

YPZ105 Programlama ve Uygulamaları I

Haftalık Ders Notları ve Laboratuvar Kılavuzu

Yapay Zeka Mühendisliği Lisans Programı — 1. Yarıyıl
Öğrenci Sürümü

Dr. Öğr. Üyesi Uğur Turhal

14 Eylül 2026

Bu kitap, **YPZ105 Programlama ve Uygulamaları I** dersinin ders notudur; Trabzon Üniversitesi, Bilgisayar ve Bilişim Fakültesi, Yapay Zeka Mühendisliği Bölümü birinci sınıf öğrencileri için yazılmıştır. Dersin resmî müfredatını hafta hafta izler: Bölüm I'in numaralı her bölümü takvimdeki bir öğretim haftasına karşılık gelir. Ders haftada dört saattir (3 saat teori + 1 saat uygulama) ve tamamı bilgisayar laboratuvarında işlenir; bu yüzden her bölüm, *dört saatlik tek bir oturumda* anlatılıp uygulanacak kadar malzeme içerir — ne daha azı, ne daha fazlası. Bir haftayı okumak, o haftanın dersine hem hazırlanmak hem de dersten sonra eksik kalan yeri tamamlamak demektir.

Neden programlama? Yapay zeka mühendisliği, dışarıdan bakıldığında modellerden ve matematikten ibaret görünür. Gerçekte bir yapay zeka projesinin her adımı bir programdır: bir dosyadan binlerce satır veriyi *okumak*, eksik ve bozuk kayıtları *temizlemek*, bir modeli veriyle *eğitmek*, sonucu bir tabloya ya da grafiğe dökerek *raporlamak*. Bu adımların hiçbiri bir düğmeye basarak yapılmaz; her biri yazılmış, okunmuş ve hatası ayıklanmış koddur. Üstelik kullandığımız yapılar da değişmez: bu dersin sekizinci haftasında yazacağımız **while** döngüsü — “bir koşul sağlandığı sürece şunu tekrarla” — dördüncü sınıfta bir sinir ağını eğitirken yazacağımız *eğitim döngüsünün* ta kendisidir; tek fark, döngünün gövdesinde toplama yerine ağırlık güncellemesi olmasıdır. Bir dosyadan sayı okuyup ortalamasını alan on beş satırlık program ile bir veri kümesini ölçekleyen program aynı iskelete sahiptir. Bu yüzden programlama, mesleğinizin yardımcı bir aracı değil, üzerine her şeyin kurulduğu *temel becerisidir*; ve bu beceri okuyarak değil, yazarak edinilir.

Notların düzeni.

- **Başlarken** bölümü, kitabın geri kalanının açıklamasız kullandığı sözleşmeleri — kutuların anlamı, kod gösterimi, sözde kod, zorluk işaretleri — bir kez toplar.
- **Bölüm I — Haftalar 1–8 ve 10–15** müfredatı aynen izler. Her hafta öğrenme çıktıları ve yeni terimlerle açılır; konu anlatımı, çözümlü örnekler, laboratuvar çalışması ve alıştırmalarla ilerler; bir özet tablosuyla kapanır.
- **9. ve 16. haftalar** ders haftası değildir: sırasıyla ara sınav (vize) ve final sınavı yapılır. Bu iki bölüm sınavın kapsamını, biçimini ve hazırlık önerilerini verir.
- **Ekler:** **A** yarıyıl içi çalışma planı (sınav takvimi, ağırlıklar, iş yükü), **B** Python kılavuzu (Anaconda, IDLE, PyCharm, Colab), **C** Python hızlı başvuru (sınav öncesi tek oturuşta gözden geçirilecek tablolar), **F** alıştırmaların cevapları.
- En sonda bir **dizin** vardır: bir terimin ilk kez nerede tanımlandığını oradan bulabilirsiniz.

Nasıl kullanılır. Bu notlar okunmak için değil, *yanında kod yazmak* için hazırlanmıştır. Kodu okumak öğrenmek değildir: bir programı anladığınızı ancak kapatıp kendiniz yeniden yazabildiğinizde anlarsınız. Bu yüzden her kod kutusunu gördüğünüzde bilgisayarınızda açık duran düzenleyiciye geçip programı elle yazın, çalıştırın, sonra bilerek bozup ne olduğuna bakın. Her kod kutusunun sağ üstündeki **karekod**, aynı dosyanın GitHub'daki kopyasına gider; dosyaların tamamı şu depodadır:

<https://github.com/uturhal/YPZ105>

Kutuların (tanım, kural, örnek, ipucu, dikkat, sık yapılan hata, laboratuvar) ne anlama geldiği ve kodun nasıl gösterildiği **Başlarken** bölümünde anlatılmıştır; oraya bakmadan devam etmeyin. Özellikle sık yapılan bir hata söz konusu olduğunda, yanlış ile doğru yan yana gösterilir:

yanlış yazım → doğru yazım

Soldaki kırmızı yazım hatalı, sağdaki yeşil yazım doğrudur. Sınavlarda kaybedilen puanların büyük kısmı bu kutularda toplanmıştır; bilinen bir konuyu okurken bile onlara göz atın.

Bir dürüstlük notu. Bu notlar yenidir ve ilk kez bu yarıyıl kullanılmaktadır. Kodların çıktıları ve metindeki sayılar otomatik olarak doğrulanmaktadır; yine de dil hataları, eksik açıklamalar ve gözden kaçmış yanlışlar bulunabilir. Bir hata bulursanız ya da bir yeri anlaşılmaz bulursanız lütfen bildirin: sonraki sürümü siz daha iyi hâle getireceksiniz.

İçindekiler

Başlarken: Bu Notlar Nasıl Kullanılır?	1
Hafta 1: Bilgisayarda Temel Kavramlar: Donanım, Veri ve Sayı Sistemleri	4
1.1 Bilgisayar nedir	4
1.2 İşlemci, bellek, depolama ve birimler	6
1.3 Sayı sistemleri	7
1.4 Verinin bellekte gösterimi	11
1.5 Yazılım katmanları	16
1.6 Yapay zeka ve programlama	18
1.7 Dosya, klasör, yol ve terminal	19
Hafta 2: Algoritma ve Akış Diyagramları I: Problemden Sıralı Çözüme	24
2.1 Problem çözme süreci	24
2.2 Algoritma nedir?	26
2.3 Değişken, atama ve ifadeler	27
2.4 Sözde kod sözleşmesi	30
2.5 Akış diyagramları	31
2.6 İzleme tablosu	32
2.7 Sıralı algoritma örnekleri	34
2.8 Algoritmayı sınamak	41
Hafta 3: Algoritma ve Akış Diyagramları II: Karar ve Tekrar Yapıları	46
3.1 Karar yapısı: EĞER-DEĞİLSE	46
3.2 Çoklu karar ve karar tablosu	49
3.3 Mantıksal koşullar: VE, VEYA, DEĞİL	51
3.4 Tekrar yapısı: koşullu döngü	53
3.5 Sayaçlı döngü ve biriktiriciler	56
3.6 İç içe döngüler	58
3.7 Klasik algoritmalar	58
3.8 Dizi kavramına giriş	63
3.9 Algoritma verimliliği	63
Hafta 4: Python'a Giriş: Giriş/Çıkış, Veri Türleri ve Değişkenler	69
4.1 Programlama dili ve Python	69
4.2 Çalışma ortamı: kurulum ve ilk program	70
4.3 Değişkenler ve atama	72
4.4 Veri türleri	75
4.5 Tür dönüşümü ve kullanıcıdan veri almak	77
4.6 Aritmetik operatörler ve işlem önceliği	79
4.7 Çıktıyı biçimlendirme: <code>print()</code> ve f-string	83
4.8 Bir programın anatomisi: Girdi – İşlem – Çıktı	84
4.9 Hata mesajlarını okumak	86
Hafta 5: Karakter Dizileri: İndeks, Dilim ve Metotlar	92
5.1 Karakter dizisi nedir, nasıl yazılır	92
5.2 İndeksleme: tek bir karaktere erişmek	94
5.3 Dilimleme: bir parçayı almak	95
5.4 Değiştirilemezlik: metin yerinde değişmez	96
5.5 Metotlar: metnin kendi işlevleri	97

5.6	Karakter kodları, karşılaştırma ve Türkçe sorunu	102
5.7	Hizalama ve biçimlendirme	104
5.8	Uygulamalar	104
Hafta 6: Koşullu İfadeler: Programa Karar Verdirmek		113
6.1	Boole türü ve karşılaştırma operatörleri	113
6.2	if deyimi: koşul, iki nokta ve blok	115
6.3	if-else: iki yoldan biri	117
6.4	if-elif-else ve sıranın önemi	118
6.5	Mantıksal operatörler: and, or, not	120
6.6	Doğruluk değeri olan nesneler ve koşullu ifade	125
6.7	İki tuzak: float eşitliği ve = ile ==	127
6.8	Uygulamalar	128
Hafta 7: Karmaşık Kararlar: İç İç Koşullar, match-case ve Girdi Doğrulama		136
7.1	İç içe koşullar	136
7.2	Karar tablosundan koda	139
7.3	Koşulları sadeleştirme: De Morgan	142
7.4	match-case: desenle seçim	143
7.5	Girdi doğrulama	146
7.6	Aralıklar ve sınır durumları	148
7.7	Menü programları ve karar ağaçları	150
7.8	Uygulamalar	152
Hafta 8: Tekrar Yapıları: while Döngüsü ve Biriktiriciler		159
8.1	Neden döngü? while'ın dört parçası	159
8.2	Sayaç kontrollü döngü	162
8.3	Biriktiriciler: toplam, çarpım, sayaç, en büyük	163
8.4	Nöbetçi değerle sonlanan döngü	165
8.5	Girdi doğrulama döngüsü	168
8.6	break, continue ve while True	169
8.7	Sonsuz döngü, birim kayması ve while ... else	172
8.8	Uygulamalar	174
Hafta 9: Ara Sınav (Vize)		183
Hafta 10: for Döngüsü, İç İçe Tekrarlar ve Seriler		184
10.1	for döngüsü ve range	184
10.2	for mu, while mi?	186
10.3	Metin üzerinde for	188
10.4	enumerate, zip ve reversed	192
10.5	İç içe döngüler	193
10.6	Döngüyle biriktirme: seriler	196
10.7	for ... else ve arama deseni	198
10.8	Adım sayısı ve verimlilik sezgisi	200
10.9	Uygulamalar	201

Hafta 11: Fonksiyonlar I: Kendi Komutlarınızı Yazmak	206
11.1 Neden fonksiyon?	206
11.2 Tanım ve çağrı: <code>def</code>	208
11.3 Parametreler ve argümanlar	209
11.4 <code>return</code> ile değer döndürmek	212
11.5 <code>print</code> mi, <code>return</code> mü? Void fonksiyonlar ve <code>None</code>	215
11.6 Yerel değişkenler ve kapsam	216
11.7 Belgeleme: docstring ve <code>help()</code>	220
11.8 Fonksiyon tasarımı ve elle sınama	220
11.9 Yukarıdan aşağıya tasarım ve <code>main()</code>	223
Hafta 12: Fonksiyonlar II: Esnek Çağrılar, Özyineleme ve Modüller	230
12.1 Varsayılan değerler ve anahtar sözcüklü argümanlar	230
12.2 Birden çok değer döndürmek	232
12.3 Yardımcı fonksiyonlar ve katmanlı tasarım	234
12.4 Özyineleme: kendini çağıran fonksiyon	237
12.5 Özyineleme mi, döngü mü?	241
12.6 Adsız fonksiyonlar: <code>lambda</code> ve <code>key</code>	243
12.7 Kendi modülünüzü yazmak	244
12.8 Fonksiyonları sınamak: <code>assert</code>	247
Hafta 13: Listeler: Çok Sayıda Veriyi Tek Adla Tutmak	255
13.1 Liste nedir: bir ad, çok değer	255
13.2 İndeksleme ve dilimleme	257
13.3 Değiştirilebilirlik: listeyi yerinde değiştirmek	258
13.4 Sorgulama, sıralama ve toplu işlevler	260
13.5 Liste üzerinde döngü ve liste üretici	262
13.6 Takma ad, kopya ve listeyi fonksiyona geçirmek	265
13.7 Arama ve sıralama algoritmaları	267
13.8 Uygulamalar	272
Hafta 14: İki Boyutlu Listeler, Demetler ve Sözlükler	279
14.1 İki boyutlu liste: listelerin listesi	279
14.2 İki boyutlu liste oluşturmak: çarpma tuzağı	282
14.3 İç içe döngüyle gezinmek: satırlar, sütunlar ve çizelgeler	283
14.4 Matris işlemleri: transpoz, toplama ve çarpma	285
14.5 İki boyutlu listenin iki yüzü: görüntü ve oyun tahtası	287
14.6 Demet (tuple): değiştirilemeyen dizi	290
14.7 Demet listeleri: kayıtlar, <code>zip</code> ve sıralama	292
14.8 Sözlük (dictionary): anahtarla erişim	294
14.9 Küme (set) ve hangi yapıyı ne zaman seçmeli	298
Hafta 15: Metin Dosyaları: Veriyi Kalıcı Kılmak	303
15.1 Neden dosya: bellek geçici, disk kalıcı	303
15.2 Dosya yolu, görelî yol ve çalışma dizini	304
15.3 Dosyayı açmak: modlar, <code>close</code> ve <code>with</code>	306
15.4 Dosyayı okumak: dört yol ve <code>strip()</code>	308
15.5 Ekleme modu ve günlük tutmak	311
15.6 Sayısal veriyi okumak, işlemek ve temizlemek	313
15.7 CSV dosyaları	315
15.8 Dosya hatalarını yakalamak	318

15.9 Kayıt güncelleme deseni ve JSON'a bakış	320
Hafta 16: Final Sınavı	329
Ek A — Yarıyıl İçi Çalışma Planı	330
A.1 Ağırlıklar ve başarı notunun hesabı	330
A.2 Sınav biçimi, devam ve mazeret	332
A.3 İş yükü ve haftalık çalışma önerisi	332
Ek B — Python Kurulum Kılavuzu	334
B.1 Hangi seçenek kime uygun?	334
B.2 Windows'ta Anaconda kurulumu	335
B.3 macOS ve Linux	335
B.4 Kurulumu doğrulama	335
B.5 IDLE ile çalışmak	336
B.6 PyCharm Community	337
B.7 Google Colab	337
B.8 Sık karşılaşılan kurulum sorunları	338
B.9 Ders kodlarını indirmek	339
Ek C — Python Hızlı Başvuru	340
C.1 Veri türleri ve dönüşümler	340
C.2 Operatörler ve işlem önceliği	340
C.3 Giriş, çıkış ve biçimlendirme	341
C.4 Metin (str) işlemleri	341
C.5 Seçme yapıları	342
C.6 Tekrar yapıları	342
C.7 Fonksiyonlar ve modüller	343
C.8 Listeler, demetler, sözlükler ve kümeler	343
C.9 Metin dosyaları	344
C.10 Sık kullanılan yerleşik fonksiyonlar, math ve random	345
C.11 Hata türleri ve olası nedenleri	345
Ek F — Alıştırma Cevapları	346
Dizin	351

Başlarken: Bu Notlar Nasıl Kullanılır?

Bu kitap, **YPZ105 Programlama ve Uygulamaları I** dersinin on dört öğretim haftasını haftalık bölümler hâlinde izler. Ders haftada dört saattir (3 saat teori + 1 saat uygulama) ve tamamı bilgisayar laboratuvarında işlenir; dolayısıyla her bölüm, dört saatlik tek bir oturumda anlatılıp *uygulanacak* kadar malzeme içerir. Aşağıdaki sözleşmeler kitabın tamamında açıklamasız kullanılır.

Kutular ne anlama gelir?

Tanım 0.1: Tanım kutusu

Yeni bir kavram ilk kez burada, kesin biçimiyle verilir. Tanımlar numaralıdır (Tanım 4.2 gibi) ve metin içinde onlara bu numarayla gönderme yapılır.

Kural 0.1 (Kural kutusu). Programlama dilinin ya da algoritma tasarımının *her zaman* geçerli bir kuralı. Kırıldığında program ya çalışmaz ya da yanlış çalışır.

Örnek 0.1 (Örnek kutusu). Bir problem cümlesi. Hemen ardından gelen **Çözüm** kutusu problemi adım adım çözer: analiz, algoritma, kod, çıktı ve **Kontrol**.

Çözüm

Çözüm kutuları hep aynı iskeleti izler: **Analiz** (girdi/çıktı/işlem), **Algoritma** (sözde kod ya da akış diyagramı), **Kod**, **Çıktı** ve **Kontrol** — elle hesaplanmış bir değerle ya da uç bir durumla karşılaştırma. Kontrol adımı isteğe bağlı değildir: çalışan bir programı doğru çalışan bir programdan ayıran şey odur.

İpucu

Mavi **İpucu** kutuları işinizi kolaylaştıran pratik bilgiler içerir: klavye kısayolları, daha kısa bir yazım, daha okunur bir yapı.

Dikkat

Turuncu **Dikkat** kutuları, sonucu sessizce yanlış çıkaran ya da programı beklenmedik biçimde durduran tuzakları işaret eder.

Sık yapılan hata

yanlış bir yazım ya da yanlış bir inanış → doğru yazım ve doğru açıklama

Kırmızı **Sık yapılan hata** kutuları, sınavlarda ve laboratuvarlarda gerçekten puan kaybettiren hataları gösterir. Bilinen bir konuyu okurken bile bu kutulara bakmaya değer.

Not

Gri **Not** kutuları arka plan bilgisi, tarihsel bir ayrıntı ya da konunun ilerideki haftalarla bağlantısını verir; ilk okumada atlanabilir.

Kod nasıl gösterilir?

Kitaptaki her tam program, **numaralı bir kod kutusu** içinde ve dosya adıyla birlikte verilir (Kod 4.3 gibi). Kutunun sağındaki **karekod** aynı dosyanın GitHub'daki kopyasına gider; telefonunuzla okutup dosyayı indirebilir ya da tarayıcıda görüntüleyebilirsiniz. Tüm dosyalar şu depoda toplanmıştır:

<https://github.com/uturhal/YPZ105>

Kodun hemen altındaki gri kutu programın **gerçek çıktısıdır**: notlar derlenirken her betik otomatik olarak çalıştırılır ve ürettiği çıktı buraya yazılır; yani basılı çıktı ile dosyanın çıktısı birbirinden farklı olamaz.

Kısa denemeler için Python'un etkileşimli kabuğu kullanılır; `>>>` işareti Python'un sizden komut beklediğini gösterir ve yazdığınız ifadenin değeri hemen altında görünür:

```
Python etkileşimli kabuk (REPL)
>>> 2 + 3 * 4
14
>>> "Merhaba" + " " + "Dünya"
'Merhaba Dünya'
```

Metin içindeki kod parçaları `print("Merhaba")` gibi daktilo yazısıyla, dosya adları `merhaba.py` gibi, klavye tuşları `Ctrl+S` gibi, menü yolları `File → Save` gibi gösterilir. Terminale yazılan komutlar `$ python merhaba.py` biçimindedir; baştaki `$` işareti komutun parçası değildir, yazılmaz.

Sözde kod ve akış diyagramları

İlk üç hafta boyunca algoritmalar Python'suz, *sözde kod* ve *akış diyagramı* ile yazılır. Sözde kodda anahtar sözcükler Türkçe ve kalındır; atama sola bakan okla gösterilir:

Sözde kod: İki sayının ortalaması

```
BAŞLA
  OKU a, b
  ort ← (a + b) / 2
  YAZ ort
BİTİR
```

Akış diyagramlarında ISO 5807 sembolleri kullanılır: oval *başla/bitir*, paralelkenar *giriş/çıkış*, dikdörtgen *işlem*, eşkenar dörtgen *karar*. Karar sembolünden çıkan oklar her zaman **Evet** ve **Hayır** ile etiketlenir.

Alıştırmalar ve cevaplar

Her bölüm alıştırmalarla biter. Zorluk üç noktayla gösterilir: ●●● tek kavram, ●● birkaç adım, ●●● birden çok konuyu birleştiren ya da tasarım kararı gerektiren problem. Her alıştırmının kısa cevabı (beklenen çıktı, anahtar fikir ya da sayısal sonuç) hemen altında gri italik olarak verilir ve kitabın sonundaki **Cevaplar** ekinde toplanır. Tam çözüm kodları sınıfta ve laboratuvarında tartışılır.

Yazım sözleşmeleri

- Kaynak kodda **İngilizce tanımlayıcılar** (`total`, `count`, `student_name`) kullanılır; açıklamalar ve ekran metinleri Türkçedir. Bu, dünya genelindeki Python topluluğunun

ve kütüphanelerin diline uyum sağlar; Türkçe karakter içeren tanımlayıcılar (**sayı**) geçerli olsa da klavye ve taşınabilirlik sorunları yarattığı için kullanılmaz.

- Ondalık ayırıcı **noktadır**: Python 3.14 yazımını tanır, **3,14** ise bir *demet* (tuple) oluşturur. Bu farkı 14. haftada ayrıntısıyla göreceksiniz.
- Girintileme **4 boşluktur**; sekme (Tab) karakteri kullanılmaz. Çoğu düzenleyici Tab tuşuna basıldığında 4 boşluk ekleyecek biçimde ayarlanabilir.
- Bir kavram ilk geçtiği yerde **kalın** yazılır ve dizine eklenir; İngilizce karşılığı gerektiğinde parantez içinde verilir: **değişken** (*Ing. variable*).

Değerlendirme

Dersin notu beş kısa sınav (her biri %5), bir ara sınav (%25) ve final sınavından (%50) oluşur. Kısa sınavlar 3., 6., 8., 12. ve 15. haftalarda dersin son 25 dakikasında laboratuvarında yapılır ve her biri o tarihe kadar işlenen konuları kapsar. Yarıyıl içi çalışma planının tamamı Ek A'dadır.

Hafta 1: Bilgisayarda Temel Kavramlar: Donanım, Veri ve Sayı Sistemleri

Öğrenme çıktıları

Bu haftanın sonunda:

1. bir bilgisayarın temel donanım bileşenlerini (işlemci, bellek, depolama, giriş/çıkış) ve von Neumann mimarisinde bir komutun nasıl yürütüldüğünü açıklayabileceksiniz;
2. bit, bayt, KB–TB ve GHz birimlerini doğru kullanıp bellek ve depolama hesapları yapabileceksiniz;
3. ikili, onluk ve onaltılık sayı sistemleri arasında iki yöntemle dönüşüm yapıp ikili sayıları toplayabileceksiniz;
4. tam sayıların, kayan noktalı sayıların, karakterlerin, görüntülerin ve sesin bellekte nasıl temsil edildiğini anlatıp bir verinin bayt boyutunu hesaplayabileceksiniz;
5. işletim sistemi, uygulama ve programlama dili katmanlarını ayırt edip bu dersin bir yapay zeka mühendisinin eğitimindeki yerini söyleyebileceksiniz;
6. dosya, klasör ve yol kavramlarını kullanarak terminalde klasörler arasında gezinip yeni klasör oluşturabileceksiniz.

Anahtar kavramlar

donanım (İng. hardware) **yazılım** (İng. software) **işlemci** (İng. CPU) **bellek** (İng. memory) **bit** **bayt** (İng. byte) **ikili sistem** (İng. binary) **onaltılık sistem** (İng. hexadecimal) **taşma** (İng. overflow) **kayan nokta** (İng. floating point) **ASCII** **Unicode** **UTF-8** **piksel** (İng. pixel) **örnekleme** (İng. sampling) **işletim sistemi** (İng. operating system) **derleyici** (İng. compiler) **yorumlayıcı** (İng. interpreter) **yol** (İng. path) **terminal**

Bu ders bir programlama dersi; ama ilk hafta tek satır program yazmayacağız. Nedeni şu: program, bilgisayara verilen bir komut listesidir ve komutun ne yaptığını anlamak için komutu *alan* makinenin nasıl çalıştığını bilmek gerekir. Bir programın bellekte nerede durduğunu, “7” ile “7”nin neden farklı şeyler olduğunu, bir fotoğrafın niçin üç megabayt tuttuğunu, klavyeden basılan **A** tuşunun neden 65 sayısı olarak saklandığını bilen biri, ilerleyen haftalarda karşılaşacağı hata mesajlarının çoğunu tahmin edebilir. Bu hafta o zemini kuruyoruz.

Bir yapay zeka mühendisi için bu zemin süs değil, iş aletidir. Yapay zeka modelleri görüntüleri piksel sayıları, metinleri karakter kodları, sesleri örnek dizileri olarak “görür”; modellerin bellek ihtiyacı bayt cinsinden hesaplanır; bir sinir ağının 32 bit yerine 16 bit sayılarla çalıştırılması saatlerce eğitim süresi kazandırır. Bu haftanın sonunda dönüşüm tablolarından başınızı kaldırıp bir sayısal fotoğrafa baktığınızda, onun arkasındaki 6 milyon sayıyı görebiliyor olacaksınız.

1.1 Bilgisayar nedir: donanım, yazılım ve von Neumann mimarisi

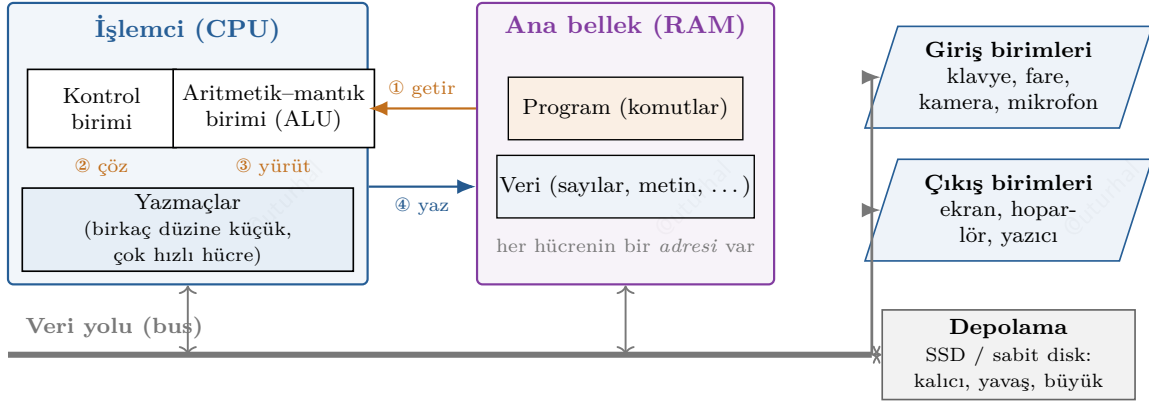
Tanım 1.1: Donanım ve yazılım

Donanım (İng. hardware), bilgisayarın elle dokunulabilen fiziksel parçalarıdır: işlemci, bellek yongaları, disk, ekran, klavye. **Yazılım** (İng. software) ise donanıma ne yapacağını söyleyen komut dizileridir: işletim sistemi, tarayıcı, oyun ve bu derste yazacağımız programlar. Donanım yazılımsız çalışmaz; yazılım donanımsız var olamaz.

Bugünkü hemen her bilgisayar — masaüstü, telefon, akıllı saat, bir arabanın motor kontrol birimi — 1945'te John von Neumann'ın adıyla anılan aynı planı izler (Şekil 1.1).

Tanım 1.2: Von Neumann mimarisi

Bir bilgisayar dört tür birimden oluşur: **işlemci** (İng. CPU, central processing unit) komutları yürütür; **ana bellek** (İng. main memory, RAM) hem çalışan programı hem de üzerinde çalıştığı veriyi tutar; **giriş/çıkış birimleri** (İng. I/O) dış dünyayla bağlantıyı sağlar; **veri yolu** (İng. bus) bunların arasında bitleri taşır. Programın da veri gibi bellekte saklanması **saklı program ilkesi** denir.



Saklı program ilkesi: komutlar da veri gibi *aynı* bellekte durur. İşlemci durmaksızın aynı döngüyü çevirir: bellekten sıradaki komutu *getir* ①, komutun ne istediğini *çöz* ②, ALU ile *yürüt* ③, sonucu yazmaya ya da belleğe *yaz* ④; sonra bir sonraki komuta geçer. Saniyede milyarlarca kez. Programlama, bu belleğe *hangi komutların* konulacağına karar vermektir.

Şekil 1.1: Von Neumann mimarisi. İşlemci, bellekten sıradaki komutu alır, ne istediğini çözer, yürütür ve sonucu yazar; sonra bir sonraki komuta geçer. Depolama (SSD/disk) bu şemada bir giriş/çıkış birimidir: işlemci diskteki veriyle doğrudan çalışamaz, önce RAM'e taşınması gerekir.

İşlemcinin içinde üç şey vardır. **Kontrol birimi** komutu okur ve “bu bir toplama, şu iki değeri al” gibi yönlendirmeler yapar. **Aritmetik-mantık birimi** (İng. ALU) toplama, çıkarma, karşılaştırma gibi asıl işlemleri yapar. **Yazmaçlar** (İng. registers) işlemcinin o an kullandığı birkaç değeri tutan, bellekten yüzlerce kat hızlı ama sayıca çok az (birkaç düzine) hücredir. İşlemci, şekildeki ①–④ adımlarını — *getir*, *çöz*, *yürüt*, *yaz* — durmaksızın yineler. Bu döngünün temposunu **saat hızı** (İng. clock speed) belirler: 3 GHz'lık bir işlemci saniyede 3 milyar döngü yapar; bir döngü yaklaşık 0,33 ns (nanosaniye, saniyenin milyarda biri) sürer. Basit bir komut bir ya da birkaç döngü alır.

Saklı program ilkesinin sonucu, aynı donanımın bir dakika hesap tablosu, bir dakika sonra oyun çalıştırabilmesidir: değişen tek şey bellekteki komutlardır. Programlama, bu belleğe hangi komutların konulacağına karar verme işidir; ilerleyen haftalarda Python'da yazdığınız her satır, sonunda bu şemadaki *getir-çöz-yürüt* döngüsüne dönüşecektir.

Dikkat

Günlük dilde “bellek” sözcüğü hem RAM hem de disk için kullanılır; bilgisayar bilimlerinde ikisi ayrı şeylerdir. Bir telefon ilanındaki “8 GB RAM / 256 GB depolama” ifadesinde *RAM* çalışan programların ve o an işlenen verinin durduğu, güç kesilince silinen hızlı bellektir; *depolama* ise fotoğrafların ve uygulamaların kalıcı olarak saklandığı yavaş ama büyük alandır. “Belleğim doldu” diyen kullanıcı çoğu zaman depolamayı kastediyordur.

1.2 İşlemci, bellek, depolama ve birimler

Tanım 1.3: Bit ve bayt

Bit (*İng. binary digit*), bilginin en küçük birimidir: 0 ya da 1 değerini alır (fiziksel olarak bir anahtarın açık/kapalı, bir kondansatörün dolu/boş olması). Sekiz bitlik gruba **bayt** (*İng. byte*) denir; bellek adresleri, dosya boyutları ve veri türleri bayt üzerinden tanımlanır. Bir bayt $2^8 = 256$ farklı değer alabilir.

Bellek ve depolama kapasiteleri bayt katlarıyla ölçülür (Tablo 1.1). İkili sistemde doğal kat $2^{10} = 1024$ 'tür; bu yüzden bilgisayar bilimlerinde geleneksel olarak $1 \text{ KB} = 1024 \text{ B}$ alınır ve bu notlarda da öyle yapılır.

Tablo 1.1: Bilgi birimleri. Bu notlarda ikili katlar kullanılır ($1 \text{ KB} = 1024 \text{ B}$); onluk tanım (SI) ve yeni ikili adlar (IEC) karşılaştırma için verilmiştir.

Birim	İkili tanım	Bayt	SI tanımı	IEC adı
bit (b)	—	1/8		
bayt (B)	8 bit	1		
kilobayt (KB)	2^{10} B	1 024	10^3	KiB
megabayt (MB)	2^{20} B	1 048 576	10^6	MiB
gigabayt (GB)	2^{30} B	1 073 741 824	10^9	GiB
terabayt (TB)	2^{40} B	1 099 511 627 776	10^{12}	TiB

Bir örnek: 16 GB RAM, $16 \times 2^{30} = 17\,179\,869\,184$ bayt, yani yaklaşık 17 milyar bayt ya da 137 milyar bittir. Böyle bir sayıyı ezberlemenize gerek yok; gereken, GB'den bayta inmek için üç kez 1024 ile çarpmayı bilmektir.

Not

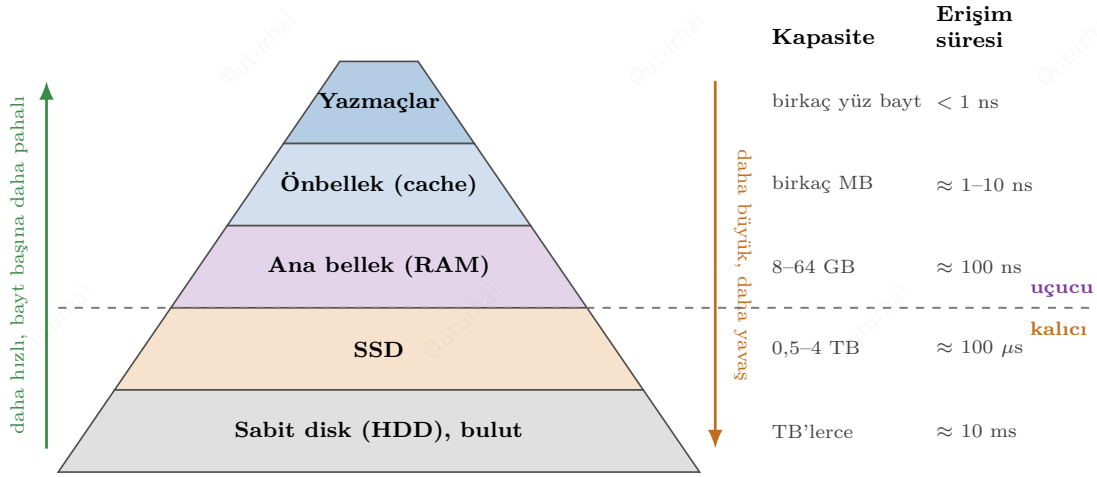
KB mi KiB mi? Disk üreticileri ve SI standardı $1 \text{ KB} = 1000 \text{ B}$, $1 \text{ GB} = 10^9 \text{ B}$ der; işletim sistemleri ise çoğu zaman 1024 tabanıyla hesaplar. Sonuç: “512 GB” yazan bir SSD bilgisayarda $512 \times 10^9 / 2^{30} \approx 476,84 \text{ GB}$ görünür — kimse sizi aldatmamıştır, iki taraf farklı tanım kullanmaktadır. Karışıklığı gidermek için IEC, ikili katlara *kibibayt* (KiB), *mebibayt* (MiB), *gibibayt* (GiB) adlarını önerdi. Bu adlar teknik belgelerde yaygınlaşıyor; günlük dilde ise “KB” hâlâ her iki anlamda kullanılıyor. Bir hesapta hangi tanımı kullandığınızı yazın; iki sonuç arasındaki yüzde 7'lik fark sınavda puan, işte ise disk alanı demektir.

Sık yapılan hata

1 KB = 1000 bit → **1 KB = 1024 bayt = 8192 bit**

Bit (b, küçük harf) ile bayt (B, büyük harf) karışır; aralarında 8 kat fark vardır. Bu karışıklığın günlük hayattaki yeri internet hızıdır: “100 Mbps” bağlantı saniyede 100 megabit taşır, yani $100/8 = 12,5 \text{ MB/s}$. Bir dosyanın 100 MB olduğunu bilip “bir saniyede iner” diyen kişi sekiz kat yanılır.

Kapasite kadar hız da önemlidir ve bellek türleri hızlarına göre bir piramit oluşturur (Şekil 1.2). İşlemcinin içindeki yazmaçlar ve önbellek en hızlı, ama en küçüktür; RAM daha büyük, daha yavaş; disk çok daha büyük, çok daha yavaştır. Her basamağı “daha yakın ama daha küçük” bir çalışma masası olarak düşünün: masanın üstü (yazmaçlar), çekmece (önbellek), oda (RAM), arşiv deposu (disk).



Her basamak, altındakinden onlarca–binlerce kat hızlı ama çok daha küçüktür. Çizginin üstündekiler *uçucudur*: güç kesilince içerikleri silinir; altındakiler *kalıcdır*. Bir program çalışmaya başladığında diskten RAM'e yüklenir; işlemci o an kullandığı birkaç değeri yazmaçlarda tutar. “Bilgisayarım yavaşladı, RAM yetmiyor” cümlesinin anlamı şudur: veri RAM'e sığmamış, işlemci 100 ns yerine 10 ms bekleten diske sık sık inmek zorunda kalmıştır — yüz bin kat fark. Kaydetmediğiniz belgenin güç kesintisinde gitmesinin nedeni de aynı şekildedir: belge RAM'deydi, diske yazılmamıştı.

Şekil 1.2: Bellek hiyerarşisi. Yukarı çıkıldıkça hız ve bayt başına maliyet artar, kapasite azalır. Çizginin üstündekiler uçucudur: güç kesilince silinir.

Piramidin en önemli sonucu şudur: bir program çalışmak için *RAM'e yüklenmek* zorundadır ve RAM'e sığmayan veri diske taşınıp geri getirilir; bu da her seferinde yüz bin kat yavaş bir işlemdir. İleride büyük veri kümeleriyle çalıştığımızda “verim RAM'e sığıyor mu?” sorusunu ilk soru olarak sormayı alışkanlık edinin.

1.3 Sayı sistemleri: ikili, onluk, onaltılık

Bilgisayar sayıları ikili tabanda tutar; bunun nedeni felsefi değil, mühendisliktir: iki durumlu bir anahtar (akım var/yok) on durumlu bir anahtardan çok daha ucuz, hızlı ve güvenilirdir. Ama ikili sayılar insan için uzun ve okunaksızdır; bu yüzden programcılar üç sistemi birlikte kullanır: ikili (makinenin dili), onluk (insanın dili) ve onaltılık (ikilinin kısa yazımı). Üçü de aynı kurala dayanır.

Kural 1.1 (Konumsal gösterim). b tabanında yazılmış $d_n d_{n-1} \dots d_1 d_0$ sayısının değeri

$$d_n \cdot b^n + d_{n-1} \cdot b^{n-1} + \dots + d_1 \cdot b^1 + d_0 \cdot b^0$$

toplamıdır. Her basamağın *ağırlığı* sağdan sola b 'nin artan kuvvetleridir ve rakamlar 0'dan $b-1$ 'e kadar gider.

Onluk sistemde ($b = 10$) bunu farkında olmadan yapıyorsunuz: $2026 = 2 \cdot 1000 + 0 \cdot 100 + 2 \cdot 10 + 6 \cdot 1$. İkili sistemde ($b = 2$) yalnız iki rakam vardır, 0 ve 1, ve ağırlıklar 1, 2, 4, 8, 16, 32, 64, 128, ... diye ikiye katlanır. Onaltılık sistemde ($b = 16$) on altı rakam gerekir; 0-9'dan sonra A-F harfleri kullanılır (Tablo 1.2). Hangi tabanda olduğunu belirtmek için sayının sağ altına taban yazılır: 10011100_2 , 156_{10} , $9C_{16}$. Programlama dillerinde onaltılık sayılar çoğu zaman $0x9C$ biçiminde, ikililer $0b10011100$ biçiminde yazılır.

Tanım 1.4: İkili sayı sistemi

İkili sistem (*İng. binary*), tabanı 2 olan konumsal sayı sistemidir. Bir ikili sayının en sağındaki bite **en az anlamlı bit** (*İng. LSB, least significant bit*), en solundakine **en anlamlı bit** (*İng. MSB, most significant bit*) denir. n bitlik bir sayı 0 ile $2^n - 1$ arasındaki 2^n farklı değeri gösterebilir.

1.3.1 İkilden onluğa: ağırlık toplamı**Ağırlık toplamı**

1. Bitlerin altına, *sağdan başlayarak* ağırlıkları yazın: 1, 2, 4, 8, ...
2. Değeri 1 olan bitlerin ağırlıklarını toplayın; 0 olanları atlayın.
3. Toplam, sayının onluk değeridir.

1.3.2 Onluktan ikiliye: bölme-kalan**Bölme-kalan**

1. Sayıyı 2'ye bölün; *bölümü* ve *kalanı* (0 ya da 1) yazın.
2. Bölümü yeni sayı olarak alıp 1. adımı yineleyin; bölüm 0 olunca durun.
3. Kalanları *aşağıdan yukarıya* (sondan başa) okuyun: ilk kalan LSB, son kalan MSB'dir.

Alternatif: sayıdan sıgıan en büyük 2 kuvvetini çıkarın, kalan için yineleyin ($156 = 128 + 28$, $28 = 16 + 12$, $12 = 8 + 4$); kullanılan kuvvetlerin bitlerine 1 yazın.

① Ağırlık toplamı: ikili → onluk

basamak	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
ağırlık	128	64	32	16	8	4	2	1
bit	1	0	0	1	1	1	0	0
katkı	128	0	0	16	8	4	0	0

$128 + 16 + 8 + 4 = 156$ (yalnız 1 olan bitler toplanır)

en anlamlı bit (MSB) solda, en az anlamlı bit (LSB) sağda

② Bölme-kalan: onluk → ikili

sayı	bölüm	kalan
156	$\div 2$	78 kalan 0
78	$\div 2$	39 kalan 0
39	$\div 2$	19 kalan 1
19	$\div 2$	9 kalan 1
9	$\div 2$	4 kalan 1
4	$\div 2$	2 kalan 0
2	$\div 2$	1 kalan 0
1	$\div 2$	0 kalan 1

$156 = 10011100_2$ (bölüm 0'da dur)

ilk kalan = LSB, son kalan = MSB

İki yöntem birbirinin tersidir ve birbirini denetler. ② ile bulduğunuz ikili sayıyı ① ile geri çevirip aynı onluk sayıyı elde ediyorsanız dönüşüm doğrudur. Ağırlıklar her basamakta ikiye katlanır (1, 2, 4, 8, 16, ...) — onluk sistemde on'a katlandığı gibi. Bölme-kalan yönteminde kalanlar *ters sırada* yazılır; en sık hata bu sırayı unutmaktır.

Şekil 1.3: İki dönüşüm yöntemi aynı sayı üzerinde. Solda 10011100_2 ağırlık toplamıyla 156'ya çevrilir; sağda 156 art arda ikiye bölünüp kalanlar ters sırada okunur. Yöntemler birbirinin tersidir: birinin sonucunu ötekiyle kontrol edebilirsiniz.

Örnek 1.1 (Onluktan ikiliye). 156 sayısını ikili sisteme çevirin ve sonucu doğrulayın.

Çözüm

Analiz. Girdi: onluk bir tam sayı. Çıktı: aynı sayının ikili yazımı. $156 < 256 = 2^8$ olduğundan sonuç en fazla 8 bit olacaktır.

Yöntem. Bölme-kalan (Şekil 1.3, sağ).

Hesap.

Sayı	Bölüm	Kalan
$156 \div 2 = 78$		0
$78 \div 2 = 39$		0
$39 \div 2 = 19$		1
$19 \div 2 = 9$		1
$9 \div 2 = 4$		1
$4 \div 2 = 2$		0
$2 \div 2 = 1$		0
$1 \div 2 = 0$		1

Kalanlar aşağıdan yukarıya: **10011100**. Yani $156_{10} = 10011100_2$.

Kontrol. Ağırlık toplamıyla geri çevirelim: 1 olan bitler 128, 16, 8 ve 4 ağırlıklarında; $128 + 16 + 8 + 4 = 156$ ✓. Ayrıca büyüklük kontrolü: 156, 128 ile 255 arasında olduğu için MSB 1 olmalı ve sayı tam 8 bit tutmalıdır; öyle de oldu.

Örnek 1.2 (İkiliden onluğa). 10110101_2 sayısının onluk değerini bulun.

Çözüm

Analiz. Sekiz bit; sonuç 0–255 arasında olacak. MSB 1 olduğundan sonuç en az 128'dir.

Yöntem. Ağırlık toplamı.

Hesap. Ağırlıkları sağdan sola yazalım:

bit	1	0	1	1	0	1	0	1
ağırlık	128	64	32	16	8	4	2	1
katkı	128	0	32	16	0	4	0	1

$128 + 32 + 16 + 4 + 1 = 181$.

Kontrol. 181'i bölme-kalanla geri çevirmek yerine hızlı bir kontrol: sayı tek olmalı çünkü LSB 1'dir ✓; $128 \leq 181 < 256$ olmalı çünkü 8 bit ve MSB 1 ✓. İsterseniz $181 - 128 = 53$, $53 - 32 = 21$, $21 - 16 = 5$, $5 - 4 = 1$, $1 - 1 = 0$ zinciriyle aynı bitlere ulaştığımızı görebilirsiniz.

**Sık yapılan hata**

$1011 = 1 + 0 + 1 + 1 = 3 \rightarrow 1011 = 8 + 0 + 2 + 1 = 11$

İkili sayıdaki 1'leri saymak sayının değerini vermez; her 1 kendi *ağırlığı* kadar katkı yapar. İkinci sık hata, ağırlıkları soldan başlatmaktır: 1011'de ağırlıklar sağdan sola 1, 2, 4, 8'dir, soldan sağa değil. Üçüncüsü, bölme-kalan yönteminde kalanları yukarıdan aşağıya okumaktır: 156 için bu 00111001 verir ki 57'dir, 156 değil.

İpucu

Aşağıdaki ikinin kuvvetlerini ezbere bilmek dönüşümleri iki kat hızlandırır:

n	0	1	2	3	4	5	6	7	8	10	16
2^n	1	2	4	8	16	32	64	128	256	1024	65536

Ayrıca $2^{20} \approx 10^6$ (bir milyon), $2^{30} \approx 10^9$ (bir milyar) ve $2^{32} = 4\,294\,967\,296$ (“dört milyar”) değerlerini bilmek bellek hesaplarında büyüklük mertebesini hızla görmeyi sağlar.

1.3.3 Onaltılık sistem

Sekiz bitlik bir sayıyı yazmak sekiz karakter alır; 32 bitlik bir bellek adresi otuz iki karakter. **Onaltılık sistem** (*İng. hexadecimal*) bu uzunluğu dörtte bir indirir, çünkü $16 = 2^4$: her onaltılık rakam tam *dört* bite karşılık gelir (Tablo 1.2). Dönüşüm bu yüzden hesap gerektirmez, gruplamayla yapılır.

Tablo 1.2: Onaltılık rakamlar ve dört bitlik karşılıkları. Harfler büyük ya da küçük yazılabilir (0x9C = 0x9c).

onluk	0	1	2	3	4	5	6	7
onaltılık	0	1	2	3	4	5	6	7
ikili	0000	0001	0010	0011	0100	0101	0110	0111
onluk	8	9	10	11	12	13	14	15
onaltılık	8	9	A	B	C	D	E	F
ikili	1000	1001	1010	1011	1100	1101	1110	1111

Dörtlü gruptama

İkiliden onaltılığa: bitleri *sağdan* başlayarak dörtlü gruplara ayırın (en soldaki grup eksikse başına 0 ekleyin), her grubu tablodan onaltılık rakama çevirin. $1011\,0101_2 \rightarrow B\,5 \rightarrow B5_{16}$. Onaltılıktan ikiliye: her rakamı dört bite yazın. $9C_{16} \rightarrow 1001\,1100_2$. Onaltılıktan onluğa ise [Kural 1.1](#) ile, 16’nın kuvvetleriyle (1, 16, 256, 4096, ...) gidilir.

Örnek 1.3 (Onaltılık dönüşümler). (a) FF_{16} ve (b) $2A7_{16}$ sayılarını onluğa çevirin; (c) 10110101_2 sayısını onaltılık yazın.

Çözüm

Analiz. (a) ve (b) için ağırlık toplamı, 16 tabanında; (c) için gruptama.

Hesap. (a) $FF_{16} = 15 \cdot 16 + 15 \cdot 1 = 240 + 15 = \mathbf{255}$.

(b) $2A7_{16} = 2 \cdot 256 + 10 \cdot 16 + 7 \cdot 1 = 512 + 160 + 7 = \mathbf{679}$.

(c) $1011\,0101_2$: sol grup $1011 = 11 = B$, sağ grup $0101 = 5$; sonuç $\mathbf{B5_{16}}$.

Kontrol. (a) İki onaltılık rakam tam bir bayttır ve FF baytın alabileceği en büyük değerdir; $2^8 - 1 = 255$ ✓. (b) $2A7$ üç rakamlı, dolayısıyla $256 \leq 679 < 4096$ olmalı ✓.

(c) [Örnek 1.2](#)’te $10110101_2 = 181$ bulmuştuk; $B5_{16} = 11 \cdot 16 + 5 = 181$ ✓. Üç sistem aynı sayıda birleşti.

✗ Sık yapılan hata

$$0x1A = 110 \rightarrow 0x1A = 1 \cdot 16 + 10 = 26$$

Onaltılık rakamları yan yana onluk rakam gibi okumak sık bir hatadır: A harfi 10'dur ama 1A “bir-on” değil, “bir onaltılık artı on birlik”tir. Aynı biçimde 10_{16} on değil, on altıdır.

Not

Onaltılık sayıları günlük hayatta görmüş olabilirsiniz. Web sayfalarındaki renk kodları (#FF8800: kırmızı FF = 255, yeşil 88 = 136, mavi 00 = 0 — turuncu), ağ kartlarının MAC adresleri (3C:22:FB:1A:9E:07), bir hata raporundaki bellek adresleri (0x7ffd3a4c) hep onaltılıktır. Programcının onaltılığı sevmesinin nedeni, her iki rakamın tam bir bayt olmasıdır: #FF8800 üç bayttır, bakınca görülür (Örnek 1.3'teki FF = 255 en parlak kanal değeridir).

1.3.4 İkili toplama

İkili toplama, onluk toplamının aynısıdır; tek fark eldenin 10'da değil 2'de verilmesidir. Dört kural yeter: $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, $1 + 1 = 10$ (yani 0 yaz, 1 elde); elde varken $1 + 1 + 1 = 11$ (1 yaz, 1 elde). Örnek: $01101011_2 + 00110101_2$ ($107 + 53$):

$$\begin{array}{r}
 \text{elde} \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\
 \quad \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \\
 + \quad \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \\
 \hline
 \quad \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0
 \end{array}$$

Sonuç $10100000_2 = 128 + 32 = 160$; gerçekten $107 + 53 = 160$ ✓. Şimdi $11111111_2 + 00000001_2$ toplamını düşünün: sonuç 100000000_2 , yani *dokuz* bit. Sekiz bitlik bir kutuya sığmaz; en soldaki 1 düşer ve geriye 00000000 kalır: $255 + 1 = 0$. Bu olaya bir sonraki bölümde ad vereceğiz.

1.4 Verinin bellekte gösterimi

Bellek yalnız bit tutar. Sayı, harf, fotoğraf, şarkı — hepsi bir *kodlama* sözleşmesiyle bit dizisine çevrilir; aynı bit dizisi bir sözleşmeye göre sayı, ötekine göre harftir. Bu bölümde en yaygın sözleşmeleri görüyoruz.

1.4.1 Tam sayılar: işaretsiz, işaretli ve taşıma

Kural 1.2 (n bitin gücü). n bit, 2^n farklı desen tutar. **İşaretsiz** (*Ing. unsigned*) yorumda bu desenler 0 ile $2^n - 1$ arasındaki sayılardır. **İşaretli** (*Ing. signed*) yorumda (ikiye tümleyen gösterimi) desenlerin yarısı negatifiere ayrılır: aralık -2^{n-1} ile $2^{n-1} - 1$ arasındadır. Her iki durumda toplam 2^n değer vardır; değişen yalnız aralığın nereye kaydırıldığıdır.

Örnek 1.4 (Kaç bit, kaç sayı?). 8, 16 ve 32 bitle gösterilebilen işaretsiz ve işaretli tam sayı aralıklarını bulun.

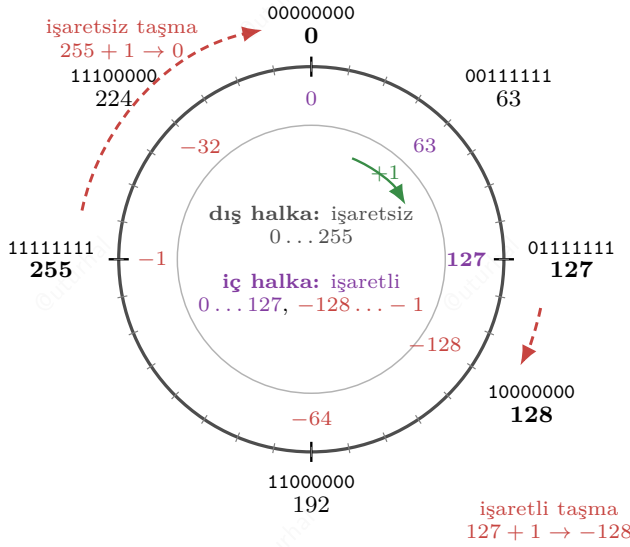
Çözüm

Analiz. Kural 1.2 doğrudan uygulanır; tek girdi bit sayısıdır.

Hesap.

Bit	Desen sayısı	İşaretsiz aralık	İşaretili aralık
8	256	0...255	-128...127
16	65 536	0...65 535	-32 768...32 767
32	4 294 967 296	0...4 294 967 295	-2 147 483 648...2 147 483 647

Kontrol. Her satırda işaretsiz en büyük değer = desen sayısı - 1 ✓; işaretili aralıkta negatiflerin sayısı (128) pozitiflerin sayısından (127) bir fazladır, çünkü sıfır pozitif tarafta yer alır: $128 + 127 + 1 = 256$ ✓. Büyüklük mertebesi: 32 bit “dört milyar” civarında olmalı, $2^{32} \approx 4,3 \times 10^9$ ✓.



İkiye tümleyen: -5 nasıl yazılır?

5 00000101
bitleri çevir 11111010
1 ekle 11111011 = -5

Aynı desen işaretsiz okunursa 251'dir.
Kural: en soldaki bit 1 ise sayı negatiftir.

Bitlerin kendisi işaretili mi işaretsiz mi olduğunu söylemez; bunu programcı (ya da veri türü) söyler. 8 bit yalnız $2^8 = 256$ farklı desen tutar: bu desenler ister 0...255 ister -128...127 diye okunur, ama her iki durumda da sınırın ardından gelen “+1” çemberde başa döner. Buna **taşma** denir.

Şekil 1.4: Sekiz bitlik sayı çemberi. Dış halka desenlerin işaretsiz, iç halka işaretili (ikiye tümleyen) okunuşudur. Çemberin üzerinde “bir ekle” saat yönünde bir adım ilerlemektir; en büyük değer ardından başa dönlür: işaretsizde $255 + 1 \rightarrow 0$, işaretilide $127 + 1 \rightarrow -128$. Sağda -5’in ikiye tümleyeni: bitleri çevir, bir ekle.

Tanım 1.5: Taşma

Bir aritmetik işlemin sonucu, ayrılan bit sayısının gösterebileceği aralığın dışına çıktığında **taşma** (İng. *overflow*) oluşur (Örnek 1.4’teki aralıklar sınırdır). Fazla bit düşer; sonuç çemberin öteki ucundan devam eder (Şekil 1.4). Altı haneli bir kilometre sayacının 999 999’dan sonra 000 000 göstermesi aynı olaydır.

İşaretili sayılarda negatifler **ikiye tümleyen** (İng. *two’s complement*) ile yazılır: sayının bitleri çevrilir ve 1 eklenir. -5 için $00000101 \rightarrow 11111010 \rightarrow 11111011$. Bu yazımın güzelliği, toplama devresinin işareti umursamamasıdır: $11111011 + 00000101 = 100000000$, düşen bit atılınca 0; yani $-5 + 5 = 0$ ✓. En soldaki bit işaret gibi davranır: 1 ise sayı negatiftir. Aynı 11111011 deseni işaretsiz okunursa 251’dir; hangisi olduğuna program karar verir.

Dikkat

Taşma sessizdir: makine hata vermez, yanlış sayıyla devam eder. 2014'te bir video paylaşım sitesinin izlenme sayacı 32 bitlik işaretli tam sayıydı ve bir video $2^{31} - 1 = 2\,147\,483\,647$ izlenmeyi aştığında sayaç negatife döndü; site sayacı 64 bite taşımak zorunda kaldı. Python'da tam sayıların büyüklük sınırı yoktur (4. haftada göreceğiz), ama yapay zekada kullanacağınız NumPy dizileri ve görüntü verileri sabit genişliktedir: 8 bitlik bir piksel değerine $255 + 10$ eklediğinizde 265 değil 9 elde edersiniz. İkinci sınıfta bu tuzağa düşen herkes bu haftayı hatırlar.

1.4.2 Kayan noktalı sayılar (kavramsal)

Ondalıklı sayılar için bilgisayar, bilim insanlarının *bilimsel gösterimini* ikiliye uyarlar: sayı, bir **anlamli kısım** (*İng. mantissa*) ile 2'nin bir **üssü** (*İng. exponent*) olarak yazılır. Örneğin $6,25 = 110,01_2$; virgül bir basamak kalacak biçimde kaydırılırsa $1,1001_2 \times 2^2$ olur. Bellekte işaret için 1 bit, üs için birkaç bit ve anlamli kısım için geri kalan bitler ayrılır; noktanın yeri üsle “kayar”, adı buradan gelir: **kayan nokta** (*İng. floating point*).

İki sonucu şimdiden bilin. Birincisi, anlamli kısım için sınırlı bit vardır: 64 bitlik standart biçim (*double*) yaklaşık 15–16 anlamli onluk basamak, 32 bitlik biçim (*float*) yaklaşık 7 basamak tutar. İkincisi ve daha şaşırtıcısı, onluk sistemde son derece sade görünen 0,1 sayısı ikili sistemde sonsuz bir kesirdir: $0,1_{10} = 0,0001\,1001\,1001 \dots_2$, tıpkı $1/3$ 'ün onlukta $0,333 \dots$ olması gibi. Bellek sonlu olduğu için sayı bir yerde kesilir ve 0,1 *tam olarak* saklanamaz. Dördüncü haftada Python'da $0,1 + 0,2$ yazıp 0,3 yerine $0,30000000000000004$ gördüğünüzde şaşırmayın: hata Python'da değil, ikili sistemin doğasındadır. Yapay zeka modelleri hız ve bellek için çoğu zaman 32 hatta 16 bitlik kayan noktalı sayılarla çalışır; “daha az bit, daha az hassasiyet” dengesini kurmak mühendisin işidir.

1.4.3 Karakterler: ASCII, Unicode ve UTF-8

Tanım 1.6: Karakter kodlaması

Karakter kodlaması (*İng. character encoding*), her karaktere bir tam sayı (kod) ve bu kodu bayta yazma kuralı veren sözleşmedir. **ASCII** (1963) 7 bitle 128 karakter tanımlar: İngiliz alfabesi, rakamlar, noktalama ve kontrol karakterleri. **Unicode** dünyadaki tüm yazı sistemlerini kapsayan 150 000'den fazla karaktere birer **kod noktası** (*İng. code point*) verir (**U+015F** gibi); **UTF-8** bu kod noktalarını 1–4 bayta yazan, ASCII ile geriye uyumlu kodlamadır ve bugün internetin ortak dilidir.

Tablo 1.3: ASCII tablosundan seçmeler. Büyük harfler 65–90, küçük harfler 97–122, rakamlar 48–57; bir harfin küçüğü büyüğünden tam 32 fazladır.

	A	B	C	D	E	F	G	H	I	J	K	L	M
kod	65	66	67	68	69	70	71	72	73	74	75	76	77
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
kod	78	79	80	81	82	83	84	85	86	87	88	89	90
	boşluk	!	,	.	?	0	1	...	9	a	b	...	z
kod	32	33	44	46	63	48	49	...	57	97	98	...	122

Örnek 1.5 (Bir sözcüğün ASCII kodları). “MERHABA” sözcüğünü ASCII kodlarıyla onluk ve ikili olarak yazın; kaç bayt ve kaç bit tuttuğunu söyleyin.

Çözüm

Analiz. Yedi büyük harf; her biri Tablo 1.3’den bir kod, her kod bir bayt.

Yöntem. Tablodan okuma, sonra her kodu 8 bite çevirme (bölme–kalan ya da $64 + \dots$ ayrıştırması; büyük harflerin hepsi 64’ten büyük olduğundan 01 ile başlar).

Hesap.

harf	M	E	R	H	A	B	A
onluk	77	69	82	72	65	66	65
ikili	01001101	01000101	01010010	01001000	01000001	01000010	01000001

Toplam 7 bayt = 56 bit.

Kontrol. Harfler alfabetik sıraya göre kodlanmıştır: A= 65, B= 66 ✓; M, alfabenin 13. harfidir, $64 + 13 = 77$ ✓; R 18. harf, $64 + 18 = 82$ ✓. İkili yazımlarda her biri 01 ile başlıyor, çünkü $64 \leq \text{kod} < 128$ ✓. Tersine çevirmek de kolaydır: 72 65 89 68 73 kodlarını tablodan okuyun.

ASCII yalnız İngilizce için yeter: ç, ğ, ı, ö, ş, ü harflerinin ASCII kodu yoktur. Bilgisayarların ilk otuz yılında her ülke kendi 8 bitlik tablosunu yaptı ve bir dosya başka bir bilgisayarda açıldığında “ÅŸ” gibi bozuk karakterler görünür oldu. Unicode bu kargaşayı tek bir numaralandırmayla bitirdi; UTF-8 ise bu numaraları öyle akıllıca bayta döker ki eski ASCII dosyaları hiç değişmeden geçerli UTF-8 kalır (Şekil 1.5).

Karakter	Kod noktası	UTF-8 baytları	Bayt
A	U+0041 (onluk 65)	01000001	1
Ş	U+015F (onluk 351)	11000101 10011111	2
€	U+20AC (onluk 8364)	11100010 10000010 10101100	3
😊	U+1F600 (onluk 128 512)	11110000 10011111 10011000 10000000	4

Renkli öncü bitler: 0 = tek baytlık karakter; 110 / 1110 / 11110 = 2 / 3 / 4 baytlık dizinin ilk baytı; 10 = devam baytı.

Karakter sayısı ile bayt sayısı aynı şey değildir. ASCII yalnız 128 karakter tanır (2^7): İngiliz alfabesi, rakamlar, noktalama. Unicode her karaktere bir *kod noktası* verir (bugün 150 000’den fazla); UTF-8 bu numarayı 1–4 bayta kodlar ve ilk 128 kod noktasında ASCII ile birebir aynıdır — eski dosyalar bozulmadan okunur. Türkçe bir metinde ç, ğ, ı, ö, ş, ü harflerinin her biri 2 bayt tutar; “Kırşehir” 8 karakter ama 10 bayttır. Dosyanın hangi kodlamayla yazıldığı bilinmezse “ÅŸ” gibi bozuk karakterler görürsünüz.

Şekil 1.5: Karakterden bayta. ASCII karakterleri UTF-8’de tek bayttır; ş gibi Türkçe harfler iki, avro simgesi üç, emoji dört bayt tutar. Renkli öncü bitler, bir baytın kaç baytlık bir dizinin parçası olduğunu söyler; böylece bir metnin ortasından okumaya başlayan program bile karakter sınırlarını bulabilir.

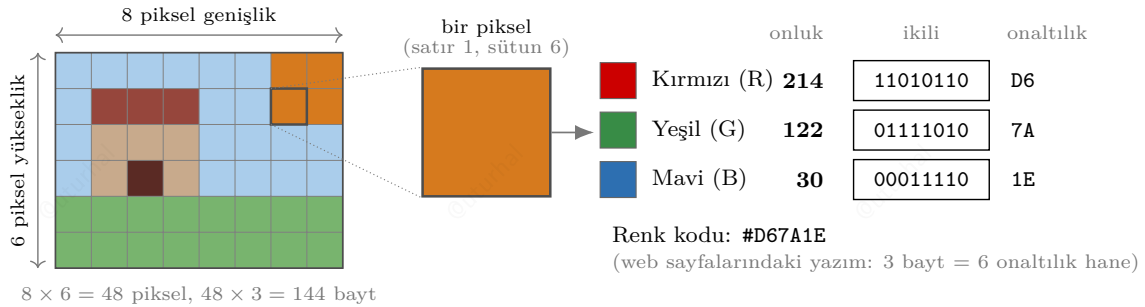
İpucu

Bir metnin “kaç karakter” ile “kaç bayt” olduğu Türkçede farklıdır: “Trabzon” 7 karakter, 7 bayttır; “Kırşehir” 8 karakter ama 10 bayttır (ı ve ş ikiyeşer bayt). Dosya boyutu, ağ kotası ve veri tabanı alanı hesaplarken bayt; kullanıcıya gösterirken karakter sayarsınız. Beşinci haftada Python’un bu ikisini nasıl ayırdığını göreceğiz.

1.4.4 Görüntü: piksel ve RGB

Tanım 1.7: Piksel ve renk derinliği

Sayısal bir görüntü, satır ve sütunlara dizilmiş **piksellerden** (İng. *pixel, picture element*) oluşur. Renkli görüntüde her pikselin rengi kırmızı, yeşil ve mavi (**RGB**) bileşenlerinin parlaklığıyla verilir; her bileşen 0–255 arasında bir bayttır. Piksel başına düşen bit sayısına **renk derinliği** denir: RGB için 24 bit (3 bayt), gri tonlu görüntü için 8 bit (1 bayt). **Şekil 1.6**, 8×6 piksellik küçük bir resmin sayı tablosuna nasıl dönüştüğünü gösterir.



Bir görüntü, piksellerden oluşan bir sayı tablosudur. Her pikselin rengi üç kanaldan (kırmızı, yeşil, mavi) parlaklığıyla verilir; her kanal 1 bayt, yani 0–255 arasında bir sayıdır. Üç bayt $2^{24} = 16\,777\,216$ farklı renk demektir. Sıkıştırılmamış boyut = genişlik $\times$ yükseklik $\times$ 3 bayt: 1920×1080 ’lık bir ekran görüntüsü 6 220 800 bayt tutar. Gri tonlu görüntüde piksel başına tek bayt yeter. Yapay zekada “görüntü işleme” bu sayı tablosu üzerinde aritmetik yapmaktır.

Şekil 1.6: Bir görüntü sayı tablosudur. 8×6 ’lık örnek resimde her piksel üç bayttır; büyütülen turuncu pikselin kanal değerleri (214, 122, 30), onaltılık yazımı #D67A1E’dir. Sıkıştırılmamış boyut = genişlik $\times$ yükseklik $\times$ bayt/piksel.

Örnek 1.6 (Bir ekran görüntüsünün boyutu). 1920×1080 piksellik (Full HD) bir RGB görüntü sıkıştırılmadan kaç bayt tutar? Sonucu MB cinsinden verin.

Çözüm

Analiz. Girdi: genişlik 1920, yükseklik 1080, piksel başına 3 bayt. Çıktı: toplam bayt ve MB.

Yöntem. Piksel sayısı $\times$ bayt/piksel; sonra iki kez 1024’e bölme.

Hesap. Piksel sayısı $1920 \times 1080 = 2\,073\,600$. Bayt sayısı $2\,073\,600 \times 3 = 6\,220\,800$ B. $6\,220\,800/1024 = 6075$ KB; $6075/1024 \approx 5,93$ MB. (Onluk tanımla 6,22 MB.)

Kontrol. Büyüklük mertebesi: iki milyon piksel, üçer bayt, altı milyon bayt, yani “6 MB civarı” ✓. Telefonunuzdaki ekran görüntülerinin 200–500 KB olması bu sonucu yanlışlamaz: PNG ve JPEG dosyaları sıkıştırılmıştır; hesapladığımız, görüntünün bel-

lekte açıldığında kapladığı ham boyuttur. Saniyede 25 kare gösteren bir Full HD video ham hâliyle saniyede 155,5 MB (onluk) üretir; video sıkıştırmasının neden kaçınılmaz olduğu buradan görülür.

✗ Sık yapılan hata

$1920 \times 1080 = 2\,073\,600$ bayt $\rightarrow 1920 \times 1080 \times 3 = 6\,220\,800$ bayt

Renkli görüntüde piksel başına üç bayt vardır; piksel sayısını bayt sayısı sanmak sonucu üç kat küçültür. Gri tonlu görüntüde ise gerçekten bir piksel = bir bayttır. İkinci sık hata birim dönüşümünde 1024 yerine 1000 ile bölüp sonucu “MB” diye yazmaktır; ikisi de kabul edilebilir, ama hangisini kullandığınızı belirtmelisiniz.

1.4.5 Ses: örnekleme

Tanım 1.8: Örnekleme ve bit derinliği

Ses, havadaki basınç değişiminin sürekli bir dalgasıdır. Bilgisayar bu dalgayı saniyede sabit sayıda ölçer — **örnekleme hızı** (*İng. sampling rate*), Hz — ve her ölçümü sabit bit sayısı ile kaydeder — **bit derinliği** (*İng. bit depth*). Stereo kayıta iki **kanal** vardır. Ham boyut:

$$\text{bayt} = \text{örnekleme hızı} \times \frac{\text{bit derinliği}}{8} \times \text{kanal sayısı} \times \text{süre (s)}.$$

Örnek 1.7 (Üç dakikalık şarkı). CD kalitesinde (44 100 Hz, 16 bit, stereo) 3 dakikalık bir kayıt sıkıştırılmadan kaç MB tutar?

Çözüm

Analiz. Girdiler: 44 100 örnek/s, 16 bit = 2 bayt, 2 kanal, 180 s.

Yöntem. Örnekleme tanımındaki formül, sonra 1024^2 'ye bölme.

Hesap. Saniyede $44\,100 \times 2 \times 2 = 176\,400$ B. Üç dakikada $176\,400 \times 180 = 31\,752\,000$ B; $31\,752\,000 / 1024^2 \approx 30,28$ MB (onluk tanımla 31,75 MB).

Kontrol. Saniyede 176 KB, dakikada yaklaşık 10 MB; üç dakika ≈ 30 MB ✓. Bir müzik CD'sinin 74 dakikası bu hızla ≈ 747 MB eder — CD'nin “yaklaşık 700 MB” diye bilinen kapasitesiyle uyumlu bir büyüklük. MP3 dosyalarının 3–5 MB olması yine sıkıştırma sayesinde; oran yaklaşık on kattır.

Bu bölümün ortak dersi şu: metin, görüntü ve ses arasında bilgisayar için özsel bir fark yoktur; hepsi bayt dizisidir ve bir sözleşmeyle anlam kazanır. Yapay zeka modelleri bu yüzden aynı matematikle hem fotoğraf hem ses hem metin işleyebilir: hepsi sayı tablosudur.

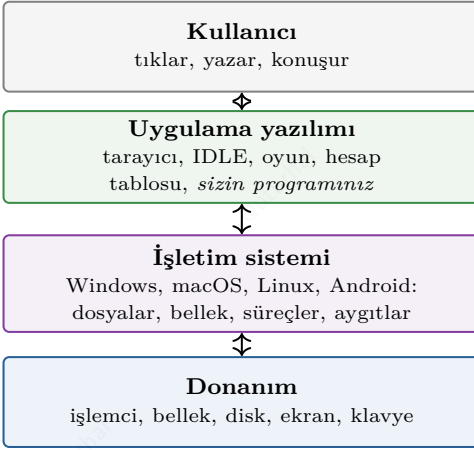
1.5 Yazılım katmanları: işletim sistemi, uygulama, programlama dili

Donanımla kullanıcı arasında birkaç yazılım katmanı vardır ve her katman yalnız komşusuyla konuşur (Şekil 1.7, sol).

Tanım 1.9: İşletim sistemi

İşletim sistemi (*İng. operating system*) — Windows, macOS, Linux, Android, iOS — donanımı yöneten ve uygulamalara ortak hizmetler sunan temel yazılımdır: dosyaları saklar ve adlandırır, belleği programlar arasında paylaştırır, aynı anda çalışan programları (*süreçleri*) sırayla işlemciye verir, klavye/ekran/ağ gibi aygıtlara erişimi düzenler. Uygulama yazılımı diske “şu bloğa yaz” demez; işletim sistemine “not.txt dosyasını aç” der.

① Yazılım katmanları



Her katman yalnız komşusuyla konuşur: programınız diske doğrudan değil, işletim sistemine “şu dosyayı aç” diyerek erişir.

② Programlama dili seviyeleri

Yüksek seviyeli dil (Python, C, Java)
toplam = fiyat + kdv
insana yakın; donanımdan bağımsız

↓ derleyici *ya da* yorumlayıcı çevirir ↓

Assembly (çevirici dili)
ADD AX, BX
her makine komutuna okunur bir ad; işlemciye özgü

↓ çevirici (assembler) ↓

Makine kodu
00000001 11011000
işlemcinin doğrudan yürüttüğü tek biçim

Aşağı inildikçe yazmak zorlaşır, çalıştırmak kolaylaşır. Bu derste en üst seviyede yazacağız; çeviri işini Python yorumlayıcısı yapacak (4. hafta).

Şekil 1.7: Solda yazılım katmanları: programınız bir *uygulamadır* ve donanıma işletim sistemi aracılığıyla erişir. Sağda programlama dili seviyeleri: aynı “topla” işi en üstte insana, en altta makineye yakın yazılır.

Uygulama yazılımı (*İng. application software*) kullanıcının bir işini yapar: tarayıcı, kelime işlemci, oyun, IDLE ve bu derste yazacağınız her program. Uygulamalar **programlama dilleriyle** yazılır ve diller üç seviyede düşünülür (Şekil 1.7, sağ):

Makine kodu

İşlemcinin doğrudan yürüttüğü ikili komutlar: 00000001 11011000. Her işlemci ailesinin kendi komut kümesi vardır; bir telefonun makine kodu bir dizüstünde çalışmaz. İnsan bunu yazmaz, okumaz.

Assembly

Her makine komutuna okunur bir ad verir: ADD AX, BX. Bire bir makine kodudur, yalnız yazımı insanca. Bir **çevirici** (*İng. assembler*) bunu makine koduna döker. Sürücü ve işletim sistemi çekirdeği gibi donanıma en yakın parçalarda kullanılır.

Yüksek seviyeli dil

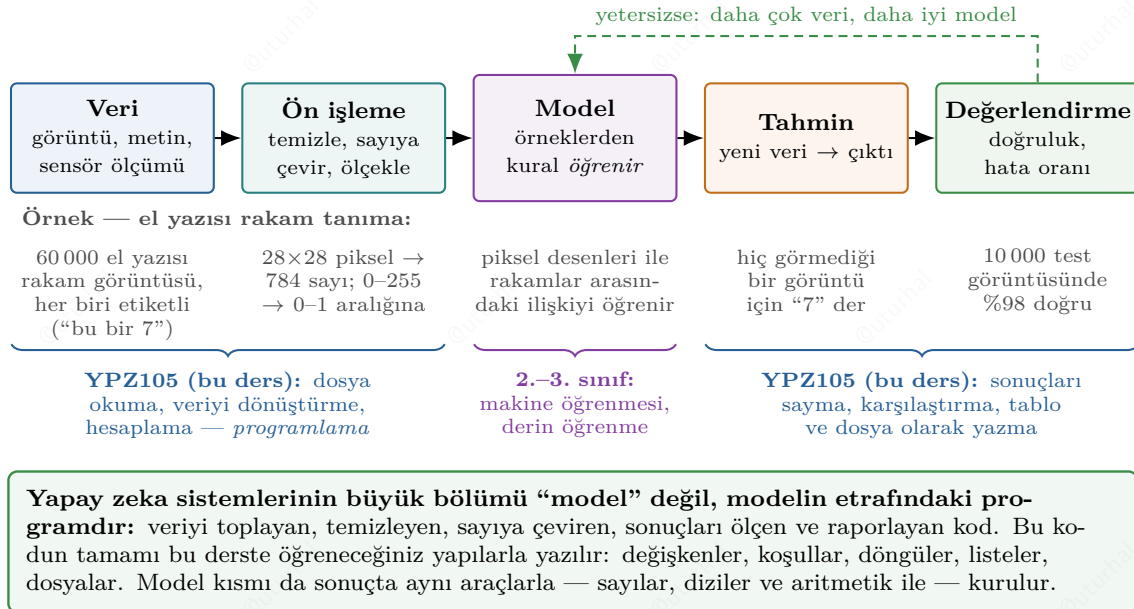
Python, C, Java, JavaScript: toplam = fiyat + kdv gibi problem dilinde yazılır; işlemciden bağımsızdır; bir satırı onlarca makine komutuna karşılık gelir. Kaynak kodu makine koduna **derleyici** (*İng. compiler*) tek seferde çevirir (C) ya da **yorumlayıcı** (*İng. interpreter*) her çalıştırmada satır satır okuyup yürütür (Python). Farkı 4. haftada, Python’u kurarken ayrıntısıyla göreceğiz.

Bu dersin dili Python’dur; ilk üç hafta ise *hiçbir* dil kullanmayız. Neden? Çünkü programlamanın zor kısmı yazım kuralları değil, çözümünü *adımlara bölmektir*; o beceriyi önce kâğıt

üzerinde, dilden bağımsız olarak kuracağız (2. ve 3. haftalar: algoritma ve akış diyagramı). Dördüncü haftada Python geldiğinde yalnız yeni bir yazım öğreneceksiniz, yeni bir düşünme biçimi değil.

1.6 Yapay zeka ve programlama

Yapay zeka (*İng. artificial intelligence*), bilgisayar biliminin bir alt alanıdır: öğrenme, akıl yürütme, algılama gibi insanla ilişkilendirilen yetenekleri makinelerle kazandırmayı amaçlar. Bugün bu alanın ağırlık merkezi **makine öğrenmesi** (*İng. machine learning*)’dir: kuralları programcının elle yazması yerine, bir *modelin* bu kuralları örnek veriden kendisinin çıkarması. Bir spam filtresine “içinde *bedava* geçen e-posta spamdır” kuralını siz yazmazsınız; on binlerce etiketli e-posta verirsiniz, kuralları model bulur. Bir yapay zeka sistemi bu yüzden hemen her zaman aynı boru hattını izler (**Şekil 1.8**): veri toplanır, temizlenip sayıya çevrilir, modele verilir, model yeni veri için tahmin üretir, tahminler ölçülür ve yetersizse başa dönülür.



Şekil 1.8: Yapay zeka boru hattı (kavramsal) ve el yazısı rakam tanıma örneği. Model, hattın yalnız bir kutusudur; diğer kutuların hepsi klasik programlama işidir ve bu derste öğreneceğiniz yapılarla yazılır.

Şeklin altındaki ayraçlar bu dersin yerini gösterir. Veriyi dosyadan okumak, satırlara ve sütunlara ayırmak, eksik değerleri temizlemek, sayıya çevirmek, sonuçları saymak ve tablo hâlinde yazmak: bunların hepsi değişken, koşul, döngü, liste ve dosya işlemleridir — yani bu dersin on dört haftası. Bir yapay zeka projesinde geçen zamanın büyük bölümünün modelde değil bu “etraf” kodda harcandığı, alanda çalışan herkesin bildiği bir gerçektir. Model kısmı da sırayla gelecek: ikinci sınıfta matris işlemleri ve NumPy, sonra makine öğrenmesi ve derin öğrenme. Bugün öğrendiğiniz bit, bayt ve sayı sistemleri o zaman geri dönecek: bir görüntü modelinin girdisi bu haftaki piksel tablosudur, “16 bit kayan nokta ile eğitim” bu haftaki kayan nokta bölümüdür, veri kümesinin RAM’e sığıp sığmayacağı bu haftaki bellek hesabıdır.

1.7 Dosya, klasör, yol ve terminalle ilk tanışma

Yazacağınız her program bir *dosya* olacak, bir *klasörde* duracak ve Python'a “şu dosyayı çalıştır” derken onun *yolunu* vereceksiniz. Üç kavram, bu hafta netleşmeli.

Tanım 1.10: Dosya, klasör, yol

Dosya (*İng. file*), diskte adıyla saklanan bir bayt dizisidir; adın noktadan sonraki kısmı, **uzantı** (*İng. extension*), içeriğin türünü söyler (**.txt** metin, **.py** Python kodu, **.jpg** görüntü). **Klasör** (*İng. folder, directory*), dosyaları ve başka klasörleri içeren kaptır; klasörler bir *ağaç* oluşturur. **Yol** (*İng. path*), bir dosyanın ağaçtaki adresidir: **mutlak yol** (*İng. absolute path*) kökten, **görelî yol** (*İng. relative path*) bulunduğunuz klasörden (**çalışma dizini** (*İng. working directory*)) başlar.

Aynı dosyanın yolu işletim sistemine göre farklı yazılır:

	Windows	macOS / Linux
kök	C:\ (sürücü harfi)	/
ayırıcı	ters bölü \	bölü /
mutlak yol	C:\Users\ayse\YPZ105\not.txt	/Users/ayse/YPZ105/not.txt
görelî yol (YPZ105'ten)	not.txt, hafta01\odev.txt	not.txt, hafta01/odev.txt
bir üst klasör	..	..
bulunduğunuz klasör	.	.

İki nokta .. “bir üst klasör” demektir; YPZ105/hafta01 içindeyken ..\not.txt (Windows) ya da ../not.txt (macOS/Linux) yolu YPZ105/not.txt dosyasına gider. Python'da yolların nasıl yazılacağını 4. haftada, dosya işlemlerini 15. haftada göreceğiz; şimdi gereken, yolu *okuyabilmek*.

Terminal (*İng. terminal, command line*) — Windows'ta *Komut İstemi* ya da *PowerShell*, macOS'ta *Terminal*, Linux'ta *Terminal* — bilgisayara fare yerine yazılı komutlarla iş yaptır-dığımız penceredir. Programcılar için vazgeçilmezdir çünkü Python programlarını çalıştırma-nın, paket kurmanın ve uzak sunuculara bağlanmanın standart yolu terminaldir. Bu hafta yalnız dört komut öğreniyoruz (Tablo 1.4); bunlar Python komutu *değildir*, işletim sisteminin kabuğuna verilen komutlardır.

Tablo 1.4: Temel terminal komutları. Windows Komut İstemi ile macOS/Linux kabuğu arasındaki farklara dikkat edin; PowerShell çoğu Linux adını da kabul eder.

İş	Windows	macOS / Linux	Açıklama
neredeyim?	cd	pwd	çalışma dizininin mutlak yolunu ya-zar
içerik listesi	dir	ls	bulunduğunuz klasördeki dosya ve klasörler
klasöre gir	cd YPZ105	cd YPZ105	çalışma dizinini değiştirir
üst klasöre çık	cd ..	cd ..	bir seviye yukarı
klasör oluştur	mkdir hafta01	mkdir hafta01	bulunduğunuz yerde yeni klasör
dosya içeriği	type not.txt	cat not.txt	metin dosyasını ekrana yazar
ekranı temizle	cls	clear	yalnız görüntüyü temizler

Aşağıda bir macOS/Linux oturumu var; \$ işareti kabuğun komut beklediğini gösterir (Windows'ta bunun yerine C:\Users\ayse> gibi bir istem görürsünüz), sonrasını kullanıcı yazar, altındaki satırlar bilgisayarın cevabıdır:

```
$ pwd
/Users/ayse
$ mkdir YPZ105
$ cd YPZ105
$ mkdir hafta01
$ ls
hafta01
$ cd hafta01
$ pwd
/Users/ayse/YPZ105/hafta01
$ cd ..
$ pwd
/Users/ayse/YPZ105
```

Oturumu adım adım okuyun: önce neredeyiz (/Users/ayse, ev klasörü); YPZ105 klasörü oluşturuldu ve içine girildi; içinde **hafta01** oluşturuldu; **ls** yalnız onu listeledi çünkü başka bir şey yok; **hafta01**'e girip mutlak yolu gördük; **cd ..** ile bir üst klasöre döndük. Bu iskeleti — YPZ105 altında hafta klasörleri — dönem boyunca kullanacağız; 4. haftada ilk Python dosyanız YPZ105/hafta04/merhaba.py olacak.

İpucu

Terminalde bir dosya ya da klasör adının ilk birkaç harfini yazıp **Tab** tuşuna basın: kabuk adı tamamlar. Hem hızlıdır hem yazım hatasını önler. **↑** tuşu önceki komutu geri getirir. Klasör ve dosya adlarında boşluk ve Türkçe karakter kullanmamayı alışkanlık edinin (**hafta01**, **odev_1.txt**): boşluklu ad terminalde tırnak ister, Türkçe karakter bazı araçlarda sorun çıkarır.

Sık yapılan hata

cd hafta01 (**hafta01** klasörü yokken) → **mkdir hafta01**, sonra **cd hafta01**
cd var olan bir klasöre *gider*, klasör *oluşturmaz*; klasör yoksa “No such file or directory” (Linux/macOS) ya da “Sistem belirtilen yolu bulamıyor” (Windows) mesajı alırsınız. Kaybolduğunuzda **pwd** (Windows'ta boş **cd**) ile nerede olduğunuza, **ls/dir** ile etrafta ne olduğuna bakın; yolun büyük/küçük harfine de dikkat edin — macOS ve Linux'ta **YpZ105** ile **YPZ105** farklı klasörlerdir.

LAB Laboratuvar Çalışması 1: Bit, bayt, dönüşüm ve terminal

Bu haftanın laboratuvarında bilgisayar yalnız bir hesap makinesi ve bir terminal olarak kullanılır; program yazılmaz. Görevleri sırayla yapın, sonuçlarınızı yanınızdaki arkadaşınızla karşılaştırın; farklı çıkan her sayı iyi bir tartışma konusudur.

Görev 1. Bilgisayarınızı tanıyın. Laboratuvar bilgisayarının sistem bilgisi ekranından işlemci saat hızını (GHz), RAM miktarını (GB) ve disk kapasitesini (GB) okuyun. RAM'i bayta çevirin (1 GB = 2^{30} B; 8 GB için 8 589 934 592 B olmalı). Diskin üreticinin onluk GB'siyle mi işletim sisteminin ikili GB'siyle mi yazıldığını iki değeri karşılaştırarak bulun. Ek: 256 GB'lık (onluk) bir diske **Örnek 1.6**'deki ham görüntüden kaç tane siğar? (Beklenen: 41 152.)

Görev 2. Dönüşüm zinciri. Öğrenci numaranızın son üç rakamını bir onluk sayı ola-

rak alın; bölme–kalanla ikiliye, gruplamayla onaltılığa çevirin; sonra ağırlık toplamıyla onluğa geri dönüp aynı sayıyı bulduğunuz doğrulayın. Ardından üç değeri de arkadaşınızla karşılıklı kontrol edin. Ek: doğum yılınızı ikiliye çevirin ($2007 = 11111010111_2$ gibi 11 bit çıkmalıdır).

Görev 3. ASCII ile mesaj (eşli). Tablo 1.3’yi ve Örnek 1.5’i örnek alarak dört–altı harflik büyük harfli bir sözcüğü onluk ASCII kodlarıyla yazın ve yalnız kodları arkadaşınıza verin; arkadaşınız sözcüğü çözsün. Sonra aynı sözcüğü *ikili* kodlarla verin. Kontrol: “YZ” → 89 90. Ek: küçük harfe çevirmek için her koda ne eklenir?

Görev 4. Fotoğraf boyutu. Telefonunuzun kamera çözünürlüğünü bulun (örneğin 3000×4000 piksel, 12 MP). Ham RGB boyutunu bayt ve MB olarak hesaplayın (3000×4000 için $\approx 34,33$ MB). Sonra galeriden gerçek bir fotoğrafın dosya boyutuna bakın ve sıkıştırma oranını (ham / dosya) bulun. Sınıfça bulduğunuz oranları karşılaştırın; neden farklı çıktıklarını tartışın.

Görev 5. Terminalde klasör yapısı. Terminali açın; `pwd` (Windows Komut İsteminde `cd`) ile nerede olduğunuzu görün. Ev klasörünüzde YPZ105 klasörünü, içinde `hafta01` klasörünü oluşturun; `hafta01`’e girin ve mutlak yolu not edin. Metin düzenleyiciyle (Not Defteri / TextEdit) bu klasöre `notlar.txt` adlı bir dosya kaydedin; içine bugünkü üç dönüşüm sonucunuzu yazın. Terminale dönüp `ls/dir` ile dosyanın görüldüğünü, `cat/type` ile içeriğinin okunduğunu doğrulayın. Son olarak `cd ..` ile YPZ105’e çıkıp `ls/dir` çıktısını eğitime gösterin.

Görev 6. Taşma deneyi (kâğıtta). 8 bitlik işaretli bir sayacı 250’den başlatıp birer birer 8 kez artırın; her adımda ikili ve onluk değeri yazın. Kaçınıcı adımda ne olur? Aynı deneyi işaretli yorumla 125’ten başlayarak yapın; iki sonucu Şekil 1.4 üzerinde gösterin.

Teslim. Görev 5’in klasör yapısı ve dosyası ders bitmeden gösterilir; kâğıt üzerindeki hesapları saklayın — Kısa Sınav 1 (3. hafta) bu haftayı da kapsar.

Özet

Kavram	Özü	Not
Von Neumann	işlemci + bellek + G/Ç + veri yolu	program da veri gibi bellekte; getir–çöz–yürüt–yaz
Bellek hiyerarşisi	yazmaç > ön bellek > RAM > SSD > disk	yukarı: hızlı, küçük, pahalı; RAM uçucu
Birimler	1 B = 8 b; 1 KB = 1024 B; MB, GB, TB	SI: 10^3 ; IEC: KiB, MiB, GiB
Dönüşüm	ikili → onluk: 1 bitlerin ağırlıklarını topla; onluk → ikili: 2’ye böl, kalanları ters oku	ağırlıklar sağdan 1, 2, 4, 8, ...; konumsal gösterim $\sum d_i b^i$
Onaltılık	4 bit = 1 rakam; A–F = 10–15	0x, renk kodları #RRGGBB
İkili toplama	$1 + 1 = 10$, elde	sıgmayan bit düşer → taşma
Tam sayılar	n bit → 2^n desen	işaretsiz $0 \dots 2^n - 1$; işaretli $-2^{n-1} \dots 2^{n-1} - 1$
Kayan nokta	anamlı kısım $\times 2^{\text{üs}}$	0,1 tam saklanamaz; 15–16 basamak

Özet (devam)

Karakter	ASCII 128; Unicode kod noktası; UTF-8 1-4 bayt	A= 65, a= 97, 0= 48; Türkçe harf 2 bayt
Görüntü	piksel $\times$ 3 bayt (RGB)	Full HD ham 6 220 800 B
Ses	hız $\times$ derinlik/8 $\times$ kanal $\times$ süre	CD: 176 400 B/s
Katmanlar	donanım < işletim sistemi < uygulama	makine kodu / assembly / yüksek seviye
Terminal	pwd/cd, ls/dir, cd ..., mkdir	yol: mutlak / görelî; .. üst klasör

Alıştırmalar

Alıştırma 1.1. ●●● Aşağıdakileri *donanım*, *işletim sistemi* ya da *uygulama yazılımı* olarak sınıflandırın: klavye, Windows 11, web tarayıcısı, RAM, SSD, Python yorumlayıcısı, Android, hesap makinesi uygulaması, ekran kartı, Not Defteri.

Cevap: donanım: klavye, RAM, SSD, ekran kartı; işletim sistemi: Windows 11, Android; uygulama: web tarayıcısı, Python yorumlayıcısı, hesap makinesi, Not Defteri

Alıştırma 1.2. ●●● (a) 45, (b) 100 ve (c) 2026 sayılarını ikiliye; (d) 1101_2 , (e) 10000000_2 ve (f) 01010101_2 sayılarını onluğa çevirin.

Cevap: (a) 101101 (b) 1100100 (c) 1111101010 (d) 13 (e) 128 (f) 85

Alıştırma 1.3. ●●● (a) $1F_{16}$, (b) $A0_{16}$ ve (c) $7FF_{16}$ sayılarını onluğa; (a) ve (b)'yi ayrıca 8 bitlik ikiliye çevirin. (d) 11001010_2 sayısını onaltılık ve onluk yazın. (e) Web renk kodu #FFFFFF hangi renktir ve onluk değeri nedir?

Cevap: (a) $31 = 00011111_2$ (b) $160 = 10100000_2$ (c) 2047 (d) $CA_{16} = 202$ (e) beyaz; 16 777 215

Alıştırma 1.4. ●●● Ev klasörü /home/ayse olan bir kullanıcı terminalde sırayla şu komutları veriyor: cd Belgeler, mkdir YPZ105, cd YPZ105, mkdir hafta01, cd ... (a) Son komuttan sonra çalışma dizini nedir? (b) hafta01 klasörünün mutlak yolu nedir? (c) Bu noktada ls ne listeler? (d) Aynı yapı Windows'ta C:\Users\ayse altında kurulsa hafta01'in yolu nasıl yazılır?

Cevap: (a) /home/ayse/Belgeler; (b) /home/ayse/Belgeler/YPZ105/hafta01; (c) YPZ105 (ve Belgeler'de önceden ne varsa); (d) C:\Users\ayse\Belgeler\YPZ105\hafta01

Alıştırma 1.5. ●●● Aşağıdaki 8 bitlik ikili toplamaları yapın; sonucu onlukla doğrulayın ve taşma olup olmadığını söyleyin (işaretsiz yorum): (a) $00101101 + 01100100$, (b) $10000001 + 10000000$.

Cevap: (a) $10010001 = 145$, taşma yok (b) 100000001 : 9 bit; 8 bitte 00000001 kalır, taşma var ($129 + 128 = 257 > 255$)

Alıştırma 1.6. ●●● (a) Üreticinin "512 GB" (512×10^9 B) dediği bir SSD, 1 GB = 2^{30} B alan bir işletim sisteminde kaç GB görünür? (b) 8 GB RAM kaç bayttır? (c) 100 Mbps'lik bir bağlantıda 1,5 GB'lık ($1,5 \times 10^9$ B) bir film en az kaç saniyede iner?

Cevap: (a) $\approx 476,84$ GB (b) 8 589 934 592 B (c) 12,5 MB/s ile 120 s = 2 dakika

Alıştırma 1.7. ●●● 16 bitlik işaretli tam sayılar için: (a) en küçük ve en büyük değer nedir? (b) -1 'in bit deseni nedir? (c) $32\,767 + 1$ işlemi ne sonuç verir? (d) Bir ses dosyasının örnekleri 16 bit işaretli tutuluyorsa bir örnek en fazla kaç farklı ses düzeyi ayırt eder?

Cevap: (a) $-32\,768 \dots 32\,767$ (b) 1111111111111111 (on altı adet 1) (c) $-32\,768$ (taşma) (d) 65 536

Alıştırma 1.8. ●●● 12 megapikselli (4000 $\times$ 3000) bir telefon fotoğrafının ham RGB boyutunu bayt ve MB (2^{20}) olarak bulun. Aynı fotoğrafın JPEG dosyası 3 MB ise sıkıştırma oranı kaçtır? Fotoğraf gri tonlu çekilseydi ham boyut ne olurdu?

Cevap: 36 000 000 B $\approx 34,33$ MB; oran ≈ 12 ; gri tonlu: 12 000 000 B

Alıştırma 1.9. ●●● (a) 80 89 84 72 79 78 ASCII kodlarının gizlediği sözcüğü bulun. (b) “YZ” sözcüğünün büyük harfli kodlarını yazın; sonra küçük harfli hâlinin (“yz”) kodlarını tablo kullanmadan bulun. (c) “Ali” ve “Ali!” kaç bayttır?

Cevap: (a) PYTHON (b) 89 90; küçük harf +32: 121 122 (c) 3 ve 4 bayt

Alıştırma 1.10. ●●● Bir stüdyo kaydı 48 000 Hz, 24 bit, stereo yapılıyor. (a) Saniyede kaç bayt üretilir? (b) Bir saatlik kayıt kaç bayt ve kaç GB (2^{30}) tutar? (c) 64 GB'lık (64×10^9 B) bir hafıza kartına en fazla kaç tam saat sığar?

Cevap: (a) 288 000 B/s (b) 1 036 800 000 B $\approx 0,97$ GB (onluk 1,04 GB) (c) 61 saat

Alıştırma 1.11. ●●● “Şişe” ve “İstanbul” sözcükleri UTF-8'de kaç karakter ve kaç bayttır? Her karakterin sabit 4 bayt tutacağı bir kodlama kullanılsaydı “Şişe” kaç bayt olurdu? UTF-8'in bu sabit uzunluklu kodlamaya göre bir üstünlüğünü ve bir bedelini yazın.

Cevap: Şişe: 4 karakter, 6 bayt; İstanbul: 8 karakter, 9 bayt; sabit 4 bayt ile Şişe 16 bayt. Üstünlük: ASCII metinler için yer ve geriye uyumluluk; bedel: n 'inci karaktere gitmek için baştan saymak gerekir

Alıştırma 1.12. ●●● Bir üniversite, öğrencilerin ilk yılda başarısız olma riskini önceden tahmin eden bir sistem kurmak istiyor. Şekil 1.8'deki boru hattının her kutusu için bu problemde ne bulunacağını bir-iki cümleyle yazın (veri neyden oluşur, ön işleme ne yapar, model neyi öğrenir, tahmin nedir, değerlendirme nasıl yapılır). Hangi kutuların işi bu derste öğreneceğiniz programlamayla yapılır?

Cevap: Veri: geçmiş öğrencilerin notları, devam kayıtları, ders yükü ve yıl sonu durumu (etiket); ön işleme: eksikleri temizleme, harf notlarını sayıya çevirme, ölçekleme; model: bu özelliklerle başarısızlık arasındaki ilişkiyi öğrenir; tahmin: yeni öğrenci için risk puanı; değerlendirme: geçmiş yılın verisinde tahminlerin doğruluğu. Model dışındaki dört kutu doğrudan programlama işidir

Hafta 2: Algoritma ve Akış Diyagramları I: Problemden Sıralı Çözüm

Öğrenme çıktıları

Bu haftanın sonunda:

1. bir problemi anlama, girdi ve çıktıyı belirleme, tasarım ve sınamaya adımlarından oluşan problem çözme sürecini küçük problemlere uygulayabileceksiniz;
2. algoritma kavramını tanımlayıp verilen bir adım listesinin algoritmanın beş özelliğini (kesinlik, sonluluk, girdi, çıktı, etkinlik) taşıyıp taşımadığını değerlendirebileceksiniz;
3. değişken, atama ve ifade kavramlarını kullanarak — tam bölme ve mod dahil — işlem önceliğine göre ifade değerlendirebileceksiniz;
4. bu kitabın sözde kod sözleşmesiyle sıralı (girdi–işlem–çıkıtı) algoritmalar yazabileceksiniz;
5. ISO 5807 sembolleriyle akış diyagramı çizip sözde kod ile akış diyagramı arasında dönüşüm yapabileceksiniz;
6. bir algoritmayı izleme tablosuyla elle çalıştırıp örnek girdi–beklenen çıktı tablosuyla sınayabileceksiniz.

Anahtar kavramlar

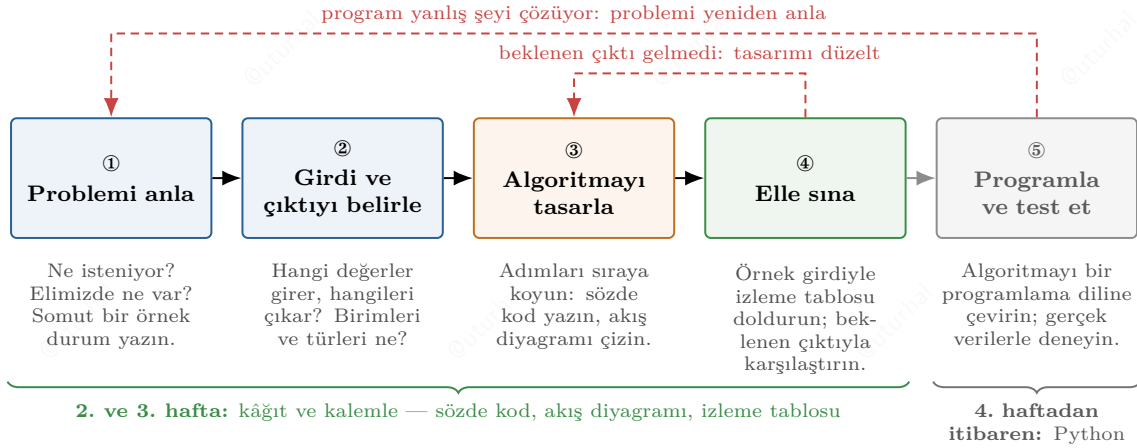
algoritma (İng. *algorithm*) **sıralı yapı** (İng. *sequence*) **sözde kod** (İng. *pseudocode*) **akış diyagramı** (İng. *flowchart*) **değişken** (İng. *variable*) **atama** (İng. *assignment*) **ifade** (İng. *expression*) **işlem önceliği** (İng. *operator precedence*) **tam bölme** (İng. *integer division*) **mod** (İng. *modulo*) **sabit** (İng. *constant*) **izleme tablosu** (İng. *trace table*) **takas** (İng. *swap*) **test durumu** (İng. *test case*)

Geçen hafta bilgisayarın ne olduğunu gördük: bellekteki bir komut listesini, hiç yorum katmadan, sırayla yürüten bir makine. Bu hafta o komut listesini *yazmayı* öğrenmeye başlıyoruz — ama henüz bir programlama diliyle değil. Önce, bir problemi bilgisayarın yürütebileceği kesin adımlara nasıl böleceğimizi öğreneceğiz; bu adım listesine *algoritma* denir. Algoritmayı iki biçimde yazacağız: Türkçe anahtar sözcüklerle *sözde kod* olarak ve standart sembollerle *akış diyagramı* olarak. Sonra da yazdığımızı, bilgisayar gibi davranarak kâğıt üzerinde çalıştıracğız.

Neden Python’a hemen geçmiyoruz? Çünkü programlamada zor olan şey dilin yazım kuralları değil, problemi adımlara bölmektir. Bir yapay zeka mühendisinin işi de büyük ölçüde budur: “müşterinin gelecek ay ayrılma olasılığını tahmin et” gibi bulanık bir isteği, önce veri akışı, sonra hesap adımları, en sonunda kod olarak yazmak. Bu haftaki her algoritma *sıralı* yapıdadır: adımlar yukarıdan aşağı, birer kez, hep aynı sırayla yürütülür. Karar vermeyi ve tekrar etmeyi 3. haftada ekleyeceğiz; 4. haftada ise bu haftanın akış diyagramlarını Python’da çalıştıracaksınız.

2.1 Problem çözme süreci

Bir problemi çözmek için oturup hemen program yazmaya başlamak, bir binayı plan çizmeden inşa etmeye benzer: küçük bir kulübe için işe yarayabilir, ama daha büyük her şeyde yıkıp yeniden yapmak gerekir. Bilgisayar bilimlerinde problem çözme, birbirini izleyen ve gerektiğinde başa dönen beş aşamalı bir döngü olarak düşünülür (Şekil 2.1).



Program yazmak, problem çözmenin son adımıdır; ilk adımı değil. Deneyimsiz programcı doğrudan ⑤'e atlar ve ne istendiğini kod yazarken anlamaya çalışır; deneyimli programcı ①-④'ü kâğıtta bitirir, çünkü kâğıttaki bir hatayı düzeltmek programdaki aynı hatayı bulmaktan kat kat ucuzdur. Kesikli oklar döngünün özüdür: sınama başarısız olduğunda geri dönülür ve yeniden denir.

Şekil 2.1: Problem çözme döngüsü. Bu hafta ve gelecek hafta ilk dört aşamada, yani kâğıt üzerinde kalıyoruz; beşinci aşama 4. haftada Python ile başlar. Kesikli oklar, sınama başarısız olduğunda geri dönüşü gösterir.

Aşamaları somut bir problemle izleyelim: “Bir öğrencinin dönem sonu notunu hesapla.”

- ① **Anlama** Cümle eksik: not nasıl hesaplanıyor? Ders bilgi paketine bakarız: vize %40, final %60. Somut bir örnek yazarız: vize 65, final 80 ise not $0,4 \cdot 65 + 0,6 \cdot 80 = 26 + 48 = 74$ olmalı. Bu örnek, ileride çözümümüzü sınayacağımız ilk *test durumu* olur.
- ② **Girdi ve çıktı**
Girdiler: vize notu ve final notu (ikisi de 0–100 arası sayılar). Çıktı: dönem sonu notu (bir sayı). Ağırlıklar girdi değildir; sabittir.
- ③ **Tasarım** Adımlar: iki notu al; $0,4 \cdot \text{vize} + 0,6 \cdot \text{final}$ hesapla; sonucu göster. Bu adımları 2.4'te sözde kod, 2.5'te akış diyagramı olarak yazacağız.
- ④ **Sınama** Adımları 65 ve 80 ile elle yürütür, 74 bulduğumuzu doğrularız. Sonra uç durumları deneriz: ikisi de 100 ise 100, ikisi de 0 ise 0 çıkmalı.
- ⑤ **Programlama**
4. haftadan itibaren: adımlar Python'a çevrilir ve gerçek girdilerle çalıştırılır.

Bu beş aşamanın en çok atlanana birincisidir. “Ne isteniyor?” sorusuna verilen yanlış bir cevap, sonraki dört aşamanın hepsini boşa çıkarır; bu yüzden deneyimli programcılar problemi kendi cümleleriyle yeniden yazar ve bir örnek girdi-çıkı çifti bulmadan tasarıma geçmez.

İpucu

Bir problemi anladığınızı sınamanın en iyi yolu, çözümü bilmeden *bir örnek girdi için çık-tının ne olması gerektiğini* yazabilmektir. Yazamıyorsanız problem henüz anlaşılmamıştır; tasarıma geçmeyin, soru sorun.

2.2 Algoritma nedir?

Tanım 2.1: Algoritma

Algoritma (*Ing. algorithm*), bir problemi çözmek için izlenecek, sonlu sayıda, kesin ve sıralı adımdan oluşan bir yönerge. Algoritmayı uygulayan — insan ya da makine — adımların ne anlama geldiğini yorumlamak zorunda kalmaz; yalnız yazılanı yapar.

Sözcük, 9. yüzyılda Bağdat'ta çalışan matematikçi El-Hârizmî'nin adından gelir; onun onluk sistemle hesap yapma kurallarını anlatan kitabı Latinceye çevrilirken adı *algoritmi* olmuştu. Kavram ise çok daha eskidir: Öklid'in iki sayının en büyük ortak bölenini bulma yöntemi (3. haftada göreceğiz) iki bin yılı aşkın süredir kullanılan bir algoritmadır. Bilgisayar, algoritmayı icat etmedi; yalnız onu yorulmadan ve hatasız uygulayan ilk makine oldu.

Günlük hayat algoritmalarla doludur: yemek tarifi, mobilya kurulum kılavuzu, ATM'den para çekme adımları, bir okul kaydının yapılması sırası. Ancak günlük tariflerin çoğu bilgisayar için yeterince kesin değildir. “Tuzu ayarlayın” cümlesini bir aşçı anlar; makine anlamaz. Bir adım listesinin gerçekten algoritma sayılması için beş özelliği taşıması gerekir (Tablo 2.1).

Tablo 2.1: Bir algoritmanın beş özelliği ve her birini bozan tipik bir adım.

Özellik	Anlamı	Bozan adım örneği
Kesinlik	Her adım tek anlamlıdır; iki kişi aynı adımdan aynı şeyi anlar.	“Biraz bekle”, “yeterince karıştır”.
Sonluluk	Adımlar sonlu sayıdadır ve yürütme bir noktada biter.	“Sonucu 2 ile çarp, sonra başa dön” (hiç bitmez).
Girdi	Dışarıdan alınan veriler (sıfır ya da daha çok) açıkça bellidir.	“İki sayının toplamını bul” — hangi iki sayı?
Çıktı	En az bir sonuç üretir ve bu sonuç dışarıya verilir.	Kareyi hesaplar, ama hiçbir yere yazmaz.
Etkinlik	Her adım, uygulayıcının yapabileceği temel bir işlemdir.	“En yakın yıldız git ve sıcaklığı ölç.”

Sık yapılan hata

Suyu biraz ısıt, sonra çayı demle → Suyu 100 °C'ye kadar ısıt, 5 g çayı ekle, 4 dakika bekle

Kesinlik eksikliği, algoritma yazmaya yeni başlayanların en sık hatasıdır. Kural şudur: bir adımı okuyan kişi “ne kadar?”, “hangisi?”, “ne zamana kadar?” diye soruyorsa adım kesin değildir. Bilgisayar bu soruları soramaz; yapamaz ya da yanlış yapar. Adımınızı, hiç çay demlememiş birine yazdığınızı düşünün.

Tanım 2.2: Sıralı yapı

Sıralı yapı (*Ing. sequence*), adımların yazıldıkları sırayla, her biri tam bir kez ve hiçbir adım atlanmadan yürütülmesidir. Yapısal programlamanın üç temel yapısından ilkidir; öteki ikisi — *karar* (bir koşula göre yol seçme) ve *tekrar* (bir adım grubunu yineleme) — 3. haftanın konusudur. Bu haftaki her algoritma yalnız sıralı yapıdan oluşur.

Sıralı yapı basit görünür; ama 2.7'de göreceğiniz gibi yalnız sıralı adımlarla bile saniyeleri saat-dakika-saniyeye ayırmak, para üstünü banknotlara bölmek, bir sayının basamaklarını ayırmak mümkündür. Üstelik sıranın kendisi bir hata kaynağıdır: iki adımın yerini değiştir-

mek doğru bir algoritmayı bozabilir; bunu iki değişkeni takas ederken göreceğiz.

Not

Bir yapay zeka modelini eğitmek de bir algoritmadır: veriyi oku, sayıya çevir, modelin parametrelerini rastgele başlat, her örnek için tahmin yap, hatayı ölç, parametreleri hatayı azaltacak yönde düzelt, bunu binlerce kez yinele, sonunda modeli kaydet. Bu listenin çoğu 3. haftada göreceğiniz *tekrar* yapısıdır; ama her yinelemenin içi, bu haftaki gibi düz bir *sıralı* adım dizisidir.

2.3 Değişken, atama ve ifadeler

Bir algoritma veriyle çalışır: okunan iki not, hesaplanan ortalama, ara sonuç olan kalan. Bu değerlerin adımlar arasında bir yerde durması ve bir adla anılması gerekir.

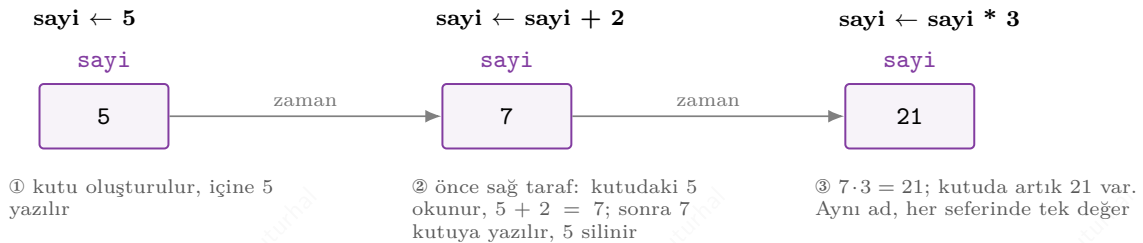
Tanım 2.3: Değişken ve atama

Değişken (*İng. variable*), üzerinde bir ad yazan ve tek bir değer tutan kutudur. Kutuya değer koymaya **atama** (*İng. assignment*) denir ve

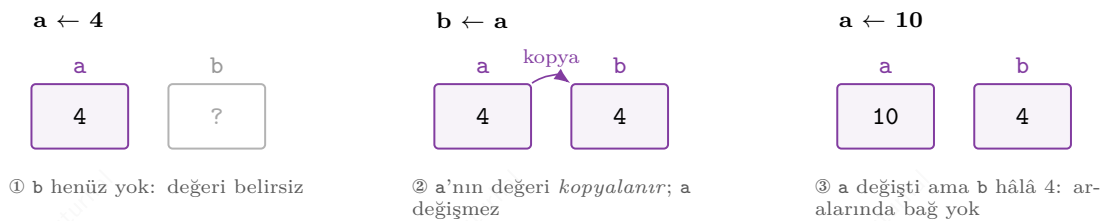
$$\text{ad} \leftarrow \text{ifade}$$

biçiminde yazılır: önce sağdaki **ifade** (*İng. expression*) hesaplanır, sonra çıkan değer soldaki kutuya yazılır; kutudaki eski değer silinir (Şekil 2.2). Bir değişkenin değeri yalnız yeni bir atama ya da **OKU** ile değişir.

Bir değişken, üç atama



İki değişken: $b \leftarrow a$ kopyalar, bağ kurmaz



Algoritma düzeyinde bir değişken, üzerinde adı yazan ve tek bir değer tutan kutudur. Atama oku ($\leftarrow$) “sağdakini hesapla, sonucu soldaki kutuya koy” demektir; bu yüzden $\text{sayi} \leftarrow \text{sayi} + 2$ bir çelişki değil, “kutudakine 2 ekle” komutudur. Bir kutunun içi ancak yeni bir atamayla ya da **OKU** ile değişir; başka bir kutunun değişmesi onu etkilemez. Python’un bu modele getirdiği küçük düzeltmeyi 4. haftada göreceğiz; bu hafta kutu benzetmesi eksiksiz çalışır.

Şekil 2.2: Değişken kutusu ve atama. Üstte tek bir kutunun üç atamayla değişen içeriği; altta $b \leftarrow a$ atamasının değeri *kopyaladığı*, iki kutu arasında kalıcı bir bağ kurmadığı görülüyor.

Ok işaretinin yönü önemlidir: veri sağdan sola akar. $\text{sayi} \leftarrow \text{sayi} + 2$ yazımı matematikte anlamsız bir denklem gibi görünür; algoritmada ise son derece olağan bir komuttur:

“kutudaki değeri oku, 2 ekle, sonucu aynı kutuya yaz.” Sağ tarafta kutunun *eski* değeri, atamadan sonra kutuda *yeni* değeri vardır.

✗ Sık yapılan hata

$5 \leftarrow \text{sayi} \rightarrow \text{sayi} \leftarrow 5$

Atamanın solunda her zaman bir değişken adı bulunur; sağında bir ifade. $5 \leftarrow \text{sayi}$ “5 kutusuna sayı yaz” demeye çalışır, oysa 5 bir kutu değil bir değerdir. Aynı biçimde $a + b \leftarrow \text{toplam}$ de yanlıştır: $a + b$ bir kutu adı değildir, hesaplanacak bir ifadedir; doğrusu $\text{toplam} \leftarrow a + b$.

Değişken adları harfle başlar, boşluk içermez ve neyi tuttuğunu söyler: *ortalama* iyidir, *x* yalnız kısa ve geçici hesaplarda kabul edilir. Bu notlardaki sözde kodda adlar küçük harfle ve Türkçe karakter kullanmadan yazılır (*ucret*, *yuzler*), çünkü 4. haftadan itibaren aynı adları Python’da kullanacaksınız ve orada *ş*, *ğ* gibi harfler kaçınılması gereken sorunlar çıkarır.

2.3.1 İfadeler ve işlemler

Tablo 2.2: Sözde kodda kullanılan aritmetik işlemler. Örneklerde $a = 7$, $b = 2$.

İşlem	Anlamı	Örnek	Sonuç
+	toplama	$a + b$	9
-	çıkarma	$a - b$	5
*	çarpma	$a * b$	14
/	bölme (sonuç ondalıklı olabilir)	a / b	3.5
//	tam bölme (bölümün tam kısmı)	$a // b$	3
mod	kalan	$a \text{ mod } b$	1
^	üs alma	$a ^ b$	49

Tablodaki ilk dört işlem hesap makinesinden tanıdık; son üçü değildir ve bu haftanın en çok işe yarayan araçlarıdır. **Tam bölme** (*Ing. integer division*) $//$ bir sayının içinde ötekinden kaç tam tane bulunduğunu, **mod** (*Ing. modulo*) ise geriye ne kaldığını verir: 17’nin içinde üç tane 5 vardır ve 2 artar, yani $17 // 5 = 3$ ve $17 \text{ mod } 5 = 2$. İkisi her zaman birlikte çalışır ve şu eşitliği sağlar:

$$(a // b) \cdot b + (a \text{ mod } b) = a \quad \text{örneğin} \quad 3 \cdot 5 + 2 = 17.$$

Bir sayı ötekine tam bölünüyorsa kalan sıfırdır: $20 // 5 = 4$, $20 \text{ mod } 5 = 0$. Bölünen bölenden küçükse bölüm sıfır, kalan sayının kendisidir: $3 // 5 = 0$, $3 \text{ mod } 5 = 3$. Bu derste tam bölme ve modu pozitif tam sayılarla kullanacağız.

i İpucu

$//$ ve mod ikilisini “bir bütünü paketlere ayırma” olarak düşünün: 17 yumurtayı 5’lik kutulara koyarsanız $17 // 5 = 3$ dolu kutu ve $17 \text{ mod } 5 = 2$ açıkta yumurta olur. Saniyeyi dakikaya (60’lık paketler), kuruşu liraya (100’lük paketler), bir sayıyı basamaklarına (10’luk paketler) ayırmak hep aynı işlemdir.

Kural 2.1 (İşlem önceliği). Bir ifadede önce *parantez içi*, sonra *üs alma* ($^$), sonra *çarpma-bölme grubu* ($*$, $/$, $//$, mod), en son *toplama-çıkarma* ($+$, $-$) yapılır. Aynı seviyedeki işlemler soldan sağa yürütülür. Karar veremediğinizde parantez kullanın: fazla parantezin bedeli yoktur, yanlış önceliğin vardır.

Kuralı üç ifadede uygulayalım. $2 + 3 * 4 ^ 2$: önce üs, $4^2 = 16$; sonra çarpma, $3 \cdot 16 = 48$; en son toplama, $2 + 48 = 50$. $(2 + 3) * 4$: parantez önce, $5 \cdot 4 = 20$. $100 - 30 - 20$: aynı seviye, soldan sağa: $70 - 20 = 50$ (sağdan başlansaydı $100 - 10 = 90$ çıkardı, yanlış). Bölme de öyledir: $100 / 10 / 2 = 10/2 = 5$.

Dikdörtgenin alanı ve çevresini hesaplayan iki atama, önceliğin neden önemli olduğunu gösterir:

Sözde kod: Dikdörtgen: parantezin gerekli olduğu yer

```
OKU en, boy
alan ← en * boy
cevre ← 2 * (en + boy)
YAZ alan, cevre
```

6 ve 4 için alan 24, çevre $2 \cdot (6+4) = 20$. Parantez unutulursa $2 * \text{en} + \text{boy} = 12+4 = 16$ olur: yazım geçerli, sonuç yanlış. Bu tür hatayı yalnız elle kontrol yakalar; bu yüzden her çözümde bir **Kontrol** adımı vardır.

Sık yapılan hata

$\text{ort} \leftarrow a + b / 2 \rightarrow \text{ort} \leftarrow (a + b) / 2$

Bölme toplamadan önce yapılır; parantezsiz yazımda yalnız b ikiye bölünür. 70 ve 85 için yanlış yazım $70 + 42,5 = 112,5$, doğru yazım $155/2 = 77,5$ verir. Bir ortalamanın en büyük girdiden büyük çıkması, formülde hata olduğunun kesin işaretidir.

2.3.2 Sabitler

Tanım 2.4: Sabit

Sabit (*İng. constant*), algoritma boyunca değeri hiç değişmeyen ve bu yüzden bir kez atayıp bir daha sol tarafta görünmeyen değişkendir. Sözde kodda adı büyük harfle yazılır: $\text{PI} \leftarrow 3.14159$, $\text{KDV_ORANI} \leftarrow 0.20$, $\text{SAATTEKI_SANIYE} \leftarrow 3600$.

Sabitin yararı iki yönlüdür. Okunabilirlik: $\text{toplam} * \text{KDV_ORANI}$, $\text{toplam} * 0.20$ yazımından daha çok şey söyler — okuyan kişi 0,20'nin ne olduğunu tahmin etmek zorunda kalmaz. Değiştirilebilirlik: KDV oranı değişirse algoritmanın tek satırı değişir, oranın geçtiği on yer aranmaz. Dönem sonu notu örneğindeki ağırlıklar (0,4 ve 0,6) tam da böyle sabitlerdir: girdi değildirlir, kullanıcıya sorulmazlar; ama ders değişirse değişebilirler.

Dikkat

Bu notlarda **ondalık ayırıcı** iki biçimde görünür. Düz yazıda Türkçe kural geçerlidir: 77,5. Sözde kodda, akış diyagramında ve izleme tablosunda ise *nokta* kullanılır: 77.5. Nedeni, 4. haftadan itibaren kullanacağınız Python'un noktayı zorunlu kılmasıdır; algoritmayı en baştan programın yazımıyla yazmak, çeviride hata payını azaltır. Sözde kodda 3,14 yazarsanız bu “3 ve 14” diye iki ayrı değer olarak okunur.

2.4 Sözde kod sözleşmesi

Tanım 2.5: Sözde kod

Sözde kod (*İng. pseudocode*), bir algoritmayı doğal dil ile programlama dili arasında bir biçimde yazmaktır: adımlar az sayıda sabit anahtar sözcükle, satır satır, ama herhangi bir dilin yazım ayrıntısına girmeden yazılır. Sözde kod bilgisayarda çalışmaz; insanın okuması ve elle yürütmesi için vardır.

Sözde kodun evrensel bir standardı yoktur; her kitap kendi sözleşmesini kurar. Bu kitabın sözleşmesi aşağıdadır ve dönem boyunca — Python’a geçtikten sonra da, çözümlerin **Algoritma** adımı — aynen kullanılır.

- Kural 2.2 (Sözde kod sözleşmesi).** (i) Algoritma **BAŞLA** ile başlar, **BİTİR** ile biter; aradaki satırlar iki boşluk içeriden yazılır.
- (ii) **OKU** *a*, *b* kullanıcıdan değer alır ve adı geçen değişkenlere, sırayla, yazar. **YAZ** ifade, ... değerleri ekrana yazar; ekrana aynen çıkacak metin çift tırnak içine alınır: **YAZ** "Ortalama:", *ort*.
- (iii) Atama *ad* ← ifade biçimindedir; Tanım 4.2 geçerlidir.
- (iv) İşlemler Tablo 2.2’deki gibidir; öncelik Kural 4.2’e uyar. Ondalık ayırıcı noktadır.
- (v) Değişken adları küçük harfle, sabitler büyük harfle yazılır; her satırda tek bir iş yapılır.
- (vi) # ile başlayan metin yorumdur; okuyucu içindir, yürütülmez.
- (vii) Karar (**EĞER**) ve tekrar (**İKEN**) sözcükleri 3. haftada eklenecektir.

Sözleşmeyi 2.1’deki dönem sonu notu probleminde uygulayalım. Tasarım adımıdaki üç cümle — “iki notu al, ağırlıklı toplamı hesapla, sonucu göster” — sözde kodda şöyle yazılır:

Sözde kod: Dönem sonu notu

```
BAŞLA
  VIZE_AGIRLIK ← 0.4          # sabitler: bir kez atanır
  FINAL_AGIRLIK ← 0.6
  OKU vize, final
  net ← vize * VIZE_AGIRLIK + final * FINAL_AGIRLIK
  YAZ "Dönem sonu notu:", net
BİTİR
```

Vize 65, final 80 girildiğinde *net* kutusuna $65 \cdot 0,4 + 80 \cdot 0,6 = 26 + 48 = 74$ yazılır ve ekranda Dönem sonu notu: 74 görünür. Öncelik kuralı gereği çarpmalar toplamadan önce yapıldığı için buraya parantez gerekmez; yine de $(vize * VIZE_AGIRLIK) + (final * FINAL_AGIRLIK)$ yazmak yanlış olmazdı.

Sık yapılan hata

YAZ *ort* (sonra) *ort* ← $(a + b) / 2$ → *ort* ← $(a + b) / 2$ (sonra) **YAZ** *ort*

Sıralı yapıda satırlar yazıldıkları sırayla yürütülür. **YAZ** *ort* satırına gelindiğinde *ort* kutusu henüz oluşmamışsa yazılacak bir şey yoktur; bilgisayar bunu hata olarak bildirir. Kural: bir değişken *kullanılmadan önce* değer almış olmalıdır — **OKU** ile ya da atamayla. İzleme tablosunda bu hata, “—” işaretli bir hücrenin okunmaya çalışılması olarak görünür.

İpucu

OKU satırından hemen önce kullanıcıya ne beklediğinizi söyleyen bir **YAZ** koymak iyi bir alışkanlıktır: **YAZ** "Vize ve final notunu girin:". Algoritmanın doğruluğunu değiştirmez; ama 4. haftada programa dönüştüğünde ekranda ne yapılacağı belli olmayan bir imleç yerine anlamlı bir soru görünür.

2.5 Akış diyagramları

Tanım 2.6: Akış diyagramı

Akış diyagramı (*İng. flowchart*), bir algoritmanın adımlarını standart geometrik sembollerle, adımlar arası geçişleri oklarla gösteren çizimdir. Semboller uluslararası ISO 5807 standardında tanımlanmıştır; bu derste standardın [Şekil 2.3](#)'teki alt kümesi kullanılır.

Sembol	Adı	Kullanımı ve sözde kod karşılığı	Örnek: iki sayının toplamı
	Uç	Diyagramın başladığı ve bittiği yer. Her diyagramda <i>tam bir</i> BAŞLA ve <i>tam bir</i> BİTİR bulunur.	
	Giriş / Çıkış	Dış dünyayla veri alışverişi: klavyeden okuma (OKU), ekrana yazma (YAZ). Paralelkenar.	
	İşlem	Hesap ve atama. Birbirini izleyen birkaç atama aynı kutuya alt alta yazılabilir.	
	Akış çizgisi	Adımların sırası. Ok ucu zorunludur; okuma yönü yukarıdan aşağı, soldan sağdır.	
	Bağlayıcı	Sütuna ya da sayfaya sığmayan diyagramı aynı harfli başka bir daireden sürdürür.	
	Karar	Bir soruya göre iki yoldan birini seçer (Evet / Hayır). 3. haftada : bu hafta kullanılmaz.	

tek giriş, tek çıkış;
her kutuya bir ok girer, bir ok çıkar

Sembolün biçimi işin türünü, içindeki yazı ne yapıldığını söyler. Bu yüzden paralelkenarın içine hesap, dikdörtgenin içine **OKU** yazılmaz. Bu haftaki her diyagram sağdaki iskeletin uzatılmış hâlidir: bir BAŞLA, bir ya da daha çok giriş, işlemler, bir ya da daha çok çıkış ve bir BİTİR — oklar hep aşağı iner. Karar sembolüyle akış ilk kez dallanacak; onu 3. haftaya bırakıyoruz.

Şekil 2.3: Bu derste kullanılan akış diyagramı sembolleri ve sağda tam bir sıralı diyagram. Karar sembolü yalnız tanıtım için gösterilmiştir; bu hafta kullanılmaz.

Akış diyagramı ile sözde kod aynı algoritmanın iki yazımıdır: diyagramdaki her kutu sözde kodda bir satıra, her ok satırlar arasındaki sıraya karşılık gelir. Diyagramın üstünlüğü görselliğidir — akışın nereden nereye gittiği bir bakışta görülür; bu üstünlük 3. haftada, akış dallanıp geri döndüğünde çok daha belirgin olacak. Sözde kodun üstünlüğü ise hızıdır: yazmak çizmekten kolaydır ve uzun algoritmalar sayfaya sığar. İkisini de bilmeniz, ekiplerde ikisinin de konuşulduğu için gerekir.

- Kural 2.3 (Akış diyagramı çizim kuralları).** (i) Her diyagramda *tam bir* BAŞLA ve *tam bir* BİTİR sembolü bulunur.
- (ii) Sıralı yapıda her sembole *bir* ok girer ve *bir* ok çıkar; oklar ok ucuyla çizilir ve birbirini kesmez.
- (iii) Akış yukarıdan aşağı ilerler; sığmayan diyagram bir *bağlayıcı* daireyle (A, B, ...) bir sonraki sütunda ya da sayfada sürer.
- (iv) Sembolün içine sözde koddaki satır aynen yazılır: paralelkenara **OKU/YAZ**, dikdörtgene atama. Birbirini izleyen birkaç atama tek dikdörtgene alt alta yazılabilir.
- (v) Sembol biçimi işin türünü belirtir; paralelkenarın içine hesap, dikdörtgenin içine **OKU** yazılmaz.

Sözde koddan akış diyagramına

1. Üste BAŞLA, alta BİTİR koyun; aralarına satır sayısı kadar yer bırakın.
2. Sözde kodun her satırını sırayla bir sembole çevirin: **OKU** ve **YAZ** → paralelkenar, atama → dikdörtgen. Sabit atamalarını ilk dikdörtgende toplayabilirsiniz.
3. Sembolleri yukarıdan aşağı oklarla bağlayın; her sembole bir giriş, bir çıkış.
4. Diyagramı okuyup sözde koda geri çevirin; aynı satırları elde etmiyorsanız çizimde bir adım eksik ya da fazladır.

Tersi de aynı yolun geri yürünmesidir: her sembol bir satır, oklar sıra.

Not

Akış diyagramı çizen ücretsiz araçlar vardır. **Flowgorithm** yalnız akış diyagramı için yapılmıştır: sembolleri menüden seçersiniz, diyagramı *çalıştırır*, kullanıcıdan değer ister ve çıktıyı gösterir — yani izleme tablosunu sizin yerinize doldurur. Genel amaçlı bir çizim aracı olan **diagrams.net** (eski adıyla draw.io) ise tarayıcıda çalışır, kurulum istemez ve akış diyagramı sembol paleti içerir; ama diyagramı çalıştırmaz. Laboratuvarda ikisini de deneyeceksiniz. Hiçbir araç, sembollerin ne anlama geldiğini bilmeyi gereksiz kılmaz; kâğıtta çizemediğinizi araçta da çizemezsiniz.

2.6 İzleme tablosu: algoritmayı elle çalıştırmak

Bir algoritma yazdınız; doğru mu? Bunu öğrenmenin bu haftaki tek yolu, algoritmayı bilgisayar gibi davranarak kendinizin çalıştırmasıdır. Bunun düzenli biçimi izleme tablosudur.

Tanım 2.7: İzleme tablosu

İzleme tablosu (*İng. trace table*), bir algoritmanın belirli bir girdi için adım adım yürütülmesinin kayıdır: her satır yürütülen bir adımı, her sütun bir değişkeni gösterir; son sütun ekrana yazılanları tutar. Tablo dolduğunda son satırdaki değerler algoritmanın o girdi için verdiği sonuçtur (**Şekil 2.4**).

İzleme tablosu doldurma

1. Algoritmadaki her değişken için bir sütun, en sağa bir *Ekrana* sütunu açın. Girdi değerlerini kâğıdın üstüne yazın.
2. **BAŞLA**'dan sonraki ilk satırdan başlayın; her satır için tabloya bir satır ekleyin.
3. Satır **OKU** ise girdi değerlerini ilgili sütunlara yazın. Atama ise sağ tarafı *tablodaki*

güncel değerlerle hesaplayıp sonucu sol taraftaki değişkenin sütununa yazın. **YAZ** ise yazılan değeri Ekran sütununa koyun.

4. Değişmeyen sütunlara bir önceki satırın değerini aynen taşıyın (ya da boş bırakıp “değişmedi” sayın; tutarlı olun).
5. **BİTİR**’de durun. Ekran sütununu beklenen çıktıyla karşılaştırın.

Sözde kod

```

1  BAŞLA
2  OKU a, b
3  toplam ← a + b
4  YAZ toplam
5  BİTİR

```

Girdi: klavyeden 3 ve 5 girilecek.

İzleme tablosu

Adım	Yürütülen satır	a	b	toplam	Ekran
1	OKU a, b	3	5	—	
2	toplam ← a + b	3	5	8	
3	YAZ toplam	3	5	8	8

① BAŞLA ve BİTİR hiçbir şeyi değiştirmez; tabloya yazılmaz

③ yalnız değişen hücre yeniden yazılır (renkli); diğerleri aynen taşınır

② yürütülen *her* satır bir tablo satırı; her değişken bir sütun

④ Ekran yalnız **YAZ** adımıyla dolar; “—” henüz değer yok demektir

İzleme tablosu, algoritmayı bilgisayarın yapacağı gibi — adım adım, yorum katmadan — kâğıtta çalıştırmaktır. Her adımda tek bir satır yürütülür, o satırın etkilediği kutu güncellenir, gerisi olduğu gibi kalır. Tablonun son satırındaki *Ekran* değeri algoritmanın çıktısıdır; bu değer beklediğinizle uyumuyorsa hata, tablonun ilk yanlış hücresinin bulunduğu satırdadır.

Şekil 2.4: İzleme tablosunun yapısı: solda sözde kod, sağda her yürütülen satıra karşılık bir tablo satırı. Renkli hücreler o adımda değişen değerlerdir; “—” henüz değer almamış değişkeni gösterir.

 Dikkat

İzleme tablosu doldururken *kâğıttaki* algoritmayı çalıştırın, aklınızdakini değil. En sık hata, “burada şunu demek istemiştım” diyerek yazılı olmayan bir adımı yapmaktır; oysa bilgisayar yalnız yazılanı yapar. Bir arkadaşınızın algoritmasını izlemek bu yüzden kendi algoritmanızı izlemekten daha güvenilirdir: arkadaşınızın ne *demek istediğini* bilmezsiniz, yalnız ne *yazdığını* görürsünüz. Laboratuvarında Görev 4 tam olarak budur.

Bu haftaki çözümlü örnekler dört yerine beş adımdan oluşur: **Analiz**, **Algoritma** (sözde kod), **Akış diyagramı**, **İzleme tablosu** ve **Kontrol**. Kod adımı 4. haftada gelecektir.

Örnek 2.1 (İki sayının ortalaması). Kullanıcıdan iki sayı alan ve aritmetik ortalamalarını yazan algoritmayı tasarlayın; 70 ve 85 girdisiyle izleyin.

Çözüm

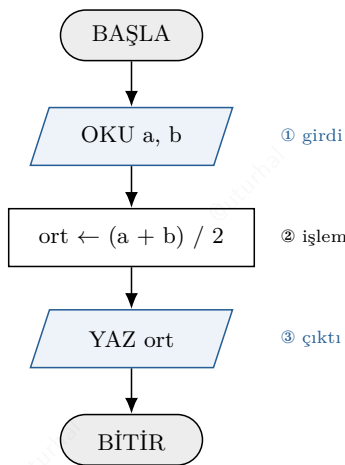
Analiz. Girdi: iki sayı, a ve b (ondalıklı olabilir). Çıktı: ortalama, tek sayı. İşlem: toplamın yarısı; toplama bölmeden önce yapılmalı, dolayısıyla parantez gerekli.

Algoritma.

Sözde kod: İki sayının ortalaması

```
BAŞLA
OKU a, b
ort ← (a + b) / 2
YAZ ort
BİTİR
```

Akış diyagramı. Şekil 2.5: bir paralelkenar (girdi), bir dikdörtgen (işlem), bir paralelkenar (çıktı). Bu üçlü, haftanın bütün diyagramlarının iskeletidir.



Aynı algoritma, sözde kodla

```
BAŞLA
OKU a, b
ort ← (a + b) / 2
YAZ ort
BİTİR
```

Diyagramdaki her kutu sözde kodda bir satırdır; oklar satırların sırasıdır. Birinden ötekine satır satır geçilir.

Her diyagramda tek BAŞLA, tek BİTİR; her kutuya bir ok girer, bir ok çıkar.

Girdi – İşlem – Çıktı, bu haftaki her algoritmanın omurgasıdır: paralelkenar dış dünyayla konuşur, dikdörtgen hesap yapar.

Parantez şarttır. $a + b / 2$ yazılıydı önce bölme yapılırdı ve 70 ile 85 için sonuç $70 + 42.5 = 112.5$ çıkardı — ortalama değil. Doğru yazımla $(70 + 85)/2 = 77.5$.

Şekil 2.5: İki sayının ortalaması: akış diyagramı ve sözde kodu yan yana. Üç bölge — girdi, işlem, çıktı — sıralı algoritmanın omurgasıdır.

İzleme tablosu. Girdi 70 ve 85:

Adım	Satır	a	b	ort	Ekran
1	OKU a, b	70	85	—	
2	$ort \leftarrow (a + b) / 2$	70	85	77.5	
3	YAZ ort	70	85	77.5	77.5

Kontrol. Elle: $(70 + 85)/2 = 155/2 = 77,5$ ✓. Makullük: ortalama iki sayının arasında olmalı, $70 < 77,5 < 85$ ✓. Uç durum: iki sayı eşitse, örneğin 50 ve 50, ortalama 50 çıkmalı; $(50 + 50)/2 = 50$ ✓. Sayılar yer değiştirirse sonuç değişmemeli: 85 ve 70 için de 77,5 ✓.

2.7 Sıralı algoritma örnekleri

Aşağıdaki örnekler yalnız sıralı yapıyla nelerin yapılabildiğini gösterir ve her biri ileriki haftalarda Python’da yeniden karşınıza çıkacaktır. Örnekleri çözümü okumadan önce kendiniz tasarlamayı deneyin; sonra karşılaştırın.

Örnek 2.2 (Sıcaklık dönüşümü). Celsius cinsinden verilen sıcaklığı Fahrenheit’a çeviren

algoritmayı yazın: $F = C \cdot 9/5 + 32$. Girdi 25 ile izleyin.

Çözüm

Analiz. Girdi: *celsius* (ondalıklı olabilir). Çıktı: *fahrenheit*. İşlem tek formüldür; $9 / 5$ bölmesi tam bölme değildir, 1,8 vermelidir.

Algoritma.

Sözde kod: Celsius'tan Fahrenheit'a

```
BAŞLA
OKU celsius
fahrenheit ← celsius * 9 / 5 + 32
YAZ fahrenheit
BİTİR
```

Akış diyagramı. Şekil 2.5 ile aynı iskelet: BAŞLA, tek girişli paralelkenar, tek dikdörtgen, tek çıkışlı paralelkenar, BİTİR. Laboratuvarında çizeceksiniz.

İzleme tablosu. Girdi 25. Öncelik: $celsius * 9 = 225$, $/ 5 = 45$, $+ 32 = 77$.

Adım	Satır	celsius	fahrenheit	Ekran
1	OKU celsius	25	—	
2	fahrenheit ← celsius * 9 / 5 + 32	25	77	
3	YAZ fahrenheit	25	77	77

Kontrol. Bilinen iki nokta formülü sınar: suyun donma noktası $0^{\circ}\text{C} = 32^{\circ}\text{F}$ ($0 \cdot 9/5 + 32 = 32$ ✓) ve kaynama noktası $100^{\circ}\text{C} = 212^{\circ}\text{F}$ ($180 + 32 = 212$ ✓). Vücut sıcaklığı 37°C için $66,6 + 32 = 98,6^{\circ}\text{F}$, tıp dizilerinden tanıdık bir sayı ✓. Yazımı $celsius * 9 // 5 + 32$ olsaydı 25 için sonuç yine 77 çıkardı ama 37 için $66 + 32 = 98$ olurdu; tam bölme ondalığı atar.

Örnek 2.3 (Saniyeyi saat, dakika ve saniyeye ayırma). Toplam saniyeyi alan ve bunu saat, dakika, saniye olarak yazan algoritmayı tasarlayın. Girdi 7384 ile izleyin.

Çözüm

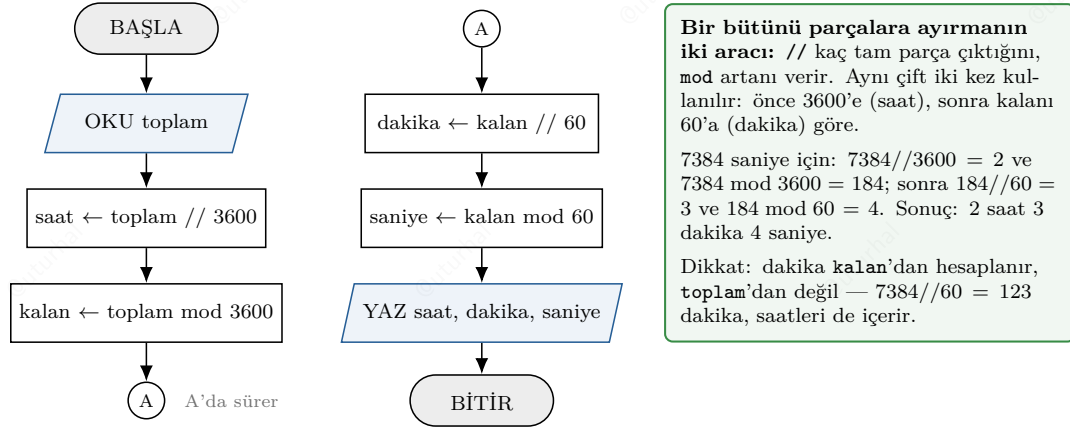
Analiz. Girdi: *toplam* (tam sayı). Çıktı: üç tam sayı. Bir saat 3600, bir dakika 60 saniyedir. Saat sayısı toplamın 3600'e tam bölümü; artan saniyeler (3600'e göre kalan) dakikaya aynı yöntemle bölünür. İki kez $//$ -mod çifti.

Algoritma.

Sözde kod: Saniye ayırıştırma

```
BAŞLA
OKU toplam
saat ← toplam // 3600
kalan ← toplam mod 3600
dakika ← kalan // 60
saniye ← kalan mod 60
YAZ saat, dakika, saniye
BİTİR
```

Akış diyagramı. Şekil 2.6. Diyagram tek sütuna sığmadığı için bağlayıcı sembolü A ile ikinci sütunda sürer.



Şekil 2.6: Saniye ayırıştırma. Aynı $//$ - $\bmod$ çifti iki kez uygulanır: önce 3600'e, sonra kalan üzerinde 60'a göre. Bağlayıcı A, diyagramın ikinci sütunda devam ettiğini gösterir.

İzleme tablosu. Girdi 7384:

Adım	Satır	toplam	saat	kalan	dakika	saniye	Ekran
1	OKU toplam	7384	—	—	—	—	
2	saat $\leftarrow$ toplam $//$ 3600	7384	2	—	—	—	
3	kalan $\leftarrow$ toplam mod 3600	7384	2	184	—	—	
4	dakika $\leftarrow$ kalan $//$ 60	7384	2	184	3	—	
5	saniye $\leftarrow$ kalan mod 60	7384	2	184	3	4	
6	YAZ saat, dakika, saniye	7384	2	184	3	4	2 3 4

Kontrol. Parçaları geri birleştirelim: $2 \cdot 3600 + 3 \cdot 60 + 4 = 7200 + 180 + 4 = 7384$ ✓. Dakika ve saniye sınırları içinde mi? $3 < 60$ ve $4 < 60$ ✓ — mod işlemi bunu garanti eder. Uç durumlar: 3600 girilirse 1 saat 0 dakika 0 saniye; 59 girilirse 0 saat 0 dakika 59 saniye; 86399 girilirse 23 saat 59 dakika 59 saniye çıkmalıdır (2.8'de tablo olarak sımayacağız).



Sık yapılan hata

$dakika \leftarrow toplam // 60 \rightarrow dakika \leftarrow kalan // 60$

Saatler ayrıldıktan sonra dakikalar *kalan* üzerinden hesaplanır, orijinal toplam üzerinden değil. $7384 // 60 = 123$ dakikadır; bu, saate çevrilmiş 120 dakikayı da içerir ve “2 saat 123 dakika” gibi anlamsız bir çıktı verir. Aynı hata para üstünde (Örnek 2.4) “87 TL için 1 elli ve 4 yirmi” biçiminde görünür. Kural: her seviyede bir öncekinin *kalanıyla* devam edin.

Örnek 2.4 (Para üstü). Bir ürünün fiyatı ile ödenen tutar (ikisi de tam TL) veriliyor. Para üstünü en az sayıda banknotla vermek için kaç adet 50, 20, 10, 5 ve 1 TL gerektiğini yazan algoritmayı tasarlayın. Girdi: fiyat 113, ödenen 200.

Çözüm

Analiz. Girdi: fiyat, ödenen (tam sayı). Çıktı: beş adet sayısı. İşlem: para üstü = ödenen - fiyat; sonra en büyük banknottan başlayarak her banknot için // adet, mod kalan; kalan bir sonraki banknota aktarılır. En küçük banknot 1 TL olduğu için son kalan doğrudan 1 TL adedidir.

Algoritma.

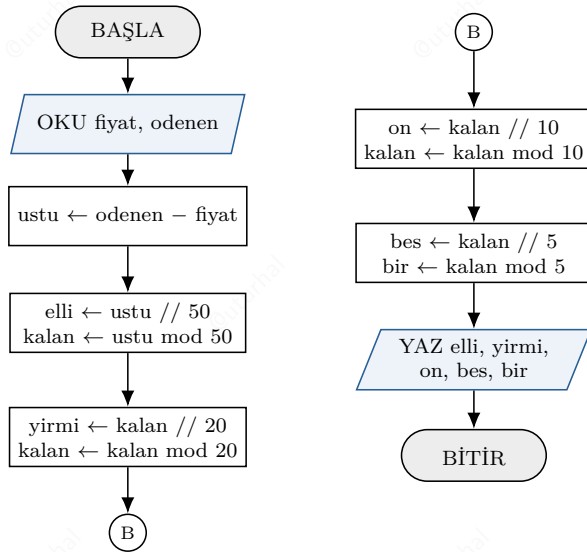
Sözde kod: Para üstü: en büyükten küçüğe

BAŞLA

```
OKU fiyat, ödenen
ustu ← ödenen - fiyat
elli ← usu // 50
kalan ← usu mod 50
yirmi ← kalan // 20
kalan ← kalan mod 20
on ← kalan // 10
kalan ← kalan mod 10
bes ← kalan // 5
bir ← kalan mod 5
YAZ elli, yirmi, on, bes, bir
```

BİTİR

Akış diyagramı. Şekil 2.7. Birbirini izleyen // ve mod atamaları, Kural 2.3 (iv) uyarınca aynı dikdörtgene yazılmıştır.



Açgözlü yöntem: en büyük banknottan başla, sıranı kadarını ver (//), artanla (mod) bir sonraki banknota geç. Banknotlar 50, 20, 10, 5 ve 1 TL.

87 TL para üstü için: $87/50 = 1$, kalan 37; $37/20 = 1$, kalan 17; $17/10 = 1$, kalan 7; $7/5 = 1$, kalan 2; yani 2 adet 1 TL. Kontrol: $50 + 20 + 10 + 5 + 2 = 87$.

kalan kutusu her adımda yeniden yazılır: yeni değeri eskisini siler, eski değere bir daha gerek yoktur.

Şekil 2.7: Para üstü algoritması. Her dikdörtgen bir banknot seviyesidir: üst satır adedi, alt satır kalanı hesaplar. **kalan** kutusu her seviyede yeniden yazılır.

İzleme tablosu. Girdi 113 ve 200. Yer kazanmak için fiyat ve ödenen sütunları ilk satırdan sonra gösterilmemiştir (değişmezler):

Adım	Satır		ustu	elli	kalan	yirmi	on	bes	bir	Ekran
1	OKU fiyat, odenen (113, 200)	—	—	—	—	—	—	—	—	
2	ustu $\leftarrow$ odenen - fiyat	87	—	—	—	—	—	—	—	
3	elli $\leftarrow$ usu // 50	87	1	—	—	—	—	—	—	
4	kalan $\leftarrow$ usu mod 50	87	1	37	—	—	—	—	—	
5	yirmi $\leftarrow$ kalan // 20	87	1	37	1	—	—	—	—	
6	kalan $\leftarrow$ kalan mod 20	87	1	17	1	—	—	—	—	
7	on $\leftarrow$ kalan // 10	87	1	17	1	1	—	—	—	
8	kalan $\leftarrow$ kalan mod 10	87	1	7	1	1	—	—	—	
9	bes $\leftarrow$ kalan // 5	87	1	7	1	1	1	—	—	
10	bir $\leftarrow$ kalan mod 5	87	1	7	1	1	1	1	2	
11	YAZ elli, yirmi, on, bes, bir	87	1	7	1	1	1	1	2	1 1 1 1 2

Kontrol. Banknotların toplamı para üstüne eşit olmalı: $1 \cdot 50 + 1 \cdot 20 + 1 \cdot 10 + 1 \cdot 5 + 2 \cdot 1 = 87$ ve $200 - 113 = 87$ ✓. Her adet, o banknotun bir büyüğünden küçük olmalı: en fazla 2 adet 20 (üçü 60 eder, 50'yi geçer), en fazla 1 adet 10, 1 adet 5, 4 adet 1 ✓. Uç durum: fiyat 200 girilirse üstü 0 ve bütün adetler 0 çıkar — deneyin. Fiyat ödenenden büyük girilirse ne olur? Algoritma negatif bir üstüyle devam eder ve anlamsız sonuç verir; bu durumu yakalamak için 3. haftanın karar yapısı gerekir.

Örnek 2.5 (iki değişkeni takas etme). İki değişkenin değerlerini karşılıklı olarak değiştiren (takas eden) algoritmayı yazın: $a = 3$, $b = 8$ girildiğinde çıktı 8 ve 3 olmalı.

Çözüm

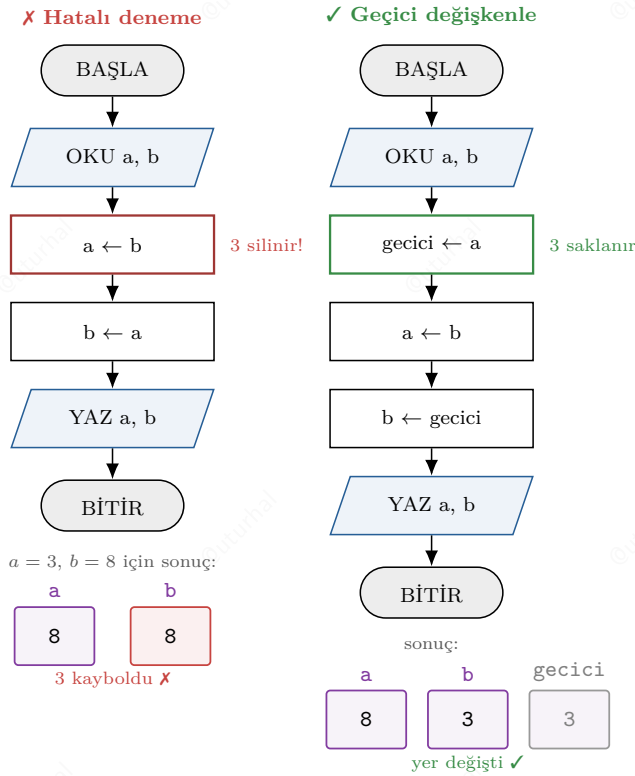
Analiz. Girdi: a , b . Çıktı: aynı iki değer, yer değiştirmiş. İlk akla gelen çözüm — $a \leftarrow b$, sonra $b \leftarrow a$ — yanlıştır: ilk atama a 'nın eski değerini siler, ikinci atama silinmiş değeri arar. Yok olacak değer önce üçüncü bir kutuya alınmalıdır.

Algoritma.

Sözde kod: Takas: geçici değişkenle

```
BAŞLA
OKU a, b
gecici  $\leftarrow$  a      # a'nın değerini güvenceye al
a  $\leftarrow$  b
b  $\leftarrow$  gecici
YAZ a, b
BİTİR
```

Akış diyagramı. Şekil 2.8; solda hatalı deneme, ortada doğru çözüm yan yana.



İki bardaktaki sıvıyı yer değiştirmek için üçüncü bir boş bardak gerekir. Bir kutuya yeni değer yazmak eskisini siler; $a \leftarrow b$ yapıldığı anda a 'daki 3 gitmiştir ve $b \leftarrow a$ artık 8'i 8 ile değiştirir.

Çözüm: yok olacak değeri önce **gecici** adlı üçüncü bir kutuya al, sonra iki atamayı yap. Üç atama, üç kutu — bilgisayar bilimlerinde *takas* (*swap*) denen bu üçlü, sıralama algoritmalarının temel taşıdır (13. hafta).

Soldaki diyagram gerçek bir tuzaaktır: yazım hatası yok, her adım kurallara uygun, yalnız *sıra* yanlış. Bu tür hatayı yalnız izleme tablosu yakalar.

Şekil 2.8: Takas. Solda iki atamalı hatalı deneme: a 'nın eski değeri ilk atamada kaybolur ve iki kutu da 8 olur. Ortada geçici değişkenli doğru çözüm: üç atama, üç kutu.

İzleme tablosu. Önce hatalı deneme, girdi 3 ve 8:

Adım	Satır	a	b	Ekran
1	OKU a, b	3	8	
2	$a \leftarrow b$	8	8	
3	$b \leftarrow a$	8	8	
4	YAZ a, b	8	8	8 8 ✗

Sonra doğru çözüm:

Adım	Satır	a	b	gecici	Ekran
1	OKU a, b	3	8	—	
2	$gecici \leftarrow a$	3	8	3	
3	$a \leftarrow b$	8	8	3	
4	$b \leftarrow gecici$	8	3	3	
5	YAZ a, b	8	3	3	8 3 ✓

Kontrol. Çıktı 8 ve 3: girdinin tersi ✓. Adım 3'te iki kutunun da 8 olması sizi ürkütmesin: 3 o sırada **gecici**'de güvendedir ve adım 4'te geri gelir. Uç durum: $a = b = 5$ için algoritma 5 ve 5 verir; takasın fark edilmemesi doğrudur. Takas üçlüsünü aklınızda tutun: 13. haftada bir listeyi sıralarken onlarca kez kullanacaksınız.

**Sık yapılan hata**

$a \leftarrow b$; $b \leftarrow a \rightarrow$ $gecici \leftarrow a$; $a \leftarrow b$; $b \leftarrow gecici$

Bu hata yazım açısından kusursuzdur: her satır **Kural 2.2**'ye uygundur. Yanlış olan tek şey *sıradır* — ve sıralı yapıda sıra her şeydir. Bu tür hataları bulmanın tek yolu izleme tablosudur; algoritmayı “okuyup” doğru görünmesine güvenmek yeterli değildir. Kısa Sınav 1’de bir takas sorusu görürseniz şaşırmayın.

Örnek 2.6 (Bir sayının basamaklarını ayırma). Üç basamaklı bir sayı alan; yüzler, onlar ve birler basamağını ayrı ayrı, ayrıca basamakların toplamını ve sayının rakamları ters sıralanmış hâlini yazan algoritmayı tasarlayın. Girdi 472.

Çözüm

Analiz. Girdi: n (üç basamaklı tam sayı). Çıktı: üç basamak, toplam, ters sayı. Onluk sistemde $472 = 4 \cdot 100 + 7 \cdot 10 + 2$ (geçen haftanın konumsal gösterimi). Yüzler basamağı 100’e tam bölüm; birler basamağı 10’a göre kalan; onlar basamağı, 100’e göre kalanın (72) 10’a tam bölümü. Ters sayı, basamakların ağırlıkları ters çevrilerek kurulur.

Algoritma.

Sözde kod: Basamak ayırma

```
BAŞLA
OKU n
yuzler  $\leftarrow n // 100$ 
onlar  $\leftarrow (n \bmod 100) // 10$ 
birler  $\leftarrow n \bmod 10$ 
toplam  $\leftarrow yuzler + onlar + birler$ 
ters  $\leftarrow birler * 100 + onlar * 10 + yuzler$ 
YAZ yuzler, onlar, birler
YAZ toplam, ters
BİTİR
```

Akış diyagramı. Yine tek yol: BAŞLA, bir giriş, beş dikdörtgen (ya da aynı dikdörtgende beş satır), iki çıkış, BİTİR. **Şekil 2.6**’daki gibi iki sütuna bölmek gerekir; laboratuvarıda çeceksiniz.

İzleme tablosu. Girdi 472; n hiç değişmediği için sütunu gösterilmemiştir:

Adım	Satır	yuzler	onlar	birler	toplam	ters	Ekran
1	OKU n (472)	—	—	—	—	—	
2	$yuzler \leftarrow n // 100$	4	—	—	—	—	
3	$onlar \leftarrow (n \bmod 100) // 10$	4	7	—	—	—	
4	$birler \leftarrow n \bmod 10$	4	7	2	—	—	
5	$toplam \leftarrow yuzler + onlar + birler$	4	7	2	13	—	
6	$ters \leftarrow birler * 100 + onlar * 10 + yuzler$	4	7	2	13	274	
7	YAZ yuzler, onlar, birler	4	7	2	13	274	4 7 2
8	YAZ toplam, ters	4	7	2	13	274	13 274

Kontrol. Basamaklar geri birleştirilirse sayı çıkmalı: $4 \cdot 100 + 7 \cdot 10 + 2 = 472$ ✓. Toplam $4 + 7 + 2 = 13$ ✓; ters sayı 274, okunuşuyla da doğru ✓. Adım 3’teki parantez, soldan sağa kuralı gereği aslında gereksizdir: $n \bmod 100 // 10$ de 7 verir.

Ama parantezsiz yazımı okuyan herkesin öncelik kuralını hatırlaması gerekir ve $n \bmod (100 // 10)$ diye yanlış okuyan $472 \bmod 10 = 2$ bulur. Niyetinizi paranteze yazın. Uç durum: 100 için basamaklar 1, 0, 0; toplam 1; ters 1 (yani 001) — ters sayının başındaki sıfırlar düşer, bu beklenen bir davranıştır.

2.8 Algoritmayı sınamak

İzleme tablosu bir girdi için algoritmanın ne yaptığını gösterir; ama bir algoritmanın *doğru* olduğuna tek girdiyle karar verilmez. Sınama, algoritmayı seçilmiş bir dizi girdiyle çalıştırıp her birinde beklenen çıktıyı aldığımızı doğrulamaktır.

Tanım 2.8: Test durumu

Test durumu (*İng. test case*), bir girdi kümesi ile o girdi için *önceden* hesaplanmış beklenen çıktının çiftidir. Birden çok test durumu bir *test tablosunda* toplanır: girdi, beklenen çıktı, gözlenen çıktı, sonuç (✓/✗). Beklenen çıktı, algoritma çalıştırılmadan — elle, problemin tanımından — bulunur; aksi hâlde algoritma kendi kendini onaylar.

Test tablosu kurma

1. *Tipik* bir girdi seçin: problemin günlük hâli (7384 saniye gibi).
2. *Sınır* girdileri ekleyin: sıfır, en küçük anlamlı değer, bir birimin tam katı (0, 59, 60, 3600), en büyük beklenen değer (86 399).
3. Her girdi için beklenen çıktıyı algoritmaya bakmadan hesaplayın.
4. Algoritmayı her girdi için izleyin, gözlenen yazın, karşılaştırın. Tek bir ✗ bile algoritmanın hatalı olduğunu gösterir; hepsinin ✓ olması ise yalnız *bu girdiler için* doğru olduğunu.

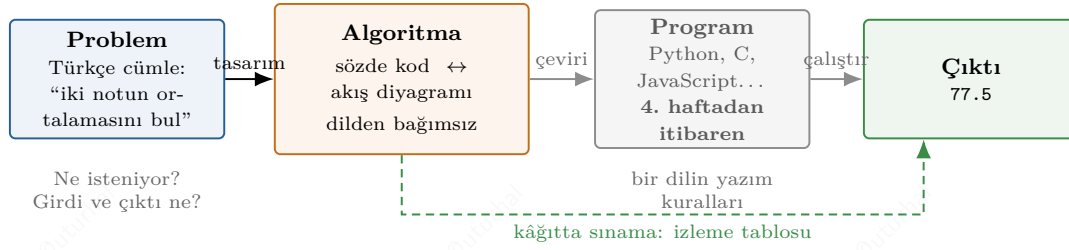
Tablo 2.3: Örnek 2.3'teki algoritma için test tablosu. Beklenen değerler problemin tanımından, gözlenenler izleme tablosundan gelir.

Girdi	Neyi sımar	Beklenen	Gözlenen	Sonuç
7384	tipik değer	2 3 4	2 3 4	✓
0	sıfır	0 0 0	0 0 0	✓
59	dakikadan az	0 0 59	0 0 59	✓
60	tam bir dakika	0 1 0	0 1 0	✓
3600	tam bir saat	1 0 0	1 0 0	✓
86 399	bir günün son saniyesi	23 59 59	23 59 59	✓

Sınır girdileri en çok hata bulan girdilerdir. 60 saniye için “1 dakika 0 saniye” yerine “0 dakika 60 saniye” veren bir algoritma, 7384 ile sırandığında hatasız görünürdü. Aynı ilke her problemde geçerlidir: para üstünde fiyatın ödenene eşit olduğu durum (0 TL üstü), ortalamada iki sayının eşit olduğu durum, basamak ayırmada 100 gibi sıfır içeren sayı. Bir algoritmayı teslim etmeden önce en az bir tipik, iki sınır girdisiyle sınamayı alışkanlık edinin.

Sekil 2.9, bu haftanın nereye bağlandığını gösterir. Bir algoritmayı kâğıtta sınamak ile programı bilgisayarda test etmek aynı işin iki hâlidir; farkı, izleme tablosunu sizin yerinize makinenin doldurmasıdır. Kâğıtta yakalanan hata ucuzdur: bir satırı çizip yeniden yazarsınız.

nız. Programda yakalanan hata ise önce *bulunmak* zorundadır — ve 4. haftada göreceğiniz gibi, bilgisayar hatanın *nerede* olduğunu söyler, *ne* olması gerektiğini değil.



Sözde kodun bu haftaki sözcükleri ve 4. haftada karşılaşacağınız Python karşılıkları

Sözde kod	OKU x	YAZ x	$x \leftarrow 5$	$a // b$	$a \bmod b$	$a ^ b$
Anlam	klavyeden al	ekrana yaz	atama	tam bölme	kalan	üs
Python (4. hafta)	<code>x = input()</code>	<code>print(x)</code>	<code>x = 5</code>	<code>a // b</code>	<code>a % b</code>	<code>a ** b</code>

Algoritma dilden bağımsızdır; program bir dile bağlıdır. Bu hafta ve gelecek hafta ürettiğiniz her sözde kod ve akış diyagramı, 4. haftadan itibaren neredeyse satır satır Python'a çevrilecek: **OKU** bir okuma komutu, **YAZ** bir yazma komutu, ok işareti eşittir olacak. Çeviri kolay, tasarım zordur — bu yüzden dönemin ilk üç haftası tasarıma ayrılmıştır. İyi bir algoritma yanlış dile çevrilemez; kötü bir algoritma hiçbir dilde doğru çalışmaz.

Şekil 2.9: Algoritmadan programa. Bu hafta ürettiğiniz sözde kod ve akış diyagramları dilden bağımsızdır; 4. haftada neredeyse satır satır Python'a çevrilecektir. Alt banttaki Python karşılıkları yalnız ön izlemedir.

LAB Laboratuvar Çalışması 2: Kâğıtta algoritma, ekranda akış diyagramı

Bu haftanın laboratuvarında Python yoktur; araçlarınız kâğıt, kalem, silgi ve bir akış diyagramı çizim programıdır. Görev 1, 3 ve 5 kâğıtta; Görev 2 bilgisayarda; Görev 4 eşli yapılır. Kâğıtlarınızı saklayın: Kısa Sınav 1 bu haftayı da kapsar.

Görev 1. Kâğıtta dört algoritma. Her biri için sözde kod ([Kural 2.2](#)) ve en az iki test durumu (girdi–beklenen çıktı) yazın; beklenen çıktıyı algoritmayı yazmadan *önce* hesaplayın. (a) Üç sınav notunun ortalaması; 70, 80, 96 → 82. (b) Metre cinsinden uzunluğu kilometre ve metre olarak yazma (`//` ve `mod`); 12 345 → 12 km 345 m. (c) Bir restoran hesabını eşit paylaşma: hesap tutarı ve kişi sayısı girdi, kişi başı tutar çıktı; 850 TL, 4 kişi → 212,5 TL. (d) 2.1'deki dönem sonu notu; ağırlıkları sabit olarak tanımlayın; vize 65, final 80 → 74.

Görev 2. Akış diyagramı programda. Flowgorithm'i açın (yoksa tarayıcıda <https://app.diagrams.net>). Flowgorithm'de: Input bloğu **OKU**, Assign bloğu **YAZ**'dır; Declare ile değişkenleri önce tanıtmamız gerekir (bu, Flowgorithm'in kendi kuralıdır, sözde kodumuzda yoktur). Önce [Örnek 2.1](#)'in, sonra [Örnek 2.3](#)'ün diyagramını kurun; Run ile çalıştırıp 70, 85 için 77.5 ve 7384 için 2 3 4 çıktığını doğrulayın. diagrams.net kullanıyorsanız sol-daki Flowchart paletinden sembolleri sürükleyin, okları sembol kenarlarındaki mavi noktalardan çekin; diyagramı PNG olarak dışa aktarın.

Görev 3. İzleme tablosu. Aşağıdaki algoritmayı izleyin ve ekrana ne yazıldığını bulun; sonra sonucu 12 ve 5 yerine 23 ve 7 için yeniden bulun. $a \leftarrow 12$; $b \leftarrow 5$; $c \leftarrow a // b$; $d \leftarrow a \bmod b$; $a \leftarrow c * 10 + d$; **YAZ** a , b , c , d . Beklenen: ilk durumda 22 5 2 2.

Görev 4. Bilgisayar gibi çalıştır (eşli). Görev 1'deki algoritmalarınızdan birini arkadaşınıza verin; arkadaşınız onu hiçbir şey sormadan, yalnız yazılımı yaparak, sizin seçtiğiniz bir girdiyle izlesin. Sonuç beklenenden farklıysa hatanın hangi satırda olduğunu *birlikte* bulun. Sonra roller değişir. Kesin olmadığı için uygulanamayan her adımı işaretleyip yeniden yazın.

Görev 5. Hatalı algoritmayı düzelt. Aşağıdaki algoritma iki sayının ortalamasını yazmak için yazılmış ama iki hatası var. Önce 70 ve 85 ile izleyerek hataları bulun, sonra düzeltin ve 77,5 çıktığını gösterin. **BAŞLA**; **OKU** *a*, *b*; **YAZ** *ort*; *ort* $\leftarrow a + b / 2$; **BİTİR**.

Görev 6. Tablodan algoritmaya (ek). Bir arkadaşınızın algoritması kayboldu, yalnız izleme tablosu kaldı: girdi 6; *x* = 6; sonra *y* = 36; sonra *z* = 42; ekranda 42. Bu tabloyu üreten sözde kodu yazın ve girdi 10 için ne yazacağını söyleyin (beklenen: 110).

Teslim. Görev 2'nin çalışan Flowgorithm dosyasını (.fprg) ya da diagrams.net görüntüsünü ders bitmeden gösterin; kâğıt çalışmalarını saklayın — 4. haftada Görev 1'deki dört algoritmayı Python'a çevireceksiniz.

Özet

Kavram	Özü	Not
Problem çözme	anla $\rightarrow$ girdi/çıktı $\rightarrow$ tasarla $\rightarrow$ elle sına $\rightarrow$ programla	önce örnek girdi-çıktı yaz; sınama başarısızsa geri dön
Algoritma	sonlu, kesin, sıralı adımlar	kesinlik, sonluluk, girdi, çıktı, etkinlik
Sıralı yapı	adımlar yukarıdan aşağı, birer kez	karar ve tekrar 3. haftada
Değişken, atama	adı olan kutu; <i>ad</i> $\leftarrow$ ifade: sağa hesapla, sola yaz	eski değer silinir; <i>b</i> $\leftarrow$ <i>a</i> kopyalar, bağlamaz
İşlemler	+ - * / // mod ^	$17//5 = 3$, $17 \bmod 5 = 2$; $(a//b) \cdot b + a \bmod b = a$
Öncelik	() > ^ > * / // mod > + -	aynı seviye soldan sağa; kararsızsan parantez
Sabit	bir kez atanır, büyük harf	<i>KDV_ORANI</i> $\leftarrow$ 0.20
Sözde kod	BAŞLA/BİTİR, OKU, YAZ, $\leftarrow$, # yorum	ondalık ayırıcı nokta; adlar küçük harf, Türkçe karactersiz
Akış diyagramı	uç, paralelkenar (G/Ç), dikdörtgen (işlem), ok, bağlayıcı	tek BAŞLA, tek BİTİR; her kutuya bir giriş bir çıkış
İzleme tablosu	satır = adım, sütun = değişken, son sütun Ekran	kâğıttakini çalıştır, aklındakini değil
//-mod zinciri	büyük birimden küçüğe: adet //, artan mod	her seviye <i>kalan</i> üzerinden sürer
Takas	<i>gecici</i> $\leftarrow$ <i>a</i> ; <i>a</i> $\leftarrow$ <i>b</i> ; <i>b</i> $\leftarrow$ <i>gecici</i>	iki atamayla olmaz: değer kaybolur
Sınama	test tablosu: girdi, beklenen, gözlenen, sonuç	beklenen algoritmaya bakmadan bulunur; sınır girdileri şart

Alıştırımlar

Alıştırma 2.1. ●●● Aşağıdaki adım listelerinin her birinde algoritmanın beş özelliğinden hangisi eksiktir? (a) “Hamura un ekle, yeterince karıştır, fırına ver.” (b) “Bir sayı al; 2 ile çarp; sonucu al; 2 ile çarp; sonucu al; ...” (c) “Bir sayı al, karesini hesapla, kâğıdı çöpe at.” (d) “İki sayının farkını hesapla ve yaz.” (e) “Ay’a git, bir taş al, ağırlığını yaz.”

Cevap: (a) kesinlik (b) sonluluk (c) çıktı (d) girdi (e) etkinlik

Alıştırma 2.2. ●●● Aşağıdaki ifadeleri işlem önceliğine göre *elle* değerlendirin.

(a) $7 + 3 * 2$

(e) $2 ^ 3 * 2$

(b) $(7 + 3) * 2$

(f) $100 - 30 - 20$

(c) $20 // 6$

(g) $9 / 2$

(d) $20 \bmod 6$

(h) $17 \bmod 5 + 17 // 5$

Cevap: (a) 13 (b) 20 (c) 3 (d) 2 (e) 16 (f) 50 (g) 4.5 (h) 5

Alıştırma 2.3. ●●● Aşağıdaki algoritmayı izleme tablosuyla çalıştırın; ekrana ne yazılır?
 $x \leftarrow 4$; $y \leftarrow x * 3$; $x \leftarrow x + y$; $y \leftarrow y - 1$; **YAZ** x , y .

Cevap: 16 11: adımlar $x = 4$; $y = 12$; $x = 16$; $y = 11$

Alıştırma 2.4. ●●● Yarıçapı verilen dairenin alanını (πr^2) ve çevresini ($2\pi r$) yazan sözde kodu yazın; π için 3,14 değerli bir sabit tanımlayın. $r = 5$ ile izleyin.

*Cevap: alan 78,5, çevre 31,4; $PI \leftarrow 3.14$; $alan \leftarrow PI * r ^ 2$; $cevre \leftarrow 2 * PI * r$*

Alıştırma 2.5. ●●● Bir aracın gittiği yolu (km) ve tükettiği yakıtı (litre) okuyup 100 km başına yakıt tüketimini ve yakıtın litre fiyatı 42,5 TL iken toplam yakıt giderini yazan sözde kodu yazın ve akış diyagramını çizin. Girdi: 450 km, 31,5 litre.

*Cevap: tüketim $31,5/450 \cdot 100 = 7$ L/100 km; gider $31,5 \cdot 42,5 = 1338,75$ TL; $LITRE_FIYATI \leftarrow 42.5$; $tuketim \leftarrow litre / km * 100$; $gider \leftarrow litre * LITRE_FIYATI$*

Alıştırma 2.6. ●●● Dakika cinsinden bir süreyi gün, saat ve dakika olarak yazan sözde kodu yazın (Örnek 2.3’ü örnek alın; bir günde 1440 dakika vardır). Girdi 10000 ile izleyin.

Cevap: 6 gün 22 saat 40 dakika: $10000//1440 = 6$, kalan 1360; $1360//60 = 22$, kalan 40

Alıştırma 2.7. ●●● Örnek 2.6’yı dört basamaklı sayılara genişletin: dört basamağı, basamak toplamını ve tersini yazın. Girdi 2026.

Cevap: basamaklar 2,0,2,6; toplam 10; ters 6202

Alıştırma 2.8. ●●● Üç değişkenin değerlerini döndüren algoritmayı yazın: a ’nın değeri b ’ye, b ’nin değeri c ’ye, c ’nin değeri a ’ya geçsin. Tek bir geçici değişken yeter. $a = 1$, $b = 2$, $c = 3$ ile izleyin.

Cevap: $a = 3$, $b = 1$, $c = 2$; $gecici \leftarrow c$; $c \leftarrow b$; $b \leftarrow a$; $a \leftarrow gecici$

Alıştırma 2.9. ●●● Aşağıdaki algoritma birim fiyatı ve adedi okuyup %20 KDV dahil

toplamı yazmalıdır. Girdi 50 ve 3 için 180 beklenirken 150,2 veriyor. İzleme tablosuyla hatalı satırı bulun ve düzeltin. OKU fiyat, adet; KDV $\leftarrow 0.20$; ara $\leftarrow$ fiyat * adet; toplam $\leftarrow$ ara + KDV; **YAZ** toplam.

Cevap: hatalı satır $toplam \leftarrow ara + KDV$ (orana tutar gibi ekleniyor); doğrusu $toplam \leftarrow ara + ara * KDV$ ya da $ara * (1 + KDV)$: $150 + 30 = 180$

Alıştırma 2.10. Örnek 2.4'teki para üstü algoritması için en az beş test durumu içeren bir test tablosu kurun: bir tipik girdi, “üstü sıfır”, “üstü tek bir banknot”, “yalnız 1 TL’ler” ve “her banknottan en az bir tane” durumları. Her durum için beklenen çıktıyı algoritmaya bakmadan yazın, sonra izleme tablosuyla doğrulayın. Ayrıca, fiyat ödenenden büyük girilirse algoritmanın ne yaptığını ve bunun neden bir sorun olduğunu bir cümleyle yazın.

Cevap: örneğin $(113, 200) \rightarrow 1\ 1\ 1\ 1\ 2$; $(200, 200) \rightarrow 0\ 0\ 0\ 0\ 0$; $(150, 200) \rightarrow 1\ 0\ 0\ 0\ 0$; $(197, 200) \rightarrow 0\ 0\ 0\ 0\ 3$; $(1, 100) \rightarrow 1\ 2\ 0\ 1\ 4$. Fiyat > ödenen ise üstü negatif olur ve anlamsız adetler çıkar; algoritma bunu fark edemez — karar yapısı (3. hafta) gerekir

Alıştırma 2.11. Geçici değişken *kullanmadan* iki sayıyı takas eden şu algoritma öneriliyor: $a \leftarrow a + b$; $b \leftarrow a - b$; $a \leftarrow a - b$. (a) $a = 3$, $b = 8$ ile izleyip çalıştığını gösterin. (b) Neden çalıştığını her satırdan sonra a ve b 'nin *eski* değerler cinsinden ne tuttuğunu yazarak açıklayın. (c) Bu yöntemin geçici değişkenli yöntemle göre bir sakıncasını yazın.

Cevap: (a) $a = 11$; $b = 3$; $a = 8 \rightarrow (8, 3)$ (b) 1. satırdan sonra $a = a_0 + b_0$; 2. satır $b = (a_0 + b_0) - b_0 = a_0$; 3. satır $a = (a_0 + b_0) - a_0 = b_0$ (c) yalnız sayılarla çalışır (metin takas edilemez); sabit genişlikli tam sayılarda $a + b$ taşabilir (1. hafta); okunması zor

Alıştırma 2.12. Aynı gün içindeki iki saat, $sa1:dk1$ ve $sa2:dk2$ biçiminde (ikincisi daha geç) okunuyor. Aradaki süreyi saat ve dakika olarak yazan sözde kodu yazın; ipucu: iki saati önce “gece yarısından bu yana geçen dakika”ya çevirin. 09:45 ve 14:20 ile izleyin ve en az üç test durumu daha yazın.

Cevap: 4 saat 35 dakika: $9 \cdot 60 + 45 = 585$; $14 \cdot 60 + 20 = 860$; fark 275; $275 // 60 = 4$, $275 \bmod 60 = 35$. $t1 \leftarrow sa1 * 60 + dk1$; $t2 \leftarrow sa2 * 60 + dk2$; fark $\leftarrow t2 - t1$; saat $\leftarrow fark // 60$; dakika $\leftarrow fark \bmod 60$

Hafta 3: Algoritma ve Akış Diyagramları II: Karar ve Tekrar Yapıları

Öğrenme çıktıları

Bu haftanın sonunda:

1. bir problemi koşullara göre dallandırıp EĞER–DEĞİLSE yapısını sözde kod ve akış diyagramı olarak yazabileceksiniz;
2. çoklu karar (EĞER–DEĞİLSE EĞER) zincirini doğru *sırayla* kurabilecek, bir karar tablosunu algoritmaya çevirebileceksiniz;
3. VE, VEYA ve DEĞİL işleçlerini doğruluk tablolarıyla açıklayıp bileşik koşullar yazabileceksiniz;
4. koşullu ve sayaçlı döngüler kurup toplam, çarpım ve sayaç biriktiricileriyle nöbetçi değerle sonlanan algoritmalar yazabileceksiniz;
5. iç içe döngülerin kaç adım yürüdüğünü sayabilecek, en büyük bulma, faktöriyel, asal testi, EBOB ve basamak toplamı gibi klasik algoritmaları izleme tablosuyla çalıştırabileceksiniz;
6. sonsuz döngü ve birim kayması (off-by-one) hatalarını izleme tablosuyla bulabilecek, iki algoritmanın adım sayılarını karşılaştırarak hangisinin daha verimli olduğunu söyleyebileceksiniz.

Anahtar kavramlar

karar yapısı (İng. selection) **koşul** (İng. condition) **karar tablosu** (İng. decision table)
mantıksal işleç (İng. logical operator) **doğruluk tablosu** (İng. truth table) **döngü** (İng. loop)
tur (İng. iteration) **biriktirici** (İng. accumulator) **sayaç** (İng. counter) **nöbetçi değer** (İng. sentinel)
sonsuz döngü (İng. infinite loop) **birim kayması** (İng. off-by-one)
dizi (İng. array) **indeks** (İng. index) **adım sayısı** (İng. step count) **ikili arama** (İng. binary search)

Geçen hafta yazdığımız her algoritma tek yolluydu: adımlar yukarıdan aşağı, birer kez, hep aynı sırayla yürütüldü. Bu, gerçek problemlerin çoğu için yetmez. Para üstü algoritmamız, ödenen tutar fiyattan küçük olduğunda anlamsız sonuçlar üretiyordu; “üç notun ortalamasını al” diyebildik ama “kaç not olduğu önceden bilinmiyorsa” diyemedik. Eksik olan iki şeydi: bir *koşula göre yol seçmek* ve bir adım grubunu *tekrar etmek*. Bu hafta bu ikisini ekliyoruz; böylece yapısal programlamanın üç temel yapısı — sıralı, karar, tekrar — tamamlanmış olacak.

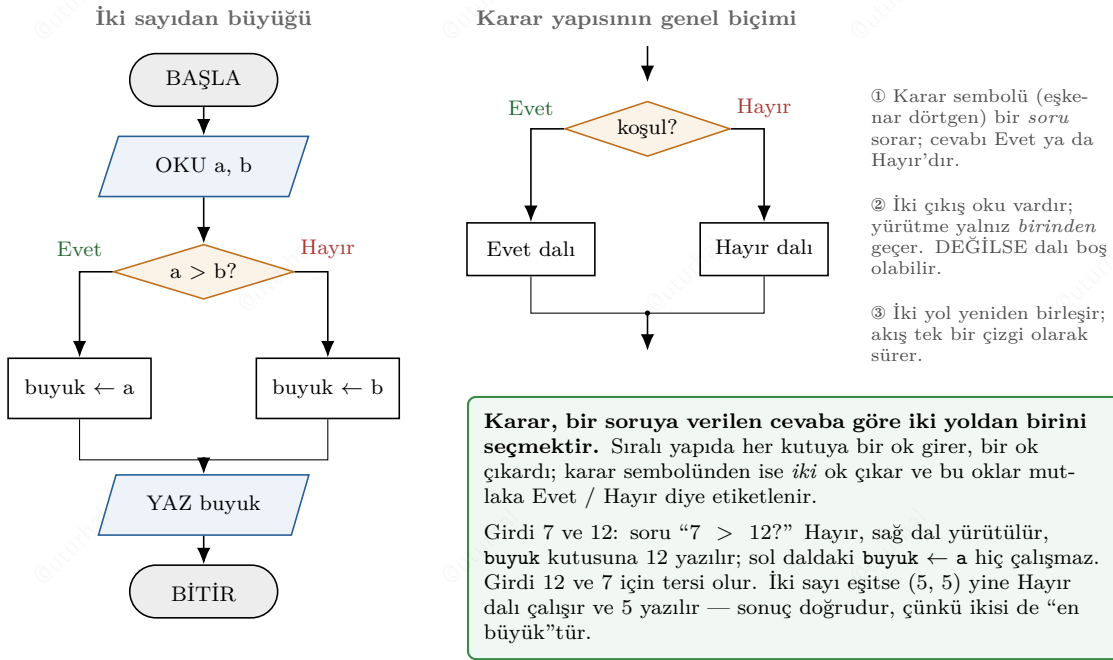
Bu üç yapıyla yazılamayan hiçbir hesap yoktur: bir yapay zeka modelini eğitmek de, bir arama motorunun sayfaları sıralaması da sonunda bu üç yapının milyonlarca kez iç içe geçmiş hâlidir. Bu hafta ayrıca yeni bir soru soruyoruz: aynı işi yapan iki algoritmadan *hangisi daha az adımda* bitirir? Bir milyon kayıtlı bir listede aradığınızı bir milyon adımda mı yoksa yirmi adımda mı bulacağınız, doğru algoritmayı seçmenize bağlıdır. Dersin son 25 dakikasında Kısa Sınav 1 (1–3. haftalar) yapılacaktır.

3.1 Karar yapısı: EĞER–DEĞİLSE

Bir algoritmanın “duruma göre” davranabilmesi için önce bir soru sorup cevaba göre farklı adımlar yürütmesi gerekir.

Tanım 3.1: Karar yapısı

Karar yapısı (*İng. selection*), bir **koşulun** (*İng. condition*) doğru olup olmamasına göre iki adım grubundan yalnız birinin yürütüldüğü yapıdır. Koşul, cevabı yalnız *doğru* ya da *yanlış* olabilen bir sorudur; “belki” yoktur. Sözde kodda **EĞER ... İSE ... DEĞİLSE ... EĞER SONU** biçiminde, akış diyagramında ise eşkenar dörtgen (karar sembolü) ile yazılır.



Şekil 3.1: Karar yapısı. Solda iki sayıdan büyüğünü bulan tam bir akış diyagramı, sağ üstte yapının genel biçimi. Karar sembolünden *iki* ok çıkar, bu oklar Evet ve Hayır diye etiketlenir ve iki yol aşağıda yeniden birleşir.

Koşul kurmak için altı karşılaştırma işleci kullanacağız: = (eşit), ≠ (eşit değil), <, ≤, >, ≥. Bunlar aritmetik işleçlerden sonra değerlendirilir: $a + 1 > b$ ifadesinde önce $a + 1$ hesaplanır.

Sözde kodda = işareti *yalnız* karşılaştırma anlamındadır; atama her zaman ← okuyla yapılır. Bu ayrım sizi 4. haftada bekleyen bir tuzığa hazırlar: Python’da atama =, karşılaştırma == ile yazılır — “ok atar, eşittir sorar”. Geçen haftanın sözde kod sözleşmesi (BAŞLA/BİTİR, OKU, YAZ, ←, # yorum, // ve mod) aynen geçerlidir; üzerine bu haftanın iki yapısını ekliyoruz.

- Kural 3.1 (Sözde kod sözleşmesi: karar ve tekrar).** (i) Karar: **EĞER** *koşul* **İSE** satırından sonra gelen ve iki boşluk içeriden yazılan satırlar koşul doğruysa yürütülür. İsteğe bağlı **DEĞİLSE** satırından sonrakiler koşul yanlışsa yürütülür. Yapı **EĞER SONU** ile kapatılır.
- (ii) Çoklu karar: **DEĞİLSE EĞER** *koşul* **İSE** satırları istendiği kadar art arda yazılabilir; koşullar *yukarıdan aşağı* sırayla sorulur ve ilk doğru olanın dalı yürütülüp yapı terk edilir.
- (iii) Tekrar: *koşul* **İKEN** satırından sonra gelen içerideki satırlar, koşul her tur ba-

şında yeniden sorularak, doğru olduğu sürece yinelenir; yapı **DÖNGÜ SONU** ile kapatılır. Akış diyagramında karar ve döngü koşulu aynı sembolle — eşkenar dörtgenle — gösterilir; ikisini ayıran, döngüde bir okun *geriye* dönmesidir.

- (iv) Koşullarda karşılaştırma işleçleri ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$) ve mantıksal işleçler (**VE**, **VEYA**, **DEĞİL**) kullanılır.
- (v) Girinti zorunludur ve yapının nerede bittiğini gösterir; iç içe yapılarda her seviye iki boşluk daha içeriden yazılır.

Sözleşmenin en kısa uygulaması bir sayının çift mi tek mi olduğunu yazan üç satırdır: **EĞER** sayı mod 2 = 0 **İSE YAZ** "çift", **DEĞİLSE YAZ** "tek", **EĞER SONU**. Girdi 12 için koşul doğrudur ($12 \bmod 2 = 0$); girdi 7 için yanlıştır ($7 \bmod 2 = 1$) ve **DEĞİLSE** dalı çalışır. İki **YAZ** satırından her zaman *tam olarak biri* yürütülür. **DEĞİLSE** dalı isteğe bağlıdır: "tutar 1000 TL'yi aşarsa kargo bedava" gibi tek dallı kurallarda yazılmaz.

Örnek 3.1 (Tutara göre indirim). Bir mağazada sepet tutarı 500 TL ve üzerindeyse %10 indirim uygulanıyor. Sepet tutarını okuyup ödenecek tutarı yazan algoritmayı tasarlayın ve 640 TL girdisiyle izleyin.

Çözüm

Analiz. Girdi: **tutar** (ondalıklı olabilir). Çıktı: **odenecek**. Eşik (500) ve oran (0,10) girdi değil sabittir. İndirim her iki durumda da hesaplanmalıdır: koşul yanlışsa indirim sıfırdır. Böylece **odenecek** $\leftarrow$ **tutar** - **indirim** satırı tek bir kez, dalların dışında yazılabilir.

Algoritma.

Sözde kod: Tutara göre indirim

```
BAŞLA
  ESIK  $\leftarrow$  500
  ORAN  $\leftarrow$  0.10
  OKU tutar
  EĞER tutar  $\geq$  ESIK İSE
    indirim  $\leftarrow$  tutar * ORAN
  DEĞİLSE
    indirim  $\leftarrow$  0
  EĞER SONU
  odenecek  $\leftarrow$  tutar - indirim
  YAZ indirim, odenecek
BİTİR
```

Akış diyagramı. Şekil 3.1'in sağ üstündeki genel biçim birebir uygulanır: bir karar sembolü, iki işlem kutusu, bir birleşme noktası, sonra ortak **odenecek** kutusu ve çıktı.

İzleme tablosu. Girdi 640 (sabitler ilk satırda gösterilmemiştir):

Adım	Yürütülen satır	tutar	indirim	Ekran
1	OKU tutar	640	—	
2	EĞER tutar $\geq$ 500 İSE $\rightarrow$ Evet	640	—	
3	indirim $\leftarrow$ tutar * 0.10	640	64	
4	odenecek $\leftarrow$ tutar - indirim	640	64	
5	YAZ indirim, odenecek	640	64	64 576

odenecek sütunu yer kazanmak için gösterilmemiştir; adım 4'te $640 - 64 = 576$ olur.

Kontrol. Elle: $640 \cdot 0,10 = 64$ ve $640 - 64 = 576$ ✓; ödenecek tutar sepetten küçük ama sıfırdan büyük, $0 < 576 < 640$ ✓. Sınırlar en öğreticisidir: tam 500 girilirse koşul *doğrudur* ($\geq$), indirim 50, ödenecek 450; 499 girilirse indirim 0, ödenecek 499. Aradaki 49 TL'lik sıçrama kuralın kendisinden gelir, alitmadan değil.

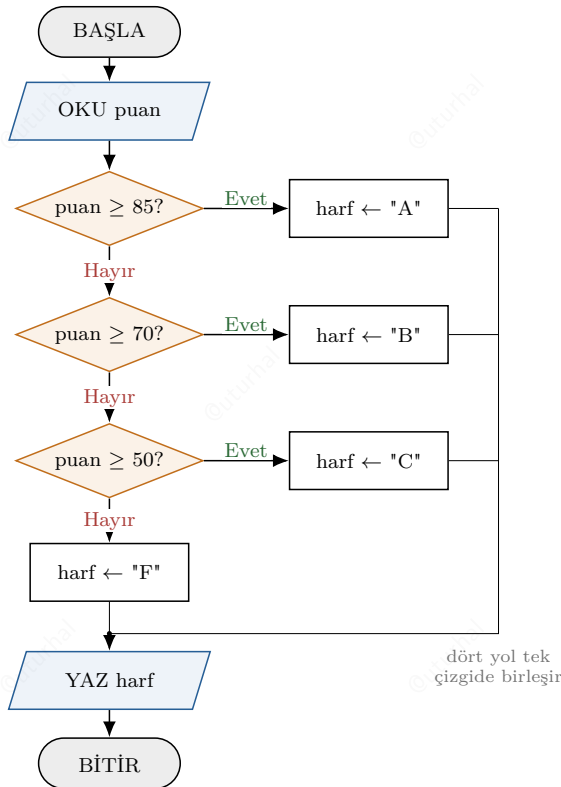
✗ Sık yapılan hata

EĞER $tutar \geq 500$ İSE $indirim \leftarrow tutar \cdot 0.10$ (DEĞİLSE yok) → DEĞİLSE $indirim \leftarrow 0$ dalmı ekleyin

Koşul yanlış olduğunda *indirim* kutusu hiç oluşmaz; bir sonraki satırda $tutar - indirim$ hesaplanmaya çalışılır ve “değer almadış değişken” hatası ortaya çıkar. Kural: bir değişken *her yoldan* değer almalıdır. Pratik çözüm, kutuyu *karar yapısından önce* sıfırlamaktır: $indirim \leftarrow 0$ satırını en başa koyarsanız **DEĞİLSE** dalına gerek kalmaz.

3.2 Çoklu karar ve karar tablosu

Kurallar çoğu zaman iki dallı değildir: bir puan A, B, C ya da F, bir bilet ücretsiz, indirimli ya da tam olabilir. Bu tür kuralları art arda gelen **DEĞİLSE EĞER** satırlarıyla yazarız.



Karar tablosu

Sıra	Koşul	harf
1	$puan \geq 85$	A
2	$70 \leq puan < 85$	B
3	$50 \leq puan < 70$	C
4	$puan < 50$	F

Diyagramda 2. ve 3. koşullar kısa yazılmıştır (“ $puan \geq 70?$ ”), çünkü o soruya ancak bir öncekinin cevabı Hayır ise ulaşılır: $puan < 85$ zaten bilinmektedir.

Çoklu karar bir merdivendir: koşullar sırayla sorulur, ilk Evet’te durulur. Girdi 72: “ $\geq 85?$ ” Hayır; “ $\geq 70?$ ” Evet → B; üçüncü soru hiç sorulmaz. Girdi 30: üç Hayır → F.

Sıra önemlidir. Sorular “ $\geq 50?$ ” ile başlasaydı 72 de, 95 de ilk soruda Evet alır ve C olurdu. En dar (en güçlü) koşul en üste yazılır; DEĞİLSE dalı hiçbir koşula uymayanları yakalar.

Sınır değerleri deneyin: $85 \rightarrow A$, $84 \rightarrow B$, $50 \rightarrow C$, $49 \rightarrow F$.

Şekil 3.2: Çoklu karar (EĞER–DEĞİLSE EĞER). Koşullar yukarıdan aşağı sorulur, ilk doğru olanın dalı yürütülür ve kalan sorular hiç sorulmaz. Sağdaki karar tablosu aynı kuralın satır satır dökümüdür.

Şekil 3.2’deki harf notu zincirini sözde koda çevirelim:

Sözde kod: Harf notu: çoklu karar

```

BAŞLA
OKU puan
EĞER puan  $\geq$  85 İSE
    harf  $\leftarrow$  "A"
DEĞİLSE EĞER puan  $\geq$  70 İSE
    harf  $\leftarrow$  "B"
DEĞİLSE EĞER puan  $\geq$  50 İSE
    harf  $\leftarrow$  "C"
DEĞİLSE
    harf  $\leftarrow$  "F"
EĞER SONU
YAZ harf
BİTİR

```

Girdi 72 için birinci soru (" ≥ 85 ?") Hayır, ikinci soru (" ≥ 70 ?") Evet: harf B olur ve üçüncü soru hiç sorulmaz. Girdi 30 için üç soru da Hayır alır ve son **DEĞİLSE** dalı çalışır: F.

Kural 3.1 (ii) uyarınca zincirdeki koşulların *sırası* anlamı belirler ve kural şudur: **en dar (en zor sağlanan) koşul en üste yazılır**. İkinci soruda " $70 \leq \text{puan} < 85$ " yazmaya gerek yoktur; oraya ancak birinci sorunun cevabı Hayır ise, yani puanın 85'ten küçük olduğu *zaten bilinirken* gelir.

Tanım 3.2: Karar tablosu

Karar tablosu (*İng. decision table*), bir kuralın bütün durumlarını satır satır listeleyen tablodur: her satırda koşullar ve o koşullar sağlandığında yapılacak iş bulunur. Karar tablosu, algoritma yazmadan *önce* hazırlanır; durumların hepsinin kapsandığını ve hiçbir durumun iki kez kapsamadığını denetlemenin en kolay yoludur.

Tablo 3.1: Bir müze biletinin yaşa göre ücreti: karar tablosu. Aralıklar bitişiktir ve üst üste binmez; "ele alınmayan yaş" yoktur.

Sıra	Koşul (yaş)	Ücret (TL)	Açıklama
1	< 7	0	okul öncesi, ücretsiz
2	7–17	40	öğrenci indirim
3	18–64	80	tam bilet
4	≥ 65	20	emekli/65 yaş indirim

Tablo 3.1'in algoritmaya çevrilmesi mekanik bir iştir: her satır bir **DEĞİLSE EĞER** olur ve koşullar, önceki satırların zaten elendiği varsayımıyla kısılır.

Sözde kod: Bilet ücreti

```

BAŞLA
OKU yas
EĞER yas  $< 7$  İSE
    ucret  $\leftarrow$  0
DEĞİLSE EĞER yas  $\leq 17$  İSE
    ucret  $\leftarrow$  40
DEĞİLSE EĞER yas  $\leq 64$  İSE
    ucret  $\leftarrow$  80
DEĞİLSE

```

```

    ucret ← 20
    EĞER SONU
    YAZ ucret
    BİTİR

```

Sınır değerleriyle sınavalım: $6 \rightarrow 0$, $7 \rightarrow 40$, $17 \rightarrow 40$, $18 \rightarrow 80$, $64 \rightarrow 80$, $65 \rightarrow 20$. Altı denemenin tamamı tabloyla uyuyor; bir kuralın doğruluğu en çok sınırlarda sınanır.

3.2.1 İç içe kararlar

Bir kararın dalı içinde başka bir karar bulunabilir. Üç sayının en büyüğü tipik örnektir:

Sözde kod: Üç sayının en büyüğü: iç içe karar

```

EĞER a > b İSE
    EĞER a > c İSE  enb ← a  DEĞİLSE  enb ← c  EĞER SONU
DEĞİLSE
    EĞER b > c İSE  enb ← b  DEĞİLSE  enb ← c  EĞER SONU
EĞER SONU

```

(Yer kazanmak için iç kararlar tek satırda gösterilmiştir. Bu kitapta zaman zaman görülen bu sıkıştırma yalnız yerden tasarruf içindir; siz her satıra tek iş düşecek biçimde, girintili yazın.) Girdi 7, 12, 9: dış koşul “ $7 > 12$?” Hayır, **DEĞİLSE** dalına girilir; iç koşul “ $12 > 9$?” Evet, **enb** = 12 olur. Dört yoldan yalnız biri yürütülür ve her girdide tam iki karşılaştırma yapılır. İç içe karar mı, **VE** ile birleştirilmiş tek karar mı? İkinci soru *yalnız* birincinin cevabına göre anlam kazanıyorsa iç içe yazın; iki soru bağımsızsa **VE** ile birleştirin.

Sık yapılan hata

EĞER puan ≥ 50 **İSE** harf $\leftarrow$ "C" **DEĞİLSE** **EĞER** puan ≥ 85 **İSE** harf $\leftarrow$ "A" $\rightarrow$ önce puan ≥ 85 , sonra ≥ 70 , sonra ≥ 50

Zincirin sırası ters yazıldığında 95 puan alan öğrenci C alır: birinci koşul zaten doğrudur ve akış oradan çıkar. Bu hata çalıştırıldığında “hata mesajı” vermez; algoritma sessizce yanlış sonuç üretir. Yakalamanın tek yolu, her dal için *birer* test girdisi denemektir: 95, 72, 60, 30.

3.3 Mantıksal koşullar: VE, VEYA, DEĞİL

Bazı kurallar tek karşılaştırmayla anlatılamaz: “puan 50 ile 100 arasında olmalı”, “cumartesi ya da pazar olmalı”, “üye *olmayan* müşteri”. Bu kurallar için üç **mantıksal işleç** (*Ing. logical operator*) kullanırız.

Tanım 3.3: Mantıksal işleçler

VE (*Ing. and*) ile birleştirilen bileşik koşul, *her iki* parça da doğruysa doğrudur. **VEYA** (*Ing. or*) ile birleştirilen koşul, parçalardan *en az biri* doğruysa doğrudur. **DEĞİL** (*Ing. not*) bir koşulun değerini tersine çevirir. Bir işlecin bütün girdi birleşimindeki sonucunu gösteren tabloya **doğruluk tablosu** (*Ing. truth table*) denir.

VE (ikisi de)			VEYA (en az biri)			DEĞİL (tersi)	
A	B	A VE B	A	B	A VEYA B	A	DEĞİL A
D	D	D	D	D	D	D	Y
D	Y	Y	D	Y	D	Y	D
Y	D	Y	Y	D	D		
Y	Y	Y	Y	Y	Y		

D = doğru, Y = yanlış. Bir koşul yalnız bu iki değerden birini alır; “belki” yoktur.

Bileşik koşulu içten dışa değerlendirin

(yıl mod 4 = 0 VE yıl mod 100 ≠ 0) VEYA yıl mod 400 = 0

yıl	mod 4 = 0	mod 100 ≠ 0	(VE)	mod 400 = 0	sonuç (VEYA)
2024	D	D	D	Y	D — artık
1900	D	Y	Y	Y	Y — artık değil
2000	D	Y	Y	D	D — artık
2026	Y	D	Y	Y	Y — artık değil

Mantıksal işlemler küçük koşulları tek bir büyük koşula bağlar. Önce karşılaştırmalar (① sütunlar), sonra **DEĞİL**, sonra **VE**, en son **VEYA** değerlendirilir; sırayı değiştirmek için parantez kullanılır.

Tablo, kuralın neden üç parçalı olduğunu gösterir: 1900 dörde bölünür ama yüze de bölündüğü için artık *değildir*; 2000 ise dört yüze bölündüğü için istisnanın istisnasıdır. **VE** bir koşulu *daraltır*, **VEYA** *genişletir*.

Şekil 3.3: VE, VEYA ve DEĞİL işlemlerinin doğruluk tabloları ve bileşik bir koşulun (artık yıl kuralı) dört yıl için adım adım değerlendirilmesi. VE bir koşulu daraltır, VEYA genişletir.

Bileşik koşullar da bir öncelik kuralına uyar: önce karşılaştırmalar, sonra **DEĞİL**, sonra **VE**, en son **VEYA** değerlendirilir. Kararsız kaldığınızda parantez kullanın — geçen haftanın aritmetik önceliğinde olduğu gibi, fazla parantezin bedeli yoktur.

İpucu

Bir aralık koşulu *iki* karşılaştırma gerektirir ve ikisinde de değişkenin adı yazılır: $\text{puan} \geq 50$ VE $\text{puan} \leq 100$. Matematikte alışık olduğunuz $50 \leq \text{puan} \leq 100$ yazımı sözde kodda ve programlama dillerinin çoğunda geçersizdir.

Sık yapılan hata

$\text{gun} = \text{"Cumartesi"} \text{ VE } \text{gun} = \text{"Pazar"} \rightarrow \text{gun} = \text{"Cumartesi"} \text{ VEYA } \text{gun} = \text{"Pazar"}$
Günlük Türkçede “cumartesi ve pazar tatildir” deriz; mantıkta ise **VE** “ikisi birden” demektir ve bir değişken aynı anda iki değer tutamayacağı için bu koşul *hiçbir zaman* doğru olmaz. Kural: aynı değişkenin farklı değerleri **VEYA** ile, farklı değişkenlerin koşulları çoğu zaman **VE** ile bağlanır.

Örnek 3.2 (Artık yıl). Bir yılın artık olup olmadığını yazan algoritmayı tasarlayın. Kural: dörde bölünen yıllar artıktır; ancak yüze bölünenler artık değildir; ancak dört yüze bölünenler yine artıktır. 2024, 1900, 2000 ve 2026 ile sınamın.

Çözüm

Analiz. Girdi: yıl (tam sayı). Çıktı: iki metinden biri. Üç parçalı kuralı tek bir bileşik koşulda toplayabiliriz: yıl (dörde bölünüyor VE yüze bölünmüyor) VEYA dört

yüze bölünüyorsa artıktır. “b’ye bölünüyor” ifadesinin sözde koddaki karşılığı $yil \bmod b = 0$ ’dır.

Algoritma.

Sözde kod: Artık yıl

BAŞLA

OKU yıl

EĞER (yıl mod 4 = 0 VE yıl mod 100 $\neq$ 0) VEYA yıl mod 400 = 0 İSE

YAZ yıl, "artık yıldır"

DEĞİLSE

YAZ yıl, "artık yıl değildir"

EĞER SONU

BİTİR

Akış diyagramı. Tek bir karar sembolü, içinde bileşik koşul; iki dalda birer **YAZ** kutusu; altta birleşme. **Şekil 3.1**’in genel biçimiyle aynıdır — koşulun uzun olması yapıyı değiştirmez. Laboratuvarında çizeceksiniz.

İzleme tablosu. Bileşik koşul, parçaları ayrı sütunlara yazılarak izlenir:

yıl	mod 4	mod 100	mod 400	(VE)	VEYA	Ekran
2024	0	24	24	doğru	doğru	artık yıldır
1900	0	0	300	yanlış	yanlış	artık yıl değildir
2000	0	0	0	yanlış	doğru	artık yıldır
2026	2	26	26	yanlış	yanlış	artık yıl değildir

Kontrol. 2024 artıktı ve Şubat 29 çekti; 2000 artıktı, 1900 değildi — kuralın üçüncü parçasının tarihsel kanıtı ✓. Sayılar: $1900 \bmod 400 = 300 \neq 0$, $2000 \bmod 400 = 0$ ✓. Büyüklük mertebesi: dört yüz yıldaki $400/4 = 100$ adaydan 3 yüzyıl dönümü düşülür, 97 artık yıl kalır; ortalama yıl uzunluğu $365 + 97/400 = 365,2425$ gün olur ve gerçek değere (365,2422) çok yakındır ✓.

Bazı sistemler 2000 yılını “yüze bölünüyor, öyleyse artık değil” diye işaretleyip 29 Şubat 2000’i takvimden düşürmüştü: kuralın üç parçasından ikisini yazıp üçüncüsünü unutmak, dört yüz yılda bir — ama tam da o yıl — yanlış sonuç verir. Bir koşulu yazdıktan sonra “hangi girdi bunu şaşırtır?” diye sormak iyi bir alışkanlıktır.

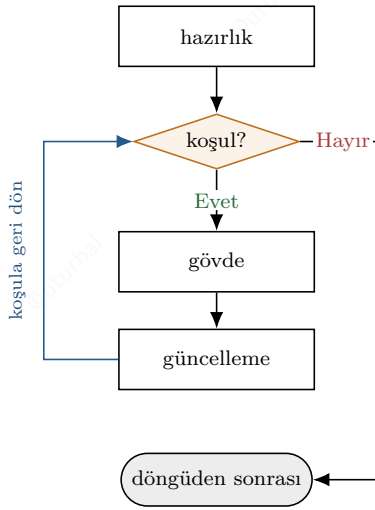
3.4 Tekrar yapısı: koşullu döngü

Üç notun ortalamasını **Örnek 3.1**’deki gibi düz bir yapıyla alabiliriz; peki not sayısı önceden bilinmiyorsa? Ya da 1’den 1000’e kadar toplamak gerekirse — bin satır mı yazacağız? İkinci temel yapı, tekrar, tam da bunun içindir.

Tanım 3.4: Döngü

Döngü (İng. *loop*), bir koşul doğru olduğu sürece aynı adım grubunun yeniden yürütülmesidir. Adım grubunun bir kez yürütülmesine **tur** (İng. *iteration*) denir. Sözde koda **koşul İKEN ... DÖNGÜ SONU** biçiminde yazılır; koşul *her turun başında* yeniden sorulur, yanlış olduğu anda döngü biter ve akış **DÖNGÜ SONU**’ndan sonraki satırdan sürer.

Koşullu döngünün genel biçimi



① **Hazırlık:** döngüden önce biriktirici ve sayaç ilk değerlerini alır ($\text{toplam} \leftarrow 0$).

② **Koşul:** her turun başında sorulur. Yanıt Hayır ise gövde bir kez bile çalışmaya-bilir.

③ **Gövde:** koşul doğru olduğu sürece yinelenen adımlar.

④ **Güncelleme:** koşulda geçen değişkeni değiştiren adım. Unutulursa döngü hiç bitmez.

Örnek: 1'den 5'e kadar toplama

```

toplam ← 0      ①
i ← 1          ①
i ≤ 5 İKEN     ②
    toplam ← toplam + i ③
    i ← i + 1    ④
DÖNGÜ SONU
YAZ toplam
  
```

tur	i	i ≤ 5?	toplam
–	1	–	0
1	1	Evet	1
2	2	Evet	3
3	3	Evet	6
4	4	Evet	10
5	5	Evet	15
–	6	Hayır	15

i ve koşul turun başındaki, toplam turun sonundaki değerdir.

Döngü, akışı geriye götüren tek yapıdır: koşul doğru olduğu sürece aynı adımlar yeniden yürütülür. Akış diyagramında bunu, gövdeden karar sembolüne geri dönen ok gösterir — sıralı yapıda böyle bir ok yoktur.

Tabloda koşul *altı* kez sorulur ama gövde *beş* kez çalışır: son soru döngüyü bitirmek içindir. Döngü bittiğinde i'nin değeri 6'dır, 5 değil — döngüden sonra sayacı kullanacaksanız bunu unutmayın.

Şekil 3.4: Koşullu döngünün genel biçimi ve dört parçası. Akışı *geriye* götüren mavi ok döngüyü karar yapısından ayırır. Sağdaki örnekte koşul altı kez sorulur ama gövde beş kez çalışır.

Şekil 3.4'te görüldüğü gibi her döngünün dört parçası vardır ve dördü de yazılmak zorundadır: ① *hazırlık* (biriktirici ve sayaçların ilk değerleri, döngüden önce), ② *koşul* (her tur başında sorulur; ilk soruda yalnızca gövde hiç çalışmaz), ③ *gövde* (yinelenen adımlar) ve ④ *güncelleme* (koşulda geçen değişkeni değiştiren adım). Dördüncüsü unutulursa koşul hiç yanlışılmaz ve döngü asla bitmez.

❌ Sık yapılan hata

$i \leftarrow 1$; $i \leq 5$ İKEN; YAZ i; DÖNGÜ SONU → gövdeye $i \leftarrow i + 1$ satırını ekleyin
Güncelleme satırı unutulduğunda i sonsuza kadar 1 kalır, koşul hep doğru çıkar ve ekrana durmaksızın 1 yazılır: **sonsuz döngü** (İng. *infinite loop*). Bilgisayarda bu, programın “kilitlenmesi” olarak görünür. İzleme tablosunda ise hemen bellidir: koşulda geçen sütun hiçbir satırda değişmiyorsa döngü bitmeyecektir. Her döngü yazdığınızda son kontrolünüz şu olsun: “koşuldaki değişkeni kim değiştiriyor?”

3.4.1 Nöbetçi değer

Kaç veri gireceğini kullanıcının kendisi belirlesin istiyorsak, verinin bittiğini haber veren özel bir değer belirleriz.

Tanım 3.5: Nöbetçi değer

Nöbetçi değer (İng. *sentinel*), bir veri dizisinin sonunu belirtmek için kullanılan, gerçek veri olamayacak özel bir değerdir: notlar okunurken –1, sıcaklıklar okunurken –999, adlar okunurken boş metin gibi. Nöbetçi değer *işlenmez*, yalnız döngüyü bitirir.

Örnek 3.3 (Nöbetçi değerle not ortalaması). Kaç tane olduğu önceden bilinmeyen sınav notlarını okuyan, -1 girilince duran ve notların ortalamasını yazan algoritmayı tasarlayın. 60, 75, 90, -1 girdisiyle izleyin.

Çözüm

Analiz. Girdi: bilinmeyen sayıda not ve sonunda -1 . Çıktı: ortalama. Biriktirici olarak **toplam** (ilk değer 0) ve **sayac** (ilk değer 0) gerekir. İlk not, döngüye girmeden önce okunmalıdır; yoksa koşul sorulduğunda **puan** kutusu henüz boş olur. Hiç not girilmezse sıfıra bölme olur; bu durumu bir kararla yakalarız.

Algoritma.

Sözde kod: Nöbetçi değerle ortalama

```
BAŞLA
toplama ← 0
sayac ← 0
YAZ "Not girin, bitirmek için -1:"
OKU puan                # ilk okuma: döngüden önce
puan ≠ -1 İKEN
    toplam ← toplam + puan
    sayac ← sayac + 1
    OKU puan                # güncelleme: bir sonraki değer
DÖNGÜ SONU
EĞER sayac > 0 İSE
    ortalama ← toplam / sayac
    YAZ ortalama
DEĞİLSE
    YAZ "Hiç not girilmedi."
EĞER SONU
BİTİR
```

Akış diyagramı. Şekil 3.4'teki genel biçim: hazırlık kutusu, ilk OKU, karar sembolü ($\text{puan} \neq -1$?), gövde ve geri dönüş oku; döngüden sonra ikinci bir karar sembolü ($\text{sayac} > 0$?).

İzleme tablosu. Girdi 60, 75, 90, -1 :

Adım	Yürütülen satır	puan	toplam	sayac	Ekran
1	$\text{toplam} \leftarrow 0$; $\text{sayac} \leftarrow 0$	—	0	0	
2	OKU puan	60	0	0	
3	koşul: $60 \neq -1 \rightarrow$ Evet	60	0	0	
4	$\text{toplam} \leftarrow \text{toplam} + \text{puan}$; $\text{sayac} \leftarrow \text{sayac} + 1$	60	60	1	
5	OKU puan; koşul: $75 \neq -1 \rightarrow$ Evet	75	60	1	
6	toplam; sayac	75	135	2	
7	OKU puan; koşul: $90 \neq -1 \rightarrow$ Evet	90	135	2	
8	toplam; sayac	90	225	3	
9	OKU puan; koşul: $-1 \neq -1 \rightarrow$ Hayır	-1	225	3	
10	EĞER sayac > 0 $\rightarrow$ Evet; $\text{ortalama} \leftarrow 225 / 3$	-1	225	3	
11	YAZ ortalama	-1	225	3	75

Kontrol. Elle: $60 + 75 + 90 = 225$ ve $225/3 = 75$ ✓. Makullük: ortalama en küçük ve en büyük notun arasında olmalı, $60 < 75 < 90$ ✓. Nöbetçi değerini toplama

katılmadığına dikkat edin: katılsaydı toplam 224, sayaç 4 ve ortalama 56 çıkardı — gözle görülür biçimde yanlış. Uç durum: ilk girdi olarak doğrudan -1 verilirse gövde hiç çalışmaz, $\text{sayac} = 0$ kalır ve **DEĞİLSE** dahi “Hiç not girilmedi” yazar; sıfıra bölme oluşmaz ✓.

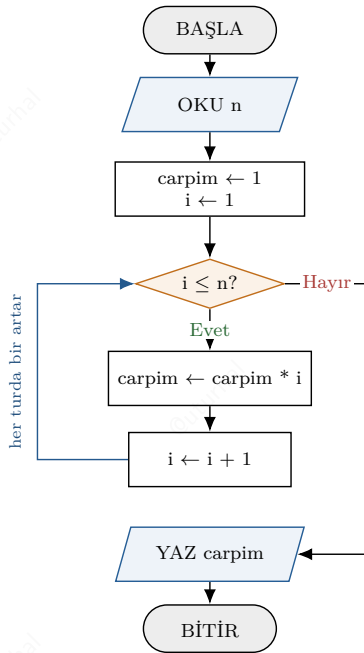
3.5 Sayaçlı döngü ve biriktiriciler

Kaç tur döneceği önceden belli olan döngüye *sayaçlı döngü* denir. Yapısı yine Tanım 3.4’teki **İKEN**’dir; farkı, koşulda bir **sayacın** (İng. *counter*) bulunması ve gövdenin sonunda sayacın bir artırılmasıdır.

Tanım 3.6: Biriktirici

Biriktirici (İng. *accumulator*), turlar boyunca kendi eski değeri üzerine ekleyerek büyüyen değişkendir: $\text{toplam} \leftarrow \text{toplam} + x$. Döngüden *önce* ilk değerini alır ve döngüden *sonra* okunur. İlk değer işlemin etkisiz elemanıdır: toplamda 0, çarpımda 1.

Faktöriyel: $n! = 1 \cdot 2 \cdot \dots \cdot n$



Üç biriktirici deseni

Amaç	İlk değer	Gövdedeki adım
toplam	$\text{toplam} \leftarrow 0$	$\text{toplam} \leftarrow \text{toplam} + x$
çarpım	$\text{carpim} \leftarrow 1$	$\text{carpim} \leftarrow \text{carpim} * x$
sayaç	$\text{sayac} \leftarrow 0$	$\text{sayac} \leftarrow \text{sayac} + 1$

İlk değer rastgele seçilmez: toplamın etkisiz elemanı 0, çarpımın 1’dir. Çarpımı 0 ile başlatırsanız sonuç her zaman 0 çıkar — biriktiriciyle ilgili en sık yapılan hata budur.

İzleme: $n = 5$ (sütunlar turun sonundaki değerler)

tur	$i \leq 5?$	i	carpim
–	–	1	1
1	Evet	2	1
2	Evet	3	2
3	Evet	4	6
4	Evet	5	24
5	Evet	6	120
–	Hayır	6	120

Gövde beş kez çalışır; altıncı soruda $i = 6 > 5$ olduğu için döngü biter.

$n = 0$ girilirse koşul daha ilk soruda yanlıştır: gövde hiç çalışmaz ve carpim ilk değeri olan 1 yazılır — matematikteki $0! = 1$ ile birebir aynı sonuç.

Sayaçlı döngü, kaç kez döneceği önceden belli olan döngüdür. Üç parçası her zaman birlikte bulunur: sayacın ilk değeri ($i \leftarrow 1$), bitiş koşulu ($i \leq n$) ve sayacın güncellenmesi ($i \leftarrow i + 1$). *Biriktirici* ise turlar boyunca büyüyen kutudur; döngüden *önce* hazırlanır, döngüden *sonra* okunur.

Şekil 3.5: Sayaçlı döngü. Solda faktöriyel algoritmasının akış diyagramı, sağda üç biriktirici deseni ve $n = 5$ için turların izlenmesi. n sıfır girilirse gövde hiç çalışmaz ve sonuç 1 olur.

Örnek 3.4 (Faktöriyel). Bir tam sayının faktöriyelini ($n! = 1 \cdot 2 \cdot \dots \cdot n$) hesaplayan algoritmayı yazın ve $n = 5$ ile izleyin.

Çözüm

Analiz. Girdi: n (negatif olmayan tam sayı). Çıktı: **carpim**. Çarpım biriktiricisi 1 ile başlar; sayaç 1'den n 'e kadar gider ve her turda biriktirici sayaçla çarpılır.

Algoritma.

Sözde kod: Faktöriyel

```
BAŞLA
OKU n
carpim ← 1
i ← 1
i ≤ n İKEN
    carpim ← carpim * i
    i ← i + 1
DÖNGÜ SONU
YAZ carpim
BİTİR
```

Akış diyagramı. Şekil 3.5'in sol yarısı. Hazırlık kutusunda iki atama, karar sembolünde $i \leq n$?, gövdede çarpma ve artırma, sonra geri dönüş oku.

İzleme tablosu. Girdi $n = 5$:

Tur	Koşul: $i \leq 5$?	i (tur başı)	carpim (tur sonu)	İşlem
–	–	1	1	hazırlık
1	Evet	1	1	$1 \cdot 1 = 1$
2	Evet	2	2	$1 \cdot 2 = 2$
3	Evet	3	6	$2 \cdot 3 = 6$
4	Evet	4	24	$6 \cdot 4 = 24$
5	Evet	5	120	$24 \cdot 5 = 120$
–	Hayır	6	120	döngü biter

Kontrol. Elle: $1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$ ✓. Uç durumlar: $n = 1$ için 1; $n = 0$ için koşul ilk soruda yanlıştır, gövde hiç çalışmaz ve biriktiricinin ilk değeri 1 yazılır — matematikteki $0! = 1$ tanımıyla birebir aynı ✓. Büyüme hızı: $10! = 3\,628\,800$.



Sık yapılan hata

carpim ← 0 (döngüden önce) → carpim ← 1

Çarpım biriktiricisi sıfırla başlatılırsa ilk turda $0 \cdot 1 = 0$ olur ve sonuç sonsuza kadar 0 kalır. Aynı hatanın tersi de yapılır: toplam biriktiricisini 1 ile başlatmak, sonucu her zaman 1 fazla gösterir. İlk değeri seçerken şunu sorun: “gövde hiç çalışmazsa hangi sonuç doğru olur?” Toplam için 0, çarpım için 1, sayaç için 0.



Dikkat

Döngü bittiğinde sayacın değeri bitiş sınırından *bir fazladır*: yukarıdaki izlemeye $i = 6$ 'dır, 5 değil. Döngüden sonra sayacı kullanacaksanız (“kaç tur döndük?” gibi) bunu hesaba katın ya da ayrı bir sayaç tutun. Bu tür bir hesap hatasına **birim kayması** (İng. *off-by-one*) denir; programlamanın en sık ve en sinsi hatasıdır. 1'den 10'a kadar toplamı isteyip koşulu $i < 10$ yazan bir algoritma 55 yerine 45 verir — yani doğruya çok yakın, ama yanlış.

3.6 İç içe döngüler

Bir döngünün gövdesinde başka bir döngü bulunabilir. Dış döngünün *her turunda* iç döngü baştan sona çalışır; toplam tur sayısı bu yüzden *çarpılır*.

Sözde kod

```
i ← 1
i ≤ 3 İKEN
  j ← 1
  j ≤ 4 İKEN
    YAZ i, "x", j, "=", i * j
    j ← j + 1
  DÖNGÜ SONU
  i ← i + 1
DÖNGÜ SONU
```

Ekrana yazılanlar (küçük mavi sayı: kaçınıcı tur)

	j = 1	j = 2	j = 3	j = 4
i = 1	1x 1= 1	1x 2= 2	1x 3= 3	1x 4= 4
i = 2	2x 1= 2	2x 2= 4	2x 3= 6	2x 4= 8
i = 3	3x 1= 3	3x 2= 6	3x 3= 9	3x 4= 12

İç döngü, dış döngünün her turunda baştan başlar. Bu yüzden $j \leftarrow 1$ satırı dış döngünün *içindedir*; dışarı alınırsa ikinci satır hiç yazılmaz.

Toplam tur sayısı $3 \times 4 = 12$ 'dir: dış döngü 3 kez, her birinde iç döngü 4 kez döner. Adım sayısı *çarpılır*, toplanmaz — 3.9'daki verimlilik tartışmasının çekirdeği budur.

iç döngü: soldan sağa bir satırı bitirir

İç içe döngüde dış döngünün bir turu, iç döngünün tamamıdır. Yürütme sırası mavi numaralarda görülür: $1 \rightarrow 4$ birinci satırı bitirir, sonra dış sayaç 2 olur ve iç sayaç yeniden 1'den başlar.

İki tuzak: (i) iç sayacın ilk değeri dış döngünün içinde verilmezse ikinci turda iç döngünün koşulu daha baştan yanlış olur ve yalnız ilk satır yazılır; (ii) iki döngüde aynı sayaç adı (i) kullanılırsa iç döngü dış sayacı bozar ve döngü ya erken biter ya da hiç bitmez. *Her döngünün kendi sayacı olur.*

Şekil 3.6: İç içe döngü: dış sayaç i satırı, iç sayaç j sütunu gezer. Hücrelerin üstündeki küçük mavi sayılar yürütme sırasını gösterir; $3 \text{ satır} \times 4 \text{ sütun} = 12 \text{ tur}$.

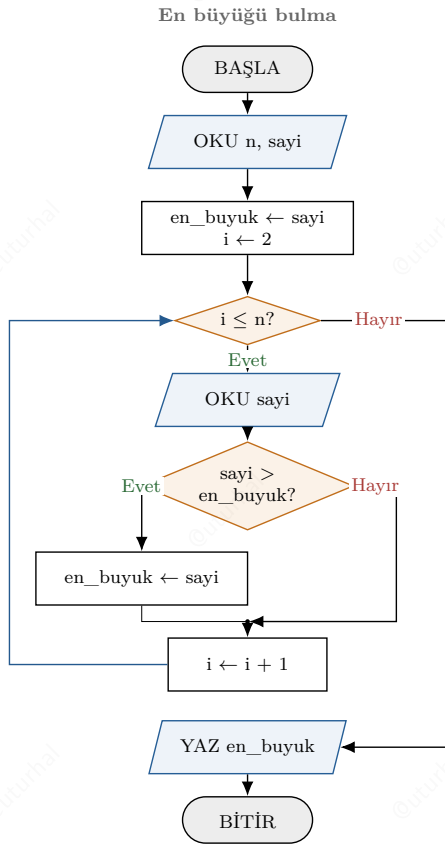
Şekil 3.6'deki algoritmayı 10×10 çarpım tablosuna genişletmek için iki sınırı 10 yapmak yeter; bu durumda ekrana $10 \cdot 10 = 100$ satır yazılır. Şeklin açıklamasındaki iki uyarıyı unutmayın: iç sayacın ilk değeri dış döngünün *içinde* verilmelidir ve her döngünün kendi sayaç adı olmalıdır.

İç içe döngüleri okurken “dış sayaç bir kez değişirken iç sayaç kaç kez değişir?” diye sorun; saat ibresi bir adım atarken dakika ibresi altmış adım atar. Bir tablonun satırları dış döngü, sütunları iç döngüdür.

3.7 Klasik algoritmalar

Aşağıdaki algoritmaları bilmek, bir müzisyenin gamları bilmesine benzer: kendileri için değil, daha büyük parçaların içinde geçtikleri için öğrenilirler. Hepsi karar ile tekrarı birleştirir ve dönem boyunca yeniden karşınıza çıkacaktır.

3.7.1 En büyüğü ve en küçüğü bulma

Girdi: $n = 5$, sayılar 23 41 17 58 34

1.	2.	3.	4.	5.
23	41	17	58	34
Okunan	Karşılaştırma	en_buyuk	Sonuç	
23	(ilk değer)	23	kutu açılır	
41	41 > 23 ✓	41	güncellenir	
17	17 > 41 ✗	41	değişmez	
58	58 > 41 ✓	58	güncellenir	
34	34 > 58 ✗	58	değişmez	

Neden ilk değer olarak 0 verilmez? Bütün sayılar negatifse ($-5, -9, -2$) sonuç 0 çıkardı — oysa 0 listede yok. Bu yüzden kutu *listenin ilk elemanı*yla açılır ve döngü ikinci elemandan başlar. En *küçüğü* bulmak için tek değişiklik karşılaştırmanın yönüdür ($sayi < en_kucuk$); aynı listede sonuç 17 olur.

En büyüğü bulma, karar ile döngünün ilk gerçek buluşmasıdır: döngü listeyi baştan sona gezer, karar her elemanda “bu, şimdiye kadarkinden büyük mü?” diye sorar.

Kutu yalnız Evet dalında değişir; Hayır dalı boştur. Diyagramda bu, karar sembolünün sağ dalının hiçbir işlem kutusuna uğramadan birleşme noktasına gitmesiyle gösterilir — *DEĞİLSE dalı boş olabilir*.

Liste n elemanlıysa karşılaştırma sayısı $n - 1$ 'dir; burada 4. Her elemana tam bir kez bakılır ve daha azına bakmak mümkün değildir: listede bakmadığınız bir eleman kalırsa en büyük o olabilir.

Şekil 3.7: En büyüğü bulma. Döngü listeyi baştan sona gezer, karar her elemanda “bu, şimdiye kadarkinden büyük mü?” diye sorar. Kutu ancak Evet dalında güncellenir; n elemanlı listede $n - 1$ karşılaştırma yapılır.

Sözde kod: En büyüğü bulma

```

BAŞLA
OKU n, sayi
en_buyuk ← sayi      # kutu ilk sayıyla açılır
i ← 2
i ≤ n İKEN
  OKU sayi
  EĞER sayi > en_buyuk İSE
    en_buyuk ← sayi
  EĞER SONU
  i ← i + 1
DÖNGÜ SONU
YAZ en_buyuk
BİTİR
  
```

5 sayılıklı 23, 41, 17, 58, 34 listesinde kutu sırasıyla 23, 41, 41, 58, 58 değerlerini alır; sonuç 58'dir (Şekil 3.7). En küçüğü bulmak için tek değişiklik karşılaştırmanın yönüdür ($sayi < en_kucuk$); aynı listede sonuç 17 olur. İki kutuyu aynı döngüde tutarak ikisini birden bulmak da mümkündür; bu durumda tur başına iki karşılaştırma yapılır.

3.7.2 Basamak toplamı

Geçen hafta üç basamaklı bir sayının basamaklarını // ve mod ile ayırmıştık; ama o çözüm yalnız üç basamak için çalışıyordu. Döngüyle, basamak sayısı ne olursa olsun çalışan bir algoritma yazabiliriz:

Sözde kod: Basamak toplamı ve basamak sayısı

```
BAŞLA
OKU n
toplam ← 0
adet ← 0
n > 0 İKEN
    toplam ← toplam + n mod 10    # son basamağı ekle
    n ← n // 10                  # son basamağı at
    adet ← adet + 1
DÖNGÜ SONU
YAZ toplam, adet
BİTİR
```

Girdi 4729 için turlar şöyle ilerler: $\text{toplam} = 9$, $n = 472$; 11 ve 47; 18 ve 4; 22 ve 0 — döngü biter. Sonuç: basamak toplamı 22, basamak sayısı 4; kontrol $4 + 7 + 2 + 9 = 22$ ✓. Uç durum: $n = 0$ girilirse gövde hiç çalışmaz ve “0 toplam, 0 basamak” çıkar; düzeltmesi bir alıştırmadır.

3.7.3 Asal testi

Örnek 3.5 (Asal testi). 2’den büyük bir tam sayının asal olup olmadığını bulan algoritmayı tasarlayın. 91 ve 97 ile izleyin.

Çözüm

Analiz. Girdi: n ($n \geq 2$). Çıktı: asal olup olmadığı, değilse bir bölen. Bölenleri 2’den başlayarak deneriz. Nereye kadar? $n = p \cdot q$ ise p ile q ’dan biri mutlaka $\sqrt{n}$ ’den küçük ya da ona eşittir; dolayısıyla $\text{bolen} * \text{bolen} > n$ olduğunda aramayı bırakabiliriz — bu, 3.9’un konusu olan bir adım sayısı kazancıdır.

Algoritma. Döngü iki koşullu: hem sınıra ulaşmamış hem de henüz bölen bulmamış olmalıyız.

Sözde kod: Asal testi

```
BAŞLA
OKU n                                # n ≥ 2 varsayılır
bolen ← 2
bolen * bolen ≤ n VE n mod bolen ≠ 0 İKEN
    bolen ← bolen + 1
DÖNGÜ SONU
EĞER bolen * bolen > n İSE
    YAZ n, "asaldır"
DEĞİLSE
    YAZ n, "asal değildir; bir böleni:", bolen
EĞER SONU
BİTİR
```

Akış diyagramı. Döngü karar sembolünde bileşik koşul, gövdede tek bir artırma kutusu, geri dönüş oku; döngüden sonra ikinci bir karar sembolü ve iki **YAZ** kutusu. Laboratuvarıda çizeceksiniz.

İzleme tablosu. Girdi $n = 91$:

Kontrol	bolen	$bolen * bolen \leq 91$	$91 \bmod bolen$	Koşul
1	2	$4 \leq 91$ doğru	1	Evet, sürdür
2	3	$9 \leq 91$ doğru	1	Evet, sürdür
3	4	$16 \leq 91$ doğru	3	Evet, sürdür
4	5	$25 \leq 91$ doğru	1	Evet, sürdür
5	6	$36 \leq 91$ doğru	1	Evet, sürdür
6	7	$49 \leq 91$ doğru	0	Hayır , dur

Döngü $bolen = 7$ iken biter. Döngüden sonraki karar: $49 > 91$ yanlış, öyleyse 91 asal değildir; bir böleni: 7.

Kontrol. $91 = 7 \cdot 13$ ✓. İkinci girdi $n = 97$: gövde sekiz kez çalışır, $bolen$ 10 olduğunda $100 \leq 97$ yanlıştır ve döngü biter; $100 > 97$ olduğu için 97 **asaldır** yazılır ✓ ($\sqrt{97} \approx 9,85$). Karekök sınırı olmasaydı 97 için 95 tur dönülürdü: aynı sonuç, on kat fazla iş.

3.7.4 EBOB: Öklid algoritması

Örnek 3.6 (EBOB — Öklid algoritması). İki pozitif tam sayının en büyük ortak bölenini (EBOB) bulan Öklid algoritmasını yazın ve 48 ile 18 için izleyin.

Çözüm

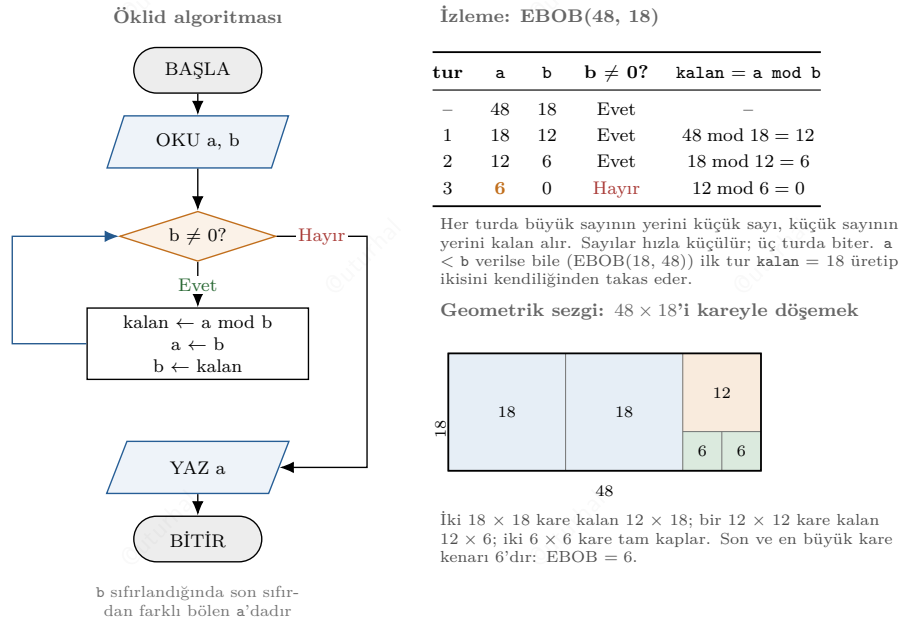
Analiz. Girdi: iki pozitif tam sayı. Çıktı: EBOB. Öklid'in gözlemi şudur: a ile b 'nin ortak bölenleri, b ile $a \bmod b$ 'nin ortak bölenleriyle aynıdır. Öyleyse sayıları, ortak bölenlerini bozmadan, kalan sıfır olana kadar küçültebiliriz.

Algoritma.

Sözde kod: EBOB: Öklid algoritması

```
BAŞLA
OKU a, b
b ≠ 0 İKEN
    kalan ← a mod b
    a ← b
    b ← kalan
DÖNGÜ SONU
YAZ a
BİTİR
```

Akış diyagramı. Şekil 3.8'in sol yarısı: tek karar sembolü ($b \neq 0?$), gövdede üç atamalı tek kutu, geri dönüş oku.



Öklid'in fikri şudur: a ile b 'nin ortak bölenleri, b ile $a \bmod b$ 'nin ortak bölenleriyle aynıdır. Bu yüzden sayıları ortak bölenlerini bozmadan küçültürebiliriz; kalan sıfır olduğunda elde kalan sayı EBOB'dur.

Kontrol: $48 = 6 \cdot 8$, $18 = 6 \cdot 3$; 8 ile 3'ün ortak böleni yok, demek ki 6 daha büyütülemez. EKOK ise $48 \cdot 18 / 6 = 144$ 'tür. Deneme yanılma ile bölen arayan bir algoritma 18 kez bölme yapardı; Öklid üç turda bitirir — *doğru algoritma, hızlı bilgisayardan daha çok kazandırır.*

Şekil 3.8: Öklid algoritması. Solda akış diyagramı, ortada EBOB(48, 18) izlemesi, sağda geometrik sezgi: 48×18 dikdörtgenini kareyle döşerken kullanılan en büyük kare kenarı EBOB'dur.

İzleme tablosu. Girdi 48 ve 18:

Tur	Yürütülen satırlar	kalan	a	b
–	OKU a, b	–	48	18
1	kalan $\leftarrow 48 \bmod 18$; a $\leftarrow$ b; b $\leftarrow$ kalan	12	18	12
2	kalan $\leftarrow 18 \bmod 12$; a $\leftarrow$ b; b $\leftarrow$ kalan	6	12	6
3	kalan $\leftarrow 12 \bmod 6$; a $\leftarrow$ b; b $\leftarrow$ kalan	0	6	0
–	koşul: $0 \neq 0 \rightarrow$ Hayır; YAZ a	0	6	0

Kontrol. $48 = 6 \cdot 8$ ve $18 = 6 \cdot 3$; 8 ile 3'ün ortak böleni olmadığına göre 6 büyütülemez ✓. İki sayının çarpımı EBOB ile EKOK'un çarpımına eşittir: $48 \cdot 18 = 864$ ve $864 / 6 = 144 = \text{EKOK}$ ✓. Uç durum: sayılar ters sırada verilirse (EBOB(18, 48)) ilk tur **kalan** = $18 \bmod 48 = 18$ üretir, **a** = 48, **b** = 18 olur; yani algoritma ikisini kendiliğinden takas eder ve bir tur kaybeder, o kadar ✓. Geometrik kontrol **Şekil 3.8**'in sağındadır.

3.7.5 Fibonacci dizisi

Her terimi kendinden önceki iki terimin toplamı olan dizi, iki değişkenin sürekli kaydırılmasıyla üretilir:

Sözde kod: İlk n Fibonacci terimi

```
onceki ← 1   simdiki ← 1   i ← 3
YAZ onceki, simdiki
i ≤ n İKEN
    yeni ← onceki + simdiki
    YAZ yeni
    onceki ← simdiki
    simdiki ← yeni
    i ← i + 1
DÖNGÜ SONU
```

$n = 10$ için ekrana 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 yazılır; onuncu terim 55'tir. Son iki atamanın *sırası* önemlidir: ters yazılırsa *onceki* ile *simdiki* aynı değeri tutar ve dizi 1, 1, 2, 4, 8, ... diye bozulur.

3.8 Dizi kavramına giriş

Şimdiye kadar her değer için ayrı bir kutu kullandık. Peki 100 öğrencinin notunu tutmak gerekirse 100 ad mı uyduracağız? Çözüm, tek bir ada bağlı *numaralandırılmış* kutulardır.

Tanım 3.7: Dizi ve indeks

Dizi (*İng. array*), aynı türden değerleri tek bir ad altında, sırayla numaralanmış kutularda tutan yapıdır. Bir kutuya, adının yanına köşeli parantez içinde **indeksi** (*İng. index*) yazılarak erişilir: *sayilar*[0], *sayilar*[1], ... Bu notlarda indeksler **sıfırdan** başlar; n elemanlı bir dizinin son elemanı *sayilar*[$n-1$]'dir.

Neden sıfırdan? Çünkü 4. haftadan itibaren kullanacağımız Python (ve C, Java, JavaScript) böyle sayar. Beş elemanlı *sayilar* = [23, 41, 17, 58, 34] dizisinde *sayilar*[0] = 23 ve *sayilar*[4] = 34'tür; *sayilar*[5] diye bir kutu *yoktur*. Dizinin asıl gücü, indeksin bir *değişken* olabilmesidir: döngünün sayacını indeks yaparak bütün elemanları tek bir görevle gezebiliriz.

Sözde kod: Dizinin toplamı ve ortalaması

```
toplam ← 0   i ← 0
i < n İKEN
    toplam ← toplam + sayilar[i]
    i ← i + 1
DÖNGÜ SONU
ortalama ← toplam / n
YAZ toplam, ortalama
```

Tanım 3.6'deki *toplam* kutusu burada da aynı işi görür: yukarıdaki beş elemanlı dizide toplam $23 + 41 + 17 + 58 + 34 = 173$, ortalama $173/5 = 34,6$ olur. Koşulun $i < n$ olduğuna dikkat edin: indeksler 0'dan $n-1$ 'e gittiği için $i \leq n$ yazmak dizinin dışına taşar. Bu, birim kaymasının dizilerdeki en yaygın biçimidir.

Dizinin Python'daki karşılığı olan *liste* 13. haftanın konusudur. Bir yapay zeka mühendisi için dizi en temel veri yapısıdır: bir görüntü piksel dizisi, bir ses örnek dizisi, bir modelin parametreleri sayı dizisidir.

3.9 Algoritma verimliliğine giriş: adım sayma

Aynı işi yapan iki algoritma yazdınız; hangisi daha iyi? Doğruluk ilk ölçüttür, ama iki algoritma da doğruysa ikinci ölçüt *kaç adımda* bitirdikleridir.

Tanım 3.8: Adım sayısı

Bir algoritmanın **adım sayısı** (*İng. step count*), girdinin büyüklüğüne (genellikle n ile gösterilir) bağlı olarak yürütülen temel işlem sayısıdır. Karşılaştırma sırasında saniye değil *adım* sayarız; çünkü saniye bilgisayardan bilgisayara değişir, adım sayısı değişmez.

Tanım 3.8 bu haftanın algoritmalarına kolayca uygulanır: en büyüğü bulma n elemanlı listede $n - 1$ karşılaştırma yapar (5 eleman için 4, 1000 eleman için 999): liste iki katına çıkarsa iş de iki katına çıkar. İç içe iki döngü ise n^2 tur döner — $n = 10$ için 100, $n = 100$ için 10 000; liste iki katına çıkınca iş *dört* katına çıkar.

3.9.1 Doğrusal arama ve ikili arama

Bir listede değer aramanın en doğal yolu baştan başlayıp tek tek bakmaktır: **doğrusal arama** (*İng. linear search*); aranan değer sondaysa ya da listede hiç yoksa n adım gerekir. Liste *sıralıysa* çok daha iyisi yapılabilir: telefon rehberini ortadan açar, aradığınız adın önde mi arkada mı olduğuna bakar ve yarısını atarsınız. Bu yönteme **ikili arama** (*İng. binary search*) denir — her adımda aday listenin *yarısı* elenir.

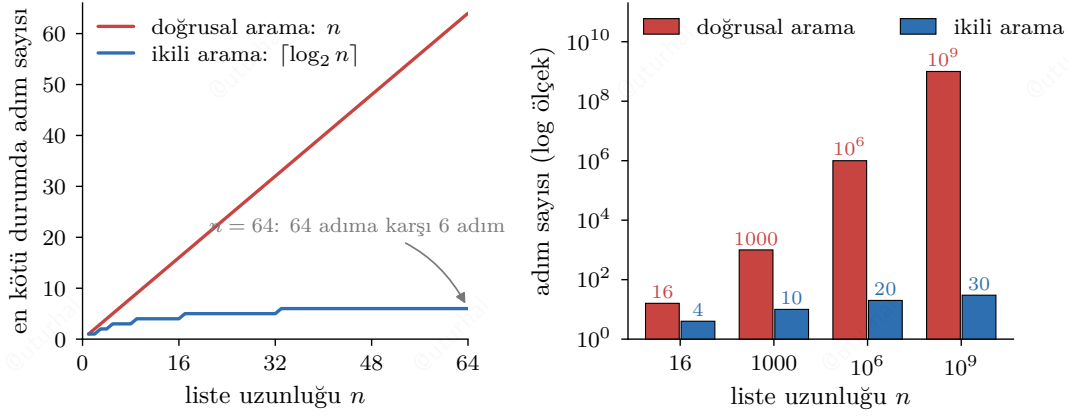
İkili arama — fikir

Aday aralığın ortasındaki elemana bak; aranan değere eşitse dur. Küçükse aday aralık sol yarı, büyükse sağ yarıdır; aralık boşalana kadar yinele. *Ön koşul*: liste sıralı olmalıdır — sırasız bir listede “sol yarıda olmalı” çıkarımı yapılamaz.

Kaç adım sürer? Aday sayısı $n, n/2, n/4, \dots$ diye iner; bire inmesi için gereken yarılama sayısı $\log_2 n$ 'dir (Tablo 3.2 ve Şekil 3.9).

Tablo 3.2: Doğrusal ve ikili aramanın en kötü durumdaki adım sayıları. İkili arama sütunu $\lceil \log_2 n \rceil$ değeridir.

Liste uzunluğu n	Doğrusal	İkili	Ne anlama gelir
16	16	4	küçük listede fark önemsiz
1000	1000	10	bir sınıf listesi
1 000 000	1 000 000	20	bir şehrin telefon rehberi
10^9	10^9	30	bir arama motorunun dizini



Şekil 3.9: Doğrusal aramanın (n) ve ikili aramanın ($\log_2 n$) adım sayıları. Solda küçük listeler için doğrudan karşılaştırma, sağda dört liste uzunluğu için logaritmik ölçekte. Liste 16'dan 10^9 'a, yani 62,5 milyon kat büyürken ikili aramanın adım sayısı yalnız 4'ten 30'a çıkar.

Tablo 3.2 şunu söyler: liste 16'dan 10^9 'a, yani 62,5 milyon kat büyürken ikili aramanın işi yalnız 7,5 kat artar. Bir milyon kayıta ikili arama, doğrusal aramadan 50 000 kat az adım yapar; hiçbir işlemci yükseltmesi bu farkı kapatmaz. Veriniz büyüdükçe *algoritma seçimi donanımdan daha önemli hâle gelir*.

İkili aramanın bedeli, listenin sıralı olmasıdır; sıralama ise kendi başına bir iş gerektirir. Kural şudur: bir listede *bir kez* arama yaparsanız doğrusal arama yeterlidir; *çok kez* arayarsanız önce sıralayıp sonra ikili aramak toplamda daha ucuzdur. Sıralama algoritmalarını ve bu hesabı 13. haftada göreceksiniz.

LAB Laboratuvar Çalışması 3: Karar, döngü ve hata avı

Bu haftanın laboratuvarında da Python yoktur: kâğıt, kalem ve bir akış diyagramı çizim programı yeter. Görev 1, 3, 4 kâğıtta; Görev 2, 5 bilgisayarda; Görev 4 eşli yapılır. Dersin son 25 dakikasında **Kısa Sınav 1** vardır (kapsam 1–3. haftalar).

Görev 1. Karar akış diyagramları. Üç kural için **Kural 3.1**'e uygun sözde kod yazıp akış diyagramını çizin; her biri için en az üç test girdisi (biri sınır) verin. (a) Bir sayının pozitif, negatif ya da sıfır olduğu ($-7 \rightarrow$ negatif, $0 \rightarrow$ sıfır). (b) **Tablo 3.1**'teki bilet ücreti; 6, 18, 65 yaşları (0, 80, 20 TL). (c) Üç kenarın üçgen oluşturup oluşturmadığı: her kenar diğer ikisinin toplamından küçük olmalı (**VE** ile tek koşul); 3, 4, 5 $\rightarrow$ üçgen, 1, 2, 9 $\rightarrow$ değil.

Görev 2. Döngüyü çalıştır. Flowgorithm'i açın (yoksa <https://app.diagrams.net> ile yalnız çizin) ve **Örnek 3.4**'in diyagramını kurup Run ile çalıştırın: 5 için 120, 0 için 1, 7 için 5040. Sonra *bilerek* güncelleme kutusunu silin, çalıştırın, ne olduğunu bir cümleyle yazın ve durdurun.

Görev 3. İzleme tablosuyla döngü takibi. Şu algoritmayı $n = 6$ için satır satır izleyin (sütunlar: i , toplam, Ekran): $\text{toplam} \leftarrow 0$; $i \leftarrow 1$; $i \leq n$ İKEN: EĞER $i \bmod 2 = 0$ İSE $\text{toplam} \leftarrow \text{toplam} + i$ EĞER SONU; $i \leftarrow i + 1$; DÖNGÜ SONU; **YAZ** toplam, i . Beklenen 12 7. Sonra $n = 7$ için yineleyin (12 8) ve toplamın neden değişmediğini açıklayın.

Görev 4. Hata avı (eşli). Aşağıdaki iki algoritmanın her birinde bir hata vardır. Arkadaşınızla önce *izleyerek* bulun, sonra düzeltip yeniden izleyin. (a) 1'den 10'a kadarki sayıların toplamını yazacak: $\text{toplam} \leftarrow 0$; $i \leftarrow 1$; $i < 10$ İKEN: $\text{toplam} \leftarrow \text{toplam} + i$; $i \leftarrow i + 1$; DÖNGÜ SONU; **YAZ** toplam. (Çıkan: 45;

beklenen: 55.) (b) Kullanıcı 0 girene kadar sayıları toplayacak: $\text{toplam} \leftarrow 0$; **OKU** sayi; $\text{sayi} \neq 0$ **İKEN**: $\text{toplam} \leftarrow \text{toplam} + \text{sayi}$; **DÖNGÜ SONU**; **YAZ** toplam. (Ne olur?)

Görev 5. Çarpım tablosu. İç içe döngüyle 1’den 5’e kadar çarpım tablosunu ekrana yazan algoritmayı sözde kod olarak yazın ve Flowgorithm’de çalıştırın. Kaç satır yazıldığını sayın (25 olmalı). Sonra iç döngünün sınırını $j \leq i$ yapın: kaç satır yazılır? (Beklenen: 15.)

Görev 6. Adım sayma (ek). Bir arkadaşınız 1–100 arasında bir sayı tutsun; siz her tahminden sonra “daha büyük / daha küçük” bilgisiyle bulun. Her seferinde aralığın ortasını söylerseniz en fazla 7 tahmin yeter ($\log_2 100 \approx 6,6$). Sonra 1’den başlayıp birer birer deneyin ve kaç adım gerektiğini yazın.

Teslim. Görev 2 ve 5’in Flowgorithm dosyalarını ders bitmeden gösterin; kâğıtlarınızı saklayın — 6. ve 8. haftalarda bu diyagramları Python’a çevireceksiniz.

Özet

Kavram	Özü	Not
Karar	EĞER koşul İSE ... DEĞİLSE ... EĞER SONU	iki daldan tam biri çalışır; DEĞİLSE isteğe bağlı
Çoklu karar	DEĞİLSE EĞER zinciri; ilk doğru koşulda durulur	sıra anlamı belirler: en dar koşul en üste
Karşılaştırma	$= \neq < \leq > \geq$; aralık: $x \geq a$ VE $x \leq b$	$=$ sorar, $\leftarrow$ atar; değişken adı iki kez yazılır
Mantıksal işleç	VE (ikisi de), VEYA (en az biri), DEĞİL (tersi)	öncelik: DEĞİL > VE > VEYA
Döngü	koşul İKEN ... DÖNGÜ SONU ; hazzırlık, koşul, gövde, güncelleme	koşul her tur başında sorulur; güncelleme yoksa sonsuz döngü
Biriktirici	$\text{toplam} \leftarrow \text{toplam} + x$	ilk değer: toplam 0, çarpım 1, sayaç 0
Nöbetçi değer	veriyi bitiren özel değer (−1)	nöbetçi işlenmez; ilk OKU döngüden önce
Sayaçlı döngü	$i \leftarrow 1$; $i \leq n$ İKEN ; $i \leftarrow i + 1$	döngü bitince $i = n + 1$; iç içe döngüde adım sayısı çarpılır
En büyük/küçük	kutuyu <i>ilk elemanla</i> aç, gezerken güncelle	$n - 1$ karşılaştırma; 0 ile başlatma
Öklid EBOB	$b \neq 0$ İKEN : $\text{kalan} \leftarrow a \bmod b$, kaydır	$\text{EBOB}(48, 18) = 6$; $\text{EKOK} = 144$
Asal testi	2’den $\sqrt{n}$ ’e kadar bölen ara	$\text{bolen} * \text{bolen} \leq n$ sınırı
Dizi, birim kayması	$\text{sayilar}[i]$, indeks 0’dan $n - 1$ ’e	döngü koşulu $i < n$; 1–10 toplamı 55, yanlışı 45
Adım sayısı	girdinin büyüklüğüne bağlı temel işlem sayısı	doğrusal arama n , ikili arama $\log_2 n$

Alıştırmalar

Alıştırma 3.1. ●●● $x = 7$ ve $y = 3$ iken aşağıdaki koşulların her birinin doğru mu yanlış mı olduğunu bulun.

- | | |
|--------------------------|----------------------------------|
| (a) $x > y$ | (e) DEĞİL ($x = y$) |
| (b) $x \bmod 2 = 0$ | (f) $x // y = 2$ |
| (c) $x > 5$ VE $y > 5$ | (g) $x > y$ VE DEĞİL ($y = 3$) |
| (d) $x > 5$ VEYA $y > 5$ | (h) $(x + y) \bmod 5 = 0$ |

Cevap: (a) D (b) Y (c) Y (d) D (e) D (f) D (g) Y (h) D (D: doğru, Y: yanlış)

Alıştırma 3.2. ●●● Aşağıdaki algoritmayı izleme tablosuyla çalıştırın; ekrana ne yazılır?
 $t \leftarrow 0$; $i \leftarrow 1$; $i \leq 6$ İKEN: EĞER $i \bmod 2 = 0$ İSE $t \leftarrow t + i$ EĞER SONU; $i \leftarrow i + 1$; DÖNGÜ SONU; **YAZ** t , i .

Cevap: 12 7: yalnız çift sayılar toplanır ($2 + 4 + 6$); döngü bitince $i = 7$

Alıştırma 3.3. ●●● (Kavramsal.) Aşağıdaki cümlelerin hangileri doğrudur? Yanlış olanları düzeltin. (a) Karar sembolünden üç ok çıkabilir. (b) Döngüyü akış diyagramında karar yapısından ayıran şey, bir okun geriye dönmesidir. (c) **DEĞİLSE** dalı her karar yapısında bulunmak zorundadır. (d) Bir döngünün gövdesi en az bir kez çalışır. (e) Çoklu kararda koşulların sırası değiştirilebilir, sonuç değişmez.

Cevap: yalnız (b) doğru. (a) iki ok; (c) isteğe bağlı; (d) gövde hiç çalışmayabilir; (e) sıra anlamı belirlir

Alıştırma 3.4. ●●● Bir sayıyı okuyup “pozitif”, “negatif” ya da “sıfır” yazan sözde kodu yazın ve akış diyagramını çizin. -7 , 0 ve 4 ile izleyin.

Cevap: $-7 \rightarrow$ negatif, $0 \rightarrow$ sıfır, $4 \rightarrow$ pozitif; üç dallı zincir, iki karar sembolü

Alıştırma 3.5. ●●● Bir kargo firmasının ücret kuralı şöyledir: şehir içi gönderide taban ücret 60 TL, şehirler arası gönderide 90 TL’dir; 5 desiyi aşan her desi için şehir içinde 10 TL, şehirler arasında 15 TL eklenir. Karar tablosunu kurun, sözde kodu yazın ve şu üç gönderiyi hesaplayın: (3 desi, şehir içi), (8 desi, şehirler arası), (12 desi, şehir içi).

Cevap: 60 ; $90 + 15 \cdot 3 = 135$; $60 + 10 \cdot 7 = 130$ TL

Alıştırma 3.6. ●●● 1 ’den n ’e kadar olan tek sayıların toplamını hesaplayan sözde kodu yazın ve $n = 9$ ile izleyin. Sonucu bir de $n = 5$ için bulun ve iki sonuca bakarak bir örüntü tahmin edin.

Cevap: $n = 9 \rightarrow 25$, $n = 5 \rightarrow 9$; ilk k tek sayının toplamı k^2

Alıştırma 3.7. ●●● Nöbetçi değerle (-1) sonlanan bir sayı dizisini okuyup en büyüğünü, en küçüğünü ve ortalamasını yazan sözde kodu yazın. 12 , 45 , 7 , 30 , -1 ile izleyin.

Cevap: en büyük 45 , en küçük 7 , toplam 94 , sayı 4 , ortalama $23,5$

Alıştırma 3.8. ●●● Basamak toplamı algoritmasını $90\,762$ için izleyin: her turda toplam ve n değerlerini yazın. Kaç basamak vardır? Ayrıca algoritmanın $n = 0$ girdisinde neden “0 basamak” dediğini açıklayın ve bunu düzeltecek bir satır önerin.

Cevap: toplam 24, 5 basamak; toplam sırasıyla 2, 8, 15, 15, 24. $n = 0$ için gövde hiç çalışmaz; düzeltme: başa EĞER $n = 0$ İSE adet $\leftarrow 1$ EĞER SONU

Alıştırma 3.9. ●●● Aşağıdaki iki algoritmanın her birinde bir hata vardır. Hatayı izleme tablosuyla bulun, türünü adlandırın (sonsuz döngü / birim kayması) ve düzeltin. (a) 1'den 10'a kadarki sayıların toplamı: $\text{toplam} \leftarrow 0$; $i \leftarrow 1$; $i < 10$ İKEN: $\text{toplam} \leftarrow \text{toplam} + i$; $i \leftarrow i + 1$; DÖNGÜ SONU; **YAZ** toplam. (b) Kullanıcı 0 girene kadar sayıları toplama: $\text{toplam} \leftarrow 0$; **OKU** sayi; $\text{sayi} \neq 0$ İKEN: $\text{toplam} \leftarrow \text{toplam} + \text{sayi}$; DÖNGÜ SONU; **YAZ** toplam.

*Cevap: (a) birim kayması: 45 çıkar, 55 olmalı ($i \leq 10$). (b) sonsuz döngü: gövdenin sonuna **OKU sayi** eklenir*

Alıştırma 3.10. ●●● Aşağıdaki iç içe döngü ekrana kaç yıldız yazar? $i \leftarrow 1$; $i \leq 4$ İKEN: $j \leftarrow 1$; $j \leq i$ İKEN: **YAZ** "*"; $j \leftarrow j + 1$; DÖNGÜ SONU; $i \leftarrow i + 1$; DÖNGÜ SONU. Aynı algoritma $i \leq n$ ile yazılıysaydı kaç yıldız yazılırdı? Sonucu n cinsinden bir formülle verin ve $n = 100$ için hesaplayın.

Cevap: $1 + 2 + 3 + 4 = 10$; genel olarak $n(n + 1)/2$; $n = 100 \rightarrow 5050$

Alıştırma 3.11. ●●● (a) Öklid algoritmasını 84 ve 36 için izleyin; kaç tur döner, EBOB kaçtır? (b) Aynı algoritmayı 1071 ve 462 için izleyin. (c) İki sayının EKOK'unu EBOB'dan hesaplayan bir satır ekleyin ve 84 ile 36 için sonucu yazın.

Cevap: (a) 2 tur, EBOB = 12 (12, 0) (b) 3 tur, EBOB = 21 (147, 21, 0) (c) $3024/12 = 252$

Alıştırma 3.12. ●●● (Kavramsal.) 1 048 576 kayıtlı sıralı bir listede bir kayıt aranıyor. (a) Doğrusal arama en kötü durumda kaç adım yapar? (b) İkili arama kaç adım yapar? (c) İkili arama neden listenin sıralı olmasını gerektirir — bir cümleyle açıklayın. (d) Liste sıralı değilse ve yalnız *bir kez* arama yapılacaksa hangi yöntemi seçersiniz, neden?

Cevap: (a) 1 048 576 (b) 20 ($2^{20} = 1\,048\,576$) (c) eleme ancak sıralı listede yapılabilir (d) doğrusal arama: sıralama daha pahalı

Müfredat: Programlama Dilinin Temel Kavramları (Giriş/Çıkış, Veri türleri, Değişkenler)

Hafta 4: Python'a Giriş: Giriş/Çıkış, Veri Türleri ve Değişkenler

Öğrenme çıktıları

Bu haftanın sonunda:

1. Python'u bilgisayarınıza kurup bir betik dosyasını IDLE ya da PyCharm ile, bir not defterini de Google Colab'da çalıştırabileceksiniz;
2. değişken tanımlayıp Python'un adlandırma kurallarına uygun, anlamlı adlar seçebileceksiniz;
3. `int`, `float`, `str` ve `bool` türlerini ayırt edip `type()` ile sorgulayabilecek, aralarında dönüşüm yapabileceksiniz;
4. `input()` ile kullanıcıdan veri alıp `print()` ve f-string ile biçimli çıktı üretebileceksiniz;
5. aritmetik operatörleri işlem önceliğine göre doğru kullanarak girdi-işlem-çıkı yapılarında küçük programlar yazabileceksiniz;
6. bir hata mesajını okuyup hatanın türünü ve yerini söyleyebileceksiniz.

Anahtar kavramlar

yorumlayıcı (İng. *interpreter*) **betik** (İng. *script*) **etkileşimli kabuk** (İng. *REPL*) **değişken** (İng. *variable*) **atama** (İng. *assignment*) **veri türü** (İng. *data type*) **tür dönüşümü** (İng. *type casting*) **sabit değer** (İng. *literal*) **ifade** (İng. *expression*) **deyim** (İng. *statement*) **f-string** **istisna** (İng. *exception*)

İlk üç hafta boyunca algoritmaları kâğıt üzerinde tasarladık: sözde kod yazdık, akış diyagramı çizdik, izleme tablosuyla adım adım çalıştırdık. Ama bir akış diyagramı kendi kendine çalışmaz; onu bilgisayarın anlayacağı bir dile çevirmek gerekir. Bu hafta o dili — **Python'u** — öğrenmeye başlıyoruz. Kâğıttaki **OKU**, bilgisayarda `input()` olacak; **YAZ**, `print()`; **ata** oku ise tek bir eşittir işareti.

Neden Python? Çünkü bir yapay zeka mühendisinin günlük araçlarının — NumPy, pandas, scikit-learn, PyTorch — hepsi Python'dan kullanılır; çünkü söz dizimi İngilizceye yakındır ve öğrenmenin ilk haftalarında sizi dilin kurallarıyla değil problemle uğraştırır; çünkü etkileşimli kabuğu sayesinde her fikri saniyeler içinde deneyebilirsiniz. Bu haftanın sonunda, ikinci haftada akış diyagramını çizdiğiniz *ortalama hesabı* programını gerçekten çalıştırmış olacaksınız.

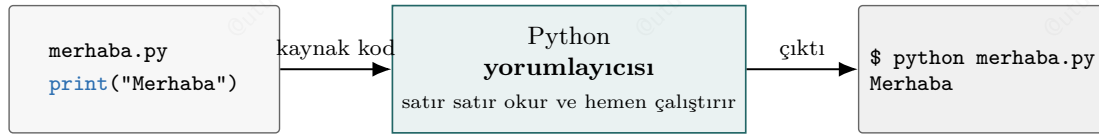
4.1 Programlama dili ve Python

Bilgisayar yalnız *makine kodunu* — işlemciye özgü ikili komutları — çalıştırır. İnsan ise makine kodu yazamaz; bu yüzden aradaki boşluğu **yüksek seviyeli programlama dilleri** doldurur. İnsanın yazdığı **kaynak kod** (İng. *source code*) bir çeviriciden geçer ve işlemcinin anlayacağı biçime dönüşür. Bu çeviri iki biçimde yapılır ([Şekil 4.1](#)):

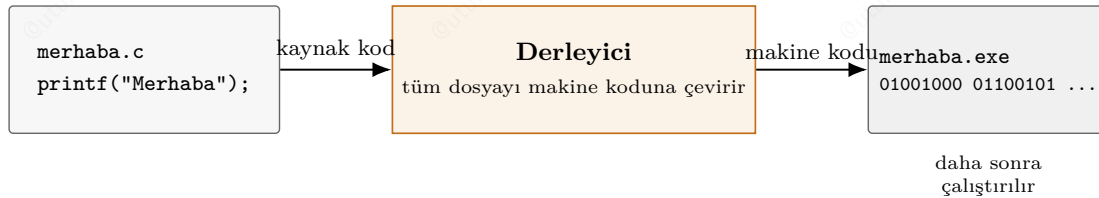
Tanım 4.1: Derleyici ve yorumlayıcı

Derleyici (İng. *compiler*), kaynak kodun *tamamını* bir kez çevirip çalıştırılabilir bir dosya (makine kodu) üretir; program daha sonra bu dosyadan çalışır. **Yorumlayıcı** (İng. *interpreter*) ise kaynak kodu *her çalıştırmada* satır satır okur ve okuduğunu hemen yürütür; ayrı bir çalıştırılabilir dosya oluşmaz. C ve C++ derlenen, Python ve JavaScript yorumlanan dillerin tipik örnekleridir.

Yorumlanan dil (Python)



Derlenen dil (C)



Yorumlayıcı, kaynak kodu ayrı bir çalıştırılabilir dosyaya dönüştürmez; her çalıştırmada kodu yeniden okur. Bu, hata bulmayı ve denemeyi kolaylaştırır (etkileşimli kabuk!), ama aynı programın derlenmiş bir C sürümünden daha yavaş çalışmasına yol açar. Yapay zeka kütüphanelerinin (NumPy, PyTorch) çekirdekleri bu yüzden C/C++ ile yazılır ve Python'dan çağrılır.

Şekil 4.1: Yorumlanan bir dil (üstte, Python) ile derlenen bir dilin (altta, C) çalışma biçimi. Python'da kaynak kod ile çıktı arasında yorumlayıcıdan başka bir şey yoktur; bu yüzden bir satır yazıp hemen sonucunu görebilirsiniz.

Yorumlayıcı yaklaşımının bedeli hızdır: aynı algoritma derlenmiş C'de çoğu zaman on ile yüz kat daha hızlı çalışır. Yapay zeka kütüphanelerinin ağır hesap yapan çekirdekleri bu yüzden C/C++ ile yazılıp Python'dan çağrılır; siz kolay dilde düşünürsünüz, hesap hızlı dilde yapılır. Bu derste bu ayrım sizi ilgilendirmez; ama ikinci sınıfta “neden NumPy dizileri düz Python listelerinden bu kadar hızlı?” diye sorduğunuzda cevabı buradadır.

Not

Python, Guido van Rossum tarafından 1991'de yayımlandı; adını yılandan değil, İngiliz komedi grubu *Monty Python*'dan alır. Bugün kullanılan sürüm ailesi Python 3'tür (2008'den bu yana); Python 2 desteği 2020'de sona ermiştir. Bu notlar Python 3.10 ve üzerini varsayar. Bir kaynakta `print "merhaba"` gibi parantezsiz bir yazım görürseniz o kaynak Python 2 içindir ve artık çalışmaz.

4.2 Çalışma ortamı: kurulum ve ilk program

Python'u çalıştırmanın üç yolu vardır (Şekil 4.2) ve bu derste üçünü de kullanacağız. Kurulum adımları ekran görüntüleriyle Ek B'de verilmiştir; burada yalnız neyin ne için olduğunu söylüyoruz.

Anaconda Python yorumlayıcısını, sık kullanılan bilimsel kütüphaneleri (NumPy, pandas, matplotlib) ve bir paket yöneticisini tek kurulumda getiren dağıtım. Laboratuvar bilgisayarlarında kuruludur; kendi bilgisayarınıza da bunu kurmanızı öneririz. Daha hafif bir seçenek, python.org'dan yalnız yorumlayıcıyı indirmektir.

IDLE Python ile birlikte gelen basit geliştirme ortamı. İki penceresi vardır: **Shell** (etkileşimli kabuk) ve **Editor** (dosya düzenleyici). Küçüktür, kurulum gerektirmez, bu haftaki her şey için yeterlidir.

PyCharm Profesyonel geliştirme ortamı (*İng. IDE*). Kod tamamlama, hata işaretleme, adım adım çalıştırma (*debugger*). Ücretsiz Community sürümü yeterlidir. Projeler büyüdüğünde (11. haftadan itibaren) IDLE'dan buna geçeceğiz.

Google Colab

Tarayıcıda çalışan, kurulum gerektirmeyen *not defteri* ortamı. Google hesabı yeter. Kod hücreler hâlinde yazılır; her hücrenin çıktısı altında kalır. Kurulum yapamayan ya da tablette çalışan öğrenciler için yedek, ileride veri analizi için ana araçtır.

1. Etkileşimli kabuk (REPL)

IDLE Shell, terminalde python

```
>>> 2 + 3
5
>>> print("selam")
selam
>>> _
```

Her satır yazılır yazılmaz çalışır ve sonuç görünür. Denemek ve öğrenmek için.

2. Betik dosyası (.py)

IDLE, PyCharm, VS Code

```
# hesap.py
a = 3
b = 4
print(a * b)
```

Komutlar dosyaya yazılır, dosya bir bütün olarak çalıştırılır. Gerçek programlar için.

3. Not defteri (Colab / Jupyter)

tarayıcıda, kurulumuz

```
[1]: x = 10
[2]: x * 2
20
[3]: _
```

Kod *hücreler* hâlinde; her hücre ayrı çalışır, çıktısı altında kalır. Veri analizi ve raporlama için.

Şekil 4.2: Python'ı çalıştırmamanın üç biçimi. Etkileşimli kabuk denemek için, betik dosyası gerçek programlar için, not defteri veri analizi ve raporlama için kullanılır.

4.2.1 Etkileşimli kabuk

IDLE'ı açtığınızda karşınıza **Shell** penceresi gelir; `>>>` işareti Python'un komut beklediğini gösterir. Yazdığınız her ifade `[Enter]` ile hemen değerlendirilir:

Python etkileşimli kabuk (REPL)

```
>>> 2 + 3
5
>>> 7 * 8
56
>>> "Merhaba" + " Dünya"
'Merhaba Dünya'
>>> 10 / 4
2.5
```

Bu pencerenin adı **REPL**'dir: *Read* (oku) – *Eval* (değerlendir) – *Print* (yazdır) – *Loop* (yinele). Kabuk, bir hesap makinesi gibi kullanılır; ama yazdıklarımız kaydedilmez, pencereyi kapattığınızda gider. Bir şeyi *denemek* istediğinizde kabuğu, bir şeyi *saklamak* istediğinizde betik dosyasını kullanın.

İpucu

Kabukta `[Alt]+[P]` önceki komutu geri getirir (macOS'ta `[Ctrl]+[P]`); uzun bir ifadeyi her seferinde yeniden yazmak zorunda kalmazsınız. Colab'da ise `[Shift]+[Enter]` hücreyi çalıştırıp bir sonrakine geçer.

4.2.2 İlk betik

IDLE'da File → New File ile düzenleyici açılır. Aşağıdaki üç satırı yazıp **merhaba.py** adıyla kaydedin (`[Ctrl]+[S]`) ve `[F5]` ile çalıştırın. Uzantı **.py** olmalıdır; IDLE bunu eklemeyiz, siz eklersiniz.



Kod 4.1 w04-01-merhaba.py — İlk Python programı

```
1 # w04-01-merhaba.py -- İlk Python programı
2 # Bir program, yukarıdan aşağıya sırayla çalışan komutlardan oluşur.
3
4 print("Merhaba Dünya!")
5 print("YPZ105 Programlama ve Uygulamaları I")
6 print("İlk programım çalıştı.")
```

```
$ python w04-01-merhaba.py
```

```
Merhaba Dünya!
YPZ105 Programlama ve Uygulamaları I
İlk programım çalıştı.
```

Üç satır, üç **deyim** (*İng. statement*): her biri Python'a bir iş yaptırır. **print()** bir **işlev**dir (*İng. function*): parantez içine ne verirsiniz ekrana yazar ve satır sonuna geçer. Tırnak içindeki metne **karakter dizisi** (*İng. string*) denir; tırnaklar metnin parçası değildir, yalnız sınırlarını gösterir; çift (") ya da tek (') tırnak fark etmez, yeter ki açtığımızla kapatın.

Yalnız kaynak kodu yazmak yetmez, kodu *çalıştırmak* gerekir. IDLE'da (F5); PyCharm'da yeşil ok; terminalde dosyanın bulunduğu klasörde `$ python merhaba.py`. Python yorumlayıcısı dosyayı açar, birinci satırdan başlayarak sırayla yürütür, son satırdan sonra durur. Bu **sıralı akış**, ikinci haftadaki akış diyagramlarının yukarıdan aşağı okunmasının birebir karşılığıdır.

✗ Sık yapılan hata

`Print("Merhaba")` → `print("Merhaba")`

Python **büyük/küçük harfe duyarlıdır**: `print`, `Print` ve `PRINT` üç ayrı addır ve yalnız ilki tanımlıdır. Diğer ikisi `NameError` verir. Aynı kural değişken adları için de geçerlidir: `toplam` ile `Toplam` farklı değişkenlerdir.

✗ Sık yapılan hata

`print("Merhaba)` → `print("Merhaba")`

Açılan her tırnak ve her parantez kapatılmalıdır. Eksik tırnak `SyntaxError: unterminated string literal`, eksik parantez `SyntaxError: '(' was never closed` hatası verir. IDLE ve PyCharm eşleşen parantezi renklendirir; bu renklendirmeye alışın.

4.3 Değişkenler ve atama

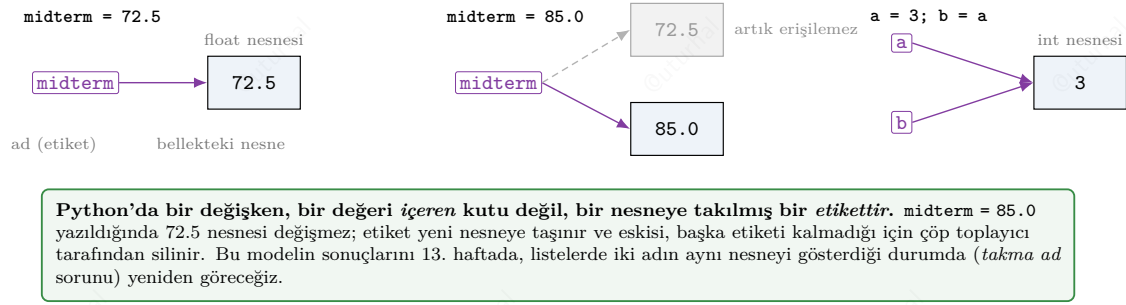
Bir program veriyle çalışır: kullanıcının girdiği yaş, hesaplanan ortalama, okunan dosya adı. Bu verilerin program çalışırken bir yerde durması ve bir adla çağrılabilmesi gerekir.

Tanım 4.2: Değişken ve atama

Değişken (*İng. variable*), bir değere verilen addır. **Atama** (*İng. assignment*) deyimi

$$\text{ad} = \text{ifade}$$

önce sağdaki **ifadeyi** (*İng. expression*) hesaplar, sonra elde edilen değeri soldaki ada bağlar. Aynı ada yeniden atama yapıldığında ad yeni değeri gösterir; eski değerle bağı kopar (Şekil 4.3).



Şekil 4.3: Python'da değişken, değeri içeren bir kutu değil, bir nesneye takılan etikettir. Yeniden atama etiketi taşır; iki ad aynı nesneyi gösterebilir.

Bu *etiket* modelinin bu hafta görünen tek sonucu, bir değişkenin türünün sabit olmamasıdır: `x = 5` ile tam sayı olan `x`, bir satır sonra `x = "beş"` ile karakter dizisi olabilir. Python bunu izin verir; iyi bir programcı ise bir adı hep aynı tür veri için kullanır, çünkü okuyan kişi (çoğu zaman üç hafta sonraki siz) adın türünü tahmin etmek zorunda kalmamalıdır.

Kural 4.1 (Ad seçme kuralları). Bir değişken adı

- (i) harf ya da alt çizgi (`_`) ile başlar; rakamla başlayamaz;
- (ii) yalnız harf, rakam ve alt çizgiden oluşur; boşluk, tire (`-`), nokta içeremez;
- (iii) Python'un **anahtar sözcüklerinden** (*Ing. keyword*) biri olamaz (Tablo 4.1);
- (iv) büyük/küçük harfe duyarlıdır.

Bunlar dilin *zorunlu* kuralıdır. Bir de topluluğun *geleneği* (PEP 8) vardır: değişken ve işlev adları küçük harfle, sözcükler alt çizgiyle ayrılarak yazılır (`student_name`, *snake_case*); sabitler büyük harfle (`MAX_SCORE`).

Tablo 4.1: Python 3'ün 35 anahtar sözcüğü. Hiçbiri değişken adı olarak kullanılamaz; çoğunu ilerleyen haftalarda öğreneceksiniz.

False	None	True	and	as	assert	async
await	break	class	continue	def	del	elif
else	except	finally	for	from	global	if
import	in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with	yield

Tablo 4.2: Geçerli, geçersiz ve geçerli ama kötü değişken adları.

Ad	Durum	Neden
total_score	✓ geçerli, iyi	küçük harf, sözcükler alt çizgiyle ayrılmış
score2	✓ geçerli	rakam başta değil, içeride olabilir
_temp	✓ geçerli	alt çizgiyle başlayabilir (özel anlamı 12. haftada)
ortalama	✓ geçerli	Türkçe ASCII ad sorunsuz; yine de İngilizce tercih edilir
sayı	✓ geçerli, kötü	Python 3 Unicode adları kabul eder; ama klavye ve taşınabilirlik sorunları yaratır
2nd_score	✗ geçersiz	rakamla başlıyor
total-score	✗ geçersiz	tire çıkarma operatörüdür: total - score okunur
total score	✗ geçersiz	boşluk içeriyor
class	✗ geçersiz	anahtar sözcük
Class	✓ geçerli, kötü	harf büyüklüğü farkı yüzünden geçerli; ama kafa karıştırır
x	✓ geçerli, kötü	neyi tuttuğu belli değil; döngü sayacı dışında kaçının
print	✓ geçerli, çok kötü	yerleşik işlevin adını gölgeler; print() artık çalışmaz

Kod 4.2 w04-02-degiskenler.py — Atama, yeniden atama, çoklu atama ve sabitler

```

1 # w04-02-degiskenler.py -- Değişkenler ve atama
2
3 # Atama: sağdaki değer hesaplanır, soldaki ada bağlanır.
4 student_name = "Ayşe Yılmaz"
5 student_no = 2026105017
6 midterm = 72.5
7 is_passing = True
8
9 print("Ad Soyad :", student_name)
10 print("Numara   :", student_no)
11 print("Vize notu:", midterm)
12 print("Geçiyor mu?", is_passing)
13
14 # Bir değişkenin değeri sonradan değiştirilebilir (variable = değişebilen).
15 midterm = 85.0
16 print("Yeni vize notu:", midterm)
17
18 # Çoklu atama ve iki değişkenin değerini takas etme
19 a, b = 3, 8
20 print("Önce : a =", a, " b =", b)
21 a, b = b, a
22 print("Sonra: a =", a, " b =", b)
23
24 # Sabitler: değişmemesi gereken değerler BÜYÜK HARFLE yazılır (bir
    ↪ gelenektir,
25 # Python zorlamaz).
```



GitHub

```

26 PI = 3.14159
27 GRAVITY = 9.81
28 print("Yerçekimi ivmesi:", GRAVITY, "m/s^2")

```

```
$ python w04-02-degiskenler.py
```

```

Ad Soyad : Ayşe Yılmaz
Numara   : 2026105017
Vize notu: 72.5
Geçiyor mu? True
Yeni vize notu: 85.0
Önce : a = 3  b = 8
Sonra: a = 8  b = 3
Yerçekimi ivmesi: 9.81 m/s^2

```

Kod 4.2’de üç ayrıntıya dikkat edin. Birincisi, `a, b = b, a` satırı iki değişkenin değerini *tek adımda* değiştirir; Python önce sağ tarafın tamamını hesaplar (8, 3), sonra sol tarafa dağıtır. Başka dillerde üçüncü bir geçici değişken gerektiren bu işlem Python’da tek satırdır. İkincisi, `PI` ve `GRAVITY` büyük harfle yazılmıştır: bu bir **sabit** (*Ing. constant*) geleneğidir, Python bu değerleri korumaz — `PI = 3` yazarsanız kabul eder — ama okuyan herkes bu ada yeniden atama yapılmaması gerektiğini anlar. Üçüncüsü, `print()` virgülle ayrılmış birden çok değer alır ve aralarına birer boşluk koyar.

Sık yapılan hata

`5 = x → x = 5`

Atamada ad **solda**, değer **sağda** durur. `=` işareti matematikteki eşitlik değildir; “sağdakini hesapla, soldakine koy” komutudur. Bu yüzden `x = x + 1` matematikte saçma, Python’da son derece olağandır: eski değere bir ekle, sonucu yine `x`’e koy. Eşitlik *sorgusu* için 6. haftada `==` kullanacaksınız.

4.4 Veri türleri

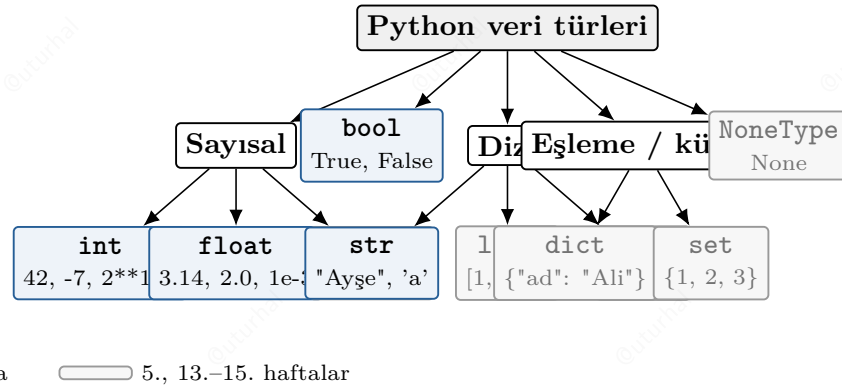
Bilgisayar için 12 ile "12" aynı şey değildir: birincisi bir sayıdır, toplanabilir; ikincisi iki karakterden oluşan bir metindir, ancak başka metinlerle birleştirilebilir. Bir değerın hangi işlemlere girebileceğini onun **veri türü** (*Ing. data type*) belirler.

Tanım 4.3: Temel veri türleri

Bu haftanın dört temel türü:

- `int` (*Ing. integer*): tam sayılar, 42, -7, 0. Python’da büyüklük sınırı yoktur.
- `float` (*Ing. floating point*): ondalıklı sayılar, 3.14, 2.0, 1.5×10^{-3} (`1.5e-3`). Yaklaşık 15–16 anlamlı basamak tutar.
- `str` (*Ing. string*): karakter dizileri, "Ayşe", 'a', "" (boş metin).
- `bool` (*Ing. boolean*): yalnız iki değer, `True` ve `False`.

Bir değerın türünü `type()` işlevi söyler.



Şekil 4.4: Python'un yerleşik veri türleri. Mavi kutular bu haftanın konusu; gri olanlar 5. haftada (str ayrıntılı), 13.-15. haftalarda (list, tuple, dict, set) gelecek.

Kod 4.3 w04-03-veri-turleri.py — Dört temel tür, type() ve türlerin davranış farkları

```

1 # w04-03-veri-turleri.py -- Temel veri türleri ve type() işlevi
2
3 count = 12                # int   : tam sayı
4 price = 49.90             # float : ondalıklı (kayan noktalı) sayı
5 city = "Trabzon"         # str   : karakter dizisi (metin)
6 is_open = False          # bool  : mantıksal değer (True / False)
7
8 print(count, "->", type(count))
9 print(price, "->", type(price))
10 print(city, "->", type(city))
11 print(is_open, "->", type(is_open))
12
13 # Metin ve sayı aynı şey değildir: "12" bir metindir, 12 bir sayıdır.
14 text_number = "12"
15 print(text_number + text_number)    # metinler birleştirilir -> 1212
16 print(count + count)               # sayılar toplanır      -> 24
17
18 # int sınırsız büyüklükte olabilir
19 big = 2 ** 100
20 print("2 üzeri 100 =", big)
21
22 # float her ondalık sayıyı tam tutamaz
23 print("0.1 + 0.2 =", 0.1 + 0.2)
24 print("0.1 + 0.2 == 0.3 ?", 0.1 + 0.2 == 0.3)
25 print("Yuvarlayarak karşılaştırma:", round(0.1 + 0.2, 10) == 0.3)
  
```

```
$ python w04-03-veri-turleri.py
```

```

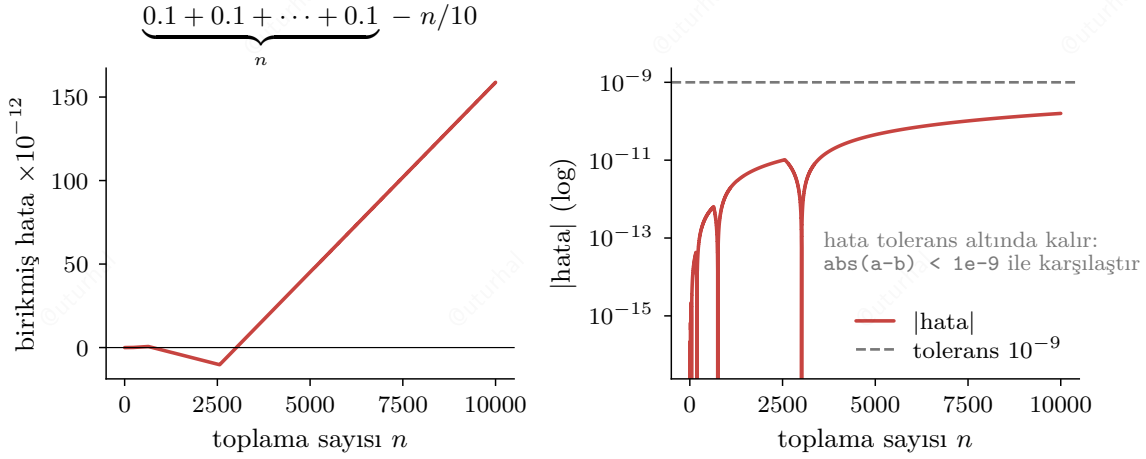
12 -> <class 'int'>
49.9 -> <class 'float'>
Trabzon -> <class 'str'>
False -> <class 'bool'>
1212
24
2 üzeri 100 = 1267650600228229401496703205376
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 ? False
Yuvarlayarak karşılaştırma: True
  
```



Çıktıdaki `<class 'int'>` gösterimi “int sınıfından” diye okunur; şimdilik *sınıf* sözcüğünü *tür* ile eş anlamlı sayın. “12” + “12” sonucunun 1212 olması yeni öğrenenlerin çoğunu şaşırtır: + operatörü iki sayı gördüğünde toplar, iki metin gördüğünde **birleştirir** (İng. *concatenation*). Bir sayı ile bir metin gördüğünde ise ne yapacağını bilemez ve `TypeError` verir; bunu 4.9’da göreceğiz.

Dikkat

Kod 4.3’in son satırları `float` türünün en önemli tuzağını gösterir: `0.1 + 0.2` sonucu `0.30000000000000004`’tür, `0.3` değil. Bunun nedeni Python değil, bilgisayarın sayıları ikili tabanda tutmasıdır: onluk tabanda $1/3$ nasıl sonsuz ondalıkla ($0,333\dots$) yazılırsa, ikili tabanda $0,1$ de sonsuz basamakla yazılır ve 64 bite sığdırılırken kesilir. **Şekil 4.5**, bu küçük hatanın toplandıkça büyüdüğünü gösterir. Kural: **iki float değeri hiçbir zaman == ile karşılaştırmayın**; farklarının küçük bir toleransın altında kalıp kalmadığına bakın, ya da `round()` kullanın. Para hesaplarında ise en güvenli yol, tutarı kuruş cinsinden *tam sayı* tutmaktır (bkz. Alıştırma 4.11).



Şekil 4.5: Solda: 0.1 değerini n kez toplayıp $n/10$ 'dan çıkarınca kalan hata; 10^4 toplamada $1,6 \times 10^{-10}$ 'a ulaşır. Sağda: aynı hatanın mutlak değeri logaritmik eksenle; 10^{-9} toleransının altında kalır, dolayısıyla toleranslı karşılaştırma güvenlidir.

Not

`bool` türü aslında `int`'in özel bir hâlidir: `True + True` sonucu 2'dir ve `True == 1` doğrudur. Bu özelliği 13. haftada, bir koşulu sağlayan elemanları saymak için `sum()` ile birlikte kullanacağız. Şimdilik `True/False` değerlerini yalnız karşılaştırma sonucu olarak düşünün.

4.5 Tür dönüşümü ve kullanıcıdan veri almak

Sözde koddaki **OKU** deyiminin Python karşılığı `input()` işlevidir.

Tanım 4.4: input() işlevi

`input(istem)` önce *istem* metnini ekrana yazar, sonra kullanıcının `[Enter]`'a basana kadar yazdıklarını bekler ve yazılanı **her zaman str türünde** döndürür. Kullanıcı 19 yazsa bile dönen değer "19" metnidir; onunla aritmetik yapmak için **tür dönüşümü** (İng. *type casting*) gerekir: `int()`, `float()`, `str()`, `bool()`.



Kod 4.4 w04-04-donusum.py — input() her zaman metin döndürür; dönüşüm işlevleri

```

1 # w04-04-donusum.py -- input() her zaman metin döndürür; tür dönüşümü gerekir
2
3 age_text = input("Yaşınızı girin: ")
4 print("Girilen değer:", age_text, "-> türü:", type(age_text))
5
6 age = int(age_text)          # metinden tam sayıya
7 print("10 yıl sonra yaşıınız:", age + 10)
8
9 height = float(input("Boyunuzu metre olarak girin: "))
10 print("Boyunuz santimetre olarak:", height * 100)
11
12 # Sayıdan metne dönüşüm: metinle birleştirmek için gerekir
13 message = "Yaş: " + str(age) + ", Boy: " + str(height) + " m"
14 print(message)
15
16 # Dönüşümler bilgi kaybedebilir
17 print(int(3.99))            # ondalık kısım ATILIR, yuvarlanmaz -> 3
18 print(int(-3.99))           # -> -3
19 print(float(7))             # -> 7.0
20 print(int("0042"))          # baştaki sıfırlar önemsiz -> 42
21 print(bool(0), bool(5), bool(""), bool("hayır"))

```

```
$ python w04-04-donusum.py
```

```

Yaşınızı girin: 19
Girilen değer: 19 -> türü: <class 'str'>
10 yıl sonra yaşıınız: 29
Boyunuzu metre olarak girin: 1.75
Boyunuz santimetre olarak: 175.0
Yaş: 19, Boy: 1.75 m
3
-3
7.0
42
False True False True

```

Çıktıda kullanıcının yazdığı değerler (19, 1.75) istemle aynı satırda görünür; bu, gerçek bir terminalde de böyledir. Kod 4.4'daki `float(input(...))` yazımı iki işlevi iç içe kullanır: önce içteki `input()` çalışır ve metni döndürür, sonra dıştaki `float()` bu metni sayıya çevirir. Bu iç içe çağrı biçimini çok kullanacaksınız.

Dönüşümlerin ayrıntıları önemlidir:

- `int(3.99)` sonucu 3'tür: ondalık kısım *atılır*, yuvarlanmaz. Yuvarlamak için `round()` kullanılır.
- `int("3.5")` **hata** verir: metin bir tam sayıya benzemiyor. Önce `float("3.5")`, sonra gerekirse `int()`.
- `int("abc")` ve `float("on iki")` `ValueError` verir. Kullanıcının yanlış girdisine karşı programı korumayı 7. haftada öğreneceğiz.
- `bool()`: sıfır ve boş metin `False`, geri kalan her şey `True`'dur. `bool("False")` bile `True`'dur — çünkü boş olmayan bir metindir.

Kullanıcıdan sayı almanın standart üç adımı

1. İstemi yazın; ne beklediğinizi ve birimi söyleyin: **"Boyunuz (m): "**. Sondaki boşluk kullanıcının yazdığıyla istemin yapışmasını öner.
2. Dönen metni uygun türe çevirin: tam sayı bekliyorsanız `int()`, ondalık bekliyorsanız `float()`. Emin değilseniz `float()` güvenli tercihtir.
3. Sonucu anlamlı bir ada atayın; `x` değil `height`.

Bu üç adım tek satıra sığar: `height = float(input("Boyunuz (m): "))`.

✗ Sık yapılan hata

`age = input("Yaş: "); print(age + 1) → age = int(input("Yaş: ")); print(age + 1)`
Bu haftanın en sık hatasıdır. `input()` metin döndürür; metinle sayı toplanamaz (`TypeError: can only concatenate str (not "int") to str`). Hata mesajındaki *concatenate* (birleştirme) sözcüğü ipucudur: Python + işaretini metin birleştirme sanmıştır.

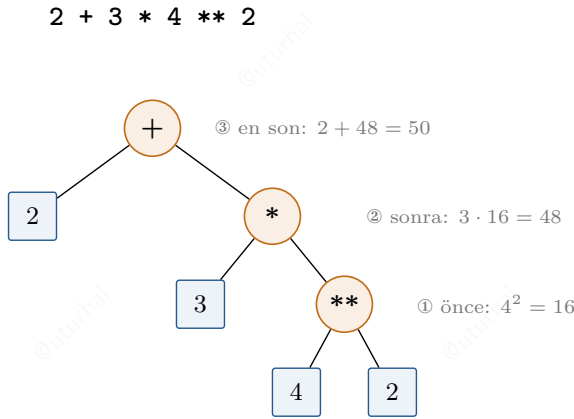
4.6 Aritmetik operatörler ve işlem önceliği

Tablo 4.3: Aritmetik operatörler. Örneklerde $a = 7$, $b = 2$.

Operatör	Anlamı	Örnek	Sonuç
+	toplama	<code>a + b</code>	9
-	çıkarma	<code>a - b</code>	5
*	çarpma	<code>a * b</code>	14
/	bölme (sonuç her zaman float)	<code>a / b</code>	3.5
//	tam bölme (aşağıya yuvarlanmış)	<code>a // b</code>	3
%	kalan (mod)	<code>a % b</code>	1
**	üs alma	<code>a ** b</code>	49
-x	işaret değiştirme	<code>-a</code>	-7

İki operatör özellikle önemlidir çünkü matematik derslerinden tanıdık değildir. **Tam bölme** `//` bölümün tam kısmını, **kalan** `%` ise artanı verir: $7 = 3 \cdot 2 + 1$, yani $7 // 2 = 3$ ve $7 \% 2 = 1$. Bu ikili, bir sayıyı parçalara ayıran her problemde karşınıza çıkar: saniyeyi saat-dakika-saniyeye, kuruşu banknot ve madenî paraya, bir sayıyı basamaklarına ayırmak. Negatif sayılarda `//` *aşağıya*, yani daha küçük sayıya yuvarlar: $-7 // 2 = -4$ (çünkü $-3,5$ 'in altındaki tam sayı -4 'tür) ve kalan bölünenin işaretini alır: $-7 \% 2 = 1$. Her durumda $(a // b) * b + a \% b == a$ eşitliği korunur.

Kural 4.2 (İşlem önceliği). Bir ifadede önce parantez içi, sonra üs alma (`**`), sonra çarpma-bölme grubu (`*`, `/`, `//`, `%`), en son toplama-çıkarma (`+`, `-`) yapılır. Aynı seviye-deki operatörler soldan sağa işlenir; tek istisna `**`'dir, o sağdan sola işlenir: $2 ** 3 ** 2 = 2^{(3^2)} = 2^9 = 512$.

**Öncelik sırası (yüksekten düşüğe)**

()	parantez
**	üs alma (sağdan sola)
-x	işaret değiştirme
* / // %	çarpma ve bölmeler
+ -	toplama, çıkarma

Aynı seviyedekiler *soldan sağa* işlenir; tek istisna **: $2 ** 3 ** 2 = 2^9 = 512$. Kararsız kaldığınızda parantez kullanın — fazladan parantezin bedeli yoktur, yanlış önceliğin vardır.

Şekil 4.6: $2 + 3 * 4 ** 2$ ifadesinin değerlendirme ağacı. Ağacın en altındaki işlem en önce yapılır; sonuç yukarı taşınır. Yaprakları sayılar, iç düğümleri operatörler olan bu ağaç, Python'un ifadeyi "gördüğü" biçimdir.

Kod 4.5 w04-06-operatorler.py — Operatörler, öncelik, kısaltılmış atama ve math modülü

```

1 # w04-06-operatorler.py -- Aritmetik operatörler ve işlem önceliği
2
3 print("7 + 2 =", 7 + 2)
4 print("7 - 2 =", 7 - 2)
5 print("7 * 2 =", 7 * 2)
6 print("7 / 2 =", 7 / 2)      # bölme: sonuç HER ZAMAN float
7 print("7 // 2 =", 7 // 2)    # tam bölme: aşağıya yuvarlanmış tam sayı
8 print("7 % 2 =", 7 % 2)      # kalan (mod)
9 print("7 ** 2 =", 7 ** 2)    # üs alma
10
11 print("-7 // 2 =", -7 // 2)  # aşağıya (daha küçük sayıya) yuvarlar: -4
12 print("-7 % 2 =", -7 % 2)   # kalan işareti bölenle aynıdır: 1
13 print("6 / 3 =", 6 / 3)     # tam bölünse bile float: 2.0
14
15 # İşlem önceliği: ** > (* / // %) > (+ -), aynı seviyede soldan sağa
16 print("2 + 3 * 4 ** 2 =", 2 + 3 * 4 ** 2)    # 2 + 3*16 = 50
17 print("(2 + 3) * 4 ** 2 =", (2 + 3) * 4 ** 2) # 5*16 = 80
18 print("2 ** 3 ** 2 =", 2 ** 3 ** 2)          # üs alma SAĞDAN sola: 2**9
19 # ↪ = 512
20 print("100 / 10 / 2 =", 100 / 10 / 2)        # soldan sağa: 5.0
21
22 # Kısaltılmış atamalar
23 total = 10
24 total += 5      # total = total + 5
25 total *= 2      # total = total * 2
26 print("total =", total)
27
28 # math modülü
29 import math
30 print("karekök(2) =", math.sqrt(2))
31 print("pi =", math.pi)
32 print("tavan(3.2) =", math.ceil(3.2), " taban(3.8) =", math.floor(3.8))
33 print("round(2.5) =", round(2.5), " round(3.5) =", round(3.5)) # bankacı
34 # ↪ yuvarlaması
35 print("round(3.14159, 2) =", round(3.14159, 2))

```




```
$ python w04-06-operatorler.py
7 + 2 = 9
7 - 2 = 5
7 * 2 = 14
7 / 2 = 3.5
7 // 2 = 3
7 % 2 = 1
7 ** 2 = 49
-7 // 2 = -4
-7 % 2 = 1
6 / 3 = 2.0
2 + 3 * 4 ** 2 = 50
(2 + 3) * 4 ** 2 = 80
2 ** 3 ** 2 = 512
100 / 10 / 2 = 5.0
total = 30
karekök(2) = 1.4142135623730951
pi = 3.141592653589793
tavan(3.2) = 4 taban(3.8) = 3
round(2.5) = 2 round(3.5) = 4
round(3.14159, 2) = 3.14
```

Kod 4.5’de iki yeni yapı var. **Kısaltılmış atama** `total += 5`, `total = total + 5` ile aynıdır; `-=`, `*=`, `/=`, `//=`, `%=`, `**=` de vardır. Biriktirme yapan her döngüde (8. hafta) bunu kullanacaksınız. **Modül** (*Ing. module*) ise başkalarının yazdığı işlevlerin toplandığı bir dosyadır; `import math` satırı onu programa getirir, sonrasında `math.sqrt`, `math.pi`, `math.floor`, `math.ceil` biçiminde erişilir. Karekök için `x ** 0.5` de yazılabilir; ama `math.sqrt(x)` ne yaptığını söyler.

Dikkat

`round()` beklediğiniz gibi çalışmayabilir: `round(2.5)` sonucu 2, `round(3.5)` sonucu 4’tür. Python “yarımları çift sayıya” yuvarlar (*bankacı yuvarlaması*); büyük veri kümelerinde toplam hatayı azaltan bu kural, IEEE 754 standardının bir parçasıdır. Üstelik `round(2.675, 2)` sonucu 2.67’dir, 2.68 değil — çünkü 2.675 bellekte aslında 2.67499999... olarak durur. Ekranı yazdırırken yuvarlama için `round()` yerine 4.7’deki f-string biçimlerini kullanın; hesapta yuvarlama gerekiyorsa kuralı bilin.

Örnek 4.1 (Saniyeyi saat, dakika ve saniyeye ayırma). Kullanıcıdan toplam saniye alan ve bunu “s saat d dakika sn saniye” biçiminde yazan bir program yazın. Örnek: 3725 saniye.

Çözüm

Analiz. Girdi: toplam saniye (tam sayı). Çıktı: saat, dakika, saniye. İşlem: bir saatte 3600, bir dakikada 60 saniye vardır; saat sayısı toplamın 3600’e tam bölümü, kalan saniyeler ise 3600’e göre kalanıdır. Aynı işlem kalan üzerinde 60 ile yinelenir.

Algoritma.

Sözde kod: Saniye ayrıştırma

```

BAŞLA
  OKU toplam
  saat ← toplam // 3600
  kalan ← toplam % 3600
  dakika ← kalan // 60
  saniye ← kalan % 60
  YAZ saat, dakika, saniye
BİTİR

```

Kod ve çıktı.

Kod 4.6 w04-05-zaman.py — Saniye ayrıştırma: // ve % birlikte

```

1 # w04-05-zaman.py -- Tam bölme (//) ve kalan (%) ile saniyeyi
  ↪ saat:dakika:saniye yapma
2
3 total_seconds = int(input("Toplam saniye: "))
4
5 hours = total_seconds // 3600           # 3600 saniyede bir saat
6 remaining = total_seconds % 3600       # saatlerden artan saniye
7 minutes = remaining // 60
8 seconds = remaining % 60
9
10 print(total_seconds, "saniye =", hours, "saat", minutes, "dakika",
  ↪ seconds, "saniye")
11
12 # Kontrol: parçaları birleştirip aynı sayıyı geri elde etmeliyiz
13 check = hours * 3600 + minutes * 60 + seconds
14 print("Kontrol:", check == total_seconds)

```

```
$ python w04-05-zaman.py
```

```

Toplam saniye: 3725
3725 saniye = 1 saat 2 dakika 5 saniye
Kontrol: True

```

Kontrol. Elle: $3725 = 1 \cdot 3600 + 125$ ve $125 = 2 \cdot 60 + 5$; yani 1 saat 2 dakika 5 saniye ✓. Programın son satırı bu kontrolü kendisi yapar: parçaları geri birleştirip $1 \cdot 3600 + 2 \cdot 60 + 5 = 3725$ elde eder ve True yazar. Küçük programlara böyle bir öz-kontrol satırı eklemek iyi bir alışkanlıktır. Ayrıca uç durumu düşünün: 3600 saniye girilirse 1 saat 0 dakika 0 saniye çıkmalıdır; 59 saniye girilirse 0 saat 0 dakika 59 saniye. İkisini de laboratuvarında deneyin.

 Sık yapılan hata

$\text{saat} = \text{toplam} / 3600 \rightarrow \text{saat} = \text{toplam} // 3600$
 / her zaman float verir: $3725/3600 = 1,0347\dots$ Saat sayısı “1” olmalı, “1.0347” değil. Tam sayı bölümü istiyorsanız // kullanın. Aynı biçimde $6 / 3$ sonucu 2.0’dır, 2 değil; bir sayının tam bölünüp bölünmediğini öğrenmenin doğru yolu % ile kalana bakmaktır: $6 \% 3 == 0$.



GitHub

4.7 Çıktıyı biçimlendirme: `print()` ve `f-string`

Programın çıktısı bir insan tarafından okunacaktır; 3.3333333333333335 değil 3.33 görmek ister. Python'da biçimli çıktının modern aracı **f-string**'dir: tırnaktan önce **f** harfi konur ve metnin içine küme parantezi ile değişkenler ya da ifadeler gömülür.

Kod 4.7 w04-07-fstring.py — `print()` seçenekleri, f-string biçim belirteçleri ve kaçış karakterleri

```
1 # w04-07-fstring.py -- print ayarları ve f-string ile biçimli çıktı
2
3 name = "Mehmet"
4 score = 87.4567
5 count = 1234567
6
7 # print: sep ayırıcıyı, end satır sonunu değiştirir
8 print("a", "b", "c")
9 print("a", "b", "c", sep="-")
10 print("Satır sonu yok", end=" | ")
11 print("aynı satırda devam")
12
13 # f-string: metnin içine {} ile değer gömme
14 print(f"Öğrenci {name}, puan {score}")
15 print(f"Puan (2 ondalık): {score:.2f}")
16 print(f"Puan (yüzde işareti gibi): {score / 100:.1%}")
17 print(f"Binlik ayırıcı: {count:,}")
18 print(f"Sağa dayalı 10 karakter: [{name:>10}]")
19 print(f"Sola dayalı 10 karakter: [{name:<10}]")
20 print(f"Ortalanmış 10 karakter : [{name:^10}]")
21 print(f"Sıfırla doldur: {42:05d}")
22 print(f"İfade de yazılabilir: {3 * 7 = }")
23
24 # Kaçış karakterleri
25 print("Tab\tile ayrılmış\tsütunlar")
26 print("Alt satıra\ngöçmek için \\n kullanılır")
27 print("Tırnak içinde \"tırnak\" yazmak")
28 print('Tek tırnaklı metinde "çift tırnak" serbesttir')
```

\$ python w04-07-fstring.py

```
a b c
a-b-c
Satır sonu yok | aynı satırda devam
Öğrenci Mehmet, puan 87.4567
Puan (2 ondalık): 87.46
Puan (yüzde işareti gibi): 87.5%
Binlik ayırıcı: 1,234,567
Sağa dayalı 10 karakter: [   Mehmet]
Sola dayalı 10 karakter: [Mehmet   ]
Ortalanmış 10 karakter : [  Mehmet  ]
Sıfırla doldur: 00042
İfade de yazılabilir: 3 * 7 = 21
Tab      ile ayrılmış      sütunlar
Alt satıra
geçmek için \n kullanılır
Tırnak içinde "tırnak" yazmak
Tek tırnaklı metinde "çift tırnak" serbesttir
```



GitHub

Tablo 4.4: Sık kullanılan f-string biçim belirteçleri. Yazım: {değer:belirteç}. Örneklerde $x = 87.4567$, $n = 1234567$, $s = "Ali"$.

Yazım	Anlamı	Sonuç
{x:.2f}	2 ondalık basamak	87.46
{x:.0f}	ondalıksız, yuvarlanmış	87
{x:8.1f}	toplam 8 karakter, 1 ondalık	87.5
{n:,}	binlik ayırıcı	1,234,567
{n:>10}	10 karaktere sağa dayalı	1234567
{s:<8}	8 karaktere sola dayalı	Ali
{s:^8}	8 karaktere ortalanmış	Ali
{n:08d}	8 hane, sıfırla doldurulmuş	01234567
{x/100:.1%}	yüzde olarak	87.5%
{x = }	ifadeyi ve değerini birlikte	x = 87.4567

Belirteçlerin genel biçimi [doldurma] [hizalama] [genişlik] [,] [.hassasiyet] [tür]'dür; hepsi isteğe bağlıdır. Tablo çıktıları üretirken genişlik ve hizalama birleştirilir:

```
1 print(f"{name:<10}{score:>8.1f}") # 10 karakter sola dayalı ad, 8 karakter
   ↪ sağa dayalı puan
```

Sütunlar böyle hizalanır (bkz. Alıştırma 4.10).

Tablo 4.5: Kaçış karakterleri (İng. *escape sequences*): ters bölü, ardındaki karaktere özel anlam verir.

Yazım	Anlamı	Örnek
\n	yeni satır	"a\nb" → iki satır
\t	sekme (tab)	"ad\tsoyad"
\\	ters bölünün kendisi	"C:\\Users"
\"	metin içinde çift tırnak	"\"alıntı\""
\'	metin içinde tek tırnak	'Ali\'nin'

İpucu

Bir metnin içinde çift tırnak geçecekse metni tek tırnakla açın ('Ali "geldi" dedi'), tek tırnak geçecekse çift tırnakla ("Ali'nin"). Kaçış karakterine böylece gerek kalmaz. Türkçe iyelik ekleri (Ayşe'nin, Trabzon'da) yüzünden Türkçe metinlerde çift tırnak çoğu zaman daha rahattır.

4.8 Bir programın anatomisi: Girdi – İşlem – Çıktı

Bu haftaki her program aynı üç bölümden oluşur ve bundan sonraki haftalarda yazacağınız programların da omurgası budur: veriyi **al**, üzerinde **işlem** yap, sonucu **göster**. İkinci haftada akış diyagramında paralelkenar–dikdörtgen–paralelkenar olarak çizdiğiniz yapı Python'da `input()`–ifadeler–`print()` olur. Aşağıdaki iki örnek bu iskeleti, çözüm kutusunun dört adımıyla (Analiz, Algoritma, Kod, Kontrol) birlikte gösterir.

Örnek 4.2 (Vücut kitle indeksi). Kütlesi (kg) ve boyu (m) verilen bir kişinin vücut kitle indeksini, $VKİ = \frac{\text{kütle}}{\text{boy}^2}$, iki ondalık basamakla yazan bir program yazın. Örnek: 70 kg, 1,75 m.

Çözüm

Analiz. İki girdi, ikisi de ondalıklı olabilir $\rightarrow$ float. Bir çıktı: VKİ, .2f biçiminde. İşlem tek formüldür; ** ile kare alınır.

Algoritma. OKU kütle, boy; vki $\leftarrow$ kütle / boy²; YAZ vki.

Kod ve çıktı.

Kod 4.8 w04-08-bmi.py — Girdi-işlem-çıktı iskeleti: VKİ

```
1 # w04-08-bmi.py -- Vücut kitle indeksi (VKİ): girdi al, hesapla,
   ↪ biçimli yazdır
2 # VKİ = kütle (kg) / boy (m) ** 2
3
4 weight = float(input("Kütleniz (kg): "))
5 height = float(input("Boyunuz (m): "))
6
7 bmi = weight / height ** 2
8
9 print(f"Kütle : {weight:.1f} kg")
10 print(f"Boy   : {height:.2f} m")
11 print(f"VKİ   : {bmi:.2f}")
```

\$ python w04-08-bmi.py

```
Kütleniz (kg): 70
Boyunuz (m): 1.75
Kütle : 70.0 kg
Boy   : 1.75 m
VKİ   : 22.86
```

Kontrol. $70/1,75^2 = 70/3,0625 = 22,857... \approx 22,86$ ✓. Sağlıklı aralığın 18,5–25 olduğunu bildiğimiz için sonuç makuldür; 2286 ya da 0,23 çıksaydı formülde bir yanlışlık (örneğin weight / height * 2 yazmak) aramamız gerekirdi. *Büyüklik mertebesi kontrolü* böyle yapılır.

Örnek 4.3 (Dairenin çevresi ve alanı). Yarıçapı santimetre olarak verilen dairenin çevresini ve alanını, birimleriyle, iki ondalık basamakla yazan bir program yazın. Örnek: $r = 3$ cm.

Çözüm

Analiz. Girdi: yarıçap (float). Çıktı: çevre $2\pi r$ (cm) ve alan πr^2 (cm²). π için math.pi kullanılır; 3.14 yazmak hem daha az doğru hem de okuyucuya “bu sayı nedir?” sorusu bıraktırır.

Kod ve çıktı.



GitHub

Kod 4.9 w04-09-daire.py — math.pi ile daire hesapları

```

1 # w04-09-daire.py -- Yarıçapı verilen dairenin çevresi ve alanı
2 import math
3
4 radius = float(input("Yarıçap (cm): "))
5
6 perimeter = 2 * math.pi * radius
7 area = math.pi * radius ** 2
8
9 print(f"Çevre: {perimeter:.2f} cm")
10 print(f"Alan : {area:.2f} cm²")

```

```
$ python w04-09-daire.py
```

```

Yarıçap (cm): 3
Çevre: 18.85 cm
Alan : 28.27 cm²

```

Kontrol. $2\pi \cdot 3 = 6\pi \approx 18,85$ ve $\pi \cdot 9 \approx 28,27$ ✓. Alanın çevreden büyük çıkması $r > 2$ için beklenir ($\pi r^2 > 2\pi r \Leftrightarrow r > 2$); $r = 1$ girilirse alan (3,14) çevreden (6,28) küçük çıkmalıdır — laboratuvarında deneyin.

İpucu

Yorum satırları `#` ile başlar ve Python tarafından yok sayılır; okuyan insan içindir. İyi bir yorum kodun *ne yaptığını* değil (kod zaten onu söyler) *neden yaptığını* ya da hangi formülü uyguladığını söyler: `# VKİ = kütle / boy**2` iyi bir yorumdur, `# x'e 1 ekle` gereksizdir. Her dosyanın başına bir satır yorumla dosyanın amacını yazmayı alışkanlık edinin; kitaptaki her kod dosyası böyle başlar.

4.9 Hata mesajlarını okumak

Programınız hata verecek; bu, öğrenmenin bir parçasıdır ve deneyimli programcılar için de günlük bir olaydır. Fark şudur: deneyimli programcı hata mesajını *okur*. Python'un hata raporuna *geri izleme* (İng. *traceback*) denir (Şekil 4.7).

```

Traceback (most recent call last):
  File "hesap.py", line 3, in <module>
    ortalama = toplam / adet
ZeroDivisionError: division by zero

```

- ① Hata, çağrı zinciriyle raporlanır
- ② Dosya adı ve satır numarası: önce buraya bakın
- ③ Hatanın olduğu satırın kendisi
- ④ Hata türü ve kısa açıklaması

Hata mesajını alttan yukarıya okuyun. En alt satır *ne* olduğunu, hemen üstü *nere* olduğunu söyler. Sık karşılaşılan türler: `SyntaxError` (yazım kuralı bozuk: eksik parantez, iki nokta), `NameError` (tanımlanmamış ad — çoğu kez yazım hatası), `TypeError` (uyumsuz türler: `"3" + 4`), `ValueError` (tür doğru, değer uygunsuz: `int("abc")`), `ZeroDivisionError` (sıfıra bölme). İngilizce mesajı olduğu gibi arama motoruna yazmak da meşru bir hata ayıklama yöntemidir.

Şekil 4.7: Bir geri izleme raporunun bölümleri. Altan yukarıya okuyun: son satır hatanın *türünü*, bir üstü hatalı *satır*, onun üstü *dosya ve satır numarasını* verir.

Bu haftaki programlarda karşılaşacağınız beş hata türü aşağıdadır. Her birini bilerek üretip mesajını okuyun; laboratuvarındaki en değerli on dakika bu olabilir.

```
1 print("Merhaba")          # parantez kapatılmadı
```

Çıktı

```
File "hata1.py", line 1
  print("Merhaba")          # parantez kapatılmadı
  ^
SyntaxError: '(' was never closed
```

SyntaxError: yazım kuralı bozuk. Program *hiç* çalışmaz; Python dosyayı okurken durur. Ok işareti sorunun yerini gösterir, ama gerçek eksik çoğu zaman okun *öncesindedir*.

```
1 total = 10
2 print(totl)                # yazım hatası: totl
```

Çıktı

```
Traceback (most recent call last):
  File "hata2.py", line 2, in <module>
    print(totl)
NameError: name 'totl' is not defined. Did you mean: 'total'?
```

NameError: tanımlanmamış bir ad kullanıldı. Nedeni hemen her zaman yazım hatası ya da büyük/küçük harf karışıklığıdır; Python 3.10'dan itibaren “şunu mu demek istediniz?” önerisi de gelir.

```
1 age = input("Yaş: ")
2 print(age + 1)             # str + int
```

Çıktı

```
Yaş: 19
Traceback (most recent call last):
  File "hata3.py", line 2, in <module>
    print(age + 1)
TypeError: can only concatenate str (not "int") to str
```

TypeError: işlem, verilen türlerle yapılamıyor. Çözüm tür dönüşümüdür: `int(age) + 1` ya da, metin isteniyorsa, `age + "1"`.

```
1 n = int("on iki")          # sayıya benzemeyen metin
```

Çıktı

```
Traceback (most recent call last):
  File "hata4.py", line 1, in <module>
    n = int("on iki")
ValueError: invalid literal for int() with base 10: 'on iki'
```

ValueError: tür doğru (bir metin verildi, `int()` metin kabul eder) ama *değer* uygunsuz. `int("3.5")` de aynı hatayı verir.

```
1 average = total / count    # count sıfır ise
```

Çıktı

ZeroDivisionError: division by zero

ZeroDivisionError: sifıra bölme. Kullanıcı sıfır girdiğinde ortaya çıkar; 6. haftada koşullu ifadelerle önleyeceğiz.

Hata ayıklama: dört adım

1. Geri izlemenin **son satırını** okuyun: hata türü ve mesaj.
2. Bir üstteki **satır numarasına** gidin; hatalı satır o ya da hemen öncesidir.
3. Satırdaki değişkenlerin **türlerini ve değerlerini** sorgulayın: hemen önüne `print(type(x), x)` ekleyin ya da kabukta deneyin.
4. Düzeltin, **yeniden çalıştırın**, aynı hatanın gittiğini ve yeni bir hatanın gelmediğini doğrulayın. Bir seferde tek şey değiştirin.

LAB Laboratuvar Çalışması 4: Kurulum ve ilk programlar

Bu haftanın laboratuvar bölümü iki parçadır: önce ortamı kurup çalıştırdığınızdan emin olacak, sonra girdi-işlem-çıkı yapılarında dört küçük program yazacaksınız. Her görevin beklenen çıktısı verilmiştir; kendi çıktınızla *karakter karakter* karşılaştırın.

Görev 1. Kurulum. Laboratuvar bilgisayarında Anaconda kurulumudur; kendi bilgisayarınıza Ek B'deki adımlarla Anaconda'yı (ya da python.org'dan Python 3'ü) kurun. Kurulumun başarılı olduğunu doğrulamak için bir terminal/komut istemi açıp `$ python --version` yazın; Python 3.1x.y biçiminde bir satır görmelisiniz. Ardından IDLE'i açıp kabukta `2 ** 10` yazın: 1024.

Görev 2. Colab. <https://colab.research.google.com> adresine Google hesabınızla girin, Yeni not defteri oluşturun, ilk hücreye `print("Merhaba Colab")` yazıp `[Shift]+[Enter]` ile çalıştırın. Not defterini YPZ105_hafta04 adıyla kaydedin; Drive'mızda görünmelidir.

Görev 3. Kendini tanıtan program. `tanitim.py`: adınızı, bölümünüzü ve doğum yılınızı *değişkenlere* atayın; `2026 - dogum_yili` ile yaşınızı hesaplayın; sonucu f-string ile tek satırda yazın. Beklenen çıktı biçimi: Ben Ayşe Yılmaz, Yapay Zeka Mühendisliği öğrencisiyim ve 19 yaşımdayım.

Görev 4. Dikdörtgen. `dikdortgen.py`: kullanıcıdan iki kenar uzunluğu alın (metre, ondalıklı olabilir); alanı ve çevreyi iki ondalık basamakla, birimiyle yazın. Girdi 3.5 ve 2: Alan: 7.00 m² ve Çevre: 11.00 m. Kontrol: kenarları 1 ve 1 girince alan 1.00, çevre 4.00 olmalı.

Görev 5. Para üstü (basit). `para_ustu.py`: ürün fiyatını ve ödenen tutarı (float) alın; para üstünü hesaplayın ve Para üstü: 12.75 TL biçiminde yazın. Sonra para üstünü *kuruş* olarak tam sayıya çevirin (`kurus = round(para_ustu * 100)`) ve kaç adet 1 TL, kaç adet 25 kuruş verildiğini `//` ve `%` ile bulun. Girdi 37.25 ve 50: 12 adet 1 TL, 3 adet 25 kuruş.

Görev 6. Not ortalaması. `ortalama.py`: üç sınav notu alın (ondalıklı); aritmetik ortalamayı *bir* ondalık basamakla yazın. Girdi 70, 85, 92: Ortalama: 82.3. Ek: ortalamanın türünü `type()` ile yazdırın ve neden float olduğunu bir yorum satırında açıklayın. Sonra üç notu da 100 girip sonucun 100.0 olduğunu doğrulayın.

Teslim. Görev 3-6 dosyalarını bugün ders bitmeden çalıştırıp gösterin; dosyaları kendinizde saklayın — 5. haftada Görev 3'ü karakter dizisi işlemleriyle genişleteceğiz.

Özet

Yapı	Yazım	Not
Ekran yazma	<code>print(a, b, sep=" ", end="\n")</code>	birden çok değer; ayırıcı ve satır sonu değiştirilebilir
Kullanıcıdan okuma	<code>s = input("İstem: ")</code>	her zaman str döndürür
Atama	<code>ad = ifade</code>	sağ hesaplanır, sola bağlanır
Çoklu atama	<code>a, b = 3, 8</code> <code>a, b = b, a</code>	takas tek satırda
Türler	<code>int float str bool</code>	<code>type(x)</code> ile sorgulanır
Dönüşüm	<code>int(s) float(s) str(n) bool(x)</code>	<code>int(3.9) → 3</code> (keser); <code>int("3.5")</code> hata
Aritmetik	<code>+ - * / // % **</code>	/ daima float; // aşağı yuvarlar
Öncelik	<code>() > ** > * / // % > + -</code>	aynı seviye soldan sağa; ** sağdan sola
Kısaltılmış atama	<code>x += 5</code>	<code>-- == /= //= %= **=</code>
f-string	<code>f"{x:.2f} {n:}, {s:>10}"</code>	ondalık, binlik, hizalama
Modül	<code>import math</code> → <code>math.sqrt(2)</code> , <code>math.pi</code>	<code>floor</code> , <code>ceil</code> de var
Yorum	<code># açıklama</code>	<i>neden'i yazın, ne'yi değil</i>
Hata türleri	<code>SyntaxError</code> <code>NameError</code> <code>TypeError</code> <code>ValueError</code> <code>ZeroDivisionError</code>	geri izlemeyi alttan yukarı okuyun



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w04>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırmalar

Alıştırma 4.1. ●●● Aşağıdaki adlardan hangileri geçerli bir Python değişken adıdır? Geçersiz olanların nedenini yazın: `2sayi`, `sayi2`, `toplam_puan`, `toplam-puan`, `class`, `Class`, `_gizli`, `ad soyad`, `print`.

Cevap: geçerli: sayi2, toplam_puan, Class, _gizli, print (yerleşik adı gölgelediği için kötü); geçersiz: 2sayi (rakamla başlar), toplam-puan (tire), class (anahtar sözcük), ad soyad (boşluk)

Alıştırma 4.2. ●●● Aşağıdaki ifadelerin her birinin `type()` sonucunu çalıştırmadan yazın, sonra kabukta doğrulayın: `5`, `5.0`, `"5"`, `5 > 3`, `7 / 7`, `7 // 7`, `"7" * 2`, `int("7") * 2`.

Cevap: `int, float, str, bool, float, int, str` (değeri "77"), `int` (değeri 14)

Alıştırma 4.3. ●● Celsius cinsinden sıcaklığı alıp Fahrenheit'a çeviren bir program yazın: $F = C \cdot \frac{9}{5} + 32$. Çıktıyı bir ondalık basamakla ve birimleriyle verin. Girdi 37 için beklenen çıktı: $37.0 \text{ }^{\circ}\text{C} = 98.6 \text{ }^{\circ}\text{F}$.

Cevap: $37 \cdot 9/5 + 32 = 98,6$; `print(f"{c:.1f} °C = {f:.1f} °F")`

Alıştırma 4.4. ●● Aşağıdaki ifadelerin değerlerini *elle*, işlem önceliğini uygulayarak bulun; sonra kabukta doğrulayın. Farklı çıkanlar için nedenini yazın.

- | | |
|-----------------------------|------------------|
| (a) $2 + 3 * 4$ | (e) $-17 // 5$ |
| (b) $(2 + 3) * 4$ | (f) $10 / 4$ |
| (c) $2 ** 3 ** 2$ | (g) $10 // 4$ |
| (d) $17 // 5 * 5 + 17 \% 5$ | (h) $2 * 3 ** 2$ |

Cevap: (a) 14 (b) 20 (c) 512 (d) 17 (e) -4 (f) 2.5 (g) 2 (h) 18

Alıştırma 4.5. ●● İki basamaklı bir sayı alan, rakamlarının toplamını ve sayının tersini (rakamları yer değiştirmiş hâlini) yazan bir program yazın. Girdi 47: toplam 11, tersi 74. İpucu: onlar basamağı $n // 10$, birler basamağı $n \% 10$.

Cevap: `onlar = n // 10, birler = n % 10; toplam onlar + birler, ters birler * 10 + onlar`

Alıştırma 4.6. ●● Bir ürünün birim fiyatını ve adedini alan; ara toplamı, %20 KDV'yi ve genel toplamı üç satırda, sağa dayalı ve iki ondalık basamakla yazan bir market fişi programı yazın. KDV oranını büyük harfli bir sabitte tutun. Girdi 49.90 ve 3: ara toplam 149.70, KDV 29.94, genel toplam 179.64.

Cevap: `KDV_RATE = 0.20; ara_toplam 49,90 * 3 = 149,70; KDV 29,94; toplam 179,64; biçim f"{'Ara toplam':<12}{ara:>10.2f} TL"`

Alıştırma 4.7. ●● Örnek 4.1'i genişletin: toplam saniyeyi gün, saat, dakika ve saniyeye ayırın. Girdi 100000: 1 gün 3 saat 46 dakika 40 saniye. Programın sonuna parçaları geri birleştiren bir kontrol satırı ekleyin.

Cevap: $100000 = 1 \cdot 86400 + 13600$; $13600 = 3 \cdot 3600 + 2800$; $2800 = 46 \cdot 60 + 40$

Alıştırma 4.8. ●● Kenarları verilen dikdörtgenin alanını, çevresini ve köşegen uzunluğunu ($\sqrt{a^2 + b^2}$, `math.sqrt`) iki ondalık basamakla yazan bir program yazın. Girdi 3 ve 4: alan 12, çevre 14, köşegen 5.

Cevap: 12.00, 14.00, 5.00; `math.sqrt(a**2 + b**2)` ya da `math.hypot(a, b)`

Alıştırma 4.9. ●●● $0.1 + 0.2 == 0.3$ ifadesi neden False verir? İki-üç cümleyle açıklayın ve iki float değerini karşılaştırmanın doğru yolunu bir satır Python koduyla gösterin.

Cevap: *float* sayıları ikili tabanda sonlu basamakla tutar; 0,1 ve 0,2 ikili tabanda sonsuz kesirdir, yuvarlanır ve toplam 0,3'ün yuvarlanmış hâliyle çakışmaz. Doğru yol: `abs(a - b) < 1e-9` ya da `math.isclose(a, b)`

Alıştırma 4.10. ●●● Öğrencinin adını, vize ve final notunu alan; ağırlıklı ortalamayı (vize %40, final %60) hesaplayan ve sonucu *başlıklı, hizalı bir tablo satırı* olarak yazan bir program yazın. Ağırlıkları sabit olarak tanımlayın. Sütunlar: ad 10 karakter sola dayalı; vize ve final 8 karakter sağa dayalı, bir ondalık; ortalama 10 karakter sağa dayalı, iki ondalık. Başlığın altına 36 tire çizin ("-" * 36). Girdi Ayşe, 72,5, 81: ortalama 77,60.

Cevap: $72,5 \cdot 0,4 + 81 \cdot 0,6 = 29 + 48,6 = 77,6$; *belirteçler* :<10, :>8.1f, :>8.1f, :>10.2f

Alıştırma 4.11. ●●● Ödenen tutarı ve ürün fiyatını alan; para üstünü *en az sayıda* 50 TL, 10 TL, 2 TL, 50 kuruş ve 25 kuruşla nasıl vereceğinizi yazan bir program yazın. Hesabı float ile değil, para üstünü kuruşa çevirip (`round((odenen - fiyat) * 100)`) *tam sayı* ile yapın; nedenini bir yorum satırında açıklayın. Girdi 100 ve 37,25: para üstü 62,75 TL = birer adet 50 TL, 10 TL, 2 TL, 50 kuruş ve 25 kuruş.

Cevap: 6275 kuruş; $6275 / 5000 = 1$, kalan 1275; $1275 / 1000 = 1$, kalan 275; $275 / 200 = 1$, kalan 75; $75 / 50 = 1$, kalan 25; $25 / 25 = 1$, kalan 0

Hafta 5: Karakter Dizileri: İndeks, Dilim ve Metotlar

Öğrenme çıktıları

Bu haftanın sonunda:

1. karakter dizilerini tek, çift ve üçlü tırnakla yazabilecek, kaçış karakterlerini ve ham metni (`r"..."`) doğru yerde kullanabileceksiniz;
2. bir metnin herhangi bir karakterine pozitif ya da negatif indeksle erişebilecek, `IndexError`'ın ne zaman oluştuğunu söyleyebileceksiniz;
3. dilimleme (`[a:b:c]`) ile bir metnin istediğiniz parçasını alabilecek, metni ters çevirebilecek; karakter dizisinin *değiştirilemez* olduğunu açıklayabileceksiniz;
4. arama, değiştirme, sınama ve hizalama metotlarını (`find`, `replace`, `isdigit`, `split`, `join`, `ljust` ...) tek tek ve zincirleyerek kullanabileceksiniz;
5. metinlerin Unicode kod noktalarına göre karşılaştırıldığını bilecek, Türkçe harflerin yarattığı sorunu tanıyıp önleyebileceksiniz;
6. ad-soyad ayırma, e-posta ve numara biçim denetimi gibi gerçek metin problemlerini döngü kullanmadan çözebileceksiniz.

Anahtar kavramlar

karakter dizisi (İng. *string*) **alt metin** (İng. *substring*) **indeks** (İng. *index*) **dilim** (İng. *slice*) **değiştirilemezlik** (İng. *immutability*) **metot** (İng. *method*) **ham metin** (İng. *raw string*) **ayırıcı** (İng. *separator*) **kod noktası** (İng. *code point*) **Unicode** **sözlük sırası** (İng. *lexicographic order*)

Geçen hafta dört veri türü tanıdık ve üçüyle — `int`, `float`, `bool` — hesap yaptık. Dördüncüsü, `str`, yalnız “ekrana yazdırılan şey” olarak göründü. Oysa bir yapay zeka mühendisinin karşılaştığı verinin büyük kısmı sayı değil *metindir*: müşteri yorumları, arama sorguları, sensör günlükleri, öğrenci kayıtları. Bu verinin modele girmeden önce temizlenmesi, parçalanması, biçimlendirilmesi gerekir; bu işin adı *metin işlemedir*.

Konunun güzel yanı, araçların tek tek çok basit olmasıdır: uzunluğu veren `len`, arayan `find`, parçalara ayıran `split`. Zor olan, bir problemi bu basit araçların birleşimi olarak *görebilmektir*. Bu hafta döngü ve koşul kullanmayacağız — ikisini de henüz görmedik; buna rağmen çözebileceğimiz problemlerin sayısına şaşıracaksınız.

5.1 Karakter dizisi nedir, nasıl yazılır

Tanım 5.1: Karakter dizisi

Karakter dizisi (İng. *string*), sıralı bir karakter topluluğudur; türü `str`'dir. Sıra önemlidir ("`abc`" ile "`cba`" farklı metinlerdir) ve bir metin sıfır karakterden ("`"`", **boş metin**) de oluşabilir. Sınırlarını **tırnaklar** belirler; tırnaklar metnin *parçası değildir*.

Python'da bir metni üç biçimde yazabilirsiniz:

- **Tek tırnak:** `'Trabzon'`
- **Çift tırnak:** `"Trabzon"` — ikisi tamamen aynıdır, yeter ki açığınızı kapatın.
- **Üç tırnak:** `"""..."""` ya da `'''...'''` — içindeki satır sonları korunur, yani **çok satırlı metin** yazmayı sağlar.

İki tırnak biçiminin olmasının pratik bir nedeni vardır: metnin içinde tırnak geçebilir. İçinde tek tırnak olan bir metni çift tırnakla, çift tırnak olanı tek tırnakla açarsanız hiçbir numaraya gerek kalmaz. Başka bir yol da 4. haftada gördüğümüz **kaçış karakteri** (İng.

escape sequence) kullanmaktır: \ ' ve \ " tırnağın kendisini, \n yeni satırı, \t sekme, \\ ters bölünün kendisini yazar. Ters bölünün bu özel anlamı Windows dosya yollarında baş ağrıtır ("C:\Users" yazarsanız \U beklenmedik bir kaçış dizisi olur); çözüm ya ters bölüyü ikilemek ya da metni **ham metin** (*Ing. raw string*) olarak açmaktır: tırnaktan önce bir r harfi koyarsanız ters bölü hiçbir özel anlam taşımaz.

Kod 5.1 w05-01-metin-temelleri.py — Tırnaklar, çok satırlı metin, kaçış karakterleri ve ham metin

```
1 # w05-01-metin-temelleri.py -- Tırnaklar, çok satırlı metin, kaçış
2 # karakterleri, ham metin; len(), + ve * operatörleri, in / not in
3
4 city = "Trabzon" # çift tırnak
5 course = 'Programlama' # tek tırnak: aynı şey
6 quote = "Ali 'geldi' dedi" # tek tırnak varsa çift ile aç
7 print(city, course, "|", quote)
8
9 # Çok satırlı metin: üç tırnak arasındaki satır sonları korunur
10 address = """Trabzon Üniversitesi
11 Yapay Zeka Mühendisliği"""
12 print(address)
13
14 # Kaçış karakterleri (\t sekme, \n yeni satır) ve ham metin
15 print("Ad\tSoyad\nAli\tKaya")
16 print("C:\\Users\\ayse", r"C:\Users\ayse") # aynı sonuç
17
18 # len(): karakter sayısı (boşluk ve noktalama dahil)
19 sentence = "Yapay zeka mühendisliği"
20 print("Karakter sayısı:", len(sentence), "| boş metin:", len(""))
21
22 # + birleştirir, * yineler
23 full_name = "Ayşe" + " " + "Yılmaz"
24 print(full_name, len(full_name), "-" * 12)
25
26 # in / not in: alt metin var mı? Sonuç True ya da False
27 print("zeka" in sentence, "Zeka" in sentence, "x" not in sentence)
```

\$ python w05-01-metin-temelleri.py

```
Trabzon Programlama | Ali 'geldi' dedi
Trabzon Üniversitesi
Yapay Zeka Mühendisliği
Ad      Soyad
Ali     Kaya
C:\Users\ayse C:\Users\ayse
Karakter sayısı: 23 | boş metin: 0
Ayşe Yılmaz 11 -----
True False True
```

İpucu

Ham metin yalnız dosya yollarında değil, ters bölünün sık geçtiği her yerde işe yarar: düzenli ifadeler (14. haftada anılacak), L^AT_EX komutları, Windows kayıt defteri yolları. Kural basittir: metinde ters bölü *görünmesini* istiyorsanız ham metin kullanın. Ham metnin tek kısıtı, ters bölü ile *bitememesidir*: r"C:\\" geçersizdir.



GitHub

5.1.1 Uzunluk, birleştirme, yinleme ve üyelik

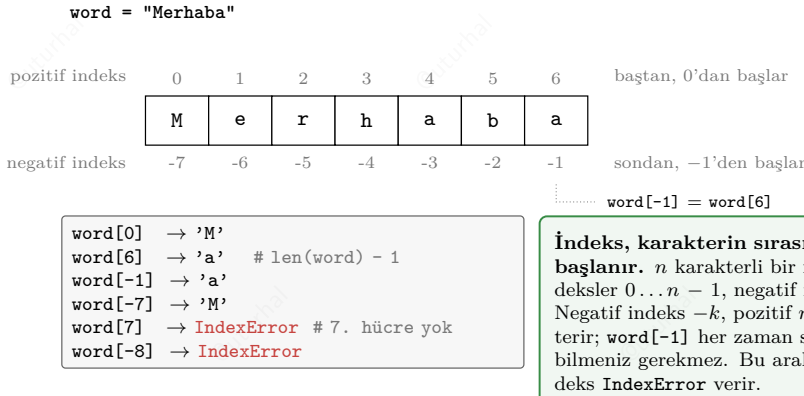
Kod 5.1'in ikinci yarısı, bir metne uygulanabilen dört temel işlemi gösterir; bunlar metot değil, yerleşik işlev ve operatörlerdir. `len(s)` karakter sayısını verir ve boşlukları, noktalama işaretlerini de sayar: `len("Yapay zeka mühendisliği")` sonucu 23'tür. `s + t` iki metni birleştirir; `s * n` bir metni n kez tekrarlar — ayrıca çizgisi çizmenin (`"-" * 30`) ve bir şifreyi maskeleyenin (`"*" * len(password)`) en kısa yolu budur, n tam sayı olmak koşuluyla. `alt in s` ise bir **alt metnin** (İng. *substring*) metinde geçip geçmediğini sorar ve `True/False` döndürür; büyük/küçük harf farkına duyarlıdır, `"Zeka" in "Yapay zeka"` yanlıştır.

5.2 İndeksleme: tek bir karaktere erişmek

Bir metin sıralı olduğuna göre, her karakterinin bir *sırası* vardır. Köşeli parantezle o sıradaki karaktere erişiriz. Python'da — ve neredeyse bütün programlama dillerinde — saymaya **sıfırdan** başlanır: ilk karakter `s[0]`, ikincisi `s[1]`'dir.

Tanım 5.2: İndeks

İndeks (İng. *index*), bir karakterin metin içindeki sırasıdır. n karakterli bir metinde geçerli indeksler **pozitif** yönde $0, 1, \dots, n - 1$ ve **negatif** yönde $-n, \dots, -2, -1$ 'dir. Negatif indeks sondan sayar: `s[-1]` son karakter, `s[-2]` sondan ikincidir. Bu aralığın dışındaki bir indeks `IndexError` verir.



Şekil 5.1: "Merhaba" metninin indeks cetveli. Üstteki sayılar baştan, alttakiler sondan sayar; ikisi aynı hücreyi gösterir. Yedi karakterli bir metinde `word[7]` diye bir hücre yoktur.

Şekil 5.1'de dikkat edilecek nokta, negatif indeksin bir "kısayol" olmasıdır: son karakteri almak için `s[len(s) - 1]` yazabilirsiniz, ama `s[-1]` hem kısa hem de uzunluğu bilmeyi gerektirmez — metnin uzunluğu her çalışmada değişeceği için bu büyük kolaylıktır.

⚠ Sık yapılan hata

`s[len(s)]` → `s[len(s) - 1]`

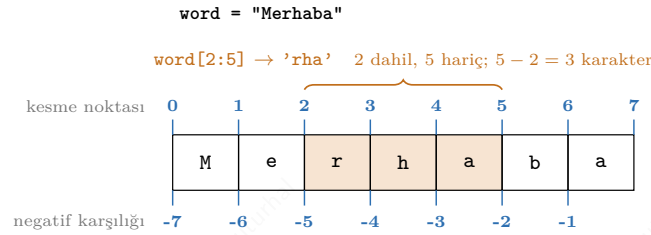
"Merhaba" yedi karakterlidir ama son karakterin indeksi 6'dır. Bir aralığın sonunu bir fazla saymak (*off-by-one* hatası) programcılığın en eski hatasıdır; bu hafta `IndexError: string index out of range` biçiminde karşınıza çıkacak. En kolay korunma yolu son-
dan erişirken negatif indeks kullanmaktır.

5.3 Dilimleme: bir parçayı almak

Tek bir karakter yerine bir *parça* istiyorsak **dilimleme** (İng. *slicing*) kullanırız.

Tanım 5.3: Dilim

$s[a:b]$ yazımı, a indeksinden başlayıp b indeksine kadar — a **dahil**, b **hariç** — olan karakterlerden oluşan *yeni* bir metin verir; uzunluğu $b - a$ 'dır. a yazılmazsa “baştan”, b yazılmazsa “sona kadar” anlaşılır. Üçüncü sayı **adımdır**: $s[a:b:c]$ her c . karakteri alır; c negatifse metin sondan başa doğru okunur, bu yüzden $s[::-1]$ metnin tersidir.



word[:3]	→ 'Mer'	# baştan 3'e
word[3:]	→ 'haba'	# 3'ten sona
word[-4:]	→ 'haba'	# son 4
word[:-1]	→ 'Merhab'	# sonuncu hariç
word[:2]	→ 'Mraa'	# adım 2
word[::-1]	→ 'abahreM'	# tersi
word[3:100]	→ 'haba'	# hata yok!

Dilim numaraları hücreleri değil, hücreler arasındaki kesme noktalarını gösterir. word[a:b], a ile b kesmeleri arasında kalan karakterlerdir: a dahil, b hariç (yarı açık aralık); uzunluğu $b - a$ 'dır. Boş bırakılan a “baştan”, boş bırakılan b “sona kadar” demektir. Bu yüzden word[:k] + word[k:] her k için metnin kendisini verir. Üçüncü sayı adımdır; -1 adım metni tersten okur.

Şekil 5.2: Dilim sayıları karakterleri değil, karakterler arasındaki kesme noktalarını gösterir. Bu bakış açısı “b neden hariç?” sorusunu kendiliğinden yanıtlar.

Yarı açık aralık (a dahil, b hariç) ilk bakışta keyfi görünür, ama iki güzel sonucu vardır. Birincisi, dilimin uzunluğu her zaman $b - a$ 'dır — çıkarma yeter, “bir ekle/bir çıkar” gerekmez. İkincisi, $s[:k] + s[k:]$ her k için metnin kendisini verir; metni bir noktadan ikiye bölmek tek satırdır.

Kod 5.2 w05-02-indeks-dilim.py — İndeksleme, dilimleme, ters çevirme ve maskeleme

```
1 # w05-02-indeks-dilim.py -- İndeksleme, dilimleme, ters çevirme
2
3 word = "Merhaba"
4 print(word[0], word[1], word[6])      # pozitif indeks: 0'dan
5 print(word[-1], word[-2], word[-7])  # negatif indeks: -1'den
6
7 # Dilimleme: word[a:b] → a dahil, b hariç
8 print(word[0:3], word[3:7])           # Mer haba
9 print(word[:3], word[3:])             # baştan / sona kadar
10 print(word[-4:], word[:-1])          # son dört / sonuncu hariç
11 print(word[:2], word[1:2], word[::-1]) # adımlı ve ters dilim
12
13 # Sınır dışı DİLİM hata vermez; kısa ya da boş metin döner
14 print(word[3:100], "[" + word[10:20] + "]")
15
16 # Uygulama: son dört hane dışında maskeleme
17 phone = "05321234567"
18 print("Telefon:", "*" * (len(phone) - 4) + phone[-4:])
19 id_no = "12345678901"
```



GitHub


```
20 print("Kimlik :", id_no[:2] + "*" * 7 + id_no[-2:])
```

```
$ python w05-02-indeks-dilim.py
```

```
M e a
a b M
Mer haba
Mer haba
haba Merhab
Mraa ehb abahreM
haba []
Telefon: *****4567
Kimlik : 12*****01
```

Dikkat

İndeks ile dilim sınır dışına çıktığında *farklı* davranır. `word[10]` hemen `IndexError` verir; oysa `word[3:100]` hata vermez, elde ne varsa onu döndürür ('haba'), `word[10:20]` ise boş metin verir. Bu “bağışlayıcılık” çoğu zaman işinize yarar (son dört haneyi almak isterken metin üç haneliyse program çökmez) ama sessizce boş sonuç üretebileceği için kontrolü size bırakır.

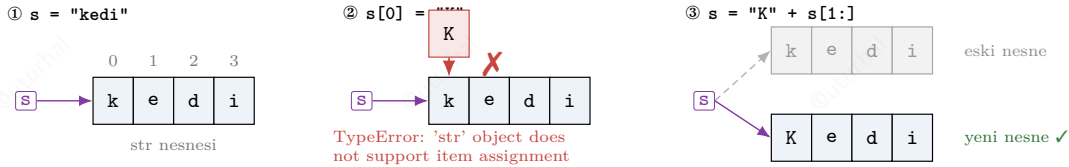
İpucu

Dört kalıbı anlamlarıyla öğrenin, ezberlemeyin: `s[:n]` “ilk n karakter”, `s[-n:]` “son n karakter”, `s[:-n]` “son n karakter hariç hepsi”, `s[::-1]` “tersi”. Telefon numarasının son dördünü göstermek, uzantıyı kesmek, bir numarayı gizlemek — hepsi bu dört kalıptan biridir.

5.4 Değiştirilemezlik: metin yerinde değişmez

Şimdi bu haftanın en önemli, en çok şaşırtan kuralına geliyoruz.

Kural 5.1 (Karakter dizisi değiştirilemez). Bir `str` nesnesi oluşturulduktan sonra içeriği *değiştirilemez* (İng. *immutable*): hiçbir karakteri yerinde değiştirilemez, silinemez, araya karakter eklenemez. `s[0] = "K"` yazmak `TypeError: 'str' object does not support item assignment` hatası verir. Bir metni “değiştirmek”, her zaman **yeni bir metin üretip** sonucu bir ada atamaktır.



Karakter dizisi değiştirilemez bir nesnedir: oluşturulduktan sonra içindeki hiçbir karakter yerinde değiştirilemez, silinemez, eklenemez (②). Bir metni “değiştirmek” her zaman *yeni bir metin üretmek* demektir (③): dilimleme, + ve bütün metod çağrıları (`s.upper()`, `s.replace(...)`) orijinale dokunmaz, yeni bir nesne döndürür. Sonucu saklamak istiyorsanız bir ada atamalısınız; en yaygın yol aynı adı yeniden kullanmaktır: `s = s.upper()`.

Şekil 5.3: Metin değiştirilemez. Soldaki nesne `s[0] = "K"` ile değiştirilemez; `s = "K" + s[1:]` ise yeni bir nesne üretir ve etiketi ona taşır.

Bu kural 4. haftadaki *etiket* modelinin doğrudan sonucudur (Şekil 5.3): değişken bir kutu

değil, bir nesneye takılı etikettir; `s = s.upper()` satırı eski nesneyi büyütmez, büyük harfli *yeni* bir nesne üretip etiketi oraya taşır.

✗ Sık yapılan hata

`name.upper()` → `name = name.upper()`

Bu haftanın en sık hatasıdır. Bütün metin metotları sonucu *döndürür*, metni *değiştirmez*. Tek başına yazılan `name.upper()` satırı hesaplanır, sonuç üretilir ve hemen çöpe atılır; `name` eskisi gibi kalır. Sonucu ya bir ada atayın, ya doğrudan yazdırın (`print(name.upper())`), ya da başka bir ifadenin içinde kullanın.

Not

Değiştirilemezlik bir eksiklik değil, bilinçli bir tasarım kararıdır: değiştirilemeyen nesneler güvenle paylaşılabilir ve sözlüklerde anahtar olabilir (14. hafta). Buna karşılık 13. haftada göreceğimiz *liste değiştirilebilir* (İng. *mutable*) bir yapıdır: `L[0] = 5` orada geçerlidir.

5.5 Metotlar: metnin kendi işlevleri

`len()` bir işlevdir ve metni parantez içine alır. `upper()` ise bir **metottur** (İng. *method*): metnin *kendi üzerinde* çağrılır ve yazımı `metin.metot(argümanlar)` biçimindedir. Aradaki fark şimdilik yalnız yazımdadır.

```
masked = comment.replace("berbat", "*****")
```

bu ifade yeni bir metin döndürür; comment değişmez

- ① nesne: üzerinde işlem yapılacak metin
- ② metot adı: noktadan sonra gelir
- ③ argümanlar: parantez, boş olsa bile zorunludur
- ④ sonuç: bir ada atanmazsa kaybolur

Metot, bir metnin “kendi üzerinde” çağrılan işlevidir. Yazımı her zaman `metin.metot(argümanlar)` biçimindedir: nokta “şunun metodu” diye okunur, parantez ise çağrıyı yapar — `s.upper` parantezsiz yazılırsa metot *çalışmaz*. Metotlar metni *değiştirmez*, yeni bir metin döndürür; bu yüzden sonucu kullanmak için ya bir ada atarsınız ya da doğrudan yazdırırsınız. Zincirlemede (sağda) çağrılar **soldan sağa** işlenir: her metot bir öncekinin *sonucuna* uygulanır.

Zincirleme: `" aYŞE kAYA ".strip().lower().title()`

```
" aYŞE kAYA "
.strip()
"aYŞE kAYA"
.lower()
"ayşe kaya"
.title()
"Ayşe Kaya"
```

Şekil 5.4: Bir metot çağrısının bölümleri ve metot zincirleme. Nokta “şunun metodu” diye okunur; parantez, argüman almasa bile zorunludur.

✗ Sık yapılan hata

`print(name.upper)` → `print(name.upper())`

Parantezi unutursanız Python metodu *çağırmas*, metodun kendisini yazdırır: `<built-in method upper of str object at 0x7f...>`. Ekranda bu tuhaf satırı görürseniz eksik parantez arayın.

5.5.1 Harf büyüklüğü ve kenar temizleme

Sekiz metot bu bölümün alet çantasıdır; hiçbiri metni *değiştirmez*, hepsi yeni metin döndürür:

- `upper()` / `lower()`: `"Trabzon"` → `'TRABZON'` / `'trabzon'`;

- `title()` / `capitalize()`: "yapay zeka" → 'Yapay Zeka' (her sözcüğün ilk harfi) / 'Yapay zeka' (yalnız ilk harf);
- `swapcase()`: "Trabzon" → 'tRABZON';
- `strip()` / `lstrip()` / `rstrip()`: " ali " → 'ali' / 'ali ' / ' ali' — yalnız uçlardaki boşlukları atarlar, aradakilere dokunmazlar.

`strip` ailesine argüman verilirse argüman bir *karakter kümesidir*: `lstrip("w.")` soldaki bütün `w` ve `.` karakterlerini, hangi sırayla gelirlirse gelsinler atar — "w." dizisini değil.

Kullanıcı ya da dosya kaynaklı her metin için ilk refleksiniz `strip()` olsun: sonda gözle görülmeyen bir boşluk kaldığında `answer == "evet"` karşılaştırması sessizce yanlış sonuç verir. Metotlar birbirine **zincirlenebilir** (İng. *method chaining*): `raw.strip().lower()` önce boşlukları atar, sonra sonucu küçültür — zincir soldan sağa okunur, her halka bir öncekinin sonucuna uygulanır.

Kod 5.3 w05-03-donusturme.py — Harf büyüklüğü, kenar temizleme ve metot zincirleme

```
1 # w05-03-donusturme.py -- Harf büyüklüğü ve kenar boşluğu metotları,
2 # metot zincirleme
3
4 text = "programlama DERS notu"
5 print(text.upper())      # hepsi büyük
6 print(text.lower())      # hepsi küçük
7 print(text.title())      # her sözcüğün ilk harfi büyük
8 print(text.capitalize()) # yalnız ilk harf büyük, gerisi küçük
9 print(text.swapcase())   # büyük -> küçük, küçük -> büyük
10 print(text)             # metot metni DEĞİŞTİRMEZ: hâlâ orijinal
11
12 # Değişikliği saklamak için sonucu bir ada atayın
13 text = text.title()
14 print(text)
15
16 # strip / lstrip / rstrip: kenarlardaki boşlukları (ve \n) atar
17 raw = "  ayşe kaya  "
18 print "[" + raw.strip() + "]", "[" + raw.rstrip() + "]"
19
20 # Argüman verilirse o karakterleri kenarlardan atar
21 print("***Dikkat***".strip("*"), "www.trabzon.edu.tr".lstrip("w. "))
22
23 # Metot zincirleme: her metot bir öncekinin sonucuna uygulanır
24 print("  aYŞE kAYA ".strip().lower().title())
```

```
$ python w05-03-donusturme.py
```

```
PROGRAMLAMA DERS NOTU
programlama ders notu
Programlama Ders Notu
Programlama ders notu
PROGRAMLAMA ders NOTU
programlama DERS notu
Programlama Ders Notu
[ayşe kaya] [  ayşe kaya]
Dikkat trabzon.edu.tr
Ayşe Kaya
```



GitHub

5.5.2 Arama, sayma ve değiştirme

`s = "kolay kod"` üzerinde: `s.find("ko")` → 0 (ilk geçişin indeksi), `s.rfind("ko")` → 6 (son geçişin indeksi), `s.find("z")` → -1 (yoksa -1, hata yok), `s.index("kod")` → 6 (find gibi, ama bulamazsa `ValueError`), `s.count("o")` → 2 (çakışmayan geçiş sayısı), `s.startswith("kol")` ve `"rapor.pdf".endswith(".pdf")` → `True`, `s.replace("o", "0")` → `'k0lay k0d'` (tüm geçişler; üçüncü argüman sayıyı sınırlar: `"aaaa".replace("a", "b", 2)` → `'bbaa'`).

`find` ile `index` arasındaki fark önemlidir: `find` bulamazsa -1 döndürüp akışı sürdürür, `index` hata fırlatıp programı durdurur. Bu hafta koşul yazamadığımız için `find`'i tercih ediyoruz.



Sık yapılan hata

`s[s.find("z"):]` → önce `s.find("z")` sonucunun -1 olup olmadığına bakın

Aranan alt metin yoksa `find` -1 döndürür ve -1 geçerli bir indekstir — son karakter! Bu yüzden bulunamayan bir aramanın sonucu dilim içinde kullanılırsa program hata vermez, sessizce *yanlış* çalışır. Sessiz yanlış, gürültülü hatadan çok daha tehlikelidir.

`replace`, aradığı metnin bütün geçişlerini değiştirir ve — [Kural 5.1](#) gereği — yeni bir metin döndürür. Bir yorumdaki yasaklı sözcüğü aynı uzunlukta yıldızla örtmek bu yüzden tek satırdır: `comment.replace(bad_word, "*" * len(bad_word))`.

5.5.3 Sınama metotları

Bir metnin “sayıdan mı oluşuyor”, “hepsi harf mi” gibi sorularının cevabı `is` ile başlayan metotlardır. Hepsi `True/False` döndürür ve hepsi *bütün* metne bakar: tek bir uygunsuz karakter sonucu `False` yapar. Boş metin için hepsi `False`'tur.

- `isdigit()`: "2026" ✓, "20.26" ✗ (nokta rakam değil);
- `isalpha()`: "Ayşe" ✓, "Ayşe Kaya" ✗ (boşluk harf değil);
- `isalnum()`: "abc123" ✓, "abc-123" ✗;
- `isspace()`: " " ✓, "" ✗;
- `isupper()` / `islower()`: "TRABZON" ve "trabzon" ✓, "Trabzon" ikisi için de ✗.

`isdigit()` girdi doğrulamanın anahtarıdır: `int(input(...))` kullanıcı harf yazdığında `ValueError` ile çöker; önce `text.isdigit()` ile bakmak çökmeyi önler. Bu haftaki programlar sonucu yalnız *yazdırıyor*; 7. haftada aynı sonuca göre karar vereceğiz.

Kod 5.4 w05-04-arama-sinama.py — Arama, sayma, değiştirme ve sınama metotları

```
1 # w05-04-arama-sinama.py -- Arama (find, rfind, index, count),
2 # değiştirme (replace) ve sınama (is...) metotları
3
4 sentence = "Yapay zeka, veri ile öğrenen sistemler tasarlar."
5
6 # find: ilk konum, yoksa -1 (hata vermez). rfind: son konum
7 print(sentence.find("zeka"), sentence.find("robot"))
8 print(sentence.find("e"), sentence.rfind("e"))
9 print(sentence.index("veri"))          # index: bulamazsa ValueError
10 print(sentence.count("e"), "aaaa".count("aa")) # çakışmayan
11 print(sentence.startswith("Yapay"), "rapor.pdf".endswith(".pdf"))
12
13 # replace: TÜM geçişleri değiştirir, yeni metin döndürür
14 print(sentence.replace("e", "3", 2)) # en fazla 2 değişiklik
15
```



GitHub

```

16 # Uygulama: sansürleme
17 comment = "Bu film berbat, oyuncular berbat, müzik berbat."
18 bad_word = "berbat"
19 print(comment.replace(bad_word, "*" * len(bad_word)))
20
21 # Sinama metotları: metnin TAMAMI koşulu sağlıyorsa True
22 print("2026105017".isdigit(), "3.5".isdigit(), "".isdigit())
23 print("Ayşe".isalpha(), "Ayşe Kaya".isalpha(), "abc123".isalnum())
24 print(" ".isspace(), "TRABZON".isupper(), "Trabzon".islower())

```

```
$ python w05-04-arama-sinama.py
```

```

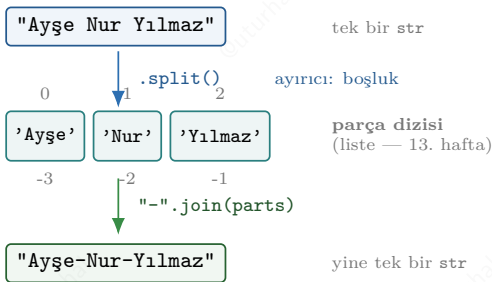
6 -1
7 36
12
7 2
True True
Yapay z3ka, v3ri ile öğrenen sistemler tasarlar.
Bu film *****, oyuncular *****, müzik *****.
True False False
True False True
True True False

```

5.5.4 split ve join: metinden parçalara, parçalardan metne

Tanım 5.4: split ve join

`s.split(ayırıcı)` metni verilen **ayırıcıdan** keserek parçalara böler. Ayırıcı verilmezse boşluklar kullanılır; art arda gelen boşluklar tek ayırıcı sayılır, baştaki ve sondakiler yok sayılır. `ayırıcı.join(parçalar)` bunun tersidir: parçaları aralarına ayırıcıyı koyarak tek metne çevirir.



```

parts = full_name.split()
len(parts)      → 3      # kaç sözcük
parts[0]        → 'Ayşe'  # ilk sözcük
parts[-1]       → 'Yılmaz' # son sözcük
"".join(parts)  → 'Ayşe Nur Yılmaz'

d, m, y = "13/09/2026".split("/")
# üç parça, üç ada tek adımda dağıtılır

```

✗ "a b".split(" ") → 3 parça (ortadaki boş)
✓ "a b".split() → 2 parça

split ile join birbirinin tersidir: biri metni ayırıcıdan keserek parçalara böler, öteki parçaları araya ayırıcıyı koyarak tek metne çevirir. `join`'in çağrıldığı nesne *ayırıcıdır*, parçalar değil — bu yüzden `"-".join(parts)` yazılır. Argümansız `split()` art arda gelen boşlukları tek ayırıcı sayar ve baştaki ile sondakileri yok sayar; bu yüzden kelime saymanın doğru yolu `len(s.split())`'tir. `split` bir *parça dizisi* (liste) döndürür; listeleri 13. haftada göreceğiz, bu hafta ondan yalnız `len()`, köşeli parantezle eleman seçme ve çoklu atama ile yararlanıyoruz.

Şekil 5.5: `split` metni parçalara böler, `join` parçaları birleştirir. `join`'in çağrıldığı nesne parçalar değil, *ayırıcıdır*.

`split`'in döndürdüğü şey aslında bir **listedir**; listeleri 13. haftada göreceğiz. Bu hafta ondan yalnız üç şekilde yararlanacağız: `len(parts)` kaç parça olduğunu söyler (kelime saymanın doğru yolu budur); `parts[0]` ve `parts[-1]` ilk ve son parçayı verir; parça sayısı kesin

biliniyorsa `day, month, year = date.split("/")` yazımı hepsini tek adımda adlandırır —
4. haftadaki `a, b = b, a` yazımının aynısı.

❌ Sık yapılan hata

`len(text.split(" "))` → `len(text.split())`

İki sözcüğün arasında iki boşluk varsa `split(" ")` aradaki boş parçayı da sayar ve kelime sayısını şişirir. Kod 5.5'in son satırları aynı cümle için 5 ve 8 sonuçlarını yan yana gösterir. Ayırıcısız `split()` bu sorunu tasarım gereği çözer; virgöl, noktalı virgöl gibi gerçek bir ayırıcı yoksa argüman vermeyin.

Kod 5.5 w05-05-split-join.py — split ile parçalama, join ile birleştirme, kelime sayma

```
1 # w05-05-split-join.py -- split ile parçalama, join ile birleştirme,
2 # kelime sayma
3
4 full_name = "Ayşe Nur Yılmaz"
5 parts = full_name.split()          # boşluklardan böler
6 print(parts, len(parts), "parça")
7 print("İlk:", parts[0], "| Son:", parts[-1])
8
9 # Ayırıcı verilebilir; parça sayısı belliyse çoklu atama
10 date = "13/09/2026"
11 day, month, year = date.split("/")
12 print(year + "-" + month + "-" + day)
13
14 record = "2026105017;Ayşe;Yılmaz;85.5"
15 fields = record.split(";")
16 print(fields[1], fields[2], "->", float(fields[3]) + 4.5)
17
18 # join: parçaları verilen ayırıcıyla birleştirir (split'in tersi)
19 print(" ".join(parts), "|", "-".join(parts))
20
21 # Uygulama: kelime sayma ve fazla boşlukları temizleme
22 text = "Yapay zeka bilgisayarların öğrenmesini sağlar"
23 print(len(text.split()), len(text.split(" "))) # 5 ve 8!
24 print(" ".join(text.split()))                # tek boşluklu temiz metin
```

\$ python w05-05-split-join.py

```
['Ayşe', 'Nur', 'Yılmaz'] 3 parça
İlk: Ayşe | Son: Yılmaz
2026-09-13
Ayşe Yılmaz -> 90.0
Ayşe Nur Yılmaz | Ayşe-Nur-Yılmaz
5 8
Yapay zeka bilgisayarların öğrenmesini sağlar
```

Bir metni parçalara ayırmanın üç sorusu

1. Ayırıcı sabit bir karakter mi? (`;`, `,`, `/`, `|`) → `split(ayırıcı)` kullanın; parça sayısı biliniyorsa çoklu atamayla adlandırın.
2. Ayırıcı boşluk mu? → argümansız `split()`; art arda boşluklar ve kenar boşlukları kendiliğinden hallolur.
3. Tek bir yerden mi kesiyorum? (uzantı, alan adı, kullanıcı adı) → `find` ya da

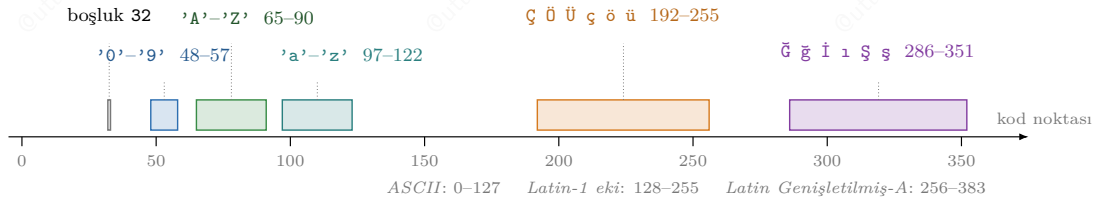


GitHub

rfind ile kesme noktasını bulup *dilimleyin*.

5.6 Karakter kodları, karşılaştırma ve Türkçe sorunu

Bilgisayar harf diye bir şey bilmez; her karakter bellekte bir *tam sayıdır*. Birinci haftada ASCII tablosunu görmüştük; Python bu sayıya **kod noktası** (İng. *code point*) der ve iki işlevle erişim verir: `ord(k)` karakterin kod noktasını, `chr(n)` kod noktasının karakterini verir.



<code>ord("Ç")</code>	199	<code>ord("Ū")</code>	220
<code>ord("ç")</code>	231	<code>ord("ü")</code>	252
<code>ord("Ğ")</code>	286	<code>ord("İ")</code>	304
<code>ord("ğ")</code>	287	<code>ord("ı")</code>	305
		<code>ord("Ş")</code>	351

Her karakter aslında bir tam sayıdır. `ord(k)` bir karakterin kod noktasını, `chr(n)` kod noktasından karakteri verir; ikisi birbirinin tersidir. Metinler *bu sayılara göre karşılaştırılır*: bütün büyük İngiliz harfleri (65-90) bütün küçüklerden (97-122) önce, rakamlar (48-57) hepsinden önce gelir. Türkçeye özgü harfler ASCII'nin dışında, çok daha büyük sayılarda durur — bu yüzden "Çınar" < "Zeynep" yanlıştır: 199 > 90.

Şekil 5.6: Unicode kod noktası cetveli. Rakamlar, büyük harfler ve küçük harfler bu sırayla dizilir; Türkçeye özgü harfler ASCII bölgesinin çok ötesindedir.

Şekil 5.6'deki bu sayılar iki işe yarar. Birincisi, bir harften bir sonrakine geçmek artık aritmetiktir: `chr(ord("K") + 1)` sonucu 'L' dir — 10. haftadaki Sezar şifrelemesinin çekirdeği budur. İkincisi, ve bu hafta önemlisi, **metinler bu sayılara göre karşılaştırılır**.

Kural 5.2 (Metin karşılaştırma kuralı). <, > ve == operatörleri iki metni soldan sağa, karakter karakter karşılaştırır. İlk *farklı* karakterde karar verilir: kod noktası küçük olan metin küçüktür; geri kalan karakterlere bakılmaz. Bütün ortak karakterler eşitse kısa olan metin küçüktür ("önek daha küçüktür"). Bu düzene **sözlük sırası** (İng. *lexicographic order*) denir.

- eşit ■ ilk fark: karar burada ■ bakılmayan karakterler
- ① `A 1 i` < `A y ş e` 0: 'A' = 'A' → devam; 1: `ord('1')` = 108 < `ord('y')` = 121 ⇒ **True**
 - ② `a b` < `a b c` ortak karakterlerin hepsi eşit; soldaki metin önce bitiyor ⇒ **True** — önek her zaman daha küçüktür
 - ③ `Ç ı n a r` < `Z e y n e p` 0: `ord('Ç')` = 199 > `ord('Z')` = 90 ⇒ **False**
 - ④ `1 0` < `9` 0: `ord('1')` = 49 < `ord('9')` = 57 ⇒ **True** — metinler sayı gibi değil, *sözlük gibi* sıralanır

İki metin soldan sağa, karakter karakter karşılaştırılır; ilk farklı karakterde karar verilir ve geri kalanına hiç bakılmaz. Karar, karakterlerin *kod noktalarına* göre verilir. Bu sıralama sözlük sırasına benzer ama onunla aynı değildir: Türkçeye özgü harfler kod noktası olarak İngiliz alfabesinin *sonrasında* durduğu için "Çınar" sıralamada "Zeynep"ten sonra gelir; aynı nedenle bütün büyük harfler bütün küçük harflerden önce gelir ("Zeynep" < "ali" doğrudur). Sayıları metin olarak karşılaştırmak da bu yüzden yanıltıcıdır: "10" < "9" doğrudur, ama 10 < 9 yanlıştır.

Şekil 5.7: Metin karşılaştırma dört örnekte. Karar her zaman ilk farklı karakterde verilir; ölçüt kod noktasıdır, alfabetik sezgi değil.

Kural 5.2'nin, Şekil 5.7'de adım adım izlenen üç şaşırtıcı sonucu vardır. "Zeynep" < "ali" doğrudur: bütün büyük harfler küçüklerden önce gelir. "Çınar" < "Zeynep" yanlıştır: ord("Ç") = 199, ord("Z") = 90'dır. "10" < "9" da doğrudur — metinler sayı gibi değil sözlük gibi sıralanır; bu yüzden sayıları sıralamadan önce int() ile çevirin.

Dikkat

Türkçe harfler ve upper()/lower(). Bu metotlar İngiliz alfabesinin kurallarıyla çalışır ve Türkçenin noktalı/noktasız i çiftini bilmez: "i".upper() de "I".upper() de 'I' verir (iki farklı harf aynı harfe dönüşür), "I".lower() 'i' verir ('ı' değil), "İ".lower() ise uzunluğu 2 olan tuhaf bir metin döndürür — "İ".lower() == "i" bu yüzden False'tur. Sonuç: "ISPARTA".lower() 'isparta', "istanbul".upper() 'ISTANBUL' verir. Pratik çözüm, dönüşümden önce sorunlu harfleri elle değiştirmektir: word.replace("I", "ı").replace("İ", "i").lower(). Bu yamayı şimdilik elle yazıyoruz; 11. haftada aynı işi yapan bir fonksiyon tanımlayacağız.

Kod 5.6 w05-06-unicode.py — ord/chr, karşılaştırma ve Türkçe harf tuzağı

```
1 # w05-06-unicode.py -- Karakter kodları (ord/chr), karşılaştırma,
2 # Türkçe harf tuzağı
3
4 # Her karakterin bir Unicode kod noktası (tam sayı) vardır
5 print(ord("A"), ord("Z"), ord("a"), ord("z"), ord("0"), ord(" "))
6 print(ord("Ç"), ord("Ü"), ord("İ"), ord("ı"), ord("Ş"))
7 print(chr(65), chr(97), chr(231), chr(960))
8 print(chr(ord("K") + 1)) # bir sonraki harf: koda 1 ekle
9
10 # Karşılaştırma kod noktalarına göre, soldan sağa yapılır
11 print("a" < "b", "ab" < "abc", "Ali" < "Ayşe") # l(108)<y(121)
12 print("Zeynep" < "ali") # True: büyük harfler küçüklerden önce
13 print("Çınar" < "Zeynep") # False! Ç (199) > Z (90)
14 print("10" < "9") # True! '1' (49) < '9' (57)
15
16 # Türkçe harf tuzağı: upper/lower İngilizce kurallarıyla çalışır
17 print("ı".upper(), "i".upper(), "I".lower()) # I I i
18 print("İ".lower() == "i", len("İ".lower())) # False 2 (!)
19 print("ISPARTA".lower(), "istanbul".upper())
20
21 # Çözüm: sorunlu harfleri önce elle değiştir, sonra çevir
22 print("ISPARTA İLİ".replace("I", "ı").replace("İ", "i").lower())
23 print("ısparta ili".replace("ı", "I").replace("i", "İ").upper())
```

\$ python w05-06-unicode.py

```
65 90 97 122 48 32
199 220 304 305 350
A a ç π
L
True True True
True
False
True
I I i
False 2
isparta ISTANBUL
ısparta İLİ
```



GitHub

ISPARTA İLİ

5.7 Hizalama ve biçimlendirme

Metin tabloları, fişler ve raporlar için sütunların hizalanması gerekir; bunun iki yolu vardır: [Tablo 5.1](#)'daki metotları kullanmak ya da f-string biçim belirteçleri.

Tablo 5.1: Hizalama ve doldurma metotları. Örneklerde `s = "YPZ105"` (6 karakter); nokta işaretleri boşlukları göstermek içindir.

Metot	Örnek	Sonuç (20 karakter)
<code>center(g)</code>	<code>s.center(20)</code>	YPZ105.....
<code>ljust(g)</code>	<code>s.ljust(20)</code>	YPZ105.....
<code>rjust(g)</code>	<code>s.rjust(20)</code>	YPZ105
<code>center(g, k)</code>	<code>s.center(20, "=")</code>	=====YPZ105=====
<code>zfill(g)</code>	<code>"42".zfill(6)</code>	000042
<code>zfill(g)</code>	<code>"-42".zfill(6)</code>	-00042

Üçü de metni *en az* verilen genişliğe tamamlar; metin zaten uzunsa kırpamaz. `zfill` numaralarda işe yarar: sıra numarasını 017 biçiminde yazmak için `str(17).zfill(3)` yeter. f-string belirteçleri aynı işi *sayılar için de* yapar — `ljust` yalnız metinlere uygulanabildiği için sayıyı önce `str()` ile çevirmek gerekir, oysa `f"{score:>8.1f}"` hem hizalar hem yuvarlar. Kural: sütun içeriği sayıysa f-string, hazır metinse metot kullanın.

f-string'den önce yaygın olan yöntem `str.format`'tır ve eski kodlarda hâlâ görülür; küme parantezleri sırayla doldurulur, `"{0} {1} {0}"` ile sıra da seçilebilir. Belirteçler f-string ile birebir aynıdır; yeni kod yazarken f-string'i tercih edin.

Metotlar, [Kod 5.4](#) ve [Kod 5.6](#)'da olduğu gibi burada da birlikte çalışır: `ljust` ile `rjust` yan yana kullanıldığında sütunları hizalı bir metin tablosu doğar. Sayılar önce `str()` ile metne çevrilmelidir:

```
1 print("Ürün".ljust(12) + "Adet".rjust(6) + "Fiyat".rjust(10))
2 print("-" * 28)
3 print("Kalem".ljust(12) + str(3).rjust(6) + "12.50".rjust(10))
```

Aynı satırı f-string ile `f"{'Kalem':<12}{3:>6}{12.5:>10.2f}"` biçiminde de yazabilirsiniz.

5.8 Uygulamalar

Şimdi bu haftanın araçlarını gerçek problemlerde birleştiriyoruz. Dört örneğin hepsi aynı dört adımla çözülür: **Analiz**, **Algoritma**, **Kod ve çıktı**, **Kontrol**. Hiçbirinde döngü ya da koşul yok — hepsi indeks, dilim ve metot.

Örnek 5.1 (Ad-soyad ayırma ve baş harfler). Kullanıcıdan ad ve soyadını *tek satırda* alan; adı ve soyadı ayrı ayrı, baş harflerini (M.Ü. biçiminde) ve bir kullanıcı adı (`mehmet.ozturk`) üreten program yazın. Kullanıcı fazladan boşluk bırakmış, harfleri gelişigüzel büyütmüş olabilir. Örnek girdi: " mehmet can ÖZTÜRK ".

Çözüm

Analiz. Girdi: tek bir metin. Çıktı: ad, soyad, baş harfler, kullanıcı adı. Sorun üç katmanlı: (i) fazla boşluklar, (ii) harf büyüklüğü, (iii) ikiden fazla sözcük olabilir (“Mehmet Can Öztürk”). İlk sözcüğü ad, son sözcüğü soyad saymak makul bir kuraldır — `parts[0]` ve `parts[-1]`. Baş harf, bir metnin sıfıncı karakteridir.

Algoritma. OKU tamad; temiz $\leftarrow$ tamad’ın fazla boşlukları atılmış hâli (`split + join`); parça $\leftarrow$ temiz’in boşluklardan bölünmüşü; ad $\leftarrow$ parça[0], soyad $\leftarrow$ parça[-1] (ilk harfleri büyütülmüş); başharf $\leftarrow$ ad[0] + "." + soyad[0] + "."; **YAZ** hepsini.

Kod ve çıktı.

Kod 5.7 w05-08-ad-soyad.py — Ad-soyad ayırma, baş harfler ve kullanıcı adı

```
1 # w05-08-ad-soyad.py -- Ad-soyadı ayırma, biçimleme, baş harfler ve
2 # kullanıcı adı üretme
3
4 full_name = input("Adınızı ve soyadınızı girin: ")
5
6 clean = " ".join(full_name.split())      # fazla boşlukları at
7 parts = clean.split()
8 first_name = parts[0].capitalize()
9 last_name = parts[-1].capitalize()
10
11 initials = first_name[0] + "." + last_name[0] + "."
12 username = (first_name + "." + last_name).lower()
13 username = username.replace("ş", "s").replace("ı", "i")
14 username = username.replace("ğ", "g").replace("ç", "c")
15 username = username.replace("ö", "o").replace("ü", "u")
16
17 print("Ad          :", first_name)
18 print("Soyad       :", last_name)
19 print("Baş harfler:", initials)
20 print("Kullanıcı   :", username)
21 print("Uzunluk     :", len(clean), "karakter,", len(parts), "sözcük")
```

\$ python w05-08-ad-soyad.py

```
Adınızı ve soyadınızı girin: mehmet can ÖZTÜRK
Ad          : Mehmet
Soyad       : Öztürk
Baş harfler: M.Ö.
Kullanıcı   : mehmet.ozturk
Uzunluk     : 17 karakter, 3 sözcük
```

Kontrol. Ham metin 25 karakterdir; temizlenmiş hâli "mehmet can ÖZTÜRK" 17 karakter ve 3 sözcüktür — program da bunu yazar. " ".join(clean.split()) kalıbı hem kenar hem ara boşlukları tek boşluğa indirir; `strip()` tek başına bunu yapamazdı. Uç durum: tek sözcük girilirse ad ile soyad aynı çıkar (`parts[0]` ile `parts[-1]` aynı parçadır); program çökmez ama sonuç anlamsızdır — böyle girdileri elemek için koşul gerekir (6. hafta).

Örnek 5.2 (E-posta adresini denetleme). Bir e-posta adresini alan; kullanıcı adı ile alan

adını ayıran ve beş kuralı **True/False** olarak raporlayan program yazın: tam bir @ var mı, kullanıcı adı boş değil mi, alan adında nokta var mı, adres boşluk içermiyor mu, kurumsal (.edu.tr) bir adres mi? Örnek girdi: " Ayse.Yilmaz@ogr.trabzon.edu.tr ".

Çözüm

Analiz. Girdi: bir metin; başında-sonunda boşluk ve gelişigüzel harf büyüklüğü olabilir, o yüzden ilk iş `.strip().lower()` zinciridir. Çıktı: iki parça ve beş mantıksal değer. Ayırma noktası @ işaretidir: `email.find("@")` konumu verir, `email[:at]` solu, `email[at + 1:]` sağ. “Tam bir @” koşulu `email.count("@") == 1` ile; bu karşılaştırmanın sonucu zaten **True/False** olduğundan doğrudan yazdırılabilir — koşul deyimine gerek yok.

Algoritma. OKU adres; temizle; konum $\leftarrow$ adres.find("@"); kullanıcı $\leftarrow$ adres[:konum]; alan $\leftarrow$ adres[konum+1:]; beş sınamayı **YAZ**.

Kod ve çıktı.

Kod 5.8 w05-09-eposta.py — E-posta adresini parçalama ve denetleme

```
1 # w05-09-eposta.py -- E-posta adresini döngüsüz ve if'siz denetleme
2
3 email = input("E-posta adresi: ").strip().lower()
4
5 at_pos = email.find("@")
6 username = email[:at_pos]
7 domain = email[at_pos + 1:]
8
9 print("Kullanıcı adı :", username)
10 print("Alan adı      :", domain)
11 print()
12 print("Tam bir @ var      :", email.count("@") == 1)
13 print("Kullanıcı adı boş değil :", len(username) > 0)
14 print("Alan adında nokta var  :", "." in domain)
15 print("Boşluk içermiyor      :", " " not in email)
16 print("Kurumsal (.edu.tr)    :", email.endswith(".edu.tr"))
```

```
$ python w05-09-eposta.py
```

```
E-posta adresi:  Ayse.Yilmaz@ogr.trabzon.edu.tr
Kullanıcı adı : ayse.yilmaz
Alan adı      : ogr.trabzon.edu.tr

Tam bir @ var      : True
Kullanıcı adı boş değil : True
Alan adında nokta var  : True
Boşluk içermiyor      : True
Kurumsal (.edu.tr)    : True
```

Kontrol. "ayse.yilmaz" 11 karakter olduğuna göre @ işareti 11. indekstedir; ogr.trabzon.edu.tr ise 18 karakterdir ve $11 + 1 + 18 = 30$, temizlenmiş adresin uzunluğuna eşittir ✓. Uç durum: adreste hiç @ yoksa `find -1` döndürür, `email[:-1]` son karakter hariç her şeyi verir ve program *sessizce yanlış* çalışır — bu haftanın sınırı tam burasıdır.



GitHub

Örnek 5.3 (Öğrenci numarasının biçimini denetleme). Öğrenci numaraları YYYYBBSSSS biçimindedir: 4 hane giriş yılı, 3 hane bölüm kodu, 3 hane sıra numarası. Bir numarayı alan; uzunluk, yalnız rakamdan oluşma, giriş yılı ve bölüm kodu denetimlerini yapan ve numarayı üç parçaya ayıran program yazın. Örnek: 2026105017.

Çözüm

Analiz. Girdi tek bir metin — *sayı değil*. Numarayı `int()` ile çevirmemeliyiz: başındaki sıfırlar kaybolur ve dilimleme yapılamaz. Dört denetim: uzunluk (`len(no) == 10`), yalnız rakam (`no.isdigit()`), yıl (`no.startswith("2026")`), bölüm (`no[4:7] == "105"`). Parçalar sabit indekslerde olduğu için üç dilim yeter.

Algoritma.

Sözde kod: Numara ayrıştırma

```
BAŞLA
OKU numara
yıl ← numara[:4]
bölüm ← numara[4:7]
sıra ← numara[7:]
YAZ dört sınavın sonucu
YAZ yıl, bölüm, sıra
BİTİR
```

Kod ve çıktı.

Kod 5.9 w05-10-ogrenci-no.py — Öğrenci numarasını denetleme ve parçalara ayırma

```
1 # w05-10-ogrenci-no.py -- Öğrenci numarasını denetleme ve parçalama
2 # Biçim: YYYYBBSSSS -- 4 hane yıl, 3 hane bölüm, 3 hane sıra no
3
4 student_no = input("Öğrenci numarası: ").strip()
5
6 year = student_no[:4]
7 dept_code = student_no[4:7]
8 order = student_no[7:]
9
10 print("Uzunluk 10 hane :", len(student_no) == 10)
11 print("Yalnız rakam :", student_no.isdigit())
12 print("Bu yıl kayıtlı :", student_no.startswith("2026"))
13 print("YZ Müh. (105) :", dept_code == "105")
14 print()
15 print("Giriş yılı :", year)
16 print("Bölüm kodu :", dept_code)
17 print("Sıra no :", order, "->", int(order))
```



GitHub

```
$ python w05-10-ogrenci-no.py
```

```
Öğrenci numarası: 2026105017
Uzunluk 10 hane   : True
Yalnız rakam      : True
Bu yıl kayıtlı    : True
YZ Müh. (105)     : True

Giriş yılı : 2026
Bölüm kodu : 105
Sıra no    : 017 -> 17
```

Kontrol. Parçaların uzunlukları $4 + 3 + 3 = 10$ 'dur ve birleşimleri ("2026" + "105" + "017") orijinal numarayı verir ✓. Sıra numarasının "017" metni olarak kalması önemlidir: `int("017")` sonucu 17'dir ve baştaki sıfır geri gelmez. "Ekranı görünen biçim" ile "hesapta kullanılan değer" bu yüzden farklı tutulur — telefon numaraları, TC kimlik numaraları ve posta kodları da hep metin olarak saklanır.

Örnek 5.4 (Palindrom denetimi). Bir sözcüğün ya da cümlemin **palindrom** olup olmadığını — tersten okunuşunun aynı olup olmadığını — bulan program yazın. Boşluklar, noktalama ve harf büyüklüğü göz ardı edilmelidir. Örnek: "Ey Edip Adana'da pide ye".

Çözüm

Analiz. Girdi: bir metin. Çıktı: True/False. Fikir üç adımdır: temizle, ters çevir, karşılaştır. Temizleme `lower()` ve art arda `replace()` çağrılıyla yapılır; ters çevirme tek bir dilimdir: `cleaned[::-1]`. Karşılaştırma `==` ile.

Algoritma. OKU metin; temiz $\leftarrow$ küçük harfli, boşluksuz, noktalamasız hâli; ters $\leftarrow$ temiz[::-1]; YAZ temiz = ters.

Kod ve çıktı.

Kod 5.10 w05-11-palindrom.py — Palindrom: temizle, ters çevir, karşılaştır

```
1 # w05-11-palindrom.py -- Palindrom: temizle, ters çevir, karşılaştır
2
3 text = input("Bir sözcük ya da cümle: ")
4
5 cleaned = text.lower().replace(" ", "").replace("'", "")
6 cleaned = cleaned.replace(",", "").replace(".", "")
7 reversed_text = cleaned[::-1]
8
9 print("Temizlenmiş   :", cleaned)
10 print("Tersi        :", reversed_text)
11 print("Palindrom mu?", cleaned == reversed_text)
```



GitHub

```
$ python w05-11-palindrom.py
```

```
Bir sözcük ya da cümle: Ey Edip Adana'da pide ye
Temizlenmiş : eyedipadanadapideye
Tersi       : eyedipadanadapideye
Palindrom mu? True
```

Kontrol. Temizlenmiş metin "eyedipadanadapideye", 19 karakterdir; tek sayıda karakteri olduğu için ortadaki karakter (indeks 9) kendisiyle eşleşir, baştan k . karakter ile sondan k . karakter aynıdır — ilk harf 'e', son harf 'e' ✓. Boş metin ve tek karakter her zaman palindromdur. Bu çözüm karakter karakter karşılaştırma yapmadığı için çok kısadır; 10. haftada aynı problemi döngüyle yeniden yazacağız.

LAB Laboratuvar Çalışması 5: Metinle çalışmak

Bu haftanın laboratuvarı geçen haftanın devamıdır: 4. haftada yazdığınız `tanitim.py` dosyasını açarak başlayın. Her görevin beklenen çıktısı verilmiştir; kendi çıktınızla *karakter karakter* karşılaştırın.

Görev 1. Tanıtımı geliştirme. `tanitim2.py`: adınızı ve soyadınızı *tek bir input()* ile alın; fazla boşlukları temizleyin (`" ".join(ad.split())`), adı ve soyadı ayırın, soyadı `upper()` ile büyütün, baş harfleri indeks 0 ile alın: Ben Ayşe YILMAZ (A.Y.), Yapay Zeka Mühendisliği öğrencisiyim.

Görev 2. Metin analiz betiği. `analiz.py`: bir cümle alın; karakter sayısını, boşluk sayısını, kelime sayısını (`len(s.split())`), ilk ve son kelimeyi, cümlelerin tersini ve "a" harfinin kaç kez geçtiğini yazdırın. Girdi Yapay zeka her yerde için: 20, 3, 4, Yapay, yerde, edrey reh akez yapaY, 3.

Görev 3. Şifre maskesi. `maske.py`: bir şifre alın; uzunluğunu, tamamen maskelenmiş hâlini (`"*" * len(password)`), yalnız ilk ve son karakteri görünen hâlini (`password[0] + "*" * (len(password) - 2) + password[-1]`) ve yalnız rakamdan oluşup oluşmadığını (`isdigit()`) yazdırın. Girdi Trabzon61 için: 9, *****, T*****1, False.

Görev 4. E-posta doğrulayıcı. `eposta.py`: Örnek 5.2'yi kendi adresinizle çalıştırın, sonra iki sına ekleyin: adres ".com" ile mi bitiyor, kullanıcı adında nokta var mı? Alan adının ilk parçasını (`ogr`) `domain.split(".")[0]` ile yazdırın.

Görev 5. Dosya adı çözümleyici. `dosya.py`: bir dosya *yolu* alın (`C:/Users/ayse/Belgeler/rapor.final.pdf`); klasörü, dosya adını, uzantısız adı ve uzantıyı ayırın (son "/" ve son "." için `rfind`), sonra uzantısı `docx` ile değiştirilmiş yeni yolu yazdırın.

Teslim. Beş dosyayı da ders bitmeden çalıştırıp gösterin ve saklayın: Görev 2 ile Görev 3'ü 10. haftada döngüyle, 11. haftada fonksiyonlarla yeniden yazacağız.

Özet

Yapı	Yazım	Not
Metin yazma	'a' "a" """çok satır"""	tırnaklar metnin parçası değildir
Kaçış / ham	"\n \t \\" r"C:\Users"	ham metin ters bölüyle bitemez
Temel işlemler	len(s) s + t s * n alt in s	n tam sayı; in harf büyüklüğüne duyarlı
İndeks	s[0] s[-1]	geçerli aralık $0 \dots n-1$, $-n \dots -1$
Dilim	s[a:b] s[a:b:c] s[::-1]	a dahil, b hariç; uzunluk $b - a$
Değiştirilemezlik	s[0] = "K" → TypeError	yeni metin üretin: s = "K" + s[1:]
Harf büyüklüğü	upper lower title capitalize swapcase	Türkçe İ/ı yanlış çevrilir
Kenar temizleme	strip lstrip rstrip	yalnız uçlar; argüman bir karakter kümesidir
Arama	find rfind index count startswith endswith	find bulamazsa -1, index hata
Değiştirme	s.replace(a, b)	tüm geçişler; üçüncü argüman sınırı
Sınama	isdigit isalpha isalnum isspace isupper islower	boş metin için hepsi False
Parçala / birleştir	s.split(";") ";".join(parts)	join'de çağrılan nesne ayrırcıdır
Hizalama	center ljust rjust zfill	yalnız uzatır, kırpamaz
Kod noktası	ord("A") = 65 chr(65) = "A"	"a" < "b", "Z" < "a": sözlük sırası



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w05>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı out/ klasöründedir.

Alıştırmalar

Alıştırma 5.1. ●●● s = "PROGRAMLAMA" olsun (11 karakter). Aşağıdaki ifadelerin sonuçlarını *çalıştırmadan* yazın, sonra kabukta doğrulayın. (a) s[0], (b) s[-1], (c) s[0:4], (d) s[:3], (e) s[7:], (f) s[-4:], (g) s[:2], (h) s[::-1], (i) s[4:4], (j) s[8:100].

Cevap: (a) 'P' (b) 'A' (c) 'PROG' (d) 'PRO' (e) 'LAMA' (f) 'LAMA' (g) 'PORMAA' (h) 'AMALMARGORP' (i) '' (j) 'AMA'

Alıştırma 5.2. ●●● Bir öğrenci name = "ayse" metnindeki ilk harfi büyütme için name[0] = "A" yazıyor ve hata alıyor. (a) Hatanın türünü ve nedenini iki cümleyle açıklayın. (b) Aynı işi yapan *iki* farklı doğru satır yazın. (c) Bir arkadaşı name.capitalize() yazıyor ama print(name) yine ayse basıyor; neden?

Cevap: (a) *TypeError*:

karakter dizisi değiştirilemez bir nesnedir, içindeki karakterler yerinde değiştirilemez. (b) `name = "A" + name[1:]` ya da `name = name.capitalize()`. (c) Metotlar metni değiştirmez, yeni metin döndürür; sonuç bir ada atanmadığı için kaybolur.

Alıştırma 5.3. ●●● Bir cümle alan ve şunları yazdıran bir program yazın: toplam karakter sayısı, boşluk sayısı, boşluksuz karakter sayısı ve "a" harfinin kaç kez geçtiği. Girdi Yapay zeka her yerde için sırasıyla 20, 3, 17, 3.

Cevap: `len(s) = 20; s.count(" ") = 3; 20 - 3 = 17; s.count("a") = 3` (büyük Y sayılmaz, "A" ayrı bir karakterdir)

Alıştırma 5.4. ●●● Bir başlık alan ve onu artı-tire çerçevesi içine alan bir program yazın; başlığın iki yanında ikişer boşluk olsun. Ayrıca başlığın büyük harfli hâlini 30 karakter genişliğinde, yıldızlarla ortalanmış olarak yazdırın. Girdi Yapay Zeka 101 (14 karakter) için çerçeve satırı 20 karakterdir.

Cevap: İç genişlik $14 + 4 = 18$, çerçeve `"+" + "-" * 18 + "+" = 20` karakter; `title.upper().center(30, "*")` her iki yana sekizer yıldız koyar

Alıştırma 5.5. ●●● 11 haneli bir telefon numarası alan ve iki biçimde yazdıran bir program yazın: 0 (532) 123 45 67 ve maskelenmiş hâli 0 (532) *** ** 67. Ayrıca operatör kodunu (532) ayrı yazdırın. Yalnız dilimleme ve + kullanın.

Cevap: `p[0]`, `p[1:4]`, `p[4:7]`, `p[7:9]`, `p[9:]` parçaları; maskede orta üç grup `"***"` ve `"**"` ile değiştirilir

Alıştırma 5.6. ●●● GG.AA.YYYY biçiminde bir tarih alan ve onu YYYY-AA-GG (ISO) ve AA/GG/YYYY (ABD) biçimlerinde yazdıran bir program yazın. Ayrıca yılın son iki hanesini ve dört yıl sonrasını (mezuniyet yılı) yazdırın. Girdi 13.09.2026 için: 2026-09-13, 09/13/2026, 26, 2030.

Cevap: `day, month, year = date.split(".")` ile üç parça; ISO için `year + "-" + month + "-" + day`; kısa yıl `year[-2:]`; `int(year) + 4 = 2030`

Alıştırma 5.7. ●●● Bir yorum metni ve bir yasaklı sözcük alan; sözcüğün bütün geçişlerini aynı uzunlukta yıldızla değiştiren bir program yazın. Sözcük cümle başında (Berbat) ya da tamamen büyük harfle (BERBAT) da geçebilir; üç biçimi de yakalayın ve kaç değişiklik yapıldığını yazdırın. Girdi Bu film berbat, gerçekten Berbat! BERBAT. ve berbat için 3 değişiklik.

Cevap: `"*" * len(bad)` maskesi; `replace` üç kez çağrılır (`bad`, `bad.capitalize()`, `bad.upper()`); sayım `text.lower().count(bad) = 3`

Alıştırma 5.8. ●●● Bir cümle alan ve şunları yazdıran bir program yazın: kelime sayısı, boşluksuz karakter sayısı, kelime başına ortalama harf sayısı (iki ondalık), ilk kelime, son kelime ve kelimeleri ters sırayla birleştirilmiş hâli. Girdi Yapay zeka her yerde için: 4, 17, 4,25, Yapay, yerde, yerde her zeka Yapay.

Cevap: `words = s.split()`; `len(words) = 4`; `len("".join(words)) = 17`; ortalama $17/4 = 4,25$; ters sıra için `" ".join(words[::-1])`

Alıştırma 5.9. ●●● Kurumsal bir öğrenci e-postasını (`ad.soyad@ogr.kurum.edu.tr`) alan; adı, soyadı ve kurum adını çıkaran ve beş biçim denetimi yapan bir program yazın: tek `@`, kullanıcı adında nokta, `ogr.` ile başlayan alan adı, `.edu.tr` ile biten adres, ad ve soyadın boş olmaması. Girdi `Ayşe.Yılmaz@ogr.trabzon.edu.tr`.

Cevap: @ konumu 11; kullanıcı adı ayşe.yilmaz, alan adı ogr.trabzon.edu.tr; kurum, alan adının noktalardan bölünmüş ikinci parçasıdır (trabzon); ad ile soyad kullanıcı adının ilk noktasından ayrılır

Alıştırma 5.10. ●● Aşağıdaki yedi metni Python'un küçükten büyüğe sıralayacağı düzende yazın ve her komşu çift için kararın hangi karakterde verildiğini belirtin: "elma", "Elma", "armut", "Armut", "10", "9", "Çilek".

Cevap: "10", "9", "Armut", "Elma", "armut", "elma", "Çilek" — karar her zaman ilk karakterde verilir: kodları sırasıyla 49, 57, 65, 69, 97, 101, 199

Alıştırma 5.11. ●●● Bir ad-soyad alan ve onu Türkçe harf kurallarına uygun biçimleyen bir program yazın: `ayşe YILMAZ` → `Ayşe Yılmaz`. Doğrudan `title()` kullanmak `Ayşe Yılmaz` verir (I harfi i'ye dönüşür); önce `I` → `ı` ve `İ` → `i` değişimlerini yapın. Programınız hem doğru hem de yanlış sonucu yazdırsın ve ikisinin eşit olup olmadığını göstereyin.

Cevap: `clean.replace("I", "ı").replace("İ", "i").lower().title()` doğru sonucu verir; `clean.title()` ise `Ayşe Yılmaz` verir, iki sonuç eşit değildir

Alıştırma 5.12. ●●● Bir dosya yolu alan (`C:/Users/ayse/Belgeler/rapor.final.pdf`) ve şunları yazdıran bir program yazın: klasör yolu, dosya adı, uzantısız ad, uzantı, yolun kaç seviye derin olduğu ("`/`" sayısı) ve uzantısı `docx` ile değiştirilmiş yeni yol. Yalnız `rfind`, `dilimleme`, `count` ve `+` kullanın.

Cevap: son "/" konumu `rfind("/")`; dosya adı `path[p+1:] = rapor.final.pdf`; uzantı son noktadan sonrası `pdf`; `path.count("/") = 4`; yeni yol `folder + "/" + stem + ".docx"`

Hafta 6: Koşullu İfadeler: Programa Karar Verdirmek

Öğrenme çıktıları

Bu haftanın sonunda:

1. karşılaştırma operatörleriyle (`==` `!=` `<` `<=` `>` `>=`) `bool` değerli koşullar kurabilecek, zincirleme karşılaştırmayı (`0 <= x < 10`) yazabileceksiniz;
2. `if`, `if-else` ve `if-elif-else` yapılarını **girinti kuralına** uygun yazabilecek, `IndentationError` mesajını okuyabileceksiniz;
3. bir `elif` zincirinde koşulların *sırasının* neden anlamı değiştirdiğini açıklayabilecek ve eşik değerli kuralları doğru sırayla kodlayabileceksiniz;
4. `and`, `or`, `not` işleçlerini doğruluk tablosu ve öncelik kurallarıyla birleştirebilecek, kısa devre değerlendirmeyi bir koruma deseni olarak kullanabileceksiniz;
5. sayıların, metinlerin ve `None`'ın doğruluk değerini bilerek `if x:` desenini ve koşullu ifadeyi (`a if k else b`) yazabileceksiniz;
6. 3. haftada tasarladığınız karar yapılarını çalışan Python programlarına çevirebilecek, programınızı **sınır değerleriyle** sınayabileceksiniz.

Anahtar kavramlar

koşul (İng. *condition*) **seçme yapısı** (İng. *selection*) **blok** (İng. *block*) **girinti** (İng. *indentation*) **karşılaştırma operatörü** (İng. *comparison operator*) **zincirleme karşılaştırma** (İng. *chained comparison*) **mantıksal operatör** (İng. *logical operator*) **doğrululuk tablosu** (İng. *truth table*) **kısa devre değerlendirme** (İng. *short-circuit evaluation*) **doğrululuk değeri** (İng. *truthiness*) **koşullu ifade** (İng. *conditional expression*) **sınır değeri** (İng. *boundary value*)

Şimdiye kadar yazdığımız Python programlarının hepsi aynı biçimde çalıştı: birinci satırdan başladı, son satırda bitti, aradaki her satırı tam olarak bir kez yürüttü. Bu *sıralı akış* bir hesap makinesi için yeterlidir, ama bir program için değildir. Gerçek programlar duruma göre davranır: not 50'nin altındaysa uyarı verir, sepet 500 lirayı geçtiyse indirim uygular, bölen sıfırsa bölmeyi hiç denemez. 3. haftada bu “duruma göre” davranışı kâğıt üzerinde tasarlamıştık — akış diyagramında eşkenar dörtgen, sözde kodda **EĞER-DEĞİLSE**. Bu hafta o eşkenar dörtgeni Python'a çeviriyoruz. Karşılığı tek bir sözcüktür: `if`.

Karar verme, bir yapay zeka mühendisinin işinin merkezindedir ve çoğu zaman gizlenmiş hâldedir: bir sınıflandırma modeli aslında “olasılık 0,5'ten büyükse pozitif de” diyen bir eşik karardır, bir veri temizleme betiği “bu alan boşsa satırı at” der. Bu haftanın `if` satırları o kararların en küçük yapı taşıdır — ve yazacağınız her koşul, 4. haftanın türleri ile 5. haftanın metin metotlarının üzerine kurulacak.

6.1 Boole türü ve karşılaştırma operatörleri

Bir kararın verilebilmesi için önce cevabı yalnız “doğru” ya da “yanlış” olabilen bir soru gerekir. Python'da bu sorunun cevabı 4. haftada tanıştığınız dördüncü temel türdür.

Tanım 6.1: `bool` türü ve karşılaştırma

`bool` türünün yalnız iki değeri vardır: `True` ve `False`. İkisi de anahtar sözcüktür; baş harfleri **büyük** yazılır ve tırnak içine alınmaz (“`True`” bir metindir, bir doğruluk değeri değil). Bir **karşılaştırma operatörü** (İng. *comparison operator*) iki değeri karşılaştırır ve sonucu her zaman bir `bool`'dur:

`==` eşit mi `!=` eşit değil mi `<` küçük mü `<=` küçük ya da eşit mi `>` büyük mü `>=` büyük ya da eşit mi

Karşılaştırmalar aritmetik işlemlerden *sonra* değerlendirilir: `a + 1 > b` ifadesinde önce `a + 1` hesaplanır.

Tanım 6.1'in söylediği şey basittir ama sonucu büyüktür: bir karşılaştırmanın sonucu sıradan bir değerdir — yazdırılabilir, bir değişkene atanabilir, başka bir ifadenin içine konabilir.

Python etkileşimli kabuk (REPL)

```
>>> 7 > 3
True
>>> type(7 > 3)
<class 'bool'>
>>> passed = 72 >= 50
>>> passed
True
```

Kod 6.1 w06-01-karsilastirma.py — Karşılaştırma operatörleri, tür karışımı ve zincirleme

```
1 # w06-01-karsilastirma.py -- bool, karşılaştırma operatörleri, zincirleme
2
3 score = 72
4 print(score > 50, score == 72, score != 72)    # True True False
5 print(type(score > 50))                       # <class 'bool'>
6
7 # Karşılaştırmanın sonucu bir değişkende saklanabilir
8 passed = score >= 60
9 print("Geçti mi?", passed)
10
11 # int ile float değerce karşılaştırılır; metin ile sayı asla eşit olmaz
12 print(5 == 5.0, "5" == 5)                    # True False
13
14 # Metinler karakter karakter, Unicode sırasına göre karşılaştırılır
15 print("elma" < "erik")                        # True: 'l' < 'r'
16 print("Zeynep" < "ali")                      # True: büyük harfler önce gelir
17
18 # Zincirleme karşılaştırma: matematikteki gibi yazılır
19 x = 7
20 print(0 <= x < 10, 0 <= x < 5)                # True False
```

\$ python w06-01-karsilastirma.py

```
True True False
<class 'bool'>
Geçti mi? True
True False
True
True
True False
```

Kod 6.1'de dört ayrıntıya dikkat edin. Birincisi, `5 == 5.0` sonucu `True`'dur: `int` ile `float` *değerce* karşılaştırılır. İkincisi, `"5" == 5` sonucu `False`'tur — bir metin hiçbir zaman bir sayıya eşit olamaz; `input()`'un metin döndürdüğünü unutup `if cevap == 5` yazmak bu



GitHub

haftanın en sinsi hatasıdır, çünkü program hata vermez, sessizce hep `False` der. Üçüncüsü, metinler 5. haftada gördüğünüz gibi Unicode sırasına göre karşılaştırılır: `"elma" < "erik"` doğrudur, çünkü ilk farklı karakterde `l`, `r` harfinden öncedir. Dördüncüsü, son satırdaki **zincirleme karşılaştırmadır** (*Ing. chained comparison*).

Tanım 6.2: Zincirleme karşılaştırma

Python'da `a <= x < b` yazımı geçerlidir ve `a <= x and x < b` ile aynı anlama gelir; `x` yalnız *bir kez* hesaplanır. Aynı yazım `0 < a < b < 100` gibi daha uzun zincirler için de çalışır. Programlama dillerinin çoğunda bulunmayan bu kolaylık, bir aralık koşulunu matematikteki gibi yazmanızı sağlar.

3. haftada bir aralık koşulunu *iki* karşılaştırmayla yazmanız gerektiğini söylemiştik: `puan ≥ 50 VE puan ≤ 100`. Python bu kuralın istisnasıdır ve `50 <= score <= 100` yazımına izin verir; ama zinciri yalnız *aynı* değişken üzerinde ve aynı yönde kurun.

6.2 if deyimi: koşul, iki nokta ve blok

Elimizde `True/False` üreten bir ifade olduğuna göre, artık “bu doğruysa şunu yap” diyebiliriz.

Tanım 6.3: if deyimi

`if` deyimi bir **koşulu** (*Ing. condition*) değerlendirir; koşul `True` ise ardından gelen **bloğu** (*Ing. block*) yürütür, `False` ise bloğu bütünüyle atlar. Yazımı:

`if koşul:` ardından girintili bir ya da daha çok satır

Başlık satırı iki nokta (`:`) ile biter; bloğun satırları başlık satırından **daha içeriden** yazılır.

Kod 6.2 w06-02-ilk-if.py — Tek if: koşul doğruysa blok çalışır, değilse atlanır

```
1 # w06-02-ilk-if.py -- tek bir if: koşul doğruysa blok çalışır, yoksa atlanır
2
3 score = float(input("Sınav notunuz: "))
4
5 if score < 50:
6     print("Uyarı: not 50'nin altında.")
7     print("Ders sorumlusuyla görüşmeniz önerilir.")
8
9 print(f"Girilen not: {score:.1f}")    # girintisiz: her durumda çalışır
10 print("Program bitti.")
```

```
$ python w06-02-ilk-if.py
```

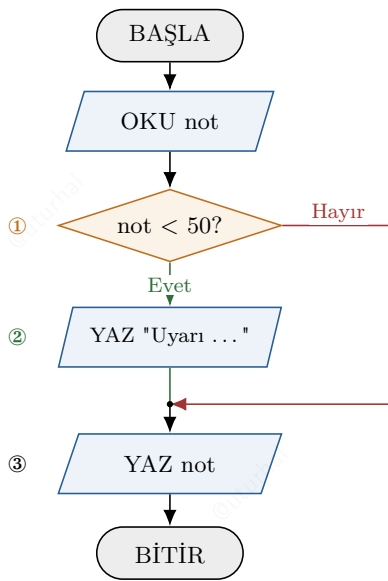
```
Sınav notunuz: 42
Uyarı: not 50'nin altında.
Ders sorumlusuyla görüşmeniz önerilir.
Girilen not: 42.0
Program bitti.
```

Kod 6.2'in çıktısında iki uyarı satırı görünüyor, çünkü girilen not 42'dir ve `42 < 50` doğrudur. Aynı programı 75 ile çalıştırsaydınız o iki satır *hiç* yazılmazdı; ama son iki satır — girintisiz olanlar — her iki durumda da yazılır. Bir bloğun nerede bittiğini Python'a söyleyen



GitHub

tek şey budur: girintinin geri dönmesi. Şekil 6.1, aynı programı 3. haftanın akış diyagramıyla yan yana koyar.



Aynı algoritma, Python'da

```

score = float(input("Sınav notunuz: "))
① if score < 50:
②     print("Uyarı: not 50'nin altında.")
    print("Ders sorumlusuyla görüşmeniz önerilir.")
③ print(f"Girilen not: {score:.1f}")
    print("Program bitti.")
  
```

- ① Karar sembolü = if satırı. Sorunun sonundaki "?" Python'da satır sonundaki **iki nokta**dır.
- ② Evet dalı = girintili blok. Dört boşluk içeride yazılan her satır yalnız koşul True ise çalışır.
- ③ Okların birleştiği yer = girintinin geri döndüğü ilk satır. Buradan sonrası her iki durumda da çalışır.

3. haftada kâğıda çizdiğiniz her karar sembolü, Python'da bir if satırıdır. Baklava dilimindeki soru koşul olur, Evet oku girintili bloğa girer, Hayır oku bloğu atlar. Diyagramda iki okun yeniden birleştiği nokta, kodda girintinin sola döndüğü satırdır — Python'da bloğun bittiğini söyleyen tek şey bu girinti değişimidir; ayrı bir "bitir" sözcüğü yoktur.

Şekil 6.1: Tek bir if: solda 3. haftanın akış diyagramı, sağda birebir karşılığı olan Python kodu. Numaralar iki gösterim arasındaki eşlemeyi kurar: karar sembolü if satırı, Evet dalı girintili blok, okların birleştiği yer ise girintinin sola döndüğü ilk satırdır.

Kural 6.1 (Blok ve girinti kuralı). (i) if, elif ve else başlık satırları iki nokta (:) ile biter.

- (ii) Bir bloğa ait bütün satırlar **aynı sayıda** boşlukla içeriden yazılır. Gelenek ve bu dersin kuralı **4 boşluktur**; sekme karakteri kullanılmaz. Blok en az bir satır içermelidir; hiçbir şey yapılmayacaksa **pass** yazılır.
- (iii) Blok, girintinin başlık satırının hizasına (ya da daha sola) dönmesiyle biter. Ayrı bir "bitir" sözcüğü *yoktur*.
- (iv) else ve elif satırları, ait oldukları if ile **tam aynı hizada** yazılır.

Bu kurallar isteğe bağlı bir biçim tercihi değil, dilin söz dizimidir; bozulduğunda program çalışmaz.

Kural 6.1'in (iii) maddesi ilk bakışta gereksiz görünür; ama "şimdilik boş bıraksam da sonra doldururum" diye yazılan boş bir if bloğu programı durdurur — boş bırakmanız gerekiyorsa **pass** yazın. Girintinin söz diziminin parçası olması ilk bakışta katı görünse de karşılığında şunu kazanırsınız: bir Python programında kodun *görünüşü* ile *anlamı* birbirinden ayrılmaz (Şekil 6.2).

✗ Sık yapılan hata

if score < 50: (sonraki satır girintisiz) → if score < 50: (sonraki satır 4 boşluk içeriden)
if satırından sonra girintili bir blok gelmezse Python `IndentationError: expected an indented block after 'if' statement on line 3` der. Hata mesajı hangi satırın

bloğunun eksik olduğunu söyler; siz de bir sonraki satırı dört boşluk içeriden yazarsınız. Ters yönde hata da vardır: hiçbir başlık satırı yokken bir satırı boşuna içeriden yazarsanız `IndentationError: unexpected indent` alırsınız.

```

1 n = int(input("Bir tam sayı girin: "))
2 if n % 2 == 0:
3     print(f"{n} çift bir sayıdır.")
4     print("İkiye tam bölünür.")
5 else:
6     print(f"{n} tek bir sayıdır.")
7 print("Program bitti.")
    4 boşluk

```

..... İki nokta: "bir blok başlıyor" işareti.

..... if bloğu (2 satır): yalnız koşul True ise çalışır. Aynı girinti = aynı blok.

..... else if ile aynı hizada olmalı; kendi bloğu var.

..... else bloğu (1 satır): koşul False ise çalışır.

..... Girinti sıfıra döndü: blok dışı. Her durumda çalışır.

Python'da bloğun sınırını girinti belirler. Başka dillerdeki { } ya da begin/end yerine, aynı bloktaki satırlar aynı sayıda boşlukla içeriden yazılır (gelenek: 4 boşluk; sekme karakteri kullanmayın). Girintiyi bir satırda unutmak ya da fazladan koymak `IndentationError` verir; bloğu bitirmek için tek yapılacak şey sonraki satırı yeniden sola yazmaktır.

Şekil 6.2: Blok ve girinti: hangi satır hangi bloğa ait? Renkli çubuklar blokların sınırını gösterir. Aynı girinti = aynı blok; girintinin sıfıra dönmesi bloğun bittiği anlamına gelir.

⚠ Dikkat

Girintide sekme (`\t`) karakteri ile boşluğu karıştırmayın: ekranda ikisi aynı görünür, ama Python için farklıdır ve karışık kullanım `TabError: inconsistent use of tabs and spaces in indentation` verir. Bir kod parçasını internetten kopyaladığınızda en sık karşılaşılan hata budur. IDLE ve PyCharm `\t` tuşuna basınca dört boşluk yazacak biçimde ayarlıdır.

6.3 if-else: iki yoldan biri

Karar sembolünün Hayır dalında da yapılacak bir iş varsa `else` bloğu yazılır. `else` satırının *koşulu yoktur* — “yukarıdaki koşul sağlanmadıysa” demektir — ve `if` ile aynı hizada yazılır, kendi girintili bloğu vardır. Şekil 6.2'deki program tam olarak bu yapıdadır: iki daldan her zaman *tam biri* çalışır, ikisi birden ya da hiçbiri olamaz; akış diyagramındaki iki okun birleştiği yer ise girintinin sola döndüğü son satırdır.

Kod 6.3 w06-03-tek-cift.py — if-else: bir tam sayı tek mi, çift mi?

```

1 # w06-03-tek-cift.py -- if-else: bir tam sayı tek mi, çift mi?
2
3 n = int(input("Bir tam sayı girin: "))
4
5 if n % 2 == 0:
6     print(f"{n} çift bir sayıdır.")
7 else:
8     print(f"{n} tek bir sayıdır.")

```



GitHub

```
$ python w06-03-tek-cift.py
```

```
Bir tam sayı girin: 17
17 tek bir sayıdır.
```

Kod 6.3, 3. haftada sözde kodla yazdığımız “çift mi tek mi” algoritmasının birebir Python karşılığıdır: $\text{sayi} \bmod 2 = 0$ koşulu $n \% 2 == 0$ olmuştur. İki işaret değişti ve ikisi de önemlidir: *mod* yerine $\%$, ve *tek* eşittir yerine *çift* eşittir. Bir koşulu yazarken “hangi durumda True olur?” diye sesli sorun: `if n % 2 == 0:` satırı “*n*’in ikiyle bölümünden kalan sıfıra eşitse” diye de, “*n* çiftse” diye de okunur; ikisi arasındaki köprüyü kurabildiğinizden emin olun.

6.4 if-elif-else ve sıranın önemi

Kurallar çoğu zaman iki dalı değildir. Bir not AA, BA, BB... ya da FF olabilir; bir bilet ücretsiz, indirimli ya da tam olabilir. Python bu tür çoklu kararları `elif` (*else if*’in kısaltması) satırlarıyla yazar.

Kod 6.4 w06-04-harf-notu.py — if-elif-else zinciri: 100’lük nottan harf notu

```
1 # w06-04-harf-notu.py -- if-elif-else zinciri: 100'lük nottan harf notu
2
3 score = float(input("Dönem sonu notu (0-100): "))
4
5 if score >= 90:
6     grade = "AA"
7 elif score >= 85:
8     grade = "BA"
9 elif score >= 80:
10    grade = "BB"
11 elif score >= 75:
12    grade = "CB"
13 elif score >= 70:
14    grade = "CC"
15 elif score >= 65:
16    grade = "DC"
17 elif score >= 60:
18    grade = "DD"
19 elif score >= 50:
20    grade = "FD"
21 else:
22    grade = "FF"
23
24 print(f"Not: {score:.1f} -> Harf notu: {grade}")
```

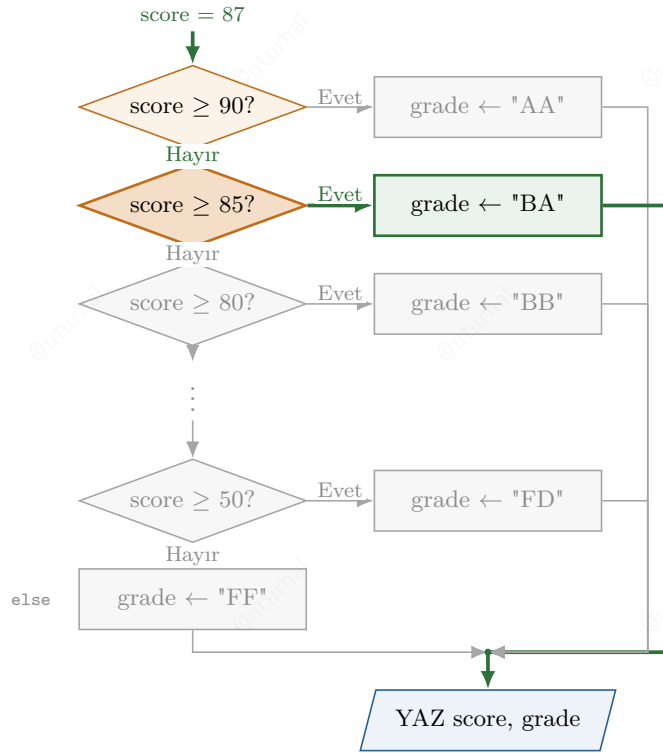
```
$ python w06-04-harf-notu.py
```

```
Dönem sonu notu (0-100): 76.5
Not: 76.5 -> Harf notu: CB
```



GitHub

Kod 6.4, sekiz `elif` ve bir `else` ile dokuz harf notunu kapsar. Çıktıda görüldüğü gibi 76,5 notu CB verir: ≥ 90 ? Hayır. ≥ 85 ? Hayır. ≥ 80 ? Hayır. ≥ 75 ? Evet — zincir burada biter, alttaki beş koşul hiç sorulmaz (**Şekil 6.3**). İşte bu yüzden `elif` zincirinde *üst* sınır yazmaya gerek yoktur: `elif 75 <= score < 80:` doğru olurdu ama gereksizdir, çünkü oraya ancak `score < 80` zaten bilinirken gelinir.



Python'da

```

if score >= 90:
    grade = "AA"
elif score >= 85:
    grade = "BA"
elif score >= 80:
    grade = "BB"
...
elif score >= 50:
    grade = "FD"
else:
    grade = "FF"
print(f"{score} -> {grade}")

```

elif = else if: "yukarıdakiler değilse, peki bu mu?" 87 ≥ 90? Hayır. 87 ≥ 85? **Evet**. Kalan koşullar (87 ≥ 80 doğru olsa bile) **hiç sorulmaz**.

Zincirde yalnız bir dal çalışır: yukarıdan aşağı ilk doğru olan. Bu yüzden koşulların *sırası* anlamı değiştirir: ≥ ile yazılmış eşikler büyükten küçüğe sıralanmalıdır. **score >= 50** en üste alınsaydı 87 de, 95 de "FD" olurdu. **else** ise "hiçbiri değilse" demektir; koşulsuzdur ve en sonda durur.

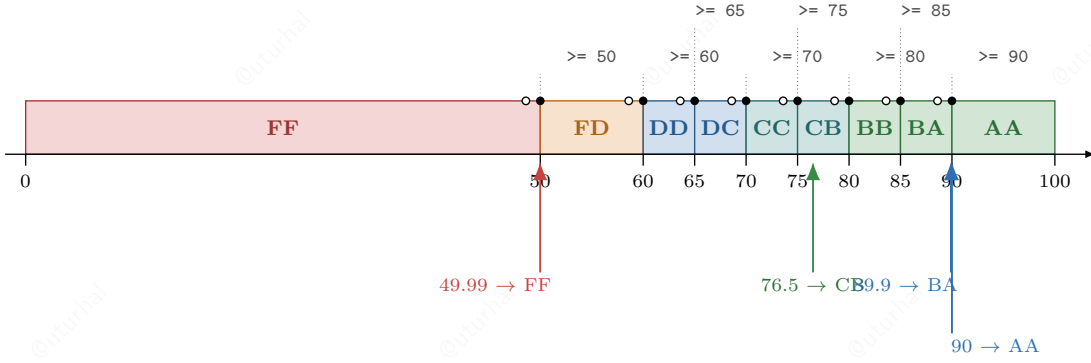
Şekil 6.3: if-elif-else zinciri. Koşullar yukarıdan aşağı sırayla sorulur; *ilk* doğru olanın bloğu çalışır ve zincirden çıkılır. **score = 87** için izlenen yol vurgulanmıştır: ikinci koşul doğru çıktığı için alttaki koşullar — doğru olsalar bile — hiç sorulmaz.



Sık yapılan hata

if score >= 50: ... elif score >= 90: ... → if score >= 90: ... elif score >= 50: ...

Zincirin sırası ters yazıldığında 95 alan öğrenci de, 52 alan öğrenci de aynı harfi alır: birinci koşul ikisi için de doğrudur ve akış oradan çıkar. Alttaki dallara *hiçbir girdi* ulaşamaz; bunlara **ulaşılamayan kod** (İng. *unreachable code*) denir. Bu hata çalıştırıldığında hata mesajı vermez — program sessizce yanlış sonuç üretir. Kural: >= ile yazılmış eşikler **büyükten küçüğe**, <= ile yazılmış eşikler **küçükten büyüğe** sıralanır.



Her **elif** bir aralığın sol ucunu söyler; sağ ucu bir önceki dal belirlemiştir. **elif** `score >= 85` satırına gelinmişse `score >= 90` zaten yanlış çıkmıştır; bu yüzden `85 <= score < 90` yazmaya gerek yoktur — zincir üst sınırı kendisi taşır. Dolu daire aralığa *dâhil* ($\geq$), boş daire *hariç* ($<$) demektir: 90 AA'dır, 89.99 BA. Programı test ederken her aralığın içinden bir sayı *ve* her eşikğin tam üstündeki sayıyı deneyin.

Şekil 6.4: Harf notu aralıkları sayı doğrusunda. Her **elif** bir aralığın *sol* ucunu söyler; sağ ucunu bir önceki dal belirlemiştir. Dolu daire aralığa *dâhil* ($\geq$), boş daire *hariç* ($<$) demektir.

Eşik değerli bir kuralı koda çevirmek

1. Kuralı bir tabloya dökün: her satırda bir aralık ve o aralığa düşen sonuç (3. haftanın *karar tablosu*); aralıkların tümü kapsansın, hiçbirisi üst üste binmesin.
2. Tabloyu eşiklerine göre sıralayın; $\geq$ kullanacaksanız büyükten küçüğe.
3. Her satırı bir **elif** yapın, *yalnız* sol sınırı yazın; son satırı koşulsuz **else** yapın.
4. Her eşik için üç sayı deneyin: eşikğin biraz altı, tam eşik, biraz üstü. Bunun adı **sınır değer analizidir** (*İng. boundary value analysis*) ve yazılım sınavının en verimli tek kuralıdır: hatalar aralığın ortasında değil, kenarlarında saklanır. Harf notu programında 89,9, 90 ve 90,1 girin; sırasıyla BA, AA, AA görmelisiniz. Ortadaki değer yanlış çıkıyorsa $>$ ile $\geq$ karıştırılmış demektir.

6.5 Mantıksal operatörler: *and*, *or*, *not*

Bazı kurallar tek karşılaştırmayla anlatılamaz: “yaşı 18’den büyük *ve* kimliği yanında olmalı”, “cumartesi *ya da* pazar olmalı”, “üye *olmayan* müşteri”. 3. haftada bunları **VE**, **VEYA**, **DEĞİL** ile yazmıştınız; Python’daki adları **and**, **or** ve **not**’tur.

Tanım 6.4: Mantıksal operatörler

a and b ifadesi *her iki* parça da **True** ise **True**’dur. **a or b** ifadesi parçalardan *en az biri* **True** ise **True**’dur. **not a** ise a’nın değerini tersine çevirir. Bir işlecin bütün girdi birleşimlerindeki sonucunu gösteren tabloya **doğruluk tablosu** (*İng. truth table*) denir. Öncelik sırası **not** $>$ **and** $>$ **or**’dur ve üçü de karşılaştırma operatörlerinden *sonra* değerlendirilir.

Kod 6.5 w06-05-mantiksal.py — **and**, **or**, **not**: doğruluk tablosu, öncelik, kısa devre

```
1 # w06-05-mantiksal.py -- and, or, not: doğruluk tablosu, öncelik, kısa devre
2
3 print("and:", True and True, True and False, False and False)
```



GitHub


```

4 print("or :", True or True, True or False, False or False)
5 print("not:", not True, not False)
6
7 # Gerçek koşullarla
8 age = 20
9 has_id = True
10 print("Sinemaya girebilir mi?", age >= 18 and has_id)
11
12 temperature = 31
13 print("Uyarı gerekli mi?", temperature < 0 or temperature > 30)
14
15 # Öncelik: not > and > or (parantez her zaman daha okunaklıdır)
16 print(True or False and False)      # True : and önce yapılır
17 print((True or False) and False)    # False
18
19 # Kısa devre: sol taraf sonucu belirliyorsa sağ taraf HİÇ hesaplanmaz
20 x = 0
21 print(x != 0 and 10 / x > 1)          # False -> 10 / 0 hiç denenmez
22 print(x == 0 or 10 / x > 1)          # True  -> yine denenmez

```

```
$ python w06-05-mantiksal.py
```

```

and: True False False
or : True True False
not: False True
Sinemaya girebilir mi? True
Uyarı gerekli mi? True
True
False
False
True

```

Önceliğin pratik sonucu şudur: `score >= 50 and score <= 100` ifadesinde parantez gerekmez, çünkü karşılaştırmalar `and`'den önce yapılır. Buna karşılık `True or False and False` ifadesi `True`, `(True or False) and False` ifadesi `False` verir (Şekil 6.5). Kararsız kaldığınız her yerde parantez kullanın. Bir değişkenin adı da *her* karşılaştırmada yazılmalıdır: `score >= 50 and <= 100` söz dizimi hatasıdır.

and — ikisi de

a	b	a and b
True	True	True
True	False	False
False	True	False
False	False	False

or — en az biri

a	b	a or b
True	True	True
True	False	True
False	True	True
False	False	False

not — tersi

a	not a
True	False
False	True

İki bool değişkenin *tam* dört birleşimi vardır; tablo hepsini listeler. Bir bileşik koşulu sınarken bu dört satırı da deneyin.

Öncelik (yüksekten düşüğe)

1	() parantez
2	+ - * / // % ** aritmetik
3	== != < <= > >= karşılaştırma
4	not
5	and
6	or

Örnek: not 3 > 5 or 2 * 3 == 6 and 0 < 1

① aritmetik	not 3 > 5 or 6 == 6 and 0 < 1
② karşılaştırma	not False or True and True
③ not	True or True and True
④ and	True or True
⑤ or	True

Karşılaştırmalar and/or'dan *önce* yapıldığı için a == 1 or a == 2 parantezsiz doğru çalışır. Ama not and'den önce geldiğinden not a and b, (not a) and b demektir — not (a and b) değil.

and bir koşulu daraltır, or genişletir. Bir and zincirine her yeni parça eklendiğinde koşulu sağlayan girdi sayısı azalır; bir or zincirine eklendiğinde artar. Tablolar 3. haftadaki VE/VEYA/DEĞİL tablolarının Python yazımıyla aynıdır; değişen yalnız işleçlerin adıdır. Öncelik sırasını ezberlemek zorunda değilsiniz: kararsız kaldığınız her yerde parantez kullanın, çünkü fazladan parantezin bedeli yoktur.

Şekil 6.5: and, or ve not işleçlerinin doğruluk tabloları, işlem önceliği merdiveni ve bileşik bir ifadenin önceliğe göre adım adım değerlendirilmesi.

❌ Sık yapılan hata

`gun == "Cumartesi" and gun == "Pazar" → gun == "Cumartesi" or gun == "Pazar"`
Günlük Türkçede “cumartesi ve pazar tatildir” deriz; mantıkta **and** “ikisi birden” demektir ve bir değişken aynı anda iki değer tutamayacağı için bu koşul *hiçbir zaman True* olmaz. Kural: aynı değişkenin farklı değerleri **or** ile, farklı değişkenlerin koşulları çoğu zaman **and** ile bağlanır. Daha tehlikeli bir biçimi `gun == "Cumartesi" or "Pazar"` yazmaktır: bu, *hata bile vermez*. “Pazar” boş olmayan bir metin olduğu için doğruluk değeri *True*’dur ve bütün koşul her gün için *True* çıkar — nedenini 6.6’da göreceksiniz.

💡 İpucu

Bir koşulun doğru yazılıp yazılmadığını anlamanın en hızlı yolu, onu *kabukta* tek başına denemektir: değişkenlere örnek değerler verin ve koşulu **if** olmadan yazın. *True/False* çıktısı, programın tamamını çalıştırmadan doğru daldan geçip geçmeyeceğinizi söyler. Karmaşık bir bileşik koşulu ise parçalara bölüp her parçayı ayrı bir *bool* değişkende saklayın (6.8’deki şifre örneğinde olduğu gibi): hem okunur hem de hangi parçanın yanlış olduğunu kabukta tek tek görebilirsiniz.

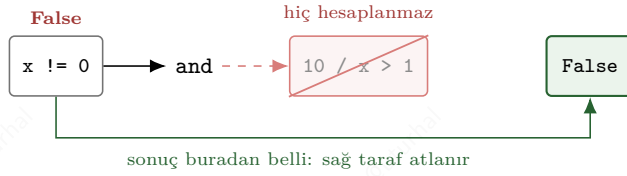
6.5.1 Kısa devre değerlendirme

Python bileşik bir koşulu soldan sağa okur ve sonucu belirlediği anda durur. **and** için soldaki *False*, **or** için soldaki *True* kararı tek başına verir; kalan parçalar *hiç hesaplanmaz*. Buna **kısa devre değerlendirme** (İng. *short-circuit evaluation*) denir. **Kod 6.5**’in son iki satırı bunu gösterir: `x = 0` iken `x != 0 and 10 / x > 1` ifadesi *False* verir ve program çökmez. Bu, bir süsleme değil *aracıdır*: tehlikeli işlemi, onu güvenli kılan koşulun **sağına** yazarsınız

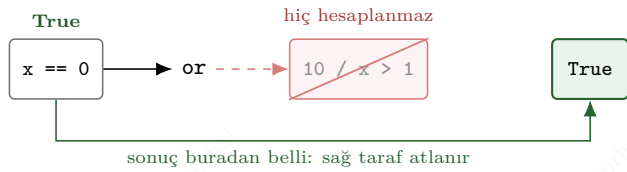
(Şekil 6.6). 7. haftada kullanıcı girdisini doğrularken bu deseni çok kullanacağız.

$x = 0$ iken iki bileşik koşul

① *and*: soldaki False ise sonuç zaten False



② *or*: soldaki True ise sonuç zaten True



Koruma deseni

```
if y != 0 and x / y > 1:
    print("büyük")
```

Sıra önemlidir: koruyan koşul ($y \neq 0$) her zaman solda durur. Yer değiştirirse x / y önce hesaplanır ve program `ZeroDivisionError` ile çöker.

Aynı desen *or* ile de kurulur: $y == 0$ or $x / y > 1$ ifadesinde de bölme yalnız güvenliyken yapılır.

Python bileşik koşulu soldan sağa okur ve sonucu belirlediği anda durur. Buna *kısa devre değerlendirme* denir. Pratik sonucu şudur: bir bölmeyi, bir indekslemeyi ya da uzun süren bir hesabı, onu güvenli kılan koşulun *sağına* yazarsanız tehlikeli işlem hiç çalışmaz. *and* için soldaki False, *or* için soldaki True kararı tek başına verir.

Şekil 6.6: Kısa devre değerlendirme. $x = 0$ iken her iki bileşik koşulda da sağ taraftaki bölme hiç yapılmaz; bu yüzden `ZeroDivisionError` oluşmaz. Sağda, bu davranışın bir koruma desenine nasıl dönüştüğü.

Örnek 6.1 (Artık yıl). 3. haftada sözde koduyla tasarladığınız artık yıl kuralını Python'a çevirin: dörde bölünen yıllar artıktır; ancak yüze bölünenler artık değildir; ancak dört yüze bölünenler yine artıktır. 2024, 1900, 2000 ve 2023 ile sınavın.

Çözüm

Analiz. Girdi: *year* (tam sayı, *int*). Çıktı: iki metinden biri. “*b*’ye bölünüyor” koşulunun Python karşılığı $\text{year \% b} == 0$ ’dır. Üç parçalı kural tek bir bileşik koşula sığar: (*dörde bölünüyor and yüze bölünmüyor*) *or* *dört yüze bölünüyor*. Koşulu ayrı bir *is_leap* değişkeninde saklamak, hem *if* satırını kısaltır hem de sonucu yeniden kullanılabilir kılar.

Algoritma.

Sözde kod: Artık yıl

BAŞLA

OKU *year*

$\text{artık} \leftarrow (\text{year mod } 4 = 0 \text{ VE } \text{year mod } 100 \neq 0) \text{ VEYA } \text{year mod } 400 = 0$

EĞER *artık* İSE

YAZ *year*, “artık yıldır”

DEĞİLSE

YAZ *year*, “artık yıl değildir”

EĞER SONU

BİTİR

Kod ve çıktı.

Kod 6.6 w06-06-artik-yil.py — Artık yıl: *and* ve *or* ile bileşik koşul

```

1 # w06-06-artik-yil.py -- artık yıl: and / or ile bileşik koşul
2
3 year = int(input("Yıl: "))
4
5 is_leap = (year % 4 == 0 and year % 100 != 0) or year % 400 == 0
6
7 if is_leap:
8     print(f"{year} artık yıldır: Şubat 29 gün çeker.")
9 else:
10    print(f"{year} artık yıl değildir: Şubat 28 gün çeker.")

```

```
$ python w06-06-artik-yil.py
```

```

Yıl: 2024
2024 artık yıldır: Şubat 29 gün çeker.

```

Kontrol. 2024: $2024 \bmod 4 = 0$ ve $2024 \bmod 100 = 24 \neq 0$, parantez içi **True** $\Rightarrow$ artık ✓. 1900: parantez içi **False** ($1900 \bmod 100 = 0$) ve $1900 \bmod 400 = 300 \neq 0 \Rightarrow$ artık değil ✓. 2000: parantez içi **False** ama $2000 \bmod 400 = 0 \Rightarrow$ artık ✓. 2023: $2023 \bmod 4 = 3 \Rightarrow$ artık değil ✓. Tarih de doğrudur: 2000 yılında 29 Şubat yaşandı, 1900'de yaşanmadı. Büyüklük mertebesi: dört yüz yılda $400/4 = 100$ adaydan üç yüzyıl dönümü düşülür, 97 artık yıl kalır; ortalama yıl uzunluğu $365 + 97/400 = 365,2425$ gün olur ve gerçek değere (365,2422) çok yakındır ✓.

Örnek 6.2 (Üçgen eşitsizliği). Üç kenar uzunluğu alan; bu kenarlarla bir üçgen oluşup oluşmadığını söyleyen, oluşuyorsa üçgenin türünü (eşkenar, ikizkenar, çeşitkenar) yazan bir program yazın. Örnek: 5, 5, 8.

Çözüm

Analiz. Girdi: üç kenar (float). Çıktı: bir metin. Önce *geçerlilik*: kenarlar pozitif olmalı ve **üçgen eşitsizliği** sağlanmalıdır — herhangi iki kenarın toplamı üçüncüden büyük olmalı. Üç eşitsizliğin hepsi gerekli olduğundan bozulmayı *or* ile yakalamak daha kısadır. Sonra *tür*: eşkenar aynı zamanda ikizkenar koşulunu da sağladığı için eşkenar önce sorulmalıdır.

Algoritma. (Yer kazanmak için dallar tek satırda gösterilmiştir.)

Sözde kod: Üçgen türü

```

OKU a, b, c
EĞER a ≤ 0 VEYA b ≤ 0 VEYA c ≤ 0 İSE YAZ "pozitif olmalı"
DEĞİLSE EĞER a+b ≤ c VEYA a+c ≤ b VEYA b+c ≤ a İSE YAZ "üçgen oluşmaz"
DEĞİLSE EĞER a = b = c İSE YAZ "eşkenar"
DEĞİLSE EĞER a = b VEYA a = c VEYA b = c İSE YAZ "ikizkenar"
DEĞİLSE YAZ "çeşitkenar"
EĞER SONU

```

Kod ve çıktı.

Kod 6.7 w06-07-ucgen.py — Üçgen eşitsizliği ve üçgen türü: zincirleme eşitlik

```

1 # w06-07-ucgen.py -- üçgen eşitsizliği ve üçgen türü
2
3 a = float(input("1. kenar: "))
4 b = float(input("2. kenar: "))
5 c = float(input("3. kenar: "))
6
7 if a <= 0 or b <= 0 or c <= 0:
8     print("Kenar uzunlukları pozitif olmalıdır.")
9 elif a + b <= c or a + c <= b or b + c <= a:
10    print("Bu üç kenarla üçgen oluşmaz.")
11 elif a == b == c:
12    print("Eşkenar üçgen.")
13 elif a == b or a == c or b == c:
14    print("İkizkenar üçgen.")
15 else:
16    print("Çeşitkenar üçgen.")

```

```
$ python w06-07-ucgen.py
```

```

1. kenar: 5
2. kenar: 5
3. kenar: 8
İkizkenar üçgen.

```

Kontrol. 5, 5, 8: kenarlar pozitif ve $5 + 5 = 10 > 8$ — üçgen oluşur ✓; üçü eşit değil ama $a = b$ olduğu için ikizkenar ✓. Sınır durumu: 2, 3, 5 girildiğinde $2 + 3$ tam olarak 5'e eşittir, $a + b \leq c$ doğrudur ve program "üçgen oluşmaz" der — doğru cevap budur, çünkü üç nokta aynı doğru üzerindedir (*yozlaşmış* üçgen). 3, 3, 3 eşkenar; 3, 4, 5 çeşitkenar (ve dik) çıkmalıdır. Koddaki $a == b == c$ yazımı 6.1'deki zincirleme karşılaştırmanın eşitlik hâlidir; $a == b$ **and** $b == c$ ile aynı anlama gelir. `float` kenarlarla çalıştığımız için $a == b$ karşılaştırması tehlikeli olabilir; bunu 6.7'de tartışıyoruz.

6.6 Doğruluk değeri olan nesneler ve koşullu ifade

`if` satırındaki ifadenin `bool` olması *zorunlu* değildir. Python her nesneye bir doğruluk değeri atar ve koşulda kullanıldığında o değeri kullanır.

Tanım 6.5: Doğruluk değeri

Bir nesnenin **doğruluk değeri** (*Ing. truthiness*), `bool()` ile öğrenilir. Şunlar **False** sayılır: 0, 0.0, boş metin `""` ve `None` ("değer yok" anlamına gelen özel nesne; 11. haftada yeniden karşılaştacağız). *Bunların dışında kalan her şey True* sayılır: sıfırdan farklı her sayı (negatifler dâhil), boş olmayan her metin — `"0"` ve `" "` bile.



GitHub

Kod 6.8 w06-08-dogruluk-degeri.py — Doğruluk değeri olan nesneler, if x: deseni ve koşullu ifade

```

1 # w06-08-dogruluk-degeri.py -- doğruluk değeri olan nesneler ve koşullu ifade
2
3 print(bool(0), bool(0.0), bool(""), bool(None))      # hepsi False
4 print(bool(7), bool(-1), bool("0"), bool(" "))      # hepsi True
5
6 name = input("Adınız (boş bırakabilirsiniz): ")
7
8 if name:                                             # name boş metin değilse True
9     print(f"Merhaba, {name}!")
10 else:
11     print("Merhaba, yabancı!")
12
13 # Koşullu ifade: değer seçen tek satırlık if-else
14 score = 58
15 status = "geçti" if score >= 50 else "kaldı"
16 print("Durum:", status)
17
18 n = -4
19 print("Mutlak değer:", n if n >= 0 else -n)

```

```
$ python w06-08-dogruluk-degeri.py
```

```

False False False False
True True True True
Adınız (boş bırakabilirsiniz):
Merhaba, yabancı!
Durum: geçti
Mutlak değer: 4

```

Kod 6.8'nin ortasındaki `if name:` satırı bu bölümün en çok kullanılan desenidir ve “name boş değilse” diye okunur. `if name != "":` yazımı da doğrudur; deneyimli Python programcıları kısa olanı tercih eder. Programı boş bir girdiyle (doğrudan **Enter**) çalıştırdığımız için çıktıda “Merhaba, yabancı!” görünür.

6.6.1 Koşullu ifade

Bazen tek yapmak istediğimiz şey, bir koşula göre *iki değerden birini seçmektir*. Bunun için dört satırlık bir if-else yerine tek satırlık **koşullu ifade** (İng. *conditional expression*) kullanılabilir:

doğruysa_değer if koşul else yanlışsa_değer

Örneğin `status = "geçti" if score >= 50 else "kaldı"` satırı, dört satırlık bir if-else bloğunun yaptığını yapar. Buradaki if bir *deyim* değil bir *ifadedir*: bir değeri vardır, bu yüzden atamanın sağında, bir `print()` çağrısının içinde ya da bir f-string'in küme parantezinde kullanılabilir. Ama onu yalnız *değer seçerken* ve satır kısa kaldığında kullanın; iki dalda farklı işler yapılacaksa normal if-else yazın.

Sık yapılan hata

`if passed == True:` → `if passed:`

`passed` zaten `True` ya da `False` olduğuna göre onu bir kez daha `True` ile karşılaştırmak gereksizdir. Aynı biçimde `if passed == False:` yerine `if not passed:` yazılır. Bu yalnız bir üslup meselesi değildir: `if passed = True:` diye yazmak çok kolaydır ve o

zaman program hiç çalışmaz.

6.7 İki tuzak: float eşitliği ve = ile ==

6.7.1 float değerleri == ile karşılaştırmayın

4. haftada $0.1 + 0.2$ toplamının 0.30000000000000004 verdiğini görmüştünüz. O zaman bu yalnız tuhaf bir çıktıydı; şimdi bir *karar* verecek olduğumuz için tehlikeli hâle geliyor:

Python etkileşimli kabuk (REPL)

```
>>> 0.1 + 0.2 == 0.3
False
>>> abs((0.1 + 0.2) - 0.3) < 1e-9
True
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
```

Birinci satır, hiçbir hata mesajı vermeden yanlış görünen bir sonuç üretir — oysa doğrudur: float sayıları ikili tabanda sonlu sayıda basamakla tutulur, 0,1 ile 0,2 yuvarlanır ve iki yuvarlanmış sayının toplamı üçüncü bir yuvarlanmış sayıya tam olarak eşit çıkmak zorunda değildir.

İki float değeri karşılaştırmak

1. **Eşitlik** sorulacaksa `==` kullanmayın; farkın mutlak değerine bakın: `if abs(a - b) < 1e-9:`. Buradaki 10^{-9} sayısına **tolerans** denir. Hazır çözüm `math.isclose(a, b)`'dir; sifıra çok yakın değerlerde `abs_tol=1e-9` verilmelidir.
2. **Büyüklik** sorulacaksa (`<`, `>`, `>=`) sorun yoktur; harf notu zincirindeki `>=` bu yüzden sorunsuz çalışır.
3. Para hesaplarında en güvenli yol, tutarı kuruş cinsinden *tam sayı* tutmaktır.

6.7.2 = atar, == sorar

3. haftada sözde kodda atamayı $\leftarrow$ oku, karşılaştırmayı `=` ile yazıyorduk. Python'da ayrım `=` ile `==` arasındadır ve yeni öğrenenlerin en sık karıştırdığı iki işaret bunlardır.

✗ Sık yapılan hata

`if score = 90:` → `if score == 90:`

Python bu hatayı *çalıştırmadan önce* yakalar ve `SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?` der — üstelik ne demek istediğinizi de tahmin eder. C ve Java gibi dillerde aynı yazım geçerli olduğu için hata sessizce geçer ve saatlerce aranır; Python'un katılığı burada sizi korur. Terside olur: `score == 90` satırını tek başına yazarsanız program hata vermez ama hiçbir şey de yapmaz — hesaplanan `True` değeri hiçbir yere gitmez.



“Eşittir atar, çift eşittir sorar.” 3. haftada sözde kodda bu ayrımı ← oku ile = arasında yapmıştık; Python’da ayrım = ile == arasındadır. İyi haber: Python bu hatayı *derleme anında* yakalar ve yukarıdaki gibi ne demek istediğinizi bile önerir — C ve Java gibi dillerde aynı yazım geçerli olduğu için hata sessizce geçer. Kötü haber: `if` satırındaki hata yüzünden program hiç başlamaz.

Şekil 6.7: `=` bir *komuttur*: sağdaki değeri hesaplar, soldaki ada bağlar ve geriye bir değer bırakmaz. `==` bir *sorudur*: iki değeri karşılaştırır ve `True` ya da `False` üretir. Altta, ikisini karıştırmamızın verdiği hata mesajı.

6.8 Uygulamalar

Bu bölümdeki örnekler haftanın bütün parçalarını bir araya getirir: karşılaştırma, `elif` zinciri, mantıksal işlemler ve 5. haftanın metin metotları.

Örnek 6.3 (Dört işlem yapan hesap makinesi). Kullanıcıdan iki sayı ve bir işlem işareti (+, -, *, /) alan; sonucu yazan bir program yazın. Sıfıra bölme denirse hesap yapmak yerine anlaşılır bir uyarı verilmeli; tanınmayan işaretler de yakalanmalıdır.

Çözüm

Analiz. Girdi: iki `float` ve bir `str`. Çıktı: tek satır. Dört işlem dört ayrı daldır; zincirin *sırası* burada sayısal eşiklerden değil *özel durumdan* gelir: “bölme **and** bölen sıfır” dalı, genel bölme dalından **önce** yazılmalıdır, yoksa program `ZeroDivisionError` ile çöker. Son `else` dalı, işaret tanınmadığında devreye girer — bir zincirin sonunda “beklenmeyen durum” dalı bırakmak iyi bir alışkanlıktır.

Algoritma.

Sözde kod: Hesap makinesi

```
OKU x, op, y
EĞER op = "+" İSE YAZ x + y
DEĞİLSE EĞER op = "-" İSE YAZ x - y
DEĞİLSE EĞER op = "*" İSE YAZ x * y
DEĞİLSE EĞER op = "/" VE y = 0 İSE YAZ "sıfıra bölme"
DEĞİLSE EĞER op = "/" İSE YAZ x / y
DEĞİLSE YAZ "tanınmayan işlem"
EĞER SONU
```

Kod ve çıktı.


Kod 6.9 w06-09-hesap-makinesi.py — Dört işlem ve sıfıra bölme koruması

```

1 # w06-09-hesap-makinesi.py -- dört işlem; sıfıra bölme kontrolü
2
3 x = float(input("Birinci sayı: "))
4 op = input("İşlem (+, -, *, /): ")
5 y = float(input("İkinci sayı: "))
6
7 if op == "+":
8     print(f"{x} + {y} = {x + y}")
9 elif op == "-":
10    print(f"{x} - {y} = {x - y}")
11 elif op == "*":
12    print(f"{x} * {y} = {x * y}")
13 elif op == "/" and y == 0:          # genel bölme dalından ÖNCE
14    print("Hata: sıfıra bölme tanımsızdır.")
15 elif op == "/":
16    print(f"{x} / {y} = {x / y}")
17 else:
18    print(f"Tanınmayan işlem: {op}")

```

```
$ python w06-09-hesap-makinesi.py
```

```

Birinci sayı: 15
İşlem (+, -, *, /): /
İkinci sayı: 4
15.0 / 4.0 = 3.75

```

Kontrol. 15 ve 4 ile bölme: $15/4 = 3,75$ ✓; girdiler float'a çevrildiği için çıktıda $15.0 / 4.0 = 3.75$ görünür. Snamanız gereken durumlar: dört işlemin her biri ($2 + 3 = 5$, $2 - 3 = -1$, $2 * 3 = 6$, $6 / 3 = 2.0$), bölen sıfır ($5 / 0 \rightarrow$ uyarı, çökme yok), tanınmayan işaret (^) ve dal sırası — iki bölme dalının yerini değiştirip $5 / 0$ girin: program bu kez çöker.

Örnek 6.4 (Vücut kitle indeksini sınıflandırmak). 4. haftada VKİ'yi hesaplayan bir program yazmıştınız. Şimdi onu genişletin: hesaplanan değeri *zayıf* ($< 18,5$), *normal* ($18,5-25$), *fazla kilolu* ($25-30$) ve *obez* (≥ 30) sınıflarından birine yerleştirsin. Örnek: 70 kg, 1,75 m.

Çözüm

Analiz. Girdi: kütle ve boy (float). Ara değer: $VKİ = \text{kütle} / \text{boy}^2$. Çıktı: VKİ ve sınıf adı. Dört sınıf, dört dal $\rightarrow$ elif zinciri; bu kez zinciri küçükten büyüğe kuruyoruz ($<$ ile). İkinci ve üçüncü dalda zincirleme karşılaştırma ($18.5 \leq \text{bmi} < 25$) kullanmak *gereksizdir* — oraya ancak $\text{bmi} \geq 18.5$ iken gelinir — ama kuralı okuyan insana aralığı bütünüyle gösterdiği için burada bilerek öyle yazdık.

Algoritma. OKU weight, boy; $\text{bmi} \leftarrow \text{weight} / \text{boy}^2$; dört dallı **EĞER-DEĞİLSE EĞER** zinciriyle sınıfı seç; **YAZ** bmi, sınıf.

Kod ve çıktı.

Kod 6.10 w06-10-vki-sinifi.py — VKİ hesabı ve dört dallı sınıflandırma

```

1 # w06-10-vki-sinifi.py -- VKİ hesabı + dört dallı sınıflandırma
2
3 weight = float(input("Kütleniz (kg): "))
4 height = float(input("Boyunuz (m): "))
5
6 bmi = weight / height ** 2
7
8 if bmi < 18.5:
9     category = "zayıf"
10 elif 18.5 <= bmi < 25:
11     category = "normal"
12 elif 25 <= bmi < 30:
13     category = "fazla kilolu"
14 else:
15     category = "obez"
16
17 print(f"VKİ: {bmi:.2f} -> {category}")

```

```
$ python w06-10-vki-sinifi.py
```

```

Kütleniz (kg): 70
Boyunuz (m): 1.75
VKİ: 22.86 -> normal

```

Kontrol. $70/1,75^2 = 70/3,0625 = 22,857... \approx 22,86$ ✓ ve $18,5 \leq 22,86 < 25$ olduğu için “normal” ✓. Sınırları üreten girdiler kolayca bulunur: boy 1,75 m ise $18,5 \cdot 3,0625 \approx 56,66$ kg, $25 \cdot 3,0625 \approx 76,56$ kg ve $30 \cdot 3,0625 \approx 91,88$ kg; bu üç kütleyle sırasıyla “normal”, “fazla kilolu” ve “obez” çıkmalıdır. float hesapları yüzünden tam sınırı yakalamak zor olabilir; bu yüzden sınıflandırmada < ve >= kullanılır, == değil.

Örnek 6.5 (Şifre kurallarını denetlemek). Bir kullanıcıdan yeni şifre alan ve şu kuralların hepsini sağlayıp sağlamadığını denetleyen bir program yazın: en az 8 karakter, yalnız harflerden oluşmamalı, yalnız rakamlardan oluşmamalı, en az bir büyük harf içermeli, boşluk içermemeli. Şifre reddedilirse *hangi* kuralın bozulduğu söylenmeli.

Çözüm

Analiz. Girdi: bir metin (input() zaten str döndürür). Çıktı: tek satır. Beş kuralın her birini 5. haftanın araçlarıyla yazabiliriz: uzunluk için len(), “yalnız harf” için isalpha(), “yalnız rakam” için isdigit(), büyük harf için password != password.lower() (küçük harfe çevrilmiş hâli kendisinden farklıysa içinde en az bir büyük harf var demektir), boşluk için " " not in password. Her kuralı ayrı bir bool değişkende saklamak koda okunabilir adlar kazandırır; kabul koşulu bu değişkenlerin and ile birleşimidir. Reddetme dallarını elif zinciriyle sıralarız, böylece kullanıcıya ilk bozulan kural söylenir.

Algoritma. Beş bool değişken hesapla; hepsi doğruysa kabul et; değilse sırayla hangisinin False olduğuna bak ve ona uygun mesajı yaz.

Kod ve çıktı.

Kod 6.11 w06-11-sifre-kontrolu.py — Şifre kuralları: adlandırılmış bool değişkenler ve and

```

1 # w06-11-sifre-kontrolu.py -- şifre kuralları: metotlar + and/or
2
3 password = input("Yeni şifre: ")
4
5 # Her kural ayrı bir bool değişkende: koşul okunur hâle gelir
6 long_enough = len(password) >= 8
7 not_only_letters = not password.isalpha()
8 not_only_digits = not password.isdigit()
9 has_upper = password != password.lower()
10 no_space = " " not in password
11
12 ok = (long_enough and not_only_letters and not_only_digits
13       and has_upper and no_space)
14
15 if ok:
16     print("Şifre kabul edildi.")
17 elif not long_enough:
18     print(f"Reddedildi: 8 karakter gerek, {len(password)} var.")
19 elif not (not_only_letters and not_only_digits):
20     print("Reddedildi: hem harf hem rakam bulunmalı.")
21 elif not has_upper:
22     print("Reddedildi: en az bir büyük harf bulunmalı.")
23 else:
24     print("Reddedildi: boşluk kullanılamaz.")

```

```
$ python w06-11-sifre-kontrolu.py
```

```

Yeni şifre: trabzon2026
Reddedildi: en az bir büyük harf bulunmalı.

```

Kontrol. Girdi trabzon2026: uzunluk $11 \geq 8$ ✓, yalnız harf değil ✓, yalnız rakam değil ✓, büyük harf ✗ — program doğru dala girer ✓. Başka girdiler: Trabzon2026 kabul edilmeli; Ayse12 uzunluk dalına ($6 < 8$), Trabzon Sehri boşluk dalına, 12345678 “hem harf hem rakam” dalına düşmeli. ayse1234 ise yalnız harf de yalnız rakam da olmadığı için üçüncü kuralı geçer ve büyük harf dalında takılır — kuralların ne söylediğini tam okumak önemlidir.

6.8.1 Yaşa ve güne göre bilet ücreti

Son uygulama, bir karar tablosunun doğrudan koda çevrilmesidir. Bir sinemanın tam bilet ücreti 180 TL’dir; çarşamba günü herkese %50, 12 yaşından küçüklerle 65 yaş ve üstüne %40, 26 yaşından küçüklerle %20 indirim uygulanır ve indirimler birleştirilmez — bu, doğrudan bir elif zinciri demektir. “En avantajlı kural uygulanır” ise sırayı belirler: en büyük indirim en üste yazılır. Gün adı büyük ya da küçük harfle yazılabileceği için karşılaştırmadan önce 5. haf-tanın lower() metodunu uyguluyoruz; yaş kuralındaki iki ayrı aralık $\text{age} < 12$ or $\text{age} \geq 65$ ile tek dalda birleşir.



Kod 6.12 w06-12-bilet.py — Karar tablosundan koda: bilet ücreti

```

1 # w06-12-bilet.py -- sinema bileti: yaş ve güne göre fiyat
2
3 FULL_PRICE = 180                                # TL
4
5 age = int(input("Yaş: "))
6 day = input("Gün (örn. Salı): ").lower()
7
8 if day == "çarşamba":                            # halk günü: yarı fiyat
9     price = FULL_PRICE * 0.5
10 elif age < 12 or age >= 65:                      # çocuk ve 65 yaş üstü
11     price = FULL_PRICE * 0.6
12 elif age <= 25:                                  # genç
13     price = FULL_PRICE * 0.8
14 else:
15     price = FULL_PRICE
16
17 print(f"Bilet fiyatı: {price:.2f} TL")

```

```
$ python w06-12-bilet.py
```

```

Yaş: 20
Gün (örn. Salı): Cumartesi
Bilet fiyatı: 144.00 TL

```

Kod 6.12’i sekiz girdiyle sınavalım. 20 yaş cumartesi: çarşamba değil, $20 \leq 25 \Rightarrow$ çarpan 0,8 ve $180 \cdot 0,8 = 144,00$ TL ✓. 20 yaş çarşamba $\rightarrow 90,00$ (gün kuralı yaş kuralını bastırır); 10 yaş $\rightarrow 108,00$; 70 yaş $\rightarrow 108,00$; 40 yaş $\rightarrow 180,00$. Sınır değerleri: 12 yaş $\rightarrow 144,00$ (çocuk indirimi biter, gençlik indirimi başlar), 25 yaş $\rightarrow 144,00$, 26 yaş $\rightarrow 180,00$, 65 yaş $\rightarrow 108,00$. Sekizinin hepsi karar tablosuyla uyuyor ✓.

LAB Laboratuvar Çalışması 6: Kâğıttan koda

Önce 3. haftada *kâğıt üzerinde* tasarladığınız iki karar algoritmasını çalışan Python programına çevireceksiniz, sonra dört yeni program yazacaksınız. Her görevde programı **sınır değerleriyle** sınavın ve denediğiniz girdi-çıkı çiftlerini bir tabloya yazın; tablo olmadan görev tamamlanmış sayılmaz.

Görev 1. Akış diyagramından koda: indirim. 3. haftadaki “tutara göre indirim” algoritmasını (sepet 500 TL ve üzerindeyse %10 indirim) `indirim.py` olarak yazın. Girdi 640: indirim 64.00, ödenecek 576.00. Sınır: 500 girildiğinde indirim 50.00 (koşul $\geq$ olduğu için *uygulanır*), 499 girildiğinde 0.00.

Görev 2. Karar tablosundan koda: müze bileti. 3. haftadaki müze bileti karar tablosunu (< 7 yaş: 0 TL; 7–17: 40 TL; 18–64: 80 TL; ≥ 65 : 20 TL) `muze.py` olarak yazın; her satır bir `elif` olmalı. Altı sınır değerini deneyin: 6, 7, 17, 18, 64, 65.

Görev 3. Harf notu ve sınır testleri. Kod 6.4’nu kendiniz yazın (kopyalamayın) ve bir test tablosu doldurun: sekiz eşğin (50, 60, 65, 70, 75, 80, 85, 90) her biri için eşğin 0,1 altı, tam eşik ve 0,1 üstü. Beklenen ile gözlenen harf notu ayrıldığı yerde $>$ ile $\geq$ karışmış demektir. Son olarak `elif score >= 50` satırını zincirin en üstüne taşıyın, 95 girin ve ne olduğunu bir cümleyle açıklayın.

Görev 4. Hesap makinesi. Örnek 6.3'teki programı kendi başınıza yeniden yazın (hesap.py): iki sayı ve bir işaret alıp sonucu yazsın; sıfıra bölmeyi yakala-sın; tanınmayan işarete uyarı versin. Beş girdiyle deneyin: $2 + 3$, $7 - 10$, $6 * 0$, $5 / 0$, $4 ^ 2$.

Görev 5. Artık yıl. Örnek 6.1'i kapatıp artık.py dosyasını sıfırdan yazın: bir yıl alıp artık olup olmadığını yazsın. 1900, 2000, 2023, 2024 ve kendi doğum yılınızla deneyin. Ek: o yılın Şubat ayının kaç gün çektiğini koşullu ifadeyle tek satırda yazdırın.

Görev 6. Şifre gücü. şifre.py: bir şifre alıp *zayıf*, *orta* ya da *güçlü* diye sınıflandırsın. Ölçütü kendiniz seçip dosyanın başına yorum satırıyla yazın; en az üç kural ve bir `elif` zinciri bulunsun. Sınıf arkadaşınızın programına ölçütünü bozan bir şifre bulmaya çalışın.

Teslim. Görev 1–3'ü Kısa Sınav 2'den önce bitirin; sınav bu becerileri ölçer. Dosyaları saklayın: 7. haftada hesap makinesine menü, 8. haftada tekrar eklenecek.

Özet

Yapı	Yazım	Not
Karşılaştırma	<code>== != < <= > >=</code> <code>0 <= x < 10</code>	sonuç her zaman <code>bool</code> ; zincir <code>and</code> ile aynı
Tek/iki dallı karar	<code>if koşul: ... else: ...</code>	<code>else</code> koşulsuzdur, <code>if</code> ile aynı hi-zada
Çok dallı karar	<code>if ...: elif ...: else:</code>	ilk doğru dal çalışır; sıra anlamı belirler
Blok	4 boşluk girinti	başlık satırı : ile biter; sekme kul-lanmayın
Mantıksal	<code>and or not</code>	öncelik <code>not > and > or</code>
Kısa devre	<code>y != 0 and x / y > 1</code>	koruyan koşul solda durur
Doğruluk değeri	<code>0 0.0 "" None → False</code>	geri kalan her şey <code>True</code> ; <code>if x:</code>
Koşullu ifade	<code>a if koşul else b</code>	<i>değer</i> seçer; deyim değil ifadedir
float eşitliği	<code>abs(a - b) < 1e-9</code>	<code>==</code> kullanmayın; <code>math.isclose</code> da olur
Atama vs. soru	<code>x = 5</code> <code>x == 5</code>	“eşittir atar, çift eşittir sorar”
Sınama	her eşikte altı, tam, üstü	sınır değer analizi
Hata türleri	<code>IndentationError</code> <code>TabError</code> <code>SyntaxError</code>	girinti ve iki nokta hataları



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w06>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın ya-nında beklenen çıktı `out/` klasöründedir.

Alıştırımlar

Alıştırma 6.1. ●●● Aşağıdaki ifadelerin değerlerini (True ya da False) çalıştırmadan yazın, sonra kabukta doğrulayın.

- | | |
|----------------|------------------------------|
| (a) $7 > 3$ | (e) $2 < 5 < 9$ |
| (b) $7 == 7.0$ | (f) $5 < 2 < 9$ |
| (c) $"7" == 7$ | (g) $"abc" < "abd"$ |
| (d) $3 != 3$ | (h) $\text{not } (4 \geq 4)$ |

Cevap: (a) True (b) True (c) False (d) False (e) True (f) False (g) True (h) False

Alıştırma 6.2. ●●● Aşağıdaki zincir bir öğrencinin harf notunu belirlemek için yazılmış, ama sırası yanlıştır (bloklar yer kazanmak için başlık satırlarının yanında gösterilmiştir).

```
if score >= 60:    grade = "DD"
elif score >= 70:  grade = "CC"
elif score >= 85:  grade = "BA"
elif score >= 90:  grade = "AA"
else:              grade = "FF"
```

- (a) `score = 95` için hangi harf atanır? (b) Hangi dallara *hiçbir* girdiyle ulaşamaz? (c) Programı çalıştırdığımızda neden hata mesajı almazsınız? (d) Zinciri düzeltin.

Cevap: (a) DD (b) CC, BA, AA (c) söz dizimi değil anlam bozuk (d) eşikleri büyükten küçüğe sıralayın

Alıştırma 6.3. ●●● Bir sayı alıp “pozitif”, “negatif” ya da “sıfır” yazan bir program yazın. Girdi $-3,5$ için çıktı: **negatif**.

Cevap: üç dal: `if n > 0`, `elif n < 0`, koşulsuz `else`

Alıştırma 6.4. ●●● Aşağıdaki ifadelerin değerlerini elle bulun. (f)–(h) şıklarında $x = 0$ 'dır; bunlarda ayrıca “sağ taraf hesaplanır mı?” sorusunu da cevaplayın.

- | | |
|--|---|
| (a) <code>True and not False</code> | (e) <code>not (5 > 3 and 2 > 4)</code> |
| (b) <code>False or 3 > 2</code> | (f) <code>x != 0 and 10 / x > 1</code> |
| (c) <code>not True or True</code> | (g) <code>x == 0 or 10 / x > 1</code> |
| (d) <code>5 > 3 and 2 > 4</code> | (h) <code>"a" in "kalem" or 1 / 0 > 0</code> |

Cevap: (a) True (b) True (c) True (d) False (e) True (f) False, sağ taraf hesaplanmaz (g) True, hesaplanmaz (h) True, hesaplanmaz

Alıştırma 6.5. ●●● Üç sayı alıp en büyüğünü yazan bir program yazın; `max()` kullanmayın. Programın sonuna `max()` ile bir kontrol satırı ekleyin. Girdi 12, 47, 33: En büyük: 47.0.

Cevap: `a >= b and a >= c, sonra b >= a and b >= c, sonra else; eşitlik için >= kullanın`

Alıştırma 6.6. ●●● Sepet tutarına göre indirim uygulayan bir program yazın: 1000 TL ve üzeri %20, 500 TL ve üzeri %10, altı indirimli. İndirim tutarını ve ödenecek tutarı iki ondalık basamakla, sağa dayalı yazın. Girdi 750: indirim 75.00, ödenecek 675.00.

Cevap: $750 \cdot 0,10 = 75$, ödenecek 675; zincirde önce ≥ 1000 , sonra ≥ 500

Alıştırma 6.7. ●●● Kademeli su faturası hesaplayın: ilk 10 m³ 12 TL/m³, sonraki 10 m³ 18 TL/m³, 20 m³ üstü 25 TL/m³. Kademeler *birikimlidir*: 27 m³ tüketen abone tüm tüketimi için 25 TL değil, kademelerin toplamını öder. Girdi 27: 475.00 TL.

Cevap: $10 \cdot 12 + 10 \cdot 18 + 7 \cdot 25 = 120 + 180 + 175 = 475$

Alıştırma 6.8. ●●● Aşağıdaki iki program parçasını inceleyin.

(A) `n = int(input("Sayı: "))`
`if n > 0:`
`print("pozitif")`
`print("işlem tamam")`

(B) `total = 0`
`score = 40`
`if score >= 50:`
`total = total + score`
`print("Eklendi.")`
`print("Toplam:", total)`

(a) (A) hangi hatayı verir ve neden? (b) (A)'yı düzeltin. (c) (B) `score = 40` ve `score = 80` için ne yazar? (d) (B)'nin son satırı dört boşluk içeriden yazılsaydı `score = 40` için ne olurdu?

Cevap: (a) **IndentationError** — iki nokta ile biten satırın ardından girintili blok gelmeli
 (b) o satırı dört boşluk içeriden yazın (c) 40 → **Toplam: 0**; 80 → **Eklendi. ve Toplam: 80**
 (d) hiçbir şey yazılmaz

Alıştırma 6.9. ●●● Örnek 6.5'teki denetimi bu kez kullanıcı adı için yazın. Kurallar: uzunluk 3–16 karakter, ilk karakter harf, boşluk yok, tamamı küçük harf. Uygun değilse ilk bozulan kuralı söyleyin. Girdi `Ayşe_yılmaz` için çıktı: **Geçersiz: yalnız küçük harf kullanın.**

Cevap: uzunluk 3–16, ilk karakter harf, boşluk yok, tamamı küçük harf; `Ayşe_yılmaz` son kuralda takılır

Alıştırma 6.10. ●●● Ay numarası (1–12) ve yıl alıp o ayın kaç gün çektiğini yazan bir program yazın. Şubat için artık yıl kuralını kullanın. Geçersiz ay numarasını da yakalayın. Girdi 2 ve 2024: 29 gün.

Cevap: 2/2024 → 29, 2/2023 → 28; 4, 6, 9, 11 → 30; geri kalan → 31

Alıştırma 6.11. ●●● $ax^2 + bx + c = 0$ denkleminin gerçek köklerini bulan bir program yazın. Diskriminant $\Delta = b^2 - 4ac$ pozitifse iki, sıfırsa bir, negatifse hiç gerçek kök vardır. $a = 0$ durumunu da yakalayın. Girdi 1, -3, 2: kökler 2 ve 1.

Cevap: $\Delta = 9 - 8 = 1$; $x_1 = 2$, $x_2 = 1$; $a \neq 0$ daki en başta olmalı

Hafta 7: Karmaşık Kararlar: İç İçe Koşullar, match-case ve Girdi Doğrulama

Öğrenme çıktıları

Bu haftanın sonunda:

1. bir problemi iç içe if'lerle, elif zinciriyle ve and ile yazabilecek; hangisinin ne zaman daha okunur olduğunu gerektirebileceksiniz;
2. iki girdinin birlikte belirlediği bir sonucu *karar tablosuna* dökebilecek ve tabloyu mekanik olarak koda çevirebileceksiniz;
3. De Morgan kurallarıyla not içeren karmaşık koşulları sadeleştirebilecek, eşdeğerliği doğruluk tablosuyla gösterebileceksiniz;
4. match-case yapısını sabit desenler, |, _ ve bekçi koşullarıyla kullanabilecek, match ile elif arasında bilinçli seçim yapabileceksiniz;
5. kullanıcı girdisini str.isdigit() ve try-except ValueError ile doğrulayabileceksiniz;
6. bir aralık koşulunun sınır değerlerini bulup test tablosu yazabilecek ve menü tabanlı küçük bir program tasarlayabileceksiniz.

Anahtar kavramlar

iç içe koşul (İng. nested if) **karar tablosu** (İng. decision table) **De Morgan kuralları** (İng. De Morgan's laws) **desen eşleme** (İng. pattern matching) **joker desen** (İng. wildcard pattern) **bekçi** (İng. guard) **girdi doğrulama** (İng. input validation) **sınır değer analizi** (İng. boundary value analysis) **test tablosu** **karar ağacı** (İng. decision tree)

Geçen hafta bir programın akışını ikiye ayırmayı öğrendiniz. O yapılarla çözülen problemlerin ortak yanı, kararın *tek* bir soruya dayanmasıydı: not kaç? sayı çift mi? Oysa gerçek problemlerde kararlar birbirine bağlanır: bir öğrencinin harf notu puanına *ve* devamsızlığına, bir kargonun ücreti ağırlığa *ve* bölgeye, bir tarihin geçerliliği güne, aya *ve* yılın artık olup olmadığına bağlıdır. Bu hafta kararları birbirine bağlamayı, bağladıktan sonra da okunur tutmayı öğreneceğiz.

İkinci konumuz daha az görünür ama en az onun kadar önemli: *programın kullanıcıya güvenmemesi*. Yaş yerine “on dokuz” yazan bir kullanıcı programınızı anında çökertir. Bu hafta girdiyi *doğrulama* ve bir aralık koşulunun en tehlikeli yerini — sınırlarını — sistemli biçimde sınamayı öğreneceksiniz. Yapay zeka mühendisliğinde bu alışkanlığın adı *veri doğrulama*dır ve bir modelin başarısı çoğu zaman modelden çok buna bağlıdır.

7.1 İç içe koşullar

Bir if bloğunun içine başka bir if yazılabilir; buna **iç içe koşul** (İng. nested if) denir. Yeni bir kural yok: bir bloğun gövdesine ne yazılabiliyorsa iç bloğa da yazılabilir. Tek yeni şey, girintinin bir seviye daha derinleşmesidir.

Tanım 7.1: İç içe koşul

Bir if (ya da else) bloğunun gövdesinde yer alan ikinci bir if deyimine **iç içe koşul** denir. İçteki koşul **yalnız** dıştaki koşul sağlandığında değerlendirilir; her iç içe seviye girintiyi dört boşluk daha derinleştirir.

Aynı karar üç ayrı biçimde yazılabilir. Bir araç kiralama kuralını ele alalım: kişi 18 yaşından büyük *ve* ehliyet sahibiyse araç kiralayabilir.

```

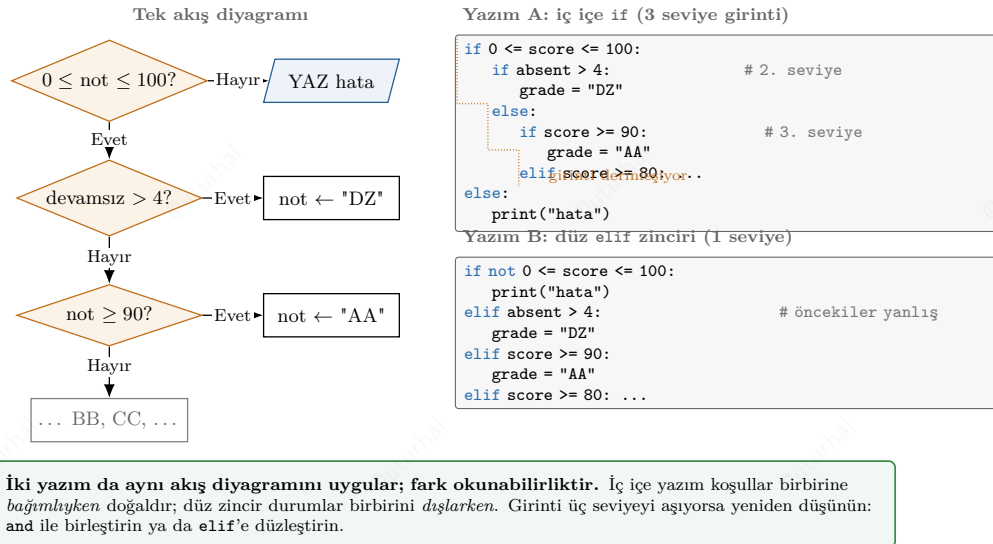
1 if age >= 18:                                # Yazım A -- iç içe
2     if has_license:
3         print("Araç kiralayabilirsiniz.")
4
5 if age >= 18 and has_license:                # Yazım B -- tek koşul, and ile
6     print("Araç kiralayabilirsiniz.")
7
8 if age < 18:                                # Yazım C -- elif zinciri
9     print("18 yaşından küçüksünüz.")
10 elif not has_license:
11     print("Ehliyetiniz yok.")
12 else:
13     print("Araç kiralayabilirsiniz.")

```

Üçü de aynı girdilerde aynı “kiralayabilirsiniz” kararını verir; seçim üslup değil *amaç* meselesidir:

- **Tek bir olumlu sonuç yetiyorsa** Yazım B en kısasıdır: girinti tek seviyede kalır. Varsayılan tercihiniz bu olsun.
- **Reddin *nedenini* söylemek gerekiyorsa** Yazım C şarttır. B ile “olmadı” diyebilirsiniz ama “neden olmadı” diyemezsiniz.
- **İçteki soru, dıştakinin cevabına bağlıysa** Yazım A zorunludur. Klasik örnek bölmedir: `if b != 0`: dışta, bölme içte. Koşul sayısı arttıkça bu bağımlılığı `and` ile yazmak okunmaz hâle gelir.

Şekil 7.1, aynı akış diyagramının iç içe ve düz `elif` yazımlarını yan yana gösterir.



Şekil 7.1: Aynı karar akışının iki yazımı: iç içe if (üstte) ve düz `elif` zinciri (altta). Düz zincir tek seviyede kalır; bedeli ilk koşulun terslenmesidir.

⚠ Dikkat

Girinti üç seviyeyi geçtiğinde kod okunmaz olmaya başlar; buna *ok başı deseni* (İng. *arrow anti-pattern*) denir. Üç çare: bağımsız koşulları `and` ile birleştirin; dışlayan durumları `elif` zincirine düzleştirin; hata durumlarını önce yakalayın.

**Dikkat**

Python'da **else**'in hangi **if**'e ait olduğunu **yalnız girinti** belirler. Aşağıdaki iki parçanın tek farkı son iki satırın girintisidir, ama farklı programlardır: $x = -2$ için birincisi hiçbir şey yazmaz, ikincisi B yazar. Girintiyi bu yüzden “biçim” değil *söz dizimi* sayın.

```

1 if x > 0:                if x > 0:
2     if x < 10:            if x < 10:
3         print("A")        print("A")
4     else:                else:
5         print("B")        print("B")
6 # (1) else İÇTEKİ if'e ait # (2) else DIŞTAKİ if'e ait

```

Örnek 7.1 (Devamsızlıklı harf notu). Dönem sonu notunu (0–100) ve devamsız hafta sayısını alan bir program yazın: dörtten fazla devamsızlık varsa harf notu, puan ne olursa olsun, DZ; değilse 90–80–70–60 eşiklerine göre AA, BB, CC, DD, altında FF. Not aralık dışındaysa hata yazılsın.

Çözüm

Analiz. İki girdi: puan (**float**) ve devamsızlık (**int**). Bir çıktı: harf notu. Üç soru birbirine bağlıdır ve *bu sırayla* sorulmalıdır: (i) puan geçerli mi? (ii) devam koşulu sağlanmış mı? (iii) puan hangi aralıkta? İkinci soru ancak birincinin cevabı “evet” ise anlamlıdır; üçüncüsü ancak ikincinin cevabı “hayır” ise. Bu bağımlılık zinciri iç içe yazımı zorunlu kılar.

Algoritma.

Sözde kod: Devamsızlıklı harf notu

```

BAŞLA
OKU puan, devamsız
EĞER 0 <= puan <= 100 İSE
    EĞER devamsız > 4 İSE harf ← "DZ"
    DEĞİLSE harf ← 90/80/70/60 eşiklerine göre AA/BB/CC/DD, aksi hâlde FF
    YAZ harf
    DEĞİLSE YAZ "hata"
BİTİR

```

Kod ve çıktı.

Kod 7.1 w07-01-harf-notu.py — İç içe if: geçerlilik, devam koşulu, not aralığı



```

1 # w07-01-harf-notu.py -- İç içe if: not aralığı → devamsızlık → harf
   ↳ notu
2 # Kural (kurgusal): 14 haftada 4'ten fazla devamsızlık DZ (devamsız)
   ↳ sayılır
3
4 score = float(input("Dönem sonu notu (0-100): "))
5 absent = int(input("Devamsız hafta sayısı: "))
6
7 if 0 <= score <= 100:                # 1. seviye: girdi geçerli mi?
8     if absent > 4:                    # 2. seviye: devam koşulu
9         grade = "DZ"
10    else:                             # 3. seviye: not aralıkları
11        if score >= 90:
12            grade = "AA"
13        elif score >= 80:
14            grade = "BB"
15        elif score >= 70:
16            grade = "CC"
17        elif score >= 60:
18            grade = "DD"
19        else:
20            grade = "FF"
21    print(f"Not: {score:.1f} Devamsızlık: {absent} hafta ->
   ↳ {grade}")
22 else:
23    print("Hata: not 0 ile 100 arasında olmalı.")

```

```
$ python w07-01-harf-notu.py
```

```

Dönem sonu notu (0-100): 78
Devamsız hafta sayısı: 2
Not: 78.0 Devamsızlık: 2 hafta -> CC

```

Kontrol. Puan 78, devamsızlık 2: ilk koşul doğru, ikincisi yanlış, zincirde $78 \geq 70$ ilk sağlanan koşul → CC ✓. Sınır değerleri: devamsızlık tam 4 iken $4 > 4$ yanlıştır, yani dört hafta *hâlâ geçerlidir*; beşte DZ başlar. Puan tam 90'da AA, 89,9'da BB; -1 ve 101 hata verir. Bu beş deneme programın bütün dallarını çalıştırır.

7.2 Karar tablosundan koda

İki (ya da daha çok) girdinin *birlikte* belirlediği bir sonuç varsa, kodu yazmadan önce bir tablo çizin. Tablo hem düşünmeyi kolaylaştırır hem de unutulmuş durumu görünür kılar (Şekil 7.2).

Tanım 7.2: Karar tablosu

Karar tablosu (İng. *decision table*), her satırı bir koşulun, her sütunu başka bir koşulun bir değerine karşılık gelen; her hücrelerinde o birleşimin sonucu yazan tablodur. İyi bir karar tablosu **tamdır** (her girdi birleşimi bir hücreye düşer) ve **ayrıktır** (hiçbiri iki hücreye düşmez).

Karar tablosu (kurgusal ücretler, TL)

ağırlık \ bölge	1 şehir içi	2 yurt içi	3 yurt dışı
hafif: $w \leq 2$	60	90	400
orta: $2 < w \leq 10$	90	150	700
ağır: $w > 10$	150	250	1200

Örnek: 3,5 kg, bölge 2 → “orta” satırı, “yurt içi” sütunu → 150 TL. Satırlar ağırlık eksenini boşluksuz ve çakışmasız böler; sütunlar bölgeyi.

Aynı tablo Python’da

```
if weight <= 2: # satır 1
    if region == 1: price = 60
    elif region == 2: price = 90
    else: price = 400
elif weight <= 10: # satır 2
    if region == 1: price = 90
    elif region == 2: price = 150
    else: price = 700
else: # satır 3
    if region == 1: price = 150
    elif region == 2: price = 250
    else: price = 1200
```

Karar tablosu, iki (ya da daha çok) girdinin birlikte belirlediği sonucu tek bakışta gösterir; koda çevirmek mekaniktir: dış if-elif-else satırı, iç if-elif-else sütunu seçer; her hücre bir yapraktır. Dokuz hücre, dokuz atama. Tablo *tam* (her girdi çifti bir hücreye düşer) ve *ayrık* (hiçbir çift iki hücreye düşmez) olmalıdır; koda bunu sağlayan şey her seviyedeki kapanış *else*’idir. Tablodaki $\leq$ ile koddaki $<=$ birebir aynı: $w = 2$ kg tam sınırdaki “hafif”tir, $w = 10$ kg “orta”.

Şekil 7.2: Karar tablosundan koda eşleme: dış if-elif-else satırı, iç if-elif-else sütunu seçer; her hücre bir atamadır.

Hücre sayısı hızla büyür: üç ağırlık sınıfı ve üç bölge $3 \times 3 = 9$ hücre eder; üçüncü bir ikili koşul (“kırılabılır mı?”) eklenince $9 \times 2 = 18$. Genel olarak n tane ikili koşulun 2^n birleşimi vardır: dört koşul 16, on koşul 1024 satır demektir. Bu yüzden karar tablosu küçük problemlerin aracıdır.

Karar tablosunu koda çevirmenin dört adımı

1. **Eksenleri seçin;** daha az değeri olan eksen sütun yapın.
2. **Her eksen boşluksuz ve çakışmasız bölün** ($w \leq 2$, $2 < w \leq 10$, $w > 10$): bir değer iki satıra ya da hiçbir satıra düşmemeli.
3. **Dış zinciri satırlardan, iç zinciri sütunlardan yazın;** her iki zincirin son dalı kapanış *else*’i olsun. Her hücre bir atamadır.

Geçersiz girdi denetimini tablodan önce yapın.

Örnek 7.2 (Kargo ücreti). Bir kargo şirketinin ücreti pakedin ağırlığına ve gideceği bölgeye bağlıdır (ücretler kurgusaldır). Ağırlığı ve bölge kodunu (1 şehir içi, 2 yurt içi, 3 yurt dışı) alan, ücreti yazan bir program yazın; geçersiz bölge kodunu reddedin.

Çözüm

Analiz. İki girdi: ağırlık (float) ve bölge (int). Bir çıktı: ücret (TL). Ağırlık sürekli bir değer olduğu için üç *aralığa* bölünür; bölge zaten üç ayrık değerdir. Geçersiz bölge kodu tablonun dışındadır, en başta elenir.

Karar tablosu. Şekil 7.2’de verilmiştir: satırlar ağırlık sınıfı (hafif $w \leq 2$, orta $2 < w \leq 10$, ağır $w > 10$), sütunlar bölge; dokuz hücre, dokuz ücret.

Kod ve çıktı.

Kod 7.2 w07-02-kargo.py — Karar tablosunun koda birebir çevirisi



```

1 # w07-02-kargo.py -- Karar tablosundan koda: kargo ücreti (ağırlık ×
   ↳ bölge)
2 # Tablo (kurgusal, TL): bölge 1 / 2 / 3 -> hafif 60/90/400,
3 #   orta 90/150/700, ağır 150/250/1200 (hafif <= 2 kg, orta <= 10 kg)
4
5 weight = float(input("Paket ağırlığı (kg): "))
6 region = int(input("Bölge (1 şehir içi, 2 yurt içi, 3 yurt dışı): "))
7
8 if region < 1 or region > 3:
9     print("Hata: bölge kodu 1, 2 ya da 3 olmalı.")
10 else:
11     if weight <= 2:                                # tablo satırı: hafif
12         if region == 1:
13             price = 60
14         elif region == 2:
15             price = 90
16         else:
17             price = 400
18     elif weight <= 10:                              # tablo satırı: orta
19         if region == 1:
20             price = 90
21         elif region == 2:
22             price = 150
23         else:
24             price = 700
25     else:                                           # tablo satırı: ağır
26         if region == 1:
27             price = 150
28         elif region == 2:
29             price = 250
30         else:
31             price = 1200
32     print(f"{weight:.1f} kg, bölge {region}: kargo ücreti {price} TL")

```

```
$ python w07-02-kargo.py
```

```

Paket ağırlığı (kg): 3.5
Bölge (1 şehir içi, 2 yurt içi, 3 yurt dışı): 2
3.5 kg, bölge 2: kargo ücreti 150 TL

```

Kontrol. 3,5 kg, bölge 2: $3,5 \leq 2$ yanlış, $3,5 \leq 10$ doğru $\rightarrow$ “orta” satırı, “yurt içi” sütunu $\rightarrow$ 150 TL ✓. Sınır değerleri: tam 2 kg “hafif”tir (çünkü $\leq$), 2,01 kg “orta”; tam 10 kg “orta”, 10,01 kg “ağır”. Bölge 0 ya da 4 girilirse tablo hiç çalışmaz. En azından her satırdan bir hücreyi ve her sınırdan bir değeri deneyin.

İpucu

Karar tablosunu kodun *içinde* de saklayın: dosyanın başına yorum satırı olarak yazın (Kod 7.2’de olduğu gibi). İyi yorum kodun ne yaptığını değil, kodun *hangi kurala* uyduğunu söyler.

7.3 Koşulları sadeleştirme: De Morgan

Koşullar birleştikçe içlerine **not** girer ve çift olumsuzlar okunmaz hâle gelir: “**not** (age < 18 or not ok)”. Böyle bir koşulu sadeleştirmenin kuralı vardır ve adı De Morgan’dır.

Kural 7.1 (De Morgan kuralları). Her a ve b doğruluk değeri için

$$\text{not } (a \text{ and } b) \equiv (\text{not } a) \text{ or } (\text{not } b) \quad \text{not } (a \text{ or } b) \equiv (\text{not } a) \text{ and } (\text{not } b)$$

Sözle: bir **not**’u parantezin içine dağıtırken **and** ile **or** yer değiştirir ve her parça terslenir. Karşılaştırmaların tersleri de bellidir: $< \leftrightarrow >=$, $> \leftrightarrow <=$, $= \leftrightarrow !=$.

Kanıt gerekmez, sayım yeter: iki doğruluk değerinin yalnız dört birleşimi vardır ve dördünde de iki taraf aynı sonucu verir (Tablo 7.1).

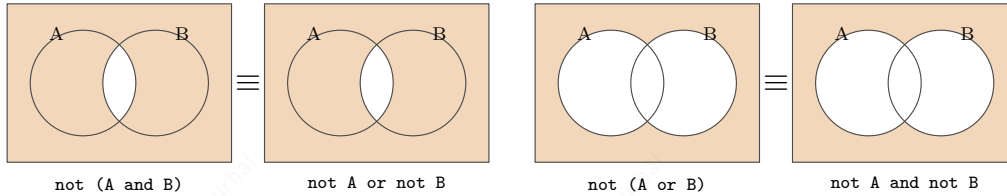
Tablo 7.1: Birinci De Morgan kuralının doğruluk tablosu. Dördüncü ve yedinci sütunlar satır satır aynıdır; eşdeğerlik budur.

a	b	a and b	not (a and b)	not a	not b	(not a) or (not b)
True	True	True	False	False	False	False
True	False	False	True	False	True	True
False	True	False	True	True	False	True
False	False	False	True	True	True	True

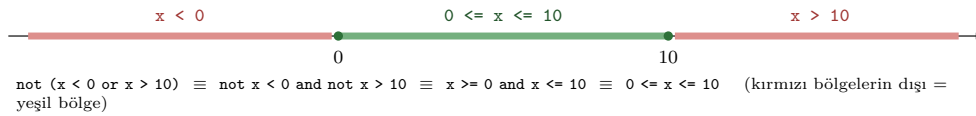
Şekil 7.3’te aynı eşdeğerlik Venn şemalarıyla ve sayı doğrusuyla gösterilmiştir.

Kural 1: “ikisi birden” değil = “en az biri” değil

Kural 2: “en az biri” değil = “ikisi de” değil



Aralık kontrolünde De Morgan



De Morgan kuralları bir **not**’u parantezin içine dağıtır: **and** ile **or** yer değiştirir, her parça terslenir ($< \leftrightarrow >=$, $= \leftrightarrow !=$). Böylece çift olumsuz koşullar tek bir aralık kontrolüne dönüşür.

Şekil 7.3: De Morgan kurallarının iki görsel kanıtı: Venn şemalarında eşdeğer ifadelerin taralı bölgeleri çıkarılır; sayı doğrusunda kırmızı bölgelerin dışı yeşil aralığa eşittir.

Kod 7.3 w07-03-de-morgan.py — De Morgan, zincirleme karşılaştırma ve or tuzağı

```
1 # w07-03-de-morgan.py -- De Morgan kuralları, zincirleme ve "or" tuzağı
2
3 print(" a      b      | not (a and b) | not a or not b")
4 print("-----+-----+-----")
5 a, b = True, True
```



```

6 print(f"{a!s:<6} {b!s:<6} | {not (a and b)!s:^13} | {not a or not b!s:^14}")
7 a, b = True, False
8 print(f"{a!s:<6} {b!s:<6} | {not (a and b)!s:^13} | {not a or not b!s:^14}")
9 a, b = False, True
10 print(f"{a!s:<6} {b!s:<6} | {not (a and b)!s:^13} | {not a or not b!s:^14}")
11 a, b = False, False
12 print(f"{a!s:<6} {b!s:<6} | {not (a and b)!s:^13} | {not a or not b!s:^14}")
13
14 # Aralık kontrolü: üç yazım, aynı anlam
15 x = 7
16 print(x >= 0 and x <= 10)          # klasik
17 print(0 <= x <= 10)                # zincirleme (tercih edilen)
18 print(not (x < 0 or x > 10))       # De Morgan ile eşdeğeri
19
20 # Tuzak: "x, 1 ya da 2'ye eşit mi?" demek için BU yazılmaz
21 print(x == 1 or 2)                  # sonuç 2 -> koşulda hep doğru sayılır!
22 print(x == 1 or x == 2)             # doğrusu

```

```
$ python w07-03-de-morgan.py
```

a	b	not (a and b)	not a or not b
True	True	False	False
True	False	True	True
False	True	True	True
False	False	True	True
True			
True			
True			
2			
False			

Kod 7.3'un ilk yarısı Tablo 7.1'i programın kendisine yazdırır; ikinci yarısı aralık kontrolünün üç eşdeğer yazımını gösterir. Üçü de True verir; `0 <= x <= 10` en okunurudur ve Python'a özgüdür — çoğu dilde zincirleme karşılaştırma yoktur.



Sık yapılan hata

`not (a and b) ≡ not a and not b` → `not (a and b) ≡ not a or not b`

En sık yapılan mantık hatası budur: `not` dağıtılırken bağlacın değişmesi unutulur. “Pazartesi ve saat 9” değilse ders yok: yanlış yazım pazartesi saat 10'da ders olduğunu sanır. Şüphelendiğinizde dört satırlık doğruluk tablosunu yazın.



Sık yapılan hata

`if x == 1 or 2:` → `if x == 1 or x == 2:`

Türkçede “x, 1 ya da 2'ye eşitse” demek doğaldır; Python'da değildir. `or` soldaki yanlışsa sağdaki değeri döndürür, yani ifadenin değeri 2 olur — ve sıfırdan farklı her sayı koşulda True sayılır: koşul *her zaman* doğrudur, program sessizce yanlış çalışır. Doğrusu `x == 1 or x == 2` ya da `case 1 | 2:` desenidir.

7.4 match-case: desenle seçim

Bir değişkeni birçok *sabit* değerle karşılaştıran `elif` zincirlerinde aynı değişken adı her satırda yeniden yazılır. Python 3.10 ile gelen `match-case` bu tekrarı ortadan kaldırır.

Tanım 7.3: match-case deyimi

match, *denetim ifadesinin* değerini altındaki **case** *desenleriyle* yukarıdan aşağıya karşılaştırır; ilk eşleşen **case**'in gövdesini çalıştırır ve deyimden çıkar. Bu haftaki üç desen biçimi:

- **sabit desen** — **case 3 :** : değer 3'e eşitse eşleşir;
- **seçenekli desen** — **case 3 | 4 | 5 :** : | burada “ya da” demektir; **or** operatörü değil, desen ayırıcısıdır;
- **joker desen** — **case _ :** : her değere uyar, **else**'in karşılığıdır ve **en sona** yazılır.

Bir **case**'e **if** ile bir koşul eklenirse buna **bekçi** (*İng. guard*) denir: desen eşleşse bile koşul yanlışsa o dal atlanır ve deneme aşağıdan sürer.

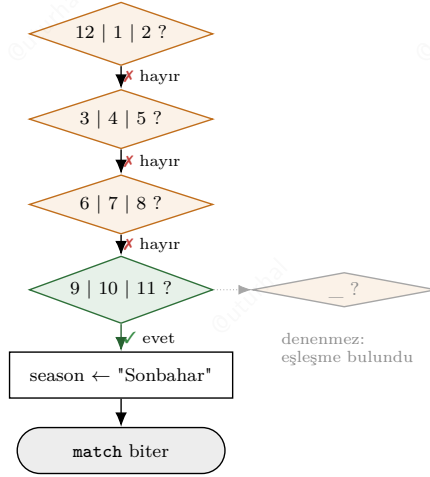
Şekil 7.4, deyimın bölümlerini ve desenlerin hangi sırayla denendiğini birlikte gösterir.

match-case deyimının bölümleri

```
match month:
    case 12 | 1 | 2:
        season = "Kış"
    case 3 | 4 | 5:
        season = "İlkbahar"
    case 6 | 7 | 8:
        season = "Yaz"
    case 9 | 10 | 11:
        season = "Sonbahar"
    case _:
        season = "geçersiz ay"
```

- ① **denetim ifadesi** — bir kez hesaplanır, sonra desenlerle karşılaştırılır.
- ② **sabit desen**; | “ya da” demektir, **or** değildir.
- ③ eşleşen ilk desenin gövdesi çalışır.
- ④ **joker desen**: her değere uyar, en sona yazılır (**else**'in karşılığı).

month = 10 için deneme sırası



match, *denetim ifadesinin* değerini desenlerle *yukarıdan aşağıya* karşılaştırır; ilk eşleşen **case**'in gövdesini çalıştırır ve **durur**. Davranış **if-elif-else** zinciriyle aynıdır; fark, aynı ifadenin her satırda yeniden yazılmamasıdır: **elif month == 9 or month == 10** yerine **case 9 | 10 | 11**. **match** yalnız *tek bir değeri* *sabit listelerle* karşılaştırdığınızda kazandırır.

Şekil 7.4: match-case deyiminin bölümleri (solda) ve desenlerin sırayla denenmesi (sağda). Eşleşme bulununca deneme durur.

Kod 7.4 w07-04-mevsim.py — match-case: sabit desenler ve | ile birleştirme

```
1 # w07-04-mevsim.py -- match-case: sabit desenler ve | ile birleştirme
2
3 month = int(input("Ay numarası (1-12): "))
4
5 match month:
6     case 12 | 1 | 2:
7         season = "Kış"
8     case 3 | 4 | 5:
9         season = "İlkbahar"
10    case 6 | 7 | 8:
11        season = "Yaz"
12    case 9 | 10 | 11:
13        season = "Sonbahar"
```



GitHub


```

14     case _:
15         season = "geçersiz ay"
16
17     print(f"{month}. ay: {season}")

```

```
$ python w07-04-mevsim.py
```

```

Ay numarası (1-12): 10
10. ay: Sonbahar

```

`match` yazımı on iki karşılaştırmanın tamamını dört satıra sığdırır ve “hangi değişkeni karşılaştırıyoruz?” sorusunu bir kez cevaplar. Bekçi kullanımını bir banka işlemi üzerinde görelim.

Kod 7.5 w07-05-banka.py — Bekçi (case ... if koşul) ile para çekme kontrolü

```

1  # w07-05-banka.py -- match-case ile guard (case ... if koşul): banka işlemi
2
3  balance = float(input("Mevcut bakiye (TL): "))
4  operation = input("İşlem (yatır / çek / bakiye): ").strip().lower()
5  amount = float(input("Tutar (TL, bakiye için 0): "))
6
7  match operation:
8      case "yatır" | "yadır":
9          balance = balance + amount
10         print(f"{amount:.2f} TL yatırıldı. Yeni bakiye: {balance:.2f} TL")
11     case "çek" | "cek" if amount > balance:
12         print(f"Yetersiz bakiye! En fazla {balance:.2f} TL çekebilirsiniz.")
13     case "çek" | "cek":
14         balance = balance - amount
15         print(f"{amount:.2f} TL çekildi. Yeni bakiye: {balance:.2f} TL")
16     case "bakiye":
17         print(f"Bakiyeniz: {balance:.2f} TL")
18     case _:
19         print(f"Tanınmayan işlem: '{operation}'")

```

```
$ python w07-05-banka.py
```

```

Mevcut bakiye (TL): 1000
İşlem (yatır / çek / bakiye): çek
Tutar (TL, bakiye için 0): 1500
Yetersiz bakiye! En fazla 1000.00 TL çekebilirsiniz.

```

Kod 7.5’da `case "çek" | "cek" if amount > balance:` satırı ile bir alttaki `case "çek" | "cek":` satırı *aynı* desene sahiptir; ayıran şey bekçidir. Sıra önemlidir: bekçili olan üstte durmalıdır, yoksa bekçisiz dal her zaman önce eşleşir ve yetersiz bakiye kontrolü hiç çalışmaz.



Dikkat

`match` eşleşmesi **değere ve türe** duyarlıdır. `input()` her zaman metin döndürdüğü için, kullanıcının yazdığı 3’ü `case 3:` yakalamaz — `case "3":` yakalar. İkisini karıştırmak sessiz bir hatadır: program çökmez, hep `_` dalına düşer. Ya girdiyi `int()` ile çevirip sayı desenleri kullanın, ya metin olarak bırakıp metin desenleri yazın; ikisini karıştırmayın.



GitHub



Sık yapılan hata

`case _: ... case 3: ...` → `case 3: ... case _: ...`

`case _` her değere uyduğu için altında kalan desenler *hiçbir zaman* denenmez. Python bunu doğrudan söz dizimi hatası olarak bildirir: `SyntaxError: wildcard makes remaining patterns unreachable`. Joker desen her zaman en sondaki `case`'tir.



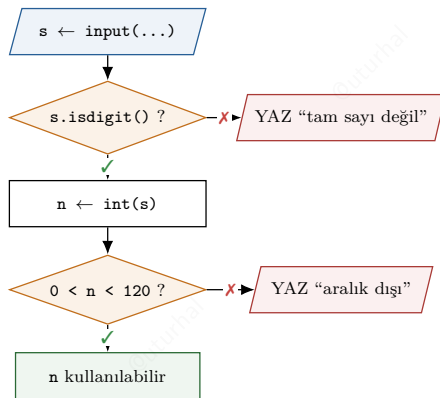
İpucu

`match` mi, `elif` mi? Soru “hangisi modern” değil, “hangisi bu koşulu daha az tekrarlar anlatıyor”dur. `match` *tek* bir değişkeni *çok sayıda sabit* değerle karşılaştırdığımızda kazandırır (ay numarası, menü seçimi, komut adı); aralık karşılaştırmalarında (`puan >= 90`) her dal için bekçi gerekeceğinden kazanç kalmaz. İki *ayrı* değişkeni birlikte sımayacaksınız `match` zaten uygun değildir: tek bir denetim ifadesi alır.

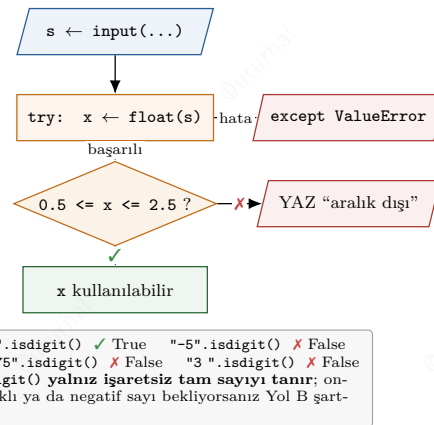
7.5 Girdi doğrulama

Şimdiye kadarki programlarımız kullanıcıya güvendi. `int(input("Yaş: "))` satırı, kullanıcı “on dokuz” yazdığında `ValueError` ile çöker ve program yarıda kalır. **Girdi doğrulama** (İng. *input validation*), bu çökmeyi anlamlı bir mesaja çevirmektir; iki yolu [Şekil 7.5](#)'te yan yana verilmiştir.

Yol A — önce biçimi denetle: `isdigit()`



Yol B — dene, hatayı yakala: `try-except`



İki yol da aynı soruyu sorar — “bu metin gerçekten sayı mı?” — ama sırayı değiştirir. Yol A önce sorar, sonra çevirir; Yol B önce çevirir, `ValueError` çıkarsa `except` bloğuna geçer ve program çökmez. Her iki yolda da doğrulama *iki katmanlıdır*: önce tür, sonra aralık.

Şekil 7.5: Girdi doğrulamanın iki yolu: önce biçimi denetle (`isdigit()`), ya da önce çevirmeyi deneyip `ValueError`'ı yakala.

Birinci yol geçen haftadan tanıdığımız `str` metotlarını kullanır: `isdigit()`, metnin tamamı rakamlardan oluşuyorsa `True` döndürür. Ama sınırları vardır: “-5” için `False` verir (eksi rakam değildir), “1.75” için `False` (nokta rakam değildir), “3 ” için bile `False` (sondaki boşluk). Yani `isdigit()` yalnız **işaretsiz tam sayı** bekliyorsanız yeterlidir.

İkinci yol dönüşümü *dener*, başarısız olursa hatayı *yakalar*:

```

1 try:
2     x = float(text)           # başarısız olursa ValueError fırlatılır
3     ok = True
4 except ValueError:           # yalnız bu hata türü yakalanır

```

```
5 ok = False
```

`try` bloğundaki bir satır `ValueError` fırlatırsa (*İng. raise*) akış anında `except` bloğuna atlar, bloğun geri kalanı çalışmaz; hata çıkmazsa `except` hiç çalışmaz.

Kod 7.6 w07-06-dogrulama.py — İki doğrulama yolu ve iki katmanlı denetim

```
1 # w07-06-dogrulama.py -- Girdi doğrulama: isdigit(), try/except, aralık
2
3 # 1) Tam sayı bekliyoruz: isdigit() ile biçim kontrolü
4 age_text = input("Yaşınız: ")
5 if age_text.isdigit():
6     age = int(age_text)
7     if 0 < age < 120:
8         print(f"Yaş kabul edildi: {age}")
9     else:
10        print("Yaş 1 ile 119 arasında olmalı.")
11 else:
12     print(f"'{age_text}' bir tam sayı değil.")
13
14 # 2) Ondalıklı sayı bekliyoruz: isdigit() yetmez ("1.75" için False verir)
15 height_text = input("Boyunuz (m): ")
16 try:
17     height = float(height_text)      # başarısız olursa ValueError fırlar
18     is_number = True
19 except ValueError:
20     is_number = False
21
22 if not is_number:
23     print(f"'{height_text}' sayı değil; ondalık ayırıcı NOKTA olmalı: 1.75")
24 elif not 0.5 <= height <= 2.5:
25     print("Boy 0.5 ile 2.5 m arasında olmalı.")
26 else:
27     print(f"Boy kabul edildi: {height:.2f} m")
```

```
$ python w07-06-dogrulama.py
```

```
Yaşınız: on dokuz
'on dokuz' bir tam sayı değil.
Boyunuz (m): 1,75
'1,75' sayı değil; ondalık ayırıcı NOKTA olmalı: 1.75
```

Kod 7.6'un çıktısında iki tipik kullanıcı hatası görünür: sayı yerine yazı ("on dokuz") ve ondalık ayırıcı olarak virgül ("1,75"). İkincisi Türkiye'de en sık karşılaşılabilecek girdi hatasıdır; bu yüzden istemin içinde beklenen biçimi *örnekle* gösterin: "Boyunuz (m, örn. 1.75): ".

Kullanıcıdan güvenli sayı almanın üç katmanı

1. **Biçim.** Metin gerçekten bir sayı mı? İşaretsiz tam sayıda `isdigit()`; her durumda `try-except ValueError` çalışır.
2. **Aralık.** Sayı *anlamlı* mı? Yaş için $0 < n < 120$, ay için $1 \leq m \leq 12$. Biçimi doğru ama değeri saçma girdi en tehlikelidir.
3. **Mesaj.** Reddederken ne beklediğinizi söyleyin: "geçersiz giriş" değil, "yaş 1 ile 119 arasında bir tam sayı olmalı".

“Doğru girene kadar tekrar sor” davranışı bir döngü gerektirir: 8. hafta.

Dikkat

`except` sözcüğünün ardına **yakalayacağınız hata türünü yazın**: `except ValueError:`. Çıplak `except:` yazımı *her* hatayı yutar — yazım hatanızı, bellek hatasını da; böyle bir program hatayı *gizlemeyi* öğrenmiş olur. Bu hafta `try-except`'i yalnız `int()` ve `float()` dönüşümü için kullanıyoruz; hata yönetiminin tamamı 15. haftada.

7.6 Aralıklar ve sınır durumları

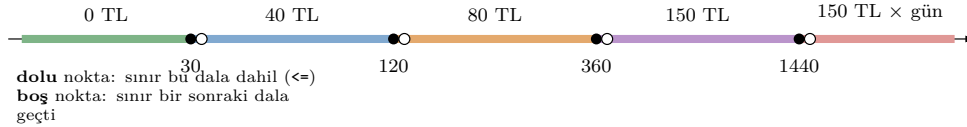
Bir programın hataları ortalarda değil, *sınırlarda* saklanır. “120 dakikaya kadar 40 TL” cümlesini `if minutes < 120:` diye yazarsanız, tam 120 dakika park eden müşteriden fazla ücret alırsınız. Bu tür hatalara **bir-eksik hatası** (*İng. off-by-one error*) denir.

Tanım 7.4: Sınır değer analizi

Bir aralık koşulunun **sınır değerleri** (*İng. boundary values*), aralığın uçlarındaki değerlerdir. **Sınır değer analizi**, her sınır için *sınırın kendisini* ve *ondan bir birim ötesini* deneyerek programı sınamaktır; sonuçlar bir **test tablosuna** yazılır (girdi, beklenen, gerçek). Beklenen ile gerçek farklıysa hata neredeyse her zaman `<` ile `<=` arasındadır.

Şekil 7.6 aşağıdaki örneğin tarifesini sayı doğrusunda gösterir ve ondan bir test tablosu türetir.

Otopark tarifesini sayı doğrusunda (eksen ölçekli değildir)



Sınır değer testleri: her sınırdaki *iki* deneme — sınırın kendisi ve bir fazlası

girdi (dakika)	0	30	31	120	121	360	361	1440	1441
beklenen (TL)	0	0	40	40	80	80	150	150	300
gerçek (TL)	...	...	...	...	...	...	...	...	...

Son satır laboratuvarında doldurulur: programı çalıştırıp gördüğünüz değeri yazın. “Beklenen” ile “gerçek” farklıysa hata sınırdadır — neredeyse her zaman `<` yerine `<=` (ya da tersi) yazılmıştır.

Bir programın hataları ortalarda değil, sınırlarda saklanır. Üç soru sorun: sınır değeri hangi dala düşüyor; aralıklar arasında boşluk kalıyor mu; aralıklar örtüşüyor mu? Sınır değeri analizi bunu mekanik bir teste dönüştürür — dokuz test, beş dalı ve dört sınırı birden kapatır.

Şekil 7.6: Otopark tarifesinin sayı doğrusu ve test tablosu. Dolu nokta sınırın soldaki dala ait olduğunu (`<=`) gösterir; dört sınır, sekiz test.

Örnek 7.3 (Otopark ücreti). Bir otoparkın tarifesini (kurgusal): ilk 30 dakika ücretsiz; 2 saate kadar 40 TL; 6 saate kadar 80 TL; 24 saate kadar 150 TL; 24 saati aşan park için *başlayan* her gün 150 TL. Park süresini dakika olarak alıp ücreti yazan bir program yazın.

Çözüm

Analiz. Bir girdi: dakika (tam sayı). Bir çıktı: ücret. Eksen beş aralığa bölünür; son aralıkta ücret sabit değil, gün sayısına bağlı bir çarpımdır. “Başlayan gün” yukarı yuvarlama demektir: `math.ceil`.

Karar tablosu (tek eksen). $d \leq 30 \rightarrow 0$ TL; $d \leq 120 \rightarrow 40$; $d \leq 360 \rightarrow 80$; $d \leq 1440 \rightarrow 150$; $d > 1440 \rightarrow 150 \times$ başlayan gün. Sınır denemeleri 30/31, 120/121, 360/361, 1440/1441 çiftleridir (Şekil 7.6).

Kod ve çıktı.

Kod 7.7 w07-07-otopark.py — Aralıklar, `math.ceil` ve `divmod` ile süre gösterimi

```
1 # w07-07-otopark.py -- Otopark ücreti: süre dilimleri ve sınır
   ↳ değerleri
2 # Tarife (kurgusal): <=30 dk 0; <=120 dk 40; <=360 dk 80; <=1440 dk
   ↳ 150 TL;
3 # 1440 dakikayı aşarsa başlayan her gün 150 TL
4 import math
5
6 minutes_text = input("Park süresi (dakika): ")
7
8 if not minutes_text.isdigit():
9     print("Hata: süre pozitif bir tam sayı olmalı.")
10 else:
11     minutes = int(minutes_text)
12     if minutes <= 30:
13         fee = 0
14     elif minutes <= 120:
15         fee = 40
16     elif minutes <= 360:
17         fee = 80
18     elif minutes <= 1440:
19         fee = 150
20     else:
21         days = math.ceil(minutes / 1440)      # başlayan gün tam gün
22         ↳ sayılır
23         fee = 150 * days
24         hours, rem = divmod(minutes, 60)
25         print(f"Süre: {hours} sa {rem} dk -> ücret: {fee} TL")
```

```
$ python w07-07-otopark.py
```

```
Park süresi (dakika): 195
Süre: 3 sa 15 dk -> ücret: 80 TL
```

Kontrol. 195 dakika: ilk iki koşul yanlış, $195 \leq 360$ doğru $\rightarrow 80$ TL ✓; $195 = 3 \cdot 60 + 15$, yani “3 sa 15 dk” ✓. Sınır değerleri: $30 \rightarrow 0$ TL ama $31 \rightarrow 40$ TL; $1440 \rightarrow 150$ TL ama $1441 \rightarrow 300$ TL, çünkü $\lceil 1441/1440 \rceil = 2$ gün. En kritik nokta $\leq$ kullanımıdır: $<$ yazsaydık tam 30, 120, 360 ve 1440 dakikaların hepsi bir üst dilime kayardı.



Sık yapılan hata

if minutes < 30: fee = 0 $\rightarrow$ if minutes <= 30: fee = 0

“30 dakikaya kadar ücretsiz” cümlesindeki “kadar” Türkçede sınırı içerir. Problem met-

nindeki “kadar”, “aşan”, “en az”, “en fazla” sözcüklerini bir operatöre çevirin: “en az 18” → ≥ 18 ; “18’i aşan” → > 18 ; “18’e kadar” → ≤ 18 . Yanlış seçilen tek bir eşittir işareti yalnız tam sınırdaki hata verir.

7.7 Menü programları ve karar ağaçları

Kullanıcıya seçenek sunan programların iskeleti hep aynıdır: seçenekleri yaz, seçimi oku, seçime göre dallan. `match` bu iskelet için biçilmiş kaftandır.

Aşağıdaki birim dönüştürücü bu iskeletin tam bir örneğidir. Seçim metin olarak okunur ve metin olarak karşılaştırılır (`case "1":`); böylece kullanıcı harf yazsa bile program çökmez, doğrudan `_` dalına düşer.

Kod 7.8 w07-08-menu.py — Menü tabanlı program: tek seçim, `match` ile

```
1 # w07-08-menu.py -- Menü tabanlı program (tek seçim): birim dönüştürücü
2 # 8. haftada bu menü döngüye alınacak: "Çıkış" seçilene kadar yinelenen.
3
4 print("=== Birim Dönüştürücü ===")
5 print("1) Kilometre -> mil")
6 print("2) Kilogram -> libre")
7 print("3) Celsius -> Fahrenheit")
8 print("4) Çıkış")
9 choice = input("Seçiminiz (1-4): ").strip()
10
11 match choice:
12     case "1":
13         km = float(input("Kilometre: "))
14         print(f"{km} km = {km * 0.621371:.2f} mil")
15     case "2":
16         kg = float(input("Kilogram: "))
17         print(f"{kg} kg = {kg * 2.20462:.2f} lb")
18     case "3":
19         c = float(input("Celsius: "))
20         print(f"{c} °C = {c * 9 / 5 + 32:.1f} °F")
21     case "4":
22         print("Güle güle!")
23     case _:
24         print(f"Geçersiz seçim: '{choice}'. 1-4 arası bir sayı girin.")
```



GitHub

\$ python w07-08-menu.py

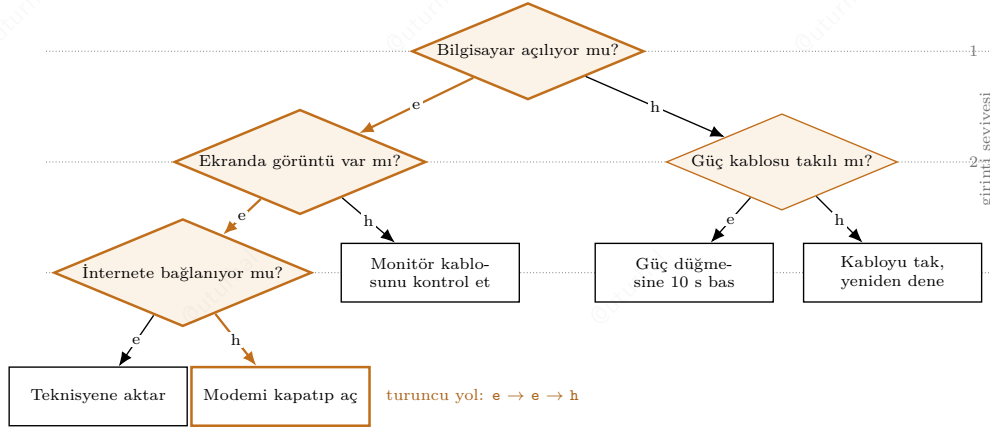
```
=== Birim Dönüştürücü ===
1) Kilometre -> mil
2) Kilogram -> libre
3) Celsius -> Fahrenheit
4) Çıkış
Seçiminiz (1-4): 2
Kilogram: 70
70.0 kg = 154.32 lb
```

Çıktıdaki hesabı doğrulayalım: seçim “2”, girdi 70 kg, sonuç $70 \times 2,20462 = 154,3234 \rightarrow 154.32$ ✓. Büyüklük mertebesi de makul — bir libre yarım kilodan biraz azdır. Seçim olarak “5”, “a” ya da boş metin girilirse program `_` dalına düşer; ama “1” seçilip değer olarak “abc” yazılırsa yine çöker: ikinci girdi doğrulanmamıştır. Onu doğrulamak laboratuvarın Görev 2’sinin işidir.

Not

Bu menü programı **bir kez** çalışır. Gerçek bir menü “Çıkış” seçilene kadar yinelenir; bunun için bir *döngü* gerekir ve döngüler 8. haftanın konusudur. O haftada bu dosyayı açıp menüyü döngüye alacağız — değişecek olan yalnız birkaç satırdır. Kodu bu yüzden saklayın.

Menüler yatay dallanmadır: tek soru, çok cevap. İç içe if’ler ise dikeydir: her cevap yeni bir soru doğurur ve bir **karar ağacı** (İng. *decision tree*) çizilir (Şekil 7.7).



İç içe if’ler bir ağaçtır: her soru bir düğüm, her cevap bir dal, her çıktı bir yapraktır. Ağaç den-
gesiz olabilir; çünkü sorular ancak bir öncekinin cevabı belliysse sorulur. Bu yapı yapay zekada da karşınıza
çıkacak: *karar ağacı* sınıflandırıcılarında soruları programcı değil algoritma seçer.

Şekil 7.7: İç içe if’lerin ağaç görünümü: her soru bir düğüm, her cevap bir dal, her öneri bir yaprak. Turuncu yol, örnek çalıştırmanın izlediği yoldur.

Kod 7.9 w07-12-chatbot.py — Karar ağacı olarak iç içe if: destek botu

```

1 # w07-12-chatbot.py -- Evet/hayır sorularıyla karar ağacı: destek "chatbot"u
2
3 print("Destek botuna hoş geldiniz. Sorulara e (evet) / h (hayır) yazın.")
4 a1 = input("Bilgisayarınız açılıyor mu? ").strip().lower()
5
6 if a1 == "e":
7     a2 = input("Ekranında görüntü var mı? ").strip().lower()
8     if a2 == "e":
9         a3 = input("İnternete bağlanabiliyor musunuz? ").strip().lower()
10        if a3 == "e":
11            print("Öneri: Sorunu ayrıntılı yazın, teknisyene aktarıyorum.")
12        else:
13            print("Öneri: Modemi 30 saniye kapatıp yeniden açın.")
14    else:
15        print("Öneri: Monitör kablosunu ve parlaklık ayarını kontrol edin.")
16 elif a1 == "h":
17     a2 = input("Güç kablosu prize takılı mı? ").strip().lower()
18     if a2 == "e":
19         print("Öneri: Güç düğmesine 10 s basılı tutun; olmazsa servise.")
20    else:
21        print("Öneri: Kabloyu takın ve yeniden deneyin.")
22 else:
23     print(f"'{a1}' anlaşılmadı; yalnız e ya da h yazın.")

```



GitHub

```
$ python w07-12-chatbot.py
```

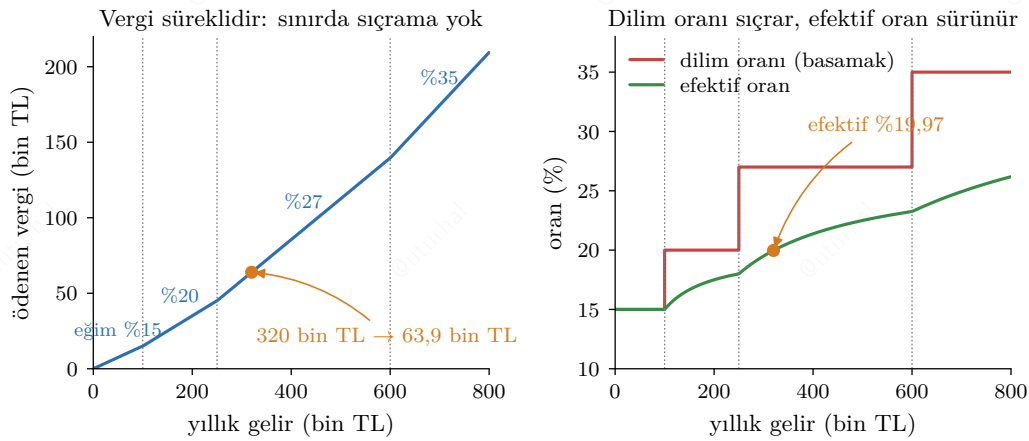
```
Destek botuna hoş geldiniz. Sorulara e (evet) / h (hayır) yazın.
Bilgisayarınız açılıyor mu? e
Ekranda görüntü var mı? e
İnternete bağlanabiliyor musunuz? h
Öneri: Modemi 30 saniye kapatıp yeniden açın.
```

Kod 7.9'un beş yaprağı vardır ve en uzun yol üç sorudan geçer. Ağacın dengesiz olması bir kusur değil, problemin doğasıdır: “ekranda görüntü var mı?” sorusu ancak bilgisayar açılıyorsa anlamlıdır.

7.8 Uygulamalar

7.8.1 Kademeli gelir vergisi

Vergi dilimleri aralık koşullarının en öğretici örneğidir; çünkü yaygın bir yanılgıyı barındırır: “bir üst dilime geçersen elime daha az para geçer.” Yanlıştır: yüksek oran tüm gelire değil, yalnız o dilimi *aşan kısma* uygulanır.



Şekil 7.8: Solda: gelire karşı vergi; dilim sınırlarında eğim değişir ama sıçrama yoktur. Sağda: dilim oranı sıçrarken efektif oran yavaşça yükselir.

Örnek 7.4 (Kademeli gelir vergisi). Yıllık geliri alıp kurgusal bir tarifeye göre vergiyi hesaplayan bir program yazın: ilk 100 000 TL için %15; 250 000'e kadar %20; 600 000'e kadar %27; üzeri için %35 — her oran yalnız kendi diliminde kalan kısma uygulanır. Vergiyi, net geliri ve efektif oranı yazdırın.

Çözüm

Analiz. Bir girdi: gelir. Dört çıktı: gelir, vergi, net, efektif oran. Kritik nokta, her dilimde *önceki dilimlerin vergisininin sabit olmasıdır*: üçüncü dilimdeki biri ilk 100 000 için 15 000, sonraki 150 000 için 30 000 öder; üzerine yalnız 250 000'i aşan kısmın %27'si eklenir.

Karar tablosu.

Koşul	Vergi	Sınırdaki
$g \leq 100\,000$	$0,15\,g$	15 000
$g \leq 250\,000$	$15\,000 + 0,20\,(g - 100\,000)$	45 000
$g \leq 600\,000$	$45\,000 + 0,27\,(g - 250\,000)$	139 500
$g > 600\,000$	$139\,500 + 0,35\,(g - 600\,000)$	—

Kod ve çıktı.

Kod 7.10 w07-09-vergi.py — Kademeli vergi: her dilimde yalnız aşan kısım

```

1 # w07-09-vergi.py -- Kademeli gelir vergisi (dilimler KURGUSAL, gerçek
   ↳ değil)
2 # 100 000'e kadar %15; 250 000'e kadar %20; 600 000'e kadar %27; üzeri
   ↳ %35 --
3 # her oran yalnız kendi dilimine düşen kısma uygulanır. Sınırlarda
   ↳ biriken
4 # vergi sırasıyla 15 000, 45 000 ve 139 500 TL'dir.
5 income = float(input("Yıllık gelir (TL): "))
6
7 if income <= 0:
8     print("Hata: gelir pozitif bir sayı olmalı.")
9 else:
10    if income <= 100_000:
11        tax = income * 0.15
12    elif income <= 250_000:
13        tax = 15_000 + (income - 100_000) * 0.20
14    elif income <= 600_000:
15        tax = 45_000 + (income - 250_000) * 0.27
16    else:
17        tax = 139_500 + (income - 600_000) * 0.35
18    print(f"Gelir   : {income:>12,.2f} TL")
19    print(f"Vergi   : {tax:>12,.2f} TL")
20    print(f"Net     : {income - tax:>12,.2f} TL")
21    print(f"Efektif oran: {tax / income * 100:.2f} %")

```

\$ python w07-09-vergi.py

```

Yıllık gelir (TL): 320000
Gelir   : 320,000.00 TL
Vergi   : 63,900.00 TL
Net     : 256,100.00 TL
Efektif oran: 19.97 %

```

Kontrol. Gelir 320 000 TL üçüncü dilimdedir: $15\,000 + 30\,000 + 18\,900 = 63\,900$ TL ✓; net 256 100 TL; efektif oran $63\,900/320\,000 = \%19,97$ — dilim oranı olan $\%27$ 'den belirgin biçimde düşük. Sınır değerleri: gelir tam 100 000 iken vergi 15 000, 250 000 iken 45 000, 600 000 iken 139 500; 100 001 iken 15 000,20. Bir liralık gelir artışı yirmi kuruşluk vergi getirir, yani “dilim atlayınca elime daha az geçer” korkusu yersizdir (Şekil 7.8).



GitHub

7.8.2 Tarih geçerliliği

Örnek 7.5 (Tarih geçerli mi?). Gün, ay ve yıl alan; tarihin geçerli olup olmadığını söyleyen bir program yazın. Ayın kaç gün çektiğini `match` ile belirleyin; şubat için artık yıl kuralını uygulayın.

Çözüm

Analiz. Üç girdi, bir çıktı. İki aşamalı karar: önce ay geçerli mi (1–12), sonra gün o ayın gün sayısını aşıyor mu. İkinci soru ancak birincinin cevabı “evet” ise sorulabilir — iç içe yazımın klasik gerekçesi. Artık yıl kuralını geçen hafta görmüştünüz: dörde bölünen yıllar artıktır, ama yüze bölünenler değildir; dört yüze bölünenler yine artıktır.

Kod ve çıktı.

Kod 7.11 w07-10-tarih.py — İç içe if ve match birlikte: tarih geçerliliği

```
1 # w07-10-tarih.py -- Tarih geçerliliği: gün/ay/yıl + artık yıl (6.
   ↪ hafta)
2
3 day = int(input("Gün: "))
4 month = int(input("Ay: "))
5 year = int(input("Yıl: "))
6
7 is_leap = (year % 4 == 0 and year % 100 != 0) or year % 400 == 0
8
9 if month < 1 or month > 12:
10     print(f"Geçersiz: {month}. ay yok.")
11 else:
12     match month:
13         case 1 | 3 | 5 | 7 | 8 | 10 | 12:
14             days_in_month = 31
15         case 4 | 6 | 9 | 11:
16             days_in_month = 30
17         case 2:
18             days_in_month = 29 if is_leap else 28
19     if 1 <= day <= days_in_month:
20         print(f"{day:02d}.{month:02d}.{year} geçerli bir tarih.")
21     else:
22         print(f"Geçersiz: {year} yılında {month}. ay {days_in_month}
   ↪ gün.")
```



GitHub

```
$ python w07-10-tarih.py
```

```
Gün: 29
Ay: 2
Yıl: 2024
29.02.2024 geçerli bir tarih.
```

Kontrol. 29.02.2024: 2024 dörde bölünür, yüze bölünmez → artık yıl → şubat 29 gün → geçerli ✓; aynı gün 2023'te geçersizdir (28 gün). Tuzak değerleri: 31.04 geçersiz (nisan 30 gün), 00.01 geçersiz, 13. ay yok; 1900 artık *değildir*, 2000 artıktır. Bu iki yıl da deneyin.

7.8.3 Taş-kâğıt-makas

Son uygulamamızda bilgisayarın seçimini `random` modülü yapar; kazananı belirleyen kural üç durumdan oluşur ve `or` ile tek koşulda toplanır.

Kod 7.12 w07-11-tas-kagit-makas.py — `random.choice` ve çok koşullu kazanç kuralı

```
1 # w07-11-tas-kagit-makas.py -- Taş-kâğıt-makas (tek el): random.choice
2 import random
3
4 OPTIONS = "TKM" # T: taş, K: kâğıt, M: makas
5 computer = random.choice(OPTIONS) # metinden rastgele bir karakter
6 user = input("Seçiminiz (T/K/M): ").strip().upper()
7
8 if len(user) != 1 or user not in OPTIONS:
9     print(f"Geçersiz seçim: '{user}'. T, K ya da M girin.")
10 else:
11     print(f"Siz: {user} Bilgisayar: {computer}")
12     if user == computer:
13         print("Berabere!")
14     elif ((user == "T" and computer == "M")
15           or (user == "K" and computer == "T")
16           or (user == "M" and computer == "K")):
17         print("Kazandınız!")
18     else:
19         print("Bilgisayar kazandı.")
```

```
$ python w07-11-tas-kagit-makas.py
```

```
Seçiminiz (T/K/M): k
Siz: K Bilgisayar: T
Kazandınız!
```

Üç seçenek ve iki oyuncu $3 \times 3 = 9$ birleşim üretir: üçü berabere, üçü sizin kazancınız, üçü bilgisayarın; adil bir oyunda her sonucun olasılığı $1/3$ 'tür. Kod 7.12'de berabere durumu tek bir eşitlikle önceden elenir, kazanç koşulu üç durumu `or` ile birleştirir, kalan her şey "bilgisayar kazandı" daldır — dokuz hücreli bir tabloyu üç satırla anlatmanın yolu budur.

Not

`random` modülü *sözde rastgele* (İng. *pseudorandom*) bir dizi üretir; dizinin başlangıcını *tohum* (İng. *seed*) belirler. Kitaptaki çıktıların aynı kalması için tohum sabitlenmiştir, sizde sonuç farklı çıkacaktır. Aynı özellik yapay zeka deneylerinde tekrarlanabilirlik için kullanılır.

LAB Laboratuvar Çalışması 7: Karar veren programlar

Önce beklenen çıktıyı yazın, sonra kodu; en sonda sınır değerlerini deneyin.

Görev 1. Girdi doğrulamalı yaş sorgusu. `yas.py`: yaşı isteyin; `metin.isdigit()` denetiminden geçmiyorsa 'abc' bir tam sayı değil. yazın, geçiyorsa `int()` ile çevirip $0 < n < 120$ aralığına bakın. Sınır denemeleri: 0 (ret), 1 (kabul), 119 (kabul), 120 (ret).

Görev 2. Menülü hesap makinesi. `hesap.py`: dört işlem ve "Çıkış" içeren bir menü yazın; seçimi `match` ile dallandırın. Seçim geçerliyse iki sayı isteyin (`try-except` ile doğrulayın), sonucu iki ondalık basamakla yazın; bölme durumunda *sıfıra bölme* kontrolünü unutmayın. Girdiler 12 ve 5 iken: 17.00, 7.00, 60.00, 2.40.

Görev 3. Kargo ücreti hesaplayıcı. `kargo.py`: [Örnek 7.2](#)'daki karar tablosunu kendi kodunuzla yazın, ama ağırlığı ve bölgeyi `try-except` ile doğrulayın. Denemeler: 2 kg / bölge 3 → 400 TL; 10.5 kg / bölge 1 → 150 TL; 0 kg → hata. Denemeleri bir test tablosuna yazın.

Görev 4. Tarih doğrulayıcı. `tarih.py`: gün, ay, yıl alın; tarihi GG.AA.YYYY biçiminde geçerli/geçersiz raporlayın. Ay uzunlukları `match` ile, artık yıl tek bir mantıksal ifadeyle. Denemeler: 31.04.2025 (geçersiz), 29.02.2024 (geçerli), 29.02.2023 (geçersiz), 00.01.2025.

Görev 5. Taş-kâğıt-makas. `tkm.py`: kullanıcıdan T, K ya da M alın (`strip().upper()`), bilgisayarın seçimini `random.choice("TKM")` ile yapın, kazananı yazın; geçersiz seçimi reddedin.

Teslim. Görev 1–3 ders bitmeden gösterilir; Görev 2'nin dosyasını saklayın.

Özet

Yapı	Yazım	Not
Karar tablosu	satır = dış zincir, sütun = iç zincir	hücre sayısı değerlerin çarpımıdır; n ikili koşul → 2^n
De Morgan	<code>not (a and b) ≡ not a or not b</code>	<code>and ↔ or</code> ; <code>< ↔ >=</code>
Desen eşleme	<code>match x: case 1 2: case _:</code>	ilk eşleşen çalışır; <code>_</code> en sonda; tür duyarlı
Biçim denetimi	<code>s.isdigit()</code>	yalnız işaretsiz tam sayı
Sınır değer analizi	her sınır için s ve $s + 1$	girdi-beklenen-gerçek tablosu



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w07>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırmalar

Alıştırma 7.1. ●● Aşağıdaki koşulları De Morgan kurallarıyla sadeleştirin; sonuçta hiçbir parantezin önünde `not` kalmamasın. (a) `not (age >= 18 and has_license)` (b) `not (x < 0 or x > 100)` (c) `not (day == 6 or day == 7)`

Cevap: (a) `age < 18 or not has_license` (b) `x >= 0 and x <= 100`, yani `0 <= x <= 100` (c) `day != 6 and day != 7`

Alıştırma 7.2. ●● Aşağıdaki iç içe yazımı, aynı davranışı koruyan tek seviyeli bir `if-elif-else` zincirine çevirin ve iki yazımı okunabilirlik açısından karşılaştırın: dışta `if score >= 0:`, içinde `if score >= 50: print("geçti")` ile `else: print("kaldı")`, dıştaki `else`'de `print("geçersiz not")`.

Cevap: `if score < 0: print("geçersiz not"); elif score >= 50: print("geçti"); else: print("kaldı")` — zincir daha kısadır ve girintisi tek seviyededir

Alıştırma 7.3. ●●● Ay numarasını (1–12) alan ve o ayın kaç gün çektiğini yazan bir program yazın: `match-case` ile, 31 ve 30 gün çeken ayları `|` ile gruplayarak, şubatı ayrı ele alarak, geçersiz numara için `_` dahilyla.

Cevap: 31 gün: 1, 3, 5, 7, 8, 10, 12; 30 gün: 4, 6, 9, 11; şubat 28 (artık yılda 29); girdi 4 için çıktı 4. ay 30 gün çeker.

Alıştırma 7.4. ●●● **Örnek 7.2**'daki kargo tarifi için bir sınır değer test tablosu yazın: her ağırlık sınırı için sınırın kendisini ve ondan biraz büyüğünü deneyin, bölgeyi 2 sabit tutun. Sütunlar: girdi, beklenen ücret, çalışan satır. Kaç satır elde ettiniz?

Cevap: 6 satır: 2 → 90 (hafif), 2,01 → 150 (orta), 10 → 150 (orta), 10,01 → 250 (ağır), artı aralık ortalarından 1 → 90 ve 5 → 150

Alıştırma 7.5. ●●● `match command.strip().lower()`: altında sırasıyla `case "sil" | "kaldır": print("A"), case "kopyala" if izin: print("B"), case "kopyala": print("C")` ve `case _: print("D")` yazılmış olsun. `command` sırasıyla `"sil"`, `"SIL"`, `"kopyala"` ve `"5"`, `izin` ise `False` iken hangi dal çalışır? `izin True` olsaydı hangi cevap değişirdi?

Cevap: "sil"→A; "SIL"→A (lower() uygulandığı için); "kopyala"→C (bekçi False); "5"→D. izin True olsaydı yalnız üçüncüsü B olurdu

Alıştırma 7.6. ●●● YKS puanını ve puan türünü (SAY, EA, SÖZ) alan bir öneri programı yazın (eşikler kurgusaldır): puan 100–560 aralığında bir sayı değilse hata; 150 altı baraj altı; 300 altı ön lisans; 400 altı lisans; 400 ve üzerinde SAY ve 480 üzeri ise tıp, bilgisayar ve yapay zeka mühendisliği, değilse mühendislik fakülteleri.

Cevap: girdi 452,5 ve say için Öneri: mühendislik fakülteleri. (çünkü 452,5 < 480); girdi abc için hata mesajı

Alıştırma 7.7. ●●● Üç kenar uzunluğu alan bir program yazın: kenarlar pozitif değilse reddedin; üçgen eşitsizliğini sağlamıyorlarsa “üçgen oluşmaz” deyin; oluşuyorsa eşkenar, ikizkenar ya da çeşitkenar olduğunu yazın.

Cevap: girdi 5, 5, 8 için Geçerli bir ikizkenar üçgen.; 1, 2, 3 için üçgen oluşmaz, çünkü 1 + 2 = 3 üçüncü kenardan büyük değildir

Alıştırma 7.8. ●●● Bir sıcaklık değeri ve birimini (C ya da F) alıp değeri diğer birime çeviren bir program yazın; değer sayı değilse ve birim C/F dışındaysa ayrı ayrı hata mesajı verin. $F = C \cdot 9/5 + 32$.

Cevap: girdi 36,6 ve c için 36.6 °C = 97.88 °F

Alıştırma 7.9. ●●● **Örnek 7.2**'daki programı genişletin: ağırlığı ve bölgeyi doğrulayın (geçersizse ayrı ayrı mesaj), “kırılabilir mi?” sorusunu sorun ve kırılabilir paketlerde ücreti %25 artırın; çıktıda ağırlık sınıfını da yazın.

Cevap: girdi 12,5 kg, bölge 1, kırılabilir e: 150 × 1,25 = 187,50 TL (“ağır” satırı)

Alıştırma 7.10. Kademeli elektrik faturası hesaplayın (tarife kurgusaldır): ilk 100 kWh 2,00 TL/kWh, 101–300 kWh arası 3,00, 300 üzeri 4,50; her fiyat yalnız kendi dilimine uygulanır. Enerji bedelini, %20 KDV'yi ve toplamı hizalı yazın; tüketimi `isdigit()` ile doğrulayın.

Cevap: 250 kWh için enerji 650,00 TL, KDV 130,00 TL, toplam 780,00 TL

Alıştırma 7.11. Bir tarihin yılın kaçınıcı günü olduğunu bulan bir program yazın: önce geçerliliği denetleyin (**Örnek 7.5**), sonra o aydan önceki ayların toplam gün sayısını `match` ile belirleyip güne ekleyin; artık yılda mart ve sonrası için bir gün ekleyin.

Cevap: 01.03.2024 → yılın 61. günü ($31 + 29 = 60$, artı 1); 31.12 normal yılda 365., artık yılda 366. gündür

Alıştırma 7.12. Kan grubu uyumunu bir karar tablosu olarak tasarlayın (kod yazmayın). Sekiz grup vardır: 0–, 0+, A–, A+, B–, B+, AB–, AB+. Kural: 0 grubu kan herkese, A grubu yalnız A ve AB'ye, B grubu yalnız B ve AB'ye, AB grubu yalnız AB'ye verilebilir; Rh– kan her alıcıya, Rh+ kan yalnız Rh+ alıcıya. (a) Kaç hücre? (b) Kaçı uyumlu? (c) A+ alıcı hangi gruplardan kan alabilir? (d) Koda çevirirken iç içe `if` mi, iki bağımsız koşulun `and`'i mi kısıdır; neden?

*Cevap: (a) $8 \times 8 = 64$ (b) 27 (c) 0–, 0+, A–, A+ (dört grup) (d) **and**: ABO uyumu ile Rh uyumu birbirinden bağımsızdır; ikisini ayrı hesaplayıp **and** ile birleştirmek 64 hücreyi iki küçük denetime indirir*

Hafta 8: Tekrar Yapıları: while Döngüsü ve Biriktiriciler

Öğrenme çıktıları

Bu haftanın sonunda:

1. bir **while** döngüsünü hazırlık, koşul, gövde ve güncelleme parçalarıyla yazabilecek, girintinin gövdeyi nasıl belirlediğini açıklayabileceksiniz;
2. üçüncü haftada sözde kodla yazdığınız "... İKEN" döngülerini Python'a çevirebilecek, sayaç kontrollü döngüleri kurabileceksiniz;
3. toplam, çarpım, sayaç ve en büyük/en küçük biriktiricilerini doğru ilk değerle kullanabileceksiniz;
4. nöbetçi değerle sonlanan döngüler ve "geçerli olana kadar sor" biçiminde girdi doğrulama döngüleri yazabileceksiniz;
5. **break**, **continue** ve **while True** desenini yerinde kullanabileceksiniz;
6. sonsuz döngü ve birim kayması hatalarını izleme tablosuyla bulup düzeltebileceksiniz.

Anahtar kavramlar

döngü (İng. *loop*) **tur** (İng. *iteration*) **sayaç** (İng. *counter*) **biriktirici** (İng. *accumulator*) **nöbetçi değer** (İng. *sentinel*) **girdi doğrulama döngüsü** (İng. *input validation loop*)
sonsuz döngü (İng. *infinite loop*) **birim kayması** (İng. *off-by-one*) **izleme tablosu** (İng. *trace table*) **bayrak değişkeni** (İng. *flag*) **break** **continue** **while True**

Şimdiye kadar yazdığınız programların hepsinin ortak bir sınırı vardı: her satır *en çok bir kez* çalışıyordu. Yedinci haftanın menü programı bir seçim yapıyor ve kapanıyordu; not ortalaması programı üç not alabiliyordu, otuz not değil. Oysa bilgisayarın insana üstünlüğü tam da burada: aynı işi yorulmadan, yanılmadan, milyonlarca kez yapabilmesinde. Bu haftanın konusu olan **tekrar yapısı** (İng. *repetition*), bu gücü programınıza katan yapıdır — ve üçüncü haftada kâğıt üzerinde tasarladığınız "... İKEN" döngülerinin Python'daki adı **while**'dir.

Döngü, üçüncü temel akış yapısıdır: *sıralı* akışı ikinci haftada, *seçme* yapısını altıncı ve yedinci haftada gördünüz; üçüncüsü tekrardır. Bu üçünün her program için yeterli olması, bilgisayar biliminin şaşırtıcı sonuçlarından biridir. Yapay zeka mühendisliği açısından bakarsanız, bir modelin "eğitilmesi" de bir döngüdür: veriyi gör, hatayı ölç, katsayıları düzelt, yeterince iyi olana kadar yinele. Bu haftanın **while** döngüleri, ileride yazacağınız eğitim döngülerinin küçük kardeşleridir.

8.1 Neden döngü? while'ın dört parçası

1'den 5'e kadar olan sayıları ekrana yazmak için beş **print()** satırı yazabilirsiniz. Peki 100'e kadar? Ya 1 milyona kadar? Kopyala-yapıştır bir çözüm değildir: dosya okunmaz hâle gelir, sınır değiştiğinde yüz satırı elle düzeltmek gerekir ve hata yapma olasılığı yüzle çarpılır. Döngü bu üç sorunu birden çözer: *kaç kez* sorusunun cevabı koda değil, bir *koşula* yazılır.

Tanım 8.1: while döngüsü

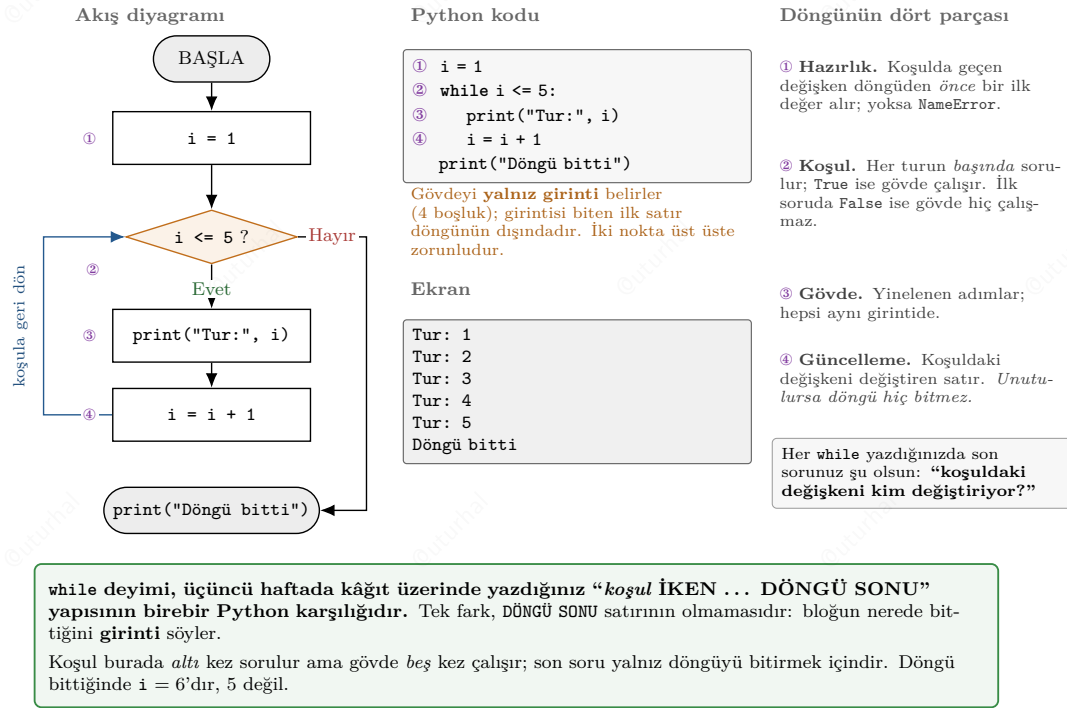
while deyimi, bir koşul **True** olduğu sürece bir blok deyimi yeniden çalıştırır:

```
while koşul:    ardından girintili gövde
```

Koşul *her turun başında* yeniden değerlendirilir; **False** olduğu anda döngü biter ve akış, gövdenin bittiği yerden — yani girintisi azalan ilk satırdan — sürer. Koşul ilk

sorulduğunda **False** ise gövde **hiç** çalışmaz. Gövdenin bir kez çalışmasına **tur** (İng. *iteration*) denir.

Bir döngü yazmak dört parçayı birden yazmaktır ve dördü de zorunludur: ① *hazırlık* (koşulda geçen değişkenin döngüden önceki ilk değeri), ② *koşul*, ③ *gövde* ve ④ *güncelleme* (koşulda geçen değişkeni değiştiren satır). Şekil 8.1, bu dört parçayı akış diyagramı ile Python kodu arasında birebir eşler.



Şekil 8.1: `while` döngüsünün akış diyagramı, Python kodu ve dört parçası. Aynı numaralar her iki tarafta aynı işi gösterir; **DÖNGÜ SONU** satırının yerini girinti alır.

Kod 8.1 w08-01-ilk-dongu.py — İlk `while` döngüsü: dört parça yan yana

```

1 # w08-01-ilk-dongu.py -- ilk while döngüsü: 1'den 5'e kadar yazdırma
2
3 i = 1                                # (1) başlangıç: sayaç ilk değerini alır
4 while i <= 5:                        # (2) koşul: doğruysa gövde yine çalışır
5     print("Tur:", i)                 # (3) gövde: girintili satırlar
6     i = i + 1                       # (4) güncelleme: sayaç bir ilerler
7 print("Döngü bitti. i =", i)        # koşul yanlışlanınca buraya gelir
  
```

\$ python w08-01-ilk-dongu.py

```

Tur: 1
Tur: 2
Tur: 3
Tur: 4
Tur: 5
Döngü bitti. i = 6
  
```



GitHub

Kod 8.1'de dikkat edilecek üç ayrıntı var. Birincisi, `while` satırının sonundaki **iki nokta**

üst üste zorunludur; altıncı haftada **if** için öğrendiğiniz kuralın aynısıdır. İkincisi, gövdeyi *yalnız girinti* belirler: dört boşlukla içeri alınmış iki satır gövdedir, üçüncü **print()** satırı döngünün dışındadır ve bir kez çalışır. Üçüncüsü, çıktının son satırı **i = 6** der — döngü bittiğinde sayaç, koşulu *yanlışlaştıran* değerdedir.

Kural 8.1 (Döngü yazmanın dört adımı). Her **while** döngüsünde şu dört soruyu sırayla cevaplayın:

- Hangi değişken değişiyor?** Ona döngüden önce bir ilk değer verin.
- Döngü ne zaman sürmeli?** Koşulu bu cevaba göre yazın (“ne zaman durmalı” değil).
- Her turda ne yapılıyor?** Gövdeyi yazın.
- Koşuldaki değişkeni kim değiştiriyor?** Cevabını gövdeye yazamıyorsanız döngünüz bitmeyecektir.

✗ Sık yapılan hata

while i <= 5 → **while i <= 5:** **if i <= 5: ...** → **while i <= 5: ...**

İki nokta üst üste unutulduğunda **SyntaxError: expected ':'**, gövde girintisiz bırakıldığında **IndentationError** alırsınız; ikisi de altıncı haftadaki **if** kurallarının aynısıdır — **while** yeni bir girinti kuralı getirmez. İkinci hata daha sinsidir: **if** ile **while** arasındaki tek harf farkı, “bir kez” ile “koşul doğru olduğu sürece” arasındaki farktır. “Neden yalnız bir satır yazdı?” diye şaşırdığınızda ilk bakacağınız yer burasıdır.

Şekil 8.2, dört parçadan biri eksik olduğunda ne olduğunu dört küçük programla gösterir: karşılaştığınız hemen her döngü hatası bu dört durumdan birine indirgenir.

✓ A — dördü de yerinde

```
i = 1           ①
while i <= 5:   ②
    print(i)    ③
    i = i + 1   ④
```

1 2 3 4 5 yazılır, döngü biter. Koşul 6 kez sorulur, gövde 5 kez çalışır.

✗ B — güncelleme ④ yok

```
i = 1
while i <= 5:
    print(i)
(i hiç değişmiyor)
```

Sonsuz döngü: 1 1 1 1 ... durmadan yazılır. Koşuldaki **i** hiç değişmediği için yanıt hep “Evet”. Durdurmak için **Ctrl+C**.

✗ C — hazırlık ① yok

```
(i = 1 satırı silindi)
while i <= 5:
    print(i)
    i = i + 1
```

Program *hiç* başlamaz: **NameError: name 'i' is not defined**. Koşul, olmayan bir değişkeni sorar.

② D — koşul baştan yanlış

```
i = 6
while i <= 5:
    print(i)
    i = i + 1
```

Hata yok, çıktı da yok: gövde **sıfır kez** çalışır. **while** bir “en az bir kez çalıştır” yapısı *değildir*.

Dört parçadan hangisi eksikse hata da ona göre. Güncelleme eksikse program *durmaz* (B); hazırlık eksikse *hiç başlamaz* (C); koşul ilk sorulduğunda yanlışsa gövde *hiç çalışmaz* (D) — ve bu üçüncüsü çoğu zaman bir hata değil, doğru davranıştır.

B ile C'nin farkını hatırlamanın kolay yolu: **C** ekrana bir şey yazmadan çöker, **B** ise durmadan yazar.

Şekil 8.2: Dört parçadan biri eksik olursa: (A) doğru döngü, (B) güncelleme yok → sonsuz döngü, (C) hazırlık yok → **NameError**, (D) koşul baştan yanlış → gövde hiç çalışmaz.

İpucu

Bir programı sonsuz döngüye soktuysanız panik yapmayın: terminalde ya da IDLE kabuğunda **Ctrl+C** çalışan programı keser (*KeyboardInterrupt*); Colab'da hücrenin durdurma düğmesi aynı işi yapar. Bu haftanın laboratuvarında bunu en az bir kez yapacaksınız — ve yapmanız iyidir.

8.2 Sayaç kontrollü döngü

Kaç tur döneceği önceden belli olan döngüye **sayaç kontrollü döngü** (*Ing. count controlled loop*) denir. Üçüncü haftada sözde kodla yazdığımız desenin Python karşılığı tam olarak şudur:

```
1 i = 1                # hazırlık
2 while i <= n:        # koşul
3     ...              # gövde
4     i = i + 1        # güncelleme (i += 1 ile aynı)
```

Sayaç yalnız birer birer artmak zorunda değildir: $i += 2$ ikiye ilerler, $i -= 1$ geri sayar. Koşul da buna göre değişir; geri sayarken $i >= 1$ yazılır, çünkü döngünün sürmesi *sayaçın hâlâ büyük olmasına* bağlıdır.

Kod 8.2 w08-02-sayac.py — Aynı sayaç deseninin üç biçimi: ileri, ikiye, geri

```
1 # w08-02-sayac.py -- sayaç kontrollü döngüler: ileri, ikiye, geri
2
3 n = int(input("Bir sayı girin: "))
4
5 # 1'den n'e kadar tüm sayılar, aynı satırda
6 i = 1
7 while i <= n:
8     print(i, end=" ")
9     i += 1
10 print()                # satırı bitir
11
12 # 2'den n'e kadar çift sayılar: sayaç ikiye artar
13 i = 2
14 while i <= n:
15     print(i, end=" ")
16     i += 2
17 print()
18
19 # n'den 1'e geri sayım: sayaç azalır, koşul ters döner
20 i = n
21 while i >= 1:
22     print(i, end=" ")
23     i -= 1
24 print("Kalkış!")
```

```
$ python w08-02-sayac.py
```

```
Bir sayı girin: 10
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10
10 9 8 7 6 5 4 3 2 1 Kalkış!
```



GitHub

Kod 8.2'te üç döngü art arda çalışır ve her biri kendi hazırlığını yeniden yapar: ikinci döngüden önce $i = 2$, üçüncüden önce $i = n$ yazılmıştır. Birinci döngüden kalan i değeri (11)

hiçbir işe yaramaz; **her döngü kendi hazırlığını kendisi yapar**. Çıktıdaki sayıların aynı satırda görünmesinin nedeni `print(i, end=" ")` yazımıdır: dördüncü haftada gördüğünüz `end` argümanı, satır sonu yerine bir boşluk koyar.

! Dikkat

Döngü bittiğinde sayacın değeri sınırdan *bir fazladır*: `while i <= 10` ile biten bir döngüden sonra `i = 11`'dir. Döngüden sonra sayacı “kaç tur döndük?” sorusunun cevabı olarak kullanacaksanız bunu hesaba katın ya da ayrı bir sayaç tutun. Bu tür hatalara **birim kayması** (*Ing. off-by-one*) denir ve 8.7’de bir izleme tablosuyla avlayacağız.

i İpucu

Bir döngüyü yazar yazmaz *küçük* bir değerle elle izleyin: $n = 3$ için sonuç doğru çıkıyorsa $n = 1000$ için de doğru çıkacaktır; birim kayması hataları küçük değerlerde de görünür ama gözle yakalanabilir. Üç satırlık bir izleme tablosu, yarım saatlik hata aramadan hızlıdır.

8.3 Biriktiriciler: toplam, çarpım, sayaç, en büyük

Döngünün asıl gücü ekrana bir şeyler yazmak değil, turlar boyunca bir sonucu *biriktirmektir*.

Tanım 8.2: Biriktirici

Biriktirici (*Ing. accumulator*), her turda kendi eski değeri üzerine yeni bir katkı ekleyerek büyüyen değişkendir: `total = total + x` (kısaca `total += x`). Biriktirici döngüden *önce* ilk değerini alır ve döngüden *sonra* okunur. İlk değer, işlemin etkisiz elemanıdır: toplamda 0, çarpımda 1, sayaçta 0.

Kod

```
total = 0   ilk değer
i = 1
while i <= 5:
    total = total + i
    i = i + 1
print(total)
```

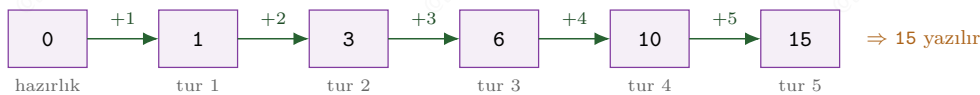
Biriktirici döngüden *önce* ilk değerini alır, gövdede *kendi eski değerinin* üzerine ekler, döngüden *sonra* okunur.

İlk değer nasıl seçilir?

Amaç	İlk değer	Gövdedeki satır
toplam	0	<code>total += x</code>
çarpım	1	<code>product *= x</code>
sayaç	0	<code>count += 1</code>
en büyük	<code>float("-inf")</code>	<code>if x > hi: hi = x</code>
en küçük	<code>float("inf")</code>	<code>if x < lo: lo = x</code>

İlk değeri seçerken tek soru: “*gövde hiç çalışmazsa hangi sonuç doğru olur?*” Toplamda 0, çarpımda 1 (etkisiz elemanlar). En büyük/en küçük için sonsuzlar yerine *ilk veriyi* de ilk değer yapabilirsiniz — bu derste çoğu zaman bunu yapacağız.

total değişkeninin turlar boyunca değeri



Biriktirici, döngünün “hafızası”dır: her tur onu bir parça büyütür, döngü bitince içinde bütün turların birikmiş sonucu durur. Zincirdeki oklar gövdenin `total = total + i` satırıdır — eşittirin sağındaki `total` eski, sol tarafındaki yeni değerdir.

En sık iki hata: biriktiriciyi döngünün *içinde* sıfırlamak (o zaman her tur baştan başlar, sonuç son terim olur) ve çarpım biriktiricisini 0 ile başlatmak (o zaman sonuç sonsuza kadar 0 kalır).

Şekil 8.3: Biriktirici deseni: `total` değişkeninin turlar boyunca aldığı değerler ve biriktirici türüne göre ilk değer seçimi.

İlk değeri seçerken tek bir soru yeter: “*gövde hiç çalışmazsa hangi sonuç doğru olur?*” Hiç sayı toplanmamışsa toplam 0’dır; hiç sayı çarpılmamışsa çarpım 1’dir (bu yüzden $0! = 1$ ’dir); hiç şey sayılmamışsa sayaç 0’dır. En büyüğü ararken ilk değer biraz daha düşündürür: `float("-inf")` (eksi sonsuz) her gerçek sayıdan küçüktür ve ilk karşılaştırmada hemen yenilir. Ama daha okunur bir yol, *ilk veriyi* ilk değer saymaktır; bu haftanın programlarında çoğunlukla bunu yapacağız.

Örnek 8.1 (Toplam ve faktöriyel: iki biriktirici bir arada). Kullanıcıdan bir n tam sayısı alan; $1 + 2 + \dots + n$ toplamını ve $n! = 1 \cdot 2 \cdot \dots \cdot n$ çarpımını *tek bir döngüde* hesaplayıp yazan bir program yazın. $n = 10$ ile deneyin.

Çözüm

Analiz. Girdi: n (tam sayı). Çıktı: iki sayı — toplam ve faktöriyel. İki ayrı biriktirici gerekir ve ilk değerleri *farklıdır*: toplam 0, çarpım 1. Sayaç 1’den n ’e gider; her turda ikisi de güncellenir. İki ayrı döngü yazmak da çalışırdı, ama aynı sayaç ikisine de hizmet ettiği için tek döngü yeter.

Algoritma.

Sözde kod: Toplam ve faktöriyel

```
BAŞLA
OKU n
toplam ← 0
carpim ← 1
i ← 1
i ≤ n İKEN
    toplam ← toplam + i
    carpim ← carpim * i
    i ← i + 1
DÖNGÜ SONU
YAZ toplam, carpim
BİTİR
```

İzleme tablosu. Küçük bir değerle, $n = 5$ ile izleyelim. Koşul ve sayaç turun *başındaki*, biriktiriciler turun *sonundaki* değerlerdir:

Tur	Koşul: $i \leq 5$?	i	total	product
—	—	1	0	1
1	Evet	1	1	1
2	Evet	2	3	2
3	Evet	3	6	6
4	Evet	4	10	24
5	Evet	5	15	120
—	Hayır	6	15	120

Kod ve çıktı.

Kod 8.3 w08-03-faktoriyel.py — İki biriktirici, tek döngü

```

1 # w08-03-faktoriyel.py -- iki biriktirici: 1 + 2 + ... + n ve n!
2
3 n = int(input("n: "))
4
5 total = 0                # toplam biriktiricisi 0'dan başlar
6 product = 1              # çarpım biriktiricisi 1'den başlar (0 değil!)
7 i = 1
8 while i <= n:
9     total += i            # total = total + i
10    product *= i          # product = product * i
11    i += 1
12
13 print(f"1 + 2 + ... + {n} = {total}")
14 print(f"{n}! = {product}")

```

```
$ python w08-03-faktoriyel.py
```

```

n: 10
1 + 2 + ... + 10 = 55
10! = 3628800

```

Kontrol. İzleme tablosu $n = 5$ için 15 ve 120 verdi; elle: $1 + 2 + 3 + 4 + 5 = 15$ ve $1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$ ✓. Programın çıktısı $n = 10$ içindir: $1 + 2 + \dots + 10 = \frac{10 \cdot 11}{2} = 55$ ✓ ve $10! = 3\,628\,800$ ✓. Uç durum: $n = 0$ girilirse koşul ilk soruda yanlıştır, gövde hiç çalışmaz ve biriktiricilerin ilk değerleri yazılır: toplam 0, çarpım 1 — matematikteki $0! = 1$ tanımıyla birebir aynı.



Sık yapılan hata

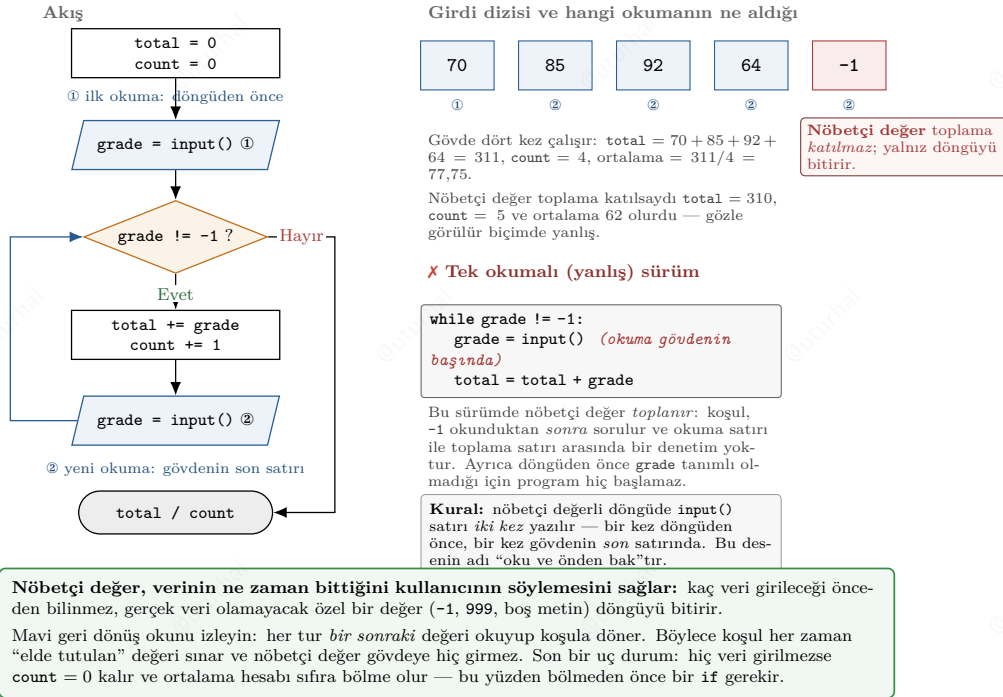
product = 0 → **product = 1** **döngünün içinde total = 0** → **döngüden önce total = 0**
 Biriktiricilerin iki klasik hatası. Çarpım biriktiricisi sıfırla başlatılırsa ilk turda $0 \cdot 1 = 0$ olur ve sonuç sonsuza kadar 0 kalır. İlk değeri hep şu soruyla seçin: “gövde hiç çalışmazsa hangi sonuç doğru olur?” İkinci hata, ilk değeri yanlışlıkla gövdeye yazmaktır: o zaman her tur sıfırdan başlar ve sonuç yalnız son terimi gösterir (1’den 10’a toplamda 55 yerine 10). Kural basit: **hazırlık satırları döngüden önce, güncelleme satırları gövdede.**

8.4 Nöbetçi değerle sonlanan döngü

Sayaç kontrollü döngülerde tur sayısı bellidir. Peki kaç veri girileceğini kullanıcının kendisi belirleyecekse? O zaman verinin bittiğini haber veren özel bir değer belirleriz.

Tanım 8.3: Nöbetçi değer

Nöbetçi değer (*İng. sentinel*), gerçek veri olamayacak, yalnız verinin bittiğini bildirmek için kullanılan özel bir değerdir: notlar okunurken -1 , sıcaklıklar okunurken 999, adlar okunurken boş metin. Nöbetçi değer **işlenmez**; yalnız döngüyü bitirir. Bu yüzden okuma satırı iki kez yazılır: bir kez döngüden *önce*, bir kez gövdenin *son* satırında.



Şekil 8.4: Nöbetçi değerle sonlanan döngünün akışı, girdi dizisinde hangi okumanın hangi değeri aldığı ve tek okumalı yanlış sürüm.

Örnek 8.2 (Nöbetçi değerle not ortalaması). Kaç tane olduğu önceden bilinmeyen sınav notlarını okuyan, -1 girilince duran; not sayısını, ortalamayı ve en yüksek notu yazan bir program yazın. Hiç not girilmezse sifıra bölme yapmasın. 70, 85, 92, 64 girdisiyle deneyin.

Çözüm

Analiz. Girdi: bilinmeyen sayıda not ve sonunda -1. Çıktı: sayı, ortalama, en yüksek. Üç biriktirici: *total* (0), *count* (0) ve *highest*. En yüksek için ilk değer sorunludur — 0 yazarsak bütün notlar negatif olduğunda yanlış olurdu; bu yüzden *ilk notu* en yüksek sayarız. Sıfıra bölmeyi önlemek için bölmeden önce bir *if* gerekir.

Algoritma.

Sözde kod: Nöbetçi değerle ortalama

```

BAŞLA
toplam ← 0 ; sayac ← 0
OKU not                      # ilk okuma: döngüden ÖNCE
not ≠ -1 İKEN
    EĞER sayac = 0 VEYA not > enyuksek İSE enyuksek ← not
    toplam ← toplam + not ; sayac ← sayac + 1
    OKU not                  # yeni okuma: gövdenin SON satırı
DÖNGÜ SONU
EĞER sayac > 0 İSE YAZ toplam / sayac DEĞİLSE YAZ "veri yok"
BİTİR
  
```

Kod ve çıktı.

Kod 8.4 w08-04-ortalama.py — Nöbetçi değer, üç biriktirici ve sıfıra bölme koruması

```

1 # w08-04-ortalama.py -- nöbetçi değerle biten döngü: -1 girilince dur
2
3 print("Notları girin; bitirmek için -1 yazın.")
4 total = 0
5 count = 0
6 highest = 0                                # geçici; ilk notta değişir
7
8 grade = float(input("Not: "))               # (1) ilk okuma döngüden ÖNCE
9 while grade != -1:                           # (2) nöbetçi değer geldi mi?
10     if count == 0 or grade > highest:        # ilk not ya da yeni rekor
11         highest = grade
12     total += grade
13     count += 1
14     grade = float(input("Not: "))           # (3) SON satır: yeni okuma
15
16 if count == 0:
17     print("Hiç not girilmedi.")
18 else:
19     print(f"{count} not girildi. Ortalama: {total / count:.2f}")
20     print(f"En yüksek not: {highest}")

```

```
$ python w08-04-ortalama.py
```

```

Notları girin; bitirmek için -1 yazın.
Not: 70
Not: 85
Not: 92
Not: 64
Not: -1
4 not girildi. Ortalama: 77.75
En yüksek not: 92.0

```

Kontrol. Elle: $70 + 85 + 92 + 64 = 311$ ve $311/4 = 77,75$ ✓. Makullük: ortalama en küçük ve en büyük notun arasında olmalı, $64 < 77,75 < 92$ ✓. Nöbetçi değerın toplama *katılmadığına* dikkat edin: katılsaydı toplam 310, sayaç 5 ve ortalama 62 çıkardı — gözle görülür biçimde yanlış. Uç durum: ilk girdi olarak doğrudan -1 verilirse gövde hiç çalışmaz, `count = 0` kalır ve program “Hiç not girilmedi” yazar; sıfıra bölme oluşmaz.



Sık yapılan hata

gövdenin başında `input()` → gövdenin sonunda `input()`

Nöbetçi değerli döngüde en sık yapılan hata, okuma satırını gövdenin *başına* koymaktır. O zaman nöbetçi değer önce okunur, sonra işlenir — yani -1 de toplama girer. Üçüncü haftada bu deseni kâğıt üzerinde görmüştünüz; Python’da da kural aynıdır: **ilk okuma döngüden önce, sonraki okumalar gövdenin son satırında.** (8.6’da bu ikilemi tek okumayla çözen `while True` desenini göreceğiz.)

8.5 Girdi doğrulama döngüsü

Yedinci haftada kullanıcı girdisini *denetlemeyi* öğrendiniz: `str.isdigit()` ile biçimi sıadı-
nız, `try-except ValueError` ile dönüşüm hatasını yakaladınız. Ama o programlar hatalı
girdide bir uyarı yazıp kapanıyordu. Artık daha iyisini yapabilirsiniz: **geçerli bir değer
girilene kadar sormak**.

Girdi doğrulama döngüsü

1. Bir **bayrak** (*Ing. flag*) değişkeni tanımlayın: `valid = False`.
2. **while not valid:** ile döngüyü açın — “geçerli olmadığı sürece sor”.
3. Gövdede girdiyi alın, çevirmeyi `try` içinde deneyin, aralığı `if` ile sıayın; her şey
uygunsa `valid = True` yazın, değilse *ne yanlış olduğunu* söyleyin ve döngü kendili-
ğinden yeniden sorsun.

Doğrulama her zaman *iki katmanlıdır*: önce tür, sonra aralık.

Kod 8.5 w08-05-dogrulama.py — “Geçerli olana kadar sor” deseni: tür ve aralık denetimi

```
1 # w08-05-dogrulama.py -- girdi doğrulama: geçerli olana kadar sor
2
3 valid = False
4 while not valid:
5     text = input("0-100 arası bir not girin: ")
6     try:
7         grade = int(text)                # 7. hafta: metni çevir
8         if 0 <= grade <= 100:
9             valid = True                # döngü sonraki turda biter
10        else:
11            print(" Hata: 0 ile 100 arasında olmalı.")
12    except ValueError:
13        print(" Hata: sayı girmelisiniz.")
14
15 print(f"Kabul edilen not: {grade}")
```

\$ python w08-05-dogrulama.py

```
0-100 arası bir not girin: abc
Hata: sayı girmelisiniz.
0-100 arası bir not girin: 105
Hata: 0 ile 100 arasında olmalı.
0-100 arası bir not girin: 87
Kabul edilen not: 87
```

Kod 8.5'daki bayrak değişkeni, koşulu okunur kılar: `while not valid` satırı İngilizce bir
cümle gibi okunur. Çıktıda üç deneme görürsünüz: `abc` tür denetimine, `105` aralık denetimine
takılır, `87` ikisini de geçer. Programın *çökmeye* dikkat edin — yedinci haftada aynı girdi
`ValueError` ile programı bitiriyordu.

⚠ Dikkat

Doğrulama döngüsünün bayrağını güncellemeyi unutursanız program sonsuza kadar aynı
soruyu sorar: kullanıcı doğru değeri girse bile `valid` hep `False` kalır. Bu, sonsuz döngünün
en can sıkıcı biçimidir, çünkü program “çalışıyor” görünür. Bayrağı *yalnız* başarı dalında
`True` yapın ve o satırı yazmadan döngüden çıkmayın.



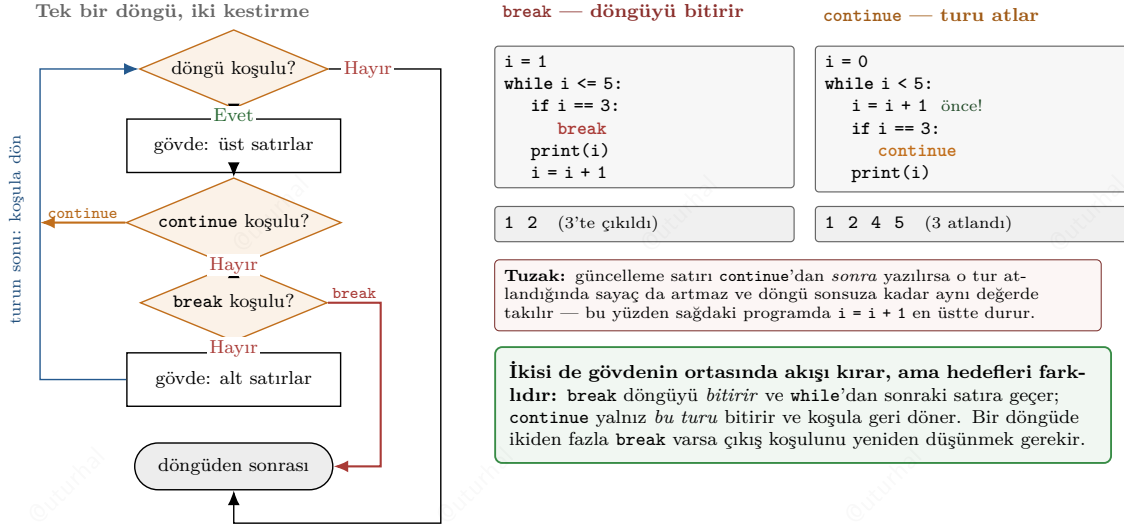
GitHub

8.6 break, continue ve while True

Bazen döngünün bitme koşulu koşul satırına sığmaz: tam ortada, bir girdi okunduktan sonra belli olur. Python bunun için iki kestirme sunar.

Tanım 8.4: break ve continue

break deyimi, içinde bulunduğu döngüyü **hemen bitirir**; akış **while** bloğundan sonraki satırdan sürer. **continue** deyimi ise yalnız **o turu** bitirir: gövdenin kalan satırları atlanır ve akış koşula geri döner. İkisi de yalnız döngü gövdesinde kullanılabilir.



Şekil 8.5: break ile continue'nun akış farkı. Kırmızı ok döngüden çıkar, turuncu ok turun sonuna atlar; ikisi de aynı gövdenin ortasından başlar.

Bu ikisi, **while True**: deseniyle birlikte kullanılır. **True** her zaman doğru olduğu için döngü kendiliğinden bitmez; çıkış *yalnız* bir **break** ile olur. Böylece “önce oku, sonra karar ver” türünden problemler tek okuma satırıyla yazılır — nöbetçi değerli döngüdeki çift okumaya gerek kalmaz.

Örnek 8.3 (Sayı tahmin oyunu). Bilgisayarın 1–100 arasında tuttuğu sayıyı tahmin ettiren bir oyun yazın. Her tahminden sonra “daha büyük” ya da “daha küçük” denilsin; doğru tahminde deneme sayısı yazılıp oyun bitsin. Aralık dışındaki tahminler uyarılsın ve *deneme sayılmasın*.

Çözüm

Analiz. Girdi: kullanıcının tahminleri (kaç tane olacağı belli değil). Çıktı: her turda bir ipucu, sonunda deneme sayısı. Bitiş koşulu “tahmin doğru” olmasıdır ama bunu ancak tahmini *okuduktan sonra* sınayabiliriz; bu yüzden **while True** ve **break**. Aralık dışı tahminler için **continue** kullanılır: sayacı artırmadan başa döner. Gizli sayı için yedinci haftanın **random** modülü yeter.

Algoritma. gizli ← rastgele sayı; deneme ← 0; *sonsuz döngü içinde:* OKU tahmin; aralık dışındaysa uyar ve turu atla (**continue**); deneme ← deneme + 1; tahmin küçükse “daha büyük”, büyükse “daha küçük”, eşitse deneme sayısını yaz ve çık (**break**).

Kod ve çıktı.

Kod 8.6 w08-06-tahmin.py — while True, break ve continue bir arada

```

1 # w08-06-tahmin.py -- sayı tahmin oyunu: while True, break ve continue
2
3 import random
4
5 secret = random.randint(1, 100)      # 1-100 arası rastgele sayı
6 attempts = 0
7 print("1 ile 100 arasında bir sayı tuttum. Bil bakalım!")
8
9 while True:                          # çıkış yalnız break ile
10     guess = int(input("Tahminin: "))
11     if guess < 1 or guess > 100:
12         print(" Aralık dışı, bu deneme sayılmaz.")
13         continue                    # kalanı atla, koşula dön
14     attempts += 1
15     if guess < secret:
16         print(" Daha büyük.")
17     elif guess > secret:
18         print(" Daha küçük.")
19     else:
20         print(f"Doğru! {attempts} denemede buldun.")
21         break                      # döngüden çık

```



```
$ python w08-06-tahmin.py
```

```

1 ile 100 arasında bir sayı tuttum. Bil bakalım!
Tahminin: 50
Daha küçük.
Tahminin: 25
Daha küçük.
Tahminin: 12
Daha büyük.
Tahminin: 0
Aralık dışı, bu deneme sayılmaz.
Tahminin: 18
Daha küçük.
Tahminin: 15
Daha büyük.
Tahminin: 16
Doğru! 6 denemede buldun.

```

Kontrol. Çıktıdaki oyunu izleyin: 50 ve 25 “daha küçük”, 12 “daha büyük” cevabını alır; üç ipucu gizli sayıyı 13–24 aralığına sıkıştırır. 18 ve 15’ten sonra kalan aralık 16–17’dir ve altıncı tahmin 16 doğrudur. Aradaki 0 tahmini aralık dışı olduğu için sayılmamıştır: yedi giriş yapılmış ama ekranda *altı* deneme yazmaktadır ✓. Her ipucunun aralığı en az yarıya indirdiğine dikkat edin — [Alıştırma 8.12](#) bunun üzerine kuruludur.

**Dikkat**

`while True` yazan bir programda gözünüz hemen `break` satırını aramalıdır. `break` yoksa döngü *hiçbir zaman* bitmez. Aynı biçimde, `continue` kullanan bir döngüde güncelleme

satırının `continue`'dan önce gelmesi gerekir; sonra gelirse atlanan turlarda sayaç artmaz ve döngü aynı değerde takılır (Şekil 8.5'teki kırmızı kutu).

Yedinci haftanın menü programı bir seçim yapıp kapanıyordu. Gerçek bir menü ise “Çıkış” seçilene kadar sürer — yani bir `while True` döngüsüdür.

Kod 8.7 w08-11-menu.py — Tekrarlı menü: 7. haftanın menüsü artık kapanmıyor

```
1 # w08-11-menu.py -- tekrarlı menü (ATM): while True, break ve continue
2
3 balance = 1000.0
4 while True:
5     print("1) Bakiye 2) Para yatır 3) Para çek 4) Çıkış")
6     choice = input("Seçiminiz: ")
7     if choice == "4":
8         print("İyi günler!")
9         break # döngüden çık, biter
10    if choice == "1":
11        print(f"Bakiyeniz: {balance:.2f} TL")
12    elif choice == "2":
13        amount = float(input("Yatırılacak tutar: "))
14        balance += amount
15    elif choice == "3":
16        amount = float(input("Çekilecek tutar: "))
17        if amount > balance:
18            print("Yetersiz bakiye!")
19            continue # kalanı atla, menüye dön
20        balance -= amount
21    else:
22        print("Geçersiz seçim.")
23        continue
24    print(f"İşlem tamam. Güncel bakiye: {balance:.2f} TL")
```

\$ python w08-11-menu.py

```
1) Bakiye 2) Para yatır 3) Para çek 4) Çıkış
Seçiminiz: 2
Yatırılacak tutar: 500
İşlem tamam. Güncel bakiye: 1500.00 TL
1) Bakiye 2) Para yatır 3) Para çek 4) Çıkış
Seçiminiz: 3
Çekilecek tutar: 2000
Yetersiz bakiye!
1) Bakiye 2) Para yatır 3) Para çek 4) Çıkış
Seçiminiz: 9
Geçersiz seçim.
1) Bakiye 2) Para yatır 3) Para çek 4) Çıkış
Seçiminiz: 3
Çekilecek tutar: 300
İşlem tamam. Güncel bakiye: 1200.00 TL
1) Bakiye 2) Para yatır 3) Para çek 4) Çıkış
Seçiminiz: 4
İyi günler!
```

Kod 8.7’de `break` ve `continue` birlikte çalışır: “4” seçilince `break` programı bitirir; yetersiz bakiye ya da geçersiz seçimde `continue` son satırı (“İşlem tamam”) atlayıp menüye döner. Bu desen — *menü, seçim, işlem, geri dön* — pek çok programın omurgasıdır.



GitHub

8.7 Sonsuz döngü, birim kayması ve while ... else

Döngülerin iki tipik hatası vardır ve ikisi de hata mesajı vermez; bu yüzden onları bulmanın tek yolu programı *elle* çalıştırmaktır.

① Güncelleme yok

```
i = 1
while i <= 5:
    print(i)
(i hiç artmıyor)
```

Koşuldaki değişkeni hiçbir satır değiştirmiyor: yanıt sonsuza kadar “Evet”.

② Yanlış yönde güncelleme

```
i = 1
while i <= 5:
    print(i)
    i = i - 1
```

Sayaç *küçülüyor*; koşul $i \leq 5$ hep doğru. Güncelleme var ama yönü ters.

③ Koşul hiç yakalanmıyor

```
x = 0
while x != 10:
    print(x)
    x = x + 3
```

x 0, 3, 6, 9, 12, ... olur; 10'a hiç eşit olmaz. != yerine < yazın.

① durumunun izleme tablosu

tur	i	$i \leq 5$?	ekran
1	1	Evet	1
2	1	Evet	1
3	1	Evet	1
4	1	Evet	1
...	...	...	...

Kırmızı sütun hiç değişmiyor. İzleme tablosunda sonsuz döngünün imzası budur: koşulda geçen değişkenin sütunu satırdan satıra aynı kalıyorsa, koşulun yanıtı da hiç değişmez. Bir döngüyü elle izlerken üç-dört satır yeter; desen kendini hemen gösterir.

Ekranda nasıl görünür? Program “kilitlenir”: ya aynı satır durmadan akar ya da hiçbir şey yazmadan bekler.

Durdurmak için terminalde ya da IDLE kabuğunda Ctrl+C; Colab'da hücrenin durdurma düğmesi.

Sonsuz döngü bir “çökme” değildir: program gayet mutlu çalışmaya devam eder, yalnız bitmez. Bu yüzden hata mesajı da yoktur — onu bulmanın yolu kodu okumak ve elle izlemektir.

Üç nedenin ortak kökü tek bir sorudur: “koşulu yanlışlaştıracak olan ne?” Cevabı yazamıyorsanız döngünüz bitmiyor demektir. `while True` ile yazılan döngülerde aynı soru şuna dönüşür: “`break` hangi satırda?”

Şekil 8.6: Sonsuz döngünün üç tipik nedeni ve izleme tablosundaki imzası: koşulda geçen sütun hiçbir satırda değişmiyor.

Tanım 8.5: Sonsuz döngü

Sonsuz döngü (*İng. infinite loop*), koşulu hiçbir zaman `False` olmayan döngüdür. Program çökmez, hata mesajı vermez; yalnız bitmez. Üç tipik nedeni vardır: güncelleme satırının hiç yazılmaması, güncellenmenin yanlış yönde olması ve koşulun `!=` ile yazılıp hedef değerin hiç yakalanmaması.

⚠ Dikkat

Bir döngünün koşulunda `float` değerlerini `==` ya da `!=` ile karşılaştırmayın. Altıncı haftada gördüğünüz gibi `0.1 + 0.2` sonucu tam olarak 0,3 değildir; bu yüzden `while x != 1.0` biçiminde bir koşul, `x` değeri “1,0’a çok yakın” olduğunda bile doğru kalabilir ve döngü bitmez. Ondalık bir değişkenle döngü kurarken `<` ya da `>` kullanın: `while x < 1.0`.

İkinci hata daha sinsidir: program *biter*, hata da vermez, yalnız cevap bir eksik ya da bir fazladır.

Tanım 8.6: Birim kayması

Birim kayması (*İng. off-by-one*), bir döngünün gerektiğinden bir tur eksik ya da bir tur fazla dönmesidir. Neredeyse her zaman koşuldaki `<` ile `<=` karışıklığından ya da sayacın yanlış değerle başlatılmasından kaynaklanır.

İzlenecek döngü

```
total = 0
i = 1
while i <= 10:
    total = total + i
    i = i + 1
print(total)
```

Tablo nasıl kurulur?

- Her değişkene bir sütun: **i**, **total**.
- Bir sütun da **koşula**: yanıtı Evet/Hayır yazın.
- Son sütun **ekrana**: yalnız print çalıştığında doldurun.
- Her satır **bir tur**: koşulu turun başındaki, biriktiriciyi turun sonundaki değerle yazın.

İzleme tablosu

tur	i	i ≤ 10?	total
–	1	–	0
1	1	Evet	1
2	2	Evet	3
3	3	Evet	6
4	4	Evet	10
		⋮	
9	9	Evet	45
10	10	Evet	55
–	11	Hayır	55

Döngü bitince $i = 11$ 'dir, 10 değil: koşul son bir kez daha sorulur ve ancak o zaman “Hayır” der.

Birim kayması (off-by-one) avı

Koşul	son tur	total
$i \leq 10$	$i = 10$	55 ✓
$i < 10$	$i = 9$	45 ✗
$i \leq 11$	$i = 11$	66 ✗

Üç yazım da *çalışır*, hiçbir hata vermez; ikisi yanlış cevap verir. Birim kayması hatalarının sinsiliği budur: sonuç doğruya çok yakındır.

Panzer: döngüyü yazar yazmaz küçük bir değerle ($n = 3$) elle izleyin. $1 + 2 + 3 = 6$ çıkmıyorsa koşul yanlış.

Sınır sorusu: “Son çalışması gereken tur hangisi?” Cevabınız $i = 10$ ise koşul $i \leq 10$ olmalıdır.

İzleme tablosu, hata ayıklamanın kalem-kâğıt sürümüdür: programı bilgisayar gibi, satır satır çalıştırıp her değişkenin değerini yazarsınız. Üçüncü haftada sözde kod için yaptığımız bu iş, Python kodunda birebir aynıdır.

İki hatayı da tablo ele verir: **sonsuz döngüde** koşul sütunu hiç “Hayır” olmaz, **birim kaymasında** ise son turun numarası beklediğinizden bir eksik ya da bir fazla çıkar.

Şekil 8.7: İzleme tablosunun kurulması ve birim kayması avı: aynı döngü üç farklı koşulla üç farklı sonuç verir, üçü de hatasız çalışır.

Şekil 8.7'ndeki üç yazımı karşılaştırm: `while i <= 10` ile 55, `while i < 10` ile 45, `while i <= 11` ile 66 elde edilir. Hiçbiri hata vermez; ikisi yanlış cevap verir. Kendinize sorun: “*son çalışması gereken tur hangisi?*” Cevabınız $i = 10$ ise koşul $i \leq 10$ olmalıdır.

İpucu

Döngünün içine geçici bir `print(i, total)` satırı koymak, elinizin altındaki en hızlı hata ayıklama aracıdır: izleme tablosunu programın kendisine doldurtmuş olursunuz. Dördüncü haftadaki `f"{i = }"` yazımı burada çok işe yarar — hem adı hem değeri yazar. Hatayı bulunca satırı silmeyi unutmayın.

8.7.1 while ... else

Python'un az bilinen bir yapısı, döngüye bağlı bir `else` bloğudur.

Not

Bir `while` döngüsüne `else` bloğu eklenebilir: bu blok, döngü **koşulu yanlışlaştığı için** bittiğinde çalışır, `break` ile çıktığında *çalışmaz*. Okunuşu “if değilse” değil, “döngü sonuna kadar gittiyse”dir. En yararlı olduğu yer arama problemleridir: aranan şey bulunca `break`, hiç bulunamazsa `else`. Yapı zorunlu değildir — aynı iş bir bayrak değişkeniyle de yapılır — ama Python kodu okurken karşınıza çıkacaktır.

Kod 8.8 w08-12-asal.py — while ... else ile asal testi

```
1 # w08-12-asal.py -- asal testi: while ... else ve i * i <= n sınırı
2
3 n = int(input("Bir tam sayı girin: "))
4
```



GitHub

```

5 i = 2
6 while i * i <= n:                # bölen aramak için  $\sqrt{n}$ 'e kadar yeter
7     if n % i == 0:
8         print(f"{n} asal değildir: {n} = {i} x {n // i}")
9         break                    # bölen bulundu; else bloğu ATLANIR
10    i += 1
11 else:                            # döngü break OLMADAN bittiyse çalışır
12     if n >= 2:
13         print(f"{n} asaldır ({i - 2} bölen denendi).")
14     else:
15         print(f"{n} asal değildir; asallar 2'den başlar.")

```

```
$ python w08-12-asal.py
```

```

Bir tam sayı girin: 97
97 asaldır (8 bölen denendi).

```

Kod 8.8, üçüncü haftada sözde kodla yazdığınız asal testinin Python karşılığıdır. Koşuldaki $i * i \leq n$ yazımı, bölen aramak için $\sqrt{n}$ 'e kadar gitmenin yeterli olduğunu söyler: n 'in $\sqrt{n}$ 'den büyük bir böleni varsa, ona eşlik eden ikinci bölen $\sqrt{n}$ 'den küçüktür ve zaten yakalanmıştır. 97 için döngü $i = 2$ ile 9 arasında döner: 8 bölen denenmiştir — 95 bölme yerine.

8.8 Uygulamalar

Bu bölümdeki dört program, döngünün klasik kullanım biçimlerini gösterir. Üçü üçüncü haftada sözde kodla tasarladığınız algoritmalarıdır; artık çalıştırılabilirler.

8.8.1 Bir sayıyı basamaklarına ayırmak

Örnek 8.4 (Basamak toplamı, basamak sayısı ve ters çevirme). Pozitif bir tam sayının basamaklarının toplamını, kaç basamaklı olduğunu ve rakamları ters sıralanmış hâlini yazan bir program yazın. 4725 ile deneyin.

Çözüm

Analiz. Girdi: pozitif tam sayı n . Çıktı: üç sayı. Dördüncü haftadaki $\%$ ve $//$ çifti burada bir döngüye girer: $n \% 10$ son basamağı verir, $n // 10$ son basamağı atar. Döngü n sıfırlanana kadar — yani basamak sayısı kadar — döner; n tükenince için aslı ayrı bir değişkende saklanır.

Algoritma. OKU n ; üç biriktiriciyi sıfırla; $n > 0$ İKEN: basamak $\leftarrow n \bmod 10$; toplam $\leftarrow$ toplam + basamak; adet $\leftarrow$ adet + 1; ters $\leftarrow$ ters $\cdot 10$ + basamak; $n \leftarrow n // 10$; DÖNGÜ SONU; YAZ üç sonucu.

İzleme tablosu. $n = 4725$ için dört tur (beşinci soruda $n = 0$ 'dır ve döngü biter):

Tur	n (tur başı)	digit	digit_sum	digit_count	reversed_n
—	4725	—	0	0	0
1	4725	5	5	1	5
2	472	2	7	2	52
3	47	7	14	3	527
4	4	4	18	4	5274

Kod ve çıktı.

Kod 8.9 w08-07-basamak.py — % ve // ile basamaklara ayırma

```

1 # w08-07-basamak.py -- basamak toplamı, sayısı ve tersi
2
3 n = int(input("Pozitif bir tam sayı girin: "))
4 original = n                # n döngüde tükenecek; aslını sakla
5
6 digit_sum = 0
7 digit_count = 0
8 reversed_n = 0
9 while n > 0:
10     digit = n % 10          # son basamak
11     digit_sum += digit
12     digit_count += 1
13     reversed_n = reversed_n * 10 + digit
14     n //= 10                # son basamağı at
15
16 print(f"{original} sayısının basamak toplamı: {digit_sum}")
17 print(f"Basamak sayısı: {digit_count}")
18 print(f"Tersi: {reversed_n}")

```

```
$ python w08-07-basamak.py
```

```

Pozitif bir tam sayı girin: 4725
4725 sayısının basamak toplamı: 18
Basamak sayısı: 4
Tersi: 5274

```

Kontrol. Elle: $4 + 7 + 2 + 5 = 18$ ✓; dört basamak ✓; ters 5274 ✓. Ters çevirme de bir biriktiricidir, yalnız toplama yerine “kaydır ve ekle” işlemiyle: her turda eski değer 10 ile çarpılıp yeni basamak eklenir. Uç durum: $n = 0$ girilirse gövde hiç çalışmaz ve üç sonuç da 0 çıkar.

8.8.2 Collatz dizisi: bitiş kanıtlanamayan döngü

Bir sayı çiftse yarıya bölün, tekse üç katının bir fazlasını alın; 1’e ulaşana kadar yineleyin. Bu basit kural **Collatz dizisini** üretir ve bugüne kadar denenmiş her başlangıç değeri için 1’e ulaşmıştır — ama *her* sayı için ulaşacağı kanıtlanamamıştır. Yani bu döngünün bittiğini kimse ispatlayamıyor.

Kod 8.10 w08-08-collatz.py — Collatz dizisi: adım sayısı ve tepe değer

```

1 # w08-08-collatz.py -- Collatz dizisi: 1'e ulaşmak kaç adım sürer?
2
3 n = int(input("Başlangıç sayısı: "))
4 steps = 0
5 peak = n                # dizide görülen en büyük değer
6
7 while n != 1:
8     if n % 2 == 0:
9         n = n // 2        # çiftse yarıya

```

```

10     else:
11         n = 3 * n + 1          # tekse üç katının bir fazlası
12         steps += 1
13         if n > peak:
14             peak = n
15
16     print(f"Adım sayısı: {steps}")
17     print(f"En büyük değer: {peak}")

```

```
$ python w08-08-collatz.py
```

```

Başlangıç sayısı: 27
Adım sayısı: 111
En büyük değer: 9232

```

Kod 8.10, 27 için 111 adım atar ve yolda 9232'ye kadar tırmanır. Bu kadar küçük bir başlangıçtan bu kadar uzun bir dizi çıkması, “küçük girdi = kısa döngü” sezgisinin her zaman doğru olmadığını gösterir. Koşuldaki $!=$ burada güvenlidir: n bir `int`'tir ve dizi 1'e tam olarak ulaşır.

8.8.3 Öklid algoritmasıyla EBOB

Örnek 8.5 (EBOB (Öklid algoritması)). İki pozitif tam sayının en büyük ortak bölenini, her adımı yazdırarak hesaplayan bir program yazın. 1071 ve 462 ile deneyin.

Çözüm

Analiz. Girdi: iki tam sayı. Çıktı: EBOB ve adım sayısı. Öklid'in gözlemi şudur: $EBOB(a, b) = EBOB(b, a \bmod b)$. Her adımda sayılar küçülür; kalan sıfır olduğunda bölen aranan cevaptır. Döngü koşulu $b \neq 0$ 'dır ve güncelleme tek satırda, dördüncü haftadaki çoklu atamayla yapılır: $a, b = b, a \% b$.

Algoritma. OKU a, b ; $b \neq 0$ **İKİ**EN: $kalan \leftarrow a \bmod b$; $a \leftarrow b$; $b \leftarrow kalan$; **DÖNGÜ SONU**; **YAZ** a . Üçüncü haftada bu algoritmayı izleme tablosuyla çalıştırmıştınız; tek yeni şey son iki satırın Python'da tek satıra inmesidir.

Kod ve çıktı.

Kod 8.11 w08-09-ebob.py — Öklid algoritması: kalan sıfır olana kadar böl

```

1  # w08-09-ebob.py -- Öklid algoritması: kalan sıfır olana kadar böl
2
3  a = int(input("Birinci sayı: "))
4  b = int(input("İkinci sayı: "))
5  first, second = a, b          # sonucu yazdırmak için sakla
6
7  steps = 0
8  while b != 0:
9      remainder = a % b
10     print(f" {a} = {a // b} x {b} + {remainder}")
11     a, b = b, remainder        # bölen bölünen olur, kalan bölen olur
12     steps += 1
13
14     print(f"EBOB({first}, {second}) = {a}    ({steps} adım)")

```



GitHub


```
$ python w08-09-ebob.py
```

```
Birinci sayı: 1071
İkinci sayı: 462
1071 = 2 x 462 + 147
462 = 3 x 147 + 21
147 = 7 x 21 + 0
EBOB(1071, 462) = 21 (3 adım)
```

Kontrol. Çıktıdaki üç satır elle doğrulanabilir: $1071 = 2 \cdot 462 + 147$, $462 = 3 \cdot 147 + 21$, $147 = 7 \cdot 21 + 0$. Demek ki $\text{EBOB}(1071, 462) = 21$ ✓. Bağımsız kontrol: $1071 = 21 \cdot 51$ ve $462 = 21 \cdot 22$; 51 ile 22'nin ortak böleni yoktur, dolayısıyla 21 gerçekten en büyüğüdür ✓. Algoritmanın hızına dikkat edin: dört basamaklı iki sayı üç adımda sonuca ulaştı.

8.8.4 Bileşik faiz: kaç yılda ikiye katlanır?

Örnek 8.6 (Birikim kaç yılda ikiye katlanır?). Ana parayı ve yıllık faiz oranını alan; birikimin ana paranın iki katına ulaşması için kaç yıl gerektiğini, her yılın sonundaki bakiyeyi yazarak hesaplayan bir program yazın. 10000 TL ve %6 ile deneyin.

Çözüm

Analiz. Girdi: ana para ve oran. Çıktı: yıl yıl bakiye ve toplam yıl sayısı. Burada tur sayısı önceden *bilinmez* — aradığımız şey zaten tur sayısıdır. Bu, **while** döngüsünün **for**'a göre üstün olduğu durumun tipik örneğidir: koşul “bakiye hedefin altında” olduğu sürece dön.

Algoritma. OKU anapara, oran; bakiye $\leftarrow$ anapara; yıl $\leftarrow$ 0; *bakiye* $<$ $2 \cdot$ *anapara*
İKEN: bakiye $\leftarrow$ bakiye $\cdot (1 + \text{oran})$; yıl $\leftarrow$ yıl + 1; **YAZ** yıl, bakiye; **DÖNGÜ SONU;** **YAZ** yıl.

Kod ve çıktı.

Kod 8.12 w08-10-faiz.py — Bileşik faiz: hedefe ulaşana kadar dön

```
1 # w08-10-faiz.py -- bileşik faiz: birikim kaç yılda ikiye katlanır?
2
3 principal = float(input("Ana para (TL): "))
4 rate = float(input("Yıllık faiz (%): ")) / 100
5
6 balance = principal
7 years = 0
8 while balance < 2 * principal:          # hedef: ana paranın iki katı
9     balance = balance * (1 + rate)      # yıl sonunda faiz eklenir
10    years += 1
11    print(f"{years:>2}. yıl sonu: {balance:>12,.2f} TL")
12
13 print(f"Para {years} yılda ikiye katlandı.")
14 print(f"72 kuralı tahmini: {72 / (rate * 100):.1f} yıl")
```

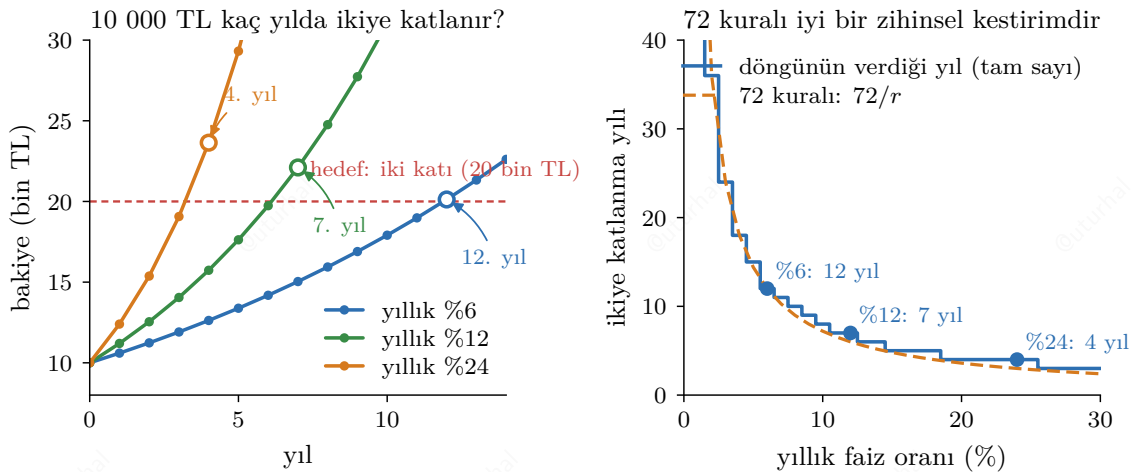


GitHub

```
$ python w08-10-faiz.py
```

```
Ana para (TL): 10000
Yıllık faiz (%): 6
1. yıl sonu: 10,600.00 TL
2. yıl sonu: 11,236.00 TL
3. yıl sonu: 11,910.16 TL
4. yıl sonu: 12,624.77 TL
5. yıl sonu: 13,382.26 TL
6. yıl sonu: 14,185.19 TL
7. yıl sonu: 15,036.30 TL
8. yıl sonu: 15,938.48 TL
9. yıl sonu: 16,894.79 TL
10. yıl sonu: 17,908.48 TL
11. yıl sonu: 18,982.99 TL
12. yıl sonu: 20,121.96 TL
Para 12 yılda ikiye katlandı.
72 kuralı tahmini: 12.0 yıl
```

Kontrol. On birinci yılın sonunda bakiye 18 982,99 TL'dir — hedefin altında; on ikinci yılın sonunda 20 121,96 TL olur ve döngü biter. Demek ki cevap 12 yıldır ✓. Kabaca kontrol: bankacıların “72 kuralı” der ki ikiye katlanma süresi yaklaşık $72/\text{oran}$ 'dır; $72/6 = 12$ ✓. **Şekil 8.8** aynı hesabı üç oran için gösterir: %12'de 7 yıl, %24'te 4 yıl. Dikkat: koşulda `float` değerleri karşılaştırılıyor ama `<` ile — eşitlik sorulmadığı için güvenlidir.



Şekil 8.8: Solda: üç faiz oranı için yıl yıl bakiye; içi boş daireler döngünün bittiği yıldır. Sağda: aynı sorunun cevabı oran oran, kesikli eğri “72 kuralı” yaklaşımı.

LAB Laboratuvar Çalışması 8: İlk döngüleriniz

Bu haftanın laboratuvarı üç bölümdür: önce üçüncü haftada kâğıtta tasarladığınız iki algoritmayı koda çevireceksiniz, sonra iki tam program yazacak, en sonunda bozuk bir programı düzelteceksiniz. Her görevde önce *izleme tablosunu* kâğıda çizin, sonra kodu yazın.

Görev 1. Akış diyagramından koda. Üçüncü haftada çizdiğiniz “1’den n ’e kadar

toplam” akış diyagramını açın ve `toplam.py` olarak yazın. Dört parçayı yorum satırlarıyla işaretleyin (`# hazırlık`, `# koşul`, `# gövde`, `# güncelleme`). $n = 100$ için beklenen çıktı: 5050. Sonra koşulu $i < n$ yapıp ne olduğunu görün ve farkı bir cümleyle yazın.

Görev 2. Geri sayım. `geri_sayim.py`: kullanıcıdan bir sayı alın ve ondan 1’e kadar geri sayıp son satırda **Kalkış!** yazın; sayılar aynı satırda, aralarında birer boşlukla görünsün (`end=" "`). Girdi 5: 5 4 3 2 1 **Kalkış!**. Ardından programı ikiye geri sayacak biçimde değiştirin ve 5 için çıktının 5 3 1 olduğunu doğrulayın.

Görev 3. Sayı tahmin oyunu. **Örnek 8.3**’i kendiniz yazın, ama aralık 1–50 olsun, en fazla 6 deneme hakkı verilsin ve hak biterse gizli sayı açıklansın. İpucu: deneme sayısını koşula taşıyın (`while attempts < 6`) ve doğru tahminde `break` kullanın.

Görev 4. Nöbetçi değerli ortalama. `ortalama.py`: sıcaklık değerleri alın, 999 girilince durun; ölçüm sayısını, ortalamayı, en düşüğü ve en yükseği yazın; hiç ölçüm yoksa “Ölçüm girilmedi” desin. Test verisi: 12,5 -3 7 0 -8,5 15 999 → ortalama 3.83, en düşük -8.5, en yüksek 15.0.

Görev 5. Menü döngüsü. Yedinci haftada yazdığınız menülü hesap makinesini `while True` ile döngüye alın: kullanıcı “Çıkış”ı seçene kadar sürsün, geçersiz seçimde `continue` ile menüye dönsün, son sonucu bir `result` değişkeninde hatırlasın.

Görev 6. Hata avı. Aşağıdaki üç programın her birinde bir hata var. Hatayı önce izleme tablosuyla bulun, türünü adlandırın (sonsuz döngü / birim kayması / yanlış ilk değer), sonra düzeltin:

- (a) `i = 1; while i <= 5: print(i)` — yalnız bu iki satır.
- (b) `total = 0; i = 1; while i < 10: total += i; i += 1` — 1’den 10’a toplam isteniyordu.
- (c) `product = 0; i = 1; while i <= 5: product *= i; i += 1` — 5! isteniyordu.

Teslim. Görev 1–4’ü ders bitmeden çalıştırıp gösterin. Görev 5 ve 6’yı bitiremezseniz evde tamamlayın; 10. haftada aynı programları for döngüsüyle yeniden yazacağız.

Özet

Yapı	Yazım	Not
Döngü	<code>while</code> koşul: ardından girintili gövde	koşul her tur <i>başında</i> sorulur; baştan yanlışsa gövde hiç çalışmaz
Dört parça	hazırlık, koşul, gövde, güncelleme	güncelleme yoksa sonsuz döngü
Sayaç	<code>i = 1; while i <= n; i += 1</code>	döngü bitince $i = n + 1$
Biriktiriciler	<code>total += x (0); product *= x (1); count += 1 (0)</code>	ilk değer: “gövde hiç çalışmazsa doğru sonuç ne?”
En büyük	<code>if x > hi: hi = x</code>	ilk değer: <code>float("-inf")</code> ya da ilk veri
Nöbetçi değer	döngüden önce bir okuma, gövdenin sonunda bir okuma	nöbetçi değer işlenmez, yalnız durdurur
Girdi doğrulama	<code>valid = False; while not valid:</code>	önce tür, sonra aralık
<code>break</code>	döngüyü hemen bitirir	<code>while True</code> ile birlikte
<code>continue</code>	yalnız o turu bitirir, koşula döner	güncelleme <code>continue</code> ’dan önce olmalı
<code>while ... else</code>	<code>else: break</code> olmadan bitince çalışır	arama problemlerinde “bulunamadı” dalı
Hata ayıklama	izleme tablosu; geçici <code>print()</code>	sonsuz döngü: değişmeyen sütun; birim kayması: kayan son tur



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w08>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırmalar

Alıştırma 8.1. ●●● Aşağıdaki döngülerin her biri *kaç tur* döner ve döngü bittiğinde *i*’nin değeri kaç olur? Çalıştırmadan, elle cevaplayın.

(a) `i = 1; while i <= 7: i += 1` (d) `i = 1; while i <= 10: i += 3`

(b) `i = 0; while i < 7: i += 1` (e) `i = 5; while i < 5: i += 1`

(c) `i = 10; while i > 0: i -= 2`

Cevap: (a) 7 tur, $i = 8$ (b) 7 tur, $i = 7$ (c) 5 tur, $i = 0$ (d) 4 tur, $i = 13$ (e) 0 tur, $i = 5$

Alıştırma 8.2. ●●● Aşağıdaki programı bir izleme tablosuyla çalıştırın (sütunlar: tur, koşul, `n`, `total`) ve ekrana ne yazıldığını bulun.

```

1 total = 0
2 n = 4
3 while n > 0:
4     total = total + n * n
5     n = n - 1
6 print(total)

```

Cevap: $16 + 9 + 4 + 1 = 30$ yazılır; döngü 4 tur döner ve biterken $n = 0$ 'dır

Alıştırma 8.3. ●● Aşağıdaki üç döngünün her biri sonsuza kadar çalışır. Her biri için (i) nedenini bir cümleyle yazın, (ii) tek satırlık bir düzeltme önerin.

(a) `i = 0; while i != 10: print(i); i += 3`

(b) `n = int(input()); while n > 0: print(n)`

(c) `password = input(); while password != "1234": print("Yanlış")`

Cevap: (a) i 0, 3, 6, 9, 12, ... olur ve 10'a hiç eşit olmaz; koşulu $i < 10$ yapın (b) güncelleme yok; gövdeye $n -= 1$ ekleyin (c) parola döngü içinde yeniden okunmuyor; gövdeye `password = input()` ekleyin

Alıştırma 8.4. ●● Aşağıdaki sözde kodu Python'a çevirin (kâğıt üzerinde) ve $n = 7$ için ekrana ne yazılacağını söyleyin.

Sözde kod: Tek sayıların toplamı

```

BAŞLA
OKU n
toplama ← 0
i ← 1
i ≤ n İKEN
    toplama ← toplama + i
    i ← i + 2
DÖNGÜ SONU
YAZ toplama
BİTİR

```

Cevap: `n = int(input()), total = 0, i = 1, while i <= n: içinde total += i ve i += 2, sonra print(total); n = 7 için 1 + 3 + 5 + 7 = 16`

Alıştırma 8.5. ●● Kullanıcıdan bir n sayısı alan; n 'den başlayarak 1'e kadar yalnız tek sayıları azalan sırada, aynı satırda yazan ve sonunda kaç sayı yazdığını bildiren bir program yazın. n çiftse bir altındaki tek sayıdan başlayın. Girdi 15: 15 13 11 9 7 5 3 1 ve 8 tek sayı yazıldı.

Cevap: *sayı azalır ($n -= 2$); 15 için sekiz sayı, 16 için de aynı sekiz sayı yazılır*

Alıştırma 8.6. ●● Sıcaklık ölçümlerini okuyan, 999 girilince duran ve şunları yazan bir program yazın: ölçüm sayısı, en düşük, en yüksek, ortalama ve sıfırın altındaki ölçümlerin sayısı. Hiç ölçüm girilmezse uygun bir ileti yazsın. Test verisi: 12,5, -3, 7, 0, -8,5, 15.

Cevap: 6 ölçüm; en düşük -8,5; en yüksek 15,0; ortalama $23/6 = 3,83$; sıfırın altında 2

ölçüm

Alıştırma 8.7. ●●● Fibonacci dizisinin $(1, 1, 2, 3, 5, 8, \dots)$ verilen bir üst sınırı aşmayan terimlerini yazan bir program yazın; sonunda kaç terim yazdığını ve son terimi bildirin. Sınır 1000: son terim 987.

Cevap: 16 terim yazılır; sınırı aşmayan en büyük terim 987'dir (bir sonraki terim $1597 > 1000$)

Alıştırma 8.8. ●●● Kullanıcıdan 1–12 arasında bir ay numarası isteyen; sayı olmayan ya da aralık dışındaki girdilerde nedenini söyleyip *yeniden soran* bir program yazın. Kaç denemede geçerli değer alındığını da yazsın. Denemeler: 13, mart, 3.

Cevap: ilk iki deneme reddedilir, üçüncüsü kabul edilir: Seçilen ay: 3 (3 denemede)

Alıştırma 8.9. ●●● Birinci haftada öğrendiğiniz “2’ye böl, kalanları ters sırada oku” yöntemini bir döngüyle programlayın: onluk tabandaki pozitif bir tam sayıyı ikili tabana çeviren bir program yazın. Girdi 45: 101101. (İpucu: basamakları bir metne biriktirin ve her yeni basamağı *sola* ekleyin.)

Cevap: $45 = 101101_2$; kalanlar sırayla 1, 0, 1, 1, 0, 1 ve ters okunur

Alıştırma 8.10. ●●● $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$ serisini, terim 10^{-6} ’nın altına düşene kadar toplayan bir program yazın; kaç terim topladığını, toplamı (10 ondalık basamakla) ve toplamın 2’ye uzaklığını yazsın.

Cevap: 20 terim toplanır; toplam 1,9999980927 ve 2’ye uzaklık $1,907 \times 10^{-6}$

Alıştırma 8.11. ●●● Borç tutarını, aylık faiz oranını ve aylık ödeme tutarını alan; borcun kaç ayda biteceğini ve toplam ne kadar ödeneceğini hesaplayan bir program yazın. Her ay önce faiz işler, sonra ödeme düşülür. Ödeme aylık faizi bile karşılamıyorsa program bunu söyleyip dursun. Girdi: 50 000 TL, aylık %2, ayda 2500 TL.

Cevap: borç 26 ayda biter; toplam ödenen 64 493,64 TL

Alıştırma 8.12. ●●● Örnek 8.3’i tersine çevirin: sayıyı *siz* tutun, bilgisayar tahmin etsin. Program her turda aralığın ortasını tahmin etsin; siz **b** (daha büyük), **k** (daha küçük) ya da **d** (doğru) yazın. En kötü durumda kaç tahmin gerektiğini de hesaplayın.

Cevap: aralığın ortası ($low + high$) // 2; 1–100 arasında en kötü durumda 7 tahmin yeter ($2^7 = 128 > 100$)

Hafta 9: Ara Sınav (Vize)

Öğrenme çıktıları

Bu hafta ders yoktur; ara sınav yapılır. Sınav 1.–8. haftaların tamamını kapsar ve yarıyıl notunun %25'ini oluşturur.

Kapsam

- **Hafta 1–3:** bilgisayarda temel kavramlar (sayı sistemleri, veri gösterimi, yazılım katmanları); algoritma, sözde kod, akış diyagramı, izleme tablosu; karar ve tekrar yapılarının akış diyagramıyla tasarımı.
- **Hafta 4–5:** değişkenler, veri türleri, tür dönüşümü, `input/print`, aritmetik operatörler ve öncelik, f-string; karakter dizisi indeksleme, dilimleme ve metotları.
- **Hafta 6–8:** karşılaştırma ve mantıksal operatörler, `if-elif-else`, iç içe koşullar, `match-case`, girdi doğrulama; `while` döngüsü, sayaç ve biriktiriciler, nöbetçi değer, `break/continue`.

Sınav biçimi

Kâğıt üzerinde, 90 dakika, 100 puan. Tipik dağılım: kavram soruları (%20), verilen kodun çıktısını bulma / izleme tablosu (%30), akış diyagramından koda çeviri (%20), problem çözümü için program yazma (%30). Hesap makinesi ve not kullanılmaz. Söz dizimi hataları, algoritma doğruysa küçük puan kaybıyla değerlendirilir; mantık hataları asıl puan kaybıdır.

Hazırlık önerileri

1. Her haftanın **Özet** tablosunu bir kâğıda kendi elinizle yeniden yazın.
2. Kırmızı **Sık yapılan hata** kutularının tamamını yeniden okuyun; sınav sorularının önemli bir kısmı bu hatalar üzerine kuruludur.
3. Alıştırmaların cevaplarına bakmadan en az birer kolay, orta ve zor alıştırmayı her haftadan yeniden çözün; sonra Ek F ile karşılaştırın.
4. Verilen bir kodun çıktısını *çalıştırmadan*, izleme tablosuyla bulma alışkanlığı kazanın; laboratuvar da arkadaşınızın yazdığı kodları böyle okuyun.
5. Eğitimci sürümündeki örnek vize (Ek S) sınıfta 8. haftanın sonunda çözülür; kendi çözümünüzü orada anlatılan puanlama ölçütleriyle notlandırın.

Hafta 10: for Döngüsü, İç İçe Tekrarlar ve Seriler

Öğrenme çıktıları

Bu haftanın sonunda:

1. **for** döngüsünü ve **range(başla, dur, adım)** işlevini kullanarak tekrar sayısı önceden belli olan problemleri çözebilecek, **dur** değerinin diziyeye *dahil olmadığını* kullanarak sınırları doğru yazabileceksiniz;
2. bir problemi gördüğünüzde **for** ile mi **while** ile mi çözüleceğine gerekçesiyle karar verebileceksiniz;
3. bir metnin karakterleri üzerinde dönerek sayma, şifreleme, tersleme ve palindrom denetimi yapabileceksiniz;
4. **enumerate**, **zip** ve **reversed** ile indeksli, eşleşmeli ve tersten dönen döngüler yazabilecek; iç içe döngülerle çarpım tablosu, desen ve ikili üretimi gibi iki boyutlu tekrarları kurabilecek, toplam adım sayısını hesaplayabileceksiniz;
5. döngü içinde biriktirerek bir seriyi toplayabilecek ve **for ... else** ile “bulunca dur” arama desenini yazabileceksiniz.

Anahtar kavramlar

for döngüsü (İng. *for loop*) **yineleyici** (İng. *iterable*) **yarı açık aralık** (İng. *half-open range*) **döngü değişkeni** (İng. *loop variable*) **iç içe döngü** (İng. *nested loop*) **desen** (İng. *pattern*) **seri** (İng. *series*) **kısmi toplam** (İng. *partial sum*) **yakınsama** (İng. *convergence*) **adım sayısı** (İng. *step count*)

Geçen hafta **while** ile tanıştınız: bir koşul doğru kaldığı sürece dönen, sayacı, koşulu ve güncellemesi sizin elinizde olan döngü. **while** her işi görür, ama sayacı ilk değerine koymayı, koşulu doğru yazmayı ve her turda artırmayı *unutmamak* zorundasınız; bu üç adımdan biri eksik kalınca ya sonsuz döngüye ya da bir eksik/bir fazla dönmeye (*off-by-one*) düşersiniz.

Bu hafta, tekrar sayısı *baştan belli* olan problemler için tasarlanmış ikinci döngüyü öğreniyoruz: **for**. “Yüz kez yap”, “1’den n’e kadar topla”, “bu metnin her karakteri için bak” cümlelerinin hepsi tek satırlık birer **for** başlığına dönüşür; sayaç, koşul ve artış o satırın içine gizlenir. **for** iç içe geçtiğinde tablolar, desenler ve ikili eşleşmeler gibi *iki boyutlu* tekrarlar da elinize geçer; 5. haftada metin metotlarıyla tek satırda çözdüğünüz problemleri — sesli harf sayma, Sezar şifrelemesi, palindrom — bu hafta karakter karakter yeniden yazacaksınız. Bir yapay zeka mühendisi için bunun anlamı büyüktür: bir veri kümesindeki her örnek üzerinde dönmek, her piksel üzerinde dönmek, bir eğitimin her adımını yinelemek — hepsi aynı **for** kalıbıdır.

10.1 for döngüsü ve range

Tanım 10.1: for döngüsü

for döngüsü, önceden belli bir *değerler dizisi* üzerinde baştan sona gezer. Yazımı **for <değişken> in <dizi>**: biçimindedir; iki nokta üst üstden sonraki girintili blok **gövdeyi** oluşturur. Her turda dizinin sıradaki değeri **döngü değişkenine** atanır ve gövde bir kez çalışır; dizi bitince döngü kendiliğinden sona erer. Üzerinde dönülebilen her nesneye **yineleyici** (İng. *iterable*) denir; bu hafta ikisini kullanacağız: **range()** ile üretilen sayı dizileri ve karakter dizileri (**str**).

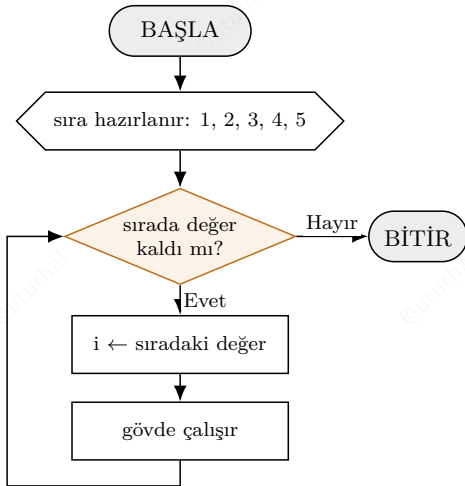
`while`'da ayrı satırlarda yazdığımız “başlangıç–koşul–artış” üçlüsü `for`'da tek bir başlığa toplanır. Sayı dizisini üreten işlev `range`'dir.

Kural 10.1 (`range` kuralı: **dur dahil değildir**). `range(başla, dur, adım)` *yarı açık* bir aralık üretir: `başla` dizinin ilk elemanıdır, `dur` ise **hiçbir zaman** üretilmez. Üç yazım biçimi vardır:

- `range(dur)` — 0'dan başlar, adım 1: `range(5)` → 0, 1, 2, 3, 4.
- `range(başla, dur)` — adım 1: `range(1, 6)` → 1, 2, 3, 4, 5.
- `range(başla, dur, adım)` — `range(0, 20, 5)` → 0, 5, 10, 15; adım negatif olabilir: `range(10, 0, -2)` → 10, 8, 6, 4, 2.

Üretilen eleman sayısı $\lceil (dur - başla) / adım \rceil$ 'dir; bu sayı sıfır ya da negatif çıkarsa dizi *boş* ve gövde hiç çalışmaz. Üç argüman da tam sayı olmalıdır; adım sıfır olamaz.

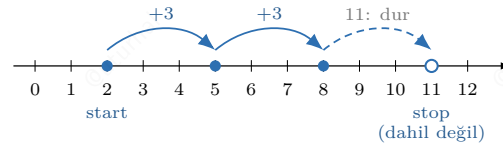
```
for i in range(1, 6): gövde
```



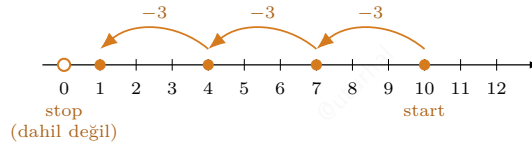
① Sayaç, koşul ve artış `range`'in içindedir; ayrı bir `i += 1` satırı yoktur. Sıra bitince döngü kendiliğinden biter.

`range(start, stop, step)` *yarı açık* bir aralıktır: **start dahil, stop dahil değil**. Pozitif adım için üretilen değer sayısı $\lceil (stop - start) / step \rceil$ 'dir: `range(1, 6)` beş, `range(2, 11, 3)` üç değer verir. `for` bu sırayı baştan sona gezer: her turda sıradaki değeri `i`'ye atar ve gövdeyi çalıştırır. `while`'daki “sayaç–koşul–artış” üçlüsü burada tek satıra sığmıştır.

`range(2, 11, 3)` → 2, 5, 8



`range(10, 0, -3)` → 10, 7, 4, 1



1 - 3 = -2 sıfırı geçti: dur. Negatif adımda `start > stop` olmalı; yoksa sıra boştur ve gövde hiç çalışmaz.

Şekil 10.1: `for` döngüsünün akışı (solda) ve `range`'in sayı doğrusu üzerinde ürettiği değerler (sağda). Dolu nokta üretilen, içi boş nokta üretilmeyen (`dur`) değeri gösterir.

Şekil 10.1'deki sol şemayı 8. haftanın `while` akış şemasıyla yan yana koyun: kutular aynıdır, fark yalnız sayacın nerede yönetildiğidir — `for`'da sayaç `range`'in içindedir, gövdeye bir `i += 1` satırı yazmanıza gerek yoktur.

Kod 10.1 w10-01-range.py — `range`'in üç biçimi, negatif adım ve boş aralık

```
1 # w10-01-range.py -- range()'in üç biçimi, negatif adım ve boş aralık
2
3 print("range(5):", end=" ")
4 for i in range(5):
5     print(i, end=" ")
6 print("\nrange(1, 6):", end=" ")
7 for i in range(1, 6):
```



GitHub

```

8     print(i, end=" ")
9     print("\nrange(0, 20, 5): ", end="")
10    for i in range(0, 20, 5):           # 5'er adımla: 0, 5, 10, 15
11        print(i, end=" ")
12    print("\nrange(10, 0, -2): ", end="")
13    for i in range(10, 0, -2):         # negatif adım: geri sayım
14        print(i, end=" ")
15    print("\nrange(5, 5):           ", end="")
16    for i in range(5, 5):             # boş aralık: gövde hiç çalışmaz
17        print(i, end=" ")
18    print("(gövde hiç çalışmadı)")
19
20    for _ in range(3):                # sayaç gerekmiyorsa adı _ olur
21        print("Python!", end=" ")
22    print()

```

```
$ python w10-01-range.py
```

```

range(5):          0 1 2 3 4
range(1, 6):       1 2 3 4 5
range(0, 20, 5):   0 5 10 15
range(10, 0, -2): 10 8 6 4 2
range(5, 5):       (gövde hiç çalışmadı)
Python! Python! Python!

```

Kod 10.1'in beşinci bölümü önemlidir: `range(5, 5)` boş bir dizidir, gövde *hiç* çalışmaz ve program hata vermeden devam eder — yanlış sınır yazdığınızda karşınıza çıkacak en sinsi hata budur. `range` ürettiği sayıları bir yere *yazmaz*; gerektiğinde birer birer hesaplar, bu yüzden kabukta yazdırırsanız sayıları değil tarafın kendisini görürsünüz. Sayıları görmek için `list(range(1, 6))` yazın: `[1, 2, 3, 4, 5]` — buradaki **liste** gösterimi 13. haftanın konusudur. `len(range(...))` ise bir döngünün kaç tur döneceğini *çalıştırmadan* söyler.

❌ Sık yapılan hata

`for i in range(1, n) → for i in range(1, n + 1)`

Bu haftanın en sık hatasıdır: “1’den *n*’e kadar” dendiğinde *n* de dahildir, ama **Kural 10.1** gereği `range` *dur* değerini üretmez. Bir eksik dönen döngü çökmez, yalnız *yanlış* sonuç verir (*n* = 100 için 5050 yerine 4950). 0’dan başlayan sayımlarda ise `range(n)` tam *n* kez döner — oraya +1 eklemek bir *fazla* dönmedir.

💡 İpucu

Döngü değişkenini gövdede kullanmayacaksanız adını alt çizgi (`_`) koyun: `for _ in range(3):` “bunu üç kez yap, kaçınıcı turda olduğum umurumda değil” demektir ve kodu okuyana “burada bir sayaç aramayın” der.

10.2 for mu, while mi?

İki döngü de aynı işi yapabilir; soru hangisinin problemi daha *dürüst* anlattığıdır. Ayrım tek bir soruya iner: *döngüye girmeden önce kaç tur döneceğinizi biliyor musunuz?* Biliyorsanız (“*n* kez”, “1’den 100’e”, “her karakter için”) **for**; bitiş ancak döngü çalışırken ortaya çıkan bir koşula bağlıysa (“kullanıcı -1 girene kadar”, “sayı 1’e inene kadar”) **while**. Diziyi gezerken erken çıkmak istiyorsanız — “bulunca dur” — **for** + **break** ikisini birleştirir. **Şekil 10.2** bu kararı satır satır özetler.

Durum	for	while
Tekrar sayısı <i>baştan</i> belli (“ <i>n</i> kez”, “1’den 100’e”)	✓ <code>range(n)</code>	mümkün, gereksiz uzun
Bir metnin <i>her karakteri</i> için bir iş	✓ <code>for ch in text</code>	indeksle, zahmetli
Bir koşul sağlanınca durmak (Collatz, EBOB, ikiye katlanma)	—	✓ <code>while</code> koşul
Kullanıcı “çık” deyinceye kadar menü	—	✓ <code>while True + break</code>
Geçerli girdi gelene kadar yeniden sormak	—	✓ doğrulama döngüsü
Sıra bitmeden “bulunca dur” (arama)	✓ <code>for + break</code>	mümkün

Aynı problem: $1 + 2 + \dots + n$

```
for version
total = 0
for i in range(1, n + 1):
    total += i
```

aynı sonuç

```
while version
total = 0
i = 1 # başlangıç
while i <= n: # koşul
    total += i
    i += 1 # artış (unutulursa sonsuz döngü)
```

Kural: kaç kez döneceğinizi (ya da neyin üzerinde döneceğinizi) *döngü başlarken* biliyorsanız `for`; döngünün ne zaman biteceğini ancak *çalışırken* öğrenecekseniz `while`. `for`, sayacı kendisi yönetir; bu yüzden “artışı unutma” ve “bir eksik/fazla dönme” hataları `for` ile çok daha seyrekler. Her `for` bir `while` ile yazılabilir, ama tersi her zaman doğal değildir.

Şekil 10.2: `for` ile `while` arasındaki karar tablosu ve aynı toplamın iki yazımı. `for` yazımında sayaç, koşul ve artış tek satırda toplanmıştır; bu yüzden “artışı unutma” hatası `for`’da yoktur.

Kod 10.2 w10-02-for-vs-while.py — Aynı toplam iki döngüyle: `while` ve `for`

```
1 # w10-02-for-vs-while.py -- Aynı problem iki döngüyle: 1'den n'e toplam
2
3 n = int(input("n: "))
4
5 total_w = 0 # while: sayaç, koşul, artış ayrı
6 i = 1
7 while i <= n:
8     total_w += i
9     i += 1
10
11 total_f = 0 # for: üçü de range'in içinde
12 for i in range(1, n + 1):
13     total_f += i
14
15 print(f"while ile : {total_w}")
16 print(f"for ile : {total_f}")
17 print(f"Gauss formülü n(n+1)/2 : {n * (n + 1) // 2}")
```

```
$ python w10-02-for-vs-while.py
```

```
n: 100
while ile : 5050
for ile : 5050
Gauss formülü n(n+1)/2 : 5050
```



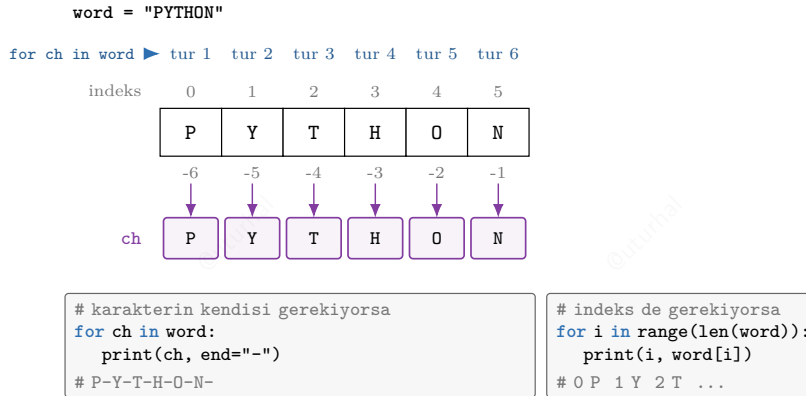
GitHub

Kod 10.2 aynı sonucu üç yoldan hesaplar ve üçünün de 5050 verdiğini gösterir. Üçüncüsü — Gauss’un $n(n+1)/2$ formülü — hiç döngü kullanmaz: *döngü, matematik bilmediğimiz yerde işe yarar.*

10.3 Metin üzerinde for

`for`'un ikinci kullanımı sayılar üzerinde değil, *karakterler* üzerinde dönmektir; bir karakter dizisi de bir yineleyicidir.

Kural 10.2 (Metin üzerinde döngü). `for ch in text:` yazıldığında döngü `len(text)` kez döner ve her turda `ch` değişkeni metnin sıradaki *karakteri* olur — bir indeks değil, karakterin kendisi. İndekse de ihtiyacınız varsa iki yol vardır: `for i in range(len(text)):` yazıp `text[i]` ile erişmek, ya da ikisini birden veren `enumerate` (10.4) kullanmak.



`for ch in word` döngüsünde `ch` bir sayı değil, karakterin kendisidir. Döngü `len(word)` kez döner; her turda sıradaki karakter `ch`'ye atanır. `word[ch]` yazmak hatadır (`TypeError`) — `ch` zaten karakterdir, indeks değildir. İndekse de ihtiyacınız varsa `range(len(word))` ile dönüp `word[i]` yazın, ya da ikisini birden veren `enumerate(word)`'u kullanın.

Şekil 10.3: Metin üzerinde `for`: döngü değişkeni karakterin kendisidir. Aynı iş indeksle de yapılabilir; seçim, gövdede indekse ihtiyaç duyup duymadığınıza bağlıdır.

Kod 10.3 w10-03-sesli-harf.py — Bir cümle'nin karakterlerini sayma: sesli harf, büyük harf, rakam

```
1 # w10-03-sesli-harf.py -- Metin üzerinde for: sesli harf, büyük harf, rakam
2
3 VOWELS = "aeioöuüAEIİÖÜÜ"
4 text = input("Bir cümle yazın: ")
5
6 vowel_count = 0 # sayaçlar döngüden ÖNCE sıfırlanır
7 upper_count = 0
8 digit_count = 0
9 for ch in text: # ch: sırayla metnin her karakteri
10     if ch in VOWELS:
11         vowel_count += 1
12     if ch.isupper():
13         upper_count += 1
14     if ch.isdigit():
15         digit_count += 1
16
17 print(f"Karakter sayısı : {len(text)}")
18 print(f"Sesli harf      : {vowel_count}")
19 print(f"Büyük harf      : {upper_count}")
20 print(f"Rakam          : {digit_count}")
```



GitHub

```
$ python w10-03-sesli-harf.py
```

```
Bir cümle yazın: Trabzon Üniversitesi 2026 YZ Mühendisliği
Karakter sayısı : 41
Sesli harf      : 13
Büyük harf      : 5
Rakam           : 4
```

Kod 10.3 üç **sayaç** (*İng. counter*) biriktiricisini aynı döngüde tutar: metni bir kez gezip üç soruyu birden cevaplar. Sayaç deseni her zaman aynıdır: döngüden *önce* sıfırla, gövdede artır, döngüden *sonra* yazdır — sıfırlamayı gövdeye koymak bu haftanın klasik hatasıdır, sonuç en çok 1 çıkar.



Sık yapılan hata

for ch in word: print(word[ch]) → **for ch in word: print(ch)**

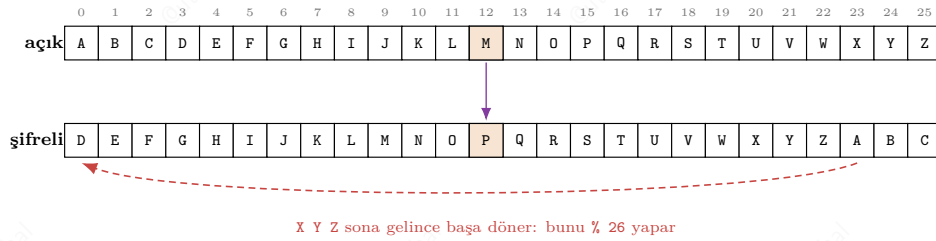
ch zaten bir karakterdir, indeks değildir; onu köşeli parantez içinde kullanmak **TypeError: string indices must be integers** verir (**Kural 10.2**). İki yazımı karıştırmamak için değişkeni adıyla ayırın: karakter üzerinde dönüyorsanız **ch**, indeks üzerinde dönüyorsanız **i**; **Şekil 10.3** bu ayrımı gösterir.

Örnek 10.1 (Sezar şifrelemesi ve çözümü). Bir mesajı ve bir kayma miktarını alan; her harfi alfabede k adım kaydırarak şifreleyen, sonra aynı programda geri çözerek doğruluğunu kendisi denetleyen bir program yazın. Harf olmayan karakterler (boşluk, rakam, noktalama) olduğu gibi kalsın. Örnek: "MERHABA DUNYA" ve $k = 3$.

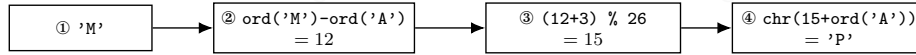
Çözüm

Analiz. Girdi: bir metin ve bir tam sayı. Çıktı: şifreli metin, çözülmüş metin ve bir doğruluk değeri. Kilit fikir 5. haftadan geliyor: harf, bellekte bir sayıdır. **ord("A")** = 65 olduğuna göre **ord(ch) - ord("A")** her büyük harfi 0–25 aralığına indirir. Kaydırdıktan sonra alfabenin sonunu başına bağlamak gerekir; bunu **% 26** yapar: 'X' → konum 23, $23 + 3 = 26$, $26 \bmod 26 = 0 \rightarrow 'A'$. Son adımda **chr** sayıyı harfe geri çevirir.

Algoritma. OKU mesaj, kayma; şifreli ← ""; **HER** karakter **İÇİN**: harfse şifreli ← şifreli + ((konum + kayma) mod 26)'nın harfi, değilse karakteri olduğu gibi ekle; **YAZ** şifreli.



Bir harfin yolculuğu (shift = 3):



Sezar şifrelemesi harfleri değil, harflerin konumlarını kaydırır. ord harfi sayıya, chr sayıyı harfe çevirir; ord('A') çıkarılınca 0...25 aralığına inilir. Kaydırmadan sonra % 26 almak, alfabenin sonunu başına bağlar: $23 + 3 = 26 \rightarrow 0$, yani 'X' $\rightarrow$ 'A'. Çözme aynı işlemin eksi yönüdür: $(\text{konum} - \text{shift}) \% 26$. Python'da negatif sayının modu da 0...25 aralığındadır, bu yüzden ek bir düzeltme gerekmez.

Şekil 10.4: Sezar şifresinde bir harfin yolculuğu: harf $\rightarrow$ konum (ord) $\rightarrow$ kaydır ve 26'ya göre kalanını al $\rightarrow$ harf (chr). Alt şerit, 3 kaydırılmış alfabadir.

Kod ve çıktı.

Kod 10.4 w10-04-sezar.py — Sezar şifreleme ve çözme: ord, chr ve % 26

```
1 # w10-04-sezar.py -- Sezar şifresi: harfleri k adım kaydır (ord/chr,
   ↪ mod 26)
2
3 message = input("Mesaj (İngiliz alfabesi, BÜYÜK harf): ")
4 shift = int(input("Kayma miktarı: "))
5
6 encrypted = ""
7 for ch in message:
8     if "A" <= ch <= "Z":
9         position = ord(ch) - ord("A")
10        encrypted += chr((position + shift) % 26 + ord("A"))
11    else:
12        encrypted += ch
13
14 decrypted = ""
15 for ch in encrypted:
16     if "A" <= ch <= "Z":
17         decrypted += chr((ord(ch) - ord("A") - shift) % 26 + ord("A"))
18    else:
19        decrypted += ch
20
21 print("Şifreli mesaj:", encrypted)
22 print("Geri çözüm   :", decrypted)
23 print("Kontrol      :", decrypted == message)
```



GitHub

```
$ python w10-04-sezar.py
```

```
Mesaj (İngiliz alfabesi, BÜYÜK harf): MERHABA DUNYA
Kayma miktarı: 3
Şifreli mesaj: PHUKDED GXQBD
Geri çözüm   : MERHABA DUNYA
Kontrol      : True
```

Kontrol. Elle: M alfabenin 13. harfidir, konumu 12'dir; $12 + 3 = 15$ ve 15. harf P'dir ✓. Aynı biçimde MERHABA → PHUKDED. Program son satırında çözülmüş metni orijinaliyle karşılaştırıp True yazar — kendi kendini denetleyen bir programdır. Uç durum: $k = 0$ ve $k = 26$ metni değiştirmez ($26 \bmod 26 = 0$), yani anlamlı kayma sayısı 25'tir. Bu, şifrenin zayıflığıdır: 25 denemeyle her mesaj kırılır (Alıştırma 10.7).



Dikkat

Kod 10.4 yalnız İngiliz alfabesinin 26 büyük harfiyle çalışır. Türkçenin Ç, Ğ, İ, Ö, Ş, Ü harfleri ASCII bölgesinin çok ötesinde, dağınık kod noktalarındadır; bu yüzden % 26 aritmetiği onlarda bozulur. Metni `upper()` ile büyümek de yetmez: `"i".upper()` sonucu 'I' dır, Türkçenin beklediği 'İ' değil. Girdiyi İngiliz alfabesiyle sınırlamak bu haftalık dürüst çözümdür.

Aynı yaklaşım 5. haftanın palindrom problemini de döngüyle çözer. Terslemenin sırrı, her yeni karakteri sonucun **başına** eklemektir (`rev = ch + rev`); sona eklerseniz metni aynen kopyalarsınız. Palindrom denetimi ise *i*. karakterle sondan *i*. karakteri (`word[n - 1 - i]`) karşılaştırır: $n/2$ karşılaştırma yeter.

Kod 10.5 w10-05-ters-palindrom.py — Döngüyle tersleme ve baştan-sondan palindrom denetimi

```
1 # w10-05-ters-palindrom.py -- Kelimeyi döngüyle tersleme ve palindrom testi
2
3 for _ in range(2):                                # iki kelime oynayacağız
4     word = input("Bir kelime: ").lower()
5
6     reversed_word = ""                             # her karakteri BAŞA ekle
7     for ch in word:
8         reversed_word = ch + reversed_word
9
10    is_palindrome = True                            # baştan ve sondan karşılaştır
11    n = len(word)
12    for i in range(n // 2):
13        if word[i] != word[n - 1 - i]:
14            is_palindrome = False
15            break                                    # tek uyumsuzluk yeter
16    print(f" tersi           : {reversed_word}")
17    print(f" dilimle aynı mı : {reversed_word == word[::-1]}")
18    print(f" palindrom mu   : {is_palindrome}")
```

```
$ python w10-05-ters-palindrom.py
```

```
Bir kelime: kayak
tersi           : kayak
dilimle aynı mı : True
```



GitHub

```

palindrom mu      : True
Bir kelime: python
tersi             : nohtyp
dilimle aynı mı  : True
palindrom mu      : False

```

"kayak" 5 karakterdir, $5/2 = 2$ karşılaştırma yapılır: (k, k) ve (a, a); ortadaki y sınanmaz. "python" içinse ilk karşılaştırma başarısız olur ve **break** döngüyü hemen bitirir. Program, döngüyle bulduğu tersi ile `word[::-1]` dilimini karşılaştırıp **True** yazar — döngü sürümünün bağımsız kanıtı.

10.4 enumerate, zip ve reversed

Üç yerleşik işlev, sık karşılaşılan üç döngü ihtiyacını kısaltır.

- `enumerate(dizi)` her turda *iki* değer verir: sıra numarası ve eleman. Numaralandırmayı 1'den başlatmak için `enumerate(dizi, start=1)`.
- `zip(dizi1, dizi2)` iki diziyi yan yana gezer; kısa olan bitince durur.
- `reversed(dizi)` diziyi sondan başa gezer; `range(len(w) - 1, -1, -1)` yazımının okunur hâlidir.

Kod 10.6 w10-06-enumerate-zip.py — enumerate, zip ve reversed ile metin döngüleri

```

1 # w10-06-enumerate-zip.py -- enumerate, zip ve reversed ile metin döngüleri
2
3 word = "PYTHON"
4
5 print("enumerate -- indeks ve karakter birlikte:")
6 for i, ch in enumerate(word):
7     print(f"{i}:{ch}", end=" ")
8
9 print("\nenumerate(start=1) -- 1'den numaralandırma:")
10 for no, ch in enumerate(word, start=1):
11     print(f"{no}. harf {ch}", end=" ")
12
13 print("\nzip -- iki metni eşleştirir, kısa olanda durur:")
14 for plain, secret in zip("ABCDE", "DEFGHIJ"):
15     print(f"{plain}->{secret}", end=" ")
16
17 print("\nreversed -- sondan başa:")
18 for ch in reversed(word):
19     print(ch, end=" ")
20 print("\naynı iş, geriye sayan range ile:")
21 for i in range(len(word) - 1, -1, -1):
22     print(word[i], end=" ")
23 print()

```

```
$ python w10-06-enumerate-zip.py
```

```

enumerate -- indeks ve karakter birlikte:
0:P 1:Y 2:T 3:H 4:O 5:N
enumerate(start=1) -- 1'den numaralandırma:
1. harf P 2. harf Y 3. harf T 4. harf H 5. harf O 6. harf N
zip -- iki metni eşleştirir, kısa olanda durur:
A->D B->E C->F D->G E->H
reversed -- sondan başa:
NOHTYP

```



GitHub

aynı iş, geriye sayan range ile:
NOHTYP

Kod 10.6'in son iki döngüsü aynı çıktıyı verir: biri **reversed**, öteki geriye sayan bir **range**. İkincisindeki **range(len(word) - 1, -1, -1)** şöyle okunur: son indeksten başla, -1'e *varmadan* dur (yani 0'da bitir), her adımda bir azal.

İpucu

zip'in kısa dizide durması bir hata değil, tasarım kararıdır: eşleşmeyen artık elemanlar sessizce atılır — **zip("abc", "12345")** üç çift verir. Eşit uzunluk gerekiyorsa döngüden önce **len(a) == len(b)** karşılaştırmasını kendiniz yapın.

10.5 İç içe döngüler

Bir döngünün gövdesi de bir döngü olabilir. Bu, bir boyutlu tekrardan *iki* boyutlu tekrara geçiştir: satır-sütun, gün-saat, oyuncu-rakip.

Kural 10.3 (İç içe döngünün adım sayısı). Dış döngü n , iç döngü her dış turda m kez dönüyorsa iç gövde toplam $n \cdot m$ kez çalışır. Dış döngü bir adım ilerlemeden önce iç döngü *baştan sona* döner; iç döngüden sonra gelen ama dış döngünün içinde kalan satırlar her dış turda bir kez çalışır. $n = m$ olduğunda toplam adım n^2 'dir.

Adımların sırasını kâğıtta bir kez yazmak bu kuralı kalıcı kılar: $i = 1$ iken j sırayla 1,2,3; sonra $i = 2$ iken yine 1,2,3; sonra $i = 3 \dots$ — üç dış tur $\times$ üç iç tur = dokuz adım. İç içe döngünün en tanıdık ürünü çarpım tablosudur: dış döngü satırları, iç döngü sütunları gezer. Hızlı bir tablo için 4. haftanın f-string genişlik belirteçleri (**f"{x:>4}"**) **print(..., end="")** ile birleştirilir.

Kod 10.7 w10-07-carpim-tablosu.py — İç içe for ile hızlı çarpım tablosu

```
1 # w10-07-carpim-tablosu.py -- İç içe for ile hızlı çarpım tablosu (1-9)
2
3 N = 9
4
5 print("    |", end="")                # başlık: sütun numaraları
6 for j in range(1, N + 1):
7     print(f"{j:>4}", end="")
8 print()
9 print("----+ " + "----" * N)
10
11 for i in range(1, N + 1):           # dış döngü: satırlar
12     print(f"{i:>3} |", end="")
13     for j in range(1, N + 1):       # iç döngü: sütunlar
14         print(f"{i * j:>4}", end="")
15     print()                         # iç döngü bitti: satır sonu
```

\$ python w10-07-carpim-tablosu.py

	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36



GitHub

5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

Kod 10.7'nde iç döngü 9 sütunu, dış döngü 9 satırı yazar; toplam 81 çarpım hesaplanır. Satır sonuna geçiren boş `print()` çağrısının iç döngünün dışında ama dış döngünün içinde olduğuna dikkat edin. Tablodaki sayıların toplamı 2025'tir; $1 + 2 + \dots + 9 = 45$ olduğundan bu 45^2 'ye eşittir — iyi bir kontrol.

❌ Sık yapılan hata

satır sonu `print()`'i iç döngünün içinde → **iç döngüden sonra, dış döngünün içinde**
İç içe döngülerde hataların çoğu girintiden doğar. Her satır için kendinize “bu satır her iç turda mı, her dış turda mı, yoksa döngüler bittikten sonra bir kez mi çalışmalı?” diye sorun; cevabı girintiye çevirin. Üç seviye: iç gövde (8 boşluk), dış gövde (4 boşluk), döngü sonrası (0 boşluk).

İç döngünün sınırı dış döngünün değişkenine de bağlı olabilir. Bir grupta *her ikiliyi* bir kez üretmek için iç döngü dış döngünün bir sonrasından başlar: dıştaki `for i in range(1, 4):` ile içteki `for j in range(i + 1, 4):` birlikte (1,2), (1,3), (2,3) çiftlerini verir. Üç elemandan 3 ikili çıkar; n eleman için $n(n-1)/2$ ikili olur ($n = 10$ için 45). İç döngü de `range(1, 4)` olsaydı kendiyile eşleşmeleri ve tekrarları da üretirdiniz — turnuva eşleşmesi ya da benzerlik karşılaştırmasında istediğiniz bu değildir.

$n = 5$ için piramit

$i = 1$					*				
$i = 2$				*	*	*			
$i = 3$			*	*	*	*	*		
$i = 4$		*	*	*	*	*	*	*	
$i = 5$	*	*	*	*	*	*	*	*	*

4 boşluk + 1 yıldız

3 boşluk + 3 yıldız

2 boşluk + 5 yıldız

1 boşluk + 7 yıldız

0 boşluk + 9 yıldız

```
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

```
# aynı satır, iç döngülerle
for j in range(n - i):
    print(" ", end="")
for j in range(2 * i - 1):
    print("*", end="")
print()
```

Desen problemlerinin tamamı tek bir soruya indirgenir: “ i . satırda kaç boşluk, kaç yıldız var?” Satır numarasıyla sayıları ilişkilendiren formülü bulduğunuzda kod kendiliğinden yazılır. Piramitte boşluk sayısı $n - i$, yıldız sayısı $2i - 1$ 'dir (tek sayılar: 1, 3, 5, ...); toplam genişlik her satırda $n - 1 + 2i - 1 \leq 2n - 1$ 'dir. `print()` her çağrıda satır sonuna geçtiği için, satırı parça parça yazarken `end=""` gerekir; satır sonunu ise iç döngüden sonraki boş `print()` koyar.

Şekil 10.5: Piramit deseninin oluşumu: i . satırda $n - i$ boşluk ve $2i - 1$ yıldız vardır. Açık gri hücreler boşluğu, mavi hücreler yıldızı gösterir.

Örnek 10.2 (Üç yıldız deseni). Kullanıcıdan satır sayısı n alan ve üç deseni alt alta çizen bir program yazın: sola dayalı üçgen, sağa dayalı üçgen ve ortalanmış piramit. $n = 5$ için piramidin son satırı 9 karakter genişliğinde olmalıdır.

Çözüm

Analiz. Üç desenin de tek sorusu şudur: i . satırda kaç boşluk, kaç yıldız var? Sol üçgende i yıldız, boşluk yok; sağ üçgende $n - i$ boşluk + i yıldız; piramitte $n - i$ boşluk + $2i - 1$ yıldız (tek sayılar: 1, 3, 5, ...). Yazdırma tekniği ortaktır: satırı parça parça `print(..., end="")` ile yazın, satır sonunda boş bir `print()` koyun. Metin yineleme operatörüyle ("`*`" * n , 5. hafta) iç döngüden tamamen de kurtulabilirsiniz.

Algoritma. HER i İÇİN 1'den n 'e: YAZ ($n - i$) boşluk ve $(2i - 1)$ yıldız, satır sonuna geç.

Kod ve çıktı.

Kod 10.8 w10-08-desenler.py — İç içe döngüyle üç yıldız deseni

```
1 # w10-08-desenler.py -- İç içe döngüyle yıldız desenleri: sol, sağ,
   ↪ piramit
2
3 n = int(input("Satır sayısı: "))
4
5 print("Sol üçgen:")
6 for i in range(1, n + 1):           # i. satırda i yıldız
7     for j in range(i):
8         print("*", end="")
9     print()
10
11 print("Sağ üçgen:")
12 for i in range(1, n + 1):           # önce n-i boşluk, sonra i yıldız
13     for j in range(n - i):
14         print(" ", end="")
15     print("*" * i)
16
17 print("Piramit:")
18 for i in range(1, n + 1):           # n-i boşluk, 2i-1 yıldız
19     print(" " * (n - i) + "*" * (2 * i - 1))
```



GitHub

```
$ python w10-08-desenler.py
```

```
Satır sayısı: 5
```

```
Sol üçgen:
```

```
*
**
***
****
*****
```

```
Sağ üçgen:
```

```
  *
  **
 ***
****
*****
```

```
Piramit:
```

```
  *
 ***
*****
*****
*****
```

Kontrol. $n = 5$ için sol üçgen $1+2+3+4+5 = 15$ yıldız, piramit $1+3+5+7+9 = 25$ yıldız kullanır; piramidin son satırı $2 \cdot 5 - 1 = 9$ karakterdir ✓. Uç durum: $n = 1$ girildiğinde her desen tek bir yıldızdan oluşur; $n = 0$ girildiğinde hiçbir satır yazılmaz (boş `range`) ve program hata vermez.

10.6 Döngüyle biriktirme: seriler

8. haftada **biriktirici** (*Ing. accumulator*) desenini öğrendiniz: döngüden önce bir değişkeni başlat, her turda güncelle, döngüden sonra kullan. `for` ile bu desen en doğal hâlini alır, çünkü “kaç terim toplayacağım” sorusunun cevabı `range`’in içindedir.

Biriktirici deseni (for ile)

1. Döngüden **önce** biriktiriciyi *etkisiz elemanla* başlat: toplam için `total = 0.0`, çarpım için `product = 1`, sayaç için `count = 0`.
2. Döngü başlığında terim sayısını belirt: `for k in range(1, n + 1):`.
3. Gövdede yalnız *tek bir terimi* hesapla ve ekle: `total += 1 / k`.
4. Döngüden **sonra** sonucu yaz; gövdeye koyarsanız her ara toplam dökülür.

Bir **seri**, sonsuz sayıda terimin toplamıdır; ilk n teriminin toplamına **kısmi toplam** denir. Bilgisayar sonsuz terim toplayamaz, ama kısmi toplamlar bir sayıya **yakınsıyorsa** yeterince çok terim alarak o sayıyı istediğimiz duyarlılıkta elde ederiz. Üç klasik seri:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \quad (\text{harmonik toplam — yakınsamaz, sonsuza gider})$$

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots \right) \quad (\text{Leibniz serisi})$$

$$e = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \cdots \quad (\text{her terim öncekinin } k\text{'ya bölümü})$$

Kod 10.9 w10-09-seriler.py — Üç seri: harmonik toplam, Leibniz ile π , $1/k!$ ile e

```

1 # w10-09-seriler.py -- Döngüyle biriktirilen seriler: harmonik toplam,  $\pi$ ,  $e$ 
2
3 import math
4
5 n = int(input("Terim sayısı n: "))
6
7 harmonic = 0.0                                #  $1 + 1/2 + 1/3 + \dots + 1/n$ 
8 for k in range(1, n + 1):
9     harmonic += 1 / k
10
11 pi_sum = 0.0                                  #  $4(1 - 1/3 + 1/5 - 1/7 + \dots)$ 
12 sign = 1
13 for k in range(n):
14     pi_sum += sign / (2 * k + 1)
15     sign = -sign                               # işaret her terimde değişir
16 pi_approx = 4 * pi_sum
17
18 e_sum = 1.0                                    #  $k = 0$  terimi:  $1/0! = 1$ 
19 term = 1.0
20 for k in range(1, n):
21     term = term / k                            #  $1/k! = (1/(k-1)!)/k$ 
22     e_sum += term
23
24 print(f"H({n}) = {harmonic:.6f}")
25 print(f" $\pi$  yaklaşımı ({n} terim) = {pi_approx:.6f} (gerçek {math.pi:.6f})")
26 print(f"hata = {abs(pi_approx - math.pi):.6f}")
27 print(f" $e$  yaklaşımı ({n} terim) = {e_sum:.10f} (gerçek {math.e:.10f})")

```

```
$ python w10-09-seriler.py
```

```

Terim sayısı n: 1000
H(1000) = 7.485471
 $\pi$  yaklaşımı (1000 terim) = 3.140593 (gerçek 3.141593)
hata = 0.001000
 $e$  yaklaşımı (1000 terim) = 2.7182818285 (gerçek 2.7182818285)

```

Kod 10.9'de küçük ama önemli bir numara var: $1/k!$ terimini her turda baştan hesaplamak yerine bir önceki terimi k 'ya bölüyoruz ($1/k! = (1/(k-1)!)/k$). Aynı fikir işaret için de geçerli: $(-1)^k$ yerine $\text{sign} = -\text{sign}$ yeter.

Örnek 10.3 (π 'yi kaç terimle, ne kadar doğru bulabiliriz?). Kod 10.9'i $n = 10$, $n = 100$ ve $n = 1000$ terim için çalıştırın. Hatanın terim sayısı ile nasıl azaldığını bulun ve π 'yi altı ondalık basamağa kadar doğru hesaplamak için kaç terim gerektiğini tahmin edin.

Çözüm

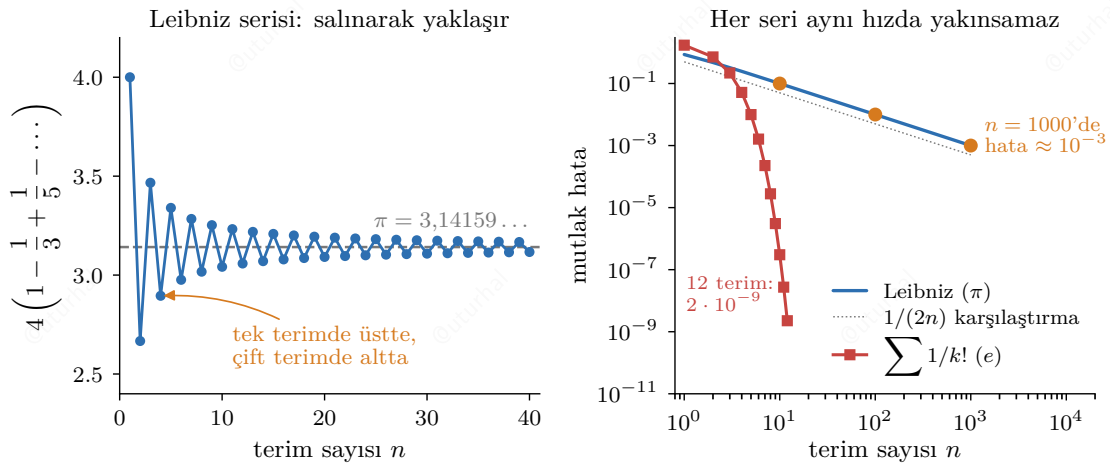
Analiz. Girdi: terim sayısı n . Çıktı: kısmi toplam ve π 'den farkı. Sorunun asıl istediği sayı değil, sayılar arasındaki *örüntü*dür.

Algoritma. Aynı programı üç farklı n ile çalıştır; $|S_n - \pi|$ değerlerini yan yana koy.

Kod ve çıktı. Program Kod 10.9'dir; üç çalıştırmanın sonucu:

n	kısmi toplam S_n	hata $ S_n - \pi $
10	3,041840	$9,98 \times 10^{-2}$
100	3,131593	$1,00 \times 10^{-2}$
1000	3,140593	$1,00 \times 10^{-3}$

Kontrol. Örüntü açıktır: terim sayısı on kat artınca hata onda birine iner, yani hata kabaca $1/(2n)$ kadardır ($n = 1000$ için $1/2000 = 0,0005$ — gözlenen $0,001$ ile aynı mertebede) ✓. Altı ondalık basamak için hatanın 10^{-6} olması, yani yaklaşık bir milyon terim gerekir. Bir milyon adımlık döngü bir saniyeden kısa sürer, ama serinin yavaşlığı ortadadır — bu seriyi *kötü* yapmaz, yalnız π hesabı için pratik olmadığını gösterir.



Şekil 10.6: Solda Leibniz serisinin kısmi toplamları: tek terim sayısında π 'nin üstünde, çift terim sayısında altında kalarak salınır. Sağda iki serinin hataları: $1/k!$ serisi 12 terimde 10^{-9} mertebesine inerken Leibniz serisi 1000 terimde ancak 10^{-3} 'e ulaşır.

Not

Harmonik toplam H_n yakınsamaz: terimler küçülse de toplam sonsuza gider — yalnız çok yavaş. $H_{1000} = 7,485471$ 'dir; $\ln n + \gamma$ yaklaşımı ($\gamma \approx 0,5772$, Euler sabiti) $7,484971$ verir. Toplamın 10'u geçmesi için 12 367 terim gerekir. “Terimler sıfıra gidiyorsa toplam sonludur” sezgisi bu yüzden yanlıştır.

Dikkat

float toplamalarında *sıra* önemlidir: çok sayıda küçük terimi büyükten küçüğe toplarsanız küçükler devasa toplamın yanında yuvarlanarak kaybolur, küçükten büyüğe toplamak daha doğrudur — 4. haftadaki $0,1 + 0,2$ sürprizinin büyük ölçekli hâli. Bir serinin son basamağına güvenmeden önce terim sayısını artırıp sonucun değişip değişmediğine bakın.

10.7 *for... else* ve arama deseni

Bir dizide “şu koşulu sağlayan eleman var mı?” diye ararken iki sonuç mümkündür: *bulundu* (döngüden erken çıkarız) ya da *yok* (dizi bitti). Python bu iki durumu ayırmak için döngüye bir *else* dalı verir.

Tanım 10.2: for ... else

Bir **for** döngüsüne bağlı **else** bloğu, döngü **break** ile **kesilmeden** doğal biçimde bittiğinde çalışır; **break** çalıştıysa *atlanır*. Adı yanıltıcıdır: burada **else**, “koşul yanlışsa” değil, “*aranan bulunamadan dizi bitti*” anlamına gelir.

Sık yapılan hata

else içteki **if** ile aynı **hizada** → **else for** ile aynı **hizada**

else'in **for** ile mi **if** ile mi eşleştiği yalnız *girintiye* bakılarak anlaşılır; ikisi çok farklı çalışır: **if**'inki her turda, **for**'unki döngü sonunda bir kez. Emin olmadığınızda bir **bayrak** (*İng. flag*) değişkeni kullanın — **found = False**, bulunca **found = True** ve **break**, döngüden sonra **if not found:**.

Örnek 10.4 (*Asal testi ve 2–100 arası asallar*). Bir sayının asal olup olmadığını bir bölen arayarak denetleyen; ardından 2–100 arasındaki bütün asalları yazan ve sayan bir program yazın. Örnek girdi: 91.

Çözüm

Analiz. n asaldır demek, 2 ile $n - 1$ arasında hiçbir sayının n 'i tam bölmemesi demektir. Bir bölen bulursak sonuç bellidir: asal değil, aramaya devam etmenin anlamı yok → **break**. Döngü hiç **break** görmeden biterse bölen yoktur → asal. Bu tam olarak **for ... else**'in tarif ettiği durumdur. $n < 2$ ayrı ele alınmalıdır: 0 ve 1 asal değildir.

Algoritma.

Sözde kod: Asal testi

```
HER bölen İÇİN 2'den n-1'e
  EĞER n mod bölen = 0 İSE YAZ "asal değil" ; DÖNGÜDEN ÇIK
DÖNGÜ KESİLMEDİYSE YAZ "asal"
```

Kod ve çıktı.

Kod 10.10 w10-10-asallar.py — for ... else ile asal testi ve asal listesi

```

1 # w10-10-asallar.py -- for ... else ile asal testi ve 2-100 arası
  ↪ asallar
2
3 n = int(input("Bir sayı: "))
4
5 if n < 2:
6     print(n, "asal değil (2'den küçük)")
7 else:
8     for divisor in range(2, n):          # bir bölen ara
9         if n % divisor == 0:
10            print(f"{n} asal değil: {divisor} ile bölünür")
11            break
12        else:                             # break OLMADAN bitti
13            print(n, "asal")
14
15 print("2-100 arası asallar:")           # aynı desen, iç döngü
  ↪ olarak
16 count = 0
17 for candidate in range(2, 101):
18     for divisor in range(2, candidate):
19         if candidate % divisor == 0:
20             break
21     else:
22         print(candidate, end=" ")
23         count += 1
24 print("\nToplam:", count, "asal")

```

```
$ python w10-10-asallar.py
```

```

Bir sayı: 91
91 asal değil: 7 ile bölünür
2-100 arası asallar:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Toplam: 25 asal

```

Kontrol. $91 = 7 \cdot 13$ olduğundan asal değildir; program ilk böleni 7 olarak bildirir — 2–6 arasında bölen olmadığı için doğrudur ✓. İkinci bölüm 2–100 arasında 25 asal bulur, en büyüğü 97'dir. Verimlilik notu: 97 için program 95 bölen dener; oysa n 'in bir böleni varsa biri mutlaka $\sqrt{n}$ 'den küçüktür, yani $\sqrt{97} \approx 9,8$ 'e kadar bakmak (8 deneme) yeterlidir (Alıştırma 10.12).

10.8 Adım sayısı ve verimlilik sezgisi

Bir programın ne kadar süreceğini ölçmenin en taşınabilir yolu saniye değil, **adım sayısı** saymaktır: döngü gövdesi kaç kez çalışıyor? Bu sayı bilgisayardan bilgisayara değişmez.

Kod 10.11 w10-11-adim-sayisi.py — Tek ve iç içe döngünün adımlarını sayaçla ölçme

```

1 # w10-11-adim-sayisi.py -- Tek ve iç içe döngünün adım sayısını sayaçla ölçme
2

```



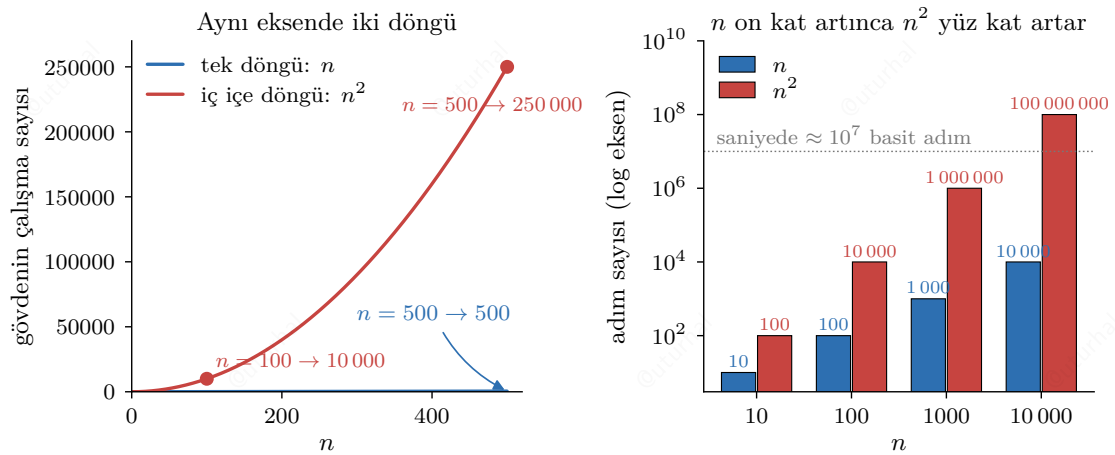
```

3 print(f'{"n":>5}{"tek döngü":>12}{"iç içe döngü":>15}{"oran":>7}')
4 for n in range(100, 501, 100):
5     single = 0
6     for i in range(n):
7         single += 1
8
9     nested = 0
10    for i in range(n):
11        for j in range(n):
12            nested += 1
13
14    print(f'{"n":>5}{"single":>12}{"nested":>15}{"nested // single:>7}')
```

```
$ python w10-11-adim-sayisi.py
```

n	tek döngü	iç içe döngü	oran
100	100	10000	100
200	200	40000	200
300	300	90000	300
400	400	160000	400
500	500	250000	500

Kod 10.11'nin çıktısı Kural 10.3'i sayıyla doğrular: $n = 100$ için tek döngü 100, iç içe döngü 10000 adımdır — oran tam olarak n 'dir; $n = 500$ için 250 000 adıma çıkar.



Şekil 10.7: Solda aynı eksende tek döngü (n) ve iç içe döngü (n^2); sağda n on kat büyüdüğünde adım sayılarının değişimi (logaritmik eksen). Yatay kesik çizgi, kabaca saniyede 10^7 basit adım varsayımdır.

Şekil 10.7'in sağ paneli asıl dersi verir: n on kat artınca tek döngü on kat, iç içe döngü yüz kat uzar. $n = 10\,000$ için iç içe döngü 10^8 adımdır; saniyede $\approx 10^7$ basit Python adımı kabul edilirse bu ≈ 10 saniye, $n = 100\,000$ için ise 10^{10} adım, yani yaklaşık bir saat eder. Bu yüzden “çalışıyor” ile “kullanılabilir” aynı şey değildir.

10.9 Uygulamalar

Not

Aynı sezgi yapay zekada doğrudan işe yarar: bir veri kümesindeki *her örnek çiftini* karşılaştıran benzerlik hesabı $n(n-1)/2$ adımdır — 1000 örnek için yarım milyon, 100 000 örnek için beş milyar. Bu yüzden büyük kümelerde “herkesi herkesle karşılaştı” yaklaşımı terk edilir.

Sayıları *görselleştirmek* de tek satırlık bir döngü işidir: `print("*" * n)` bir sayıyı çubuk hâline getirir; sola dayalı bir etiket ve ayraçla birlikte yazdırılırsa hizalı bir histogram kurulur. Aynı fikir harf frekanslarına (Alıştırma 10.8), not dağılımlarına ve — 13. haftada listelerle — gerçek veri kümelerine uygulanır.

Örnek 10.5 (Mükemmel sayılar). Bir sayının kendisi hariç bölenlerinin toplamı kendisine eşitse o sayıya **mükemmel sayı** (İng. *perfect number*) denir. 1–1000 arasındaki bütün mükemmel sayıları, bölenlerinin toplamı biçiminde yazan bir program yazın.

Çözüm

Analiz. İki döngü gerekir: dış döngü adayları, iç döngü her adayın bölenlerini gezer. Bir sayının kendisi dışındaki en büyük böleni en fazla $n/2$ 'dir; iç döngü bu yüzden `range(1, number // 2 + 1)` ile sınırlanır.

Algoritma. **HER** sayı **İÇİN** 2'den 1000'e: toplam $\leftarrow 0$; **HER** bölen **İÇİN** 1'den sayı/2'ye: bölüyorsa toplam $\leftarrow$ toplam + bölen; **EĞER** toplam = sayı **İSE YAZ** sayı ve bölenleri.

Kod ve çıktı.

Kod 10.12 w10-12-mukemmel.py — İç içe döngüyle 1–1000 arası mükemmel sayılar

```
1 # w10-12-mukemmel.py -- 1-1000 arası mükemmel sayılar (bölen toplamı =
   ↪ sayı)
2 LIMIT = 1000
3 for number in range(2, LIMIT + 1):
4     divisor_sum = 0
5     divisors_text = ""
6     for divisor in range(1, number // 2 + 1): # en büyük bölen: n/2
7         if number % divisor == 0:
8             divisor_sum += divisor
9             divisors_text += f"{divisor} + "
10    if divisor_sum == number:
11        print(f"{number} = {divisors_text[:-3]}") # sondaki ' + '
   ↪ atılır
```

\$ python w10-12-mukemmel.py

```
6 = 1 + 2 + 3
28 = 1 + 2 + 4 + 7 + 14
496 = 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248
```

Kontrol. Elle: 6'nın bölenleri 1, 2, 3 ve toplamları 6 ✓; 28'in bölenleri 1, 2, 4, 7, 14 ve toplamları 28 ✓. Üçüncü mükemmel sayı 496'dır ve program üçünü de bulur; dördüncüsü 8128'dir, sınırın dışındadır — LIMIT değerini 10 000 yapıp deneyin. Karşı örnek: 12'nin bölen toplamı 16 > 12, yani 12 mükemmel değil “bolluk” sayısıdır. Tarama kabaca $\sum_{n=2}^{1000} n/2 \approx 250\,000$ adımdır; sınırı 100 kat büyütürseniz adım sayısı 10 000 kat artar.



GitHub

LAB Laboratuvar Çalışması 10: Döngüler iş başında

Üç zorunlu, üç seçmeli görev. Her görevde önce kâğıda “kaç tur dönecek, her turda ne olacak?” yazın; kodu ondan sonra açın.

Görev 1. Isınma (kabukta). Şu çağrılar kaç değer ürettiğini önce *tahmin edin*, sonra `len(range(...))` ile doğrulayın: `range(7)`, `range(1, 8)`, `range(0, 30, 4)`, `range(20, 2, -3)`, `range(4, 4)`, `range(5, 1)`.

Görev 2. Desen çizimi. `desen.py`: kullanıcıdan n alın ve dört deseni sırayla çizin — sol üçgen, sağ üçgen, piramit ve *kum saati* (ters piramit + piramit). $n = 4$ için kum saati 7 satır olmalıdır. Her deseni yazmadan önce “ i . satırda kaç boşluk, kaç yıldız?” tablosunu doldurun.

Görev 3. Sezar programı. `sezar.py`: mesajı, kaymayı ve işlemi (“s” şifrele / “c” çöz) alın; 7. haftanın `if-elif` yapısıyla işlemi seçip sonucu yazın. Kendi adınızı 5 kaydırıp yan sıradaki arkadaşınıza yalnız şifreli metni verin; kaymayı deneyerek bulmasını isteyin.

Görev 4. π yaklaşımı. `pi.py`: Leibniz serisiyle $n = 10, 100, 1000, 10\,000$ için dört satırlık bir tablo yazdırın: terim sayısı, yaklaşım (.6f), hata (.2e). Hatanın her satırda onda birine indiğini gözleyin.

Görev 5. Çarpım tablosu. `carpim.py`: n alıp $n \times n$ tabloyu başlık satırı ve ayraç çizgisiyle hizalı yazın; sütun genişliğini en büyük çarpıma göre seçin ($n = 12$ için 4 karakter yeter). Kontrol: bütün sayıların toplamı $(1 + 2 + \dots + n)^2$, $n = 9$ için 2025 olmalıdır.

Görev 6. Asal bulucu. `asal.py`: iki sayı alın ve aralarındaki asalları her satırda 10 tane olacak biçimde yazdırın; sonunda sayısını bildirin (2–100 arası 25). Ek: iç döngünün sınırını $n - 1$ yerine `math.isqrt(n) + 1` yapıp sonucun değişmediğini, adım sayısının düştüğünü bir sayaçla gösterin.

Teslim. Görev 1–3’ü ders bitmeden çalıştırıp gösterin; dosyaları saklayın (11. haftada `sezar.py` ve `asal.py` fonksiyonlara bölünecek).

Özet

Yapı	Yazım	Not
Sayılar üzerinde	<code>for i in range(1, n + 1):</code>	<code>dur</code> dahil değil
Sayı dizisi	<code>range(başla, dur, adım)</code>	$\lceil (dur - başla)/adım \rceil$ eleman
Geri sayım	<code>range(10, 0, -1)</code>	negatif adım: <code>başla > dur</code>
Metin üzerinde	<code>for ch in text:</code>	<code>ch</code> karakterin <i>kendisi</i>
İndeks ve eleman	<code>for i, ch in enumerate(s):</code>	<code>start=1</code> ile 1’den başlar
İki diziyi eşleme	<code>for a, b in zip(s1, s2):</code>	kısa olan bitince durur
Tersten gezme	<code>for ch in reversed(s):</code>	<code>range(n-1, -1, -1)</code> ile aynı
İç içe döngü	<code>dış for + iç for</code>	toplam adım $n \cdot m$; n^2
Tüm ikililer	<code>for j in range(i + 1, n):</code>	$n(n - 1)/2$ ikili
Biriktirme	<code>total = 0 ... total += x</code>	başlatma döngüden <i>önce</i>
Arama deseni	<code>for ... break + else:</code>	<code>else: break</code> olmadıysa
Desen satırı	<code>print(" " * a + "*" * b)</code>	<code>end=""</code> ile parça parça



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w10>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırmalar

Alıştırma 10.1. ●●● Aşağıdaki çağrılarının her biri kaç değer üretir ve bu değerler nelerdir? Çalıştırmadan yazın, sonra `len(range(...))` ile doğrulayın. (a) `range(1, 10, 2)` (b) `range(10, 1, -2)` (c) `range(0, 7, 3)` (d) `range(5, 5)` (e) `range(3, 0)`
 Cevap: (a) 5 değer: 1,3,5,7,9 (b) 5 değer: 10,8,6,4,2 (c) 3 değer: 0,3,6 (d) boş (e) boş

Alıştırma 10.2. ●●● Aşağıdaki `while` döngüsünü, aynı değerleri aynı sırayla üreten tek satırlık bir `for` başlığına çevirin: `i = 2` ile başlayıp `while i < 20:` koşuluyla dönen, gövdesinde `print(i)` ve `i += 3` bulunan döngü. Kaç tur döner, son değer nedir? Hangi döngüyü tercih edeceğinizi bir cümleyle gerekçelendirin.

Cevap: `for i in range(2, 20, 3):` — 6 tur; 2,5,8,11,14,17; son değer 17. Tekrar sayısı belli olduğu için `for`

Alıştırma 10.3. ●●● Bir tam sayı alan ve rakamlarının toplamını döngüyle bulan bir program yazın: sayıyı `str()` ile metne çevirip karakterleri üzerinde dönün, her karakteri `int()` ile sayıya çevirerek biriktirin. Girdi 2026: toplam 10, basamak sayısı 4.

Cevap: $2 + 0 + 2 + 6 = 10$; basamak sayısı `len(str(abs(n))) = 4`

Alıştırma 10.4. ●●● 1 ile n arasındaki çift ve tek sayıların toplamını ayrı iki döngüyle, `range`'in adım parametresini kullanarak hesaplayan bir program yazın. İki toplamın farkını ve toplamlarının $n(n+1)/2$ 'ye eşit olduğunu da yazdırın. Girdi 100.

Cevap: çift toplam 2550, tek toplam 2500, fark 50, toplam 5050

Alıştırma 10.5. ●●● Fibonacci dizisinin ilk n terimini numaralandırarak yazdıran bir program yazın. İlk iki terim 1 ve 1'dir; her terim kendinden önceki iki terimin toplamıdır. 4. haftanın çoklu atama yazımını (`a, b = b, a + b`) kullanın. Girdi 10.

Cevap: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55; onuncu terim 55

Alıştırma 10.6. ●●● 32–126 arasındaki yazdırılabilir ASCII karakterlerini, her satırda dört sütun olacak biçimde, kodu ve karakteri yan yana yazan bir program yazın (satır sonu için `sayac` ile `%` operatörü). Sonunda toplam karakter ve satır sayısını bildirin.

Cevap: 95 karakter, dört sütunda 24 satır ($\lceil 95/4 \rceil$); $65 \rightarrow A$, $97 \rightarrow a$, 32 $\rightarrow$ boşluk

Alıştırma 10.7. ●●● Kayma miktarı bilinmeyen bir Sezar mesajını kıran bir program yazın: olası bütün kaymaları tek tek deneyip 25 adayı listeleyin, anlamlı olanı gözle seçin. Sinama metni: "PHUKDED GXQBD".

Cevap: doğru kayma 3; çözüm MERHABA DUNYA; denenecek kayma sayısı 25

Alıştırma 10.8. ●● Bir metindeki harflerin sıklığını yıldızlarla gösteren bir histogram programı yazın: her farklı harf için bir satır, yanında sayısı kadar yıldız ve parantez içinde sayı. Boşluklar sayılmasın, her harf yalnız *bir kez* listelensin (daha önce yazdığınız `continue` ile atlayın). Girdi: "programlama".

Cevap: 11 karakter, 7 farklı harf; a 3, r 2, m 2, kalanlar birer kez

Alıştırma 10.9. ●●● Sayı tahmin oyununu `while` ile değil, *en fazla beş deneme* veren bir `for` döngüsüyle yazın: bilgisayar 1–100 arasında bir sayı tutsun (`random.randint`), her tahminden sonra “daha büyük”/“daha küçük” ipucu versin, doğru tahminde `break` ile çıksın, `else` dalında “denemeler bitti” yazsın. Kaç deneme her zaman yeterdi?

Cevap: 5 deneme yetmeyebilir: her adımda aralığı ikiye bölerek oynanırsa $\lceil \log_2 100 \rceil = 7$ deneme her zaman yeter

Alıştırma 10.10. ●●● Yarı yüksekliği n olan bir *elmas* deseni çizen program yazın: üst yarı büyüyen bir piramit, alt yarı küçülen bir ters piramittir; orta satır bir kez yazılır. Alt yarı için geriye sayan bir `range` kullanın. $n = 4$ için toplam satır sayısını da yazdırın.

Cevap: $2n - 1 = 7$ satır; en geniş satır $2n - 1 = 7$ yıldız; toplam yıldız sayısı 25

Alıştırma 10.11. ●●● e sayısını $\sum 1/k!$ serisiyle hesaplayan, tek bir sonuç yerine *terim sayısı–yaklaşım–hata* tablosu yazdıran bir program yazın (1’den 12 terime kadar; hata sütunu .2e biçiminde). Hata kaç terimde 10^{-9} ’un altına iner?

Cevap: beş terimde 2,708333 (hata $9,95 \times 10^{-3}$); 12 terimde hata $2,26 \times 10^{-9}$; 13. terimde 10^{-9} ’un altına iner ($1,73 \times 10^{-10}$)

Alıştırma 10.12. ●●● Aşağıdaki üç döngü kurgusunda en içteki satır kaç kez çalışır? Önce n cinsinden formül, sonra $n = 100$ için sayı verin. (a) `for i in range(n):` içinde `for j in range(n):` (b) `for i in range(n):` içinde `for j in range(i):` (c) üç kat iç içe, her biri `range(n)`. Ayrıca Örnek 10.4’deki asal testinde iç döngüyü $\sqrt{n}$ ile sınırlamak 97 için kaç denemeyi kaçta indirir?

Cevap: (a) $n^2 = 10\,000$ (b) $n(n-1)/2 = 4950$ (c) $n^3 = 1\,000\,000$; asal testinde 95 deneme yerine 8 deneme (2–9 arası)

Hafta 11: Fonksiyonlar I: Kendi Komutlarınızı Yazmak

Öğrenme çıktıları

Bu haftanın sonunda:

1. tekrar eden kodu tanıyıp tek bir fonksiyona dönüştürebilecek, bunun bakım maliyetini neden düşürdüğünü açıklayabileceksiniz;
2. `def` ile konumsal parametrelili fonksiyonlar tanımlayıp çağırabilecek, bir çağrının akışını ve çağrı yığını adımları adım adım izleyebileceksiniz;
3. parametre ile argümanı ayırt edebilecek, sıra ve sayı uyumsuzluğunun ne zaman `TypeError` verdiğini, ne zaman sessizce yanlış sonuç ürettiğini söyleyebileceksiniz;
4. `return` ile değer döndüren fonksiyonlarla ekrana yazan (`void`) fonksiyonları ayırtabilecek, `print` ile `return` farkını gösterebileceksiniz;
5. yerel ve global adları, gölgelemeyi ve `global` bildiriminin neden kaçınıldığını açıklayabilecek, fonksiyonunuza docstring yazıp `help()` ile okutabileceksiniz;
6. büyük bir problemi yukarıdan aşağıya fonksiyonlara bölüp `main()` düzeninde bir program yazabilecek ve fonksiyonunuzu bilinen girdi-beklenen çıktı tablosuyla elle smayabileceksiniz.

Anahtar kavramlar

fonksiyon (İng. *function*) **çağrı** (İng. *call*) **parametre** (İng. *parameter*) **argüman** (İng. *argument*) **konumsal argüman** (İng. *positional argument*) **dönüş değeri** (İng. *return value*) **void fonksiyon** (İng. *void function*) **çağrı yığını** (İng. *call stack*) **çerçeve** (İng. *frame*) **yerel değişken** (İng. *local variable*) **kapsam** (İng. *scope*) **gölgeleme** (İng. *shadowing*) **belgeleme dizesi** (İng. *docstring*) **yukarıdan aşağıya tasarım** (İng. *top-down design*) **yeniden düzenleme** (İng. *refactoring*)

On haftadır Python'un hazır fonksiyonlarını kullanıyorsunuz: `len("Trabzon")` yedi sayısını nasıl sayıyor, `math.sqrt(16)` karekökü nasıl buluyor, `random.randint(1, 6)` zarı nasıl atıyor bilmiyorsunuz. Her seferinde yaptığınız şey aynıydı: bir ada değer(ler) vermek ve karşılığında bir sonuç almak. Bu hafta o kutuları *kendiniz* yapmayı öğreniyorsunuz.

Bunun iki sebebi var. Birincisi çok pratik: programlarınız büyüdü. 10. haftanın Sezar şifresi programında aynı yirmi satırı şifrelemek ve çözmek için iki kez yazdınız. Kopyalanan her satır, ileride düzeltilecek her hatanın kopya sayısı kadar tekrarlanması demektir. İkincisi daha derin: fonksiyon *düşünme birimini* büyütür. “Şu sayının asal olup olmadığını anla” cümlesi, o işi yapan altı satırlık döngüden daha kolay akılda tutulur. Fonksiyon, bir fikre ad vermenin yoludur; ad verdiğiniz fikirleri birleştirerek çok daha büyük problemler çözebilirsiniz. Yapay zeka kütüphanelerinin tamamı — `model.fit(X, y)` — bu fikrin üstüne kuruludur.

11.1 Neden fonksiyon?

Müfredat bu haftanın amacını üç sözcükle söylüyor: *tekrar kullanılabilir kod tanımlamak, kodu düzenlemek ve kodu basitleştirmek*. Üçünü de tek bir örnek üzerinde görelim.

Bir sağlık kioskunun programını yazıyorsunuz. Kullanıcıdan kütle, boy ve yaş alacaksınız ve üçünün de pozitif olduğundan emin olmak istiyorsunuz. 8. haftada öğrendiğiniz girdi doğrulama döngüsü tam olarak bu işi yapar — ama üç kez yazılması gerekir (Şekil 11.1, solda). Bu kod *çalışır*; sorun çalışıp çalışmaması değil. Ertesi hafta müdür “uyarı mesajını değiştirilim” dediğinde üç kopyadan ikisini düzeltir, birini atlarsınız. Program artık tutarsızdır ve bunu kimse fark etmez.

ÖNCE — aynı blok üç kez

```
w = float(input("Kütle: "))
while w <= 0:
    print("Pozitif girin.")
    w = float(input("Kütle: "))
```

```
h = float(input("Boy: "))
while h <= 0:
    print("Pozitif girin.")
    h = float(input("Boy: "))
```

```
a = float(input("Yaş: "))
while a <= 0:
    print("Pozitif girin.")
    a = float(input("Yaş: "))
```

SONRA — tek tanım, üç çağrı

```
def read_positive(prompt):
    v = float(input(prompt))
    while v <= 0:
        print("Pozitif girin.")
        v = float(input(prompt))
    return v
```

```
w = read_positive("Kütle: ")
h = read_positive("Boy: ")
a = read_positive("Yaş: ")
```

✓ Mesajı düzeltmek: **tek** yerde değişiklik — ve üç yer birden düzelir. Ana program artık *ne* yaptığını okutuyor, *nasıl* yaptığını değil. Yeni bir girdi gerekirse tek satır eklenir.

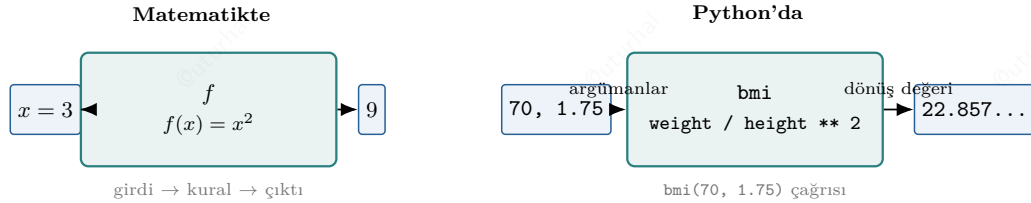
✗ Mesajı düzeltmek: **üç** yerde değişiklik. Biri unutulursa program tutarsız davranır — ve bunu kimse fark etmez.

Aynı kodu ikinci kez yazmak üzereyken durun: orada bir fonksiyon vardır. Kopyala-yapıştır kod ilk gün zaman kazandırır, ertesi hafta faturayı keser: her düzeltme, her iyileştirme, her yeni özellik kopya sayısı kadar tekrarlanır ve biri mutlaka atlanır. Davranışı değiştirmeden yapıyı düzelterken bu işleme *yeniden düzenleme* denir. Doğru yapıldığının kanıtı basittir: çıktı, düzenlemeden önceki çıktının aynısıdır.

Şekil 11.1: Tekrar eden bir blok, tek bir fonksiyona dönüşüyor. Sağdaki sürümde uyarı mesajı programda *bir* yerde geçer; onu düzeltmek üç girdiyi birden düzeltir.

Tanım 11.1: Fonksiyon

Fonksiyon (İng. *function*), bir ada bağlanmış, gerektiğinde çağrılabilen kod bloğudur. Dışarıdan aldığı değerlere **argüman**, sonuçta geri verdiği değere **dönüş değeri** (İng. *return value*) denir. Fonksiyonu *kullanan* kişinin gövdesini bilmesi gerekmez; ne aldığı ve ne verdiğini bilmesi yeter. Bu ayrıma **soyutlama** (İng. *abstraction*) denir ve programlamanın en güçlü fikridir (Şekil 11.2).



Bir fonksiyon, adı olan bir kara kutudur: içine değerler verirsiniz (*argümanlar*), kutu gövdesindeki adımları çalıştırır ve size tek bir sonuç verir (*dönüş değeri*). Kutuyu *kullanan* kişinin içeri bilmesi gerekmez — yalnız ne aldığı ve ne verdiğini bilir. `len("abc")`, `math.sqrt(16)` ve `round(2.7)` çağrılarını on haftadır böyle kullanıyorsunuz; bu hafta kutuları kendiniz yapmayı öğreniyorsunuz.

Şekil 11.2: Matematikteki fonksiyon ile Python'daki fonksiyon aynı fikirdir: girdi → kural → çıktı.

Üç kazancı adlandıralım. **Tekrar kullanım:** bir kez yazılır, istenildiği kadar çağrılır — farklı değerlerle, hatta (12. haftada göreceğiniz gibi) başka programlarda. **Düzen:** ana program *ne* yapıldığını okutan kısa bir listeye dönüşür; *nasıl* yapıldığı fonksiyonların içindedir. **Sadeleştirme:** büyük bir problem, tek başına sınanabilen küçük parçalara bölünür; beş satırlık bir fonksiyonun doğruluğunu göstermek, elli satırlık bir gövdeninkini göstermekten çok daha kolaydır.

Bu haftaya kadar hiç fonksiyon yazmadınız ama sürekli fonksiyon *çağırdınız*: `print`, `input`, `len`, `int`, `round`, `range`, `ord` ve `chr` yerleşiktir; `math.sqrt` ile `random.randint` mo-

düllerden gelir. Bu hafta yazacaklarınızla onlar arasında *hiçbir* yapısal fark yoktur.

11.2 Tanım ve çağrı: `def`

Bir fonksiyon iki ayrı olayla yaşar: bir kez *tanımlanır*, sonra istenildiği kadar *çağrılır*.

Kural 11.1 (Fonksiyon tanımının yazımı). `def` ad(parametreler): satırı bir fonksiyon tanımını başlatır. Kurallar: (i) `def` anahtar sözcüğünü fonksiyonun **adı** izler; ad, değişken adlarıyla aynı kurallara uyar ve gelenek olarak küçük harfle, alt çizgiyle yazılır (`letter_grade`); (ii) adı parantez izler, parantez içi boş olabilir ama parantezler *zorunludur*; (iii) satır iki nokta üst üste ile biter; (iv) **gövde** (*Ing. body*), bir seviye içeriden (4 boşluk) yazılan en az bir satırdır, girinti biter bitmez fonksiyon da biter.

Tanım, gövdeyi *çalıştırmaz*; yalnız adı ve gövdeyi Python'un belleğine kaydeder. Gövde ancak `ad(...)` biçiminde bir **çağrı** (*Ing. call*) yapıldığında çalışır.

Kod 11.1 w11-01-ilk-fonksiyon.py — İlk fonksiyon: tanım, çağrı ve çalışma sırası

```
1 # w11-01-ilk-fonksiyon.py -- İlk fonksiyon: tanım, çağrı ve çalışma sırası
2
3 def print_header():
4     """Üç satırlık bir başlık yazar."""
5     print("=" * 30)
6     print("    YPZ105 - Fonksiyonlar")
7     print("=" * 30)
8
9
10 # Ana program buradan başlar; yukarıdaki tanım henüz hiçbir şey yazdırmadı.
11 print("1) Ana program başladı.")
12 print_header() # çağrı: akış gövdeye atlar, sonra döner
13 print("2) İlk çağrı bitti, ana programa dönüldü.")
14 print_header() # aynı gövde ikinci kez çalışır
15 print("3) Program bitti.")
```

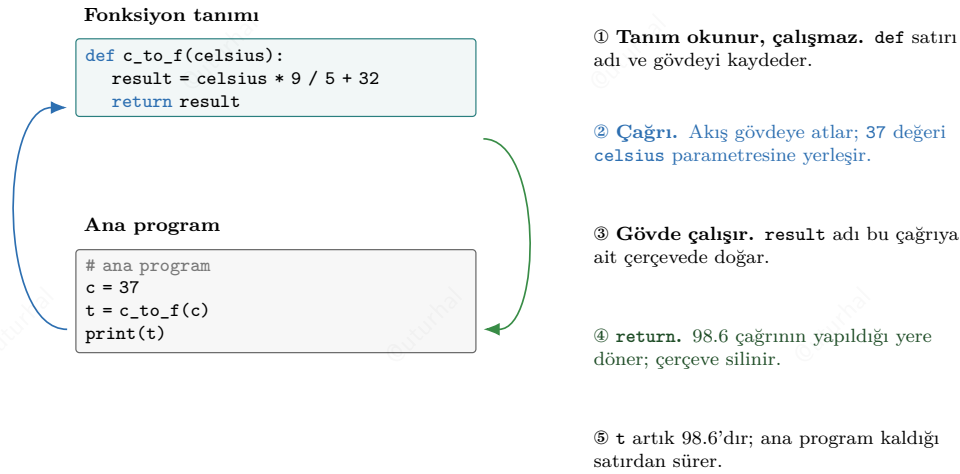
\$ python w11-01-ilk-fonksiyon.py

```
1) Ana program başladı.
=====
    YPZ105 - Fonksiyonlar
=====
2) İlk çağrı bitti, ana programa dönüldü.
=====
    YPZ105 - Fonksiyonlar
=====
3) Program bitti.
```

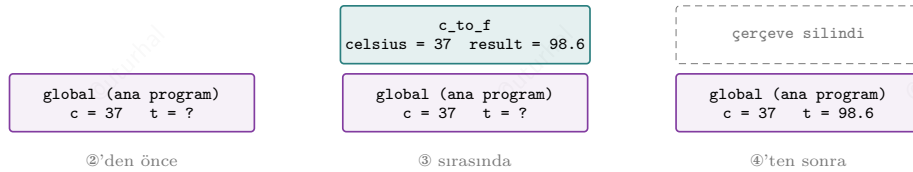
Kod 11.1'in çıktısındaki sıraya dikkat edin: ekrana ilk yazılan şey 1) *Ana program başladı.* satırıdır, oysa `def` satırı dosyada çok daha önce geçiyor. Sebebi **Kural 11.1**'in son cümlesidir: tanım yalnız *kaydeder*. Başlık iki kez görüldü, çünkü fonksiyon iki kez *çağrıldı* — fonksiyonun bütün mesleği budur. **Şekil 11.3** bir çağrının akışını beş adımda ve üç anlık görüntüde gösterir; oradaki **çerçeve** (*Ing. frame*) kavramı 11.6'daki kapsam tartışmasının da temelidir.



GitHub



Çağrı yığınının üç anı (*Ing. call stack*):



Çağrı, akışı fonksiyonun gövdesine taşır; **return** geri getirir. Her çağrıda Python bir *çerçeve* — o çağrının yerel adlarını tutan bir kutu — açar ve *çağrı yığınının* üstüne koyar. En üstteki çerçeve çalışır, altındakiler bekler; **return** çalışınca çerçeve silinir ve dönüş değeri çağrının yapıldığı yere yerleşir. Bir hata mesajındaki **Traceback** listesi tam olarak bu yığının fotoğrafıdır.

Şekil 11.3: Bir çağrının akışı ve çağrı yığını. Çağrı akışı gövdeye taşır, **return** geri getirir; her çağrı yığında kendi çerçevesini açar.



Sık yapılan hata

`print_header` → `print_header()`

Parantezsiz yazılan ad, fonksiyonu *çağırmaz*; yalnız fonksiyon nesnesinin kendisini ifade eder. Program hata vermez, hiçbir şey de yapmaz — bulunması en sinir bozucu hata türüdür. Kabukta `<function print_header at 0x7f...>` görürsünüz: “bu bir fonksiyondur” demektir, “çalıştırdım” değil. Çağırarak için parantez şarttır, boş olsa bile.

İkinci tuzak sıradadır: bir fonksiyon *çağrıldığı anda* tanımlanmış olmalıdır. Tanımı çağrıdan sonraya yazarsanız **NameError: name 'print_header' is not defined** alırsınız. Bu yüzden bütün **def**'ler dosyanın başına, ana program en sona yazılır — 11.9'daki **main()** düzeni bunu kurallaştırır.



İpucu

Fonksiyon adları *eylem* bildirmelidir: `calculate_area`, `draw_box`, `read_score`. Bir soruya evet/hayır cevabı veren fonksiyonlar `is_` ya da `has_` ile başlar: `is_prime`, `is_leap`, `is_palindrome`. Bu gelenek çağrıyı okunur kılar: `if is_prime(n):` satırı neredeyse İngilizcedir.

11.3 Parametreler ve argümanlar

`print_header()` her çağrıda aynı şeyi yapar; oysa fonksiyonların asıl gücü, her çağrıda *farklı* değerlerle çalışabilmeleridir. Bunu parantez içine yazılan adlar sağlar.

Tanım 11.2: Parametre ve argüman

Parametre (*İng. parameter*), `def` satırının parantezleri içinde yazılan addır; o fonksiyonun gövdesinde kullanılabilen, başlangıç değeri çağrıdan gelen bir yerel değişkendir. **Argüman** (*İng. argument*) ise çağrıda parantez içine yazılan değerdir. Kısacası: *parametre tanımda, argüman çağrıdadır*. Argümanlar parametrelere soldan sağa, yazıldıkları sıraya göre yerleşir; bu yüzden onlara **konumsal argüman** (*İng. positional argument*) denir (Şekil 11.4).

Tanım — parametreler

```
def bmi( weight , height ):
```

Çağrı — argümanlar

```
bmi( 70 , 1.75 ) ✓ 22.857...
```

Sayı tutmazsa: `TypeError`

```
bmi(70)
TypeError: bmi() missing 1 required
positional argument: 'height'
```

Sıra karışırsa: hata yok, sonuç yanlış

```
def bmi( weight , height ):
```

```
bmi( 1.75 , 70 ) ✗ 0.000357...
```

Python sırayı denetlemez: 1,75 kg ağırlığında, 70 m boyunda bir kişinin VKİ'sini dürüstçe hesaplar. Böyle hataları yalnız *kontrol* adımı yakalar.

Parametre tanımda, argüman çağrıdadır. `def` satırındaki adlar (`weight`, `height`) birer boş kutudur; çağrıdaki değerler (70, 1.75) bu kutulara *soldan sağa, sırayla* yerleşir — bu yüzden onlara *konumsal argüman* denir. Sayıları uyuşmazsa Python hemen `TypeError` verir; sıraları karışırsa hiçbir uyarı almazsınız, yalnızca yanlış bir sayı alırsınız.

Şekil 11.4: Konumsal argümanlar parametrelere sırayla yerleşir. Sayı uyumsuzluğu `TypeError` verir; sıra karışıklığı hiç uyarı vermez.

Örnek 11.1 (Çizgi ve üçgen çizen iki fonksiyon). Bir karakteri istenen sayıda yan yana yazan `print_line` ve `n` satırlık bir yıldız üçgeni çizen `draw_triangle` fonksiyonlarını yazın; her ikisini de farklı argümanlarla çağırın.

Çözüm

Analiz. İkisi de ekrana bir görüntü çizer; hesap yapıp değer üretmezler. Birincinin girdisi bir karakter ve bir sayı, ikincisinin girdisi yalnız satır sayısıdır. Çizim işi 5. haftanın `"*" * n` yinelemesi ve 10. haftanın `for` döngüsüyle yapılır.

Fonksiyon arayüzü tasarımı. `print_line(ch, n)`: parametreler sırayla karakter ve tekrar sayısı; dönüş değeri *yok* — işi ekrana yazmaktır. `draw_triangle(n)`: tek parametre, satır sayısı; dönüş değeri yine *yok*. Böyle fonksiyonlara 11.5'te *void* diyeceğiz.

Kod ve çıktı.

Kod 11.2 w11-02-parametre.py — Parametrelili fonksiyonlar: print_line ve draw_triangle

```

1 # w11-02-parametre.py -- Parametrelili fonksiyonlar: print_line,
  ↪ draw_triangle
2
3 def print_line(ch, n):
4     """ch karakterini n kez yan yana yazar."""
5     print(ch * n)
6
7
8 def draw_triangle(n):
9     """n satırlık yıldız üçgeni çizer."""
10    for i in range(1, n + 1):
11        print("*" * i)
12
13
14 print_line("-", 20)           # ch = "-", n = 20
15 print_line("*", 5)           # aynı gövde, başka argümanlar
16 print()
17 draw_triangle(4)             # n = 4

```

```
$ python w11-02-parametre.py
```

```

-----
*****

*
**
***
****

```

Kontrol. Kod 11.2’de `print_line("-", 20)` çağrısı `ch` parametresine "-", `n`’ye 20 yerleştirir; ilk satırdaki tireler tam 20 ✓, ikincideki yıldızlar 5 ✓. `draw_triangle(4)` dört satır çizmeli, *i*’inci satırda *i* yıldız: 1,2,3,4 ✓. Uç durum: `draw_triangle(0)` hiçbir şey çizmez, çünkü `range(1, 1)` boştur. `ch` ve `n` adları yalnız gövdenin içinde vardır.

❌ Sık yapılan hata

`print_line("-")` → `print_line("-", 20)`

TypeError: print_line() missing 1 required positional argument: 'n'. Koonsal argümanların *sayısı* parametrelerin sayısına eşit olmalıdır: ne eksik, ne fazla. Fazla vererseniz mesaj da simetrik: `takes 2 positional arguments but 3 were given`. Bir parametreye varsayılan değer vererek onu isteğe bağlı kılmayı 12. haftada öğreneceksiniz; bu hafta bütün argümanlar zorunludur.

⚠ Dikkat

Sayı uyumsuzluğunu Python yakalar, **sıra karışıklığını yakalayamaz**. `bmi` fonksiyonu için `bmi(1.75, 70)` yazarsanız hiçbir hata almazsınız: Python, 1,75 kg ağırlığında ve 70 m boyunda bir kişinin vücut kitle indeksini dürüstçe hesaplar ve 0,000357 gibi bir sayı



GitHub

verir. Böyle hataları yalnız *Kontrol* adımı — sonucun büyüklük mertebesine bakmak — yakalar. Parametre adlarını anlamlı seçmek (*weight*, *height*) ve çağırıcı yazarken tanımı bir kez okumak en iyi korunmadır.

11.4 *return* ile değer döndürmek

`print_line` ekrana bir şey yazar ve biter. Çoğu fonksiyonsa hesap yapar ve sonucu *çağırana* teslim eder. Bunu *return* deyimi sağlar.

Tanım 11.3: *return* deyimi

return ifade deyimi iki iş yapar: (i) *ifadeyi* hesaplar ve elde edilen değeri çağırıcının yapıldığı yere gönderir; (ii) fonksiyonu *o anda* sonlandırır — gövdenin geri kalanı ne olursa olsun çalışmaz. Bir fonksiyonda birden çok *return* bulunabilir; ilk çalışan hangisiyse fonksiyon onunla biter. *return*'den sonra gelen aynı girintideki satırlara **ölü kod** (*Ing. dead code*) denir.

Dönen değer, tıpkı bir sabit ya da değişken gibi, bir *ifadenin* parçasıdır: bir değişkene atanabilir, bir hesaba girebilir, başka bir fonksiyona argüman olabilir, bir koşulda karşılaştırılabilir ya da doğrudan yazdırılabilir.

Örnek 11.2 (*Sıcaklık dönüşümü ve vücut kitle indeksi*). Celsius sıcaklığını Fahrenheit'a çeviren bir fonksiyon (`celsius_to_fahrenheit`) ile vücut kitle indeksini hesaplayan bir fonksiyon (`bmi`) yazın; dönüş değerlerini dört ayrı biçimde kullanan bir ana program ekleyin.

Çözüm

Analiz. İki bağımsız hesap: $F = C \cdot 9/5 + 32$ ve $VKI = \text{kütle}/\text{boy}^2$. İkisi de ekrana bir şey *yazmaz*; yazma işi çağırıcının işidir.

Fonksiyon arayüzü tasarımı.

Ad	Parametreler (sırayla)	Dönüş değeri
<code>celsius_to_fahrenheit</code>	<code>celsius</code> — sıcaklık, °C	float, °F cinsinden sıcaklık
<code>bmi</code>	<code>weight</code> (kg), <code>height</code> (m)	float, kg/m ²

Her fonksiyon tek bir iş yapıyor ve tek bir sayı döndürüyor; arayüz tasarımı gövdeyi yazmadan *önce* yapılır.

Kod ve çıktı.

Kod 11.3 w11-03-donus-degeri.py — Dönüş değerinin dört kullanımı

```

1 # w11-03-donus-degeri.py -- return ile değer döndürme (sıcaklık, VKİ)
2
3 def celsius_to_fahrenheit(celsius):
4     """Celsius sıcaklığı Fahrenheit'a çevirir."""
5     return celsius * 9 / 5 + 32
6
7
8 def bmi(weight, height):
9     """Vücut kitle indeksi: kütle (kg) / boy (m) karesi."""
10    return weight / height ** 2
11
12
13 # 1) Dönüş değeri bir değişkene atanır
14 f = celsius_to_fahrenheit(37)
15 print("37 °C =", f, "°F")
16
17 # 2) Doğrudan bir ifadenin içinde kullanılır
18 print("Kaynama - donma farkı:",
19       celsius_to_fahrenheit(100) - celsius_to_fahrenheit(0), "°F")
20
21 # 3) f-string içinde biçimlenir
22 print(f"VKİ: {bmi(70, 1.75):.2f}")
23
24 # 4) Bir karşılaştırmanın parçası olur
25 print("Normal aralıkta mı?", bmi(70, 1.75) < 25)

```

```
$ python w11-03-donus-degeri.py
```

```

37 °C = 98.6 °F
Kaynama - donma farkı: 180.0 °F
VKİ: 22.86
Normal aralıkta mı? True

```

Kontrol. Kod 11.3'un çıktısını elle doğrulayalım: $37 \cdot 9/5 + 32 = 66,6 + 32 = 98,6$ ✓. İkinci satır bir uç durum sınavıdır: kaynama ile donma noktası arasındaki fark Celsius'ta 100, Fahrenheit'ta 180 derecedir — çıktı da 180,0 verdi ✓. VKİ için $70/1,75^2 = 70/3,0625 = 22,857... \approx 22,86$ ✓; sağlıklı aralık 18,5–25 olduğuna göre büyüklük mertebesi doğru ve son satır True ✓.

Örnek 11.3 (Harf notu fonksiyonu ve erken dönüş). 0–100 arasındaki bir puanı A, B, C, D, F harf notuna çeviren `letter_grade(score)` fonksiyonunu yazın. Fonksiyonu 100'den 60'a onar onar azalan puanlarla, ayrıca 89,9 ve 59,9 sınır değerleriyle deneyin.

Çözüm

Analiz. Girdi: bir sayı. Çıktı: tek harfli bir `str`. 6. haftada bu işi `if-elif-else` zinciriyle yapmıştık; şimdi aynı zinciri bir fonksiyonun içine alıp her dalda `return` ediyoruz. Böylece `else` dallarına bile gerek kalmaz: bir `return` çalıştıysa zaten aşağıya inilmez.

Fonksiyon arayüzü tasarımı. Ad `letter_grade`; tek parametre `score`; dönüş değeri her durumda bir `str`. Son `return "F"` satırı bunu garanti eder: hiçbir koşul tutmasa bile bir değer döner. Bir fonksiyonun *tek bir türde* değer döndürmesi genel bir kuraldır: `is_prime` her zaman `bool`, `bmi` her zaman `float` verir; bazen sayı bazen metin döndüren bir fonksiyon, onu çağıran her satırı “acaba bu sefer ne geldi?” diye kontrol etmeye zorlar.

Kod ve çıktı.

Kod 11.4 `w11-05-harf-notu.py` — Birden çok `return` noktası: erken dönüş

```
1 # w11-05-harf-notu.py -- Birden çok return noktası: harf notu (erken
   ↪ dönüş)
2
3 def letter_grade(score):
4     """0-100 arası puanı A, B, C, D, F harf notuna çevirir."""
5     if score >= 90:
6         return "A"
7     if score >= 80:
8         return "B"
9     if score >= 70:
10        return "C"
11    if score >= 60:
12        return "D"
13    return "F"                # hiçbir koşul tutmadıysa buraya gelinir
14
15
16 for score in range(100, 55, -10):
17     print(f"{score:>5} -> {letter_grade(score)}")
18
19 # Sınır değerleri: 90 A verir, 89.9 B vermeli
20 print(f"{89.9:>5} -> {letter_grade(89.9)}")
21 print(f"{59.9:>5} -> {letter_grade(59.9)}")
```

```
$ python w11-05-harf-notu.py
```

```
100 -> A
 90 -> A
 80 -> B
 70 -> C
 60 -> D
89.9 -> B
59.9 -> F
```

Kontrol. Kod 11.4’te, 7. haftadaki sınır değer analizi gereği her eşik iki yandan sınanıyor: $90 \rightarrow A$ ve $89,9 \rightarrow B$ ✓; $60 \rightarrow D$ ve $59,9 \rightarrow F$ ✓. Çıktıdaki harfler yukarıdan aşağıya A, A, B, C, D gidiyor; puan azalırken harfin “iyileşmesi” mümkün olmadığından zincirin sırası da doğrudur. Sıra bozulsaydı (`score >= 60` en başa yazılsaydı) bütün geçer notlar D olurdu.



GitHub

11.5 print mi, return mü? Void fonksiyonlar ve None

Bu bölüm haftanın en sık yapılan hatasını konu alır. Gövdesi `print(x * x)` olan bir `square_print` fonksiyonunu `a = square_print(4)` diye çağırırsanız ekranda 16 görürsünüz — işe yarıyor sanırsınız. Ama `print(a)` satırı `None`, `a + 1` satırı ise `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'` verir.

Tanım 11.4: Void fonksiyon ve None

`return` deyimi olmayan (ya da yalnız çıplak `return` yazan) bir fonksiyon, çağırana `None` döndürür. `None`, “değer yok” anlamına gelen özel bir Python nesnesidir; türü `NoneType`’tır ve hiçbir aritmetik işleme girmez. Sonuç döndürmek yerine yalnız bir yan etki üreten (ekrana yazan, dosyaya kaydeden) fonksiyonlara **void fonksiyon** (İng. *void function*) denir. `print()` de böyledir: `x = print("a")` yazarsanız `x` `None` olur.

✗ Sık yapılan hata

`sonuc = yazdir(3) → sonuc = hesapla(3)`

Bu haftanın bir numaralı hatası. Ekran yazan bir fonksiyonun sonucu bir değişkene atanamaz; atarsanız değişkene `None` gelir ve hata, bir sonraki hesapta `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'` olarak patlar. Hata mesajında `NoneType` sözcüğünü gördüğünüz anda düşünecek şey bellidir: *bir yerde return yazmayı unuttum.*

print mi, return mü? Üç soruluk karar

1. Fonksiyonun ürettiği değer başka bir hesapta kullanılabilir mi? **Evet** ise `return`.
2. Fonksiyonun tek işi ekranda bir görüntü oluşturmak mı (çizgi, tablo, menü)? **Evet** ise `print`; fonksiyon void kalır.
3. Kararsız kaldıysanız `return` seçin ve yazdırmayı çağırana bırakın. Dönen bir değeri yazdırmak kolaydır; yazdırılmış bir değeri geri almak imkânsızdır.

Kod 11.5 w11-04-print-vs-return.py — print ile return farkı: değer nereye gider?

```
1 # w11-04-print-vs-return.py -- print ile return farkı: değer nereye gider?
2
3 def square_print(x):
4     """Karesini EKRANA yazar; çağırana bir şey döndürmez."""
5     print(x * x)
6
7
8 def square_return(x):
9     """Karesini ÇAĞIRANA döndürür; ekrana bir şey yazmaz."""
10    return x * x
11
12
13 a = square_print(4)           # 16 ekrana yazılır ...
14 print("a =", a)              # ... ama a'ya None gelir
15 print(type(a))
16
17 b = square_return(4)         # ekrana hiçbir şey yazılmaz ...
18 print("b =", b)              # ... değer b'de saklanır
19 print("b + 1 =", b + 1)      # ve hesapta kullanılabilir
```



GitHub

```
20 print(square_return(4))    # dönen değeri BİZ yazdırıyoruz
21 square_return(4)          # dönen değer kullanılmadı: kaybolur
```

```
$ python w11-04-print-vs-return.py
```

```
16
a = None
<class 'NoneType'>
b = 16
b + 1 = 17
16
```

① Yalnız print eden fonksiyon (void)

```
def square_print(x):
    print(x * x)
a = square_print(4)
```

↓ değer ekrana gider

16 ← ekran

a → None çağırana hiçbir değer dönmedi

```
print(a + 1)
TypeError: unsupported operand type(s)
for +: 'NoneType' and 'int'
```

② return eden fonksiyon

```
def square_return(x):
    return x * x
b = square_return(4)
```

↓ değer çağırana gider

b → 16

(boş) ← ekrana hiçbir şey yazılmadı

```
print(b + 1) # 17
print(square_return(3) + square_return(4))
# 25 -- dönen değer hesapta kullanılır
```

print ekrana yazar, return çağırana değer verir. İkisi birbirinin yerine geçmez: print eden bir fonksiyonun sonucu bir değişkene atanamaz, çünkü return'ü olmayan her fonksiyon None döndürür — “değer yok” anlamına gelen özel bir nesne. Bu haftanın en sık hatası budur. Kural: hesabı fonksiyon yapar, return eder; yazdırmayı çağırana yapar. Yalnız ekrana bir şey çizen fonksiyonlar (print_line, draw_triangle) void kalabilir.

Şekil 11.5: Aynı hesabı yapan iki fonksiyon. Solda değer ekrana gider ve kaybolur; sağda çağırana gider ve kullanılabilir.

Şekil 11.5 iki yolun farkını tek bakışta özetler. Kod 11.5’in son iki satırı ise ince bir ayrımı gösterir: `print(square_return(4))` ekranda 16 gösterir, ama bunu yazdıran fonksiyon değil *bizim* `print` çağrımızdır; hemen ardındaki yalnız `square_return(4)` satırında dönen 16 hiçbir yere atanmadığı için sessizce kaybolur.

11.6 Yerel değişkenler ve kapsam

`square_return` fonksiyonunun içindeki `x`, fonksiyon bittikten sonra nereye gider? Cevap: silinir.

Tanım 11.5: Kapsam ve yerel değişken

Bir adın erişilebildiği program bölgesine o adın **kapsamı** (*İng. scope*) denir. Bir fonksiyonun gövdesinde ilk kez atanan her ad — parametreler dahil — o fonksiyona ait **yerel değişkendir** (*İng. local variable*): yalnız o çağırının çerçevesinde yaşar ve fonksiyon bitince silinir. Fonksiyonların dışında, dosya düzeyinde tanımlanan adlar **global** (*İng. global*) kapsamdadır ve fonksiyonların içinden *okunabilir*.



Sık yapılan hata

`def f(): total = 5 ... print(total) → def f(): return 5 ... print(f())`

Fonksiyonun içinde hesaplanan bir değere dışarıdan erişmeye çalışmak `NameError: name 'total' is not defined` verir. Yerel adlar dışarıya sızmaz; değeri dışarı çıkarmanın tek yolu `return`'dür. Bu "hata" aslında bir özelliktir: sayesinde iki farklı fonksiyonun içinde aynı adı korkusuzca kullanabilirsiniz.

Kod 11.6 w11-06-kapsam.py — Yerel/global adlar, gölgeleme ve global bildirimi

```
1 # w11-06-kapsam.py -- Yerel ve global değişkenler, kapsam ve gölgeleme
2
3 MAX_SCORE = 100                                # global (modül düzeyi) ad
4 count = 0                                       # global ad
5
6
7 def percent(score):
8     ratio = score / MAX_SCORE                  # global ad fonksiyon içinden OKUNABİLİR
9     return ratio * 100                         # ratio yereldir: fonksiyon bitince silinir
10
11
12 def shadow_demo():
13     count = 99                                # aynı adlı YENİ YEREL ad: globali gölgeler
14     print(" shadow_demo içinde count =", count)
15
16
17 def increase():
18     global count                               # "count ile globali kastediyorum" bildirimi
19     count = count + 1
20
21
22 print("percent(85) =", percent(85))
23 shadow_demo()
24 print("shadow_demo sonrası count =", count)    # 0: global değişmedi
25 increase()
26 increase()
27 print("increase() x2 sonrası count =", count)  # 2: global değişti
```

```
$ python w11-06-kapsam.py
```

```
percent(85) = 85.0
shadow_demo içinde count = 99
shadow_demo sonrası count = 0
increase() x2 sonrası count = 2
```

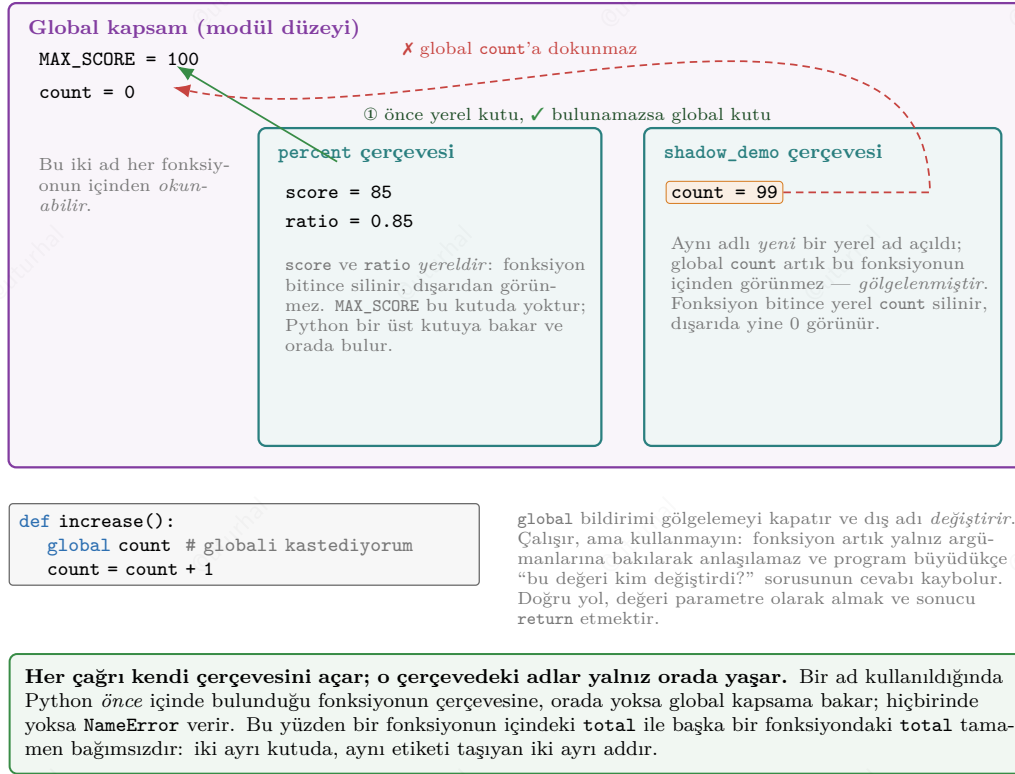


Dikkat

global çalışır, ama kullanmayın. Bir fonksiyon global bir adı değiştirmeye başladığı anda, o fonksiyonun ne yaptığını anlamak için yalnız kendisine bakmak yetmez; “bu değeri kim değiştirdi?” sorusunun cevabı program büyüdükçe kaybolur. Doğru yol neredeyse her zaman aynıdır: **değeri parametre olarak al, sonucu return et.** `increase`'in temiz hâli `def increase(count): return count + 1`'dir; çağırın taraf `count = increase(count)` yazar. Büyük harfli sabitler bu uyarının dışındadır: onlar okunur, değiştirilmez.



GitHub



Şekil 11.6: Global kapsam ve iki çağrı çerçevesi. Bir ad önce yerel kutuda aranır, bulunamazsa global kutuda; aynı adlı bir yerel değişken globali gölgeler.

Kod 11.6’de üç davranışı yan yana, Şekil 11.6’te aynı üçünü kutu kutu görüyorsunuz. percent global MAX_SCORE adını sorunsuz okur. shadow_demo kendi içinde count = 99 yazar; bu bir atama olduğu için Python yeni bir yerel count açar ve globali gölgeler — fonksiyon bitince dışarıdaki count hâlâ 0’dır. increase ise global count bildirimiyle gerçekten global adı değiştirir; iki çağrıdan sonra count 2 olur.

Örnek 11.4 (8. ve 10. haftanın döngüleri fonksiyona dönüşüyor). Faktöriyel, en büyük ortak bölen (Öklid) ve basamak toplamı hesaplarını — daha önce ana programda döngüyle yazdığımız üç algoritmayı — birer fonksiyona dönüştürün ve dönüş değerlerinin başka hesaplarda kullanılabildiğini gösterin.

Çözüm

Analiz. Gövdeler zaten bilinen döngülerdir; yeni olan tek şey sonucu print etmek yerine return etmeleridir.

Fonksiyon arayüzü tasarımı. factorial(n) → n! (0! = 1); gcd(a, b) → EBOB; digit_sum(n) → basamakların toplamı. Üçü de tam sayı alır, int döndürür ve ekrana yazmaz. digit_sum içinde n döngüde değiştirilir; yerel bir ad olduğu için çağıranın değişkenine dokunmaz — kod bunu da smar.

Kod ve çıktı.



Kod 11.7 w11-10-sayi-fonksiyonlari.py — Üç klasik algoritma, üç fonksiyon

```

1 # w11-10-sayi-fonksiyonlari.py -- 8. ve 10. haftanın döngüleri
  ↪ fonksiyon oldu
2
3 def factorial(n):
4     """n! = 1 * 2 * ... * n (0! = 1)"""
5     result = 1
6     for i in range(2, n + 1):
7         result *= i
8     return result
9
10
11 def gcd(a, b):
12     """En büyük ortak bölen (Öklid algoritması)."""
13     while b != 0:
14         a, b = b, a % b
15     return a
16
17
18 def digit_sum(n):
19     """Pozitif tam sayının basamakları toplamı."""
20     total = 0
21     while n > 0:
22         total += n % 10
23         n //= 10          # n yerel bir addır: dışarıyı etkilemez
24     return total
25
26
27 print("5! =", factorial(5), " 10! =", factorial(10))
28 print("gcd(48, 18) =", gcd(48, 18), " gcd(17, 5) =", gcd(17, 5))
29 number = 2026
30 print("digit_sum(2026) =", digit_sum(number), " number hâlâ", number)
31 # Dönüş değerleri başka ifadelerde birleştirilebilir
32 print("digit_sum(9875) =", digit_sum(9875),
33       " EKOK(12, 18) =", 12 * 18 // gcd(12, 18))

```

```
$ python w11-10-sayi-fonksiyonlari.py
```

```

5! = 120  10! = 3628800
gcd(48, 18) = 6  gcd(17, 5) = 1
digit_sum(2026) = 10  number hâlâ 2026
digit_sum(9875) = 29  EKOK(12, 18) = 36

```

Kontrol. Kod 11.7'nin her satırı elle doğrulanabilir: $5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$ ✓; $10! = 3628800$ ✓. EBOB(48,18): $48 = 2 \cdot 18 + 12$, $18 = 1 \cdot 12 + 6$, $12 = 2 \cdot 6 + 0 \rightarrow 6$ ✓; 17 ile 5 aralarında asal olduğundan sonuç 1 ✓. $2 + 0 + 2 + 6 = 10$ ✓ ve $9 + 8 + 7 + 5 = 29$ ✓. Son satır dönüş değerlerinin birleştirilebildiğini gösteriyor: $\text{EKOK}(12, 18) = 12 \cdot 18 / \text{EBOB}(12, 18) = 216 / 6 = 36$ ✓. Ayrıca `digit_sum(number)` çağırısından sonra `number` hâlâ 2026'dır — yerel `n`'in değişmesi dışarıyı etkilemedi ✓.

11.7 Belgeleme: docstring ve help()

Bir fonksiyonu yazan siz, onu üç hafta sonra kullanacak olan da sizsiniz — ve o siz, bu sizi hatırlamayacak. Python bunun için standart bir yer ayırır.

Tanım 11.6: Belgeleme dizesi

Belgeleme dizesi (*İng. docstring*), bir fonksiyonun gövdesindeki *ilk* deyim olarak yazılan karakter dizisidir; genellikle üç tırnak ("""...""") arasında yazılır ve birden çok satır olabilir. Yorum satırından farkı, programın içinde bir *veri* olarak saklanmasıdır: `help(ad)` çağırısı onu ekrana basar, düzenleyiciler yazarken ipucu olarak gösterir. İyi bir docstring üç şeyi söyler: fonksiyon *ne yapar*, parametreler *ne bekler* (anlam ve birim), dönüş değeri *nedir*.

Kod 11.8 w11-07-docstring.py — Docstring ve help()

```
1 # w11-07-docstring.py -- Belgeleme dizesi (docstring) ve help()
2 import math
3
4
5 def circle_area(radius):
6     """Yarıçapı verilen dairenin alanını döndürür.
7
8     radius: yarıçap (float); Dönüş: alan, aynı birimin karesi (float)
9     """
10    return math.pi * radius ** 2
11
12
13 print("circle_area(3) =", circle_area(3))
14 help(circle_area)          # kendi fonksiyonumuzun belgesi
15 help(len)                  # yerleşikğin belgesi de aynı yolla
```

```
$ python w11-07-docstring.py
```

```
circle_area(3) = 28.274333882308138
Help on function circle_area in module __main__:

circle_area(radius)
    Yarıçapı verilen dairenin alanını döndürür.

    radius: yarıçap (float); Dönüş: alan, aynı birimin karesi (float)

Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.
```

Kod 11.8'un çıktısına bakın: `help(circle_area)` ile `help(len)` aynı biçimde çalışıyor — kendi fonksiyonunuz Python'un yerleşikleriyle aynı sistemin parçasıdır.

11.8 Fonksiyon tasarımı ve elle sınama

Bir fonksiyonun doğru çalıştığını nasıl bilirsiniz? Gövdeye bakıp “doğru görünüyor” demek yetmez. Doğru yöntem, fonksiyonu *cevabını zaten bildiğiniz* girdilerle çağırıp sonucu karşılaştırmaktır.

Bir fonksiyonu elle sınamak

1. **Bilinen değerler:** cevabı elden hesaplanabilen 2-3 girdi ($EBOB(48, 18) = 6$ gibi).
2. **Sınır değerler:** eşikler ve hemen yanları (90 ve 89,9) — 7. haftadaki sınır değer analizi burada da geçerlidir.
3. **Uç durumlar:** en küçük anlamlı girdi (0, 1, boş metin), en büyük makul girdi.
4. **Tersine çevirme:** bir dönüşümün tersi ilk değeri geri vermelidir — $f_to_c(c_to_f(37))$ yine 37 olmalıdır.
5. **Tabloyu yazın:** girdi, beklenen çıktı ve gerçek çıktı üç sütunda yan yana; göz kararı karşılaştırma yanlıtır. Bu kontrolleri programa yaptıran `assert` deyimi 12. haftada gelecek.

Kod 11.9 dört geometri fonksiyonundan oluşan küçük bir kütüphanedir. Hiçbiri ekrana yazmaz; tabloyu basan kısım ana programdadır — hesap ile gösterimin ayrılması tam da 11.5'te tartıştığımız ilkedir. “Beklenen” sütunundaki sayılar programdan değil elden gelir: $\pi \cdot 1^2 = 3,1416$; $\pi \cdot 3^2 = 28,27$; $2\pi \cdot 3 = 18,85$; $3 \cdot 4 = 12$; $2(3 + 4) = 14$. İki sütun her satırda birbirini tutuyor ✓. Bir tutarlılık kontrolü daha: $r = 3$ için alan (28,27) çevreden (18,85) büyüktür, çünkü $\pi r^2 > 2\pi r$ eşitsizliği $r > 2$ için doğrudur.

Kod 11.9 w11-09-geometri.py — Dört fonksiyonluk geometri kütüphanesi ve test tablosu

```

1 # w11-09-geometri.py -- Geometri kütüphaneciği ve elle test tablosu
2 import math
3
4
5 def circle_area(r):
6     """Dairenin alanı: pi * r^2"""
7     return math.pi * r ** 2
8
9
10 def circle_perimeter(r):
11     """Dairenin çevresi: 2 * pi * r"""
12     return 2 * math.pi * r
13
14
15 def rectangle_area(width, height):
16     """Dikdörtgenin alanı."""
17     return width * height
18
19
20 def rectangle_perimeter(width, height):
21     """Dikdörtgenin çevresi."""
22     return 2 * (width + height)
23
24
25 # Test tablosu: bilinen girdi -> beklenen çıktı
26 print(f'{"Çağrı":<26}{ "Sonuç":>9}{ "Beklenen":>10}')
27 print("-" * 45)
28 print(f'{"circle_area(1)":<26}{circle_area(1):>9.4f}{3.1416:>10.4f}')
29 print(f'{"circle_area(3)":<26}{circle_area(3):>9.2f}{28.27:>10.2f}')
30 c = circle_perimeter(3)
31 print(f'{"circle_perimeter(3)":<26}{c:>9.2f}{18.85:>10.2f}')
32 print(f'{"rectangle_area(3, 4)":<26}{rectangle_area(3, 4):>9}{12:>10}')
33 p = rectangle_perimeter(3, 4)
34 print(f'{"rectangle_perimeter(3, 4)":<26}{p:>9}{14:>10}')
```



GitHub

```
$ python w11-09-geometri.py
```

Çağrı	Sonuç	Beklenen
circle_area(1)	3.1416	3.1416
circle_area(3)	28.27	28.27
circle_perimeter(3)	18.85	18.85
rectangle_area(3, 4)	12	12
rectangle_perimeter(3, 4)	14	14

Örnek 11.5 (Asal testi fonksiyonu ve 2–100 arasındaki asallar). `is_prime(n)` fonksiyonunu yazın ve onu kullanarak 2 ile 100 arasındaki asal sayıları yazdırın, sayılarını sayın.

Çözüm

Analiz. Girdi bir tam sayı, çıktı bir bool. 10. haftada bayrak değişkeniyle yazdığımız teste artık gerek yok: bölen bulunur bulunmaz `return False` ile çıkılır, döngü sağ salım biterse `return True` çalışır. Bölen aramak $\sqrt{n}$ 'e kadar yeter.

Fonksiyon arayüzü tasarımı. `is_prime(n)`: `is_` öneki dönüşün bool olduğunu duyurur; 2'den küçük sayıları fonksiyon ilk satırda halleder.

Kod ve çıktı.

Kod 11.10 w11-08-asal.py — `is_prime` ve onunla asal listesi

```
1 # w11-08-asal.py -- is_prime(n) fonksiyonu ve onunla 2-100 arasındaki
  ↪ asallar
2
3 def is_prime(n):
4     """n asal ise True, değilse False döndürür."""
5     if n < 2:
6         return False
7     for divisor in range(2, int(n ** 0.5) + 1):
8         if n % divisor == 0:
9             return False          # bölen bulundu: hemen çık
10    return True                    # döngü bitti, bölen yok
11
12
13 print("is_prime(2) =", is_prime(2), " is_prime(91) =", is_prime(91))
14 print("2-100 arasındaki asallar:")
15 count = 0
16 for n in range(2, 101):
17     if is_prime(n):
18         print(n, end=" ")
19         count += 1
20 print()
21 print("Toplam:", count)
```

```
$ python w11-08-asal.py
```

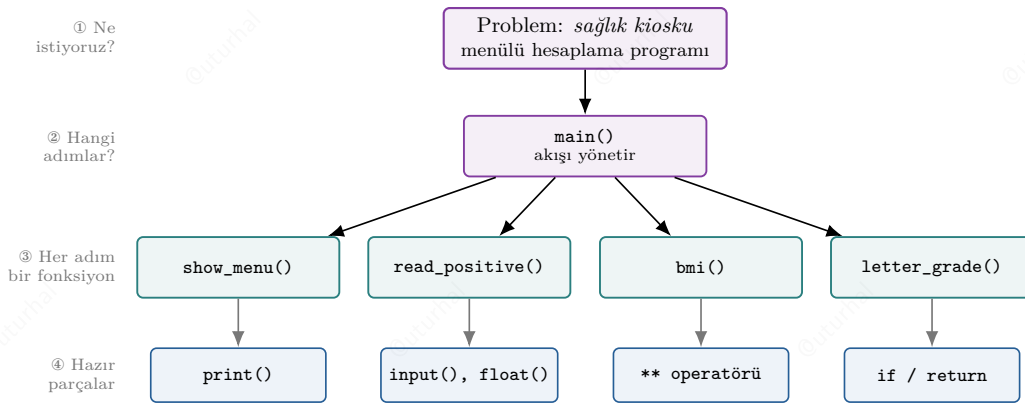
```
is_prime(2) = True is_prime(91) = False
2-100 arasındaki asallar:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Toplam: 25
```

Kontrol. Kod 11.10'da 2 asaldır ✓; $91 = 7 \cdot 13$ olduğu için asal değildir ✓ — bu,

“tek sayı ise asaldır” yanlıgısını kıran iyi bir örnektir. 97 asaldır ✓. 100’e kadar 25 asal sayı bulunduğu bilinen bir gerçektir; program da 25 saydı ✓; listenin ilk elemanları 2,3,5,7,11, sonuncusu 97 ✓. Asıl kazanç ana programın okunabilirliğidir: `if is_prime(n):` satırı, altı satırlık bölen aramasını tek bir cümleye indirir.

11.9 Yukarıdan aşağıya tasarım ve `main()`

Fonksiyonları öğrendiğimize göre, artık programı yazmaya *koddan* değil *plandan* başlayabilirsiniz. **Yukarıdan aşağıya tasarım** (İng. *top-down design*), bir problemi önce büyük adımlara, sonra her adımı daha küçük adımlara bölerek çözme yöntemidir. Her adım bir fonksiyon olur; en üstteki fonksiyon (geleneksel adıyla `main`) yalnız adımları sırayla çağırır, hiçbir ayrıntıyı kendisi yapmaz. Ortaya çıkan yapıya **fonksiyon hiyerarşisi** denir (Şekil 11.7).



Yukarıdan aşağıya tasarım: büyük problemi, her biri tek bir iş yapan küçük parçalara bölmek. Önce en üstteki soruyu yazarsınız, sonra onu çözmek için gereken adımları sıralarsınız; her adım bir fonksiyon olur. Bir kutunun içini yazarken yalnız o kutuyla ilgilenirsiniz — altındaki diğerlerin *nasıl* çalıştığını değil, *ne verdiğini* bilmeniz yeter. Alt sıradaki hazır parçalar (`print`, `input`, `math.sqrt`) tam olarak aynı modeldir; sizin yazdıklarınızla aralarında hiçbir fark yoktur.

Şekil 11.7: Bir problemin fonksiyon hiyerarşisi. Her düzey bir alt düzeyi *kullanır*, nasıl çalıştığını bilmez.

İpucu

`main()` düzeninin üç kuralı: (i) bütün `def`’ler dosyanın başında, (ii) `main` en sonda tanımlanır, (iii) dosyanın en son satırı tek bir `main()` çağrısıdır. Bu düzen, dosyayı açan herkesin programın nereden başladığını tek bakışta görmesini sağlar. 12. haftada bu son satırı `if __name__ == "__main__":` koşuluna alarak dosyayı hem program hem de kütüphane olarak kullanılabilir hâle getireceksiniz.

Bunu somut bir program üzerinde yapalım. 10. haftada yazdığımız döngülü menü programı — kitabın deposunda `w11-11-menu-once.py` — tek parça bir `while` gövdesidir: girdi doğrulama döngüsü iki kez tekrarlanır (Şekil 11.1’deki desenin aynısı), harf notu zinciri menünün ortasına gömülüdür ve gövde bir ekrana sığmaz. Şimdi onu *davranışını hiç değiştirmeden* fonksiyonlara böleceğiz; buna **yeniden düzenleme** (İng. *refactoring*) denir. Önce arayüzleri tasarlıyoruz:

Ad	Parametreler	Dönüş değeri
<code>show_menu</code>	yok	yok (void: yalnız yazar)
<code>read_positive</code>	<code>prompt</code> — istem metni	float, pozitif sayı
<code>bmi</code>	<code>weight, height</code>	float
<code>letter_grade</code>	<code>score</code>	str
<code>main</code>	yok	yok (akışı yönetir)

Program dosyasının son satırı yalnızca `main()` çağrısıdır.

Kod 11.11 `w11-12-menu-sonra.py` — Menü programı, `main()` düzeninde

```

1  # w11-12-menu-sonra.py -- Aynı menü programı, fonksiyonlara ayrılmış (SONRA)
2  # Her fonksiyon tek bir iş yapar; main() yalnız akışı yönetir.
3
4  def show_menu():
5      """Menü seçeneklerini yazar."""
6      print()
7      print("1) VKİ hesapla")
8      print("2) Harf notu")
9      print("0) Çıkış")
10
11
12  def read_positive(prompt):
13      """Pozitif bir sayı okur; geçerli olana kadar yeniden sorar."""
14      value = float(input(prompt))
15      while value <= 0:
16          print("Pozitif bir değer girin.")
17          value = float(input(prompt))
18      return value
19
20
21  def bmi(weight, height):
22      """Vücut kitle indeksi: kütle (kg) / boy (m) karesi."""
23      return weight / height ** 2
24
25
26  def letter_grade(score):
27      """0-100 puanı A-F harf notuna çevirir."""
28      if score >= 90:
29          return "A"
30      if score >= 80:
31          return "B"
32      if score >= 70:
33          return "C"
34      if score >= 60:
35          return "D"
36      return "F"
37
38
39  def main():
40      """Ana akış: menüyü göster, seçimi işle, çıkışa kadar yinele."""
41      while True:
42          show_menu()
43          choice = input("Seçiminiz: ")

```



GitHub


```

44     if choice == "1":
45         weight = read_positive("Kütle (kg): ")
46         height = read_positive("Boy (m): ")
47         print(f"VKİ: {bmi(weight, height):.2f}")
48     elif choice == "2":
49         score = float(input("Puan (0-100): "))
50         print(f"Harf notu: {letter_grade(score)}")
51     elif choice == "0":
52         print("Görüşmek üzere!")
53         break
54     else:
55         print("Geçersiz seçim.")
56
57
58 main()                                # program buradan başlar

```

```
$ python w11-12-menu-sonra.py
```

```

1) VKİ hesapla
2) Harf notu
0) Çıkış
Seçiminiz: 1
Kütle (kg): -5
Pozitif bir değer girin.
Kütle (kg): 70
Boy (m): 1.75
VKİ: 22.86

1) VKİ hesapla
2) Harf notu
0) Çıkış
Seçiminiz: 2
Puan (0-100): 85
Harf notu: B

1) VKİ hesapla
2) Harf notu
0) Çıkış
Seçiminiz: 0
Görüşmek üzere!

```

Kontrol. Kod 11.11 için yeniden düzenlemenin doğruluk ölçütü tektir: *çıkış değişmemelidir*. Düzenlemeden önceki sürüm de aynı girdi dizisiyle (1, -5, 70, 1.75, 2, 85, 0) birebir aynı satırları üretir ✓— bunu laboratuvar da kendiniz doğrulayacaksınız. Sayısal kontroller: $70/1.75^2 = 22.86$ ✓; $85 \rightarrow B$ ✓; -5 girişi reddedilip yeniden soruluyor ✓. Kazanç da sayılabilir: tekrar eden girdi doğrulama bloğu iki kopyadan tek fonksiyona indi ve `main` gövdesi tek ekrana sığar hâle geldi.

LAB Laboratuvar Çalışması 11: Fonksiyonlarla düzenlemek

Önce var olan kodu fonksiyonlara bölecek, sonra küçük bir fonksiyon kütüphanesi yazacaksınız. Her görevde **docstring** yazmayı unutmayın.

Görev 1. Kötü kodu kurtarma. Aşağıdaki programı `kotu.py` adıyla yazın (h ve a için de aynı blok yinelenir), çalıştırın ve çıktısını not edin. Sonra tekrarlanan

bloğu `read_positive(prompt)` fonksiyonuna dönüştürüp programı yeniden yazın. **Ölçüt:** çıktı birebir aynı kalmalı.

```
1 w = float(input("Kütle (kg): "))
2 while w <= 0:
3     print("Pozitif bir değer girin.")
4     w = float(input("Kütle (kg): "))    # aynı blok h ve a
5     ↪ için de var
6 print(f"VKİ: {w / h ** 2:.2f}, yaş: {a:.0f}")
```

Görev 2. Geometri kütüphaneciği. `geometri.py`: `circle_area`, `circle_perimeter`, `rectangle_area`, `rectangle_perimeter` ve `square_area` fonksiyonlarını yazın; her satırında çağrı, sonuç ve *elle hesapladığınız* beklenen değer bulunan bir test tablosu basın (uç durum: yarıçap 0). Kontrol: `square_area(4)` ile `rectangle_area(4, 4)` aynı sayıyı vermelidir. Docstring'leri `help()` ile okutun; sonra tabloya *yanlış* bir beklenen değer koyup yakalandığını görün.

Görev 3. Asal sayılar. `asal.py`: `is_prime(n)`'i bayrak değişkeni kullanmadan yazın; 2–200 arasındaki asalları onarlı satırlar hâlinde yazdırıp sayın. Kontrol: 100'e kadar 25, 200'e kadar 46 asal vardır; `is_prime(1)`, `is_prime(0)`, `is_prime(-7)` False vermelidir.

Görev 4. Void fonksiyonlarla çizim. `cizim.py`: `draw_box(width, height)` fonksiyonunu, üst ve alt kenarı için [Örnek 11.1](#)'deki `print_line`'i çağırarak yazın; void olmalı. Kontrol: `draw_box(5, 3)` beş sütun ve üç satırlık bir çerçeve çizmeli.

Görev 5. Menüü düzenleme. `w11-11-menu-once.py`'yi depodan indirip `main()` düzenine geçirin; menüye “asal mı?” seçeneğini ekleyin (Görev 3'teki `is_prime`). **Ölçüt:** `main` ekrana sığmalı, çıktı değişmemeli.

Teslim. Görev 1–3'ü ders bitmeden gösterin; dosyaları saklayın, 12. haftada `geometri.py`'yi bir *modüle* dönüştüreceksiniz.

Özet

Yapı	Yazım	Not
Tanım ve çağrı	<code>def ad(p1, p2): + gövde; ad(a1, a2)</code>	tanım gövdeyi çalıştırmaz; parantezsiz ad çağırılmaz
Parametre / argüman	tanımda parametre, çağrıda argüman	konumsal: soldan sağa eşleşir; eksik/-fazla sayı <code>TypeError</code> , yanlış sıra <i>hata vermez</i>
Değer döndürme	<code>return ifade</code> ; birden çok <code>return</code>	fonksiyonu o anda bitirir, sonrası ölü kod; <code>else</code> gerekmez
<code>print</code> vs <code>return</code>	ekrana yazar / çağırana verir	<code>return</code> 'süz fonksiyon <code>None</code> döner
Yerel / global ad	gövdede atanan ad / dosya düzeyindeki ad	yerel ad dışarıdan görünmez, global içtenden okunur; aynı adlı yerel ad globali gölgeler, <code>global</code> ile yazılabilir — kaçının
Docstring, <code>main()</code>	"""...""" gövdenin ilk satırında; <code>def</code> 'ler üstte, <code>main()</code> çağrısı en altta	<code>help(ad)</code> ile okunur; programın başladığı yer tek bakışta görünür



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w11>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırmalar

Alıştırma 11.1. ●●● Programın ekrana ne yazacağını *çalıştırmadan* satır satır yazın, sonra doğrulayın.

```
1 def kare_yaz(x):
2     print(x * x)
3
4 def kare_ver(x):
5     return x * x
6
7 a = kare_yaz(3)
8 b = kare_ver(3)
9 print(a, b, kare_ver(2) + kare_ver(3))
```

Cevap: önce 9 (fonksiyonun kendi yazdığı), sonra None 9 13 satırı

Alıştırma 11.2. ●●● `print` satırları hangi değerleri yazdırır? Her biri için nedenini bir cümleyle yazın.

```
1 toplam = 10
2
3 def ekle(x):
4     toplam = x + 1
5     print("içeride:", toplam)
6
7 ekle(5)
8 print("dışarıda:", toplam)
```

Cevap: içeride: 6 ve dışarıda: 10; fonksiyondaki atama yeni bir yerel ad açar ve global `toplam` adını gölgeler

Alıştırma 11.3. ●●● Sıcaklık dönüşümü için iki fonksiyon yazın ($F = C \cdot 9/5 + 32$): `celsius_to_fahrenheit(celsius)` ve `fahrenheit_to_celsius(fahrenheit)`. 100 °C, 212 °F ve -40 °C değerlerini dönüştürün; son satırda 37 °C'yi çevirip geri çevirerek gidiş-dönüş kontrolü yapın.

Cevap: 212,0 °F; 100,0 °C; -40,0 °F (iki ölçeğin kesiştiği tek nokta); gidiş-dönüş kontrolü True

Alıştırma 11.4. ●●● `def kdv(fiyat, oran):` biçiminde tanımlanmış bir fonksiyon için aşağıdaki çağrılarının her biri ne yapar? Hata verenler hangi hatayı verir, hata vermeyenler hangi sonucu üretir?

- (a) `kdv(100, 0.2)` (d) `kdv(100, 0.2, 5)`
 (b) `kdv(0.2, 100)` (e) `kdv`
 (c) `kdv(100)` (f) `sonuc = kdv(100, 0.2)`

Cevap: (a) doğru çağrı, 20,0 (b) hata yok, sonuç anlamsız: "fiyat" 0,2 TL sayılmıştır (c) TypeError: missing 1 required positional argument: 'oran' (d) TypeError: takes 2 positional arguments but 3 were given (e) çağrı değildir, fonksiyon nesnesini ifade eder (f) dönen değer sonuc'a atanır

Alıştırma 11.5. ●● `is_leap(year)` fonksiyonunu yazın: 4'e bölünen yıl artıktır, ancak 100'e bölünüyorsa değildir; 400'e bölünüyorsa yine artıktır. 1896, 1900, 1904, 2000, 2023, 2024 ve 2100 için deneyin, dönen değeri bir `if` koşulunda kullanın.

Cevap: 1896, 1904, 2000, 2024 artık (True); 1900, 2023, 2100 değil (False)

Alıştırma 11.6. ●● `draw_box(width, height)` void fonksiyonunu yazın: içi boş bir kutu çizsin, üst ve alt kenarı için `print_line(ch, n)`'i çağırın. `draw_box(5, 3)`, `draw_box(8, 4)` ve `draw_box(3, 2)` çağrılarını deneyin.

Cevap: 5 × 3 kutu: üç satır; ilk ve son satır #####, ortadaki # #. 3 × 2 kutuda iç satır yoktur

Alıştırma 11.7. ●● `is_palindrome(text)` fonksiyonunu yazın: boşluklar atılıp küçük harfe çevrilen metin tersiyle aynıysa `True` döndürsün. "kayak", "Ey Edip Adanada pide ye", "Trabzon", tek harfli bir metin ve boş metinle deneyin.

Cevap: True, True, False, True, True

Alıştırma 11.8. ●● 10. haftadaki Sezar şifreleme döngüsünü `caesar(text, shift)` fonksiyonuna dönüştürün: büyük ve küçük harfleri `shift` kadar kaydırsın, diğer karakterlere dokunmasın. `caesar("MERHABA", 3)` sonucunu yazdırın, sonra negatif kaydırmayla çözün; son satırda gidiş-dönüş kontrolü yapın.

Cevap: caesar("MERHABA", 3) → PHUKDED; caesar("PHUKDED", -3) → MERHABA; gidiş-dönüş kontrolü True

Alıştırma 11.9. ●●● Bir sayı, kendisi dışındaki pozitif bölenlerinin toplamına eşitse *mükemmel sayıdır*. `is_perfect(n)` fonksiyonunu yazıp önce 6 ve 12 ile sınavın, sonra 1–10 000 arasındaki bütün mükemmel sayıları bulun.

Cevap: is_perfect(6) True (1 + 2 + 3 = 6), is_perfect(12) False (1 + 2 + 3 + 4 + 6 = 16); 10 000'e kadar mükemmel sayılar: 6, 28, 496, 8128

Alıştırma 11.10. ●●● `gcd(a, b)` fonksiyonunu (Öklid) yazın; onu kullanarak `lcm(a, b)` ve bir kesri sadeleştirip ekrana yazan void `print_simplified(num, den)` fonksiyonlarını yazın. EKOK(12,18), EKOK(4,6), EKOK(7,5) ile 12/18, 45/60, 7/5 kesirlerinin sadeleşmiş hâlini yazdırın.

Cevap: 36, 12, 35; 12/18 = 2/3, 45/60 = 3/4, 7/5 = 7/5 (zaten sade)

Alıştırma 11.11. İki öğrenci için ad, vize ve final notu alıp ağırlıklı ortalamayı (vize %40, final %60) ve harf notunu yazan bir program yazın. *Baştan* fonksiyonlarla tasarlayın: puan okuyan (ve 0–100 dışını reddeden), ortalama hesaplayan, harf notu veren, raporu yazan (void) birer fonksiyon ve akışı yöneten `main()`. Ağırlıkları büyük harfli sabitlerde tutun.

Cevap: Ayşe: $72,5 \cdot 0,4 + 81 \cdot 0,6 = 29 + 48,6 = 77,60 \rightarrow C$; Mehmet: $90 \cdot 0,4 + 95 \cdot 0,6 = 36 + 57 = 93,00 \rightarrow A$

Alıştırma 11.12. Bir sayının *dijital kökü*, basamakları toplamı tek basamaklı kalana kadar yinelenince elde edilen sayıdır ($9875 \rightarrow 29 \rightarrow 11 \rightarrow 2$). `digit_sum(n)` ve onu kullanan `digital_root(n)` fonksiyonlarını yazın; 9875, 2026, 7 için sonucu yazdırın ve birkaç sayı için dijital kökün $1 + (n - 1) \bmod 9$ kuralına uyduğunu doğrulayın.

Cevap: $9875 \rightarrow 2$, $2026 \rightarrow 1$ ($10 \rightarrow 1$), $7 \rightarrow 7$; kural her denemede `True`

Hafta 12: Fonksiyonlar II: Esnek Çağrılar, Özyineleme ve Modüller

Öğrenme çıktıları

Bu haftanın sonunda:

1. parametrelere varsayılan değer verebilecek, varsayılanlıların neden sağda durduğunu açıklayabileceksiniz;
2. anahtar sözcüklü argümanlarla çağrı yazabilecek, konumsal ve anahtar sözcüklü argümanların karıştığı bir çağrıda hangi değerın hangi parametreye gittiğini söyleyebileceksiniz;
3. bir fonksiyondan birden çok değer döndürüp bunları $x, y = f()$ biçiminde açabilecek, `*args` ile değişken sayıda argüman alabileceksiniz;
4. bir işi yardımcı fonksiyonlara bölüp katmanlı bir tasarım kurabileceksiniz;
5. özyinelemeli fonksiyon yazabilecek (taban durumu + kendine indirgeme), çağrı ağacını çizebilecek, `RecursionError`'ın nedenini açıklayabilecek ve özyineleme ile döngü arasında gerekçeli seçim yapabileceksiniz;
6. kendi `.py` dosyanızı bir modül hâline getirip `import` edebilecek, `if __name__ == "__main__":` deseninin neden gerektiğini anlatabilecek ve fonksiyonlarınızı `assert` ile sınayabileceksiniz.

Anahtar kavramlar

varsayılan parametre değeri (İng. *default parameter value*) **anahtar sözcüklü argüman** (İng. *keyword argument*) **demet** (İng. *tuple*) **paketleme** (İng. *packing*) **açma** (İng. *unpacking*) **değişken sayıda argüman** (İng. *varargs*) **yardımcı fonksiyon** (İng. *helper function*) **özyineleme** (İng. *recursion*) **taban durumu** (İng. *base case*) **özyinelemeli adım** (İng. *recursive step*) **çağrı ağacı** (İng. *call tree*) **adsız fonksiyon** (İng. *lambda*) **modül** (İng. *module*) **ad uzayı** (İng. *namespace*) **doğrulama deyimi** (İng. *assertion*)

Geçen hafta fonksiyonun ne olduğunu öğrendiniz: bir ada bağlanmış kod bloğu, konumsal parametreler, `return`, kapsam. Bu hafta aynı kutuyu *esnetiyoruz*. Bir fonksiyonun bazı argümanları isteğe bağlı olamaz mı — `print("a")` ile `print("a", end="")` nasıl aynı fonksiyon? Bir fonksiyon iki sayıyı birden veremez mi? Ve bir fonksiyon *kendini* çağırabilir mi? Üçünün de cevabı “evet”tir.

Bir genişleme daha var: şimdiye kadar yazdığınız her fonksiyon, onu kullanan programla aynı dosyada duruyordu; oysa `math.sqrt` başka bir dosyadan geliyor. Bu haftanın sonunda sizin `geometry.py` dosyanız da tam olarak `math` gibi `import` edilebilir bir **modül** olacak — yapay zeka kütüphanelerini kullanmak, başkalarının modüllerini çağırmaktan ibarettir.

12.1 Varsayılan değerler ve anahtar sözcüklü argümanlar

11. haftada bütün argümanlar zorunluydu: `print_line(ch, n)` fonksiyonunu iki argümansız çağıramıyordunuz. Farkı yaratan şey, tanımdaki bir eşittir işaretidir.

Tanım 12.1: Varsayılan parametre değeri

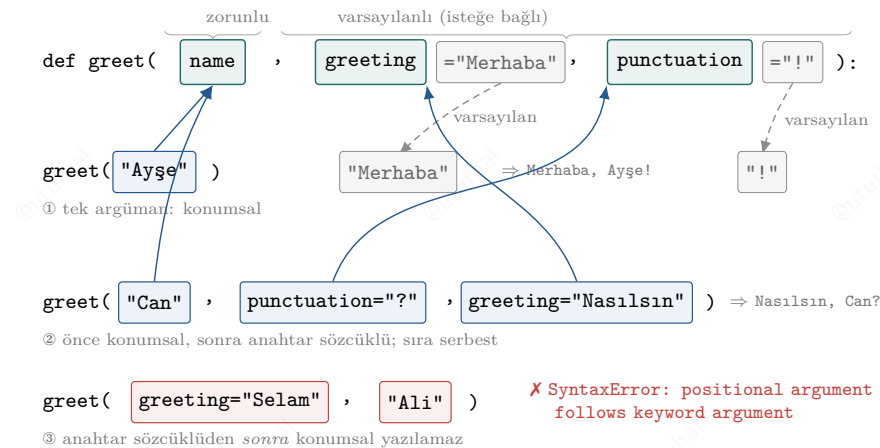
`def` satırında bir parametrenin yanına `=değer` yazılırsa o parametre isteğe bağlı hâle gelir: çağrıda karşılık gelen argüman verilmezse parametre bu **varsayılan değeri** (İng. *default value*) alır. Varsayılan değer `def` satırının *okunduğu anda* bir kez hesaplanır. İyi bir varsayılan, *en sık kullanılan* değerdir (`rate=0.20`).

Kural 12.1 (Varsayılanlı parametreler sağda durur). Varsayılan değeri olan parametreler, parametre listesinin *sonunda* yazılmak zorundadır. `def f(a, b=2, c=3):` geçerlidir; `def f(a=1, b):` ise dosya *okunurken*, program hiç başlamadan `SyntaxError` verir: kunksal argümanlar soldan sağa yerleştiği için “atlanabilir” olanlar sağda toplanmalıdır — yoksa `f(5)` çağrısındaki 5’in hangi parametreye ait olduğu belirsiz kalır.

Varsayılan değerler bir sorun doğurur: üç isteğe bağlı parametreden yalnız *üçüncüsünü* değiştirmek isterseniz, konumsal argümanlarla önce ilk ikisini de yazmak zorunda kalırsınız. Çözüm, argümanı adıyla söylemektir.

Tanım 12.2: Anahtar sözcüklü argüman

Çağrıda **ad=değer** biçiminde yazılan argümana **anahtar sözcüklü argüman** (*Ing. keyword argument*) denir. Böyle bir argüman parametrelere konuma göre değil *ada* göre yerleşir; kendi aralarındaki sıra önemsizdir. Konumsal argümanlar her zaman *önce* yazılır: anahtar sözcüklerden sonra gelen konumsal argüman **SyntaxError: positional argument follows keyword argument** verir. Kullanılan ad, çağırının değil *parametrenin* adı olmalıdır.



Eşleme kuralı. Python önce *konumsal* argümanları soldan sağa parametrelere dağıtır, sonra *anahtar sözcüklü* argümanları adlarına göre yerleştirir, en son boş kalan parametreleri tanımdaki *varsayılan* değerlerle doldurur. Varsayılanı olmayan bir parametre boş kalırsa `TypeError: greet() missing 1 required positional argument` alınır.

Şekil 12.1: Argümanların parametrelere eşlenmesi. Konumsal argümanlar sırayla, anahtar sözcük-lüler adla yerleşir; geri kalanı varsayılan doldurur.

Şekil 12.1 eşleme kuralının tamamını gösteriyor.

Kod 12.1 w12-01-varsayilan.py — Varsayılan değerler, anahtar sözcüklü argümanlar ve karışık çağrılar

```
1 # w12-01-varsayilan.py -- Varsayilan ve anahtar sözcüklü argümanlar
2
3 def greet(name, greeting="Merhaba", punctuation="!"):
4     """Kişiyi selamlar; selam sözcüğü ve noktalama isteğe bağlıdır."""
5     print(f"{greeting}, {name}{punctuation}")
6
7
8 def price with tax(price, rate=0.20):
```




```

9      """Fiyata KDV ekler; oran verilmezse %20 kullanılır."""
10     return price * (1 + rate)
11
12
13     greet("Ayşe")                # eksikler varsayılandan tamamlanır
14     greet("Mehmet", "Günaydın")  # konumsal: sıra parametre sırasıdır
15     greet("Ali", punctuation="...") # anahtar sözcüklü: ada göre eşleşir
16     greet(greeting="Selam", name="Fatma") # sıra serbest
17     greet("Can", punctuation="?", greeting="Nasılsın") # önce konumsal
18     print(f"{price_with_tax(120):.2f} TL ve "
19           f"{price_with_tax(15, rate=0.01):.2f} TL")
20
21 # print() de aynı kurala uyar: sep ve end varsayılanlı, anahtar sözcüklü
22 print("2026", "09", "13", sep="-")
23 print("Yükleniyor", end="")
24 print("... tamam")

```

```
$ python w12-01-varsayılan.py
```

```

Merhaba, Ayşe!
Günaydın, Mehmet!
Merhaba, Ali...
Selam, Fatma!
Nasılsın, Can?
144.00 TL ve 15.15 TL
2026-09-13
Yükleniyor... tamam

```

Kod 12.1'in son satırları bu haftanın en rahatlatıcı keşfidir: `sep` ve `end` birer anahtar sözcüklü argümandır ve `print`'in imzasında varsayılanları vardır. Yani dört haftadır kullandığınız `end=""` yazımı özel bir söz dizimi değil, tam olarak bu bölümün konusuydu.

❌ Sık yapılan hata

`def kdv(oran=0.20, fiyat):` → `def kdv(fiyat, oran=0.20):`

Kural 12.1'in ihlali; program *hiç başlamadan* `SyntaxError` verir. Kural tek cümledir: **zorunlular solda, varsayılanlılar sağda**. İkinci sık hata çağrı tarafındadır: `greet(greeting="Selam", "Ali")`. Bir kez anahtar sözcüklü argüman yazdıysanız sonrasında konumsal yazamazsınız; doğrusu `greet("Ali", greeting="Selam")`'dir. Üçüncüsü yanlış ad kullanmaktır: `price_with_tax(price=120, ratio=0.1)` çağrısı `TypeError: unexpected keyword argument 'ratio'` verir, çünkü parametrenin adı `rate`'tir.

Not

Varsayılan değer *bir kez* hesaplanması, değeri değiştirilebilen türlerde bir tuzağa dönüşür: bütün çağrılar aynı nesneyi paylaşır. Bu hafta varsayılan olarak yalnız sayı, metin ve `True/False` kullanıyoruz; bunlar değiştirilemez olduğu için tuzak işlemez. 13. haftada listeleri öğrendiğinizde `def topla(x, kutu=[])` yazmanın neden sürpriz sonuçlar verdiğine döneceğiz.

12.2 Birden çok değer döndürmek

11. haftanın kuralı “bir fonksiyon tek türde tek bir değer döndürür” idi. Bazı problemler buna direnir: bir bölmenin hem bölümü hem kalanı aynı hesabın parçalarıdır. Python bunu ayrı bir mekanizmayla değil, virgülle çözer.

Tanım 12.3: Çoklu dönüş ve açma

`return a, b` yazıldığında Python iki değeri tek bir **demet** (İng. *tuple*) hâlinde **paketler** (İng. *packing*); fonksiyon yine tek bir değer döndürmüştür. Çağırana tarafta `x, y = f()` yazılırsa paket **açılır** (İng. *unpacking*) ve değerler sırayla soldaki adlara dağıtılır; ad sayısı tutmazsa `ValueError` oluşur. (Demetin kendisini 14. haftada ayrıntılı göreceğiz; burada yalnız “birden çok dönüş değeri” aracıdır.)

Fonksiyonun içi

```
def divide(a, b):
    quotient = a // b
    remainder = a % b
    return quotient, remainder
```

quotient

3

remainder

2

① `return` satırındaki virgül
iki değeri *tek bir pakete* koyar

② demet (İng. *tuple*)
— tek bir dönüş değeri

(3, 2)

Çağırana taraf

`q, r = divide(17, 5)`

q

3

r

2

③ açma: soldaki iki ad, paket-
teki iki değeri sırayla alır

`pair = divide(17, 5)`

pair

(3, 2)

④ tek ad yazılırsa paket
açılmadan o ada girer

Python'da bir fonksiyon aslında hep *tek bir değer döndürür*. `return quotient, remainder` satırındaki virgül, iki değeri bir *demete* paketler; dönen tek değer işte o demettir. `q, r = divide(17, 5)` yazdığınızda ise ters işlem olur: soldaki ad sayısı paketteki değer sayısına eşitse Python paketi *açar* ve değerleri sırayla dağıtır. Sayılar tutmazsa `ValueError: too many values to unpack` alırsınız. Yerleşik `divmod(17, 5)` de tam olarak bu biçimde (3, 2) döndürür.

Şekil 12.2: `return a, b` paketler, `x, y = f()` açar. Tek bir ada atarsanız paket açılmadan o ada girer.

Kod 12.2 w12-02-coklu-donus.py — Çoklu dönüş, açma, `divmod` ve `*args`

```
1 # w12-02-coklu-donus.py -- Birden çok değer döndürme ve *args
2
3 def divide(a, b):
4     """a'yı b'ye böler; bölümü ve kalanı birlikte döndürür."""
5     return a // b, a % b          # iki değer, virgülle
6
7
8 def stats(x, y, z):
9     """Üç sayının en küçüğünü, en büyüğünü ve ortalamasını döndürür."""
10    return min(x, y, z), max(x, y, z), (x + y + z) / 3
11
12
13 def total(*numbers):
14     """Kaç tane verilirse verilsin, sayıların toplamını döndürür."""
15     result = 0
16     for n in numbers:          # numbers bir demettir (14. hafta)
17         result += n
18     return result
19
20
21 q, r = divide(17, 5)          # iki değeri iki ada aç
22 print(f"17 = {q} * 5 + {r}", "| divmod:", divmod(17, 5))
23 lowest, highest, average = stats(72, 91, 64)
24 print(f"En düşük {lowest}, en yüksek {highest}, ortalama {average:.1f}")
```



GitHub

```

25 pair = divide(17, 5) # tek ada atarsanız paket açılmaz
26 print("Tek ad:", pair, type(pair))
27 print(total(3, 4), total(10, 20, 30, 40), total())

```

```
$ python w12-02-coklu-donus.py
```

```

17 = 3 * 5 + 2 | divmod: (3, 2)
En düşük 64, en yüksek 91, ortalama 75.7
Tek ad: (3, 2) <class 'tuple'>
7 100 0

```

Şekil 12.2’deki mekanizmayı Kod 12.2 çalışırken görüyorsunuz. `divide(17, 5)` çağrısı $17 = 3 \cdot 5 + 2$ eşitliğinin iki sayısını birden veriyor; yerleşik `divmod(17, 5)` de $(3, 2)$ döndürüyor — çoklu dönüş, Python’un kendi kütüphanesinde de kullanılan sıradan bir desendir. `stats` ise 72, 91 ve 64 için 64, 91 ve 75,7 veriyor ($227/3 = 75,66\dots$).

Dikkat

Açarken ad sayısı tutmalıdır: `q, r, s = divide(17, 5)` satırı `ValueError: not enough values to unpack (expected 3, got 2)` verir; tersi durumda mesaj `too many values to unpack` olur. Bir fonksiyonun kaç değer döndürdüğü docstring’inde yazılmaktadır.

Bir de karşı yön var: argüman *sayısı* çağrıdan çağrıya değişebilir mi? `print("a")` ve `print("a", "b", "c")` çağrılarının ikisi de çalışıyor; bunu sağlayan yazım, parametre adının önündeki yıldızdır.

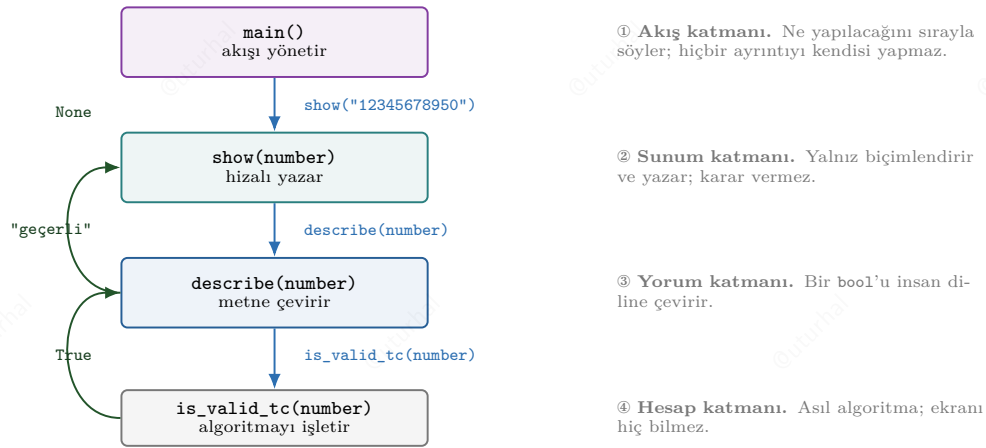
Tanım 12.4: Değişken sayıda argüman: `*args`

`def f(*args):` biçiminde tanımlanan fonksiyon, kendisine verilen *bütün* konumsal argümanları tek bir demet hâlinde `args` adına toplar; gövdede bu ad üzerinde `for` ile dolaşılır. Ad `args` olmak zorunda değildir (`*numbers` da olur); anlamı veren yıldızdır. Yerleşik `print`, `max` ve `min` böyle çalışır.

Kod 12.2’un son satırı bunu gösteriyor: `total(3, 4)` 7, `total(10, 20, 30, 40)` 100, hiç argüman verilmeyen `total()` ise 0 veriyor — boş demet üzerinde döngü hiç dönmediği için biriktiricinin ilk değeri doğrudan sonuç olur.

12.3 Yardımcı fonksiyonlar ve katmanlı tasarım

11. haftada `draw_box` fonksiyonu `print_line`’ı çağırmişti: bir fonksiyon, işinin bir parçasını başka bir fonksiyona *devredebilir*. Yalnız başka fonksiyonlara hizmet etmek için yazılan küçük fonksiyonlara **yardımcı fonksiyon** (İng. *helper function*) denir.



Bir fonksiyonun başka bir fonksiyonu çağırması yeni bir kural gerektirmez: çağrı yığınının bir çerçeve daha eklenir, o kadar. Kazanç, işi *katmanlara* ayırmaktır. Yalnız bir alt katmanı çağıran *yardımcı fonksiyonlar* sayesinde en alttaki hesap fonksiyonu ekranı, en üstteki akış fonksiyonu da algoritmayı bilmek zorunda kalmaz. Yarın çıktı biçimi değişirse yalnız *show*, doğrulama kuralı değişirse yalnız *is_valid_tc* düzeltilir. Bir fonksiyonun *kendini* çağırması da bu resmin içindedir — özyineleme, aynı kutunun bir alt katmanı olmasıdır.

Şekil 12.3: Katmanlı tasarım: her katman yalnız bir altındakini çağırır. Çağrı aşağı iner, sonuç yukarı döner.

Şekil 12.3'daki dört katman tek bir ilkeyi uygular: *hesap yapan fonksiyon ekranı bilmez, ekrana yazan fonksiyon hesabı bilmez*. Böylece çıktı biçimi değişince yalnız üst katman, kural değişince yalnız alt katman düzeltilir.

Örnek 12.1 (Birim dönüştürücü: yardımcı fonksiyonlarla katmanlı tasarım). km–mil ve °C–°F dönüşümlerini yapan menülü bir program yazın. Her dönüşüm ayrı bir fonksiyon olsun; girdiyi doğrulayan ve seçime göre doğru dönüşümü çağıran ayrı fonksiyonlar bulunsun. Dönüşümü çağıran katman, hem sonucu hem de sonucun *birimini* öğrensin.

Çözüm

Analiz. Girdi: bir menü seçimi ve bir pozitif sayı. Çıktı: dönüştürülmüş değer ve birimi. Asıl tasarım kararı, tek satırlık formülleri *kimin* çağıracağıdır.

Fonksiyon arayüzü tasarımı.

Ad	Parametreler	Dönüş değeri
km_to_mile, celsius_to_fahrenheit	tek sayı	float
read_positive	prompt	float, pozitif
convert	choice, value	iki değer: sonuç ve birim
main	yok	yok (akışı yönetir)

convert bu haftanın iki yeniliğini birden kullanır: yardımcılarından uygun olanı *çağırır* ve sonucu birimiyle birlikte *çift* olarak döndürür.

Kod ve çıktı.



GitHub

Kod 12.3 w12-03-birim.py — Katmanlı birim dönüştürücü: yardımcılar, convert ve main

```

1 # w12-03-birim.py -- Yardımcı fonksiyonlar ve main(): birim dönüştürücü
2
3 KM_PER_MILE = 1.609344
4
5
6 def km_to_mile(km):
7     return km / KM_PER_MILE
8
9
10 def celsius_to_fahrenheit(c):
11     return c * 9 / 5 + 32
12
13
14 def read_positive(prompt):
15     """Geçerli (pozitif) bir sayı girilene kadar sorar."""
16     while True:
17         text = input(prompt)
18         if text.replace(".", "", 1).isdigit() and float(text) > 0:
19             return float(text)
20         print("Lütfen pozitif bir sayı girin.")
21
22
23 def convert(choice, value):
24     """Uygun yardımcıyı çağırır; sonucu VE birimini döndürür."""
25     if choice == 1:
26         return km_to_mile(value), "mil"
27     return celsius_to_fahrenheit(value), "°F"
28
29
30 def main():
31     print("1) km -> mil      2) °C -> °F")
32     choice = int(input("Seçiminiz (1-2): "))
33     value = read_positive("Değer: ")
34     result, unit = convert(choice, value) # iki değer birden
35     print(f"Sonuç: {result:.2f} {unit}")
36
37
38 main()

```

```
$ python w12-03-birim.py
```

```

1) km -> mil      2) °C -> °F
Seçiminiz (1-2): 1
Değer: abc
Lütfen pozitif bir sayı girin.
Değer: 42.195
Sonuç: 26.22 mil

```

Kontrol. Kod 12.3 bir maratonun uzunluğunu dönüştürüyor: $42,195/1,609344 = 26,2187\dots \approx 26,22$ mil ✓ — maraton mesafesinin 26,2 mil olduğu bilinen bir gerçek-

tir, büyüklük mertebesi doğru. Girdi doğrulama da sınıyor: `abc` reddedilip soru yineleniyor ✓. Sabiti *tek* bir yerde tutmak (`KM_PER_MILE`) ters dönüşüm eklendiğinde gidiş-dönüş tutarlılığını garanti eder.

12.4 Özyineleme: kendini çağıran fonksiyon

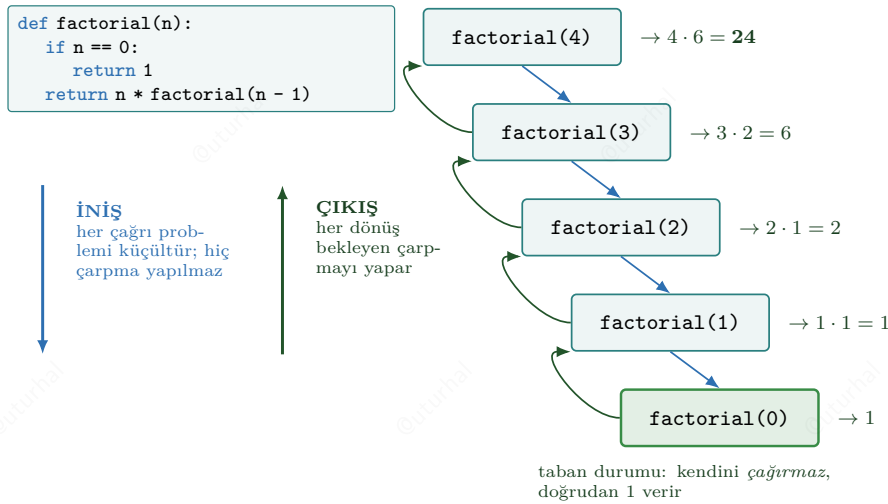
Bir fonksiyon başka bir fonksiyonu çağırabiliyorsa, *kendini* de çağırabilir mi? Evet — ve bu, programlamanın en zarif fikirlerinden biridir.

Tanım 12.5: Özyineleme

Gövdesinde kendi adını çağıran fonksiyona **özyinelemeli** (*Ing. recursive*) fonksiyon denir. İki parçası vardır: (i) **taban durumu** (*Ing. base case*), kendini çağırmadan doğru-
dan sonuç veren en küçük durum; (ii) **özyinelemeli adım** (*Ing. recursive step*), problemi *aynı türden ama daha küçük* bir probleme indirgeyip kendini çağıran kısım. Her çağrı taban durumuna bir adım daha yaklaşımdır.

Faktöriyel tanımın kendisi zaten özyinelemelidir: $0! = 1$ (taban durumu) ve $n! = n \cdot (n-1)!$ (özyinelemeli adım); kod, bu tanımın birebir çevirisidir.

Kural 12.2 (Özyineleme yazmanın üç sorusu). Bir özyinelemeli fonksiyon yazmadan önce şu üç soruyu *yazılı* olarak cevaplayın: (i) **Taban durumu nedir?** Hangi girdide cevabı çağrı yapmadan biliyorum? (ii) **Küçültme nasıl olur?** Problemi kendi türünden hangi daha küçük probleme indirgiyorum? (iii) **Küçülme taban durumuna varıyor mu?** Üçünden biri eksikse fonksiyon ya yanlış sonuç verir ya da `RecursionError` ile durur.



Özyineleme iki yönlü çalışır. Aşağı inerken hiçbir çarpma yapılmaz: her çağrı “*n* çarpı bir küçüğünün sonucu” diye bekler ve çağrı yığımında yerini alır — burada aynı anda beş çerçeve açıktır. Taban durumuna varılınca yön değişir: değerler yukarı doğru çarpılarak geri taşınır ve en üstteki çağrı 24’ü teslim eder. Taban durumu unutulursa iniş hiç bitmez ve Python `RecursionError` verir.

Şekil 12.4: Özyinelemeli `factorial(4)`: çağrılar aşağı iner, değerler yukarı döner. Çarpmalar ancak dönüşte yapılır.



GitHub

Kod 12.4 w12-04-ozyineleme.py — Üç klasik özyineleme; faktöriyel her çağrıyı girintili yazdırıyor

```

1 # w12-04-ozyineleme.py -- Faktöriyel, basamak toplamı ve EBOB
2
3 def factorial(n, depth=0):
4     """n! değerini özyinelemeyle hesaplar; çağrıları girintili yazar."""
5     pad = " " * depth
6     print(f"{pad}factorial({n}) çağrıldı")
7     if n == 0:                                # taban durumu
8         print(f"{pad}taban durumu -> 1")
9         return 1
10    result = n * factorial(n - 1, depth + 1)
11    print(f"{pad}factorial({n}) = {n} * factorial({n - 1}) = {result}")
12    return result
13
14
15 def digit_sum(n):
16     """Son basamak + kalan sayının basamak toplamı."""
17     if n < 10:                                # taban durumu: tek basamaklı sayı
18         return n
19     return n % 10 + digit_sum(n // 10)
20
21
22 def gcd(a, b):
23     """Öklid: EBOB(a, b) = EBOB(b, a mod b); taban EBOB(a, 0) = a."""
24     if b == 0:
25         return a
26     return gcd(b, a % b)
27
28
29 print("4! =", factorial(4))
30 print("digit_sum(4729) =", digit_sum(4729),
31       "| gcd(1071, 462) =", gcd(1071, 462),
32       "| gcd(48, 18) =", gcd(48, 18))

```

```
$ python w12-04-ozyineleme.py
```

```

factorial(4) çağrıldı
  factorial(3) çağrıldı
    factorial(2) çağrıldı
      factorial(1) çağrıldı
        factorial(0) çağrıldı
          taban durumu -> 1
        factorial(1) = 1 * factorial(0) = 1
      factorial(2) = 2 * factorial(1) = 2
    factorial(3) = 3 * factorial(2) = 6
  factorial(4) = 4 * factorial(3) = 24
4! = 24
digit_sum(4729) = 22 | gcd(1071, 462) = 21 | gcd(48, 18) = 6

```

Kod 12.4'nin girintili çıktısı Şekil 12.4'in kendisidir: satırlar aşağı doğru genişleyerek `factorial(0)`'a iner, sonra aynı girintiler geri kapanarak 1, 1, 2, 6, 24 değerlerini yukarı taşır. Bir özyinelemeli fonksiyonu anlamamanın en hızlı yolu budur: gövdeye geçici bir `print` koyup çağrı zincirini görmek. Aynı dosyadaki `digit_sum` ve `gcd` de aynı desenedir — taban durumu artı kendine indirgeme: $4 + 7 + 2 + 9 = 22$ ✓ ve $\text{EBOB}(1071, 462) = 21$ (ortak

çarpanlar $3 \cdot 7$) ✓.

✗ Sık yapılan hata

`def factorial(n): return n * factorial(n - 1)` → ilk satıra `if n == 0: return 1` ekleyin
Taban durumu olmayan bir fonksiyon kendini sonsuza kadar çağırır. Python bunu bellek tükenene kadar sürdürmez: yığında biriken çerçeve sayısı bin civarına geldiğinde durur.

Çıktı

```
File "hata.py", line 2, in factorial
    return n * factorial(n - 1)
[Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

`RecursionError` gördüğünüzde iki olasılık vardır: ya taban durumunu yazmayı unuttunuz, ya da yazdınız ama *ona hiç varılmıyor* — örneğin `factorial(-1)` çağırısı `n == 0` durumuna hiç uğramaz; `n <= 0` yazmak bunu da kapatır.

i İpucu

Özyinelemeli bir fonksiyonda hata ararken gövdenin ilk satırına geçici bir `print("giriyorum:", n)` ve `return`'den hemen önce bir `print("çıkıyorum:", n)` koyun. Ekrandaki girinti düzeni doğrudan çağrı ağacını çizer: satırların açılıp kapanmadığını görürseniz taban durumuna varılmıyor demektir. İşiniz bitince iki satırı silmeyi unutmayın.

Örnek 12.2 (Üs alma: aynı sonuç, iki maliyet). b^n değerini iki ayrı yolla hesaplayan fonksiyonlar yazın: (a) döngüyle, (b) üssü her adımda yarıya indiren *hızlı* özyinelemeyle. İkisinin yaptığı çarpma sayısını bir sayaçla ölçüp karşılaştırın ve her satırda iki sonucun eşit olduğunu `assert` ile sıayın.

Çözüm

Analiz. İki sürüm de aynı sayıyı üretmelidir; fark *maliyettedir*. Hızlı sürümün fikri şudur: b^n için $h = b^{n/2}$ bir kez hesaplanır ve n çiftse $h \cdot h$, tekse $h \cdot h \cdot b$ döndürülür. (Doğrudan özyineleme, yani $b^n = b \cdot b^{n-1}$, tam olarak döngü kadar — n tane — çarpma yapar; ilginç olan üçüncü yoldur.)

Algoritma. [Kural 12.2](#)'in üç sorusu: taban durumu $n = 0$ ($b^0 = 1$); küçültme, üssü tam bölmeyle ikiye ayırmak; tam bölme üssü her adımda kesin olarak azalttığı için taban durumuna mutlaka varılır. Adımlar: ① üs 0 ise 1 döndür; ② değilse h 'yi üssün yarısı için hesapla; ③ üs çiftse $h \cdot h$, tekse $h \cdot h \cdot b$ döndür.

Kod ve çıktı.



Kod 12.5 w12-06-us-alma.py — Döngülü ve hızlı özyinelemeli üs alma; çarpımlar sayılıyor

```

1 # w12-06-us-alma.py -- Üs alma: döngü ve hızlı özyineleme
2
3 mults = 0                                # ölçüm sayacı: kaç çarpma yapıldı?
4
5
6 def power_loop(base, exponent):
7     """Döngüyle: exponent kez çarpılır."""
8     global mults
9     result = 1
10    for _ in range(exponent):
11        result *= base
12        mults += 1
13    return result
14
15
16 def power_fast(base, exponent):
17     """Hızlı üs alma: üs her adımda YARIYA iner."""
18     global mults
19     if exponent == 0:                    # taban durumu
20         return 1
21     half = power_fast(base, exponent // 2)
22     mults += 1
23     if exponent % 2 == 0:
24         return half * half
25     mults += 1
26     return half * half * base
27
28
29 print(f"{'n':>3} {'2**n':>22} {'döngü':>7} {'hızlı':>7}")
30 for n in (10, 20, 40, 64):
31     counts = []
32     for power in (power_loop, power_fast):
33         mults = 0
34         assert power(2, n) == 2 ** n    # ikisi de aynı sayıyı vermeli
35         counts.append(mults)
36     print(f"{'n':>3} {'2 ** n':>22} {'counts[0]':>7} {'counts[1]':>7}")

```

\$ python w12-06-us-alma.py

n	2**n	döngü	hızlı
10	1024	10	6
20	1048576	20	7
40	1099511627776	40	8
64	18446744073709551616	64	8

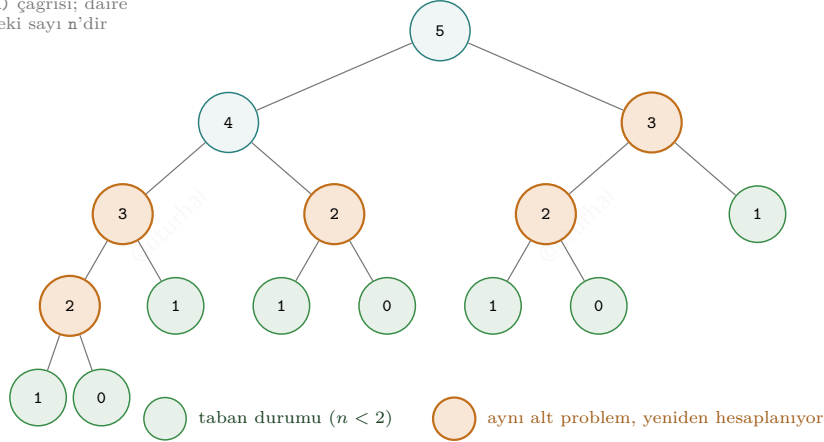
Kontrol. Kod 12.5'ün tablosunda $2^{10} = 1024$ ✓ ve $2^{64} = 18\,446\,744\,073\,709\,551\,616$ ✓ — ikincisi, 1. haftada gördüğünüz 64 bitlik bir sayacın alabileceği değer sayıdır. Asıl kazanç son sütundadır: 2^{64} için döngü 64 çarpma yaparken hızlı sürüm yalnız 8 çarpma yapıyor ✓; beklenen sayı $\log_2 64 = 6$ ile $2\log_2 64 = 12$ arasında olmalıydı. Programın içindeki `assert` satırı ayrıca iki sürümün *aynı* sayıyı ürettiğini her turda

sınıyor.

12.5 Özyineleme mi, döngü mü?

Özyineleme her zaman iyi bir fikir değildir. Bunu en açık gösteren örnek Fibonacci dizisidir: $F_0 = 0$, $F_1 = 1$ ve $F_n = F_{n-1} + F_{n-2}$. Tanım yine özyinelemelidir — ama bu kez fonksiyon kendini *iki* kez çağırır.

fib(n) çağrısı; daire içindeki sayı n'dir



Toplam 15 çağrı, ama yalnız 6 farklı alt problem: fib(2) üç, fib(1) beş kez yeniden hesaplanıyor.

Faktöriyelde her çağrı bir çağrı doğurur, Fibonacci'de *iki*. Bu yüzden ağaç zincir değil, ikiye ayrılan bir ağaçtır ve boyu her adımda neredeyse ikiye katlanır: fib(5) 15, fib(20) 21 891, fib(30) ise 2 692 537 çağrı ister. Sebebi açık — aynı alt problem defalarca sıfırdan hesaplanıyor. Döngülü sürüm son iki değeri saklayarak aynı işi n adımı bitirir. Özyineleme her zaman değil, alt problemler *tekrar etmediğinde* iyi bir seçimdir.

Şekil 12.5: fib(5) çağrı ağacı. Aynı alt problem defalarca sıfırdan hesaplanıyor; turuncu daireler tekrarları gösterir.

Şekil 12.5'ndeki ağaçta fib(5) için 15 çağrı var, oysa yalnız 6 farklı alt problem mevcut. Aşağıdaki Kod 12.6'nın tablosu israfın nereye vardığını gösteriyor: $n = 10$ için 177, $n = 20$ için 21 891, $n = 30$ için 2 692 537 çağrı. Döngülü sürüm aynı sonuçları sırasıyla 10, 20 ve 30 adımda verir.

Kod 12.6 w12-05-fibonacci.py — Fibonacci: özyinelemeli ve döngülü; çağrı sayıları sayaçla ölçülüyor

```
1 # w12-05-fibonacci.py -- Fibonacci: özyineleme, döngü, çağrı sayısı
2
3 calls = 0                                     # her çağrıda artan sayaç (ölçüm için)
4
5
6 def fib(n):
7     """n. Fibonacci sayısı: fib(0)=0, fib(1)=1, fib(n)=fib(n-1)+fib(n-2)"""
8     global calls
9     calls += 1
10    if n < 2:                                   # iki taban durumu: 0 ve 1
11        return n
12    return fib(n - 1) + fib(n - 2)             # kendini İKİ kez çağırır
13
```



GitHub

```

14
15 def fib_loop(n):
16     """Aynı dizi döngüyle: son iki değeri taşı."""
17     previous, current = 0, 1
18     for _ in range(n):
19         previous, current = current, previous + current
20     return previous
21
22
23 print("İlk 12 Fibonacci sayısı:", end=" ")
24 for n in range(12):
25     print(fib_loop(n), end=" ")
26 print()
27 print(f"{n':>3} {'fib(n)':>8} {'özyineleme':>14} {'döngü':>9}")
28 for n in (5, 10, 20, 30):
29     calls = 0
30     print(f"{n:>3} {fib(n):>8} {calls:>18} {n:>11}")

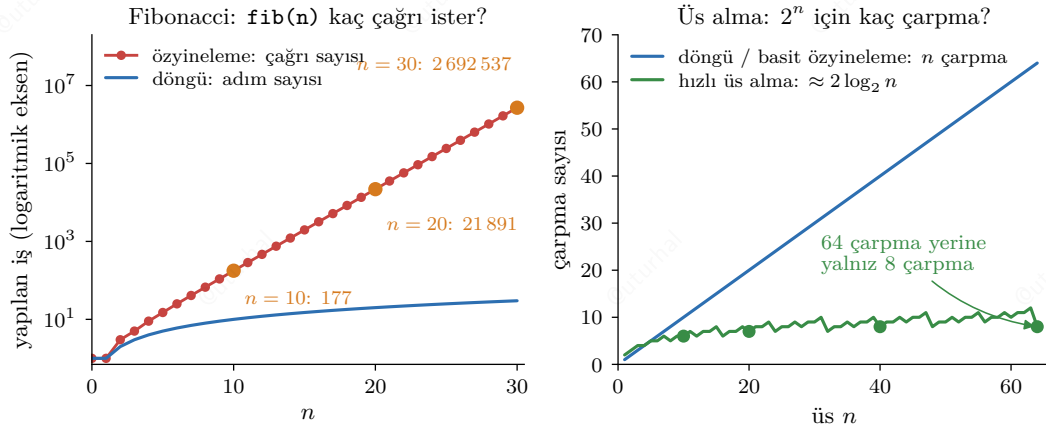
```

```
$ python w12-05-fibonacci.py
```

```

İlk 12 Fibonacci sayısı: 0 1 1 2 3 5 8 13 21 34 55 89
n  fib(n)  özyineleme  döngü
5   5      15        5
10  55     177       10
20  6765   21891    20
30  832040 2692537   30

```



Şekil 12.6: Solda Fibonacci: özyinelemeli çağrı sayısı üstel, döngü adımı doğrusal (dikey eksen logaritmik). Sağda üs alma: hızlı sürüm n yerine $\approx 2 \log_2 n$ çarpma yapar.

Şekil 12.6 iki yüzü yan yana koyuyor: solda özyineleme kaybediyor, sağda kazanıyor. Fark, alt problemlerin *tekrar edip etmemesidir*; Fibonacci’de $\text{fib}(n-1)$ ile $\text{fib}(n-2)$ ağaçları büyük ölçüde örtüşür, hızlı üs almada ise her adım problemi gerçekten ikiye böler.

Özyineleme mi, döngü mü? Dört soruluk karar

1. Problemin *tanımı* kendi kendine mi atıf yapıyor (faktöriyel, Hanoi, bir metnin tersi)? Evet ise özyineleme kodu kısaltır ve okunur kılar.
2. Alt problemler *tekrar ediyor* mu? Ediyorsa (Fibonacci) saf özyineleme pahalıdır; döngü seçin.

3. Derinlik ne kadar? Girdi binden büyük olabiliyorsa `RecursionError` riski vardır; döngü güvenlidir.
4. İkisi de uygunsa *okunabilir* olamı seçin. Her özyinelemeli fonksiyon bir döngüyle, her döngü de özyinelemeyle yazılabilir; seçim bir yapabilirlik değil, tasarım sorunudur.

12.6 Adsız fonksiyonlar: lambda ve key

11. haftada bir fonksiyonun adını parantezsiz yazmanın onu *çağırmadığını* görmüştünüz: fonksiyonlar da birer *değerdir*; bir değişkene atanabilir, başka bir fonksiyona argüman olarak verilebilir.

Tanım 12.6: lambda ifadesi

lambda parametreler: ifade yazımı, adı olmayan küçük bir fonksiyon *üretir*. Gövdesi tek bir ifadedir ve bu ifadenin değeri kendiliğinden döndürülür; `return` yazılmaz, birden çok satır olamaz. `kare = lambda x: x * x` tanımı ile `def kare(x): return x * x` tanımı aynı işi yapar.

`lambda`'nın asıl kullanım yeri, bir fonksiyonu *argüman* olarak isteyen yerlerdir; en sık karşılaşılanı `key` parametresidir. `sorted`, `max` ve `min`, karşılaştırma yapmadan *önce* her elemene `key` fonksiyonunu uygular.

Kod 12.7 w12-07-lambda.py — lambda ile adsız fonksiyonlar; key ile ölçüt değiştirme

```
1 # w12-07-lambda.py -- Adsız fonksiyonlar ve key parametresi
2
3 def square_def(x):                                # aynı işi yapan iki tanım
4     return x * x
5
6
7 square = lambda x: x * x
8
9 print(square_def(7), square(7))
10
11 # key: karşılaştırma ÖNCESİ her elemene uygulanan fonksiyon
12 print(max(3, -7, 5))                             # olduğu gibi en büyük
13 print(max(3, -7, 5, key=abs))                     # mutlak değeri en büyük olan
14 print(min("Trabzon", "Ankara", "İzmir", key=len)) # en kısa metin
15
16 # Karakterleri sıralama: büyük harfler Unicode'da küçüklerden önce gelir
17 word = "PythOn"
18 print("".join(sorted(word)))                      # 'O' ve 'P' başa
19 print("".join(sorted(word, key=lambda c: c.lower()))) # harf sırası
```

```
$ python w12-07-lambda.py
```

```
49 49
5
-7
İzmir
OPhnty
hnOPty
```



GitHub

Kod 12.7'nin çıktısı bu fikri üç kez gösteriyor. `max(3, -7, 5)` doğal olarak 5 verir;

`max(3, -7, 5, key=abs)` ise `-7` verir, çünkü artık karşılaştırılan sayılar 3, 7 ve 5'tir — ama dönen değer yine *asıl* elemandır. `key=len` ise en kısa metni seçtirir. Son iki satır bir metnin karakterlerini sıralıyor: düz sıralamada büyük harfler öne geçer ("`PythOn`" → "`OPhnty`"), çünkü Unicode kod noktalarında büyük harfler küçüklerden önce gelir (5. hafta); `key=lambda c: c.lower()` eklendiğinde sonuç "`hnOPty`" olur.

! Dikkat

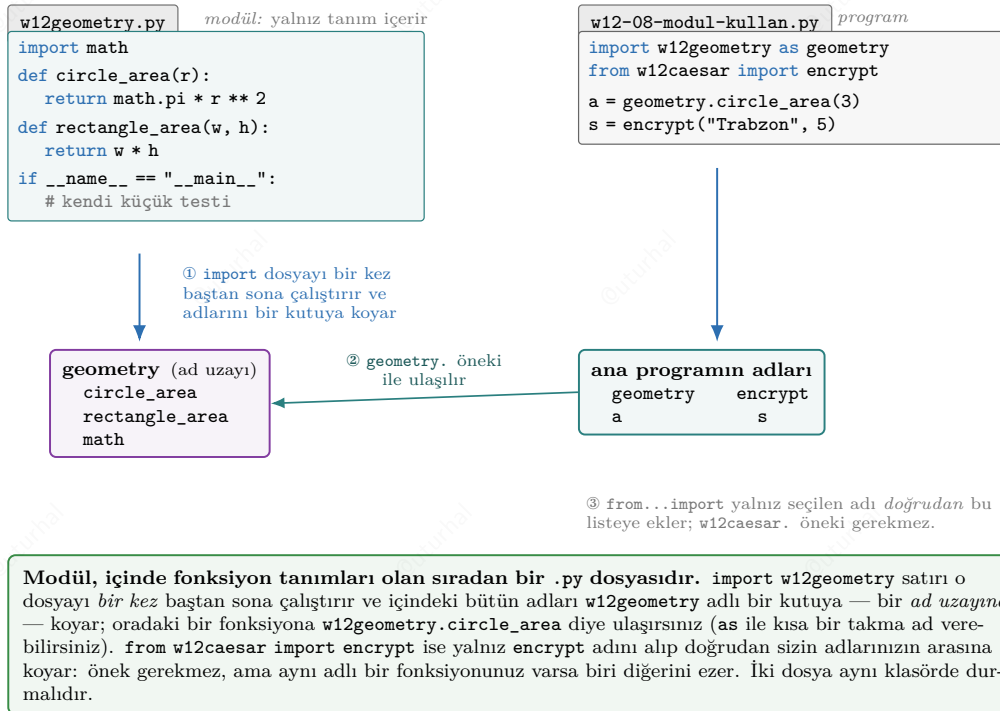
`lambda`'yı bir ada atamak çalışır, ama `def` kullanmak neredeyse her zaman daha iyidir: `def` bir ad, bir docstring ve okunur bir hata mesajı verir. `lambda`'yı yalnız *tek seferlik* ve çok kısa olduğunda, özellikle `key=` içinde kullanın.

12.7 Kendi modülünüzü yazmak

`import math` satırını on bir haftadır yazıyorsunuz. `math`, içinde fonksiyon tanımları olan bir dosyadan başka bir şey değildir — sizin `geometry.py` dosyanız gibi.

Tanım 12.7: Modül

Modül (*İng. module*), içindeki tanımlar başka programlardan kullanılmak üzere yazılmış bir `.py` dosyasıdır. `import` ad satırı (uzantısız) o dosyayı *bir kez* baştan sona çalıştırır ve bütün adlarını ad adlı bir **ad uzayına** (*İng. namespace*) koyar; `ad.fonksiyon()` diye ulaşılır. `import ...as` kısa takma ad verir; `from ad import f, g` ise seçilen adları doğrudan sizin adlarınıza ekler.



Şekil 12.7: Modül ve onu kullanan program: iki dosya, iki içe aktarma biçimi ve ad uzayı.

Şekil 12.7'daki iki dosya aynı klasörde durur. Python, `import` satırındaki adı önce çalışan programın klasöründe arar; bu yüzden kendi modülünüz hazır kütüphanelerle aynı kolaylıkla bulunur — ama aynı sebeple, dosyanıza `math.py` adını vererseniz gerçek `math` modülünü

gölgeler. (Depodaki dosya adları hafta önekiyle başlar; kendi bilgisayarınızda `geometry.py` adını verin ve `import geometry` yazın.)

Bir modül dosyasını doğrudan çalıştırdığınızda kendi küçük testlerini yazdırmalı, ama bu testler modülü `import` eden programın ekranına karışmamalıdır. Python bu ayrımı tek bir değişkenle çözer.

Kural 12.3 (`if __name__ == "__main__":` deseni). Python her dosyaya `__name__` adlı bir değişken verir. Doğrudan çalıştırılan dosyada bu değişkenin değeri `"__main__"`, `import` edilen dosyada ise *modülün adıdır*. Bu yüzden bu koşulun içine yazılan satırlar yalnız dosya *program olarak* çalıştırıldığında yürür, `import` edildiğinde atlanır. Kural: dosyanın *tanımları* üstte, *çalışan kısmı* bu bölüm içinde durur.

Aynı dosyanın iki ayrı yaşamı vardır: `$ python w12geometry.py` komutunda `__name__` değişkeni `"__main__"` olur ve testler çalışır; `import w12geometry` satırından geçildiğinde ise `"w12geometry"` olur ve blok atlanır. 11. haftada programın son satırı yalnız `main()` çağrısıydı; artık o satır bu bölümün içine girer. [Kod 12.8](#) bu düzenin tam örneğidir: yukarıda yalnız tanımlar, aşağıda yalnız dosyaya özel testler.

Kod 12.8 `w12geometry.py` — Geometri modülü: yalnız tanımlar ve kendi testleri

```
1 # w12geometry.py -- Geometri modülü (sizde: geometry.py)
2 """Düzlem geometrisi yardımcıları.
3
4 Kullanım: import w12geometry as geometry
5           print(geometry.circle_area(3))
6 """
7 import math
8
9
10 def circle_area(r):
11     """Yarıçapı r olan dairenin alanı."""
12     return math.pi * r ** 2
13
14
15 def circle_perimeter(r):
16     """Yarıçapı r olan dairenin çevresi."""
17     return 2 * math.pi * r
18
19
20 def rectangle_area(width, height):
21     """Dikdörtgenin alanı."""
22     return width * height
23
24
25 def square_area(side):
26     """Karenin alanı; işi dikdörtgen fonksiyonuna devreder."""
27     return rectangle_area(side, side)
28
29
30 def triangle_area(a, b, c):
31     """Heron formülü; kenarlar üçgen oluşturmuyorsa 0 döndürür."""
32     if a + b <= c or a + c <= b or b + c <= a:
33         return 0
34     s = (a + b + c) / 2
35     return math.sqrt(s * (s - a) * (s - b) * (s - c))
```



GitHub

```

36
37
38 if __name__ == "__main__":          # yalnız DOĞRUDAN çalıştırılırsa
39     print("w12geometry kendini test ediyor...")
40     assert square_area(4) == rectangle_area(4, 4) == 16
41     assert triangle_area(3, 4, 5) == 6.0
42     assert triangle_area(1, 2, 10) == 0
43     assert abs(circle_area(1) - math.pi) < 1e-9
44     print("Tüm testler geçti.")

```

```
$ python w12geometry.py
```

```

w12geometry kendini test ediyor...
Tüm testler geçti.

```

Örnek 12.3 (Geometri modülü ve onu kullanan program). 11. haftanın laboratuvarında yazdığınız geometri fonksiyonlarını gerçek bir modüle dönüştürün: dosyanın kendi testleri `assert` ile yazılsın ve yalnız doğrudan çalıştırıldığında işlesin. Sonra bu modülü *başka* bir dosyadan kullanan, kullanıcıdan yarıçap alıp alan ve çevre hesaplayan bir program yazın; program ayrıca Sezar şifre modülünden yalnız iki fonksiyonu `from ...import` ile alsın.

Çözüm

Analiz. İki ayrı dosya gerekir. Birincisi yalnız *tanım* içerir ve hiçbir şey yazdırmaz (kendi testleri hariç); ikincisi kullanıcıyla konuşur ve hesabı modüle devreder — [Şekil 12.3](#)'daki katman ayrımının iki *dosyaya* taşınmış hâli.

Fonksiyon arayüzü tasarımı. Modülde beş fonksiyon var (daire alanı ve çevresi, dikdörtgen ve kare alanı, üçgen alanı); hepsi sayı döndürür, hiçbirini ekrana yazmaz. `square_area` işi `rectangle_area`'ya devreder — modülün kendi içinde de yardımcı fonksiyon kullanılır. `triangle_area` geçersiz kenarlarda 0 döndürür.

Kod ve çıktı. Modül [Kod 12.8](#)'dedir; onu kullanan program aşağıdadır. `import w12geometry as geometry` modülün tamamını takma adla, `from w12caesar import encrypt, decrypt` ise yalnız iki adı alır.

Kod 12.9 w12-08-modul-kullan.py — Modülleri kullanan program: iki içe aktarma biçimi bir arada

```

1 # w12-08-modul-kullan.py -- Kendi modüllerimizi kullanan ana program
2 import w12geometry as geometry          # modülün tamamı, takma adla
3 from w12caesar import encrypt, decrypt  # yalnız iki ad alınır
4
5 r = float(input("Yarıçap (cm): "))
6 print(f"Alan : {geometry.circle_area(r):.2f} cm²")
7 print(f"Çevre: {geometry.circle_perimeter(r):.2f} cm")
8
9 # from ... import ... ile gelen adlar modül adı yazılmadan kullanılır
10 secret = encrypt("Trabzon 2026", 5)
11 print("Şifreli:", secret, "| Çözülen:", decrypt(secret, 5))
12 print("Modülün adı:", geometry.__name__, "| Bu dosya:", __name__)

```



GitHub

```
$ python w12-08-modul-kullan.py
```

```
Yarıçap (cm): 3
Alan : 28.27 cm2
Çevre: 18.85 cm
Şifreli: Ywfgets 2026 | Çözülen: Trabzon 2026
Modülün adı: w12geometry | Bu dosya: __main__
```

Kontrol. Modül doğrudan çalıştırıldığında “kendini test ediyor” satırını yazdı (Kod 12.8) ✓; `import` edildiğinde o satırlar Kod 12.9’in çıktısında görünmüyor ✓ — Kural 12.3’in gözle görülür kanıtı. Sayısal kontroller: $\pi \cdot 3^2 = 28,274 \dots \approx 28,27$ ✓ ve $2\pi \cdot 3 = 18,849 \dots \approx 18,85$ ✓ — ikisi de 11. haftanın test tablosuyla birebir aynı. Sezar kontrolü gidiş-dönüşle yapıyor: "Trabzon 2026" → "Ywfgets 2026" → "Trabzon 2026" ✓; rakamlar ve boşluk dokunulmadan kalıyor.

✗ Sık yapılan hata

`import w12geometry.py` → `import w12geometry`
`import` satırında *uzantı yazılmaz*; yazarsanız `ModuleNotFoundError` alırsınız — Python noktadan sonrasını “modülün içindeki bir modül” sanar. Aynı hata iki dosya farklı klasörlerde durduğunda da çıkar. En sinsisi ise modül dosyasına hazır bir kütüphanenin adını vermektir (`math.py`): kendi dosyanız gerçeğini gölgeler.

12.8 Fonksiyonları sınamak: `assert`

11. haftada bir fonksiyonu “bilinen girdi – beklenen çıktı” tablosuyla elle sınamıştınız. Program büyüdükçe bu yöntem çöker: kimse yirmi satırlık bir çıktıyı her değişiklikten sonra tek tek okumaz. Karşılaştırmayı programa yaptıralım.

Tanım 12.8: `assert` deyimi

`assert` koşul deyimi koşulu sınar: `True` ise *hiçbir şey yapmaz*, program sessizce devam eder; `False` ise `AssertionError` ile durur ve `Traceback` hangi satırın düştüğünü gösterir. `assert` koşul, "mesaj" biçiminde ikinci bir argüman verilirse mesaj da yazdırılır. Sessizlik başarı demektir.

Örnek 12.4 (Test fonksiyonlarıyla *sınama*). Harf notu veren ve palindrom denetleyen iki fonksiyon için birer *test fonksiyonu* yazın. Testler sınır değerleri ve uç durumları kapsasın; hepsi geçerse ekrana kısa bir onay satırı yazılsın.

Çözüm

Analiz. Test fonksiyonu, sınanan fonksiyonu bilinen girdilerle çağırıp sonucu beklenen değerle `assert` eder. Beklenen değerler programdan değil *elden* gelir — yoksa test yalnız “kod kendisiyle tutarlı” demiş olur.

Algoritma. Girdileri 11. haftanın listesine göre seçin: bilinen değerler (100 → AA), sınır değerler ve hemen yanları (90 → AA ama 89,9 → BB), uç durumlar (boş metin) ve en az bir olumsuz durum.

Kod ve çıktı.

Kod 12.10 w12-09-test-assert.py — `assert` ve `test` fonksiyonlarıyla sınama

```

1 # w12-09-test-assert.py -- assert ve test fonksiyonlarıyla sınama
2
3 def letter_grade(score):
4     """0-100 arası puanı harf notuna çevirir."""
5     if score >= 90:
6         return "AA"
7     if score >= 70:
8         return "BB"
9     if score >= 50:
10        return "CC"
11    return "FF"
12
13
14 def is_palindrome(text):
15     """Boşlukları ve harf büyüklüğünü yok sayar.
16
17     >>> is_palindrome("Trabzon")
18     False
19     """
20     cleaned = text.replace(" ", "").lower()
21     return cleaned == cleaned[::-1]
22
23
24 def test_letter_grade():
25     assert letter_grade(100) == "AA"
26     assert letter_grade(90) == "AA"           # sınır: tam eşikte
27     assert letter_grade(89.9) == "BB"        # sınır: bir altında
28     assert letter_grade(50) == "CC"
29     assert letter_grade(49) == "FF"
30     print("test_letter_grade: geçti")
31
32
33 def test_is_palindrome():
34     assert is_palindrome("Kayak") is True    # harf büyüklüğü
35     assert is_palindrome("") is True         # uç durum: boş metin
36     assert is_palindrome("Trabzon") is False
37     print("test_is_palindrome: geçti")
38
39
40 test_letter_grade()
41 test_is_palindrome()

```

```
$ python w12-09-test-assert.py
```

```
test_letter_grade: geçti
test_is_palindrome: geçti
```

Kontrol. Kod 12.10'in çıktısı üç satırdan ibarettir — ve bu iyi haberdır: `assert`'ler sessiz kaldığına göre sekiz sınamanın hepsi geçmiştir ✓. Testin gerçekten çalıştığını görmek için bir satırı bilerek bozun: `letter_grade(89.9)` için "AA" bekleyin; program o satırda `AssertionError` ile durur ✓. Sınır değerler iki yandan sınanıyor (90 →

"AA", 89,9 $\rightarrow$ "BB"); zincirin sırası bozulsaydı bu ikisinden biri mutlaka düşerdi.

Not

Kod 12.10'teki `is_palindrome` fonksiyonunun docstring'inde bir `>>` satırı ve hemen altında beklenen çıktı var. Bu süs değil: Python'un `doctest` modülü bir dosyayı `$ python -m doctest dosya.py -v` komutuyla tarar, docstring'lerdeki `>>` örneklerini çalıştırır ve altlarındaki beklenen çıktıyla karşılaştırır. Böylece belge ile test aynı yerde durur. Bu derste `doctest`'i tanımanız yeterlidir; sınamayı `assert` ile yapacağız.

Örnek 12.5 (Kimlik numarası doğrulayıcı). 11 haneli bir kimlik numarasının geçerli olup olmadığını denetleyen `is_valid_tc` fonksiyonunu yazın. Kurallar: numara tam 11 rakamdan oluşur, ilk hane 0 olamaz, onuncu hane tek konumlardaki ilk beş hanenin toplamının 7 katından çift konumlardaki dört hanenin toplamı çıkarılıp 10'a bölününce kalan sayıya eşit olmalı, on birinci hane ise ilk on hanenin toplamının 10 ile bölümünden kalan olmalıdır. Sonucu okunur biçimde yazan yardımcı fonksiyonlar da ekleyin.

Çözüm

Analiz. Girdi bir metindir (başında sıfır olabileceği ve aritmetik yapılmayacağı için sayı değil), çıktı bir `bool`. İş üç katmana ayrılır: biçim denetimi, kontrol basamaklarının hesabı, sonucun sunumu.

Algoritma.

Sözde kod: Kimlik numarası doğrulama

```
FONKSİYON is_valid_tc(number)
    EĞER uzunluk  $\neq$  11 VEYA rakam değil VEYA ilk hane = "0" İSE
        DÖNDÜR yanlış
    odd_sum  $\leftarrow$  1., 3., 5., 7., 9. hanelerin toplamı
    even_sum  $\leftarrow$  2., 4., 6., 8. hanelerin toplamı
    EĞER (odd_sum * 7 - even_sum) mod 10  $\neq$  10. hane İSE DÖNDÜR yanlış
    DÖNDÜR (ilk 10 hanenin toplamı) mod 10 = 11. hane
```

Kod ve çıktı.



Kod 12.11 `w12-10-tc-dogrula.py` — Kimlik doğrulayıcı: biçim, iki kontrol basamağı ve sunum katmanı

```

1 # w12-10-tc-dogrula.py -- 11 haneli kimlik numarası doğrulayıcısı
2 # NOT: Numaralar gerçek kişilere ait değil, sınama için kurgusaldır.
3
4 def is_valid_tc(number):
5     """Biçimi ve iki kontrol basamağını doğrular.
6
7     10. hane = ((1,3,5,7,9. haneler)*7 - (2,4,6,8. haneler)) % 10
8     11. hane = (ilk 10 hanenin toplamı) % 10
9     """
10    if len(number) != 11 or not number.isdigit() or number[0] == "0":
11        return False
12    odd_sum = 0 # 1,3,5,7,9. haneler (indeks
13    ↪ 0,2,4,6,8)
14    even_sum = 0 # 2,4,6,8. haneler (indeks 1,3,5,7)
15    for i in range(0, 9, 2):
16        odd_sum += int(number[i])
17    for i in range(1, 8, 2):
18        even_sum += int(number[i])
19    if (odd_sum * 7 - even_sum) % 10 != int(number[9]):
20        return False
21    return (odd_sum + even_sum + int(number[9])) % 10 ==
22    ↪ int(number[10])
23
24 def describe(number):
25     """Sonucu okunur metne çevirir (yorum katmanı)."""
26     return "geçerli" if is_valid_tc(number) else "GEÇERSİZ"
27
28 def show(number):
29     """Numarayı ve sonucu hizalı yazar (sunum katmanı)."""
30     print(f"{number:>12}: {describe(number)}")
31
32
33 for tc in ("12345678950", "10000000146", "12345678901",
34           "0234567895", "123a"):
35     show(tc)
36 print("Girdiğiniz numara", describe(input("Numara girin: ").strip()))

```

```
$ python w12-10-tc-dogrula.py
```

```

12345678950: geçerli
10000000146: geçerli
12345678901: GEÇERSİZ
0234567895: GEÇERSİZ
123a: GEÇERSİZ
Numara girin: 10000000146
Girdiğiniz numara geçerli

```

Kontrol. Kod 12.11'nin ilk sınama numarası 12345678950 elle doğrulanabilir: tek

konumlardaki haneler $1 + 3 + 5 + 7 + 9 = 25$, çift konumlardakiler $2 + 4 + 6 + 8 = 20$; onuncu hane $(25 \cdot 7 - 20) \bmod 10 = 155 \bmod 10 = 5$ ✓ ve numaranın onuncu hanesi gerçekten 5. İlk on hanenin toplamı $1+2+\dots+9+5 = 50$, on birinci hane $50 \bmod 10 = 0$ ✓. 12345678901 yalnız son hanesiyle ayrılıyor ve reddediliyor ✓ — tek basamaklık bir yazım hatasını yakalamak kontrol basamağının tüm amacıdır. 0234567895 hem 10 haneli hem sıfırla başlıyor ✓; 123a rakam olmayan karakter içeriyor ✓. Bu algoritma numaranın *tutarlı* olduğunu gösterir, *var olduğunu* değil — rastgele on bir rakamın tutarlı çıkma olasılığı yaklaşık $1/100$ 'dür.

LAB Laboratuvar Çalışması 12: Kendi kütüphaneni yaz

Bu hafta iki dosyayla çalışacaksınız: bir *modül* ve onu kullanan bir *program*. İkisini aynı klasöre koyun. Görev 1 ve 2 herkesten beklenir.

Görev 1. Geometri modülü ve testleri. 11. haftanın *geometri.py* dosyasını *geometry.py* adıyla yeniden düzenleyin: `print` çağrılarını `assert`'lere çevirip `__name__` bloğuna alın. `circle_area` için varsayılanlı bir parametre ekleyin: `circle_area(r, digits=2)` sonucu yuvarlasın. **Ölçüt:** dosyayı doğrudan çalıştırınca “tüm testler geçti” yazmalı. Kontrol: `square_area(4)` ile `rectangle_area(4, 4)` eşit, `triangle_area(1, 2, 10)` 0 olmalı.

Görev 2. Modülü kullanan program. *alan.py*: `import geometry` desin, kullanıcıdan şekil ve ölçü alıp sonucu yazsın. Sonra aynı programın başına `from geometry import circle_area` ekleyip bu fonksiyonu öneksiz çağırın. **Ölçüt:** modülün kendi test satırları bu programın çıktısında *görünmemeli*; nedenini bir yorum satırıyla açıklayın.

Görev 3. Kimlik doğrulayıcı. *kimlik.py*: `is_valid_tc(number)` ve onu sınavan `test_is_valid_tc()` fonksiyonlarını yazın. En az altı sınaama olsun: iki geçerli numara, yanlış uzunluk, sıfırla başlayan, rakam olmayan karakter içeren ve son basamağı bozulmuş bir numara. Kontrol: 12345678950 geçerli, 12345678901 geçersiz olmalı.

Görev 4. Özyineleme atölyesi. *ozyineleme.py*: `factorial(n)`, `fib(n)` ve `reverse(text)` fonksiyonlarını özyinelemeyle yazın; her birinin yanına döngülü sürümünü koyup 0–15 arası girdilerde `assert` ile eşitliği sınavın. Sonra `fib`'e çağrı sayacı ekleyip $n = 5, 10, 15, 20$ için sayıyı yazdırın. Kontrol: `fib(20) = 6765` ve çağrı sayısı 21 891 olmalı.

Görev 5. Sayı tahmin oyununu bölme. 8. haftadaki oyunu *tahmin.py* içinde dört fonksiyona bölün: `pick_secret(low=1, high=100)` (varsayılanlı!), `read_guess(prompt)`, `give_hint(guess, secret)` (metin döndürsün, yazdırmasın) ve `play()`. Oyunun sonunda tahmin sayısını *döndürün*; `main` bunu yazsın. **Ölçüt:** `play()` gövdesi on satırı geçmemeli.

Teslim. Görev 1–3'ü ders bitmeden gösterin. Dosyaları saklayın: 13. haftada listeleri öğrenince *geometry.py*'ye toplu hesap yapan fonksiyonlar ekleyeceksiniz.

Özet

Yapı	Yazım	Not
Varsayılan değer	<code>def f(a, b=2):</code>	varsayılanlılar <i>sağda</i> ; değer tanımda bir kez hesaplanır
Anahtar sözcüklü ar-güman	<code>f(1, b=2); f(b=2, a=1)</code>	önce konumsal, sonra anahtar sözcüklü; adlar <i>parametre</i> adlarıdır
Çoklu dönüş, <i>*args</i>	<code>return a, b / x, y = f();</code> <code>def f(*args):</code>	tek bir demet döner; ad sayısı tutmazsa ValueError
Yardımcı fonksiyon	bir fonksiyon başkasını çağırır	katman: hesap ekranı, ekran hesabı bilmez
Özyineleme	taban durumu + kendine indirgeme	küçülme taban durumuna varmalı; yoksa RecursionError
Özyineleme vs döngü	tanım kendine atıf yapıyorsa özyineleme	alt problemler tekrar ediyorsa (Fibonacci) döngü
lambda ve key	<code>sorted(s, key=lambda c: c.lower())</code>	tek ifadelik adsız fonksiyon; yalnız key gibi yerlerde
Modül, <code>__name__</code>	<code>import ad; if __name__ == "__main__":</code>	uzantı yazılmaz; blok yalnız doğrudan çalıştırmada girer
assert	<code>assert f(90) == "AA"</code>	doğruysa sessiz; yanlışsa AssertionError



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w12>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırmalar

Alıştırma 12.1. ●●● Aşağıdaki tanım için verilen çağrılarının her biri ne yapar? Hata verenler hangi hatayı verir, hata vermeyenler ne yazdırır?

```
1 def etiket(ad, onek="Sayın", sonek="."):
2     print(f"{onek} {ad}{sonek}")
```

- | | |
|-------------------------------|--------------------------------------|
| (a) etiket("Ayşe") | (d) etiket(onek="Merhaba", ad="Ali") |
| (b) etiket("Ayşe", "Değerli") | (e) etiket(onek="Merhaba", "Ali") |
| (c) etiket("Ayşe", sonek="!") | (f) etiket() |

Cevap: (a) Sayın Ayşe. (b) Değerli Ayşe. (c) Sayın Ayşe! (d) Merhaba Ali. (e) **SyntaxError: positional argument follows keyword argument** (f) **TypeError: missing 1 required positional argument: 'ad'**

Alıştırma 12.2. ●●● Aşağıdaki iki fonksiyonun ikisi de kendini çağırıyor, ama ikisi de hatalı. Her biri çalıştırıldığında ne olur, neden? Her birini tek satırlık bir değişiklikle düzeltin.

```

1 def toplam(n):                # 1 + 2 + ... + n bekleniyor
2     return n + toplam(n - 1)
3
4 def geri_say(n):              # n, n-1, ..., 1 yazdırması bekleniyor
5     if n == 0:
6         return
7     print(n)
8     geri_say(n)

```

Cevap: İkisi de RecursionError: maximum recursion depth exceeded verir. Birincide taban durumu yoktur (if n == 0: return 0 eklenmeli); ikincide taban durumu vardır ama ona hiç varılmaz, çünkü çağrı küçülüyor (geri_say(n - 1) olmalı)

Alıştırma 12.3. ●● Varsayılan üssü 2 olan `power(base, exponent=2)` fonksiyonunu yazın. Üç çağrı deneyin: yalnız taban vererek, iki argümanı da konumsal vererek ve iki argümanı da anahtar sözcüklü *ters sırada* vererek.

Cevap: $3^2 = 9$, $2^{10} = 1024$, $5^3 = 125$

Alıştırma 12.4. ●● Üç sayının en küçüğünü ve en büyüğünü *birlikte* döndüren `min_max(a, b, c)` fonksiyonunu yazın (yerleşik `min` ve `max`'ı kullanmadan). Kullanıcıdan üç sayı alıp sonucu iki ayrı ada açın, farklarını yazdırın ve son satırda sonucu yerleşik fonksiyonlarla karşılaştırarak doğrulayın. Girdi 5, 2, 9 ile deneyin.

Cevap: En küçük 2, en büyük 9, fark 7; doğrulama satırı True

Alıştırma 12.5. ●● `*args` kullanarak, kendisine verilen sözcüklerin en uzununu döndüren `longest(*words)` fonksiyonunu yazın. Dört sözcükle, tek sözcükle ve *hiç* sözcük vermeden çağırın.

Cevap: Trabzon; Python; boş metin (")

Alıştırma 12.6. ●● Bir sayının basamak sayısını *özyinelemeyle* bulan bir fonksiyon yazın: adı `count_digits` olsun, sayıyı metne çevirmeden çalışsın. 4729, 7, 1 000 000 ve 2^{64} için deneyin; her satırda `len(str(n))` ile karşılaştırarak doğrulayın.

Cevap: 4, 1, 7, 20 basamak; dört satırda da karşılaştırma aynı sayıyı verir

Alıştırma 12.7. ●● Bir metni *özyinelemeyle* ters çeviren `reverse(text)` fonksiyonunu yazın (dilimleme kullanmadan). "Trabzon", bir palindrom ve boş metinle deneyin; son satırda sonucu `[::-1]` ile karşılaştırarak doğrulayın.

Cevap: nozbarT; palindromun tersi kendisidir (True); boş metnin tersi boş metin; doğrulama True

Alıştırma 12.8. ●●● Kendi metin araçları modülünüzü yazın: `textutils.py` içinde `count_vowels(text)`, `is_palindrome(text)` ve varsayılan ayırıcılı `initials` fonksiyonları bulunsun (ikinci parametre `separator`, varsayılanı nokta). Modülün kendi testleri `__name__` bloğunda `assert` ile yazılsın. Sonra başka bir dosyadan `import textutils` deyip üç fonksiyonu da kullanın.

Cevap: count_vowels("Programlama") = 4; initials("Uğur Turhal") = U.T.; initials("ayşe yılmaz kaya", "-") = A-Y-K-; modülü import eden program test satırlarını görmez

Alıştırma 12.9. ●●● EBOB'u özyinelemeyle hesaplayan `gcd(a, b, verbose=False)` fonksiyonunu yazın: `verbose` doğruysa her çağrıyı yazdırsın. Onu kullanarak `lcm(a, b)` fonksiyonunu yazın. 1071 ve 462 için adımları ve çağrı sayısını gösterin, EKOK'u hesaplayın ve sonucun iki sayıya da bölündüğünü doğrulayın.

Cevap: EBOB(1071, 462) = 21, 4 çağrı (1071, 462 → 462, 147 → 147, 21 → 21, 0); EKOK = 1071 · 462 / 21 = 23 562; doğrulama True

Alıştırma 12.10. ●●● Onluk bir sayıyı ikili gösterimine özyinelemeyle çeviren `to_binary(n)` fonksiyonunu yazın (`bin` kullanmadan; sonuç bir metin olsun). 37, 255, 0, 1 ve 1024 için deneyin; her satırda `int(sonuc, 2)` ile geri çevirerek doğrulayın.

Cevap: 37 → 100101, 255 → 11111111, 0 → 0, 1 → 1, 1024 → 1000000000; beş satırda da doğrulama True

Alıştırma 12.11. ●●● *Hanoi kuleleri*: üç direk ve farklı boyutlarda n disk vardır; diskler başlangıçta büyükten küçüğe A direğinde durur. Her hamlede bir direğin en üstteki diski alınıp başka bir direğe konur ve büyük bir disk asla küçüğün üstüne konamaz. Dört parametrelili `hanoi(n, source, target, spare)` fonksiyonunu yazıp hamleleri numaralı biçimde yazdırın ve sayın. $n = 3$ için çalıştırın; 5, 10, 15 ve 20 disk için hamle sayısını da yazdırın.

Cevap: $n = 3$ için 7 hamle; 5 → 31, 10 → 1023, 15 → 32 767, 20 → 1 048 575; genel formül $2^n - 1$

Hafta 13: Listeler: Çok Sayıda Veriyi Tek Adla Tutmak

Öğrenme çıktıları

Bu haftanın sonunda:

1. liste oluşturabilecek, uzunluğunu ölçebilecek, pozitif ve negatif indeksle erişebilecek, dilimleyebileceksiniz;
2. listenin *değiştirilebilir* olmasının ne demek olduğunu açıklayabilecek; **append**, **insert**, **extend**, **remove**, **pop**, **del** ve **clear** işlemlerini doğru kullanabileceksiniz;
3. liste üzerinde üç döngü biçiminden (eleman, **enumerate**, indeks) doğru olanı seçebilecek ve basit liste üreticileri yazabileceksiniz;
4. **takma ad** ile **kopya** farkını bir bellek şemasıyla anlatabilecek, bir listeyi fonksiyona geçirmenin sonuçlarını öngörebileceksiniz;
5. doğrusal arama, ikili arama, kabarcık ve seçmeli sıralamayı kodlayabilecek ve adım sayılarını karşılaştırabileceksiniz;
6. bir sayı listesinin ortalama, medyan, mod ve frekans bilgilerini hesaplayan programlar yazabileceksiniz.

Anahtar kavramlar

liste (İng. *list*) **eleman** (İng. *element*) **indeks** (İng. *index*) **dilim** (İng. *slice*) **değiştirilebilirlik** (İng. *mutability*) **metot** (İng. *method*) **takma ad** (İng. *aliasing*) **kopya** (İng. *copy*) **yan etki** (İng. *side effect*) **liste üretici** (İng. *list comprehension*) **doğrusal arama** (İng. *linear search*) **ikili arama** (İng. *binary search*) **kabarcık sıralaması** (İng. *bubble sort*) **seçmeli sıralama** (İng. *selection sort*) **frekans** (İng. *frequency*)

Şimdiye kadar her değer kendi adı vardı: **midterm**, **final**, **radius**. Sekizinci haftada bir sınıfın not ortalamasını hesapladık, ama notları bir biriktiriciye ekleyip *unuttuk*; döngü bittiğinde elimizde yalnız toplam kalmıştı. “Kaç kişi ortalamanın üstünde?” ya da “ikinci en yüksek not kaç?” diye sorulduğunda yapabileceğimiz tek şey bütün notları yeniden istemektir. Kırk öğrenci için kırk değişken açmak ise ne yazılabilir ne de okunabilir bir çözümdür. Bu haftanın konusu olan **liste** tam bu boşluğu doldurur: tek bir adla, sıralı ve indeksli olarak istediğiniz kadar değeri tutar — ve hepsine geri dönebilirsiniz.

Liste, yapay zeka mühendisliğinde de her şeyin başladığı yerdir: bir veri kümesi sayı listesidir, bir modelin tahminleri listedir, bir metin işlenirken önce kelime listesine dönüşür. İkinci sınıfta kullanacağınız NumPy dizileri ve pandas sütunları, buradaki listenin hızlandırılmış hâlinde başka bir şey değildir. Bu yüzden bu haftanın iki fikri — *indeksle erişim* ve *değiştirilebilirlik* — dersin geri kalanının taşıyıcı kolonlarıdır.

13.1 Liste nedir: bir ad, çok değer

Tanım 13.1: Liste

Liste (İng. *list*), birden çok değeri *sıralı* biçimde tutan bir Python nesnesidir. Değerler köşeli parantez içinde, virgülle ayrılarak yazılır:

```
scores = [72, 85, 91, 58, 77]
```

Listenin her değerine **eleman** (İng. *element*) denir. Elemanlar aynı türden olmak zorunda değildir; listenin uzunluğu `len()` ile ölçülür ve program çalışırken değişebilir.

Bir liste dört yolla kurulur: doğrudan köşeli parantezle, boş başlatıp sonradan doldurarak ([]), başka bir diziyi çevirerek (`list(range(1, 6))`) ve yineleyerek (`[0] * 4` — sayacı listeleri böyle açılır). Metinlerde öğrendiğiniz üç işlem listede de aynen çalışır: `+` uç uca ekler, `*` yineler, `in` üyelik sorar. Beşinci haftada “metin bir dizidir” demiştik; artık ikinci bir dizi türümüz var ve aradaki tek — ama belirleyici — fark, listenin *değiştirilebilir* olmasıdır (13.3).

Kod 13.1 w13-01-liste-temel.py — Liste oluşturma, uzunluk, indeks ve dilimleme

```
1 # w13-01-liste-temel.py -- Liste oluşturma, uzunluk, indeks, dilimleme
2
3 scores = [72, 85, 91, 58, 77]      # köşeli parantez, virgülle ayrılmış
4 mixed = ["Ayşe", 19, 3.85, True]  # farklı türler bir arada olabilir
5 numbers = list(range(1, 6))       # range'i listeye çevir
6 zeros = [0] * 4                   # yineleme: dört tane 0
7
8 print(scores, type(scores))
9 print(mixed, numbers, zeros, [])
10 print("Eleman sayısı:", len(scores))
11 print("İlk:", scores[0], "Son:", scores[-1], "Üçüncü:", scores[2])
12
13 # Dilimleme [başla:bitir:adım] -- bitir hariç; sonuç YENİ bir listedir
14 print("İlk üç  :", scores[:3], "| son iki:", scores[-2:])
15 print("Atlamalı:", scores[:2], "| tersten:", scores[::-1])
16
17 # Metinle aynı işlemler: birleştirme, yineleme, üyelik
18 print("Birleştirme:", scores + [100, 0], "| yineleme:", [1, 2] * 3)
19 print("Üyelik:", 85 in scores, 60 in scores)
20 print("Parçalama:", scores[:2] + scores[2:] == scores)
```

\$ python w13-01-liste-temel.py

```
[72, 85, 91, 58, 77] <class 'list'>
['Ayşe', 19, 3.85, True] [1, 2, 3, 4, 5] [0, 0, 0, 0] []
Eleman sayısı: 5
İlk: 72 Son: 77 Üçüncü: 91
İlk üç  : [72, 85, 91] | son iki: [58, 77]
Atlamalı: [72, 91, 77] | tersten: [77, 58, 91, 85, 72]
Birleştirme: [72, 85, 91, 58, 77, 100, 0] | yineleme: [1, 2, 1, 2, 1, 2]
Üyelik: True False
Parçalama: True
```

Kod 13.1’in çıktısında `type(scores)` sonucu `<class 'list'>`’tir: liste de `int` gibi bir türdür. `mixed` listesi dört ayrı türü bir arada tutar; Python buna izin verir, ama pratikte bir listenin elemanları aynı türdendir — onunla yapacağınız işlem toplama, sıralama ya da ortalama ise ancak tek türde anlamlıdır.

❌ Sık yapılan hata

`list = [72, 85, 91] → scores = [72, 85, 91]`

`list`, Python’un yerleşik tür dönüştürme işlevidir. Ona bir liste atarsanız aynı programda bir daha `list(range(5))` yazamazsınız: `TypeError: 'list' object is not callable`. Aynı tuzak `sum`, `min`, `max`, `sorted` ve `len` için de geçerlidir — dördüncü haftada `print` için gördüğümüz “yerleşik adı gölgeleme” hatasının liste hâli. Liste adlarını **çoğul**, döngü değişkenini **tekil** seçin: `for score in scores:` satırı böylece kendi kendini açıklar.



GitHub

13.2 İndeksleme ve dilimleme

Beşinci haftada bir metnin karakterlerine indeksle erişmişsiniz; liste için kurallar *birebir* aynıdır, hücrelerde karakter yerine herhangi bir nesne durur.

Tanım 13.2: Liste indeksi ve dilim

n elemanlı bir listede geçerli indeksler pozitif yönde $0, 1, \dots, n - 1$, negatif yönde $-n, \dots, -1$ 'dir; `items[-1]` son elemandır ve aralık dışı bir indeks `IndexError` verir. `items[a:b:c]` yazımı, a dahil b hariç olmak üzere her c . elemandan oluşan **yeni** bir liste üretir; `items[::-1]` listenin tersi, `items[:]` ise tam bir kopyasıdır.

`scores = [72, 85, 91, 58, 77]`

`len(scores) = 5` hücre

pozitif indeks	0	1	2	3	4	baştan, 0'dan
	72	85	91	58	77	
negatif indeks	-5	-4	-3	-2	-1	sondan, -1'den

```
scores[0] → 72
scores[4] → 77 # len(scores) - 1
scores[-1] → 77 # sonuncu
scores[-5] → 72
scores[2] = 100 # HÜCRE DEĞİŞİR
scores[5] → IndexError # 5. hücre yok
```

Liste, sıralı ve indeksli hücrelerden oluşur; numaralandırma 0'dan başlar. n elemanlı bir listede pozitif indeksler $0 \dots n - 1$, negatif indeksler $-n \dots -1$ 'dir; $-k$ ile $n - k$ aynı hücreyi gösterir. Metinden tek farkı şudur: metinde hücrenin içeriği *okunur*, listede *değiştirilebilir* de — `scores[2] = 100` geçerli bir deyimdir. Aralık dışı bir indeks `IndexError` verir.

Şekil 13.1: Liste, indeksli hücrelerden oluşur. Üstteki sayılar baştan, alttakiler sondan sayar. Metinden tek farkı, bir hücrenin içeriğinin değiştirilebilmesidir.

`scores = [72, 85, 91, 58, 77]`

`scores[1:4] → [85, 91, 58]` 1 dahil, 4 hariç; $4 - 1 = 3$ eleman

kesme noktası	0	1	2	3	4	5
	72	85	91	58	77	
negatif karşılığı	-5	-4	-3	-2	-1	

```
scores[:3] → [72, 85, 91] # ilk üç
scores[3:] → [58, 77] # 3'ten son
scores[-2:] → [58, 77] # son iki
scores[:-1] → [72, 85, 91, 58] # sonuncu
scores[:2] → [72, 91, 77] # adım 2
scores[::-1] → [77, 58, 91, 85, 72] # tersi
scores[2:99] → [91, 58, 77] # hata yok
scores[9:] → [] # boş liste
```

Dilim sayıları hücreleri değil, hücreler arasındaki kesme noktalarını gösterir; sonuç her zaman yeni bir listedir. `scores[a:b]` yarı açık aralıktır: a dahil, b hariç; uzunluğu $b - a$ 'dır. Bu yüzden `scores[:k] + scores[k:]` her k için listenin kendisini verir ve `scores[:]` listenin tam bir kopyasıdır. Şema 5. haftadaki metin dilimlemesinin aynıdır — tek fark, hücrelerde karakter yerine herhangi bir nesnenin durmasıdır.

Şekil 13.2: Dilim sayıları hücreleri değil, hücreler arasındaki kesme noktalarını gösterir; şema 5. haftadaki metin dilimlemesinin aynıdır.

Şekil 13.2'nin son iki satırı, dilimlemenin indekslemeden daha *bağışlayıcı* olduğunu gösterir: `scores[5]` hemen `IndexError` verirken `scores[2:99]` elde ne varsa onu, `scores[9:]` ise boş liste döndürür. Bu çoğu zaman işinize yarar, ama bir hatayı sessizce yutabilir; sonucun boş olup olmadığını kontrol etmek size düşer.

❌ Sık yapılan hata

`son = scores[len(scores)]` → `son = scores[len(scores) - 1]`

Beş elemanlı bir listenin son indeksi 4'tür. *Birim kayması* (İng. *off-by-one*) hatası bu hafta `IndexError: list index out of range` biçiminde karşınıza çıkar; en kolay korunma yolu `scores[-1]` yazmaktır. Dilimlemenin bu haftaki en önemli kullanımı ise `kopya = liste[:]` kalıbıdır; 13.6'da neden hayati olduğunu göreceksiniz.

13.3 Değiştirilebilirlik: listeyi yerinde değiştirmek

Tanım 13.3: Değiştirilebilirlik

Bir nesne kurulduktan sonra içeriği değiştirilebiliyorsa **değiştirilebilir** (İng. *mutable*), değiştirilemiyorsa **değiştirilemez** (İng. *immutable*) sayılır. Metinler, sayılar ve mantıksal değerler değiştirilemezdir: `s[0] = "X"` hata verir, metin üzerindeki her işlem *yeni* bir metin üretir. Listeler değiştirilebilirdir: `scores[0] = 100` var olan nesnenin bir hücreğini yerinde günceller.

deyim	listenin yeni durumu	deyimin değeri
	0 1 2 3 4	
① <code>items = [...]</code>	süt çay bal	—
② <code>items.append("un")</code>	süt çay bal un	<code>None</code>
③ <code>items.insert(1, "tuz")</code>	süt tuz çay bal un	<code>None</code>
④ <code>last = items.pop()</code>	süt tuz çay bal un	<code>'un'</code>
⑤ <code>items.remove("çay")</code>	süt tuz bal	<code>None</code>

Liste metotlarının çoğu listeyi *yerinde* değiştirir ve geriye `None` döndürür; bu yüzden `items = items.append("un")` yazmak listeyi yok eder. Yalnız `pop()` bir değer döndürür — çıkardığı elemanı. `remove` değere, `pop` ve `del` indekse göre siler; `insert` araya sokar ve sağındaki elemanların indeksini bir artırır.

Şekil 13.3: `append`, `insert`, `pop` ve `remove` listeyi yerinde değiştirir. Sağ sütun her deyimin geriye ne döndürdüğünü verir: `pop` dışındakiler `None` döndürür.

Kural 13.1 (Yerinde değiştiren metotlar `None` döndürür). `append`, `insert`, `extend`, `remove`, `sort`, `reverse` ve `clear` listeyi *yerinde* değiştirir ve `None` döndürür; sonuçlarını bir ada atamak listeyi kaybetmek demektir. Yalnız `pop()` bir değer döndürür: çıkardığı elemanı.

Kabukta `names.append("Can")` yazdığınızda ekranda hiçbir şey görünmez — dönen değer `None`'dir ve kabuk onu göstermez; `print(names.append("Can"))` yazınca ortaya çıkar. `names = names.sort()` ise **Kural 13.1**'in neden kural olarak yazıldığını gösterir: liste sıralanır, sonra `names` adına `None` atanır ve sıralı liste erişilemez olur.

❌ Sık yapılan hata

`scores = scores.append(95)` → `scores.append(95)`

Bu haftanın en sık hatası. Doğru yazımda atama *yoktur*. Yanlış yazımdan sonra `len(scores)` satırı `TypeError: object of type 'NoneType' has no len()` verir ve öğrenciler hatayı `len` satırında arar — oysa hata bir önceki satırdadır.

Kod 13.2 w13-02-degistir.py — Eleman atama, ekleme, silme ve dilime atama

```

1 # w13-02-degistir.py -- Liste değiştirilebilir: eleman atama ve metotlar
2
3 shopping = ["süt", "ekmek", "yumurta"]
4 shopping[1] = "kepekli ekmek" # eleman atama: o yerin içeriği değişir
5 print("atama :", shopping)
6
7 shopping.append("peynir") # sona ekle
8 shopping.insert(0, "su") # indeksle ekle, gerisini sağa kaydır
9 shopping.extend(["çay", "şeker"]) # başka listenin elemanlarını ekle
10 print("ekleme:", shopping)
11
12 shopping.remove("yumurta") # DEĞERE göre sil (ilk eşleşen)
13 last = shopping.pop() # SON elemanı çıkar ve döndür
14 first = shopping.pop(0) # indeksteki çıkar ve döndür
15 del shopping[1] # indeksteki sil; değer döndürmez
16 print("silme :", shopping, "| pop:", last, first)
17
18 shopping[0:2] = ["kahve", "fındık", "bal"] # 2 yere 3 eleman
19 print("dilim :", shopping, len(shopping))
20
21 shopping.clear() # hepsini sil; liste nesnesi boşalır
22 print("clear :", shopping, len(shopping))

```

\$ python w13-02-degistir.py

```

atama : ['süt', 'kepekli ekmek', 'yumurta']
ekleme: ['su', 'süt', 'kepekli ekmek', 'yumurta', 'peynir', 'çay', 'şeker']
silme : ['süt', 'peynir', 'çay'] | pop: şeker su
dilim : ['kahve', 'fındık', 'bal', 'çay'] 4
clear : [] 0

```

Kod 13.2’de bir *dilime* atama da yapılabildiğini görüyorsunuz: iki hücrenin yerine üç eleman kondu ve liste bir uzadı. `del` ise atama değil bir deyimdir: `del shopping[1]` indeksteki elemanı, `del shopping[1:3]` bir dilimi siler.



GitHub

Tablo 13.1: Listeyi değiştiren işlemler ve sorgular. “Döndürür” sütunu [Kural 13.1](#)’in özetidir; son iki satırın örneğinde `rolls = [4, 6, 2, 6, 3, 6, 1, 2]` ve `v = 6`.

İşlem	Ne yapar	Döndürür	Yoksa
<code>s[i] = v</code>	<i>i</i> . hücreyi değiştirir	—	<code>IndexError</code>
<code>s.append(v)</code>	sona bir eleman ekler	<code>None</code>	—
<code>s.insert(i, v)</code>	<i>i</i> . sıraya ekler, gerisini kaydırır	<code>None</code>	sona ekler
<code>s.extend(t)</code>	<i>t</i> ’nin elemanlarını ekler	<code>None</code>	—
<code>s.remove(v)</code>	<i>v</i> ’yi ilk bulduğu yerden siler	<code>None</code>	<code>ValueError</code>
<code>s.pop()</code> / <code>s.pop(i)</code>	elemanı listeden çıkarır	eleman	<code>IndexError</code>
<code>del s[i]</code> / <code>s.clear()</code>	siler / listeyi boşaltır	—	<code>IndexError</code>
<code>s.index(v)</code> / <code>s.count(v)</code>	ilk sıra: 1 / tekrar: 3	sayı	<code>ValueError</code> / 0

⚠ Dikkat

`remove` değere, `pop` ve `del` *indekse* göre siler: `marks.remove(0)` “0 değerini sil” demektir, “ilk elemanı sil” demek değil — onun için `marks.pop(0)` ya da `del marks[0]` yazılır.

13.4 Sorgulama, sıralama ve toplu işlevler

Bir listeyi değiştirmeden de çok şey sorabilirsiniz ([Tablo 13.1](#)); sıralama ise iki ayrı biçimde gelir ve bu ayrım sürekli karşınıza çıkar.

Kod 13.3 w13-03-sorgula-sirala.py — Üyelik, arama, sayma ve sıralama

```

1 # w13-03-sorgula-sirala.py -- Üyelik, arama, sayma, sıralama
2
3 rolls = [4, 6, 2, 6, 3, 6, 1, 2]          # 8 zar atışı
4
5 print("6 var mı:", 6 in rolls, "| ilk 6:", rolls.index(6))
6 print("kaç tane 6:", rolls.count(6), "| eleman:", len(rolls))
7 print("min:", min(rolls), "max:", max(rolls), "toplam:", sum(rolls))
8
9 print("sorted():", sorted(rolls), "| özgün:", rolls)
10 rolls.sort()                          # YERİNDE sıralar; None döndürür
11 print("sort() :", rolls)
12 rolls.sort(reverse=True)              # azalan sıra
13 print("azalan :", rolls)
14 rolls.reverse()                       # ters çevirir, sıralamaz
15 print("reverse():", rolls)
16
17 names = ["Zeynep", "ali", "Mert", "ayşe"] # key= ile harf büyüklüğü
18 print(sorted(names), sorted(names, key=str.lower))

```

```
$ python w13-03-sorgula-sirala.py
```

```

6 var mı: True | ilk 6: 1
kaç tane 6: 3 | eleman: 8
min: 1 max: 6 toplam: 30
sorted(): [1, 2, 2, 3, 4, 6, 6, 6] | özgün: [4, 6, 2, 6, 3, 6, 1, 2]
sort() : [1, 2, 2, 3, 4, 6, 6, 6]
azalan : [6, 6, 6, 4, 3, 2, 2, 1]
reverse(): [1, 2, 2, 3, 4, 6, 6, 6]

```



GitHub

```
['Mert', 'Zeynep', 'ali', 'ayşe'] ['ali', 'ayşe', 'Mert', 'Zeynep']
```



Dikkat

`sorted(liste)` yeni bir sıralı liste üretir, özgün listeye dokunmaz — özgün sıra önemliyse bunu kullanın. `liste.sort()` ise yerinde sıralar ve `None` döndürür — özgün sıra gerekmiyorsa ve liste büyükse bunu kullanın. Aynı ayırım `liste.reverse()` ile `liste[::-1]` arasında da vardır. İkisi de on ikinci haftanın `key` parametresini kabul eder: `sorted(names, key=str.lower)` harf büyüklüğünü yok sayar; `reverse=True` azalan sıra verir.



İpucu

Kod 13.3'deki gibi bir listeyi hem sıralı hem de özgün sırasıyla kullanacaksanız, sıralı hâli ayrı bir ada verin: `ordered = sorted(scores)`. Böylece `scores` girildiği sırayı, `ordered` ise sıralı hâli tutar — “hangisi neydi?” sorusu hiç doğmaz.

Örnek 13.1 (En büyük iki eleman). Bir not listesinin en yüksek ve *ikinci* en yüksek elemanını, listeyi sıralamadan ve yalnız **bir kez** dolaşarak bulan bir program yazın. Örnek liste: [67, 89, 45, 92, 78, 83].

Çözüm

Analiz. Girdi: en az iki elemanlı sayı listesi. Çıktı: iki sayı. Kısıt: tek geçiş. Sekizinci haftanın “en büyüğü bulma” biriktiricisi artık *iki* biriktirici taşıyacak.

Algoritma. `first` ilk elemanla, `second` “yok” ile başlar. İkinciden itibaren her eleman için: (i) eleman `first`'ten büyükse eski `first` *ikinciliğe* iner ve eleman birinci olur; (ii) değilse ama `second`'dan büyükse yalnız `second` güncellenir. Birinci madde-deki devir satırı unutulursa program ikinci en büyüğü kaçırmaz — bu algoritmanın tek kritik noktası budur.

Kod ve çıktı.

Kod 13.4 w13-05-en-buyuk-iki.py — Tek geçişte en büyük iki eleman

```
1 # w13-05-en-buyuk-iki.py -- Tek geçişte en büyük iki elemanı bulma
2
3 scores = [67, 89, 45, 92, 78, 83]
4
5 first = scores[0]           # şimdiye kadarki en büyük
6 second = None              # ikinci en büyük: henüz yok
7 for s in scores[1:]:       # ilk eleman zaten first'te
8     if s > first:
9         second = first      # eski birinci ikinciliğe iner
10        first = s
11    elif second is None or s > second:
12        second = s
13 print("En büyük:", first, "İkinci:", second)
14
15 ordered = sorted(scores)    # kontrol: sondan iki eleman aynı mı?
16 print("Sıralı:", ordered, "|", ordered[-1], ordered[-2])
```



GitHub

```
$ python w13-05-en-buyuk-iki.py
```

```
En büyük: 92 İkinci: 89
Sıralı: [45, 67, 78, 83, 89, 92] | 92 89
```

Kontrol. Elle: en büyük 92; onu çıkarınca kalanın en büyüğü 89 ✓. Program aynı kontrolü `sorted(scores)` ile de yapar: `[45, 67, 78, 83, 89, 92]` listesinin son iki elemanı 92 ve 89’dur. Uç durum: bütün elemanlar eşitse `([5, 5, 5])` birinci de ikinci de 5 olur — “ikinci en büyük” ile “ikinci *farklı* en büyük” aynı şey değildir.

13.5 Liste üzerinde döngü ve liste üretici

Onuncu haftada `for` döngüsünü metin üzerinde kullanmıştınız; listede de aynı biçimde çalışır. Ama listelerde üç ayrı biçim vardır ve seçim, döngüde ne yapacağımıza bağlıdır.

Liste üzerinde döngünün üç biçimi

1. **Yalnız değer gerekiyorsa:** `for s in scores:` — en okunaklısı; toplama, sayma, karşılaştırma işlerinde bunu kullanın.
2. **Sıra numarası da gerekiyorsa:** `for i, s in enumerate(scores, start=1):` — numaralı liste basmak için.
3. **Elemanı değiştirmek gerekiyorsa:** `for i in range(len(scores)):` ve gövdede `scores[i] = ...` — yalnız bu biçimde listeyi yerinde güncelleyebilirsiniz.

Örnek 13.2 (Nöbetçi değerle not listesi). Notları `-1` girilene kadar tek tek alan; kalan notları tek satırda boşlukla ayrılmış olarak da kabul eden; ortalamayı yazan, notları numaralandıran ve herkese 5 puan ekleyen (ama 100’ü aşmayan) bir program yazın.

Çözüm

Analiz. Girdi iki biçimde gelir: nöbetçi değerle biten döngü (8. hafta) ve bir satırda birden çok sayı (`split`, 5. hafta); ikisi de aynı listede birikir. Üç döngü biçiminin üçü de kullanılacak.

Algoritma. Boş liste aç → nöbetçi değere kadar `append` ile doldur → okunan satırı `split` ile parçala ve her parçayı `int`’e çevirip ekle → toplamı elemanlar üzerinde hesapla → `enumerate` ile numaralı bas → `range(len(...))` ile yerinde güncelle.

Kod ve çıktı.


Kod 13.5 w13-04-dongu-girdi.py — Nöbetçi değer, split ve üç döngü biçimi

```

1 # w13-04-dongu-girdi.py -- Listeyi girdiyle doldurma ve üç döngü biçimi
2
3 scores = []
4 while True:                                # nöbetçi değer: -1 girilince dur
5     value = int(input("Not (-1 = bitir): "))
6     if value == -1:
7         break
8     scores.append(value)
9
10 for piece in input("Kalan notlar: ").split(): # split parçalara
11     ↪ ayırır
12     scores.append(int(piece))
13
14 print("Liste:", scores, "| eleman:", len(scores))
15
16 total = 0                                  # 1) yalnız DEĞER gerekiyorsa
17 for s in scores:
18     total += s
19
20 print(f"Ortalama: {total / len(scores):.1f}")
21
22 for i, s in enumerate(scores, start=1):    # 2) SIRA NUMARASI
23     ↪ gerekiyorsa
24     print(f" {i}. öğrenci: {s}")
25
26 for i in range(len(scores)):               # 3) elemanı DEĞİŞTİRMEK gerekiyorsa
27     scores[i] = min(scores[i] + 5, 100)
28
29 print("Güncel:", scores)

```

```
$ python w13-04-dongu-girdi.py
```

```

Not (-1 = bitir): 72
Not (-1 = bitir): 85
Not (-1 = bitir): -1
Kalan notlar: 58 91
Liste: [72, 85, 58, 91] | eleman: 4
Ortalama: 76.5
1. öğrenci: 72
2. öğrenci: 85
3. öğrenci: 58
4. öğrenci: 91
Güncel: [77, 90, 63, 96]

```

Kontrol. Girilen notlar 72, 85, 58, 91; toplamı 306 ve $306/4 = 76,5$ ✓. `min(..., 100)` koruması bu girdide devreye girmez (en yüksek not $91 + 5 = 96$); 97 gibi bir not girilseydi 102 yerine 100 yazılırdı. Uç durum: kullanıcı hemen `-1` girip ikinci satırı boş bırakırsa liste boş kalır ve `total / len(scores)` sıfıra bölme hatası verir.

Sık yapılan hata

`for s in scores: s = s + 5` → `for i in range(len(scores)): scores[i] += 5`

Birinci biçimde `s`, hücrenin kendisi değil hücredeki değere takılan geçici bir addır; ona atama yapmak listeyi değiştirmez, yalnız `s` adını bir sonraki turda üzerine yazılacak yeni

bir değere bağlar. Döngü bittiğinde liste hiç değişmemiştir.

Tanım 13.4: Liste üretici

Liste üretici (*İng. list comprehension*), bir diziden yeni liste üreten tek satırlık yazımdır:

[ifade for x in dizi] ve [ifade for x in dizi if koşul]

Birincisi her eleman için *ifade*yi hesaplar; ikincisi yalnız *koşulu* sağlayanları alır. Sonuç her zaman yeni bir listedir; kaynak dizi değişmez.

Kod 13.6 w13-08-uretec.py — Liste üretici, süzgeç ve split/join

```
1 # w13-08-uretec.py -- Liste üretici: döngüden tek satıra
2
3 scores = [72, 85, 91, 58, 77, 96, 45]
4
5 squares = [] # döngüyle: 1..5 sayılarının kareleri
6 for n in range(1, 6):
7     squares.append(n ** 2)
8 print("döngüyle:", squares)
9
10 # Aynı iş liste üreticiyle: [ifade for değişken in dizi]
11 print("üreteçle:", [n ** 2 for n in range(1, 6)])
12
13 # Süzgeçli biçim: [ifade for değişken in dizi if koşul]
14 print("Geçenler:", [s for s in scores if s >= 50])
15
16 # Dönüştürme: YENİ liste üretir, scores değişmez
17 print("Eklenmiş:", [min(s + 5, 100) for s in scores], "| özgün:", scores)
18
19 # True = 1 olduğu için koşulu sağlayanlar sum ile sayılır
20 print("80 ve üstü kaç kişi:", sum([s >= 80 for s in scores]))
21
22 # Metin ile liste arasında: split ayırır, join birleştirir
23 print("yapay zeka mühendisliği".split())
24 print(" | ".join([str(s) for s in scores]))
```

```
$ python w13-08-uretec.py
```

```
döngüyle: [1, 4, 9, 16, 25]
üreteçle: [1, 4, 9, 16, 25]
Geçenler: [72, 85, 91, 58, 77, 96]
Eklenmiş: [77, 90, 96, 63, 82, 100, 50] | özgün: [72, 85, 91, 58, 77, 96, 45]
80 ve üstü kaç kişi: 3
['yapay', 'zeka', 'mühendisliği']
72 | 85 | 91 | 58 | 77 | 96 | 45
```

Kod 13.6’te iki numara var. `sum([s >= 80 for s in scores])` bir koşulu sağlayanları *sayar* — dördüncü haftada “True aslında 1’dir” demiştik; çıktıda 80 ve üstü not sayısı 3’tür. `split` ve `join` ise metinle liste arasında gidiş dönüş sağlar; `join` yalnız *metin* elemanı kabul ettiği için sayı listesini önce `[str(s) for s in scores]` ile çevirmek gerekir.

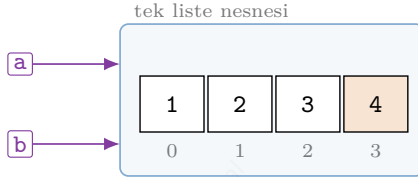
Üreteç kodu kısaltmak için değil *okunaklı* kılmak içindir: tek cümleyle anlatılabilen işler üreteçle yazılır, içinde birden çok koşul ya da uzun bir hesap varsa açık `for` döngüsü daha iyidir. Ölçü basit — bir satıra sığmıyorsa üreteç kullanmayın.

13.6 Takma ad, kopya ve listeyi fonksiyona geçirmek

Dördüncü haftada “değişken, bir kutu değil, bir nesneye takılan etikettir” demiş ve sonuçlarını 13. haftada göreceğimizi söylemiştik. O hafta geldi.

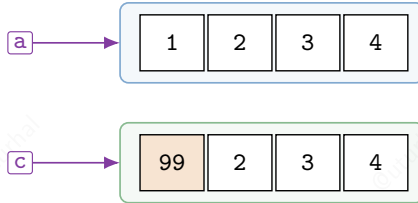
Kural 13.2 (Atama kopyalamaz). $b = a$ deyimi a 'nın gösterdiği nesnenin kopyasını üretmez; yalnız aynı nesneye ikinci bir ad takar. Bu iki ada **takma ad** (İng. *alias*) denir. Liste değiştirilebilir olduğu için b üzerinden yapılan her değişiklik a üzerinden de görünür. Gerçek kopya istiyorsanız açıkça isteyin: $a[:]$, $\text{list}(a)$ ya da $a.\text{copy}()$.

① $a = [1, 2, 3]$ $b = a$ $b.\text{append}(4)$



b 'nin eklediği eleman a 'da da görünür.

② $c = a[:]$ $c[0] = 99$



İki ayrı nesne: c 'deki değişiklik a 'ya geçmez.

```
a is b → True # aynı nesne
a == b → True # aynı içerik
a is c → False # ayrı nesne
a == c → False # içerik de ayrıldı
```

4. haftanın etiket modeli aynen geçerlidir: $b = a$ bir kopya değil, aynı nesneye takılan ikinci etikettir. Liste değiştirilebilir olduğu için bu fark artık görünür — sayı ve metinlerde fark edilmez, çünkü onlar yerinde değişmez. Kopya istiyorsanız açıkça isteyin: $a[:]$, $\text{list}(a)$ ya da $a.\text{copy}()$. is aynı nesne mi diye, == aynı içerik mi diye sorar; == doğruyken is yanlış olabilir.

Şekil 13.4: Takma ad ile kopya. Üstte iki ad tek nesneyi gösterir; altta iki ayrı nesne vardır. is aynı nesne mi diye, == aynı içerik mi diye sorar.

Kod 13.7 w13-06-takma-ad.py — Takma ad, kopya, is ve ==

```
1 # w13-06-takma-ad.py -- Takma ad (aynı nesne) ile kopya (ayrı nesne)
2
3 a = [1, 2, 3]
4 b = a # kopya DEĞİL: aynı nesneye takılan ikinci etiket
5 b.append(4)
6 print("a =", a, " b =", b, "| a is b:", a is b)
7
8 c = a[:] # dilimleme yeni liste üretir → gerçek kopya
9 d = list(a) # aynı iş; a.copy() de aynı
10 c[0] = 99
11 print("a =", a, " c =", c)
12 print("a is c:", a is c, "| a == c:", a == c, "| a == d:", a == d)
13
14 x = 5 # sayı değiştirilemez: takma ad sorunu çıkmaz
15 y = x
16 y += 1 # y YENİ bir int nesnesine bağlanır
17 print("x =", x, " y =", y)
```

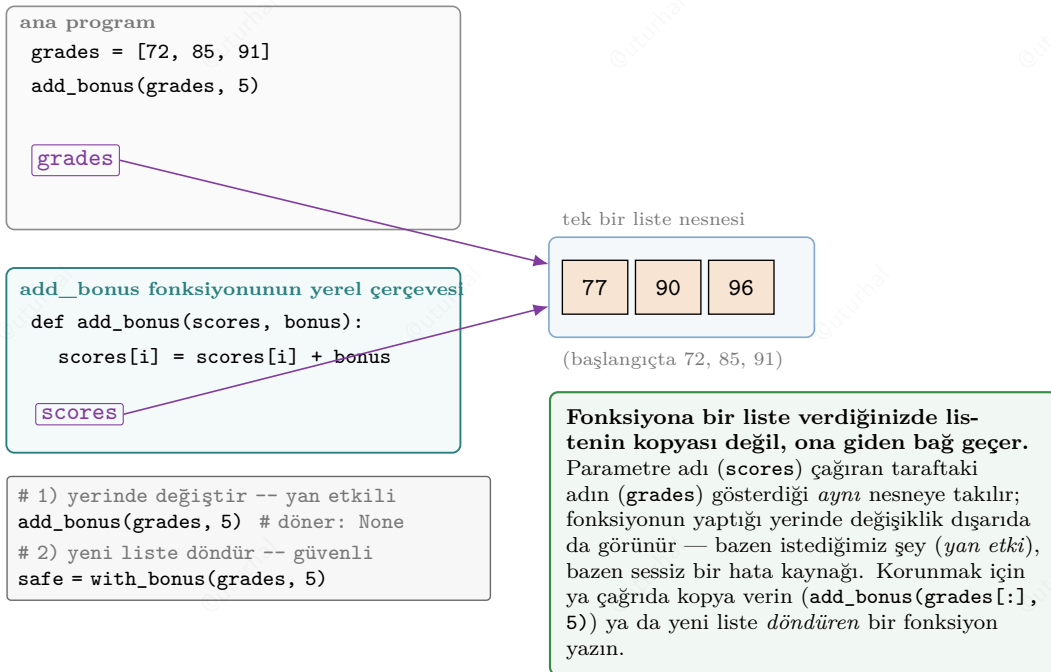


GitHub

```
$ python w13-06-takma-ad.py
a = [1, 2, 3, 4] b = [1, 2, 3, 4] | a is b: True
a = [1, 2, 3, 4] c = [99, 2, 3, 4]
a is c: False | a == c: False | a == d: True
x = 5 y = 6
```

Kod 13.7'in son üç satırı bu sorunun neden ancak şimdi ortaya çıktığını gösterir: $y = x$ sonrası $y += 1$ yazılsa bile x değişmez, çünkü sayılar değiştirilemezdir — $+=$ var olan nesneyi değiştirmez, adı yeni bir nesneye bağlar. Takma ad her zaman vardı; ancak değiştirilebilir bir nesnede görünür hâle gelir. Kopya almanın üç yolu (`a[:]`, `list(a)`, `a.copy()`) aynı işi yapar ve **sığ kopya** (İng. *shallow copy*) üretir: listenin kendisi yenidir, içindeki nesneler paylaşılır — bu ayrım 14. haftada iç içe listelerde önem kazanacak.

On birinci haftada bir fonksiyona sayı verdiğinizde fonksiyonun onu değiştirmesinin dışarıyı etkilemediğini görmüştünüz. Liste için durum farklıdır — ve nedeni yeni bir kural değil, aynı etiket modelidir (Şekil 13.5).



Şekil 13.5: Fonksiyona liste verildiğinde nesnenin kopyası değil, ona giden bağ geçer; parametre adı çağıran taraftaki adla aynı nesneyi gösterir.

Kod 13.8 w13-07-fonksiyon-liste.py — Listeyi değiştiren ve yeni liste döndüren iki fonksiyon

```
1 # w13-07-fonksiyon-liste.py -- Fonksiyona liste geçirmek
2
3 def add_bonus(scores, bonus):
4     """Listeyi YERİNDE değiştirir; değer döndürmez (yan etki)."""
5     for i in range(len(scores)):
6         scores[i] = min(scores[i] + bonus, 100)
7
8
9 def with_bonus(scores, bonus):
10    """Özgün listeye dokunmaz; YENİ liste döndürür."""
11    return [min(s + bonus, 100) for s in scores]
```



```

12
13
14 grades = [72, 85, 91, 58, 97]
15 print("add_bonus dönüşü:", add_bonus(grades, 5))    # None!
16 print("grades      :", grades)                       # değişti
17
18 safe = [72, 85, 91, 58, 97]
19 print("with_bonus dönüşü:", with_bonus(safe, 5))
20 print("safe      :", safe)                           # dokunulmadı

```

```
$ python w13-07-fonksiyon-liste.py
```

```

add_bonus dönüşü: None
grades           : [77, 90, 96, 63, 100]
with_bonus dönüşü: [77, 90, 96, 63, 100]
safe             : [72, 85, 91, 58, 97]

```

Kod 13.8'daki iki fonksiyon aynı işi iki felsefeyle yapar. `add_bonus` listeyi yerinde değiştirir: hızlıdır, bellek harcamaz, ama bir **yan etkisi** (*Ing. side effect*) vardır — çağıranın listesi habersizce değişir. `with_bonus` yeni liste döndürür: özgün veri korunur, bellekte ikinci liste açılır. İkisi de doğrudur; yanlış olan hangisini yazdığımızı belli etmemektir. Kural: yerinde değiştirenin adı bir *eylem*, yeni liste döndürenin adı bir *sonuç* olsun ve docstring'in ilk cümlesi bunu söylesin. Özgün listeyi korumanın en ucuz yolu ise kopyayı çağrıda vermektir: `add_bonus(grades[:], 5)`.

Dikkat

Kural 13.2 fonksiyon çağrılarında da geçerlidir; on ikinci haftanın varsayılan parametre değerlerinde ise listeler ayrıca özel bir tuzak taşır: `def ekle(x, liste=[]):` yazarsanız varsayılan liste *bir kez* oluşturulur ve bütün çağrılarda paylaşılır. Doğrusu varsayılanı `None` yapıp gövdede `if liste is None: liste = []` yazmaktır.

13.7 Arama ve sıralama algoritmaları

Üçüncü haftada bir telefon rehberinde ad aramanın iki yolunu — baştan sona bakmak ve rehberi ortadan açmak — kâğıt üzerinde karşılaştırmıştık. Artık ikisini de kodlayabiliriz; Şekil 13.6 aynı listede aynı hedefin iki yolla nasıl arandığını gösteriyor.

Doğrusal arama — hedef 57

	0	1	2	3	4	5	6	7	
	4	9	15	23	31	42	57	68	7 adım ✓
adım:	1	2	3	4	5	6	7		

İkili arama — aynı hedef, *sıralı* listede

1. adım	4	9	15	23	31	42	57	68	$23 < 57 \Rightarrow$ sağ yarı
2. adım					31	42	57	68	$42 < 57 \Rightarrow$ sağ yarı
3. adım							57	68	3 adım ✓ bulundu

Doğrusal arama her listede çalışır ama en kötü durumda n adım ister; **ikili arama** yalnız *sıralı* listede çalışır ve her adımda kalan aday sayısını yarıya indirir. Sekiz elemanda fark küçüktür (7'ye karşı 3); 100 000 elemanda doğrusal arama 100 000, ikili arama 17 adımdır — çünkü $2^{17} > 100\,000$. 3. haftada bunu telefon rehberini ortadan açarak anlatmıştık; şimdi kodu elimizde.

Şekil 13.6: Aynı listede aynı hedef: doğrusal arama yedi, ikili arama üç karşılaştırma yapar. Gri hücreler, ikili aramanın o adımda elemediği bölgedir.

Örnek 13.3 (Doğrusal arama ve ikili arama). Sıralı bir sayı listesinde hedef arayan iki fonksiyon yazın: biri baştan sona baksın, öteki her adımda listeyi ikiye bölsün. İkisi de indeksle birlikte kaç karşılaştırma yaptığını döndürsün ve $[4, 9, 15, 23, 31, 42, 57, 68]$ listesinde 57, 4 ve listede olmayan 50 için karşılaştırılsın.

Çözüm

Analiz. Doğrusal arama her listede çalışır ve en kötü durumda n karşılaştırma yapar. İkili arama yalnız *sıralı* listede doğru sonuç verir; her adımda kalan aday sayısını yarıya indirir. İki sınır (*low*, *high*) ve bir orta nokta (*mid*) yeterlidir.

Algoritma. İkili aramanın izleme tablosu, hedef 57 için:

adım	low	high	mid	items[mid]	karar
1	0	7	3	23	$23 < 57 \Rightarrow \text{low} = 4$
2	4	7	5	42	$42 < 57 \Rightarrow \text{low} = 6$
3	6	7	6	57	$57 = 57 \Rightarrow$ bulundu, indeks 6

Kod ve çıktı.



Kod 13.9 w13-09-arama.py — Doğrusal ve ikili arama; adım sayıları

```

1 # w13-09-arama.py -- Doğrusal arama ile ikili arama: adım
  ↪ karşılaştırması
2
3 def linear_search(items, target):
4     """Baştan sona bakar; (indeks, adım) döndürür, yoksa (-1, n)."""
5     for i in range(len(items)):
6         if items[i] == target:
7             return i, i + 1          # i + 1 = yapılan karşılaştırma
8             ↪ sayısı
9     return -1, len(items)
10
11 def binary_search(items, target):
12     """SIRALI listeyi ortadan böler; (indeks, adım) döndürür."""
13     low, high, steps = 0, len(items) - 1, 0
14     while low <= high:
15         mid = (low + high) // 2
16         steps += 1
17         if items[mid] == target:
18             return mid, steps
19         elif items[mid] < target:
20             low = mid + 1             # hedef sağ yarıda
21         else:
22             high = mid - 1            # hedef sol yarıda
23     return -1, steps
24
25
26 data = [4, 9, 15, 23, 31, 42, 57, 68]
27 for target in [57, 4, 50]:
28     i1, s1 = linear_search(data, target)
29     i2, s2 = binary_search(data, target)
30     print(f"hedef {target:>2}: doğrusal {i1:>2} ({s1} adım)"
31           f" | ikili {i2:>2} ({s2} adım)")
32
33 big = list(range(0, 200000, 2))    # 100 000 sıralı çift sayı
34 print("n =", len(big), "| doğrusal:", linear_search(big, 199998),
35       "| ikili:", binary_search(big, 199998))

```

```
$ python w13-09-arama.py
```

```

hedef 57: doğrusal  6 (7 adım) | ikili  6 (3 adım)
hedef  4: doğrusal  0 (1 adım) | ikili  0 (3 adım)
hedef 50: doğrusal -1 (8 adım) | ikili -1 (3 adım)
n = 100000 | doğrusal: (99999, 100000) | ikili: (99999, 17)

```

Kontrol. Tablo ile çıktı birebir uyuyor: 57 için ikili arama 3, doğrusal arama 7 adım. Listede olmayan 50 için doğrusal arama bütün listeyi tarar (8 adım), ikili arama yine 3 adımda “yok” der. İlk eleman olan 4 içinse durum tersine döner: doğrusal arama 1, ikili arama 3 adım harcar — üstünlük ölçeğe ortaya çıkar. 100 000 elemanlı listede doğrusal arama son elemanı 100 000 adımda, ikili arama 17 adımda bulur; çünkü her adım aday sayısını yarıya indirir ve $2^{17} = 131\,072 > 100\,000$ ✓.

Dikkat

İkili arama **sıralı olmayan** bir listede hata vermez — yanlış cevap verir: aradığı değer listede olduğu hâlde -1 döndürebilir. Çöküş yok, mesaj yok, yalnız sessiz bir yanlış. Bir fonksiyonun ön koşulunu docstring’e yazmanın ve on ikinci haftanın **assert**’ünün değeri buradadır.

Kabarcık sıralaması: [5, 1, 4, 2]

	0	1	2	3	
1. geçiş 5 > 1	5	1	4	2	takas ✓
5 > 4	1	5	4	2	takas ✓
5 > 2	1	4	5	2	takas ✓
2. geçiş 1 > 4?	1	4	2	5	hayır ✗
4 > 2	1	4	2	5	takas ✓
3. geçiş 1 > 2?	1	2	4	5	hayır ✗
sonuç: 6 karşılaştırma, 4 takas					

Her geçişte komşular karşılaştırılır; büyük olan bir sağa taşınır, böylece kalanın en büyüğü sona “kabarır” (yeşil hücreler). Bir sonraki geçiş bir eleman kısadır: $3 + 2 + 1 = 6$ karşılaştırma; n elemanda toplam $n(n - 1)/2$. Karşılaştırma sayısı listenin sırasından bağımsızdır; *takas* sayısı ise değişir — zaten sıralı bir listede sıfırdır.

Şekil 13.7: Kabarcık sıralamasının üç geçişi. Turuncu çerçeve o anda karşılaştırılan komşu çifti, yeşil hücreler yerine oturmuş kuyruğu gösterir.

Şekil 13.7, kabarcık sıralamasının dört elemanlı bir listede nasıl ilerlediğini karşılaştırma karşılaştırma gösteriyor; aşağıdaki örnek aynı fikri on elemanlı bir listede kodluyor ve ikinci bir algoritmayla karşılaştırıyor.

Örnek 13.4 (Kabarcık ve seçmeli sıralama). Bir listeyi `sort()` kullanmadan sıralayan iki fonksiyon yazın: kabarcık sıralaması komşuları karşılaştırıp takas etsin, seçmeli sıralama kalanın en küçüğünü başa taşınsın. İkisi de kaç karşılaştırma ve kaç takas yaptığını döndürsün; [64, 25, 12, 22, 11, 90, 45, 33, 78, 5] listesinde karşılaştırılsın.

Çözüm

Analiz. İkisi de iç içe iki döngü kullanır: dış döngü “kaçıncı geçiş”, iç döngü “bu geçişte nereye kadar”. Onuncu haftada iç içe döngünün n^2 adım demek olduğunu görmüştük; burada tam sayısını hesaplayacağız. İkisi de listeyi *yerinde* sıraladığı için çağırmadan önce kopya almak gerekir (13.6).

Algoritma. Kabarcık sıralamasının ilk dört geçişinde listenin durumu:

geçiş	geçiş sonundaki liste	takas
başlangıç	[64, 25, 12, 22, 11, 90, 45, 33, 78, 5]	—
1	[25, 12, 22, 11, 64, 45, 33, 78, 5, 90]	8
2	[12, 22, 11, 25, 45, 33, 64, 5, 78, 90]	6
3	[12, 11, 22, 25, 33, 45, 5, 64, 78, 90]	3
4	[11, 12, 22, 25, 33, 5, 45, 64, 78, 90]	2

Her geçişte kalanın en büyüğü sona “kabarır”: önce 90, sonra 78, sonra 64 yerine oturur.

Kod ve çıktı.

Kod 13.10 w13-10-siralama.py — Kabarcık ve seçmeli sıralama; karşılaştırma ve takas sayıları

```

1 # w13-10-siralama.py -- Kabarcık ve seçmeli sıralama; adım sayıları
2
3 def bubble_sort(items):
4     """Komşuları karşılaştırıp ters sıradaysa takas eder; yerinde."""
5     comparisons = swaps = 0
6     for i in range(len(items) - 1):          # geçiş numarası
7         for j in range(len(items) - 1 - i):  # sondaki i eleman yerinde
8             comparisons += 1
9             if items[j] > items[j + 1]:
10                 items[j], items[j + 1] = items[j + 1], items[j]
11                 swaps += 1
12     return comparisons, swaps
13
14
15 def selection_sort(items):
16     """Kalan kısmın en küçüğünü bulup başa taşır; yerinde."""
17     comparisons = swaps = 0
18     for i in range(len(items) - 1):
19         smallest = i
20         for j in range(i + 1, len(items)):
21             comparisons += 1
22             if items[j] < items[smallest]:
23                 smallest = j
24         if smallest != i:
25             items[i], items[smallest] = items[smallest], items[i]
26             swaps += 1
27     return comparisons, swaps
28
29
30 data = [64, 25, 12, 22, 11, 90, 45, 33, 78, 5]
31 a, b = data[:], data[:]          # kopyalar: özgün liste bozulmasın
32 c1, s1 = bubble_sort(a)
33 c2, s2 = selection_sort(b)
34 print("kabarcık:", a, f"| {c1} karşılaştırma, {s1} takas")
35 print("seçmeli :", b, f"| {c2} karşılaştırma, {s2} takas")
36 print("Kontrol :", a == b == sorted(data), "| özgün:", data)

```

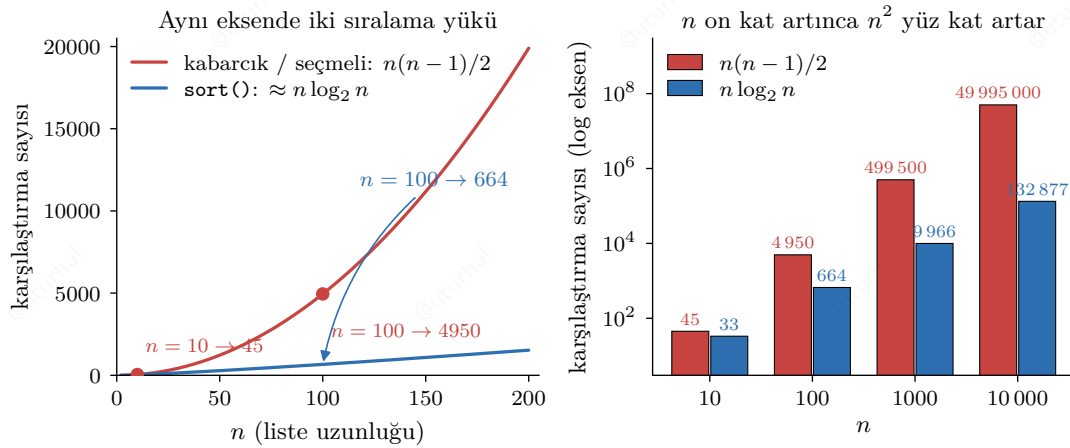


GitHub

```
$ python w13-10-siralama.py
```

```
kabarcık: [5, 11, 12, 22, 25, 33, 45, 64, 78, 90] | 45 karşılaştırma, 24
↳ takas
seçmeli : [5, 11, 12, 22, 25, 33, 45, 64, 78, 90] | 45 karşılaştırma, 4
↳ takas
Kontrol : True | özgün: [64, 25, 12, 22, 11, 90, 45, 33, 78, 5]
```

Kontrol. İki algoritma da aynı sıralı listeyi üretir ve son satır bunu `a == b == sorted(data)` ile doğrular ✓. Karşılaştırma sayısı ikisinde de 45'tir; bu tesadüf değildir: $9 + 8 + \dots + 1 = 10 \cdot 9/2 = 45$. Takas sayıları ayrışır: kabarcık 24, seçmeli 4 takas yapar — seçmeli sıralama her geçişte en çok bir takas yaptığı için takas pahalıysa tercih edilir. Son satırdaki `data` çıktısı özgün listenin hiç bozulmadığını da gösterir; kopya almasaydık ikinci algoritma zaten sıralı bir liste alacaktı.



Şekil 13.8: Kabarcık ve seçmeli sıralamanın karşılaştırma sayısı $n(n-1)/2$, Python'un yerleşik `sort()` işlevinininki ise kabaca $n \log_2 n$ ile artar.

Şekil 13.8 bu haftanın verimlilik dersini tek resimde toplar. On elemanda 45'e karşı 33: fark yok denecek kadar az. Yüz elemanda 4950'ye karşı 664; bin elemanda 499500'e karşı 9966; on bin elemanda 49995000'e karşı 132877. Liste on kat uzadığında kendi algoritmalarımızın işi yaklaşık *yüz* kat, `sort()`'unki yaklaşık on üç kat artar. Kendi sıralama algoritmanızı yazmak öğrenmek içindir; gerçek işte `sorted()` kullanın.

13.8 Uygulamalar

Örnek 13.5 (Not listesi istatistikleri). Bir sınavın notlarını tutan listenin ortalamasını, medyanını ve modunu *elle* hesaplayan; sonuçları `statistics` modülünün verdikleriyle yan yana yazan; son olarak harf notu dağılımını bir sayaç listesiyle çıkaran bir program yazın. Liste: [72, 85, 91, 58, 77, 85, 64, 96].

Çözüm

Analiz. Ortalama `sum/len`'dir. Medyan için önce `sorted` gerekir; eleman sayısı çift olduğundan ortadaki *iki* elemanın ortalaması alınır. Mod, `count` ile en çok tekrar edenini bulmaktır. Harf dağılımı için sözlüğe gerek yok: beş elemanlı bir *sayaç listesi*

(counts = [0] * 5) yeterlidir.

Algoritma. Sıralı kopya al → ortalama → medyan → mod → eşik listesiyle sayaçları doldur → yıldızla histogram bas.

Kod ve çıktı.

Kod 13.11 w13-11-not-istatistik.py — Ortalama, medyan, mod ve frekans

```

1 # w13-11-not-istatistik.py -- Not listesi: istatistikler ve frekans
2 import statistics
3
4 scores = [72, 85, 91, 58, 77, 85, 64, 96]
5 n = len(scores)
6 mean = sum(scores) / n
7
8 ordered = sorted(scores)          # medyan: sıralı listenin ortası
9 if n % 2 == 1:
10     median = ordered[n // 2]
11 else:
12     median = (ordered[n // 2 - 1] + ordered[n // 2]) / 2
13
14 mode = scores[0]                  # mod: en çok tekrar eden değer
15 for s in scores:
16     if scores.count(s) > scores.count(mode):
17         mode = s
18
19 print(f"{' ':10}{'elle':>8}{'statistics':>12}")
20 print(f"{'Ortalama':10}{mean:>8.2f}{statistics.mean(scores):>12.2f}")
21 print(f"{'Medyan':10}{median:>8.2f}{statistics.median(scores):>12.2f}")
22 print(f"{'Mod':10}{mode:>8}{statistics.mode(scores):>12}")
23 above = [s for s in scores if s > mean]
24 print(f"Ortalama üstü: {above} ({len(above)} öğrenci)")
25
26 labels = ["A", "B", "C", "D", "F"]  # frekans: sayaç listesi (sözlük
    ↪ yok)
27 limits = [90, 80, 70, 60, 0]
28 counts = [0] * 5
29 for s in scores:
30     for k in range(5):
31         if s >= limits[k]:
32             counts[k] += 1
33             break
34 for k in range(5):
35     print(f" {labels[k]} {'*' * counts[k]:<6}{counts[k]}")
36 print("Toplam:", sum(counts))

```



```
$ python w13-11-not-istatistik.py
```

	elle	statistics
Ortalama	78.50	78.50
Medyan	81.00	81.00
Mod	85	85

Ortalama üstü: [85, 91, 85, 96] (4 öğrenci)

A ** 2
B ** 2
C ** 2
D * 1
F * 1

Toplam: 8

Kontrol. Elle: toplam 628, $628/8 = 78,5$ ✓. Sıralı liste [58, 64, 72, 77, 85, 85, 91, 96]; ortadaki iki eleman 77 ve 85, ortalamaları 81,0 ✓. 85 iki kez, diğerleri birer kez geçtiği için mod 85'tir; üç değer de **statistics** sonuçlarıyla uyuşması bağımsız bir doğrulamadır. Harf dağılımı $2 + 2 + 2 + 1 + 1 = 8$ verir — toplamın eleman sayısına eşit olması hiçbir notun iki kez ya da sıfır kez sayılmadığını gösterir; sayaç listeleriyle çalışırken her zaman bu toplamı kontrol edin. Ortalamanın üstünde 85, 91, 85, 96 olmak üzere 4 öğrenci vardır.

Not

Medyan, ortalamadan farklı olarak uç değerlerden etkilenmez: [30, 32, 35, 31, 300] listesinin ortalaması 85,6, medyanı 32'dir. Maaş, ev fiyatı ve süre gibi çarpık dağılan verilerde haberlerde “ortanca” sözcüğünü bu yüzden duyarsınız.

Bu haftanın ikinci uygulaması, yedinci haftada tek seçimlik, sekizinci haftada döngülü yazdığınız menünün arkasına bir liste koyar: dört seçenek dört liste işlemine karşılık gelir. İki koruma gerekir — eklemekten önce **name** **in** **items** sorgusu (aynı ürün iki kez girilmesin) ve silmeden önce yine aynı sorgu, çünkü **remove** olmayan bir değer için **ValueError** verir.

Kod 13.12 w13-12-alisveris.py — Menü döngüsü ve liste işlemleri

```
1 # w13-12-alisveris.py -- Alışveriş listesi: menü döngüsü + liste metotları
2
3
4 def show(items):
5     """Listeyi numaralı satırlar hâlinde yazar."""
6     if len(items) == 0:
7         print(" (liste boş)")
8     for i, item in enumerate(items, start=1):
9         print(f" {i}. {item}")
10
11
12 items = []
13 while True:
14     choice = input("[1] Ekle [2] Sil [3] Listele [4] Çık > ")
15     if choice == "1":
16         name = input(" Ürün: ")
17         if name in items:
18             print(" Zaten listede.")
19         else:
```



GitHub

```

20         items.append(name)
21     elif choice == "2":
22         name = input(" Silinecek ürün: ")
23         if name in items:
24             items.remove(name)
25         else:
26             print(" Listede yok:", name)
27     elif choice == "3":
28         show(items)
29     elif choice == "4":
30         print("Son liste:", items, "| ürün sayısı:", len(items))
31         break
32     else:
33         print(" Geçersiz seçim.")

```

```
$ python w13-12-alisveris.py
```

```

[1] Ekle [2] Sil [3] Listele [4] Çık > 1
  Ürün: süt
[1] Ekle [2] Sil [3] Listele [4] Çık > 1
  Ürün: ekmek
[1] Ekle [2] Sil [3] Listele [4] Çık > 1
  Ürün: yumurta
[1] Ekle [2] Sil [3] Listele [4] Çık > 1
  Ürün: süt
  Zaten listede.
[1] Ekle [2] Sil [3] Listele [4] Çık > 2
  Silinecek ürün: ekmek
[1] Ekle [2] Sil [3] Listele [4] Çık > 2
  Silinecek ürün: peynir
  Listede yok: peynir
[1] Ekle [2] Sil [3] Listele [4] Çık > 3
  1. süt
  2. yumurta
[1] Ekle [2] Sil [3] Listele [4] Çık > 4
Son liste: ['süt', 'yumurta'] | ürün sayısı: 2

```

Kod 13.12'nün oturumunda üç ürün eklendi, dördüncü ekleme (*süt*) yinelenen olduğu için reddedildi, *ekmek* silindi, olmayan *peynir*'i silme denemesi programı çökertmeden uyarı verdi ve çıkışta iki ürün kaldı. `show` içindeki boş liste kontrolü de gereklidir: `enumerate` boş liste üzerinde hiç dönmeyiz.

LAB Laboratuvar Çalışması 13: Not defteri ve algoritmalar

Bu haftanın laboratuvarı dönemin ilk “gerçek” uygulamasını kurar: veriyi saklayan, üzerinde işlem yapan ve menüyle yönetilen bir program. Görev 1–3 herkes içindir; 4–5 erken bitirenler için.

Görev 1. Not defteri (menülü). `not_defteri.py`: bir `scores` listesi ve beş seçenek — not ekle, indekse göre sil, numaralı listele, istatistik (eleman sayısı, en küçük, en büyük, ortalama, medyan), çık. Boş listede istatistik istenirse “önce not ekleyin” yazsın, çökmesin; notu 0–100 aralığında doğrulayın (8. haftanın girdi doğrulama döngüsü).

Görev 2. Alışveriş listesini genişletin. Kod 13.12'ye iki seçenek ekleyin: alfabetik sıralama (`sort`, yerinde) ve listeyi virgülle ayrılmış tek satır olarak yazma

(join). Sıralamadan önce özgün sırayı bir kopyada saklayıp geri yükleyebilirsiniz — kopyayı `items[:]` ile alın.

Görev 3. Arama karşılaştırması. Kod 13.9'daki iki fonksiyonu kendi dosyanıza alın ve `list(range(1000))` üzerinde 0, 499, 999 ve listede olmayan 1000 hedefleri için adım sayılarını tablo hâlinde yazdırın. Hangi hedefte doğrusal arama kazanır, neden?

Görev 4. Sıralama karşılaştırması. Kod 13.10'yı alın; `random` ile (`random.seed(2026)`) 20 sayılık bir liste üretin ve iki algoritmayı listenin kopyaları üzerinde çalıştırın. Sonra aynı işi *sıralı* liste üzerinde tekrarlayın: hangi sayı değişti, hangisi değişmedi?

Görev 5. Takma ad avı. Şunu yazıp çalıştırın ve çıktının neden beklediğiniz gibi olmadığını bulup tek satır değiştirerek düzeltin: `ilk = [10, 20, 30], yedek = ilk, ilk.append(40), print(yedek)`. Sonra `ilk` ile `yedek` için `is` ve `==` sonuçlarını da yazdırın.

Teslim. Görev 1–3 dosyalarını ders bitmeden çalıştırıp gösterin ve Görev 1'i saklayın: 14. haftada her nota bir ad ekleyecek, 15. haftada notları bir dosyaya yazacaksınız.

Özet

Yapı	Yazım	Not
Oluşturma	<code>[], [1, 2, 3], list(range(5)), [0] * n</code>	son biçim sayaç listesi için
İndeks	<code>s[0], s[-1], s[i] = v, len(s)</code>	metinden farkı: atama yapılabilir
Dilim	<code>s[a:b], s[:k], s[-k:], s[::-1]</code>	<i>a</i> dahil, <i>b</i> hariç; yeni liste
Ekleme, silme	<code>s.append(v), s.insert(i, v), s.extend(t), s.remove(v), s.pop(i), del s[i]</code>	<code>pop</code> dışında hepsi <code>None</code> ; <code>remove</code> gere, ötekiler indekse göre
Sorgu, toplu	<code>v in s, s.index(v), s.count(v), min, max, sum</code>	<code>index</code> yoksa <code>ValueError</code>
Sıralama	<code>s.sort()</code> yerinde; <code>sorted(s)</code> yeni	<code>key=</code> , <code>reverse=True</code>
Döngü	<code>for x in s / enumerate(s) / range(len(s))</code>	değiştirmek için üçüncüsü
Üreteç	<code>[f(x) for x in s if k(x)]</code>	sonuç yeni liste
Kopya	<code>b = a takma ad; a[:], list(a), a.copy()</code>	<code>is</code> nesne, <code>==</code> içerik
Metin↔liste	<code>"a b".split(), " ".join(parts)</code>	<code>join</code> yalnız metin alır
Arama, sıralama	doğrusal n , ikili $\log_2 n$ adım; kabarcık / seçmeli $n(n-1)/2$ karşılaştırma	ikili arama sıralı liste ister; gerçek işte <code>sorted()</code>



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w13>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırma

Alıştırma 13.1. ●●● `nums = [10, 20, 30, 40, 50]` için aşağıdakilerin değerlerini *çalıştırmadan* yazın, sonra kabukta doğrulayın:

- | | |
|----------------------------|--------------------------------|
| (a) <code>nums[1]</code> | (e) <code>nums[::-1]</code> |
| (b) <code>nums[-2]</code> | (f) <code>len(nums[2:])</code> |
| (c) <code>nums[1:3]</code> | (g) <code>nums[5]</code> |
| (d) <code>nums[:2]</code> | (h) <code>nums[2:99]</code> |

Cevap: (a) 20 (b) 40 (c) [20, 30] (d) [10, 20] (e) [50, 40, 30, 20, 10] (f) 3 (g) IndexError (h) [30, 40, 50]

Alıştırma 13.2. ●●● Aşağıdaki program çalıştıktan sonra `a`, `b` ve `c` listelerinin içerikleri ne olur? Cevabınızı bir bellek şemasıyla (adlar ve nesneler) açıklayın.

```
1 a = [1, 2, 3]
2 b = a
3 a.append(4)
4 c = a[:]
5 a[0] = 99
```

Cevap: a = [99, 2, 3, 4], b = [99, 2, 3, 4], c = [1, 2, 3, 4]

Alıştırma 13.3. ●●● `sort()` ile `sorted()` arasındaki farkı iki cümleyle açıklayın; ardından şu programın neden `None` yazdığını söyleyip tek satır değiştirerek düzeltin.

```
1 names = ["ali", "veli", "ayse"]
2 sirali = names.sort()
3 print(sirali)
```

Cevap: sort() listeyi yerinde sıralar ve None döndürür; sorted() özgün listeye dokunmadan yeni bir sıralı liste döndürür. Düzeltme: sirali = sorted(names)

Alıştırma 13.4. ●●● Kullanıcıdan beş sayı alıp listeye koyan; listeyi, eleman sayısını, en küçüğü, en büyüğü, toplamı, bir ondalıklı ortalamayı, sıralı hâlini ve tersini yazan bir program yazın. Girdi 12,7,23,7,15 için ortalama 12,8 olmalıdır.

Cevap: toplam 64, ortalama 64/5 = 12,8; sıralı [7, 7, 12, 15, 23]; ters [15, 7, 23, 7, 12]

Alıştırma 13.5. ●●● `numbers = [3, 8, 12, 7, 5, 20, 15]` için *liste üreticiyle* şunları hesaplayın: (a) çift sayılar, (b) her elemanın karesi, (c) 10'dan büyük elemanlar, (d) çift sayıların toplamı. Sonra (a) şikkını açık bir `for` döngüsüyle de yazıp iki sonucun eşit olduğunu gösterin.

Cevap: (a) [8, 12, 20] (b) [9, 64, 144, 49, 25, 400, 225] (c) [12, 20, 15] (d) 40

Alıştırma 13.6. ●●● Bir listedeki yinelenen değerleri atan, ilk görülme sırasını koruyan ve özgün listeye dokunmayan bir `unique(items)` fonksiyonu yazın. [4, 7, 4, 2, 7,

9, 2, 4] için sonuç ne olur? Her değerin kaç kez geçtiğini `count` ile de yazdırın.

Cevap: [4, 7, 2, 9]; dört farklı değer; sayımlar $4 \rightarrow 3, 7 \rightarrow 2, 2 \rightarrow 2, 9 \rightarrow 1$

Alıştırma 13.7. ●● a = [3, 8, 1, 9] ve b = [5, 8, 2, 1] için şunları hesaplayan bir program yazın: birleşimin sıralı hâli, ortak elemanlar, yinelenensiz birleşim ve yalnız a'da bulunanlar.

Cevap: birleşik [1, 1, 2, 3, 5, 8, 8, 9]; ortak [8, 1]; tekil birleşim [1, 2, 3, 5, 8, 9]; yalnız a'da [3, 9]

Alıştırma 13.8. ●● Bir listeyi *dilimleme kullanmadan*, yerinde ters çeviren bir fonksiyon yazın: baştaki ile sondakini takas edin, sonra bir içerdeki, ortada buluşana kadar. [5, 3, 9, 1, 7] için sonuç ne olur, kaç takas yapılır? Sonucu [: :-1] ile karşılaştırarak doğrulayın.

Cevap: [7, 1, 9, 3, 5]; $5//2 = 2$ takas

Alıştırma 13.9. ●●● 12 zar atışının sonucu [3, 6, 1, 6, 4, 2, 6, 3, 5, 1, 4, 6]. Sözlük kullanmadan, yedi elemanlı bir sayaç listesiyle her yüzün kaç kez geldiğini sayın; yıldızlarla histogram basın, en sık gelen yüzü bulun ve sayaç toplamının atış sayısına eşit olduğunu doğrulayın.

Cevap: $1 \rightarrow 2, 2 \rightarrow 1, 3 \rightarrow 2, 4 \rightarrow 2, 5 \rightarrow 1, 6 \rightarrow 4$; en sık gelen yüz 6; sayaç toplamı 12

Alıştırma 13.10. ●●● Bir sayı listesinin medyanını döndüren `median(items)` fonksiyonu yazın; eleman sayısı tek ise ortadaki elemanı, çift ise ortadaki iki elemanın ortalamasını versin ve özgün listeye *dokunmasın*. [7, 3, 9] ve [7, 3, 9, 4] için sonuçları yazdırın; sonra [30, 32, 35, 31, 300] maaş listesinde ortalama ile medyayı karşılaştırıp farkı bir cümleyle açıklayın.

Cevap: [7, 3, 9] $\rightarrow 7$; [7, 3, 9, 4] $\rightarrow 5,5$; maaşlarda ortalama 85,6, medyan 32

Alıştırma 13.11. ●●● [2, 5, 9, 14, 20, 27, 35, 44, 54, 65] listesinde ikili arama yapan bir fonksiyon yazın; bulunan indeksi, karşılaştırma sayısını ve *bulunamazsa* değerin sıralamayı bozmadan ekleneceği yeri döndürsün. 44, 21, 2 ve 65 için sonuçları yazdırın; 21'i bulduğunuz yere ekleyip listenin hâlâ sıralı olduğunu doğrulayın.

Cevap: 44 $\rightarrow$ indeks 7, 2 adım; 21 $\rightarrow$ yok, 3 adım, 5. sıraya eklenir; 2 $\rightarrow$ indeks 0, 3 adım; 65 $\rightarrow$ indeks 9, 4 adım

Alıştırma 13.12. ●●● Bir listeyi k adım *sağa* döndüren iki fonksiyon yazın: biri dilimlemeyle (son k eleman başa gelir), öteki tek tek yerleştirerek. k liste uzunluğundan büyük olabilir. [1, 2, 3, 4, 5] listesini $k = 0, 1, 2, 7$ için döndürüp iki fonksiyonun aynı sonucu verdiğini gösterin.

Cevap: $k = 0 \rightarrow [1, 2, 3, 4, 5]$; $k = 1 \rightarrow [5, 1, 2, 3, 4]$; $k = 2 \rightarrow [4, 5, 1, 2, 3]$; $k = 7 \rightarrow [4, 5, 1, 2, 3]$ (çünkü $7 \bmod 5 = 2$)

Hafta 14: İki Boyutlu Listeler, Demetler ve Sözlükler

Öğrenme çıktıları

Bu haftanın sonunda:

1. iç içe listelerle satır-sütun düzeninde veri tutabilecek, bir hücreye `m[i][j]` ile erişebileceksiniz;
2. `[[0]*n]*m` yazımının neden beklenmedik sonuç verdiğini açıklayabilecek ve iki boyutlu listeyi liste üreticiyle güvenle oluşturabileceksiniz;
3. iç içe döngüyle satır ve sütun toplamaları hesaplayabilecek, transpoz alabilecek, iki matrisi çarpabilecek ve sonucu hizalı bir çizelge olarak yazdırabileceksiniz;
4. demetin listeden farkını söyleyebilecek, paketleme-açma, takas ve çoklu dönüşün aynı mekanizma olduğunu gösterebilecek, `zip` ile kayıt listesi üretip `sorted` ile istediğiniz alana göre sıralayabileceksiniz;
5. sözlükte ekleme, güncelleme, silme, arama ve döngü işlemlerini yapabilecek, `get` ile `KeyError`'dan korunabilecek ve frekans sayma desenini uygulayabileceksiniz;
6. bir problem için liste, demet, sözlük, küme ya da iki boyutlu liste arasından doğru yapıyı gerekçesiyle seçebileceksiniz.

Anahtar kavramlar

iki boyutlu liste (İng. *two-dimensional list*) **matris** (İng. *matrix*) **satır** (İng. *row*) **sütun** (İng. *column*) **transpoz** (İng. *transpose*) **demet** (İng. *tuple*) **paketleme** (İng. *packing*)
açma (İng. *unpacking*) **kayıt** (İng. *record*) **sözlük** (İng. *dictionary*) **anahtar** (İng. *key*)
değer (İng. *value*) **frekans sayma** (İng. *frequency counting*) **küme** (İng. *set*) **özet** (İng. *hash*)

Geçen hafta liste sayesinde tek bir adla yüzlerce değeri tutabildiniz. Ama gerçek veriler nadiren tek sıradır. Bir sınıfın üç sınavdaki notları bir çizelgedir: satırlar öğrenciler, sütunlar sınavlar. Bir görüntü bir piksel ızgarasıdır; bir XOX tahtası üç satır üç sütundur. Üstelik verilerin çoğunda eleman sıra numarasıyla değil bir *adla* aranır: “Trabzon’un plaka kodu”, “kalemin fiyatı”, “bu kelime kaç kez geçti”. Bu hafta bu üç ihtiyaç için üç araç öğreniyoruz — **iki boyutlu liste**, **demet**, **sözlük** — ve yanlarına küçük bir ek koyuyoruz: **küme**.

Hiçbiri sıfırdan yeni bir fikir değil. İki boyutlu liste, elemanları yine liste olan bir listedir; geçen haftanın takma ad (*aliasing*) konusu burada sizi tekrar karşılayacak. Demet, değiştirelemeyen bir listedir; 4. haftadaki `a, b = b, a` takası ile 12. haftadaki çoklu dönüş meğer hep demetmiş. Sözlük ise indisin yerine anahtarı koyar. Bir yapay zeka mühendisi için bu üçü günlük ekmektir: eğitim verisi bir sayı matrisidir, bir modelin ayarları bir sözlüktür, bir veri noktası çoğu zaman bir demettir.

14.1 İki boyutlu liste: listelerin listesi

Dört öğrencinin üç sınavdan aldığı notları üç ayrı listede de tutabilirsiniz; ama o zaman “üçüncü öğrencinin tüm notları” gibi basit bir soru üç listeyi birden gezmeyi gerektirir ve sınav sayısı değişince program baştan yazılır. Doğru araç, elemanları *liste* olan bir listedir.

Tanım 14.1: İki boyutlu liste

İki boyutlu liste (İng. *two-dimensional list*), elemanlarının her biri yine bir liste olan listedir. Dıştaki liste **satırları** (İng. *row*) tutar; her satır kendi içinde **sütunlara** (İng. *column*) ayrılır. r satır ve c sütundan oluşan böyle bir yapıya **matris** (İng. *matrix*) de

denir ve $r \times c$ boyutlu olduğu söylenir. Bir hücreye erişim iki indisle yapılır: $m[i][j]$, önce i . satırı seçer, sonra o satırın j . elemanını alır (Şekil 14.1).

		sütun indisi j			
		0	1	2	3
satır indisi i	0	3	8	1	6
	1	5	2	9	4
	2	7	0	3	8

$m[1][2]$

$m[1] = [5, 2, 9, 4]$
bir satır = bir liste

```
len(m)    → 3 # satır sayısı
len(m[0]) → 4 # sütun sayısı
m[1][2]   → 9 # vurgulanan hücre
m[-1][-1] → 8 # sağ alt
```

İki boyutlu liste, elemanları yine liste olan bir listedir. Dıştaki liste *satırları* tutar; $m[1]$ bir satırın tamamıdır, $m[1][2]$ o satırın 2. elemanıdır. İndis sırası her zaman **önce satır, sonra sütun**'dur ve ikisi de 0'dan sayılır. r satır, c sütunlu bir matriste geçerli indisler $0 \dots r-1$ ve $0 \dots c-1$ 'dir; dışına çıkmak `IndexError` verir.

Şekil 14.1: İki boyutlu listede indisler. Satır indisi i soldan, sütun indisi j üstten okunur; ikisi de sıfırdan başlar. Turuncu hücre $m[1][2]$, yani 1. satırın 2. elemanıdır.

`len(m)` *satır* sayısını verir, çünkü dış listenin elemanları satırlardır; sütun sayısı için bir satırın uzunluğuna bakılır: `len(m[0])`. Bu ayrım yeni öğrenenlerin en sık takıldığı noktadır. İndisleri her zaman "`m[satır][sütun]`" diye sesli okuyun ve değişkenlere i (satır), j (sütun) demeyi alışkanlık edinin; o zaman üstteki hücrenin $m[i-1][j]$, sağdakinin $m[i][j+1]$ olduğu kendiliğinden gelir.

Kod 14.1 w14-01-matris-temel.py — İç içe liste: erişim, satır ve sütun gezintisi

```
1 # w14-01-matris-temel.py -- iç içe liste: erişim ve iç içe döngüyle gezinme
2
3 m = [[3, 8, 1, 6],           # 3 satır, 4 sütun
4       [5, 2, 9, 4],
5       [7, 0, 3, 8]]
6
7 print("Satır sayısı:", len(m), " Sütun sayısı:", len(m[0]))
8 print("1. satır:", m[1])      # bir satır = bir liste
9 print("m[1][2] =", m[1][2], " sağ alt:", m[-1][-1])
10
11 # Bir sütunu toplamak: sütun indisi sabit, satırlar üzerinde döngü
12 col_total = 0
13 for row in m:
14     col_total += row[2]
15 print("2. sütunun toplamı:", col_total)
16
17 # Tüm elemanları gezmek: dış döngü satır, iç döngü sütun
18 total = 0
19 for i in range(len(m)):
20     for j in range(len(m[i])):
21         total += m[i][j]
```



GitHub


```

22 print("Tüm elemanların toplamı:", total)
23
24 # İndis gerekmiyorsa aynı gezinmenin daha okunaklı biçimi
25 count_even = 0
26 for row in m:
27     for value in row:
28         if value % 2 == 0:
29             count_even += 1
30 print("Çift eleman sayısı:", count_even)
31
32 m[2][1] = 10 # tek bir hücreyi değiştirme
33 print("Yeni 2. satır:", m[2])

```

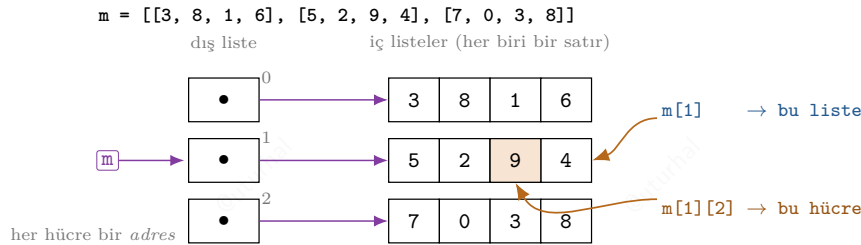
```
$ python w14-01-matris-temel.py
```

```

Satır sayısı: 3 Sütun sayısı: 4
1. satır: [5, 2, 9, 4]
m[1][2] = 9 sağ alt: 8
2. sütunun toplamı: 13
Tüm elemanların toplamı: 56
Çift eleman sayısı: 6
Yeni 2. satır: [7, 10, 3, 8]

```

Kod 14.1’de üç ayrıntıya dikkat edin. Birincisi, `m[1]` bir sayı değil bir *listedir*: ekrana `[5, 2, 9, 4]` yazılır; negatif indisler de çalışır, `m[-1][-1]` sağ alt köşeyi verir. İkincisi, tüm elemanları gezmenin iki biçimi vardır: indislerle (iki iç içe `range` döngüsü) ve doğrudan elemanlarla (`for row in m` içinde `for value in row`). *İndise ihtiyacınız yoksa ikinci biçimi kullanın*. Üçüncüsü, bir sütunu toplamak için sütun indisi sabit tutulur ve satırlar üzerinde dolaşılır: 2. sütunun toplamı $1 + 9 + 3 = 13$, tüm elemanların toplamı ise 56’dır.



Dış liste sayıları değil, iç listelere giden referansları tutar. Bu yüzden `m[1]` bir sayı değil, bir listenin kendisidir; ona ikinci bir indis uygulayınca (`m[1][2]`) sayıya ulaşırsınız. Erişim iki adımdır: önce satır referansı izlenir, sonra o satırın içinde ilerlenir. Aynı modelin bir sonucu da şudur: iki satır *aynı* iç listeyi gösteriyorsa, birini değiştirmek diğerini de değiştirir — geçen haftanın takma ad sorunu, bu hafta iki boyutta geri geliyor.

Şekil 14.2: İki boyutlu listenin bellek modeli. Dış listenin her hücresi bir iç listeye *referans* tutar; bu yüzden erişim iki adımdır ve iki satır aynı listeyi gösterebilir.

Bellek modeli (Şekil 14.2) geçen haftanın etiket fikrinin devamıdır: dış liste sayıları değil, iç listelerin adreslerini tutar. Bu ayrıntı kuramsal görünebilir; bir sonraki bölümde çok pratik bir sonuca yol açacak.

✗ Sık yapılan hata

`m[1, 2] → m[1][2]`

Virgüllü yazım matematikte ve NumPy’da anlamlıdır, düz Python listesinde değil. Python `m[1, 2]` gördüğünde köşeli parantezin içini bir *demet* sanar ve `TypeError: list`

indices must be integers or slices, not tuple der. Listede her boyut için ayrı bir köşeli parantez açılır.

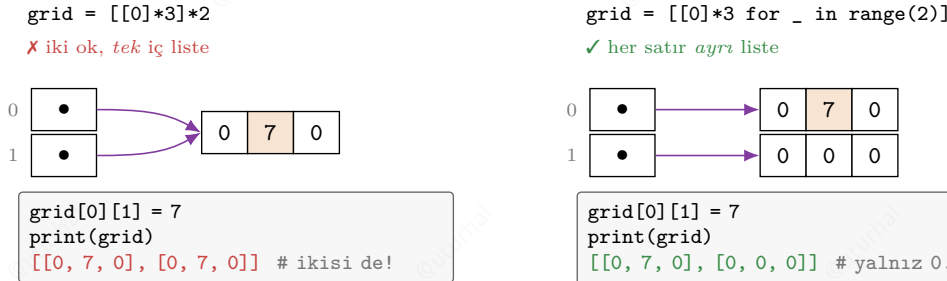
14.2 İki boyutlu liste oluşturmak: çarpma tuzağı

Boş bir tahta ya da sıfırlarla dolu bir çizelge sık gerekir. Geçen hafta tek boyutlu listede `[0] * 5` yazımı işe yaramıştı; aynı fikri iki boyuta taşımak çok doğal görünür:

Python etkileşimli kabuk (REPL)

```
>>> grid = [[0] * 3] * 2
>>> grid[0][1] = 7
>>> grid
[[0, 7, 0], [0, 7, 0]]
```

Tek bir hücreye atama yaptık, iki satır birden değişti. Python hata vermedi; program sessizce yanlış çalışıyor — bulunması en zor hata türü budur.



liste * n listeyi kopyalamaz; aynı nesneye n tane referans koyar. Solda dış listenin iki hücresi de aynı iç listeyi gösterdiği için `grid[0][1] = 7` yazmak “iki” satırı birden değiştirir — aslında tek satır vardır, iki kez görünür. Sağdaki liste üretici `[0]*3` ifadesini *her dönüşte yeniden* çalıştırır, böylece birbirinden bağımsız iki liste oluşur. Kural: iç içe liste oluştururken dış çarpma (*) kullanmayın, liste üretici kullanın. Yalnız *tek* boyutlu listede `[0]*3` güvenlidir, çünkü içindekiler değiştirilemez sayılardır.

Şekil 14.3: Solda `[[0]*3]*2`: dış listenin iki hücresi de *aynı* iç listeyi gösterir. Sağda liste üretici: her satır için ayrı bir liste oluşur.

Nedeni **Şekil 14.3**'te görülüyor ve bellek modelinin doğrudan sonucudur: * operatörü listeyi *kopyalamaz*, aynı nesneye birden çok referans koyar. Geçen hafta `b = a` yazdığımızda iki adın aynı listeyi gösterdiğini öğrenmiştiniz; burada olan budur, yalnız adlar yerine dış listenin hücreleri var. Ortada iki satır yoktur; tek bir satır iki kez görünür.

❌ Sık yapılan hata

`grid = [[0] * cols] * rows` → `grid = [[0] * cols for _ in range(rows)]`

Dıştaki çarpma tekrarlanan *referanslar* üretir. Liste üretici ise `[0] * cols` ifadesini her dönüşte *yeniden* çalıştırır ve birbirinden bağımsız satırlar oluşturur. İçteki `[0] * cols` güvenlidir: içindekiler sayıdır, sayılar zaten değiştirilemez.

İki boyutlu liste oluşturmanın üç yolu

1. **Değerler baştan belliyse** doğrudan yazın, her satırı ayrı bir kod satırına hizalayarak: `m = [[3, 8, 1], [5, 2, 9]]`.
2. **Sabit bir değerle dolduracaksanız** liste üretici kullanın:

```
grid = [[0] * cols for _ in range(rows)].
```

3. **Değerler** hesaplanacaksa iç içe liste üretici yazın:
`t = [[i * j for j in range(5)] for i in range(4)]` bir çarpım tablosu üretir.

Üçünde de ortak ölçüt şudur: *her satır ayrı bir liste nesnesi olmalıdır.*



Dikkat

Şüphelendiğinizde sınama kolaydır: bir hücreye işaret değeri yazıp tüm matrisi yazdırın; işaret birden çok satırda görünüyorsa satırlar aynı nesnedir. Aynı tuzak hazır bir satırı kopyalamadan kullanmakta da vardır: `m = [row, row]` yazarsanız iki satır yine aynı listedir; doğrusu `m = [row[:], row[:]]`'dir.

14.3 İç içe döngüyle gezinmek: satırlar, sütunlar ve çizelgeler

İki boyutlu listenin tipik işlemleri **iç içe döngü** ile yapılır: dış döngü satırları, iç döngü o satırın sütunlarını dolaşır. Üç desen her programda karşınıza çıkar. **Tüm elemanlar üzerinde biriktirme:** toplam, sayaç, en büyük. **Satır bazlı işlem:** her satır tek başına bir listedir, geçen haftanın araçları (`sum`, `max`, `sorted`) doğrudan çalışır. **Sütun bazlı işlem:** sütun diye bir liste *yoktur*; sütun indisi sabit tutulup satırlarda dolaşarak toplanır. Çizelge çıktısı ise 4. haftanın f-string genişlik belirteçleriyle hizalanır.

Örnek 14.1 (Sınıf not çizelgesi). Dört öğrencinin üç sınavdan (Vize, Final, Proje) aldığı notlar bir matriste tutuluyor. Her öğrencinin ortalamasını satır sonunda, her sınavın ortalamasını da çizelgenin altında gösteren hizalı bir çizelge yazdıran program yazın.

Çözüm

Analiz. Girdi: 4×3 boyutlu not matrisi, öğrenci ve sınav adları. Çıktı: hizalı bir çizelge, satır ve sütun ortalamaları. İşlem: satır ortalaması `sum(row) / len(row)`; sütun ortalaması için sabit `j` ile satırlarda dolaşma.

Algoritma. (1) Başlık satırını sabit genişliklerle kur ve altına çizgi çek. (2) Her satır `i` için ortalamayı hesapla, adı ve notları hizalı yaz. (3) Her sütun `j` için toplamı sıfırdan başlat, satırlarda dolaşarak biriktir, satır sayısına böl ve yaz.

Kod ve çıktı.



Kod 14.2 w14-03-not-cizelgesi.py — Satır ve sütun ortalamaları, hizalı çizelge

```

1 # w14-03-not-cizelgesi.py -- 2B çizelge: satır ve sütun ortalamaları
2
3 students = ["Ayşe", "Mehmet", "Zeynep", "Can"]
4 exams = ["Vize", "Final", "Proje"]
5 grades = [[85, 90, 78],
6           [60, 72, 65],
7           [95, 88, 100],
8           [70, 55, 80]]
9
10 header = f"{'Öğrenci':<10}"
11 for exam in exams:
12     header += f"{exam:>7}"
13 header += f"{'Ort.':>8}"
14 print(header)
15 print("-" * len(header))
16
17 for i in range(len(students)):          # her satır bir öğrenci
18     avg = sum(grades[i]) / len(grades[i])
19     line = f"{students[i]:<10}"
20     for grade in grades[i]:
21         line += f"{grade:>7}"
22     print(line + f"{avg:>8.2f}")
23
24 print("-" * len(header))
25 line = f"{'Sınav ort.':<10}"
26 for j in range(len(exams)):              # her sütun bir sınav
27     col_total = 0
28     for i in range(len(students)):
29         col_total += grades[i][j]
30     line += f"{col_total / len(students):>7.2f}"
31 print(line)

```

```
$ python w14-03-not-cizelgesi.py
```

Öğrenci	Vize	Final	Proje	Ort.
Ayşe	85	90	78	84.33
Mehmet	60	72	65	65.67
Zeynep	95	88	100	94.33
Can	70	55	80	68.33
Sınav ort.	77.50	76.25	80.75	

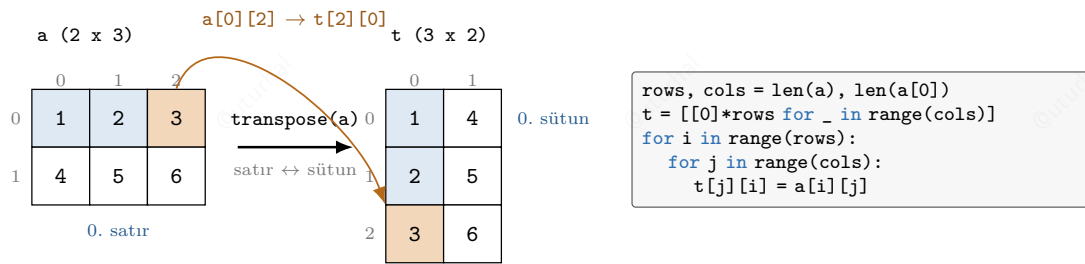
Kontrol. Ayşe'nin ortalaması elle: $(85 + 90 + 78)/3 = 253/3 = 84,33$ ✓. Vize sütununun ortalaması: $(85 + 60 + 95 + 70)/4 = 310/4 = 77,50$ ✓. İki yönden gelen sonuçlar birbirini denetler: satır ortalamalarının toplamı 312,67, dörde bölümü 78,17; sütun ortalamalarının toplamı 234,50, üçe bölümü yine 78,17 ✓ — ikisi de tüm notların genel ortalamasıdır (938/12). Çizelgedeki en yüksek satır ortalaması 94,33 (Zeynep), en yüksek sütun ortalaması 80,75 (Proje) ✓.

İpucu

Başlık satırını da veri satırlarıyla *aynı* genişlik belirteçleriyle yazın: `f"{'Vize':>7}"` ile `f"{'grade:>7}"` aynı genişlikte olduğu için sütunlar kaymaz. Ayırıcı çizgiyi elle saymayın; `print("-" * len(header))` başlığın kendi uzunluğunu kullanır, başlık değişince çizgi kendiliğinden uyar.

14.4 Matris işlemleri: transpoz, toplama ve çarpma

Transpoz satırları sütun yapar: $t[j][i] = m[i][j]$. $r \times c$ boyutlu bir matrisin transpozu $c \times r$ boyutludur; sonuç matrisini bu *ters* boyutla oluşturmak gerekir (Şekil 14.4). Ters yazılırsa program çoğu zaman `IndexError` verir; kare matrislerde ise hata vermez, sessizce yanlış çalışır.



Transpoz, her elemanın **satır ve sütun indislerini yer değiştirir**: $t[j][i] = a[i][j]$. $r \times c$ boyutlu bir matrisin transpozu $c \times r$ boyutludur; hedef matrisi bu *ters* boyutla oluşturmayı unutmayın. Transpozun transpozu özgün matristir — bu, kodunuzu sınamanın en kolay yoludur. Sütun üzerinde işlem yapmak gerektiğinde iki seçeneğiniz vardır: sabit sütun indisiyle satırlarda dolaşmak, ya da bir kez transpoz alıp sütunları satır gibi işlemek.

Şekil 14.4: Transpoz, her elemanın iki indisini takas eder. 2×3 boyutlu bir matrisin transpozu 3×2 boyutludur; 0. satır 0. sütun olur.

Toplama yalnız *aynı boyutlu* matrislerde tanımlıdır ve eleman elemandır: $(a + b)[i][j] = a[i][j] + b[i][j]$. Tek bir iç içe liste üretici yeter:

```
1 def add(a, b):
2     return [[a[i][j] + b[i][j] for j in range(len(a[0]))]
3             for i in range(len(a))]
```

Çarpma eleman eleman *değildir*. a matrisi $r \times n$, b matrisi $n \times c$ boyutluysa çarpım $r \times c$ boyutludur ve

$$(ab)[i][j] = \sum_{k=0}^{n-1} a[i][k] \cdot b[k][j]$$

olur: a 'nın i . satırı ile b 'nin j . sütunu karşılıklı çarpılıp toplanır. İki matrisin çarpılabilmesi için **birincinin sütun sayısı ikincinin satır sayısına eşit olmalıdır**; bu yüzden kodda üç iç içe döngü vardır (i, j, k).

Örnek 14.2 (Transpoz ve matris çarpımı). Transpoz ve çarpma işlemlerini birer fonksiyon

olarak yazın; $a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ ve $b = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$ üzerinde deneyin. Sütun toplamlarını transpozdan elde edin ve doğruluğu programın kendisi `assert` ile denetlesin.

Çözüm

Analiz. Girdi: iki matris; a 2×3 , b 3×2 olduğundan ab çarpımı 2×2 boyutlu çıkacaktır. **Algoritma.** Her fonksiyonda: sonucun boyutunu belirle, sonuç matrisini liste üreticiyle oluştur, iç içe döngüyle doldur, döndür. Çarpmada en içteki k döngüsü tek bir hücrenin değerini biriktirir.

Kod ve çıktı.

Kod 14.3 w14-02-matris-islem.py — Transpoz ve matris çarpımı

```
1 # w14-02-matris-islem.py -- transpoz ve matris çarpımı
2
3 def transpose(m):
4     """t[j][i] = m[i][j]; boyut ters: r x c -> c x r."""
5     t = [[0] * len(m) for _ in range(len(m[0]))]
6     for i in range(len(m)):
7         for j in range(len(m[0])):
8             t[j][i] = m[i][j]
9     return t
10
11 def multiply(a, b):
12     """a (r x n) ile b (n x c) çarpımı; sonuç r x c."""
13     result = [[0] * len(b[0]) for _ in range(len(a))]
14     for i in range(len(a)):
15         for j in range(len(b[0])):
16             for k in range(len(b)):
17                 result[i][j] += a[i][k] * b[k][j]
18     return result
19
20 a = [[1, 2, 3], [4, 5, 6]]
21 b = [[7, 8], [9, 10], [11, 12]]
22 print("Satır toplamları:", [sum(row) for row in a])
23 print("Sütun toplamları:", [sum(c) for c in transpose(a)])
24 print("Transpoz:", transpose(a))
25 print("a x b      :", multiply(a, b))
26
27 assert transpose(transpose(a)) == a # 12. hafta: assert
28 assert sum([sum(r) for r in a]) == sum([sum(c) for c in transpose(a)])
29 print("Kontroller geçti.")
```

```
$ python w14-02-matris-islem.py
```

```
Satır toplamları: [6, 15]
Sütun toplamları: [5, 7, 9]
Transpoz: [[1, 4], [2, 5], [3, 6]]
a x b      : [[58, 64], [139, 154]]
Kontroller geçti.
```

Kontrol. Çarpımın sol üst hücresi elle: $1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 58$ ✓; sağ alt hücre

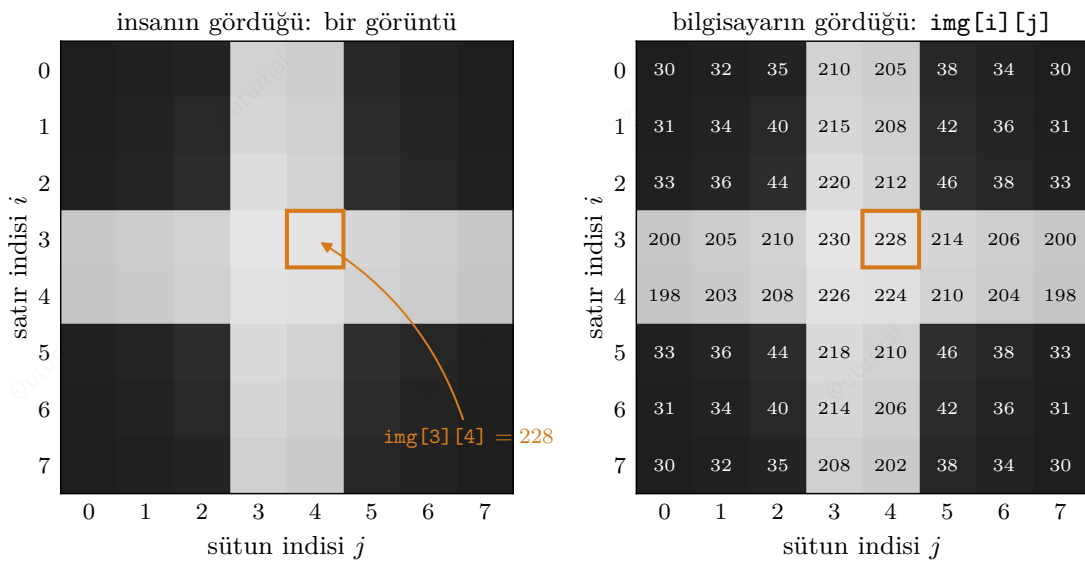


GitHub

$4 \cdot 8 + 5 \cdot 10 + 6 \cdot 12 = 154$ ✓. Satır toplamaları [6, 15], sütun toplamaları [5, 7, 9]; ikisinin de toplamı 21'dir ✓ — bütün matrisi iki ayrı yönden toplamış olduk. Programdaki `assert` satırı ise transpozun transpozunun özgün matrise eşit olduğunu doğrular; bu eşitlik bozulursa indisleri karıştırmışsınız demektir.

14.5 İki boyutlu listenin iki yüzü: görüntü ve oyun tahtası

İki boyutlu listenin yapay zekadaki en yaygın karşılığı **görüntüdür**: gri seviye bir görüntü, her hücresi 0 (siyah) ile 255 (beyaz) arasında bir parlaklık değeri tutan bir sayı ızgarasıdır (Şekil 14.5). Birinci haftada “bir görüntü piksellerden oluşur” demiştik; şimdi o ızgarayı gerçekten programlayabiliyorsunuz.



Şekil 14.5: Solda gördüğünüz 8×8 görüntü, sağda aynı görüntünün sayıları. Bilgisayar için ikisi aynı şeydir: bir sayı matrisi. Çerçevesi hücre `img[3][4]`, değeri 228.

Örnek 14.3 (Görüntü eşikleme). 5×5 gri seviye bir görüntünün ortalama parlaklığını ve en parlak pikselini bulun; sonra bir eşik değeri (128) kullanarak görüntüyü ikili hâle getirin ve `#/.` karakterleriyle çizin. Kaç piksel beyaz olmuştur?

Çözüm

Analiz. Girdi: piksel matrisi ve eşik. Çıktı: ortalama parlaklık, en büyük değer, ikili görüntü ve beyaz piksel sayısı. İşlem: ortalama için tüm elemanların toplamı piksel sayısına ($5 \cdot 5 = 25$) bölünür; eşikleme her piksel için tek bir karşılaştırmadır ve *yeni* bir matris üretir — özgün görüntü bozulmaz.

Algoritma. Her satır için yeni bir satır listesi oluştur; satırdaki her piksel eşikten büyük ya da eşitse 1, değilse 0 ekle; satırı sonuç matrisine ekle. Beyaz piksel sayısı, ikili matristeki bütün 1'lerin toplamıdır.

Kod ve çıktı.



Kod 14.4 w14-04-esikleme.py — Piksel matrisi: ortalama parlaklık ve eşikleme

```

1 # w14-04-esikleme.py -- gri seviye görüntü = piksel matrisi; eşikleme
2
3 image = [[ 20,  35, 210,  40,  25],      # 0 siyah, 255 beyaz
4           [ 30,  45, 220,  50,  35],
5           [200, 215, 235, 205, 190],
6           [ 25,  40, 225,  45,  30],
7           [ 15,  30, 215,  35,  20]]
8 THRESHOLD = 128
9
10 def threshold(img, t):
11     """t ve üstü 1, altı 0; YENİ bir görüntü döndürür."""
12     result = []
13     for row in img:
14         result.append([1 if p >= t else 0 for p in row])
15     return result
16
17 total = 0
18 brightest = 0
19 for row in image:
20     total += sum(row)
21     brightest = max(brightest, max(row))
22 pixels = len(image) * len(image[0])
23 print(f"{pixels} piksel, ortalama parlaklık {total / pixels:.1f}, "
24       f"en parlak {brightest}")
25
26 binary = threshold(image, THRESHOLD)
27 white = 0
28 for row in binary:
29     white += sum(row)
30 print(f"Eşik {THRESHOLD} ile {white} beyaz piksel:")
31 for row in binary:
32     print("".join("#" if p == 1 else "." for p in row))

```

```
$ python w14-04-esikleme.py
```

```

25 piksel, ortalama parlaklık 97.4, en parlak 235
Eşik 128 ile 9 beyaz piksel:
..#..
..#..
#####
..#..
..#..

```

Kontrol. 25 pikselin toplamı 2435'tir; ortalama $2435/25 = 97,4$ ✓. Görüntüde bir dikey ve bir yatay parlak şerit vardır; eşikten büyük piksellerin sayısı 9 çıkmalıdır: $5 + 5 - 1 = 9$, çünkü ortak hücre bir kez sayılır ✓. Çizilen ikili görüntü de tam olarak bir artı işareti göstermektedir. Uç durum: eşik 0 verilirse her piksel beyaz olur (25), eşik 256 verilirse hiçbiri (0) — ikisini de laboratuvarında deneyin.

Not

Gerçek görüntü işleme ve makine öğrenmesi kodlarında bu matrisler düz Python listeleriyle değil, **NumPy** dizileriyle tutulur. NumPy bir dizinin tamamına tek satırda işlem uygular ($\text{img} > 128$ bütün matrisi bir kerede eşikler) ve hesabı C ile yazılmış çekirdeklerde yaptığı için yüzlerce kat hızlı çalışır. NumPy bu dersin kapsamı dışındadır ve sınavlarda sorulmaz; ikinci sınıfta öğreneceksiniz. Ama oradaki her fikrin karşılığını bu hafta öğreniniz: `img[i][j]`, satır/sütun toplamı, transpoz, matris çarpımı.

İkinci yüz **oyun tahtasıdır**. Üç satır üç sütunluk bir XOX tahtası, her hücreinde "X", "O" ya da boşluk tutan bir matristir. Kazananı bulmak için bakılacak yerler bellidir: 3 satır, 3 sütun ve 2 köşegen, toplam 8 çizgi. Bu çizgileri tek bir listede toplamak, kazanan kontrolünü tek bir döngüye indirir.

Kod 14.5 w14-11-tic-tac-toe.py — XOX: tahta gösterimi, aday çizgiler, kazanan

```
1 # w14-11-tic-tac-toe.py -- 3x3 tahta: gösterim ve kazanan kontrolü
2
3 EMPTY = " "
4
5 def show(board):
6     print("  0  1  2")
7     for i in range(3):
8         print(f"{i}  " + " | ".join(board[i]))
9
10 def lines_of(board):
11     """Kazanmaya aday 8 üçlü: 3 satır, 3 sütun, 2 köşegen."""
12     triples = [row for row in board]
13     triples += [[board[i][j] for i in range(3) for j in range(3)]
14                 for i in range(3)]
15     triples.append([board[i][i] for i in range(3)])
16     triples.append([board[i][2 - i] for i in range(3)])
17     return triples
18
19 def winner(board):
20     """Kazanan işareti, kimse kazanmadıysa None döndürür."""
21     for t in lines_of(board):
22         if t[0] != EMPTY and t[0] == t[1] == t[2]:
23             return t[0]
24     return None
25
26 def is_full(board):
27     for row in board:
28         if EMPTY in row:
29             return False
30     return True
31
32 game = [[EMPTY] * 3 for _ in range(3)] # her satır AYRI bir liste
33 for i, j, mark in [(1, 1, "X"), (0, 0, "O"), (0, 1, "X"),
34                   (2, 2, "O"), (2, 1, "X")]:
35     game[i][j] = mark
36 show(game)
37 print("Aday üçlü:", len(lines_of(game)), " kazanan:", winner(game),
38       " dolu mu?", is_full(game))
39
40 draw = [["X", "O", "X"], ["X", "O", "O"], ["O", "X", "X"]]
41 show(draw)
```



GitHub

```
41 print("Kazanan:", winner(draw), " dolu mu?", is_full(draw))
```

```
$ python w14-11-tic-tac-toe.py
```

```

  0  1  2
0  0  | X  |
1     | X  |
2     | X  | 0
Aday üçlü: 8  kazanan: X  dolu mu? False
  0  1  2
0  X  | 0  | X
1  X  | 0  | 0
2  0  | X  | X
Kazanan: None  dolu mu? True
```

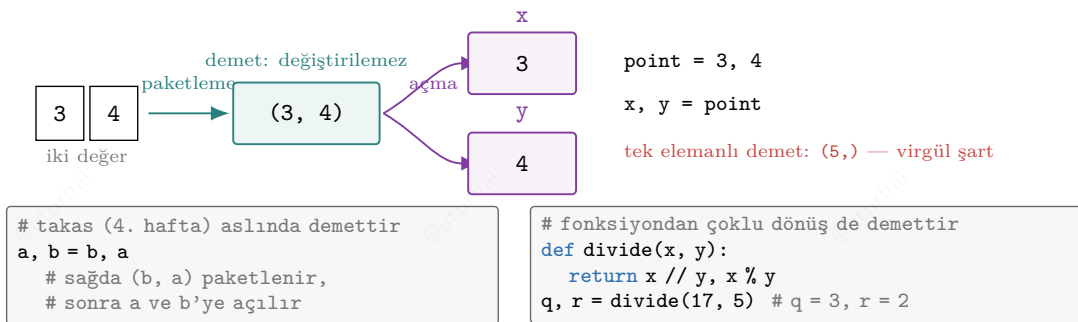
Kod 14.5’te tahta, satırları ayrı listeler olacak biçimde liste üreticiyle kuruluyor — bu haftanın tuzağına düşülmemesi gereken tipik bir yer. Birinci tahtada X işaretleri 1. sütunun tamamındadır, dolayısıyla kazanan X’tir ve tahtada hâlâ boş hücre olduğu için doluluk **False** çıkar. İkinci tahtada hiçbir üçlü aynı işaretten oluşmaz: kazanan **None**, tahta dolu — yani berabere. Uç durum önemlidir: boş tahtada **winner** **None** döndürmelidir; bunu sağlayan koşul `t[0] != EMPTY`’dir, yoksa üç boşluk “kazanır”dı.

14.6 Demet (tuple): değiştirilemeyen dizi

Bir noktanın koordinatları, bir tarihin gün–ay–yıl üçlüsü, bir öğrencinin ad–puan ikilisi... Bunların ortak yanı, birlikte gezen *sabit sayıda* alandan oluşmalarıdır. Python bu iş için listeden ayrı bir tür sunar.

Tanım 14.2: Demet

Demet (İng. *tuple*), oluşturulduktan sonra **değiştirilemeyen** bir dizidir. Genelde parantezle yazılır, (3, 4); ama demeti oluşturan parantez değil *virgüldür*: `point = 3, 4` de bir demet üretir. Tek elemanlı demet (5,) biçiminde, sondaki virgülle yazılır. İndisleme, dilimleme, `len`, `in`, `count` ve `index` listedeki gibi çalışır; `append`, `sort` gibi *değiştiren* işlemler yoktur.



Demeti oluşturan parantez değil, virgüldür. Virgülle ayrılmış değerler kendiliğinden tek bir demet nesnesinde *paketlenir*; eşitliğin solunda aynı sayıda ad varsa demet tekrar *açılır*. Dördüncü haftadaki takas da, 12. haftadaki çoklu dönüş de bu tek mekanizmanın uygulamasıdır. Demet oluştuktan sonra **değiştirilemez**: eleman ataması **TypeError** verir — bu “dondurulmuşluk” onu sözlük anahtarı olarak kullanılabilir kılan özelliktir.

Şekil 14.6: Paketleme ve açma. Virgülle ayrılmış değerler bir demette toplanır; eşitliğin solundaki adlar demeti tekrar açar. Takas ve fonksiyondan çoklu dönüş aynı mekanizmadır.

Sekil 14.6'daki iki işlemin adı vardır: değerlerin bir demette toplanmasına **paketleme** (İng. *packing*), demetin birden çok değişkene dağıtılmasına **açma** (İng. *unpacking*) denir. Bu ikisini aslında haftalardır kullanıyorsunuz: 4. haftadaki `a, b = b, a` takası sağda bir demet paketler, solda onu açar; 12. haftada bir fonksiyondan iki değer döndürdüğünüzde dönen şey bir demetti.

Kod 14.6 w14-05-demet.py — Demet: yazım, paketleme, açma, takas, çoklu dönüş

```
1 # w14-05-demet.py -- demet: değiştirilemez dizi, paketleme ve açma
2
3 point = (3, 4)
4 print(point, type(point), " x =", point[0], " uzunluk:", len(point))
5
6 single = (5,)          # tek elemanlı demet: virgül şart!
7 not_tuple = (5)        # bu yalnız parantezli bir sayıdır
8 print(type(single), type(not_tuple))
9
10 trabzon = 41.0027, 39.7168 # paketleme: parantez gerekmez
11 lat, lon = trabzon        # açma
12 print(f"Enlem: {lat} Boylam: {lon}")
13
14 a, b = 3, 8
15 a, b = b, a              # takas: sağda paketle, solda aç
16 print("a =", a, " b =", b)
17
18 def divide(x, y):
19     """Bölümü ve kalanı birlikte, demet olarak döndürür."""
20     return x // y, x % y
21
22 q, r = divide(17, 5)      # dönen demet açılır
23 print(f"17 = {q} * 5 + {r}")
24
25 colors = ("kırmızı", "yeşil", "mavi")
26 color_list = list(colors) # listeye çevir, değiştir, geri çevir
27 color_list.append("sarı")
28 colors = tuple(color_list) # YENİ bir demet; eskisi değişmedi
29 print(colors, len(colors), colors.count("sarı"), colors.index("mavi"))
```

\$ python w14-05-demet.py

```
(3, 4) <class 'tuple'> x = 3 uzunluk: 2
<class 'tuple'> <class 'int'>
Enlem: 41.0027 Boylam: 39.7168
a = 8 b = 3
17 = 3 * 5 + 2
('kırmızı', 'yeşil', 'mavi', 'sarı') 4 1 2
```

Kod 14.6'in çıktısında `divide(17, 5)` çağrısı $17 = 3 \cdot 5 + 2$ eşitliğini kurar. Sonda demetle liste arasındaki dönüşüm gösteriliyor: `list(...)` bir liste kopyası verir, `append` yapılır, `tuple(...)` ile yeniden demete çevrilir. Özgün demet değişmemiştir; `colors` adı yeni bir demete bağlanmıştır — tıpkı 5. haftada karakter dizilerinde olduğu gibi.

❌ Sık yapılan hata

`single = (5)` → `single = (5,)`

Parantezler tek başına demet yapmaz; (5) yalnızca parantez içine alınmış bir sayıdır ve türü `int`'tir. Tek elemanlı demet sondaki virgülle yazılır. Ters yönde de bir tuzak



GitHub

vardır: `print("Sonuç:", (x, y))` yazımı demeti tek bir değer olarak basar; iki ayrı değer istiyorsanız virgüllü parantezin dışına alın.

“Değiştirilemiyorsa ne işe yarıyor?” sorusunun üç cevabı var. **Güvence:** bir demeti fonksiyona geçirdiğinizde fonksiyonun onu değiştiremeyeceğini bilirsiniz — listelerde bu güvence yoktur ve geçen hafta bunun sürprizlerini gördünüz. **Niyet bildirimi:** sabit alanlı bir ikili demet yapmak, okuyucuya “bu bir koordinattır, sıralanacak bir dizi değil” der. **Sözlük anahtarı olabilme:** bir sonraki bölümde göreceğiniz gibi sözlük anahtarları değiştirilemez olmak zorundadır; demet olabilir, liste olamaz.

14.7 Demet listeleri: kayıtlar, zip ve sıralama

Gerçek veriler çoğu zaman *kayıtlardan* oluşur: her kayıt birkaç alanlı bir satırdır. Python’da bunun en doğal karşılığı demetlerden oluşan bir listedir: `[("Ayşe", 85), ("Mehmet", 60)]`. Dış yapı listedir, çünkü kayıt sayısı değişebilir; her kayıt demettir, çünkü alan sayısı sabittir.

İki ayrı listeyi kayıt listesine çevirmenin kısa yolu, 10. haftada tanıştığınız *zip*’tir: karşılıklı elemanları ikili demetler hâlinde eşler. Kayıtları istediğiniz alana göre sıralamak için ise 12. haftadaki anahtarlı sıralama kullanılır; anahtar fonksiyonu her kayıttan sıralamada kullanılacak alanı seçer.

```
1 records = list(zip(names, scores))           # [(ad, puan), ...]
2 ranked = sorted(records, key=lambda r: r[1]) # 1. alana göre
```

Örnek 14.4 (Puan sıralaması ve özet). Ad listesi ve puan listesi verilen beş öğrenci için: kayıt listesi oluşturun; en düşük kaydı, en yüksek kaydı ve ortalamayı *tek bir fonksiyondan* döndürün; sonra kayıtları puana göre büyükten küçüğe sıralı bir tablo hâlinde yazdırın.

Çözüm

Analiz. Girdi: iki paralel liste. Çıktı: kayıt listesi, üçlü özet ve sıralı tablo. İşlem: *zip* ile eşleme; tek geçişte üç değeri biriktirme; anahtarlı sıralama.

Algoritma. *summarize* listeyi bir kez gezer: her kayıtta demeti *name*, *score* olarak açar, toplamı biriktirir, en küçük ve en büyük kaydı günceller; sonunda üç değeri *bir demet olarak* döndürür. Çağırın taraf bunu üç ada açar.

Kod ve çıktı.



Kod 14.7 w14-06-kayitlar.py — zip, demet açma, çoklu dönüş ve anahtarlı sıralama

```

1 # w14-06-kayitlar.py -- demet listesi = kayıtlar; zip, çoklu dönüş,
  ↪ sıralama
2
3 names = ["Ayşe", "Mehmet", "Zeynep", "Can", "Elif"]
4 scores = [85, 60, 95, 70, 78]
5
6 records = list(zip(names, scores)) # her kayıt bir (ad, puan) demeti
7 print(records[:3], "...")
8
9 def summarize(recs):
10     """En düşük kaydı, en yüksek kaydı ve ortalamayı döndürür."""
11     lowest = highest = recs[0]
12     total = 0
13     for name, score in recs: # demeti döngüde aç
14         total += score
15         if score < lowest[1]:
16             lowest = (name, score)
17         if score > highest[1]:
18             highest = (name, score)
19     return lowest, highest, total / len(recs)
20
21 low, high, avg = summarize(records)
22 print(f"En düşük: {low[0]} ({low[1]}), en yüksek: {high[0]}
  ↪ ({high[1]})")
23 print(f"Ortalama: {avg:.1f}")
24
25 # Puana göre büyükten küçüğe: anahtar, demetin 1. elemanı
26 by_score = sorted(records, key=lambda rec: rec[1], reverse=True)
27 for rank, (name, score) in enumerate(by_score, start=1):
28     print(f"{rank:<3} {name:<9} {score:>4}")
29 print("Alfabetik ilk iki:", sorted(records)[:2]) # anahtarsız: ada
  ↪ göre

```

```
$ python w14-06-kayitlar.py
```

```

[('Ayşe', 85), ('Mehmet', 60), ('Zeynep', 95)] ...
En düşük: Mehmet (60), en yüksek: Zeynep (95)
Ortalama: 77.6
1  Zeynep      95
2  Ayşe       85
3  Elif       78
4  Can        70
5  Mehmet     60
Alfabetik ilk iki: [('Ayşe', 85), ('Can', 70)]

```

Kontrol. Ortalama elle: $(85 + 60 + 95 + 70 + 78)/5 = 388/5 = 77,6$ ✓. En yüksek 95 (Zeynep), en düşük 60 (Mehmet); sıralı tablonun ilk ve son satırları da bu ikisidir ✓. Sıralamanın ikinci bir kanıtı: tabloda puanlar yukarıdan aşağı azalarak gitmelidir (95,85,78,70,60) ✓. Anahtarsız `sorted` ise demetleri ilk alandan karşılaştırdığı için alfabetik sıra verir; programın son satırı bu farkı gösterir.

İpucu

`enumerate` ile demet açmayı birleştirebilirsiniz: döngü başlığını

```
1 for rank, (name, score) in enumerate(by_score, start=1):
```

biçiminde yazarsanız her adımda hem sıra numarasını hem de kaydın iki alanını ayrı adlara açmış olursunuz. İçteki parantezler gereklidir.

14.8 Sözlük (dictionary): anahtarla erişim

Şimdiye kadarki tüm yapılarda eleman *sıra numarasıyla* bulunuyordu. Oysa günlük hayat-taki aramaların çoğu bir *adla* yapılır: bir ilin plaka kodu, bir ürünün fiyatı, bir kelimenin kaç kez geçtiği. Bunları iki paralel listeyle tutmak mümkündür ama hem hataya açıktır hem de her aramada listeyi baştan taramayı gerektirir.

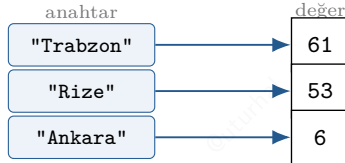
Tanım 14.3: Sözlük

Sözlük (*İng. dictionary*), **anahtar-değer** (*İng. key-value*) çiftlerinden oluşan bir yapıdır. Süslü parantezle yazılır: `{"Trabzon": 61, "Rize": 53}`. Anahtarlar **benzersiz** ve **değiştirilemez** olmalıdır (`str`, `sayı` ya da `demet`); değerler için kısıt yoktur. `d[k]` okuma ve yazma yapar; `d[k] = v` anahtar yoksa ekler, varsa günceller. Boş sözlük `{}` ile yazılır.

Sözlük, matematikteki *eşleme* fikrinin programdaki karşılığıdır: her anahtar tam olarak bir değere bakar (Şekil 14.7). Tersisi doğru olmak zorunda değildir; iki farklı anahtar aynı değeri gösterebilir, ama bir anahtarın iki değeri olamaz.

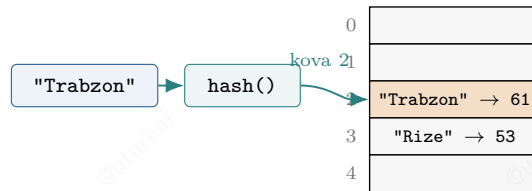
Anahtar → değer eşlemesi

`plates = {"Trabzon": 61, "Rize": 53}`



her anahtar bir kez geçer;
değerler tekrarlanabilir

Anahtar, yerini tek adımda bulur



baştan taranmaz: kaç kayıt olursa olsun
erişim aynı sürede biter

Listede eleman *sırasıyla*, sözlükte *anahtarla* bulunur. Sözlük anahtarı bir sayıya (özet, *hash*) çevirir ve değeri doğrudan o gözde arar; bu yüzden "Trabzon" in `plates` sorgusu sözlükte binlerce kayıt olsa bile ucuzdur, listede ise en kötü durumda her elemana bakmak gerekir. Özeti hesaplanabilmesi için anahtarın **değiştirilemez** olması şarttır: `str`, `sayı` ve `demet` anahtar olabilir; liste olamaz (`TypeError: unhashable type: 'list'`). Değer için böyle bir kısıt yoktur — değer liste de sözlük de olabilir.

Şekil 14.7: Sözlük bir eşlemedir: her anahtar tek bir değere bakar. Anahtar bir özet (*hash*) değerine çevrilip yeri doğrudan bulunduğu için arama, kayıt sayısından neredeyse bağımsızdır.

Kod 14.8 w14-07-sozluk.py — Sözlük: ekleme, güncelleme, silme, arama ve döngü

```
1 # w14-07-sozluk.py -- sözlük: anahtar-değer çiftleri, ekle/sil/ara/döngü
2
3 plates = {"Trabzon": 61, "Rize": 53, "Ankara": 6}
4 print(plates, len(plates), plates["Trabzon"])
5
```



GitHub

```

6 plates["Giresun"] = 28          # anahtar yoksa ekler
7 plates["Rize"] = 53             # varsa günceller
8 print(plates)
9
10 print("Rize" in plates, "Artvin" in plates, 61 in plates) # in: ANAHTAR
11 print(plates.get("Artvin"), plates.get("Artvin", "kayıt yok"))
12
13 del plates["Rize"]
14 code = plates.pop("Giresun")    # pop silinen değeri döndürür
15 print("Silinen Giresun kodu:", code, "->", plates)
16
17 for city in plates:             # döngü yalnız anahtarları verir
18     print(city, end=" ")
19 print()
20 for city, code in plates.items(): # (anahtar, değer) çiftleri
21     print(f"{city:<8} {code:>3}")
22 print("Kodlar:", list(plates.values()))
23
24 coords = {(41.0, 39.7): "Trabzon"} # anahtar demet de olabilir
25 print(coords[(41.0, 39.7)])

```

```
$ python w14-07-sozluk.py
```

```

{'Trabzon': 61, 'Rize': 53, 'Ankara': 6} 3 61
{'Trabzon': 61, 'Rize': 53, 'Ankara': 6, 'Giresun': 28}
True False False
None kayıt yok
Silinen Giresun kodu: 28 -> {'Trabzon': 61, 'Ankara': 6}
Trabzon Ankara
Trabzon 61
Ankara 6
Kodlar: [61, 6]
Trabzon

```

Kod 14.8'te üç ayrıntı özellikle önemlidir. Birincisi, `in` operatörü **anahtarlar**a bakar: `"Rize" in plates` doğru, `61 in plates` yanlıştır — 61 bir değerdir. Değerlerde aramak isterseniz `plates.values()` üzerinde arayın; aynı karışıklık döngüde de olur, `for x in plates` anahtarları verir. İkincisi, `get` yöntemi olmayan anahtar için hata vermez, `None` (ya da verdiğiniz varsayılanı) döndürür. Üçüncüsü, anahtar ve değeri birlikte istiyorsanız `items()` kullanın ve her adımda demeti açın: `for city, code in plates.items():`.



Sık yapılan hata

`code = plates["Artvin"]` → `code = plates.get("Artvin", 0)`

Sözlükte olmayan bir anahtarı köşeli parantezle okumak programı `KeyError` ile durdurur. Kullanıcıdan gelen ya da var olduğundan emin olmadığınız her anahtar için ya önce `if key in d:` diye sorun ya da `get` kullanın. `get`'in ikinci argümanı “yoksa bunu döndür” demektir; varsayılan verilmezse `None` döner.

14.8.1 Frekans sayma deseni

Sözlüğün en sık kullanılan deseni, bir dizideki değerlerin kaç kez geçtiğini saymaktır: kelime sayacı, harf sayacı, zar yüzü sayacı... Desen iki biçimde yazılır. Birincisinde anahtar var mı diye sorulur, varsa artırılır, yoksa 1 atanır; ikincisinde `get` ile tek satıra inilir:


```
1 counts[item] = counts.get(item, 0) + 1      # yoksa 0'dan başla
```

İkisi de aynı sonucu verir; ikincisi kısadır ve profesyonel kodda daha sık görülür. Sayım bittikten sonra “en çok geçenler” listesi için sözlük `items()` ile demet listesine çevrilir ve sayıya göre sıralanır — yani bir önceki bölümün tekniği.

Örnek 14.5 (Kelime frekansı). Bir metindeki kelimelerin kaç kez geçtiğini sayan, en sık geçen üç kelimeyi yıldızlardan oluşan küçük bir histogramla gösteren ve metinde kaç farklı kelime olduğunu yazan bir program yazın. Sayımı iki biçimde de yapıp sonuçların aynı olduğunu programın kendisi doğrulasın.

Çözüm

Analiz. Girdi: bir metin. Çıktı: frekans sözlüğü, en sık üç kelime ve histogram, farklı kelime sayısı. İşlem: metin `split()` ile kelime listesine ayrılır (5. hafta); sözlükte sayaç tutulur; `items()` demet listesine çevrilip sayıya göre azalan sıralanır.

Algoritma. Kelime listesini gez; her kelime için sayacı bir artır (yoksa sıfırdan başlat). Sonra çiftleri değere göre azalan sırala ve ilk üçünü yaz.

Kod ve çıktı.

Kod 14.9 w14-08-kelime-frekans.py — Frekans sayma deseni ve en sık üç kelime

```
1 # w14-08-kelime-frekans.py -- frekans sayma deseni: metin -> sözlük
2
3 text = "elma armut elma kiraz muz elma armut kiraz elma muz kivi armut"
4 words = text.split()
5
6 counts = {}                                # 1. biçim: önce sor, sonra artır
7 for word in words:
8     if word in counts:
9         counts[word] += 1
10    else:
11        counts[word] = 1
12
13 counts2 = {}                               # 2. biçim: get ile tek satır
14 for word in words:
15     counts2[word] = counts2.get(word, 0) + 1
16
17 print(f"{len(words)} kelime, {len(counts)} farklı kelime")
18 print(counts)
19 print("İki biçim aynı sonucu verir:", counts == counts2)
20
21 # Sözlüğü (kelime, sayı) demetlerine çevirip sayıya göre sırala
22 pairs = sorted(counts.items(), key=lambda kv: kv[1], reverse=True)
23 print("En sık üç kelime:")
24 for word, n in pairs[:3]:
25     print(f" {word:<8} {n:>2} {'*' * n}")
```




```
$ python w14-08-kelime-frekans.py

12 kelime, 5 farklı kelime
{'elma': 4, 'armut': 3, 'kiraz': 2, 'muz': 2, 'kivi': 1}
İki biçim aynı sonucu verir: True
En sık üç kelime:
elma      4  ****
armut     3  ***
kiraz     2  **
```

Kontrol. Metinde 12 kelime vardır ve bunların 5'i farklıdır. Sayıların toplamı kelime sayısına eşit olmalıdır: $4 + 3 + 2 + 2 + 1 = 12$ ✓. En sık kelime **elma** (4 kez); metne bakarak elle sayıldığında da 4 bulunur ✓. Programın kendi kontrolü de geçer: iki farklı sayma biçimi birbirinin aynı sözlüğünü üretir (**True**).

14.8.2 İç içe yapılar

Sözlüğün değeri her şey olabilir — liste de, sözlük de. Bu, iki yaygın kalıp doğurur. **Sözlük listesi** kayıtları alan adlarıyla tutar: `{"ad": "Ayşe", "no": 2601}` kaydında alanlar indisle değil adla okunur, dolayısıyla sıra değişse de kod bozulmaz. **Liste değerli sözlük** ise grupta içindir: bir güne birden çok ders, bir yazara birden çok kitap düşer.

Kod 14.10 w14-09-ic-ice-yapilar.py — Sözlük listesi (öğrenci kayıtları) ve liste değerli sözlük (ders programı)

```
1 # w14-09-ic-ice-yapilar.py -- sözlük listesi ve liste değerli sözlük
2
3 students = [
4     {"ad": "Ayşe", "no": 2601, "notlar": [85, 90, 78]},
5     {"ad": "Mehmet", "no": 2602, "notlar": [60, 72, 65]},
6     {"ad": "Zeynep", "no": 2603, "notlar": [95, 88, 100]},
7 ]
8
9 for student in students:                # her kayıt bir sözlük
10     grades = student["notlar"]
11     print(f"{student['no']:<6}{student['ad']:<9}"
12           f"{sum(grades) / len(grades):>7.2f}")
13
14 for student in students:                # bütün kayıtlara yeni bir alan
15     student["durum"] = "geçti" if min(student["notlar"]) >= 60 else "kaldı"
16 print(students[1])
17
18 schedule = {"Pazartesi": ["Matematik", "Fizik"],
19             "Salı": ["Programlama"],
20             "Çarşamba": []}
21 schedule["Salı"].append("Lineer Cebir") # değer bir liste: yerinde ekle
22
23 total_lessons = 0
24 for day, lessons in schedule.items():
25     total_lessons += len(lessons)
26     text = ", ".join(lessons) if lessons else "(ders yok)"
27     print(f"{day:<10}: {text}")
28 print("Haftalık ders sayısı:", total_lessons)
```



GitHub

```
$ python w14-09-ic-ice-yapilar.py
```

```
2601 Ayşe      84.33
2602 Mehmet    65.67
2603 Zeynep     94.33
{'ad': 'Mehmet', 'no': 2602, 'notlar': [60, 72, 65], 'durum': 'geçti'}
Pazartesi : Matematik, Fizik
Salı      : Programlama, Lineer Cebir
Çarşamba : (ders yok)
Haftalık ders sayısı: 4
```

Kod 14.10’de üç teknik bir arada: her kayıttan bir liste çekmek; bütün kayıtlara *sonradan* yeni bir alan eklemek; ve bir günün ders listesine `append` ile eleman eklemek. Sonuncusu yalnız *o anahtarın değeri zaten bir liste olduğu için* çalışır; anahtar yoksa önce boş liste atanmalıdır:

```
1 if day not in schedule:
2     schedule[day] = []           # önce boş liste
3     schedule[day].append(lesson) # sonra ekle
```

Bu araçlarla artık kullanılabilir programlar yazabilirsiniz: deposundaki `w14-12-rehber.py`, menü döngüsünü (8. hafta) ve fonksiyon düzenini (11. hafta) bu haftanın sözlüğüyle birleştiren küçük bir telefon rehberidir; laboratuvarıda onu genişleteceksiniz.

14.9 Küme (set) ve hangi yapıyı ne zaman seçmeli

Bazen bir veriden tek istediğiniz şey “hangi farklı değerler var?” ya da “bu değer var mı?” sorusunun cevabıdır. Sıra da, eşleşen bir değer de gerekmez.

Tanım 14.4: Küme

Küme (*İng. set*), **yinelenen eleman içermeyen, sırasız** bir topluluktur. Süslü parantezle yazılır (`{3, 7, 9}`) ya da bir diziden üretilir (`set(liste)`). İndislenemez ve dilimlenemez; yapabileceğiniz şeyler üyelik sorgusu (`in`), ekleme (`add`), çıkarma (`discard`) ve küme işlemleridir: birleşim `|`, kesişim `&`, fark `-`, simetrik fark `^`. Boş küme `set()` ile yazılır — `{}` boş *sözlüktür*.

Kod 14.11 w14-10-kume.py — Küme: benzersizlik ve küme işlemleri

```
1 # w14-10-kume.py -- küme: benzersiz elemanlar ve küme işlemleri
2
3 numbers = [3, 7, 3, 1, 7, 7, 9, 1]
4 unique = set(numbers)           # yinelenenler bir kez kalır
5 print(unique, len(unique), sorted(unique))
6 print("7 var mı?", 7 in unique, " 5 var mı?", 5 in unique)
7
8 morning = {"Ayşe", "Mehmet", "Zeynep", "Can"}
9 evening = {"Zeynep", "Can", "Elif"}
10 print("Birleşim:", sorted(morning | evening))
11 print("Kesişim :", sorted(morning & evening))
12 print("Fark     :", sorted(morning - evening))
13 print("Simetrik:", sorted(morning ^ evening))
14
15 morning.add("Deniz")
16 morning.discard("Ayşe")         # yoksa hata vermez
```



GitHub

```

17 print("Sabah grubu:", sorted(morning))
18
19 text = "veri veri model model model tahmin"
20 print(f"{len(text.split())} kelime, {len(set(text.split()))} farklı")
21
22 print(type(set()), type({}))          # boş küme set(); {} sözlüktür

```

```
$ python w14-10-kume.py
```

```

{1, 3, 9, 7} 4 [1, 3, 7, 9]
7 var mı? True 5 var mı? False
Birleşim: ['Ayşe', 'Can', 'Elif', 'Mehmet', 'Zeynep']
Kesişim : ['Can', 'Zeynep']
Fark    : ['Ayşe', 'Mehmet']
Simetrik: ['Ayşe', 'Elif', 'Mehmet']
Sabah grubu: ['Can', 'Deniz', 'Mehmet', 'Zeynep']
6 kelime, 3 farklı
<class 'set'> <class 'dict'>

```

Kümenin en sık kullanımı tek satırlıktır: `len(set(words))` bir metindeki farklı kelime sayısını verir, `sorted(set(numbers))` bir listedeki yinelenenleri atıp sıralar. Geçen hafta “listeden yinelenenleri çıkarma” problemini döngüyle çözmüştünüz; küme aynı işi tek adımda yapar — ama sırayı korumaz, bu yüzden ikisi birbirinin yerine geçmez.

Yapı	Yazım	Erişim	Değişir mi?	Ne zaman kullanılır
liste	[3, 8, 1]	a[2]	evet	sıra önemli, tekrar serbest
demet	(41.0, 39.7)	p[0]	hayır	birlikte duran sabit alanlar
sözlük	{"Trabzon": 61}	d["Trabzon"]	evet	ada göre arama, sayaç
küme	{3, 8, 1}	x in s	evet	benzersizlik, kesişim/fark
2B liste	[[1, 2], [3, 4]]	m[i][j]	evet	tablo, matris, görüntü

Yapıyı, veriye nasıl ulaşacağınız belirler. Üç soruyu sırayla sorun: ① *Elemana neyle ulaşacağım?* Sıra numarasıyla ise liste, bir adla ise sözlük, satır-sütun ikiliyle ise iki boyutlu liste. ② *Sonradan değişecek mi?* Değişmeyecekse ve birlikte duran birkaç alandan oluşuyorsa (enlem-boylam, ad-puan) demet; hem güvence verir hem sözlük anahtarı olabilir. ③ *Yalnız “var mı?” diye soracak mıyım?* Öyleyse küme: yinelenenleri kendiliğinden atar ve üyelik sorgusu ucuzdur. Bu yapılar birbirinin rakibi değildir; gerçek programlar iç içe kullanır — sözlük listesi, liste değerli sözlük, demet listesi.

Şekil 14.8: Bu haftanın karar tablosu. Yapıyı seçen şey verinin kendisi değil, veriye *nasıl ulaşacağınız*dır.

Şekil 14.8 bu haftanın özetidir ve bir programa başlarken sorulacak üç soruyu sıralar. Sorular birbirini dışlamaz: gerçek programlar bu yapıları iç içe kullanır — bir sınıf çizelgesi “listelerin listesi”, bir öğrenci kaydı “sözlüklerin listesi”, bir ders programı “liste değerli sözlük”, bir envanter “demet değerli sözlük”tür.

LAB Laboratuvar Çalışması 14: Veriyi doğru kaba koymak

Bu haftanın laboratuvarı iki boyutlu listeyi başlayıp sözlükle bitiyor. Her görevin sonunda bir **kontrol ölçütü** verilmiştir; kendi çıktınızı onunla karşılaştırın.

Görev 1. Not çizelgesi. `cizelge.py`: beş öğrenci ve dört sınavlık bir not matrisi oluşturun. Her öğrencinin ortalamasını satır sonunda, her sınavını son satırda, başlıklar hizalı olacak biçimde yazdırın. **Kontrol:** satır ortalamalarının ortalaması ile sütun ortalamalarının ortalaması aynı olmalı.

Görev 2. Boş tahta ve tuzak. `tahta.py`: önce `g = [[0] * 3] * 3` ile bir tahta kurup `g[0][0] = 1` yazın ve basın; ne gördüğünüzü bir yorum satırında açıklayın. Sonra aynı tahtayı liste üreticiyle kurup farkı gösterin. **Kontrol:** ikinci sürümde yalnız bir hücre değişmeli.

Görev 3. XOX kazananı. `xox.py`: `w14-11-tic-tac-toe.py`'yi depodan indirip `play(board, i, j, mark)` fonksiyonu ekleyin: hücre boşsa işareti koysun, doluyorsa uyarı; her hamleden sonra kazananı denetlesin. **Kontrol:** dolu bir hücreye oynamak tahtayı değiştirmemeli.

Görev 4. Telefon rehberi. `rehber.py`: `w14-12-rehber.py`'yi indirip bir “sil” seçeneği ekleyin (`pop` ile; olmayan ad çökertmesin) ve çıkarken kayıtları ada göre sıralı listeletin. **Kontrol:** olmayan bir adı silmeye çalışmak uyarı vermelidir.

Görev 5. Envanter. `envanter.py`: anahtar ürün adı, değeri (`adet`, `fiyat`) demeti olan bir sözlük kurun; menüden giriş, satış, toplam değer ve azalan stok listesi yapılabilir. **Kontrol:** elindekinden fazlasını satmak reddedilmeli.

Görev 6. Kelime frekansı. `frekans.py`: kendi yazdığınız birkaç cümleyi küçük harfe çevirip noktalamayı `replace` ile atın, frekansları sayın ve en sık beş kelimeyi histogramla yazdırın. **Kontrol:** frekansların toplamı, metindeki kelime sayısına eşit olmalı.

Teslim. Görev 1–3'ü ders bitmeden gösterin; dosyaları saklayın — 15. haftada rehber ve envanter programlarını bir metin dosyasıyla *kalıcı* hâle getireceğiz.

Özet

Yapı	Yazım	Not
İki boyutlu liste	<code>m = [[3, 8], [5, 2]]; m[i][j]</code>	<code>len(m)</code> satır, <code>len(m[0])</code> sütun sayısı; <code>m[i]</code> bir listedir
Güvenli oluşturma	<code>[[0] * cols for _ in range(rows)]</code>	<code>[[0]*cols]*rows</code> aynı satırı yineler — kullanmayın
Gezinme	<code>for row in m: + for v in row:</code>	indis gerekiyorsa <code>range(len(m))</code> ; sütun için <code>j</code> sabit
Transpoz, çarpım	<code>t[j][i] = m[i][j];</code> $(ab)[i][j] = \sum_k a[i][k] b[k][j]$	= boyut ters: $r \times c \rightarrow c \times r$; çarpımda birincinin sütunu = ikincinin satırı
Demet	<code>p = (3, 4); p = 3, 4; (5,)</code>	değiştirilemez; paketleme/açma; <code>a, b = b, a</code> ; çoklu dönüş bir demettir
Kayıt listesi	<code>list(zip(names, scores))</code>	<code>sorted(recs, key=lambda r: r[1])</code> ile alan bazlı sıralama
Sözlük	<code>d = {"Trabzon": 61}; d[k] = v</code>	anahtar benzersiz ve değiştirilemez; <code>in</code> anahtara bakar
Okuma, silme, dolma	<code>d.get(k, 0); del d[k], d.pop(k); d.items()</code>	<code>d[k]</code> yoksa <code>KeyError</code> ; <code>for k, v in d.items():</code> demeti açar
Frekans sayma	<code>d[x] = d.get(x, 0) + 1</code>	sonra <code>sorted(d.items(), key=..., reverse=True)</code>
Küme	<code>set(liste); a b, a & b, a - b</code>	yenilenen yok, sırasız; boş küme <code>set()</code>



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w14>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı out/ klasöründedir.

Alıştırmalar

Alıştırma 14.1. ●●● Aşağıdaki matris için istenen değerleri *çalıştırmadan* yazın; sonucunun neden hata verdiğini de söyleyin.

```
1 t = [[4, 9, 2],
2      [3, 5, 7],
3      [8, 1, 6]]
```

`t[0][2], t[2][0], t[-1][1], len(t), len(t[1]), sum(t[1]), t[3][0]`.

Cevap: 2, 8, 1, 3, 3, 15; `t[3][0]` *IndexError* verir — dış listede 0, 1, 2 indisleri vardır, 3 yoktur

Alıştırma 14.2. ●●● Aşağıdaki üç satırın çıktısını tahmin edin ve *neden* böyle olduğunu iki-üç cümleyle açıklayın. Aynı sonucu doğru biçimde elde etmek için tek satırı nasıl değiştirirsiniz?

```
1 grid = [[0] * 3] * 2
2 grid[0][1] = 7
3 print(grid)
```

Cevap: `[[0, 7, 0], [0, 7, 0]]`; * kopya değil aynı iç listeye iki referans üretir.

Doğrusu: `grid = [[0] * 3 for _ in range(2)]`

Alıştırma 14.3. ●●● Üç şubenin dört günlük satışlarını tutan bir matris için her şubenin toplamını satır sonunda, her günün toplamını son satırda yazdıran bir program yazın. Satışlar: `[[120, 95, 140, 110], [80, 130, 90, 160], [200, 175, 185, 190]]`.

Cevap: şube toplamları 465, 460, 750; gün toplamları 400, 400, 415, 460; genel toplam iki yoldan da 1675

Alıştırma 14.4. ●●● `column_means(m)` fonksiyonu yazın: her sütunun ortalamasını bir liste olarak döndürsün. Örnek 14.1'deki not matrisiyle oynayın ve en yüksek ortalamalı sınavı yazdırın.

Cevap: `[77,50; 76,25; 80,75]`; en yüksek ortalama Proje sınavında (80,75)

Alıştırma 14.5. ●●● `min_max_avg(values)` fonksiyonu yazın: bir sayı listesinin en küçüğünü, en büyüğü ve ortalamasını *tek bir demet* olarak döndürsün. `[12, 47, 3, 28, 47, 19]` listesiyle oynayın; dönen değerin türünü ve uzunluğunu da yazdırın.

Cevap: `(3, 47, 26.0)`; tür *tuple*, uzunluk 3; ortalama $156/6 = 26$

Alıştırma 14.6. ●●● Adları ["Ayşe", "Mehmet", "Zeynep", "Can", "Elif", "Burak"], puanları [85, 60, 95, 70, 78, 88] olan altı öğrenci için zip ile kayıt listesi oluşturun; puana göre büyükten küçüğe sıralı bir tablo basın; sınıf ortalamasını ve ortalamanın üstünde kalanları yazdırın.

Cevap: sıralama Zeynep 95, Burak 88, Ayşe 85, Elif 78, Can 70, Mehmet 60; ortalama $476/6 = 79,33$; ortalama üstü Ayşe, Zeynep, Burak

Alıştırma 14.7. ●●● Üç kayıtlık bir ülke-başkent sözlüğü kurun. Sırayla bir kayıt ekleyin, birini güncelleyin, `get` ile hem var olan hem olmayan bir ülkeyi arayın, bir kaydı silin, kalanları alfabetik yazdırın ve başkent→ülke biçiminde *ters* bir sözlük üretin.

Cevap: ekleme sonrası 4 kayıt; olmayan ülke için get varsayılanı döndürür; "Paris" anahtar değil değer olduğu için in ile aranınca False çıkar; silme sonrası 3 kayıt ve ters sözlük de 3 kayıtlıdır

Alıştırma 14.8. ●●● "yapay zeka mühendisliği bölümü" metnindeki harflerin frekansını sayın (boşlukları saymayın). Farklı ve toplam harf sayısını yazdırın; en sık geçen beş harfi histogramla gösterin.

Cevap: 17 farklı harf, toplam 27 harf; en sık harfler a, i ve u (üçer kez); sayıların toplamı boşluksuz metnin uzunluğuna eşittir

Alıştırma 14.9. ●●● Menülü bir envanter programı yazın. Sözlüğün anahtarı ürün adı, değeri (adet, fiyat) demeti olsun. Menü: giriş, satış, toplam envanter değeri, stoğu 10'un altına düşenler ve çıkış. Elde olandan fazla satış reddedilsin. Başlangıç: {"kalem": (120, 7.5), "defter": (45, 32.0), "silgi": (8, 4.25)}.

Cevap: başlangıç envanter değeri $120 \cdot 7,5 + 45 \cdot 32 + 8 \cdot 4,25 = 2374,00$ TL; 20 kalem satılıp 30 silgi girildikten sonra 2351,50 TL

Alıştırma 14.10. ●●● Bir kare matrisin *sihirli kare* olup olmadığını denetleyen `is_magic(m)` fonksiyonu yazın: bütün satır, sütun ve köşegen toplamaları eşitse sihirlidir. Yukarıdaki Alıştırma 14.1 matrisiyle ve [[1, 2, 3], [4, 5, 6], [7, 8, 9]] ile sınavın; her çizginin toplamını da yazdırın.

Cevap: birinci matris sihirlidir, her çizginin toplamı 15; ikincisi değildir; $n = 3$ için denetlenecek çizgi sayısı $3 + 3 + 2 = 8$

Alıştırma 14.11. ●●● Boyut denetimi yapan bir `multiply(a, b)` fonksiyonu yazın: birincinin sütun sayısı ikincinin satır sayısına eşit değilse `None` döndürsün, eşitse çarpımı hesaplasın. a (2×3) ile b (3×2), a ile birim matris (3×3) ve a ile a çarpımlarını deneyin.

Cevap: $ab = [[58, 64], [139, 154]]$; a ile birim matrisin çarpımı a 'nın kendisi; $a \cdot a$ için boyutlar uymaz, `None`

Alıştırma 14.12. ●●● Aşağıdaki beş durum için hangi veri yapısını seçerdiniz? Her biri için seçiminizi *birer cümleyle* gerekçelendirin. (a) Bir ürünün barkodundan fiyatını bulmak. (b) Bir haftalık sıcaklık ölçümlerini saklamak. (c) 30 öğrencinin 5 dersten aldığı notlar. (d) Bir metinde geçen farklı harfler. (e) Bir şehrin enlem-boylam çifti.

Cevap: (a) sözlük (b) liste (c) iki boyutlu liste (d) küme (e) demet

Hafta 15: Metin Dosyaları: Veriyi Kalıcı Kılmak

Öğrenme çıktıları

Bu haftanın sonunda:

1. bir programın verisini neden ve nasıl kalıcı kıldığını açıklayabilecek, görelili ile mutlak yolu ayırt edebilecekleriniz;
2. `open()` işlevinin `r`, `w`, `a` ve `x` modlarını doğru seçebilecek, `with` deyimiyle dosyayı güvenli açıp kapatabilecekleriniz;
3. bir metin dosyasına `write()` ve `print(..., file=)` ile yazabilecek; `read`, `readline`, `readlines` ve dosya üzerinde `for` ile okuyabilecekleriniz;
4. dosyadan okunan satırları `strip()` ve `split()` ile ayrıştırıp sayıya çevirebilecek, liste ve sözlük işlemleriyle istatistik üretebilecekleriniz;
5. CSV dosyalarını hem elle hem `csv` modülüyle okuyup yazabilecek, JSON ile bir sözlüğü diske kaydedebilecekleriniz;
6. `FileNotFoundError` gibi dosya hatalarını `try-except-else-finally` ile yakalayabilecek ve “oku-değiştir-geri yaz” desenini uygulayabilecekleriniz.

Anahtar kavramlar

kalıcılık (*İng. persistence*) **dosya** (*İng. file*) **metin dosyası** (*İng. text file*) **dosya yolu** (*İng. file path*) **görelili yol** (*İng. relative path*) **mutlak yol** (*İng. absolute path*) **çalışma dizini** (*İng. working directory*) **dosya modu** (*İng. file mode*) **tampon** (*İng. buffer*) **satır sonu** (*İng. newline*) **ekleme modu** (*İng. append mode*) **günlük** (*İng. log*) **CSV** (*İng. comma-separated values*) **ayrıştırma** (*İng. parsing*) **kodlama** (*İng. encoding*) **JSON**

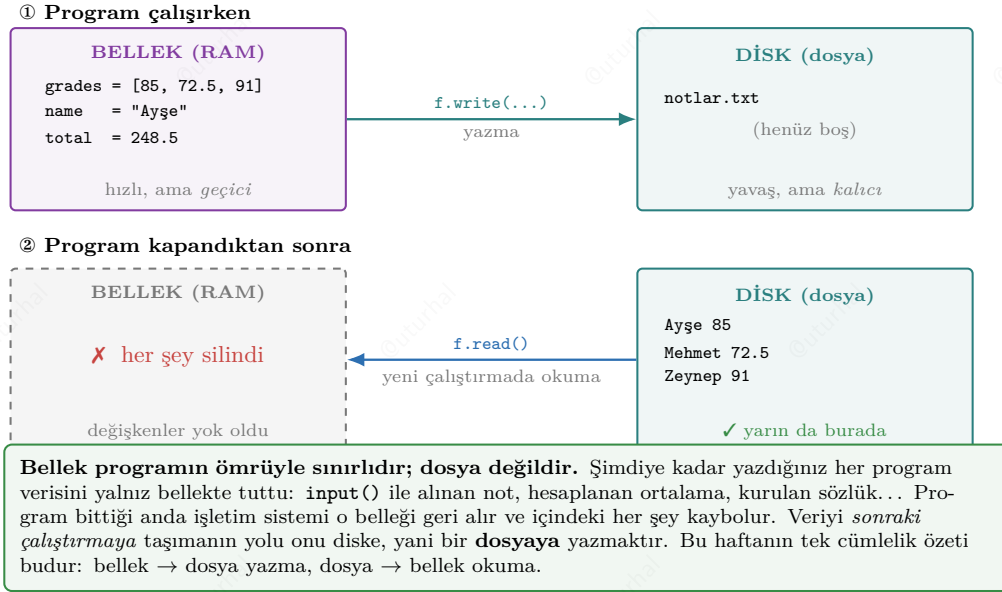
On dört haftadır yazdığınız her program aynı sessiz kusuru paylaşıyordu: çalışırken her şeyi biliyor, kapandığında hiçbir şey hatırlamıyordu. Geçen hafta kurduğunuz telefon rehberi, envanter ve not çizelgesi programları buna en iyi örnektir — onlarca kayıt girersiniz, program biter, kayıtlar biter. Çünkü bir programın değişkenleri *bellekte* durur ve bellek, işletim sistemi tarafından program bittiği anda geri alınır. Gerçek bir uygulamayı ödev programından ayıran şey büyük ölçüde budur: gerçek uygulama, verisini bir sonraki çalıştırmaya taşır.

Bu hafta o köprüyü kuruyoruz. Metin dosyası en basit ve en taşınabilir kalıcı saklama biçimidir: herhangi bir düzenleyiciyle açılır, herhangi bir dille okunur, otuz yıl sonra da okunabilir. Yapay zeka mühendisliği açısından da somut bir anlamı vardır — bir modeli eğitecek veri kümesi size neredeyse her zaman bir metin ya da CSV dosyası olarak gelir; ikinci sınıfta öğreneceğiniz `pandas`’ın `read_csv` işlevi, bu haftaki `open` ve `split` işlerini sizin yerinize yapan kalın bir sarmalayıcıdan ibarettir.

15.1 Neden dosya: bellek geçici, disk kalıcı

Tanım 15.1: Kalıcılık ve dosya

Kalıcılık (*İng. persistence*), bir verinin onu üreten program bittikten sonra da var olmaya devam etmesidir. **Dosya** (*İng. file*), işletim sisteminin diskte tuttuğu, bir adı ve bir içeriği olan bayt dizisidir. İçeriği insanın okuyabileceği karakter dizilerinden oluşuyorsa **metin dosyası** (*İng. text file*), ham baytlardan oluşuyorsa **ikili dosya** (*İng. binary file*) denir. Bu hafta yalnız metin dosyalarıyla çalışacağız.



Şekil 15.1: Program çalışırken veri bellekte, program kapandıktan sonra yalnız diskte kalır. Yazma bellekten diske, okuma diskten belleğe taşır; bu haftanın tamamı bu iki oktan ibarettir.

Şekil 15.1'deki iki ok bu haftanın iki temel işlemidir. Soldaki kutunun içindekiler size tanıdık; sağdaki kutuda ise *tür* yoktur, diskte yalnız karakterler vardır. Bu ayrıntı haftanın en önemli kuralını doğurur: dosyaya yazarken her şey metne çevrilir, dosyadan okurken her şey metin olarak gelir — sayı istiyorsanız `int()` ya da `float()` ile siz çevirirsiniz.

Not

Metin dosyası ile ikili dosyanın farkı uzantı değil içeriktir. `.txt`, `.csv`, `.py`, `.json` metin dosyalarıdır; `.jpg`, `.mp3`, `.pdf`, `.xlsx` ikili dosyalardır — Not Defteri'nde açarsanız anlamsız işaretler görürsünüz. Python ikili dosyaları da açabilir ("`rb`", "`wb`"), ama bu dersin kapsamı dışındadır.

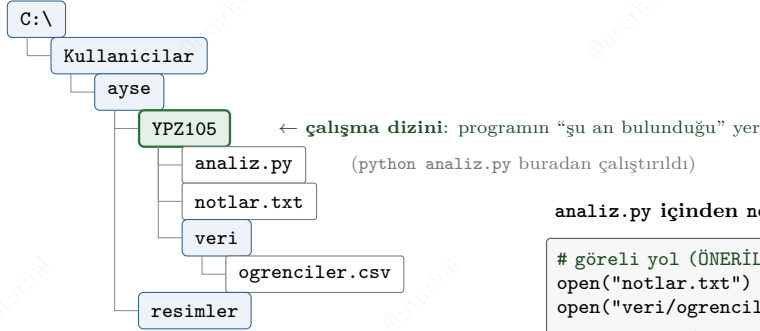
15.2 Dosya yolu, görelî yol ve çalışma dizini

Bir dosyayı açmak için önce onu bulmak gerekir. Birinci haftada klasör ağacından ve terminalde `$ cd` komutundan söz etmiştik; şimdi aynı kavramlar Python'un içine giriyor.

Tanım 15.2: Dosya yolu, görelî ve mutlak

Dosya yolu (*İng. path*), bir dosyanın klasör ağacındaki adresidir. **Mutlak yol** (*İng. absolute path*) kökten başlar (`C:\Kullanıcılar\ayşe\notlar.txt` ya da `/home/ayşe/notlar.txt`) ve her yerden aynı dosyayı gösterir. **Görelî yol** (*İng. relative path*) ise **çalışma dizininden** (*İng. working directory*) başlar: "`notlar.txt`" çalışma dizinindeki dosya, "`veri/ogrenciler.csv`" oradaki bir alt klasördeki dosya, "`../ortak.txt`" bir üst klasördeki dosyadır. Çalışma dizinini `os.getcwd()` söyler.

Klasör ağacı



analiz.py içinden notlar.txt

```

# görelî yol (ÖNERİLEN)
open("notlar.txt")
open("veri/ogrenciler.csv")

# mutlak yol (taşınmaz)
open("C:/Kullanıcılar/ayse/YPZ105/notlar.txt")

# bir üst klasöre çıkmak
open("../ortak.txt")

```

- ① Görelî yol çalışma dizininden başlar.
- ② Mutlak yol kökten (C:\ ya da /) başlar.
- ③ os.getcwd() çalışma dizinini söyler.

"Dosyayı bulamıyor" hatalarının çoğu yol hatasıdır, yazım hatası değil.

`open("notlar.txt")` yazdığınızda Python dosyayı betiğin yanında değil, *çalışma dizininde* arar; IDLE'da betiği kaydettiğiniz klasör, terminalde ise `cd` ile girdiğiniz klasör budur. Mutlak yol yazmak sorunu bilgisayarınızda çözer ama programı taşıyamaz kılar: aynı dosya arkadaşınızın bilgisayarında başka bir yoldadır. Bu derste veri dosyalarını betiğin yanında tutup **görelî yol** kullanacağız.

Şekil 15.2: Aynı dosyaya iki farklı adresle ulaşmak. Görelî yol çalışma dizinine (yeşil klasör) bağlıdır; mutlak yol bağlı değildir ama programı başka bir bilgisayara taşıyamazsınız.

Şekil 15.2'nin anlattığı ayrım tek bir soruya iner: *program hangi klasörden çalıştırıldı?* IDLE'da `F5` ile bu, betiği kaydettiğiniz klasördür; terminalde `$ cd` ile girdiğiniz klasör. Veri dosyalarınızı betiğin yanında tutmak en az sürprizli yoldur — bu kitabın `code/w15/` klasöründe de öyle yapılmıştır.

İpucu

Ters bölü (`\`) tuzağına dikkat: Python'da ters bölü kaçış karakteridir, `"C:\notlar.txt"` içindeki `\n` bir satır sonu olur. Üç çözüm: düz bölü (`"C:/veri/notlar.txt"` — Windows'ta da çalışır), ikilenmiş ters bölü ya da ham metin (`r"C:\veri"`). En kolayı düz bölüdür.

Sık yapılan hata

`open("C:/Users/ayse/Desktop/notlar.txt")` → `open("notlar.txt")`

Mutlak yol yazmak programı *yalnız sizin bilgisayarınızda* çalışır kılar. Arkadaşınıza gönderdiğinizde, laboratuvar bilgisayarında açtığınızda ya da kendi klasörünüzün adını değiştirdiğinizde `FileNotFoundError` alırsınız. Veri dosyasını betiğin yanına koyup görelî yol kullanın; bu dersin bütün ödev ve projelerinde bu beklenir.

15.3 Dosyayı açmak: modlar, close ve with

Tanım 15.3: open() işlevi

`open(yol, mod, encoding=...)` bir dosyayı açar ve üzerinde okuma/yazma yapılabilen bir *dosya nesnesi* döndürür. **Mod** (*İng. mode*), dosyanın ne amaçla açıldığını söyleyen kısa bir metindir: **"r"** okuma (varsayılan), **"w"** yazma, **"a"** sona ekleme, **"x"** yalnız yoksa oluşturma. **encoding** ise karakterlerin baytlara nasıl çevrileceğini belirler; bu derste her zaman **"utf-8"** yazacağız.

Mod seçimi masum bir ayrıntı değildir: yanlış mod veri kaybettirir. [Şekil 15.3](#) dört modun dosya varken ve yokken ne yaptığını karşılaştırır.

Mod	Ne yapar	Dosya VARSA	Dosya YOKSA	İmleç başta nerede
"r"	okur	içeriği okunur	FileNotFoundError	en başta
"w"	yazar	içerik SİLİNİR	yeni dosya açılır	en başta (boş dosya)
"a"	sona ekler	içerik korunur	yeni dosya açılır	en sonda
"x"	yazar	FileExistsError	yeni dosya açılır	en başta

Aynı dosya, üç farklı mod: önce "Ali" yazılmış bir dosya var

```
# mod "w"
f.write("Veli\n")
Sonuç dosya:
Veli
```

```
# mod "a"
f.write("Veli\n")
Sonuç dosya:
Ali
Veli
```

```
# mod "x"
f.write("Veli\n")
Sonuç:
FileExistsError
```

Modu seçmek, dosyanın kaderini seçmektir. "w" ile açılan bir dosya *o anda*, daha siz tek bir karakter yazmadan boşaltılır; dolayısıyla var olan bir günlüğe, sayaca ya da kayıt dosyasına "w" ile dokunmak eski verinin tamamını yok eder. Eklemek istiyorsanız "a", kazayla üzerine yazmaktan korunmak istiyorsanız "x" kullanın. Okuma ("r") varsayılan moddur: `open("a.txt")` ile `open("a.txt", "r")` aynı şeydir.

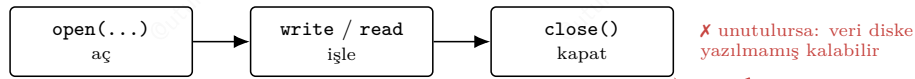
Şekil 15.3: Dört açma modu. "w" dosyayı açar açmaz boşaltır — dönemin en pahalı tek harfidir.

⚠ Dikkat

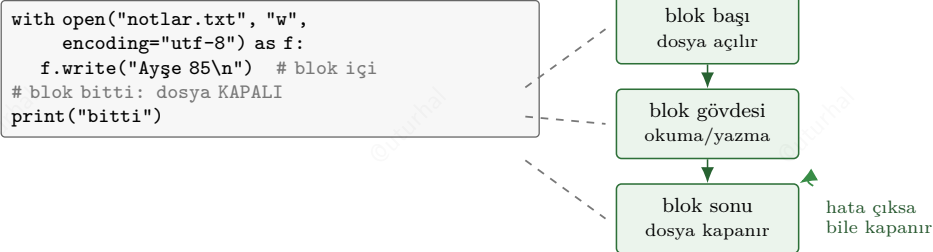
`open("notlar.txt", "w")` satırı, siz hiçbir şey yazmasanız bile dosyanın eski içeriğini **siler**. Var olan bir dosyaya dokunmadan önce hangi modda açtığınızı iki kez okuyun. Eklemek istiyorsanız "a", var olanı kazayla ezmek istiyorsanız "x" kullanın; "x" dosya varsa **FileExistsError** verir ve veriniz kurtulur.

Açılan her dosya kapatılmalıdır. Kapatma, yalnız bir nezaket kuralı değildir: Python yazdıklarınızı önce bir **tampona** (*İng. buffer*) biriktirir ve tampon dolduğunda ya da dosya kapandığında diske aktarır. Kapatmayı unutursanız son yazdıklarınız diske hiç ulaşmayabilir.

① Elle: open ... close



② with bloğu: kapatma kendiliğinden



Her açılan dosya kapatılmalıdır; with bunu sizin yerinize yapar. Python yazdıklarımızı önce bir *tampona* (buffer) biriktirir ve tampon dolduğunda ya da dosya kapandığında diske aktarır. `close()` çağrılmazsa — program bir hatayla yarıda kesildiğinde sık olur — son yazılanlar diske hiç ulaşmayabilir. `with` deyimi bloktan *nasıl* çıkıldığından bağımsız olarak dosyayı kapatır; bu yüzden bu derste dosya her zaman `with` ile açılır.

Şekil 15.4: Bir dosyanın yaşam döngüsü. Üstte elle açıp kapatma, altta `with` bloğu: blok nasıl biterse bitsin — normal ya da bir hatayla — dosya kapanır.

Kural 15.1 (Dosya her zaman `with` ile açılır). Bir dosyayı açmanın doğru biçimi

```
with open(yol, mod, encoding="utf-8") as f:
```

deyimidir. Bloğun gövdesi dosya *açıkken* çalışır; blok bittiğinde — normal biterse de, bir hata fırlatırsa da — Python dosyayı kendiliğinden kapatır. Bu yüzden `with` kullandığımızda `f.close()` yazmanız *gerekmez* ve yazmamalısınız. Blok dışında `f` hâlâ bir addır, ama kapalı bir dosyayı gösterir: üzerinde okuma/yazma denemek `ValueError: I/O operation on closed file` verir.

Şekil 15.4'nün üst şeridi *neyin yanlış gidebileceğini*, alt şeridi Kural 15.1'i gösterir; bundan sonraki bütün örneklerde dosya `with` ile açılacaktır.

Kod 15.1 w15-01-yazma.py — Dosyaya yazmanın üç yolu ve `with` bloğu

```

1 # w15-01-yazma.py -- dosyaya yazma: write, print(file=), writelines
2
3 # 1) Klasik yol: aç - yaz - kapat
4 f = open("selam.txt", "w", encoding="utf-8")
5 f.write("Merhaba dosya!\n")
6 f.write("Bu satırı Python yazdı.\n")
7 f.close() # kapatmayı unutmayın!
8
9 # 2) Önerilen yol: with bloğu; blok bitince dosya kendiliğinden kapanır
10 names = ["Ali", "Elif", "Kerem"]
11 with open("isimler.txt", "w", encoding="utf-8") as f:
12     f.write("Ayşe") # write() satır sonu EKLEMEZ
13     f.write("Mehmet\n") # \n'yi biz koyarız
14     print("Zeynep", file=f) # print() satır sonunu kendi ekler
15     f.writelines([name + "\n" for name in names]) # listeyi yazar
16
17 # Yazdıklarımızı geri okuyup denetleyelim
  
```



GitHub

```

18 with open("selam.txt", "r", encoding="utf-8") as f:
19     print(f.read(), end="")
20
21 with open("isimler.txt", encoding="utf-8") as f:    # "r" varsayılandır
22     content = f.read()
23     print(repr(content))                          # \n karakterleri görünür olsun
24     print(content, end="")

```

```
$ python w15-01-yazma.py
```

```

Merhaba dosya!
Bu satırı Python yazdı.
'AyşeMehmet\nZeynep\nAli\nElif\nKerem\n'
AyşeMehmet
Zeynep
Ali
Elif
Kerem

```

Kod 15.1'da dört ayrıntı önemlidir. `f.write()` satır sonu **eklemez**: iki ardışık yazma dosyada AyşeMehmet tek satırını oluşturur. `print("Zeynep", file=f)` aynı işi yapar ama satır sonunu kendisi ekler; ekrana yazan `print()` ile aynı işlevdir, yalnız hedefi değişmiştir. `f.writelines(liste)` listedeki metinleri arka arkaya yazar ve o da satır sonu eklemez. Son olarak `repr()` ile yazdırılan çıktıda `\n` karakterleri *görünür* hâle gelir — dosya sorunlarını ayıklarken bu numarayı çok kullanacaksınız.

❌ Sık yapılan hata

`f.write(count)` → `f.write(str(count))`
`write()` yalnız karakter dizisi kabul eder; bir sayı vererseniz `TypeError: write() argument must be str, not int` alırsınız. Sayıları `str()` ile ya da f-string ile çevirin: `f.write(f"{name} {score}\n")`. Buna karşılık `print(count, file=f)` sayıyı kabul eder, çünkü `print` her değeri kendiliğinden metne çevirir.

⚠ Dikkat

`encoding="utf-8"` yazmayı alışkanlık edinin. Yazmazsanız Python işletim sisteminin varsayılmasını kullanır: Linux ve macOS'ta genellikle UTF-8, Windows'ta Türkçe bölge ayarında cp1254. O zaman bir bilgisayarda yazılan dosya başkasında ÅŸ, Å¼ gibi bozuk karakterlerle açılır ya da `UnicodeDecodeError` verir. “ş, ğ, ı, İ” harflerinin bozulmasının neredeyse tek nedeni budur.

15.4 Dosyayı okumak: dört yol ve `strip()`

Okumanın dört biçimi vardır ve aralarındaki fark, dosyanın *ne kadarını* bir kerede belleğe aldığımızdır.

Kod 15.2 w15-02-okuma.py — Bir metin dosyasını okumanın dört yolu

```

1 # w15-02-okuma.py -- bir metin dosyasını okumanın dört yolu
2
3 # 1) read(): tüm dosya tek bir metin (str) olarak
4 with open("siir.txt", encoding="utf-8") as f:
5     text = f.read()
6     print(type(text), len(text), "karakter")

```



GitHub

```

7 print(text[:33] + "...")
8
9 # 2) readline(): her çağrıda bir satır; satır sonu \n dahildir
10 with open("siir.txt", encoding="utf-8") as f:
11     first = f.readline()
12     second = f.readline()
13 print(repr(first))
14 print(repr(second.strip()))          # strip() \n'yi atar
15
16 # 3) readlines(): tüm satırlar bir listede
17 with open("siir.txt", encoding="utf-8") as f:
18     lines = f.readlines()
19 print(len(lines), "satır; sonuncusu:", repr(lines[-1]))
20
21 # 4) for satır in dosya: en iyi yol (hepsini belleğe almaz)
22 with open("siir.txt", encoding="utf-8") as f:
23     for number, line in enumerate(f, start=1):
24         print(f"{number:2d}: {line.strip()}")

```

```
$ python w15-02-okuma.py
```

```

<class 'str'> 366 karakter
Sabah olur, sis iner Karadeniz'in...
"Sabah olur, sis iner Karadeniz'in tepelerine,\n"
'fındık dalları eğilir sabahın serin nefesinde.'
8 satır; sonuncusu: 'şehir uyur, deniz uyumaz, dalgalar sayar geceyi.\n'
1: Sabah olur, sis iner Karadeniz'in tepelerine,
2: fındık dalları eğilir sabahın serin nefesinde.
3: Bir martı sesi düşer boz denizin üstüne,
4: kayıklar döner limana, ağlar dolu umutla.
5: Çay bahçeleri yeşil bir merdiven gibi yükselir,
6: her basamağında bir emek, bir sabır saklıdır.
7: Akşam olur, ışıklar yanar Boztepe'den bakınca;
8: şehir uyur, deniz uyumaz, dalgalar sayar geceyi.

```

- `f.read()` tamamını tek bir `str` olarak verir; kısa dosyalarda en kolayı, dev dosyada belleği doldurur.
- `f.readline()` bir sonraki satırı verir, dosya sonunda `""` döner; başlık satırını ayrı okumak için uygundur.
- `f.readlines()` bütün satırları bir *liste* yapar; indeksle erişmek ya da sıralamak gerekiyorsa iyidir, yine hepsi bellektedir.
- `for line in f:` satır satır dolaşır, yalnız o anki satırı bellekte tutar — bellek dostudur ve bu dersin varsayılan yoludur.

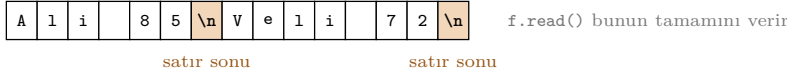
Bir metin dosyasını satır satır işlemenin iskeleti

1. Biriktiricileri döngüden *önce* hazırlayın (`total = 0`, boş liste, boş sözlük).
2. `with open(...) as f:` ile açın, `for line in f:` ile dolaşın.
3. Her satırda önce `strip()`, sonra gerekiyorsa `split()`, en son `int()/float()` uygulayın.
4. Boş ve bozuk satırları `continue` ile atlayın.
5. Sonuçları döngüden *sonra*, blok dışında yazdırın.

Üçüncü adımdaki `strip()`, bu haftanın en sık unutulmuş çağrısıdır: dosyadan gelen her

satırın sonunda görünmez bir satır sonu karakteri vardır (Şekil 15.5).

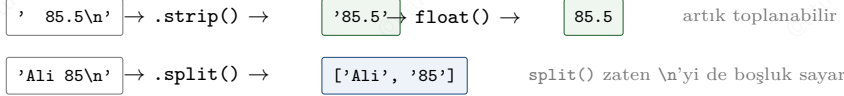
① Diskte dosya, *tek* bir karakter dizisidir — satırları `\n` ayırır



② Satırlara bölündüğünde `\n` hâlâ oradadır



③ `strip()` baştaki ve sondaki boşlukları ve `\n`'yi atar



`float("85.5\n") -> 85.5` # sayıya çevirme sondaki boşluğu hoş görür
`"85\n" == "85" -> False` # karşılaştırma ve sözlük anahtarı görmez

Dosyadan okunan her satırın sonunda görünmez bir `\n` vardır ve dosyadan gelen her şey `str`'dir. Bu iki cümle, bu haftanın hatalarının yarısını açıklar: `print(line)` çıktısını neden çift satır aralıklı yazar (`print` kendi satır sonunu da ekler), iki ad neden eşit çıkmaz, `"85" + 5` neden `TypeError` verir. Kural basittir: *okuduğun satırı önce `strip()`, sonra gerekiyorsa `int()`/`float()` ile çevir.*

Şekil 15.5: Dosya, diskte tek bir karakter dizisidir; satırları `\n` ayırır ve okurken bu karakter satırın içinde kalır.



Sık yapılan hata

`print(line)` -> `print(line.strip())`

Bir dosyayı satır satır yazdırdığınızda çıktının çift aralıklı çıkmasının nedeni budur: satırın kendi `\n`'i *artı* `print`'in eklediği satır sonu. Ya `strip()` uygulayın ya da `print(line, end="")` yazın. Aynı görünmez karakter karşılaştırmaları da bozar: dosyadan okunan `"Ayşe\n"` ile kullanıcının yazdığı `"Ayşe"` eşit değildir ve sözlükte iki ayrı anahtar olurlar.

Bu iskeletin tipik bir uygulaması, 13. haftadaki “en büyüğü bulma” deseninin dosya üzerinde yinelenmesidir: liste yerine dosya dolaşılır, karşılaştırılan büyüklük `len(line)` olur, satır numarasını `enumerate(f, start=1)` verir.

Kod 15.3 `w15-03-en-uzun-satir.py` — En uzun satır: dosya üzerinde “en büyüğü bulma” deseni

```
1 # w15-03-en-uzun-satir.py -- dosyadaki en uzun satırı ve numarasını bulma
2
3 longest = ""
4 longest_no = 0
5 with open("siir.txt", encoding="utf-8") as f:
6     for number, line in enumerate(f, start=1):
7         line = line.strip() # \n uzunluğa katılmasın
8         if len(line) > len(longest):
9             longest = line
10            longest_no = number
11
12 print(f"En uzun satır {longest_no}. satır ({len(longest)} karakter):")
13 print(longest)
```



GitHub

```
$ python w15-03-en-uzun-satir.py
```

```
En uzun satır 8. satır (48 karakter):
şehir uyur, deniz uyumaz, dalgalar sayar geceyi.
```

Kod 15.3'in doğruluğunu elle denetleyelim: sekiz satırın uzunlukları 45, 46, 40, 41, 47, 45, 46 ve 48'dir, en büyüğü 48 ile sonuncusudur ✓. Eşit uzunlukta iki satır olsaydı > karşılaştırması *ilkini* tutardı (>= sonuncusunu); boş bir dosyada döngü hiç dönmez ve program "0. satır" der — hata vermez ama anlamsızdır.

15.5 Ekleme modu ve günlük tutmak

Bazı dosyalar hiç silinmez, yalnız büyür: bir sunucunun olay kaydı, bir programın hata raporu, bir cihazın ölçüm defteri. Bunların tamamı **günlük** (*İng. log*) dosyalarıdır ve hepsi "a" moduyla yazılır.

Kod 15.4 w15-04-gunluk.py — Ekleme modu: zaman damgalı günlük tutma

```
1 # w15-04-gunluk.py -- ekleme ("a") moduyla günlük (log) tutma
2 from datetime import datetime
3
4 LOG_FILE = "gunluk.txt"
5
6
7 def log(message):
8     """Mesajı zaman damgasıyla günlüğün SONUNA ekler."""
9     stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
10    with open(LOG_FILE, "a", encoding="utf-8") as f:
11        f.write(f"{stamp} | {message}\n")
12
13
14 def show_log():
15     """Kayıtları numaralayıp listeler (zaman damgası dosyada kalır)."""
16    with open(LOG_FILE, encoding="utf-8") as f:
17        for number, line in enumerate(f, start=1):
18            stamp, message = line.strip().split(" | ", 1)
19            print(f"{number}. {message}")
20
21
22 while True:
23     text = input("Kayıt (q = bitir): ")
24     if text == "q":
25         break
26     log(text)
27     print(" -> günlüğe eklendi")
28
29 print("Günlükteki kayıtlar:")
30 show_log()
```

```
$ python w15-04-gunluk.py
```

```
Kayıt (q = bitir): Program başlatıldı
-> günlüğe eklendi
Kayıt (q = bitir): Ayşe'nin notu güncellendi
-> günlüğe eklendi
Kayıt (q = bitir): Yedek alındı
-> günlüğe eklendi
```



GitHub


```
Kayıt (q = bitir): q
Günlükteki kayıtlar:
1. Program başlatıldı
2. Ayşe'nin notu güncellendi
3. Yedek alındı
```

Kod 15.4'te üç şey birleşiyor. `log()` dosyayı *her çağrıda* açıp kapatır: program çökse bile o ana kadarki kayıtlar diskte durur. `datetime.now()` ile alınan **zaman damgası** her satırın başına konur; bir günlükte “ne oldu” kadar “ne zaman oldu” da geçerlidir. Okuma tarafında `line.strip().split(" ", 1)` satırı ikiye ayırır — ikinci argüman “en çok bir kez böl” demektir, böylece mesajdaki bir dikey çizgi satırı bölmez.

Örnek 15.1 (Ziyaretçi sayacı). Her çalıştırıldığında kaçınıcı kez çalıştırıldığını söyleyen bir program yazın. Sayı bir dosyada saklansın; dosya ilk çalıştırmada kendiliğinden oluşsun. Program üç kez çalıştırıldığında sırasıyla 1, 2 ve 3 yazmalıdır.

Çözüm

Analiz. Girdi yok; çıktı bir sayı. Durum bilgisi tek bir tam sayıdır ve tek satırlık bir dosyada tutulabilir. İşlem üç adımdır: oku, bir artır, geri yaz. İlk çalıştırmada dosya yoktur; `os.path.exists()` ile bunu önceden sorup sayacı sıfırdan başlatmak gerekir. **Algoritma.** Dosya varsa içindeki metni `int`'e çevir, yoksa sayacı sıfır yap; bir artır; “w” moduyla yeni değeri yaz (tek değer tutulduğu için üzerine yazmak doğrudur); değeri ekrana bas.

Kod ve çıktı.

Kod 15.5 w15-05-sayac.py — Oku-artır-yaz: bir sayıyı dosyada saklamak

```
1 # w15-05-sayac.py -- ziyaretçi sayacı: oku, artır, yaz (dosya yoksa
   ↳ oluştur)
2 import os
3
4 COUNTER_FILE = "sayac.txt"
5
6
7 def visit():
8     """Sayacı bir artırır, yeni değeri dosyaya yazar ve döndürür."""
9     if os.path.exists(COUNTER_FILE):
10         with open(COUNTER_FILE, encoding="utf-8") as f:
11             count = int(f.read().strip())
12     else:
13         count = 0                                # ilk ziyaret: dosya henüz yok
14         count += 1
15         with open(COUNTER_FILE, "w", encoding="utf-8") as f:
16             f.write(str(count))                    # write() sayı kabul etmez: str()
17     return count
18
19
20 # Programın üç kez çalıştırılmasını taklit ediyoruz
21 for run in range(3):
22     print(f"Hoş geldiniz! Siz {visit()}. ziyaretçisiniz.")
23
24 with open(COUNTER_FILE, encoding="utf-8") as f:
25     print("Dosyanın son içeriği:", repr(f.read()))
```



GitHub


```
$ python w15-05-sayac.py
```

```
Hoş geldiniz! Siz 1. ziyaretçisiniz.
Hoş geldiniz! Siz 2. ziyaretçisiniz.
Hoş geldiniz! Siz 3. ziyaretçisiniz.
Dosyanın son içeriği: '3'
```

Kontrol. Program üç kez çalıştırılmış (`for run in range(3)` bunu taklit ediyor) ve çıktı 1, 2, 3 olmuştur ✓; dosyanın son içeriği '3' tek karakteridir. Dosya silinirse sayaç sıfırdan başlar — istenen davranış budur. Burada "w" doğru seçimdir, çünkü dosyada *tek* bir değer tutulur; günlükte durum tam tersiydi.

15.6 Sayısal veriyi okumak, işlemek ve temizlemek

Dosyadan gelen her şey metindir; istatistik yapmak için önce sayıya çevirmek gerekir. Aşağıdaki iki örnek, gerçek veriyle çalışırken hep birlikte gelen iki işi gösterir: hesaplamak ve *bozuk satırlarla başa çıkmak*.

Örnek 15.2 (Not dosyasından istatistik). Her satırında bir ad ve bir puan bulunan `notlar.txt` dosyasını okuyup öğrenci sayısını, puan toplamını, ortalamayı ve en yüksek puanı alan öğrenciyi yazan bir program yazın.

Çözüm

Analiz. Girdi: "Ayşe 85" biçiminde satırlar. Çıktı: sayı, toplam, ortalama ve en yüksek puanın sahibi. Her satır `split()` ile ikiye ayrılır, ikinci parça `float()` ile sayıya çevrilir. Toplam ve sayaç 8. haftanın biriktirici deseni, "en yüksek" ise 13. haftanın en büyüğü bulma deseni; `best_name` ile `best_score` birlikte güncellenir.

Algoritma. Dört biriktirici sıfırlanır (`total`, `count`, `best_score`, `best_name`); her satır için ad ve puan ayrılır, puan sayıya çevrilir, toplama ve sayaca eklenir, puan en iyiden büyükse ikili güncellenir; döngü bitince sonuçlar yazılır.

Kod ve çıktı.

Kod 15.6 w15-06-notlar-istatistik.py — Satır → split → float → istatistik

```

1 # w15-06-notlar-istatistik.py -- notlar.txt: toplam, ortalama, en
  ↪ yüksek
2
3 total = 0.0
4 count = 0
5 best_name = ""
6 best_score = -1.0
7
8 with open("notlar.txt", encoding="utf-8") as f:
9     for line in f:
10         name, score_text = line.split() # "Ayşe 85\n" -> ["Ayşe",
11         ↪ "85"]
12         score = float(score_text)      # dosyadan gelen her şey
13         ↪ metindir
14         total += score
15         count += 1
16         if score > best_score:
17             best_score = score
18             best_name = name
19
20 print(f"Öğrenci sayısı : {count}")
21 print(f"Toplam          : {total:.1f}")
22 print(f"Ortalama        : {total / count:.2f}")
23 print(f"En yüksek      : {best_name} ({best_score})")

```

```
$ python w15-06-notlar-istatistik.py
```

```

Öğrenci sayısı : 8
Toplam          : 593.5
Ortalama        : 74.19
En yüksek      : Selin (94.5)

```

Kontrol. Dosyadaki sekiz puanı elle toplayalım: $85 + 72,5 + 91 + 58 + 66,5 + 79 + 94,5 + 47 = 593,5$ ✓. Ortalama $593,5/8 = 74,1875 \approx 74,19$ ✓; bu değer en küçük (47) ile en büyük (94,5) puanın arasında kaldığı için makuldür ve en yüksek puan 94,5 ile Selin'indir ✓. Büyüklük mertebesi kontrolü: bir ortalama 100'ü aşamaz; 593,5 görseydik *toplamı* ortalama sanmış olurduk.

Kod 15.6'teki `name, score_text = line.split()` satırı, 14. haftanın demet açma yazımıdır. Satırda üç kelime olsaydı ("Ayşe Kaya 85") `ValueError: too many values to unpack` alırdınız; adlarda boşluk olabilen veriler için ya sondan bölün (`line.rsplit(" ", 1)`) ya da CSV kullanın — 15.7'nin var oluş nedeni budur.

Örnek 15.3 (Bozuk veriyi temizlemek). `kirli_veri.txt` dosyasında günlük sıcaklık ölçümleri var; ama dosyada boş satırlar ve sayıya çevrilemeyen satırlar da bulunuyor. Bozuk satırları numarasıyla birlikte bildiren, geçerli olanlardan ortalama, en düşük ve en yüksek sıcaklığı hesaplayan bir program yazın.

Çözüm

Analiz. Girdi: her satırında bir sayı *olması beklenen* bir dosya. Üç satır türü vardır: geçerli sayı, boş satır, çevrilemeyen metin. Boş satır `strip()` sonrası `""` olmasıyla tanınır; bozuk satır ancak `float()` denendiğinde belli olur — 7. haftada tanıdığınız `try-except ValueError` yapısı burada tam yerindedir.

Algoritma. Her satır için: kırp; boşsa boş sayacı artır ve *atla*; değilse `float`'a çevirmeyi dene, başarsan listeye ekle, `ValueError` alırsan bozuk sayacı artır ve satır numarasını bildir. Döngü bitince `sum`, `min`, `max` ile istatistik.

Kod ve çıktı.

Kod 15.7 w15-07-temizleme.py — Veri temizleme: boş satır, bozuk satır ve try-except

```
1 # w15-07-temizleme.py -- kirli veri: boş satırları ve hatalı sayıları
   ↪ atla
2
3 values = []
4 blank = 0
5 bad = 0
6
7 with open("kirli_veri.txt", encoding="utf-8") as f:
8     for number, line in enumerate(f, start=1):
9         text = line.strip()
10        if text == "":                # boş satır
11            blank += 1
12            continue
13        try:
14            values.append(float(text))
15        except ValueError:            # sayıya çevrilemeyen satır
16            bad += 1
17            print(f" {number}. satır atlandı: {repr(text)}")
18
19 print(f"Geçerli: {len(values)}, boş: {blank}, hatalı: {bad}")
20 print(f"Ortalama sıcaklık: {sum(values) / len(values):.2f} °C")
21 print(f"En düşük: {min(values)}, en yüksek: {max(values)}")
```

```
$ python w15-07-temizleme.py
```

```
4. satır atlandı: 'abc'
7. satır atlandı: 'hata'
Geçerli: 6, boş: 2, hatalı: 2
Ortalama sıcaklık: 24.40 °C
En düşük: 22.8, en yüksek: 26.3
```

Kontrol. Dosyada on satır var: altısı geçerli, ikisi boş, ikisi (`abc` ve `hata`) çevrilemez ✓. Geçerli değerlerin toplamı 146,4, ortalama $146,4/6 = 24,4$ ✓; en düşük 22,8, en yüksek 26,3 ✓ ve ortalama ikisinin arasında kaldığına göre hesap tutarlıdır. Bir ayrıntı: dosyadaki `" 26.3 "` satırının başında ve sonunda boşluk vardır — `strip()` olmasaydı boşluklu bir boş satır “bozuk” sayılırdı.

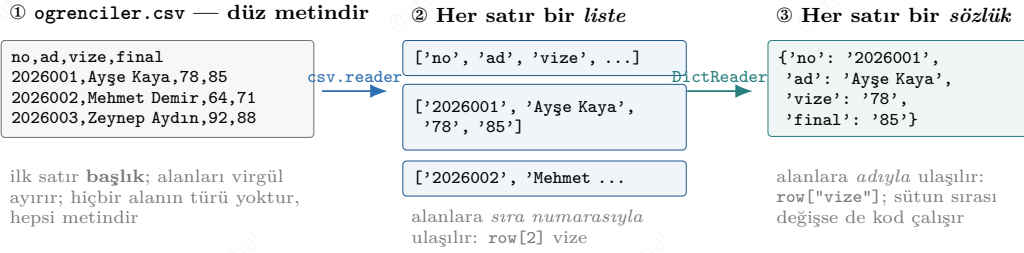
15.7 CSV dosyaları

Bir satırda birden çok alan tutmak gerektiğinde metin dosyasının en yaygın biçimi CSV'dir. Elektronik tablo programları, veri tabanları ve makine öğrenmesi veri kümeleri hep bu bi-

çimde alışverişe girer.

Tanım 15.4: CSV

CSV (*İng. comma-separated values*), her satırı bir kayıt, her kaydı virgülle ayrılmış alanlar olarak tutan düz metin biçimidir. İlk satır genellikle **başlık satırıdır** (*İng. header*) ve sütun adlarını verir. Bir alanın içinde virgül geçiyorsa o alan çift tırnak içine alınır. Uzantısı **.csv**'dir, ama dosya sıradan bir metin dosyasıdır.



Virgül içeren alan: elle split(",") neden yetmez

```
2026007,"Kaya, Ayşe",78,85
```

✗ `line.split(",")` → 5 alan: ad ikiye bölündü
 ✓ `csv.reader` → 4 alan: tırnak içi virgül korunur

CSV, düz metin olduğu için her programın açabildiği ortak dildir; ama biçim kuralları görüldüğünden inceliklidir. Alan içinde virgül, tırnak ya da satır sonu geçebilir; bunları tırnaklama kuralları yönetir. Kendi ayrıştırıcınızı yazmak yerine csv modülünü kullanın: bölme, tırnak açma ve satır sonu işlerini o hallederken siz veriye bakarsınız.

Şekil 15.6: Aynı CSV dosyasının üç görünüşü: düz metin, `csv.reader` ile satır listeleri, `csv.DictReader` ile sözlükler. Sağ altta, elle `split(",")` yapmanın neden yetmediği.

Şekil 15.6'nın sol sütunu CSV'yi 5. haftadaki `split` ile de ayrıştırabileceğinizi gösterir; orta ve sağ sütun `csv` modülünün iki yüzüdür: `reader` her satırı bir *liste*, `DictReader` bir *sözlük* olarak döndürür. İkincisinde alanlara adıyla ulaşırsınız, sütunların yeri değişse bile kod çalışır.

Kod 15.8 w15-08-csv-okuy.py — CSV'yi iki yoldan okumak: elle `split` ve `csv` modülü

```
1 # w15-08-csv-okuy.py -- CSV'yi elle (split) ve csv modülüyle okuma; istatistik
2 import csv
3
4 # 1) Elle: her satırı virgülden böl
5 with open("ogrenciler.csv", encoding="utf-8") as f:
6     header = f.readline().strip().split(",")
7     print("Sütunlar:", header)
8     for number, line in enumerate(f):      # başlıktan sonraki satırlar
9         if number < 2:                      # ilk iki kayıt yeter
10             print(line.strip().split(","))
11
12 # 2) csv modülü: bölme işini o yapar; alanlar yine METİN gelir
13 midterms = []
14 finals = []
15 best_name = ""
16 best_final = 0.0
17 with open("ogrenciler.csv", encoding="utf-8", newline="") as f:
18     reader = csv.reader(f)
19     next(reader)                            # başlık satırını atla
```



GitHub

```

20     for row in reader:
21         no, name, midterm, final = row
22         midterms.append(float(midterm))
23         finals.append(float(final))
24         if float(final) > best_final:
25             best_final = float(final)
26             best_name = name
27
28     print(f"Vize ortalaması : {sum(midterms) / len(midterms):.2f}")
29     print(f"Final ortalaması: {sum(finals) / len(finals):.2f}")
30     print(f"En yüksek final : {best_name} ({best_final:.0f})")

```

```
$ python w15-08-csv-okuy.py
```

```

Sütunlar: ['no', 'ad', 'vize', 'final']
['2026001', 'Ayşe Kaya', '78', '85']
['2026002', 'Mehmet Demir', '64', '71']
Vize ortalaması : 73.67
Final ortalaması: 76.17
En yüksek final : Zeynep Aydın (88)

```

Kod 15.8'nun çıktısında dikkat edilecek nokta, alanların *yine metin* olarak gelmesidir: `csv` modülü bölme işini yapar, tür dönüşümünü yapmaz. `float(midterm)` çağrıları bu yüzden kaçınılmazdır. Ayrıca `next(reader)` satırı başlık satırını atlar; bu yazım, 13. haftadaki “listenin ilk elemanını atla” fikrinin dosya üzerindeki karşılığıdır.



Dikkat

`csv` modülüyle açılan her dosyada `newline=""` yazın: `open("veri.csv", encoding="utf-8", newline="")`. Yazmazsanız Windows'ta ürettiğiniz dosyada satırlar arasında birer boş satır oluşur (satır sonu dönüşümü iki kez uygulanır). Bu derste ayrıca `lineterminator="\n"` veriyoruz ki çıktılar her işletim sisteminde aynı olsun.

Örnek 15.4 (CSV'den sonuç dosyası üretmek). `ogrenciler.csv` dosyasında öğrenci numarası, ad, vize ve final notu var. Ağırlıklı ortalamayı (%40 vize, %60 final) hesaplayıp numara, ad, ortalama ve geçti/kaldı bilgisini içeren yeni bir `sonuclar.csv` dosyası üreten bir program yazın.

Çözüm

Analiz. Girdi bir CSV, çıktı başka bir CSV. Alanlara adıyla ulaşmak istediğimiz için okumada `DictReader`, yazmada `writer` kullanacağız. Ağırlıkları 12. haftadaki gibi büyük harfli sabitlerde tutuyoruz. Geçme ölçütü 60'tır ve 6. haftadaki koşullu ifadeyle tek satırda yazılır.

Algoritma. Her kayıt için ortalamayı hesapla, durumu belirle, dört alanlı bir satırı listeye ekle; sonra başlık satırını ve biriken satırları yeni dosyaya yaz; son olarak yazdığını geri okuyup göster.

Kod ve çıktı.

Kod 15.9 w15-09-csv-yaz.py — DictReader ile oku, writer ile yaz

```

1 # w15-09-csv-yaz.py -- DictReader ile okuma, writer ile yeni bir CSV
  ↪ yazma
2 import csv
3
4 MIDTERM_WEIGHT = 0.4
5 FINAL_WEIGHT = 0.6
6
7 rows = []
8 with open("ogrenciler.csv", encoding="utf-8", newline="") as f:
9     for record in csv.DictReader(f): # her satır bir sözlük
10         average = (float(record["vize"]) * MIDTERM_WEIGHT
11                   + float(record["final"]) * FINAL_WEIGHT)
12         status = "Geçti" if average >= 60 else "Kaldı"
13         rows.append([record["no"], record["ad"], f"{average:.1f}",
14                     ↪ status])
15
16 with open("sonuclar.csv", "w", encoding="utf-8", newline="") as f:
17     writer = csv.writer(f, lineterminator="\n")
18     writer.writerow(["no", "ad", "ortalama", "durum"]) # başlık
19     ↪ satırı
20     writer.writerows(rows)
21
22 # Yazılan dosyayı geri okuyup gösterelim
23 with open("sonuclar.csv", encoding="utf-8") as f:
24     print(f.read(), end="")

```

```
$ python w15-09-csv-yaz.py
```

```

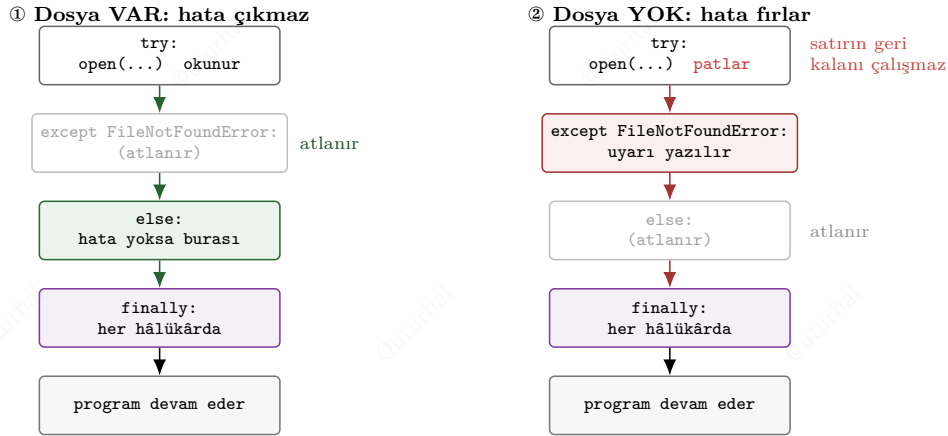
no,ad,ortalama,durum
2026001,Ayşe Kaya,82.2,Geçti
2026002,Mehmet Demir,68.2,Geçti
2026003,Zeynep Aydın,89.6,Geçti
2026004,Ali Çelik,58.0,Kaldı
2026005,Elif Şahin,75.4,Geçti
2026006,Kerem Yılmaz,77.6,Geçti

```

Kontrol. İlk öğrenci: $78 \cdot 0,40 + 85 \cdot 0,60 = 31,2 + 51 = 82,2$ ✓. Ali Çelik: $55 \cdot 0,40 + 60 \cdot 0,60 = 22 + 36 = 58,0 < 60$, yani “Kaldı” ✓; altı öğrencinin beşi geçmiştir. Her ağırlıklı ortalama iki notun arasında kalmalıdır: $55 \leq 58,0 \leq 60$ ✓. Sınır kontrolü: tam 60 alan öğrenci ≥ 60 ölçütüyle geçer — yönetmelik sınırı dışlıyorsa ölçüt > 60 olmalıdır.

15.8 Dosya hatalarını yakalamak

Dosya işlemleri, programınızın *dış dünyayla* konuştuğu yerdir: dosya silinmiş olabilir, adı yanlış yazılmış olabilir, izniniz olmayabilir, disk dolu olabilir. Bu yüzden dosya kodu her zaman hataya hazırlıklı yazılır.



Dört bloğun işi farklıdır. **try:** patlayabilecek satırlar — yalnız onlar, bir tanesi yeter. **except:** o hata çıkarsa ne yapılacağı; hata türünü yazın, çıplak **except:** her şeyi yutar ve gerçek arızayı gizler. **else:** hata çıkmadıysa yapılacaklar; okunan veriyi işleyen kod buraya konur, böylece **try** kısa kalır. **finally:** hata çıksa da çıkmasa da çalışır — temizlik yeri. Yedinci haftada **ValueError** için tanıdığınız bu yapı burada **FileNotFoundError** ile aynı biçimde çalışır.

Şekil 15.7: try-except-else-finally yapısının iki senaryodaki akışı. Sol: hata çıkmaz, except atlanır. Sağ: hata çıkar, try'ın geri kalanı ve else atlanır. finally her iki durumda da çalışır.

Şekil 15.7, 7. haftada **ValueError** için tanıdığınız yapının tam hâlidir. **else** ve **finally** isteğe bağlıdır: **else** “hata çıkmadıysa devam et” der ve **try** bloğunu kısa tutmanızı sağlar, **finally** ise “ne olursa olsun bunu yap” der.

Kod 15.10 w15-10-hata.py — FileNotFoundError ve arkadaşlarını yakalamak

```

1 # w15-10-hata.py -- dosya hatalarını yakalama: try / except / else / finally
2 import os
3
4
5 def read_first_line(filename):
6     """Dosyanın ilk satırını döndürür; sorun varsa None döndürür."""
7     try:
8         with open(filename, encoding="utf-8") as f:
9             first = f.readline().strip()
10    except FileNotFoundError:
11        print(f" HATA: '{filename}' bulunamadı.")
12        first = None
13    except PermissionError:
14        print(f" HATA: '{filename}' için okuma izni yok.")
15        first = None
16    else:
17        print(f" Okundu: {first}")
18    finally:
19        print(f" ({filename} denemesi bitti)")
20    return first
21
22
23 for name in ["siir.txt", "yok.txt"]:
24     print(f"{name}:")
25     result = read_first_line(name)
26     print(" dönen değer:", repr(result))
27

```



GitHub

```

28 # Açmayı denemeden önce sormak: os.path.exists
29 print("notlar.txt var mı?", os.path.exists("notlar.txt"))
30 print("yok.txt var mı?   ", os.path.exists("yok.txt"))

```

```
$ python w15-10-hata.py
```

```

siir.txt:
Okundu: Sabah olur, sis iner Karadeniz'in tepelerine,
(siidr.txt denemesi bitti)
dönen değır: "Sabah olur, sis iner Karadeniz'in tepelerine,"
yok.txt:
HATA: 'yok.txt' bulunamadı.
(yok.txt denemesi bitti)
dönen değır: None
notlar.txt var mı? True
yok.txt var mı?   False

```

Kod 15.10'da iki yaklaşım yan yana duruyor: “önce dene, sonra özür dile” (aç, olmazsa yakala) ve “önce sor” (`os.path.exists("notlar.txt")`). Python topluluğu genellikle birinciyi tercih eder, çünkü kontrol ile açma arasında dosya silinmiş olabilir; ama “dosya yoksa sıfırdan başla” durumlarında `os.path.exists` daha okunaklıdır — Örnek 15.1'te öyle kullanmıştık.

Sık yapılan hata

except: → **except FileNotFoundError:**

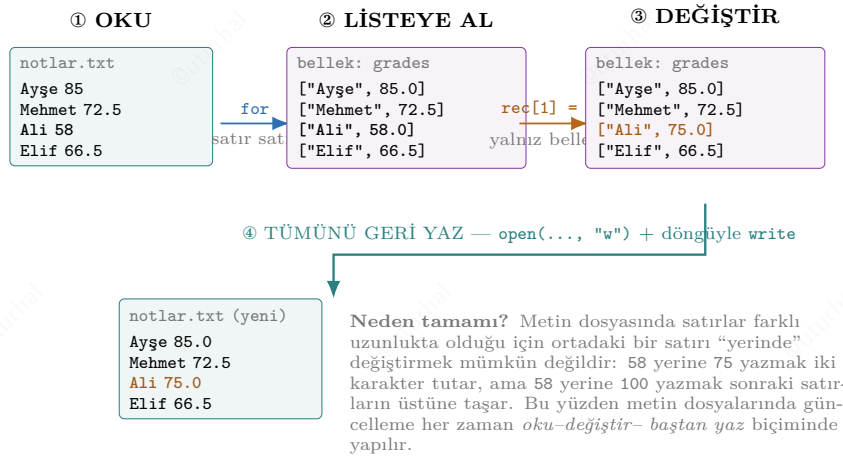
Çıplak **except:** her hatayı yakalar — yazım hatanızı, `KeyboardInterrupt` ile programı durdurma isteğinizi, bellek hatasını da. Program “çalışıyor” görünür ama yanlış çalışır ve gerçek arıza gizlenir. Hangi hatayı beklediğinizi yazın; birden çok tür bekliyorsanız birden çok **except** bloğu ya da **except (FileNotFoundError, PermissionError):** yazın.

Dosya açan bir programı sağlamlaştırmanın üç adımı

1. **Hangi hata?** Okumada `FileNotFoundError`, yazmada `PermissionError`, “x” modunda `FileExistsError`, satırı sayıya çevirirken `ValueError`.
2. **Ne söylenecek?** “Hata oldu” değil, dosyanın adı ve yapılacak iş: ‘notlar.txt’ bulunamadı; dosyayı programın yanına koyun.
3. **Ne yapılacak?** Varsayılanla devam etmek, yeni bir ad istemek ya da temiz bir mesajla durmak. Sessizce yanlış sonuç üretmek seçenek değildir.

15.9 Kayıt güncelleme deseni ve JSON'a bakış

Bir dosyaya sonradan bir şey *eklemek* ile dosyayı *güncellemek* farklı işlerdir. Ekleme ucuzdur: “a” modu imleci sona koyar, yazarsınız, bitti. Güncelleme pahalıdır ve dönemin bütün parçalarını birleştirir; bir kayıt dosyasını güncelleyen her program aynı dört adımı izler (Şekil 15.8).



Dosya, programın başında ve sonunda; iş ortada, bellekte yapılır. Bu desen 13. ve 14. haftadaki liste ve sözlük işlemlerinizi olduğu gibi korur: dosyadan gelen satırları bir listeye çevirirsiniz, bildiğiniz yöntemlerle (arama, `append`, `sort`, sözlük güncelleme) üzerinde çalışırsınız, sonra listeyi baştan yazarsınız. Güncelleme bittikten sonra yazmayı unutursanız değişiklik program kapanınca kaybolur — kaydetmeyi unutulan bir belge gibi.

Şekil 15.8: Oku-listeye al-değiştir-tümünü geri yaz. Metin dosyasında satırlar farklı uzunlukta olduğu için ortadaki bir satır “yerinde” değiştirilemez.

Örnek 15.5 (Kalıcı not defteri). 13. haftanın laboratuvarında yazdığınız not defteri programını kalıcı hâle getirin: program açılışta notları dosyadan okusun; listeleme, ekleme, güncelleme ve ortalama seçenekleri olsun; her değişiklikten sonra dosya güncellensin.

Çözüm

Analiz. Girdi: menü seçimleri ve yeni kayıtlar. Çıktı: ekrana liste ve ortalama, diske güncel dosya. Veri yapısı 13. haftadaki gibi `[ad, puan]` listelerinden oluşan bir listedir; program üç fonksiyona ayrılır (11. hafta): `load_grades`, `save_grades`, `show`. Menü döngüsü 8. haftanın `while True` kalıbıdır.

Algoritma. Başlangıçta `grades = load_grades()`; sonra sonsuz menü döngüsü. Ekleme ve güncelleme seçenekleri önce *listeyi* değiştirir, hemen ardından `save_grades(grades)` ile dosyanın tamamını yeniden yazar; “çık” döngüyü `break` ile bitirir.

Kod ve çıktı.

Kod 15.11 w15-11-notlar.py — Dönemin bütün parçaları bir arada: dosya, liste, fonksiyon, menü



```

1 # w15-11-notlar.py -- kalıcı not dosyası:
  ↪ listele/ekle/güncelle/ortalama
2 GRADES_FILE = "notlar.txt"
3
4
5 def load_grades():
6     """Dosyayı okur; [ad, puan] çiftlerinin listesini döndürür."""
7     grades = []
8     with open(GRADES_FILE, encoding="utf-8") as f:
9         for line in f:
10             name, score = line.split()
11             grades.append([name, float(score)])
12     return grades
13
14
15 def save_grades(grades):
16     """Listenin TAMAMINI yeniden yazar ("w" eskisini siler)."""
17     with open(GRADES_FILE, "w", encoding="utf-8") as f:
18         for name, score in grades:
19             f.write(f"{name} {score}\n")
20
21
22 grades = load_grades()                # 1. adım: dosyadan listeye
23 while True:
24     print("1) Listele 2) Ekle 3) Güncelle 4) Ortalama 5) Çık")
25     choice = input("Seçim: ")
26     if choice == "1":
27         for name, score in grades:
28             print(f" {name:<8}{score:>6.1f}")
29     elif choice == "2":
30         grades.append([input("Ad: "), float(input("Puan: "))])
31         save_grades(grades)            # 2-3. adım: değiştir, geri yaz
32     elif choice == "3":
33         target = input("Kimin notu? ")
34         found = [r for r in grades if r[0] == target]
35         if found:
36             found[0][1] = float(input("Yeni puan: "))
37             save_grades(grades)
38         else:
39             print(" Bulunamadı")
40     elif choice == "4":
41         total = sum(score for name, score in grades)
42         print(f" Ortalama: {total / len(grades):.2f}")
43     elif choice == "5":
44         break
45
46 print("Dosyanın son hâli:")
47 with open(GRADES_FILE, encoding="utf-8") as f:
48     print(f.read(), end="")

```

```
$ python w15-11-notlar.py
```

```
1) Listele 2) Ekle 3) Güncelle 4) Ortalama 5) Çık
Seçim: 3
Kimin notu? Ali
Yeni puan: 75
1) Listele 2) Ekle 3) Güncelle 4) Ortalama 5) Çık
Seçim: 2
Ad: Deniz
Puan: 88
1) Listele 2) Ekle 3) Güncelle 4) Ortalama 5) Çık
Seçim: 4
Ortalama: 77.61
1) Listele 2) Ekle 3) Güncelle 4) Ortalama 5) Çık
Seçim: 5
Dosyanın son hâli:
Ayşe 85.0
Mehmet 72.5
Zeynep 91.0
Ali 75.0
Elif 66.5
Kerem 79.0
Selin 94.5
Burak 47.0
Deniz 88.0
```

Kontrol. Örnek oturumda Ali'nin notu 58'den 75'e çekildi ve Deniz 88 puanla eklendi; dosyada dokuz satır kaldı. Ortalamayı elle doğrulayalım: $85 + 72,5 + 91 + 75 + 66,5 + 79 + 94,5 + 47 + 88 = 698,5$ ve $698,5/9 = 77,611... \approx 77,61$ ✓. Güncellemeden önceki ortalama 74,19 idi; bir öğrencinin notu 17 puan arttığı ve ortalamanın üstünde bir öğrenci eklendiği için yükselmesi beklenir ✓. Uç durumlar: olmayan bir ada güncelleme denemesi "Bulunamadı" der ve *dosyaya dokunmaz*; liste boşken ortalama istenirse `ZeroDivisionError` çıkar — laboratuvarıda bunu denetlemeniz istenecek.

Metin dosyası tek boyutlu veriler için yeterlidir; iç içe yapıları — sözlük içinde liste, liste içinde sözlük — satırlara sığdırmak zahmetlidir. Bunun için **JSON** (İng. *JavaScript Object Notation*) biçimi vardır; Python'da tek satırlık iki işlevle kullanılır.

Kod 15.12 `w15-12-json-ayarlar.py` — Bir sözlüğü JSON olarak kaydetmek ve geri yüklemek

```
1 # w15-12-json-ayarlar.py -- bir sözlüğü JSON dosyasına yazma ve geri okuma
2 import json
3
4 settings = {
5     "kullanici": "Ayşe",
6     "yazi_boyutu": 14,
7     "bildirimler": True,
8     "son_dosyalar": ["notlar.txt"],
9 }
10
11 with open("ayarlar.json", "w", encoding="utf-8") as f:
12     json.dump(settings, f, indent=2, ensure_ascii=False)
13
```



GitHub

```

14 # JSON dosyası düz metindir; içine bakalım
15 with open("ayarlar.json", encoding="utf-8") as f:
16     print(f.read())
17
18 # Geri yükleme: türler korunur (int, bool, list)
19 with open("ayarlar.json", encoding="utf-8") as f:
20     loaded = json.load(f)
21
22 print(type(loaded), loaded == settings)
23 print(loaded["yazi_boyutu"] + 2, type(loaded["bildirimler"]))

```

```
$ python w15-12-json-ayarlar.py
```

```

{
  "kullanici": "Ayşe",
  "yazi_boyutu": 14,
  "bildirimler": true,
  "son_dosyalar": [
    "notlar.txt"
  ]
}
<class 'dict'> True
16 <class 'bool'>

```

Kod 15.12'un çıktısındaki son iki satır önemlidir: `loaded == settings` True'dur (sözlük olduğu gibi geri gelmiştir) ve `loaded["yazi_boyutu"] + 2` aritmetik yapabilmektedir — JSON türleri korur. Düz metin dosyasında ikisi de doğru değildir; orada her şey yeniden metindir. `ensure_ascii=False` Türkçe harflerin dosyada okunabilir kalmasını sağlar; yazılmazsa Ayşe yerine Ay\u015fe görürsünüz.

Not

Hangi biçim ne zaman? **Düz metin:** tek türden, satır satır veriler (günlük, sayaç, ad-puan listesi). **CSV:** satır-sütun tabloları; Excel ve veri analizi araçlarıyla alışveriş. **JSON:** iç içe, adlandırılmış yapılar (ayarlar, kayıtlar, web servisi verisi). Üçü de metin dosyasıdır; farkı *içindeki düzendir*. Veri tabanları (SQL) ileriki yılların konusudur.

LAB Laboratuvar Çalışması 15: Programlarınızı kalıcı kılmak

Dönemin son laboratuvarı bir *bitirme çalışmasıdır*: geçen iki haftada yazdığınız programlara hafıza kazandıracaksınız. Her görevin sonundaki **kontrol ölçütünü** kendi çıktınızla karşılaştırın.

Görev 1. Isınma: çalışma dizini. `nereede.py`: `import os` ile `print(os.getcwd())` yazdırın; aynı klasöre elle bir `deneme.txt` oluşturup programdan okuyun, sonra dosyayı bir üst klasöre taşıyıp `open("../deneme.txt")` ile okuyun. **Kontrol:** iki okuma da aynı metni vermelidir.

Görev 2. Kalıcı not defteri. `w15-11-notlar.py`'yi indirip iki şey ekleyin: (i) ada göre `sil` seçeneği (olmayan ad çökertmesin), (ii) liste boşken ortalama istenirse "kayıt yok" yazsın. **Kontrol:** bütün kayıtları silip ortalama isteyin — program çökmemelidir.

Görev 3. Rehberi kalıcı kılmak. 14. haftadaki telefon rehberi açılışa `rehber.txt`'den okusun, çıkarken kaydedsin; her satır `ad,telefon` biçiminde olsun. **Kontrol:** programı kapatıp açtığınızda kayıtlar yerinde olmalı.

Görev 4. CSV'den sınıf raporu. `ogrenciler.csv`'den ağırlıklı ortalamaları, geçme oranını, en yüksek ve en düşük ortalamayı hesaplayıp ekrana *ve* `rapor.txt`'ye yazın (`print(..., file=f)`). **Kontrol:** sınıf ortalaması 75,17, geçme oranı %83,3.

Görev 5. Veri temizleme. `kirli_veri.txt`'den geçerli satırların ortalamasını, en küçüğünü ve en büyüğünü bulun; atlanan satırları numarasıyla `hatalar.log`'a *ekleme* modunda yazın ve programı iki kez çalıştırın. **Kontrol:** günlük iki katına çıkmalı, ortalama değişmemeli (24,40).

Görev 6. Eksik dosyayı yakalamak. Görev 4'ü olmayan bir dosya adıyla çalıştırın; sonra `try-except FileNotFoundError` ekleyip programın anlaşılır bir mesajla, *çökmeden* bitmesini sağlayın ve `finally` bloğuna “işlem tamamlandı” satırı koyun. **Kontrol:** her iki durumda da son satır yazılmalı.

Teslim. Görev 1–2'yi ders bitmeden gösterin; 3–6 notlandırılmaz ama finalin en iyi hazırlığıdır.

Dönemin sonu

Burası dönemin son öğretim haftasıydı. On beş hafta önce bilgisayarın verileri ikili sayılarla nasıl tuttuğuyla başladık; kâğıtta akış diyagramı çizdik, sonra onları Python'a çevirdik. Sırasıyla değişkenleri ve türleri (4), karakter dizilerini (5), kararları (6–7), döngüleri (8, 10), fonksiyonları (11–12), listeleri, matrisleri, demetleri ve sözlükleri (13–14) öğrendiniz; bu hafta hepsini bir arada kullanıp veriyi diske yazdınız. **Örnek 15.5** bu birleşimin küçük bir portresidir — içinde dosya, liste, fonksiyon, döngü, koşul, f-string ve tür dönüşümü birlikte çalışır. Buradan sonrası iki yöne ayrılır: *derinleşme* (nesne yönelimli programlama, veri yapıları ve algoritmalar, veri tabanları) ve *genişleme* (NumPy, pandas, matplotlib ve makine öğrenmesi kitaplıkları). İkisinin de ön koşulu aynıdır ve bu derste tamamlandı: bir problemi adımlara bölebilmek, o adımları doğru veri yapısıyla ifade edebilmek ve sonucu *denetleyebilmek*.

Final'e hazırlık: dört oturumluk plan

Final, dönemin tamamını (1–15. haftalar) kapsar; ağırlıklı olarak *kod okuma* ve *kâğıtta kısa program yazma* sorularından oluşur.

- 1. Temeller (1–7).** Sayı sistemleri; değişken, tür ve tür dönüşümü; `//` ve `%`; f-string biçimleri; koşullar ve `elif` sırası; De Morgan. Her haftanın özet tablosunu okuyup alıştırmaları cevapları kapatarak yeniden çözün.
- 2. Tekrar ve fonksiyon (8, 10–12).** `while/for` seçimi; biriktirici, sayaç, nöbetçi değer; `range` sınırları; `return` ile `print` farkı; kapsam; varsayılan parametre; özyineleme. En az beş programı *elle izleyin*.
- 3. Veri yapıları ve dosya (13–15).** Liste metotları; takma ad ve kopya; 2B liste; demet; sözlük ve frekans sayma; dosya modları; `strip`; CSV. Bu üç haftanın kodlarını *çalıştırmadan* çıktısını tahmin edin, sonra çalıştırıp karşılaştırın.
- 4. Karma çalışma.** Vize ve beş kısa sınavı yeniden çözün; sonra “dosyadan oku–hesapla–dosyaya yaz” biçiminde kendinize üç problem uydurup çözün — en etkili hazırlık budur.

Kâğıda yazılan kodda girinti, iki nokta üst üste ve tırnak kapatma aranır; Ek C'deki hızlı başvuru sayfasını sınav öncesi son gün okuyun.

Özet

İş	Yazım	Not
Açma (kural)	<code>with open(yol, mod, encoding="utf-8") as f:</code>	blok bitince kendiliğinden kapanır; <code>close()</code> yazmayın
Modlar	"r" oku · "w" yaz · "a" ekle · "x" yoksa oluştur	"w" var olan dosyayı siler ; "r" varsayılandır
Yazma	<code>f.write(metin)</code> · <code>print(deg, file=f)</code>	write satır sonu <i>eklemez</i> ve yalnız str alır
Okuma	<code>f.read()</code> · <code>f.readline()</code> · <code>f.readlines()</code>	sırasıyla: tümü tek metin, bir satır, satır listesi
Satır satır	<code>for line in f:</code>	bellek dostu; tercih edilen yol
Satırı işleme	<code>line.strip()</code> · <code>line.split()</code> · <code>float(parca)</code>	her satırın sonunda <code>\n</code> vardır; dosyadan gelen her şey str 'dir
CSV	<code>csv.reader(f)</code> · <code>csv.DictReader(f)</code> · <code>csv.writer(f)</code>	liste / sözlük döndürür; <code>newline=""</code> ile açın
Dosya var mı	<code>os.path.exists(yol)</code>	"dosya yoksa sıfırdan başla" durumlarında
Hata yakalama	<code>try / except FileNotFoundError / else / finally</code>	çıplak except : yazmayın
Güncelleme deseni	oku → listeye al → değiştir → "w" ile tümünü yaz	metin dosyasında satır yerinde değiştirilemez
JSON	<code>json.dump(d, f)</code> · <code>json.load(f)</code>	türleri korur; <code>ensure_ascii=False</code> ile Türkçe okunur kalır
Yol	görelî "veri/a.csv" · mutlak "C:/.../a.csv"	görelî yol <code>os.getcwd()</code> dizininden başlar



Bu haftanın tüm örnek kodları

<https://github.com/uturhal/YPZ105/tree/main/code/w15>

Dosyaları indirip kendi bilgisayarınızda çalıştırın; her dosyanın yanında beklenen çıktı `out/` klasöründedir.

Alıştırmalar

Alıştırma 15.1. ●●● `sayilar.txt` dosyasında iki satır var: 10 ve 20. Aşağıdaki iki program, *her biri bu başlangıç dosyası üzerinde ayrı ayrı çalıştırılıyor*. Her birinden sonra dosyanın içeriği ne olur? Bir günlük dosyası için hangisini seçersiniz, neden?

(a)

```
1 with open("sayilar.txt", "w", encoding="utf-8") as f:
2     f.write("30\n")
```

(b)

```
1 with open("sayilar.txt", "a", encoding="utf-8") as f:
2     f.write("30\n")
```

*Cevap: (a) dosyada yalnız 30 kalır (1 satır); 10 ve 20 silinir. (b) 10, 20, 30 (3 satır).
Günlük için "a": günlük birikmelidir*

Alıştırma 15.2. ●●● Bir öğrenci dosyayı `with` kullanmadan açıyor ve `close()` çağırmaı unutuyor. Bundan doğabilecek üç ayrı sorunu yazın; sonra aynı kodu `with` ile yeniden yazmanın neden üçünü birden çözdüğünü bir cümleyle açıklayın.

Cevap: (i) tampondaki veri diske yazılmaz, dosya eksik ya da boş kalır; (ii) araya bir hata girerse `close()` satırına hiç ulaşılmaz; (iii) dosya açık kahr, işletim sistemi kaynağı tutulur ve bazı sistemlerde dosya kilitlenir. `with`, bloktan nasıl çıkılırsa çıkılsın dosyayı kapatır

Alıştırma 15.3. ●●● Kullanıcıdan üç şehir adı alıp her birini `sehirler.txt` dosyasına ayrı satır olarak yazan, sonra dosyayı geri okuyup numaralandırarak listeleyen bir program yazın. Girdi Trabzon, Rize, Giresun.

Cevap: Dosya üç satır olur; yazarken `f.write(city + "\n")`, okurken `line.strip()` gerekir. Çıktı: 1. Trabzon, 2. Rize, 3. Giresun

Alıştırma 15.4. ●●● `siir.txt` dosyasının satır, kelime ve karakter sayısını (satır sonları hariç) bulan bir program yazın; ayrıca satır başına ortalama kelime sayısını iki ondalık basamakla yazdırın.

Cevap: 8 satır, 52 kelime, 358 karakter; satır başına ortalama 6,50 kelime

Alıştırma 15.5. ●●● `notlar.txt` dosyasını okuyup kaç öğrencinin geçtiğini (≥ 60) ve kaçının kaldığını yazan; ayrıca harf notu dağılımını ($AA \geq 90$, $BA \geq 85$, $BB \geq 80$, $CB \geq 75$, $CC \geq 70$, $DC \geq 65$, $DD \geq 60$, altı FF) bir sözlükle sayıp yıldızlı bir histogram olarak basan bir program yazın.

Cevap: Geçen 6, kalan 2; dağılım AA 2, BA 1, BB 0, CB 1, CC 1, DC 1, DD 0, FF 2 (toplam 8)

Alıştırma 15.6. ●●● `siir.txt` dosyasının satırlarını ters sırada `ters.txt` dosyasına yazan ve sonucu ekranda numaralandırarak gösteren bir program yazın. Hangi okuma yöntemi bu iş için uygundur, neden?

Cevap: `readlines()` uygundur: satırları listeye alır, `reverse()` listeyi ters çevirir. Sonuçta 8 satır; yeni dosyanın ilk satırı eski dosyanın son satırıdır

Alıştırma 15.7. ●●● `kelimeler.txt` dosyasındaki kelimeleri küçük harfe çevirip noktalamayı temizleyerek sayın; toplam kelime sayısını, farklı kelime sayısını ve en sık geçen beş kelimeyi (eşitlikte alfabetik sırayla) yazdırın. Frekansların toplamının kelime sayısına eşit olduğunu programın kendisi denetlesin.

Cevap: 50 kelime, 31 farklı kelime; en sık beşi veri 7, model 4, ister 3, iyi 3, de 2

Alıştırma 15.8. ●●● Bir CSV dosyasına kayıt ekleyen `add_student(record)` fonksiyonunu yazın: hedef dosya `yeni_ogrenciler.csv` olsun; dosya yoksa önce başlık satırını, varsa yalnız kaydı yazsın (ekleme modu). İki kayıt ekleyin; ikincisinin adında virgül olsun ("Kaya, Ayşe"). Sonra dosyayı hem ham metin olarak hem de `csv.reader` ile

okuyup gösterin.

Cevap: Başlık yalnız bir kez yazılır; virgüllü ad dosyada çift tırnak içine alınır ve csv.reader onu tek alan olarak okur — her satır 4 alandır

Alıştırma 15.9. ●●● `ogrenciler.csv`'den bir sınıf raporu üretip `rapor.txt` dosyasına yazan bir program yazın. Rapor şunları içersin: her öğrencinin adı, ağırlıklı ortalaması (%40 vize, %60 final) ve geçti/kaldı durumu; sonra sınıf ortalaması, geçme oranı, en yüksek ve en düşük ortalama. Dosyaya yazarken `print(..., file=f)` kullanın.

Cevap: Sınıf ortalaması 75,17; geçme oranı %83,3 (6 öğrencinin 5'i); en yüksek Zeynep Aydın 89,6, en düşük Ali Çelik 58,0

Alıştırma 15.10. ●●● `notlar.txt` dosyasında adı verilen öğrencinin puanını güncelleyen bir program yazın: oku, listeye al, değiştir, tümünü geri yaz. Ad dosyada yoksa uyarı versin ve dosyaya hiç dokunmasın. Girdi Burak ve 72.

Cevap: Burak'ın puanı 47'den 72'ye çıkar; dosya yine 8 satırdır. Ad bulunmazsa "w" ile açma adımı hiç çalıştırılmaz

Alıştırma 15.11. ●●● 14. haftadaki telefon rehberini JSON ile kalıcı kılın. İki fonksiyon yazın: biri dosya varsa onu yükleyip sözlük döndürsün, yoksa boş sözlükle başlasın; öteki sözlüğün tamamını `rehber.json` dosyasına yazsın. Ekleme, arama ve silme sözlük üzerinde yapılsın, her değişiklikten sonra kaydedilsin. İki kayıt ekleyip kaydedin, sonra dosyayı yeniden yükleyerek ("ikinci çalıştırma") kayıtların korunduğunu gösterin.

Cevap: İlk yüklemede "0 kayıt", ikinci yüklemede "2 kayıt" yazar; olmayan bir adı silmek `pop(ad, None)` ile çökmeyen ve bir kayıt silinince geriye 1 kayıt kalır

Alıştırma 15.12. ●●● Aşağıdaki üç kod parçasının her biri farklı bir nedenle yanlıştır. Her biri için (i) ne olacağını, (ii) nedenini ve (iii) doğru yazımını yazın.

(a)

```
1 with open("notlar.txt", "w", encoding="utf-8") as f:
2     lines = f.readlines()
```

(b)

```
1 with open("notlar.txt", encoding="utf-8") as f:
2     for line in f:
3         total = total + int(line)
```

(c)

```
1 with open("notlar.txt") as f:
2     text = f.read()
```

Cevap: (a) "w" dosyayı sildi ve yazma için açtı; okuma `io.UnsupportedOperation: not readable` verir — veri zaten gitmiştir, doğrusu "r". (b) satır "Ayşe 85\n" biçimindedir; `int()` `ValueError` verir, önce `split()` gerekir (ayrıca `total` döngüden önce tanımlanmalı). (c) `encoding` yazılmadığı için işletim sisteminin varsayılan kodlaması kullanılır; Türkçe harfler bozulabilir ya da `UnicodeDecodeError` çıkar

Hafta 16: Final Sınavı

Öğrenme çıktıları

Final sınavı dönemin tamamını (1.–15. haftalar) kapsar ve yarıyıl notunun %50'sini oluşturur. Ağırlık, ara sınavdan sonra işlenen konulardadır: döngüler, fonksiyonlar, listeler, demetler, sözlükler ve dosya işlemleri.

Kapsam ve ağırlık

Konu grubu	Haftalar	Yaklaşık ağırlık
Temel kavramlar, algoritma, akış diyagramı	1–3	%10
Değişkenler, türler, metinler, koşullar	4–7	%20
Döngüler (<code>while</code> , <code>for</code> , iç içe)	8, 10	%20
Fonksiyonlar, modüller, özyineleme	11–12	%20
Listeler, 2B listeler, demetler, sözlükler	13–14	%20
Dosya işlemleri, CSV, istisnalar	15	%10

Sınav biçimi

Kâğıt üzerinde, 120 dakika, 100 puan. Kavram ve kod okuma soruları (%30), verilen fonksiyonu tamamlama ya da hatasını bulma (%20), fonksiyon yazma (%20), liste/sözlük/dosya kullanan bir problemin tam çözümü (%30). Son soru, dönem boyunca laboratuvarında geliştirilen “not defteri” uygulamasına benzer bütünlük bir problemidir.

Hazırlık önerileri

1. Ek C'deki hızlı başvuru sayfasını kapatıp aynısını kendiniz yazmayı deneyin; yazamadığınız her satır bir çalışma konusudur.
2. 13.–15. haftaların laboratuvar çalışmalarını (not defteri, telefon rehberi, dosyaya kayıt) sıfırdan, notlara bakmadan yeniden yazın.
3. Her haftadan bir “zor” alıştırmayı fonksiyonlarla yeniden yapılandırın.
4. Eğitimci sürümündeki örnek final (Ek S) 15. haftanın son saatinde çözülür.

Ek A — Yarıyıl İçi Çalışma Planı

Bu ek, dersin yarıyıl içi değerlendirme takvimini bir arada verir. Önce ağırlıklar ve başarı notunun hesabı, sonra üniversitenin *Yarıyıl İçi Çalışma Planı* formuyla aynı bilgileri taşıyan plan tablosu (Tablo A.1) gelir: her değerlendirme bileşeninin türü, kapsamı, yapılacağı tarih ve sonucun ilan edileceği tarih. Tarihler, 2026–2027 eğitim–öğretim yılı Güz Yarıyılı’nda derslerin **29 Eylül 2026 Salı** günü başladığı ve dersin her hafta salı günü yapıldığı varsayımıyla hesaplanmıştır; akademik takvimde bir değişiklik olursa sınav tarihleri de aynı oranda kayar ve derste duyurulur.

A.1 Ağırlıklar ve başarı notunun hesabı

Yarıyıl notu yedi bileşenden oluşur; ağırlıkların toplamı %100’dür.

Bileşen	Adet	Birim ağırlık	Toplam ağırlık
Kısa sınav (3., 6., 8., 12., 15. hafta)	5	%5	%25
Ara sınav — vize (9. hafta)	1	%25	%25
Final sınavı (16. hafta)	1	%50	%50
Toplam	7	—	%100

Her sınav 100 puan üzerinden değerlendirilir. Başarı notu şu ağırlıklı ortalamadır:

$$\text{Başarı notu} = 0,05 \sum_{i=1}^5 \text{KS}_i + 0,25 \cdot \text{Vize} + 0,50 \cdot \text{Final}.$$

Örnek A.1 (Başarı notu hesabı). Bir öğrencinin kısa sınav notları sırasıyla 80, 70, 90, 60, 85; vize notu 65; final notu 72 olsun. Başarı notu kaçtır?

Çözüm

Kısa sınavların toplamı $80 + 70 + 90 + 60 + 85 = 385$ olduğundan katkısı $0,05 \cdot 385 = 19,25$ puandır. Vizenin katkısı $0,25 \cdot 65 = 16,25$, finalin katkısı $0,50 \cdot 72 = 36,00$ puandır.

$$19,25 + 16,25 + 36,00 = 71,50$$

Kontrol. Ağırlıkların toplamı $5 \cdot 0,05 + 0,25 + 0,50 = 1$ olduğundan sonuç, notların en küçüğü (60) ile en büyüğü (90) arasında kalmalıdır; 71,5 bu aralıktadır. Bütün notlar 100 olsaydı sonuç 100, hepsi 0 olsaydı 0 çıkardı.

İpucu

Finalin tek başına %50 ağırlıkta olması, yarıyıl içinde kaybedilen puanın telafi edilebileceği anlamına gelir; ama tersi de doğrudur: kısa sınavların tamamından tam puan alan bir öğrenci bile finalden 40 alırsa başarı notu $25 + 0,25 \cdot \text{Vize} + 20$ ile sınırlanır. Kısa sınavlar asıl olarak *geri bildirim* içindir: hangi konuyu anlamadığınızı vize gelmeden önce söylerler.

TRABZON ÜNİVERSİTESİ — BİLGİSAYAR VE BİLİŞİM FAKÜLTESİ — YAPAY ZEKA MÜHENDİSLİĞİ BÖLÜMÜ
2026-2027 EĞİTİM-ÖĞRETİM YILI GÜZ YARIYILI — YARIYIL İÇİ ÇALIŞMA PLANI

Dersin Kodu	Dersin Adı	T	U	K	Öğretim Elemanı Unvanı	Adı-Soyadı	Yarıyıl İçi Çalışma Türü (Sınav/Ödev/Proje/Diğer)	İçeriği / Konusu	Teslim/Sınav Tarihi	Değerlendirme Tarihi	Öğretim Elemanı İmza
YPZ105	Programlama ve Uygulamaları I	3	1	3,5	Dr. Öğr. Üyesi	Uğur Turhal	Sınav (Kısa Sınav 1 — %5)	Dersin 1.–3. haftalarında yer alan konulara yönelik sorular: bilgisayarda temel kavramlar ve sayı sistemleri, algoritma kavramı, sözde kod, akış diyagramı, izleme tablosu.	13.10.2026	20.10.2026	
							Sınav (Kısa Sınav 2 — %5)	Dersin 4.–6. haftalarında yer alan konulara yönelik sorular: değişkenler, veri türleri ve tür dönüşümü, giriş/çıkış, karakter dizisi işlemleri, seçme yapıları.	03.11.2026	10.11.2026	
							Sınav (Kısa Sınav 3 — %5)	Dersin 6.–8. haftalarında yer alan konulara yönelik sorular: iç içe koşullar, karar tablosu, girdi doğrulama, while döngüsü, sayaç ve biriktiriciler.	17.11.2026	24.11.2026	
							Sınav (Ara Sınav — %25)	Dersin 1.–8. haftalarında yer alan konuların tamamı. Kâğıt üzerinde, 90 dakika, 100 puan; kavram soruları, kod çıktısı bulma, akış diyagramından koda çeviri ve program yazma.	24.11.2026	01.12.2026	
							Sınav (Kısa Sınav 4 — %5)	Dersin 10.–12. haftalarında yer alan konulara yönelik sorular: for döngüsü ve range , iç içe döngüler, fonksiyon tanımı, parametre ve return , özyineleme.	15.12.2026	22.12.2026	
							Sınav (Kısa Sınav 5 — %5)	Dersin 13.–15. haftalarında yer alan konulara yönelik sorular: tek ve iki boyutlu listeler, demet, sözlük, küme; metin dosyası okuma ve yazma.	05.01.2027	12.01.2027	
							Sınav (Final — %50)	Dönemin tamamı (1.–15. haftalar), ağırlık 10.–15. haftalarda. Kâğıt üzerinde, 120 dakika, 100 puan.	12.01.2027	19.01.2027	

Tablo A.1: YPZ105 yarıyıl içi çalışma planı (2026-2027 Güz Yarıyılı). T: teorik saat, U: uygulama saati, K: yerel kredi. Ödev ve proje yoktur. Tarihler, derslerin 29.09.2026 Salı başladığı varsayımıyla.

A.2 Sınav biçimi, devam ve mazeret

Beş kısa sınav, ilgili haftanın dersinin **son 25 dakikasında** laboratuvarında, kâğıt üzerinde yapılır; ayrı bir sınav saati ilan edilmez. Her kısa sınav yalnız yukarıdaki tabloda yazan haftaların konularını kapsar ve 100 puan üzerinden değerlendirilir. Ara sınav ve final kâğıt üzerinde, sırasıyla 90 ve 120 dakikadır; hesap makinesi, not ve bilgisayar kullanılmaz. Sınavlarda söz dizimi hataları algoritma doğruysa küçük puan kaybıyla değerlendirilir; asıl puan kaybı mantık hatalarındadır.

Devam zorunluluğu ve mazeret sınavı konusunda **üniversitenin ilgili yönetmeliği esastır**; aşağıdakiler yalnız genel çerçevedir ve yönetmelikle çeliştiği yerde geçersizdir. Derse devam, teorik saatlerin en az %70'i ve uygulama saatlerinin en az %80'i olarak aranır; ders tamamen laboratuvarında işlendiği için pratikte her iki koşul da aynı yoklamayla izlenir. Belgelenmiş bir mazereti (sağlık raporu, resmî görev) olan öğrenci, mazeretini yönetmelikte belirtilen süre içinde bölüme bildirmek koşuluyla **ara sınav** için mazeret sınavına girebilir. **Kısa sınavların mazereti yoktur**: bunun yerine, ağırlıkça en düşük bir kısa sınav yerine geçmek üzere dönem sonunda bir telafi kısa sınavı uygulanıp uygulanmayacağı dönem başında derste duyurulur. Final sınavının mazereti, yönetmelikte tanımlı bütünleme ya da mazeret düzenlemelerine tabidir.

Dikkat

Laboratuvar görevleri notlandırılmaz; ancak kısa sınav ve vize sorularının önemli bir kısmı doğrudan laboratuvar görevlerinden türetilir. “Notu etkilemiyor” diye atlanan bir görev, iki hafta sonra sınavda karşınıza çıkar.

A.3 İş yükü ve haftalık çalışma önerisi

Ders 4 AKTS'dir. Resmî ders bilgi paketindeki iş yükü dağılımı şöyledir:

Etkinlik	Sayı	Süre (saat)	Toplam (saat)
Derse katılım (teorik)	14	3	42
Uygulama / pratik	14	1	14
Ders sonrası bireysel çalışma	14	2	28
Kısa sınavlara hazırlık	5	2	10
Ara sınav (vize)	1	2	2
Ara sınava hazırlık	1	15	15
Final sınavı	1	2	2
Finale hazırlık	1	20	20
Toplam iş yükü			133
AKTS kredisi			4

Kısa sınavlar dersin kendi saati içinde yapıldığı için ayrıca sınav süresi olarak sayılmamıştır.

Bu dağılımın haftalık karşılığı nettir: **3 saat ders + 1 saat uygulama + 2 saat bireysel çalışma = haftada 6 saat**. Dördü laboratuvarında geçer; geri kalan iki saati kendiniz ayırmak zorundasınız. Bu iki saati şöyle bölmenizi öneririz:

- **İlk 30 dakika (dersten sonraki gün):** haftanın özet tablosunu kapalı kitap kendi elinizle yeniden yazın, sonra kitapla karşılaştırın. Unuttuğunuz satır, tekrar etmeniz gereken konudur.
- **Orta 60 dakika:** o haftanın alıştırmalarından en az üçünü — birer tane kolay, orta ve zor — cevaba bakmadan çözün. Kod yazan alıştırmaları gerçekten bilgisayarda çalıştırın; kâğıtta “çalışır” sanılan kod sıkça çalışmaz.
- **Son 30 dakika:** kitaptaki kod dosyalarından birini açın, bilerek bozun (bir tırnağı silin, bir girintiyi kaydırın, bir değişken adını yanlış yazın) ve çıkan hata mesajını okuyun. Hata mesajlarını tanımak, sınavda en hızlı kazanılan puandır.

Sınav haftalarında bu ritim yetmez: tabloda görüldüğü gibi vize için 15, final için 20 saatlik ayrı bir hazırlık öngörülmüştür. Vize öncesindeki iki haftayı ve finalden önceki son üç haftayı buna göre planlayın; bu saatleri sınavdan önceki geceye sıkıştırmak, bu derste işe yaramaz.

Not

Programlamada ilerlemenin ölçüsü okunan sayfa değil, *yazılan satırdır*. Yukarıdaki iki saatin en az bir saatini klavye başında geçirmiyorsanız plan işlemiyor demektir. Haftada yazdığınız satır sayısını bir kenara not etmek, çalışma alışkanlığınızı izlemenin şaşırtıcı derecede iyi bir yoludur.

Ek B — Python Kurulum Kılavuzu

Bu ek, dersin ilk haftalarında bir kez yapılacak işi anlatır: bilgisayarınızda çalışan bir Python ortamı kurmak. Laboratuvar bilgisayarlarında her şey kurulumdur; buradaki adımlar *kendi* bilgisayarınız içindir. Ekran görüntüsü yoktur: kurulum programlarının görünümü sürümünden sürüme değişir, ama adımların sırası ve doğrulama yöntemi değişmez. Her bölümün sonunda, işin gerçekten olduğunu kanıtlayan bir **doğrulama** adımı vardır; o adımı atlamayın.

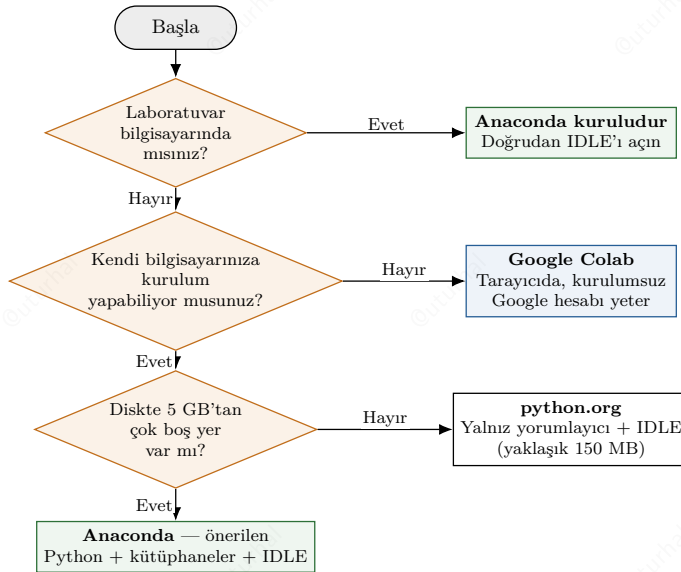
Dikkat

Kurulum sırasında karşılaştığınız ekranları okumadan İleri tuşuna basmayın. Bu ekteki sorunların neredeyse tamamı, okunmadan geçilen tek bir onay kutusundan doğar.

B.1 Hangi seçenek kime uygun?

Python'u çalıştırmanın bu derste kullanacağımız üç yolu vardır. Üçü birbirinin alternatifi değildir: biri ana ortamınız olur, diğerleri yedek kalır. Hangisinin size uyduğunu [Şekil B.1](#)'deki karar akışıyla bulabilirsiniz.

Seçenek	Kime	Artısı	Eksisi
Anaconda	Laboratuvar ve ev bilgisayarları; <i>önerilen</i>	Python, IDLE, paket yöneticisi ve bilimsel kütüphaneler tek kurulumda gelir	Büyüktür (indirme yaklaşık 1 GB, diskte 4–5 GB)
python.org	Diskte yeri az olan, hafif kurulum isteyen	Küçüktür (yaklaşık 150 MB); yalnız yorumlayıcı ve IDLE	Kütüphaneleri sonradan pip ile kurmak gerekir
Google Colab	Kurulumu tamamlayamayan; tablette çalışan; yedek isteyen	Kurulum yok; tarayıcıda çalışır; Google hesabı yeter	İnternet ister; dosyalar bilgisayarınızda değil, Drive'dadır



Nasıl okunur: Bu derste üç seçeneğin de kullanılacağı anlar vardır. *Anaconda* laboratuvarın ve evin ana ortamıdır. *python.org* kurulumu diskte yer yoksa yeterlidir: bu dersin hiçbir haftası NumPy/pandas gerektirmez. *Colab* kurulumu tamamlanana kadar yedektir; 15. haftada veri dosyası yüklemek için de işe yarar. Hangi yolu seçerseniz seçin, B.4'teki üç doğrulama adımı yapın.

Şekil B.1: Hangi kurulum seçeneğini seçmelisiniz? Karar akışı. Laboratuvarda çalışıyorsanız hiçbir şey kurmanız gerekmez; evde ise disk alanınız belirleyicidir.

Not

Bu dersin on beş haftasının hiçbiri NumPy, pandas ya da başka bir dış kütüphane gerektirmez; yalnız Python'un kendisiyle gelen `math`, `random`, `statistics`, `csv` ve `json` modülleri kullanılır. Yani “hafif” kurulum bu ders için tamamen yeterlidir. Anaconda'yı önermemizin nedeni ikinci sınıfta başlayacak derslerdir: orada kütüphaneleri tek tek kurmak yerine hazır bulmak işinizi kolaylaştırır.

B.2 Windows'ta Anaconda kurulumu**Adım adım kurulum
(Windows 10/11)**

1. **İndirin.** <https://www.anaconda.com/download> adresine gidin ve Windows sürümünü (64-bit grafik yükleyici) indirin. Dosya adı `Anaconda3-20xx.xx-Windows-x86_64.exe` biçimindedir ve yaklaşık 1 GB'tır. E-posta soran bir kutu çıkarsa altındaki “atla” bağlantısını kullanabilirsiniz.
2. **Çalıştırın.** İndirilen `.exe` dosyasına çift tıklayın. `Next` → `I Agree` → kurulum türü olarak `Just Me (recommended)` seçin. Bilgisayarı birden çok kişi kullanıyorsa bile “Just Me” doğru seçimdir: yönetici parolası istemez ve daha az sorun çıkarır.
3. **Kurulum klasörü.** Varsayılanı — kullanıcı klasörünüzün altındaki `anaconda3` — kabul edin. Klasör yolunda **Türkçe karakter ya da boşluk olmamalıdır**; kullanıcı adınız “Şeyma Öz” gibiyse yolu elle `C:\anaconda3` yapın.
4. **“Add to PATH” uyarısı.** Gelen ekranda iki onay kutusu vardır. `Add Anaconda3 to my PATH environment variable` kutusu **işaretsiz** bırakılmalıdır (kurulum programı da bunu kırmızıyla önerir); bunun yerine `Register Anaconda3 as my default Python` kutusu **işaretleli** kalsın. `PATH`'e eklemek, bilgisayarda başka bir Python varsa ikisini birbirine karıştırır. Anaconda'yı kendi başlatıcısı olan `Anaconda Prompt` ile kullanacağız.
5. **Bekleyin.** Kurulum 5–15 dakika sürer. Sonunda gelen tanıtım ekranlarındaki onay kutularını (`Launch Anaconda Navigator vb.`) temizleyip `Finish` diyebilirsiniz.
6. **Açın.** Başlat menüsünde `Anaconda Prompt` ve `IDLE (Python 3.x)` kısayollarını göreceksiniz. Bundan sonra Python ile ilgili her komutu `Anaconda Prompt` içinde çalıştırın, sıradan `Komut İstemi` içinde değil.

İpucu

Kurulumdan sonra `Anaconda Prompt`'u `Başlat` menüsünde bulup sağ tıklayın ve `Görev çubuğuna sabitle` deyin. Dönem boyunca en çok bu pencereyi açacaksınız.

B.3 macOS ve Linux

macOS. Anaconda'nın `.pkg` yükleyicisini indirip çift tıklamak yeterlidir; işlemcisi Apple Silicon (M1/M2/M3...) olan makinelerde “Apple Silicon” sürümünü, daha eski makinelerde “Intel” sürümünü seçin. Kurulumdan sonra `Terminal` uygulamasını açıp aşağıdaki doğrulama komutlarını çalıştırın. macOS'ta sistemle birlikte gelen eski bir Python bulunabilir; bu yüzden `$ python3 --version` yazımını (sondaki 3 ile) tercih edin.

Linux. Çoğu dağıtımda Python 3 zaten kuruludur; `$ python3 --version` ile sürümü kontrol edin. Sürüm 3.10'dan eskiyse dağıtımınızın paket yöneticisiyle güncelleyin (Ubuntu/Debian: `$ sudo apt install python3 idle3`). Anaconda isterseniz indirilen `.sh` betiğini `$ bash Anaconda3-*.sh` komutuyla çalıştırın (yıldızın yerine dosyanın tam adını yazın) ve sorulara varsayılan cevapları verin.

B.4 Kurulumu doğrulama

Kurulum bittğinde üç şeyi sırayla kontrol edin. Üçü de beklenen çıktıyı vermeden bir sonraki bölüme geçmeyin.

1. **Yorumlayıcı var mı?** `Anaconda Prompt` (macOS/Linux'ta `Terminal`) açın ve yazın:

Çıktı

```
$ python --version
Python 3.12.4
```

Sürüm numarasının son iki hanesi farklı olabilir; önemli olan **3.10 ya da daha yenisi** olmasıdır. Anaconda kurduysanız ayrıca `$ conda --version` komutu `conda 2x.x.x` biçiminde bir satır vermelidir.

2. **IDLE açılıyor mu?** Başlat menüsünden IDLE uygulamasını açın. `>>` işaretiyle biten bir pencere görmelisiniz. İşaretin yanına yazın:

Python etkileşimli kabuk (REPL)

```
>>> 2 ** 10
1024
```

1024 sayısını gördüyseniz Python çalışıyor demektir.

3. **Türkçe karakterler doğru mu?** Aynı kabukta yazın:

Python etkileşimli kabuk (REPL)

```
>>> print("Çığır açan ölçüm: şüphesiz!")
Çığır açan ölçüm: şüphesiz!
```

Harfler bozuk çıkıyorsa (Ã§, Ä± gibi) kurulum değil, kullandığınız terminalin kod çözümü sorunludur; IDLE'da doğru görünüyorsa sorun yoktur.



Sık yapılan hata

Komut İstemi'nde `python --version` → Anaconda Prompt'ta `python --version`
Sıradan Komut İstemi penceresinde Python tanınmaz (`'python' is not recognized...`), çünkü kurulumda PATH'e ekleme yapmadık. Bu bir hata değil, seçtiğimiz kurulum biçiminin doğal sonucudur.

B.5 IDLE ile çalışmak

IDLE, Python ile birlikte gelen basit geliştirme ortamıdır ve bu dersin ilk on haftası için fazlasıyla yeterlidir. İki penceresi vardır ve ikisini karıştırmamak önemlidir:

Shell `>>` işaretli etkileşimli kabuk. Yazdığınız her satır hemen çalışır ve sonucu altında görünür. Denemek, bir metodun ne yaptığını görmek, küçük hesap yapmak içindir. Buraya yazdıklarınız **kaydedilmez**.

Editor Program dosyası yazdığınız pencere. File → New File ile açılır. Burada yazdıklarınız siz çalıştırana kadar hiçbir şey yapmaz.

İlk programınızı yazıp çalıştırmak

1. IDLE'da File → New File (`Ctrl+N`) ile boş bir düzenleyici açın.
2. İki satır yazın: `ad = input("Adınız: ")` ve `print("Merhaba", ad)`.
3. File → Save (`Ctrl+S`) ile kaydedin. Dosya adını `merhaba.py` yazın — **uzantıyı kendiniz yazmalısınız**. Uzantı `.py` değilse IDLE renklendirme yapmaz ve `F5` çalışmaz.
4. Dosyayı, yolunda Türkçe karakter ve boşluk olmayan bir klasöre koyun; örneğin `C:\ypz105\hafta04`.
5. `F5` (Run → Run Module) ile çalıştırın. Shell penceresi öne gelir, programınız orada çalışır ve girdi ister.
6. Bir sonraki çalıştırmada yaptığınız değişikliklerin geçerli olması için *önce kaydedin*; IDLE kaydetmeden çalıştırmaya izin vermez, sorar.

Girintiyi 4 boşluğa ayarlamak. Python'da girinti söz dizimin parçasıdır (6. haftada göreceksiniz) ve bu derste girinti **4 boşluktur**, sekme değil. IDLE'da Options → Configure IDLE → Windows sekmesindeki Indentation Width değerini 4 yapın. Aynı pencerenin Fonts/Tabs sekmesinden yazı tipi boyutunu da büyütebilirsiniz; projeksiyona bakarak kod yazacağınız için 12–14 punto iyi bir seçimdir.

Dikkat

Sekme (Tab) ile boşlukları aynı dosyada karıştırmayın. Ekranda aynı görünürler ama Python için farklıdır ve TabError ya da IndentationError ile karşılaşabilirsiniz. Kural basittir: girinti için **yalnız boşluk** kullanın; düzenleyicinizi Tab tuşuna basıldığında 4 boşluk ekleyecek biçimde ayarlayın.

B.6 PyCharm Community

PyCharm, profesyonel bir geliştirme ortamıdır (*İng. IDE*): kod tamamlama, hata işaretleme, dosyalar arası gezinme ve adım adım çalıştırma (*debugger*) sunar. Ücretsiz **Community** sürümü bu ders için fazlasıyla yeterlidir.

PyCharm kurulumu ve ilk proje

1. <https://www.jetbrains.com/pycharm/download> adresinden **Community Edition** yükleyicisini indirin. Sayfada önce ücretli “Professional” sürümü görünür; aşağı inip ücretsiz olanı seçin.
2. Kurulumda varsayılanları kabul edin; istenirse Create Desktop Shortcut ve Add “Open Folder as Project” kutularını işaretleyin.
3. İlk açılıшта New Project deyin. Proje klasörü olarak C:\ypz105 gibi Türkçe karakter içermeyen bir yol verin.
4. **Yorumlayıcı seçimi** en kritik adımdır: Python Interpreter bölümünde Previously configured interpreter → Add Interpreter → Conda Environment yolunu izleyip Anaconda'nın Python'unu seçin. Ne seçtiğinizi bilmiyorsanız varsayılan Virtualenv seçeneği de çalışır; önemli olan sürümün 3.10 ya da üstü olmasıdır.
5. Projeye sağ tıklayıp New → Python File ile `merhaba.py` oluşturun, iki satırı yazın ve **[Shift]+[F10]** (ya da yeşil üçgen) ile çalıştırın. Çıktı alttaki Run panelinde görünür.

Ne zaman IDLE'dan PyCharm'a geçilir? Programınız tek dosyadan çıkıp birkaç dosyaya yayıldığında — yani 12. haftada kendi modülünüzü yazdığınızda. O noktadan itibaren “hangi dosya hangi fonksiyonu çağırıyor” sorusunu IDLE ile takip etmek zorlaşır. İlk haftalarda PyCharm'a geçmenin bir yararı yoktur; aksine, öğrenilecek şeylerin sayısını ikiye katlar.

B.7 Google Colab

Colab, tarayıcıda çalışan bir *not defteri* ortamıdır: hiçbir şey kurmanız gerekmez, kod **hücreler** hâlinde yazılır ve her hücrenin çıktısı hemen altında kalır.

Colab'da ilk not defteri

1. <https://colab.research.google.com> adresine gidin ve Google hesabınızla girin. Hesabınız yoksa önce bir Google hesabı açmanız gerekir.
2. Dosya → Yeni not defteri deyin. Boş bir kod hücresi açılır.
3. Hücreye `print("Merhaba Colab")` yazıp **[Shift]+[Enter]** ile çalıştırın. İlk çalıştırma, sunucu ayrıldığı için birkaç saniye sürer.
4. **+** Kod düğmesiyle yeni hücre ekleyin. Bir hücrede tanımladığınız değişken sonraki hücrelerde de geçerlidir; hücreleri **yukarıdan aşağıya** çalıştırın.
5. Dosya → Kaydet (**[Ctrl]+[S]**) ile kaydedin. Not defteri Drive'mızdaki Colab Notebooks klasörüne `.ipynb` uzantısıyla kaydedilir. Adını YPZ105_hafta04 gibi anlamlı bir şey yapın.
6. **Dosya yüklemek** (15. hafta için): sol kenardaki klasör simgesine tıklayın, yukarı ok simgesiyle bilgisayarınızdaki bir `.txt` ya da `.csv` dosyasını yükleyin. Yüklenen dosya `/content/`

klasörüne düşer ve kodunuzdan `open("veri.txt")` ile açılır.

⚠ Dikkat

Colab'a yüklediğiniz dosyalar oturum kapandığında (birkaç saat işlem yapılmazsa) **silinir**; not defterinin kendisi Drive'da kalır ama `/content/` geçicidir. Üzerinde çalıştığınız veri dosyasını her oturumda yeniden yükleyin ya da Drive'ı bağlayın.

Not

Not defteri ile betik arasındaki fark yalnız görünüş değildir: not defterinde hücreleri istediğiniz sırada çalıştırabilirsiniz, bu yüzden “bende çalışıyordu” diyen ama başkasında çalışmayan kodlar çoğu zaman not defterlerinden çıkar. Bu derste teslim edilen her şey `.py` dosyasıdır; Colab'ı deneme ve yedek ortamı olarak kullanın.

B.8 Sık karşılaşılan kurulum sorunları

Belirti	Olası neden	Çözüm
'python' is not recognized as an internal or external command	Kurulumda PATH'e ekleme yapılmadı (doğrusu budur) ya da kurulum hiç tamamlanmadı	Komutu Anaconda Prompt içinde çalıştırın. Orada da tanınmıyorsa kurulum tamamlanmamıştır; yeniden kurun.
\$ python --version çıktısı Python 2.7.x	Bilgisayarda eski bir Python var ve önce o bulunuyor	\$ python3 --version deneyin. Bu ders Python 3.10+ ister; Python 2 ile hiçbir örnek çalışmaz.
Kurulum “cannot create directory” ya da benzeri bir hatayla duruyor	Kullanıcı klasörünüzde Türkçe karakter var (C:\Users\Şeyma)	Kurulum yolunu elle C:\anaconda3 yapın. Proje dosyalarınızı da Türkçe karakter içermeyen bir klasörde tutun.
Yükleyici başlamıyor, “engellendi” uyarısı çıkıyor	Antivirüs ya da Windows SmartScreen bilinmeyen yayıncıyı engelliyor	İndirmeyi yalnız resmî adresten yapın; Daha fazla bilgi → Yine de çalıştır deyın ya da antivirüsü kurulum boyunca geçici olarak durdurun.
Kurulum yarıda “yetersiz disk alanı” diyor	Anaconda açıldığında 4–5 GB yer kaplar	python.org kurulumunu (yaklaşık 150 MB) seçin; bu ders için yeterlidir.
IDLE açılıyor ama [F5] bir şey yapmıyor	Dosya kaydedilmemiş ya da uzantısı .py değil	File → Save As ile adı .py uzantısıyla yeniden kaydedin.
Türkçe harfler ÃŞ, Â gibi görünüyor	Terminalin karakter kodlaması farklı	IDLE'da doğru görünüyorsa sorun yoktur. Komut isteminde düzeltmek için \$ chcp 65001 komutunu kullanın.
pip ile paket kurulamıyor	Ağ kısıtı ya da yanlış istem penceresi	Bu dersin hiçbir haftası dış paket gerektirmez; şimdilik göz ardı edin.

i İpucu

Bir kurulum sorununu sınıfa ya da eğitime sorarken üç şeyi birlikte söyleyin: **ne yazdınız, ne beklediniz, ne gördünüz**. Hata mesajının tamamını ekran görüntüsüyle değil, metin olarak kopyalayın; böylece aranabilir olur. Bu alışkanlık dönem boyunca kod hataları için de geçerlidir.

B.9 Ders kodlarını indirmek

Kitaptaki her tam program, GitHub'daki ders deposunda aynı dosya adıyla durur. Deponun tamamını indirmek için:

1. <https://github.com/uturhal/YPZ105> adresini açın.
2. Yeşil Code düğmesine tıklayın ve Download ZIP deyin.
3. İnen YPZ105-main.zip dosyasını sağ tıklayıp Tümünü ayıkla ile açın; Türkçe karakter içermeyen bir klasöre çıkarın.
4. code/ klasörü haftalara ayrılmıştır (w04/, w05/...); her haftanın out/ alt klasöründe programların beklenen çıktıları vardır. Kendi çıktınızı bunlarla karşılaştırabilirsiniz.

Tek bir dosya isterseniz kitaptaki kod kutusunun sağ üstündeki karekodu telefonunuzla okutmanız yeterlidir; doğrudan o dosyanın GitHub sayfasına gider. Aşağıdaki karekod ise deponun tamamına gider:



YPZ105 ders deposu

<https://github.com/uturhal/YPZ105>

Kodlar MIT, ders notu CC BY-NC-SA 4.0 lisanslıdır.

Not

İndirdiğiniz dosyaları okumak yetmez; her birini açıp **yeniden** yazın. Kopyalanan kod öğretmez. Depodaki dosyalar, kendi yazdığınızı karşılaştırmak ve bir yerde takıldığınızda bakmak içindir.

Ek C — Python Hızlı Başvuru

Bu ek, notlarda haftalara yayılmış yazımları tek yerde toplar. Öğretmek için değil, *hatırlatmak* için hazırlanmıştır: bir yapıyı ilk kez burada görüyorsanız ilgili haftaya dönün. Sağdaki sütun her zaman tek cümledir; ayrıntı, uç durum ve tuzaklar haftalardaki **Dikkat** ve **Sık yapılan hata** kutularındadır. Tablolarda **x**, **n** sayıları; **s** bir metni; **L** bir listeyi gösterir.

C.1 Veri türleri ve dönüşümler

Tür	Örnek değerler	Ne zaman
int	42 -7 0 1_000	Tam sayılar; sayma, indeks, adet. Basamak sayısı sınırsızdır.
float	3.14 2.0 1e-3 -0.5	Ondalık sayılar; ölçüm, ortalama, oran. Yaklaşık tutulur.
str	"merhaba" 'a' ""	Metin; tek ya da çift tırnak, ikisi de aynı.
bool	True False	Mantıksal değer; karşılaştırmaların sonucu.

Yazım	Anlamı
type(x)	x değerinin türünü söyler: <code><class 'int'></code> .
int(s) int(x)	Tam sayıya çevirir; <code>int(3.9) → 3</code> (keser, yuvarlamaz), <code>int("3.5")</code> hata verir.
float(s)	Kayan noktalı sayıya çevirir; <code>float("3.5") → 3.5</code> .
str(x)	Metne çevirir; <code>print</code> birleştirmesinde gerekir.
bool(x)	0, 0.0, "" ve boş liste <code>False</code> ; diğer her şey <code>True</code> .
round(x, n)	n ondalık basamağa yuvarlar; n yazılmazsa en yakın tam sayıya.

C.2 Operatörler ve işlem önceliği

Yazım	Anlamı
+ - *	Toplama, çıkarma, çarpma. + ve * metinlerde de çalışır.
/	Bölme; sonuç daima float 'tır (<code>6 / 3 → 2.0</code>).
//	Tam bölme; sonucu aşağı yuvarlar (<code>-7 // 2 → -4</code>).
%	Kalan; <code>n % 2</code> sıfırsa n çifttir.
**	Üs alma; sağdan sola birleşir (<code>2 ** 3 ** 2 → 512</code>).
== !=	Eşit mi, farklı mı. = atar, == sorar.
< <= > >=	Karşılaştırma; zincirlenebilir: <code>0 <= x < 10</code> .
and or not	Mantıksal ve, veya, değil; and/or kısa devre yapar.
in not in	Üyelik: metinde alt metin, listede eleman, sözlükte anahtar arar.
+= -= *= /=	Kısaltılmış atama: <code>x += 5</code> , <code>x = x + 5</code> demektir.

Öncelik (yüksekten düşüğe). Aynı satırdakiler soldan sağa değerlendirilir; tek istisna ******, o sağdan sola işler.

#	Operatörler	Açıklama
1	()	Parantez her şeyin önündedir.
2	**	Üs alma.
3	-x +x	Tekli işaret.
4	* / // %	Çarpma ve bölmeler.
5	+ -	Toplama ve çıkarma.
6	< <= > >= == !=	Karşılaştırmalar.
7	not	Mantıksal değil.
8	and	Mantıksal ve.
9	or	Mantıksal veya.

C.3 Giriş, çıkış ve biçimlendirme

Yazım	Anlamı
<code>print(a, b)</code>	Değerleri aralarında boşlukla yazar ve satır sonu koyar.
<code>print(a, b, sep="-")</code>	Değerlerin arasına konan ayırıcıyı değiştirir.
<code>print(a, end="")</code>	Satır sonunu kaldırır; sonraki çıktı aynı satıra gelir.
<code>s = input("İstem: ")</code>	Kullanıcıdan okur; daima str döndürür.
<code>n = int(input("Sayı: "))</code>	Okunamı sayıya çevirmenin standart yazımı.
<code>f"Merhaba {ad}"</code>	f-string: süslü parantez içindeki ifade değeriyle değişir.
<code>f"{x:.2f}"</code>	İki ondalık basamak: $3.14159 \rightarrow 3.14$.
<code>f"{n:5d}"</code>	Tam sayıyı 5 karakter genişlikte sağa yaslar.
<code>f"{n:,}"</code>	Binlik ayırıcı ekler: $1000000 \rightarrow 1,000,000$.
<code>f"{s:>10}"</code> <code>f"{s:<10}"</code>	Metni 10 karakterlik alanda sağa / sola yaslar.
<code>f"{s:^10}"</code>	Metni 10 karakterlik alanda ortalar.
<code>f"{x:.1%}"</code>	Yüzde biçimi: $0.256 \rightarrow 25.6\%$.
<code>f"{x = }"</code>	Hata ayıklama: <code>x = 7</code> biçiminde ad ve değeri birlikte yazar.

Kaçış	Anlamı	Kaçış	Anlamı
<code>\n</code>	Satır sonu	<code>\\</code>	Ters bölü karakterinin kendisi
<code>\t</code>	Sekme (hizalama)	<code>\"</code>	Çift tırnak (çift tırnaklı metinde)
<code>\r</code>	Satır başı	<code>\'</code>	Tek tırnak (tek tırnaklı metinde)

C.4 Metin (str) işlemleri

Yazım	Anlamı
<code>len(s)</code>	Karakter sayısı.
<code>s[0]</code> <code>s[-1]</code>	İlk ve son karakter; indeks 0'dan başlar.
<code>s[a:b]</code> <code>s[a:b:c]</code>	Dilim: a'dan b'ye (b dahil değil), c adımıyla.
<code>s[::-1]</code>	Metni ters çevirir.
<code>s + t</code> <code>s * 3</code>	Birleştirme ve yinleme.
<code>"a" in s</code>	Alt metin var mı? True/False .
<code>s.upper()</code> <code>s.lower()</code>	Tümü büyük / küçük harf. Türkçe i-İ çiftinde dikkatli olun.
<code>s.title()</code> <code>s.capitalize()</code>	Her kelimenin / yalnız ilk harfin baş harfini büyütür.
<code>s.strip()</code> <code>s.lstrip()</code> <code>s.rstrip()</code>	Baştaki ve sondaki boşlukları siler.
<code>s.replace(a, b)</code>	Tüm a geçişlerini b ile değiştirir; yeni metin döndürür.
<code>s.find(a)</code> <code>s.index(a)</code>	İlk geçişin indeksi; find bulamazsa -1, index hata verir.
<code>s.count(a)</code>	Kaç kez geçtiğini sayar.
<code>s.startswith(a)</code> <code>s.endswith(a)</code>	İle başlıyor / bitiyor mu?
<code>s.split()</code> <code>s.split(",")</code>	Metni parçalara ayırıp liste döndürür.

Yazım	Anlamı
"-".join(L)	Listenin elemanlarını aralarına ayırıcı koyarak birleştirir.
s.isdigit() s.isalpha()	Tümü rakam / harf / boşluk mu?
s.isspace()	
s.center(n) s.ljust(n)	n karakterlik alanda ortalar / yaslar.
s.rjust(n)	
s.zfill(n)	Başına sıfır ekleyerek n karaktere tamamlar.
ord(c) chr(k)	Karakterin kod noktası / koda karşılık gelen karakter.

Dikkat

Metinler **değiştirilemez**: `s.upper()` metni yerinde büyütmez, *yeni* bir metin döndürür. Sonucu saklamak istiyorsanız `s = s.upper()` yazmalısınız.

C.5 Seçme yapıları

```

1 if kosul:
2     ...
3 elif baska_kosul:      # istenildiği kadar elif olabilir
4     ...
5 else:                  # en fazla bir else, en sonda
6     ...
7
8 deger = a if kosul else b      # koşullu ifade (tek satırlık seçim)

```

```

1 match secim:
2     case 1:
3         ...
4     case 2 | 3:          # birden çok değer
5         ...
6     case n if n > 10:    # koşullu (guard) durum
7         ...
8     case _:              # hiçbirini tutmazsa
9         ...

```

Girdi doğrulamanın standart yazımı:

```

1 try:
2     n = int(input("Sayı: "))
3 except ValueError:
4     print("Sayı girmediniz.")

```

C.6 Tekrar yapıları

```

1 while kosul:          # koşul doğru olduğu sürece
2     ...                # gövdede koşulu değiştirmeyi unutmayın
3
4 for i in range(5):     # 0 1 2 3 4
5     ...
6
7 for c in "merhaba":    # metnin karakterleri
8     ...
9

```

```

10 for i, c in enumerate(s):      # indeks ve eleman birlikte
11     ...

```

Yazım	Anlamı
range(n)	0, 1, ..., n-1.
range(a, b)	a'dan b-1'e. Üst sınır dahil değildir.
range(a, b, c)	c adımla; c negatifse geriye sayar.
break	İçinde bulunduğu döngüyü hemen bitirir.
continue	Gövdenin kalanını atlar, sonraki yinelemeye geçer.
while True: ...break	"En az bir kez çalışsın" deseni.
for ...else:	Döngü break ile kesilmediyse else bloğu çalışır.
zip(a, b)	İki diziyi ikiyeşerli eşler.
reversed(L)	Diziyi sondan başa dolaşır.

C.7 Fonksiyonlar ve modüller

```

1 def alan(en, boy=1):          # boy'un varsayılan değeri var
2     """Dikdörtgenin alanını döndürür."""
3     return en * boy           # return yoksa fonksiyon None döndürür
4
5 a = alan(3, 4)                # konumsal argüman
6 b = alan(boy=5, en=2)         # anahtar sözcüklü argüman
7 x, y = bolum_ve_kalan(17, 5)  # birden çok değer demet olarak döner
8
9 if __name__ == "__main__":   # dosya doğrudan çalıştırıldıysa
10     main()

```

Yazım	Anlamı
import math	Modülün tamamını alır; math.sqrt(2) biçiminde kullanılır.
from math import sqrt	Yalnız bir adı alır; sqrt(2) yazılır.
import kendi_modul	Aynı klasördeki kendi_modul.py dosyasını alır.
help(alan)	Fonksiyonun docstring'ini yazar.
assert kosul, "mesaj"	Koşul yanlışsa programı hatayla durdurur; sınama için.

C.8 Listeler, demetler, sözlükler ve kümeler

Liste	Anlamı
L = [3, 8, 1] L = []	Oluşturma; liste değiştirilebilir .
L[0] = 9	Elemanı yerinde değiştirir.
L.append(x)	Sona bir eleman ekler.
L.insert(i, x)	i indeksine ekler, kalanları kaydırır.
L.extend(M) L + M	Başka bir listenin elemanlarını ekler / birleştirir.
L.remove(x)	İlk x elemanını siler; yoksa hata.
L.pop() L.pop(i)	Son / i'inci elemanı çıkarır ve döndürür.
del L[i] L.clear()	Elemanı / tüm listeyi siler.
L.index(x) L.count(x)	İlk geçişin indeksi / kaç kez geçtiği.
L.sort() L.reverse()	Listeyi <i>yerinde</i> sıralar / ters çevirir.
sorted(L) list(reversed(L))	Yeni bir sıralı / ters liste döndürür.
L2 = L[:] L2 = list(L)	Kopya çıkarır; L2 = L yalnız takma addır.

Liste	Anlamı
<code>[x * 2 for x in L]</code>	Liste üretici: her elemana bir işlem uygular.
Demet (tuple) <code>t = (3, 8) a, b = t</code>	Değiştirilemez dizi; paketleme ve açma.
Sözlük (dict) <code>d = {"ali": 90} d = {}</code> <code>d["ali"] d.get("ali", 0)</code> <code>d["veli"] = 75</code> <code>"ali" in d del d["ali"]</code> <code>d.keys() d.values()</code> <code>d.items()</code>	Anahtar-değer çiftleri. Değere erişim; get anahtar yoksa hata vermez. Ekler ya da var olanı günceller. Anahtar var mı / anahtarı sil. Anahtarlar, değerler, çiftler üzerinde döngü için.
Küme (set) <code>k = {1, 2, 3} set(L)</code> <code>a b a & b a - b</code>	Yinelenensiz, sırasız topluluk. Birleşim, kesişim, fark.

C.9 Metin dosyaları

```

1 with open("veri.txt", "r", encoding="utf-8") as f:
2     for satir in f:                # satir satir oku
3         print(satir.strip())       # satir sonunu at
4
5 with open("cikti.txt", "w", encoding="utf-8") as f:
6     f.write("ilk satir\n")         # write satir sonu EKLEMEZ

```

Yazım	Anlamı
"r"	Okuma (varsayılan); dosya yoksa FileNotFoundError .
"w"	Yazma; dosya varsa içeriğini siler , yoksa oluşturur.
"a"	Ekleme; yazılanlar dosyanın sonuna eklenir.
"x"	Oluşturma; dosya zaten varsa hata verir.
<code>with open(...) as f:</code>	Blok bitince dosyayı kendiliğinden kapatır; tercih edilen yazım.
<code>f.read() f.readline()</code>	Tamamını / bir satırı / satır listesini okur.
<code>f.readlines()</code>	
<code>f.write(s) print(x, file=f)</code>	Dosyaya yazar.
<code>os.path.exists(yol)</code>	Dosya var mı? (import os gerekir)

C.10 Sık kullanılan yerleşik fonksiyonlar, *math* ve *random*

Yerleşik	Anlamı	<i>math</i> / <i>random</i>	Anlamı
<code>len(x)</code>	Eleman/karakter sayısı	<code>math.pi</code>	π sayısı
<code>max(L)</code> <code>min(L)</code>	En büyük / en küçük	<code>math.sqrt(x)</code>	Karekök
<code>sum(L)</code>	Toplam	<code>math.floor(x)</code>	Aşağı yuvarlar
<code>sorted(L)</code>	Sıralı yeni liste	<code>math.ceil(x)</code>	Yukarı yuvarlar
<code>round(x, n)</code>	Yuvarlama	<code>math.factorial(n)</code>	Faktöriyel
<code>abs(x)</code>	Mutlak değer	<code>math.log(x, t)</code>	Logaritma
<code>type(x)</code>	Tür	<code>random.random()</code>	[0, 1) arası sayı
<code>enumerate(L)</code>	İndeks + eleman	<code>random.randint(a, b)</code>	a-b arası tam sayı (ikisi dahil)
<code>zip(a, b)</code>	İkişerli eşleme	<code>random.choice(L)</code>	Listeden rastgele eleman
<code>range(a, b)</code>	Sayı dizisi	<code>random.shuffle(L)</code>	Listeyi yerinde karıştırır

C.11 Hata türleri ve olası nedenleri

Hata	Sık görülen neden	İlk bakılacak yer
<code>SyntaxError</code>	Eksik iki nokta, kapanmayan parantez ya da tırnak	Belirtilen satır ve bir üstü ; parantez/tırnak eşleşmesi
<code>IndentationError</code>	Blok girintisi eksik ya da tutarsız	if, while, for, def satırından sonraki girinti
<code>TabError</code>	Aynı dosyada sekme ile boşluk karışmış	Düzenleyicinin “sekmeyi boşluğa çevir” ayarı
<code>NameError</code>	Tanımlanmamış ad kullanıldı; yazım ya da büyük/küçük harf hatası	Adın yazımı; değişkenin gerçekten atandığı satır
<code>TypeError</code>	İşlem, verilen türlerle yapılamıyor (" <code>5</code> " + <code>1</code>)	type() ile değişkenlerin türü; eksik int() dönüşümü
<code>ValueError</code>	Tür doğru ama değer uygunsuz (<code>int("on")</code>)	Kullanıcıdan gelen metin; isdigit() ile ön kontrol
<code>ZeroDivisionError</code>	Sıfıra bölme	Bölen değişkenin sıfır olabileceği durumlar
<code>IndexError</code>	Var olmayan indekse erişim	Döngü sınırları; <code>len(L)</code> ile son indeks <code>len(L)-1</code>
<code>KeyError</code>	Sözlükte olmayan anahtar	in ile kontrol ya da <code>d.get(k, varsayilan)</code>
<code>AttributeError</code>	Nesnede olmayan metot çağrıldı (<code>L.upper()</code>)	Değişkenin türü; metodun o türe ait olup olmadığı
<code>FileNotFoundError</code>	Dosya yok ya da yol yanlış	Çalışma dizini; dosya adının uzantısı ve yazımı
<code>UnboundLocalError</code>	Yerel değişken atanmadan kullanıldı	Fonksiyon içinde aynı adı hem okuyup hem atama

İpucu

Geri izlemeyi (*traceback*) **alttan yukarı** okuyun: en alt satır hatanın türünü ve mesajını, bir üstteki satır hatanın hangi dosyanın kaçınıcı satırında oluştuğunu söyler. Hata o satırda değilse neredeyse her zaman bir önceki satırdadır.

Ek F — Alıştırma Cevapları

Notlardaki her alıştırmanın kısa cevabı bulunduğu yerde verilmiştir; burada, bir haftanın tamamı tek seferde kontrol edilebilsin diye toplanmıştır. Numara alıştırma numarasıdır: **4.6**, 4. haftanın altıncı alıştırmasıdır.

Not

Cevap, çözüm değildir. Buradaki değerler ve çıktılar yalnız varış noktasını söyler; oraya mantıklı bir yoldan gidip gitmediğinizi söylemez. Çalışan ama anlaşılabilir, doğru çıktı veren ama yanlış yapılandırılmış bir program, sınavda ve gerçek hayatta pek az değer taşır. Tam çözümler laboratuvarında ve derste tartışılır.

Cevapların tamamı **verify/** altındaki doğrulama betikleriyle bağımsız olarak yeniden hesaplanır ve her derlemede yeniden denetlenir; dolayısıyla sizin sonucunuzla buradaki bir cevabın uyuşmaması, iki yönde de ciddiye alınmaya değer.

Hafta 1

- 1.1** donanım: klavye, RAM, SSD, ekran kartı; işletim sistemi: Windows 11, Android; uygulama: web tarayıcısı, Python yorumlayıcısı, hesap makinesi, Not Defteri
- 1.2** (a) 101101 (b) 1100100 (c) 11111101010 (d) 13 (e) 128 (f) 85
- 1.3** (a) $31 = 00011111_2$ (b) $160 = 10100000_2$ (c) 2047 (d) $CA_{16} = 202$ (e) beyaz; 16 777 215
- 1.4** (a) /home/ayse/Belgeler; (b) /home/ayse/Belgeler/YPZ105/hafta01; (c) YPZ105 (ve Belgeler'de önceden ne varsa); (d) C:\Users\ayse\Belgeler\YPZ105\hafta01
- 1.5** (a) 10010001 = 145, taşma yok (b) 100000001: 9 bit; 8 bitte 00000001 kalır, taşma var ($129 + 128 = 257 > 255$)
- 1.6** (a) $\approx 476,84$ GB (b) 8 589 934 592 B (c) 12,5 MB/s ile 120 s = 2 dakika
- 1.7** (a) -32 768...32 767 (b) 1111111111111111 (on altı adet 1) (c) -32 768 (taşma) (d) 65 536
- 1.8** 36 000 000 B $\approx$ 34,33 MB; oran $\approx$ 12; gri tonlu: 12 000 000 B
- 1.9** (a) PYTHON (b) 89 90; küçük harf +32: 121 122 (c) 3 ve 4 bayt
- 1.10** (a) 288 000 B/s (b) 1 036 800 000 B $\approx$ 0,97 GB (onluk 1,04 GB) (c) 61 saat
- 1.11** Şişe: 4 karakter, 6 bayt; İstanbul: 8 karakter, 9 bayt; sabit 4 bayt ile Şişe 16 bayt. Üstünlük: ASCII metinler için yer ve geriye uyumluluk; bedel: n 'inci karaktere gitmek için baştan saymak gerekir
- 1.12** Veri: geçmiş öğrencilerin notları, devam kayıtları, ders yükü ve yıl sonu durumu (etiket); ön işleme: eksikleri temizleme, harf notlarını sayıya çevirme, ölçekleme; model: bu özelliklerle başarısızlık arasındaki ilişkiyi öğrenir; tahmin: yeni öğrenci için risk puanı; değerlendirme: geçmiş yılın verisinde tahminlerin doğruluğu. Model dışındaki dört kutu doğrudan

programlama işidir

Hafta 2

- 2.1** (a) kesinlik (b) sonluluk (c) çıktı (d) girdi (e) etkinlik
- 2.2** (a) 13 (b) 20 (c) 3 (d) 2 (e) 16 (f) 50 (g) 4.5 (h) 5
- 2.3** 16 11: adımlar $x = 4$; $y = 12$; $x = 16$; $y = 11$
- 2.4** alan 78,5, çevre 31,4; $PI \leftarrow 3.14$; $alan \leftarrow PI * r^2$; $cevre \leftarrow 2 * PI * r$
- 2.5** tüketim $31,5/450 \cdot 100 = 7$ L/100 km; gider $31,5 \cdot 42,5 = 1338,75$ TL; $LITRE_FIYATI \leftarrow 42.5$; $tuketim \leftarrow litre / km * 100$; $gider \leftarrow litre * LITRE_FIYATI$
- 2.6** 6 gün 22 saat 40 dakika: $10\,000//1440 = 6$, kalan 1360; $1360//60 = 22$, kalan 40
- 2.7** basamaklar 2, 0, 2, 6; toplam 10; ters 6202
- 2.8** $a = 3$, $b = 1$, $c = 2$; $gecici \leftarrow c$; $c \leftarrow b$; $b \leftarrow a$; $a \leftarrow gecici$
- 2.9** hatalı satır toplam $\leftarrow ara + KDV$ (orana tutar gibi ekleniyor); doğrusu toplam $\leftarrow ara + ara * KDV$ ya da $ara * (1 + KDV)$: $150 + 30 = 180$
- 2.10** Örneğin $(113, 200) \rightarrow 1\,1\,1\,1\,2$; $(200, 200) \rightarrow 0\,0\,0\,0\,0$; $(150, 200) \rightarrow 1\,0\,0\,0\,0$; $(197, 200) \rightarrow 0\,0\,0\,0\,3$; $(1, 100) \rightarrow 1\,2\,0\,1\,4$. Fiyat > ödenen ise üstü negatif olur ve anlamsız adetler çıkar; algoritma bunu fark edemez — karar yapısı (3. hafta) gerekir
- 2.11** (a) $a = 11$; $b = 3$; $a = 8 \rightarrow (8, 3)$ (b) 1. satırdan sonra $a = a_0 + b_0$; 2. satır $b = (a_0 + b_0) - b_0 = a_0$; 3. satır $a = (a_0 + b_0) - a_0 = b_0$ (c) yalnız sayılarla çalışır (metin takas edilemez); sabit genişlikli tam sayılarda $a + b$ taşabilir (1. hafta); okunması zor
- 2.12** saat 35 dakika: $9 \cdot 60 + 45 = 585$; $14 \cdot 60 + 20 = 860$; fark 275; $275//60 = 4$, $275 \bmod 60 = 35$. $t1 \leftarrow sa1 * 60 + dk1$; $t2 \leftarrow sa2 * 60 + dk2$; $fark \leftarrow t2 - t1$; $saat \leftarrow fark // 60$; $dakika \leftarrow fark \bmod 60$

Hafta 3

- 3.1 (a) D (b) Y (c) Y (d) D (e) D (f) D (g) Y (h) D (D: doğru, Y: yanlış)
- 3.2 12 7: yalnız çift sayılar toplanır ($2 + 4 + 6$); döngü bitince $i = 7$
- 3.3 yalnız (b) doğru. (a) iki ok; (c) isteğe bağlı; (d) gövde hiç çalışmayabilir; (e) sıra anlamı belirler
- 3.4 $-7 \rightarrow$ negatif, $0 \rightarrow$ sıfır, $4 \rightarrow$ pozitif; üç dallı zincir, iki karar sembolü
- 3.5 $60; 90 + 15 \cdot 3 = 135; 60 + 10 \cdot 7 = 130$ TL
- 3.6 $n = 9 \rightarrow 25, n = 5 \rightarrow 9$; ilk k tek sayının toplamı k^2
- 3.7 en büyük 45, en küçük 7, toplam 94, sayaç 4, ortalama 23,5
- 3.8 toplam 24, 5 basamak; toplam sırasıyla 2, 8, 15, 15, 24. $n = 0$ için gövde hiç çalışmaz; düzeltme: başa EĞER $n = 0$ İSE adet $\leftarrow 1$ EĞER SONU
- 3.9 (a) birim kayması: 45 çıkar, 55 olmalı ($i \leq 10$). (b) sonsuz döngü: gövdenin sonuna OKU sayı eklenir
- 3.10 $10 + 2 + 3 + 4 = 10$; genel olarak $n(n+1)/2$; $n = 100 \rightarrow 5050$
- 3.11 (a) 2 tur, EBOB = 12 (12, 0) (b) 3 tur, EBOB = 21 (147, 21, 0) (c) $3024/12 = 252$
- 3.12 (a) 1048576 (b) 20 ($2^{20} = 1048576$) (c) eleme ancak sıralı listede yapılabilir (d) doğrusal arama: sıralama daha pahalı

Hafta 4

- 4.1 geçerli: sayi2, toplam_puan, Class, _gizli, print (yerleşik adı gölgelediği için kötü); geçersiz: 2sayi (rakamla başlar), toplam-puan (tire), class (anahtar sözcük), ad soyad (boşluk)
- 4.2 int, float, str, bool, float, int, str (değeri "77"), int (değeri 14)
- 4.3 $37 \cdot 9/5 + 32 = 98,6$; `print(f"{c:.1f} °C = {f:.1f} °F")`
- 4.4 (a) 14 (b) 20 (c) 512 (d) 17 (e) -4 (f) 2,5 (g) 2 (h) 18
- 4.5 `onlar = n // 10, birler = n % 10`; toplam `onlar + birler, ters birler * 10 + onlar`
- 4.6 `KDV_RATE = 0.20`; ara toplam $49,90 \cdot 3 = 149,70$; KDV 29,94; toplam 179,64; biçim `f"{'Ara toplam':<12}{ara:>10.2f} TL"`
- 4.7 $100\,000 = 1 \cdot 86\,400 + 13\,600$; $13\,600 = 3 \cdot 3600 + 2800$; $2800 = 46 \cdot 60 + 40$
- 4.8 12.00, 14.00, 5.00; `math.sqrt(a**2 + b**2)` ya da `math.hypot(a, b)`
- 4.9 float sayıları ikili tabanda sonlu basamakla tutar; 0,1 ve 0,2 ikili tabanda sonsuz kesirli, yuvarlanır ve toplam 0,3'ün yuvarlanmış hâliyle çakışmaz. Doğru yol: `abs(a - b) < 1e-9` ya da `math.isclose(a, b)`

- 4.10 $2,5 \cdot 0,4 + 81 \cdot 0,6 = 29 + 48,6 = 77,6$; belirteçler `<10, :>8.1f, :>8.1f, :>10.2f`
- 4.11 6275 kuruş; $6275//5000 = 1$, kalan 1275; $1275//1000 = 1$, kalan 275; $275//200 = 1$, kalan 75; $75//50 = 1$, kalan 25; $25//25 = 1$, kalan 0

Hafta 5

- 5.1 (a) 'P' (b) 'A' (c) 'PROG' (d) 'PRO' (e) 'LAMA' (f) 'LAMA' (g) 'PORMAA' (h) 'AMALMARGORP' (i) " (j) 'AMA'
- 5.2 (a) `TypeError`: karakter dizisi değiştirilemez bir nesnedir, içindeki karakterler yerinde değiştirilemez. (b) `name = "A" + name[1:]` ya da `name = name.capitalize()`. (c) Metotlar metni değiştirmez, yeni metin döndürür; sonuç bir ada atanmadığı için kaybolur.
- 5.3 `len(s) = 20`; `s.count(" ") = 3`; $20 - 3 = 17$; `s.count("a") = 3` (büyük Y sayılmaz, "A" ayrı bir karakterdir)
- 5.4 İç genişlik $14 + 4 = 18$, çerçeve `"+" + "-" * 18 + "+" = 20` karakter; `title.upper().center(30, "*")` her iki yana sekizer yıldız koyar
- 5.5 `p[0]`, `p[1:4]`, `p[4:7]`, `p[7:9]`, `p[9:]` parçaları; maskede orta üç grup `****` ve `***` ile değiştirilir
- 5.6 `day, month, year = date.split(".")` ile üç parça; ISO için `year + "-" + month + "-" + day`; kısa yıl `year[-2:]`; `int(year) + 4 = 2030`
- 5.7 `"*" * len(bad)` maskesi; `replace` üç kez çağrılır (`bad`, `bad.capitalize()`, `bad.upper()`); sayım `text.lower().count(bad) = 3`
- 5.8 `words = s.split()`; `len(words) = 4`; `len("".join(words)) = 17`; ortalama $17/4 = 4,25$; ters sıra için `"".join(words[::-1])`
- 5.9 @ konumu 11; kullanıcı adı `ayse.yilmaz`, alan adı `ogr.trabzon.edu.tr`; kurum, alan adının noktalardan bölünmüş ikinci parçasıdır (`trabzon`); ad ile soyad kullanıcı adının ilk noktasından ayrılır
- 5.10 "10", "9", "Armut", "Elma", "armut", "elma", "Çilek" — karar her zaman ilk karakterde verilir: kodları sırasıyla 49, 57, 65, 69, 97, 101, 199
- 5.11 `clean.replace("I", "ı").replace("İ", "i").lower().title()` doğru sonucu verir; `clean.title()` ise Ayşe Yılmaz verir, iki sonuç eşit değildir
- 5.12 `on "/" konumu rfind("/")`; dosya adı `path[p+1:] = rapor.final.pdf`; uzantı son noktadan sonrası `pdf`; `path.count("/") = 4`; yeni yol `folder + "/" + stem + ".docx"`

Hafta 6

- 6.1 (a) True (b) True (c) False (d) False (e) True (f) False (g) True (h) False
- 6.2 (a) DD (b) CC, BA, AA (c) söz dizimi değil anlam bozuk (d) eşikleri büyükten küçüğe sıralayın
- 6.3 üç dal: `if n > 0, elif n < 0, koşulsuz else`
- 6.4 (a) True (b) True (c) True (d) False (e) True (f) False, sağ taraf hesaplanmaz (g) True, hesaplanmaz (h) True, hesaplanmaz
- 6.5 `a >= b and a >= c, sonra b >= a and b >= c, sonra else;` eşitlik için `>=` kullanın
- 6.6 $750 \cdot 0,10 = 75$, ödenecek 675; zincirde önce ≥ 1000 , sonra ≥ 500
- 6.7 $10 \cdot 12 + 10 \cdot 18 + 7 \cdot 25 = 120 + 180 + 175 = 475$
- 6.8 (a) `IndentationError` — iki nokta ile biten satırın ardından girintili blok gelmeli (b) o satırı dört boşluk içeriden yazın (c) 40 → Toplam: 0; 80 → Eklendi. ve Toplam: 80 (d) hiçbir şey yazılmaz
- 6.9 uzunluk 3-16, ilk karakter harf, boşluk yok, tamamı küçük harf; `Ayşe_yılmaz` son kuralda takılır
- 6.10 $2024 \rightarrow 29$, $2/2023 \rightarrow 28$; 4, 6, 9, 11 → 30; geri kalan → 31
- 6.11 $\Delta = 9 - 8 = 1$; $x_1 = 2$, $x_2 = 1$; `a == 0` dalı en başta olmalı

Hafta 7

- 7.1 (a) `age < 18 or not has_license` (b) `x >= 0 and x <= 100`, yani `0 <= x <= 100` (c) `day != 6 and day != 7`
- 7.2 `if score < 0: print("geçersiz not"); elif score >= 50: print("geçti"); else: print("kaldı")` — zincir daha kırsadır ve girintisi tek seviyededir
- 7.3 31 gün: 1, 3, 5, 7, 8, 10, 12; 30 gün: 4, 6, 9, 11; şubat 28 (artık yılda 29); girdi 4 için çıktı 4. ay 30 gün çeker.
- 7.4 6 satır: 2 → 90 (hafif), 2,01 → 150 (orta), 10 → 150 (orta), 10,01 → 250 (ağır), artı aralık ortalarından 1 → 90 ve 5 → 150
- 7.5 `"sil"→A; "SIL"→A (lower() uygulandığı için); "kopyala"→C (bekçi False); "5"→D. izin True olsaydı yalnız üçüncüsü B olurdu`
- 7.6 girdi 452,5 ve `say` için Öneri: mühendislik fakülteleri. (çünkü $452,5 < 480$); girdi `abc` için hata mesajı
- 7.7 girdi 5, 5, 8 için Geçerli bir ikizkenar üçgen.; 1, 2, 3 için üçgen oluşmaz, çünkü $1 + 2 = 3$ üçüncü kenardan büyük değildir
- 7.8 girdi 36,6 ve `c` için $36,6 \text{ } ^\circ\text{C} = 97,88 \text{ } ^\circ\text{F}$
- 7.9 girdi 12,5 kg, bölge 1, kırılabilir e: $150 \times 1,25 = 187,50$ TL ("ağır" satırı)
- 7.10 50 kWh için enerji 650,00 TL, KDV 130,00 TL, toplam 780,00 TL
- 7.11 01.03.2024 → yılın 61. günü ($31 + 29 = 60$, artı 1); 31.12 normal yılda 365., artık yılda

366. gündür

- 7.12 (a) $8 \times 8 = 64$ (b) 27 (c) 0-, 0+, A-, A+ (dört grup) (d) `and`: ABO uyumu ile Rh uyumu birbirinden bağımsızdır; ikisini ayrı hesaplayıp `and` ile birleştirmek 64 hücreyi iki küçük denetime indirir

Hafta 8

- 8.1 (a) 7 tur, `i = 8` (b) 7 tur, `i = 7` (c) 5 tur, `i = 0` (d) 4 tur, `i = 13` (e) 0 tur, `i = 5`
- 8.2 $16 + 9 + 4 + 1 = 30$ yazılır; döngü 4 tur döner ve biterken `n = 0`'dır
- 8.3 (a) `i = 0, 3, 6, 9, 12, ...` olur ve 10'a hiç eşit olmaz; koşulu `i < 10` yapın (b) güncelleme yok; gövdeye `n -= 1` ekleyin (c) parola döngü içinde yeniden okunmuyor; gövdeye `password = input()` ekleyin
- 8.4 `n = int(input()), total = 0, i = 1, while i <= n: içinde total += i ve i += 2, sonra print(total); n = 7 için 1 + 3 + 5 + 7 = 16`
- 8.5 sayaç ikiye azalır (`n -= 2`); 15 için sekiz sayı, 16 için de aynı sekiz sayı yazılır
- 8.6 ölçüm; en düşük -8,5; en yüksek 15,0; ortalama $23/6 = 3,83$; sıfırın altında 2 ölçüm
- 8.7 16 terim yazılır; sınırı aşmayan en büyük terim 987'dir (bir sonraki terim $1597 > 1000$)
- 8.8 ilk iki deneme reddedilir, üçüncüsü kabul edilir: Seçilen ay: 3 (3 denemede)
- 8.9 $45 = 101101_2$; kalanlar sırayla 1, 0, 1, 1, 0, 1 ve ters okunur
- 8.10 0 terim toplanır; toplam 1,9999980927 ve 2'ye uzaklık $1,907 \times 10^{-6}$
- 8.11 borç 26 ayda biter; toplam ödenen 64 493,64 TL
- 8.12 `rahgim` ortası `(low + high) // 2`; 1-100 arasında en kötü durumda 7 tahmin yeter ($2^7 = 128 > 100$)

Hafta 10

- 10.1 (a) 5 değer: 1, 3, 5, 7, 9 (b) 5 değer: 10, 8, 6, 4, 2 (c) 3 değer: 0, 3, 6 (d) boş (e) boş
- 10.2 `for i in range(2, 20, 3):` — 6 tur; 2, 5, 8, 11, 14, 17; son değer 17. Tekrar sayısı belli olduğu için `for`
- 10.3 $+ 0 + 2 + 6 = 10$; basamak sayısı `len(str(abs(n))) = 4`
- 10.4 ift toplam 2550, tek toplam 2500, fark 50, toplam 5050
- 10.5 1, 2, 3, 5, 8, 13, 21, 34, 55; onuncu terim 55
- 10.6 5 karakter, dört sütunda 24 satır ($\lceil 95/4 \rceil$); 65 → A, 97 → a, 32 → boşluk
- 10.7 doğru kayma 3; çözüm MERHABA DUNYA; denecek kayma sayısı 25
- 10.8 1 karakter, 7 farklı harf; a 3, r 2, m 2, kalanlar birer kez

- 10.9 deneme yetmeyebilir: her adımda aralığı ikiye bölerek oynanırsa $\lceil \log_2 100 \rceil = 7$ deneme her zaman yeter
- 10.10 $n - 1 = 7$ satır; en geniş satır $2n - 1 = 7$ yıldız; toplam yıldız sayısı 25
- 10.11 beş terimde $2,708333$ (hata $9,95 \times 10^{-3}$); 12 terimde hata $2,26 \times 10^{-9}$; 13. terimde 10^{-9} 'un altına iner ($1,73 \times 10^{-10}$)
- 10.12 (a) $n^2 = 10000$ (b) $n(n - 1)/2 = 4950$ (c) $n^3 = 1000000$; asal testinde 95 deneme yerine 8 deneme (2-9 arası)

Hafta 11

- 11.1 önce 9 (fonksiyonun kendi yazdığı), sonra None 9 13 satırı
- 11.2 çeride: 6 ve dışarıda: 10; fonksiyondaki atama yeni bir yerel ad açar ve global toplam adını gölgeler
- 11.3 $12,0^\circ \text{F}$; $100,0^\circ \text{C}$; $-40,0^\circ \text{F}$ (iki ölçeğin keştiği tek nokta); gidiş-dönüş kontrolü True
- 11.4 (a) doğru çağrı, 20,0 (b) hata yok, sonuç anlamsız: "fiyat" 0,2 TL sayılmıştır (c) `TypeError: missing 1 required positional argument: 'oran'` (d) `TypeError: takes 2 positional arguments but 3 were given` (e) çağrı değildir, fonksiyon nesnesini ifade eder (f) dönen değer sonuc'a atanır
- 11.5 1896, 1904, 2000, 2024 artık (True); 1900, 2023, 2100 değil (False)
- 11.6 3×3 kutu: üç satır; ilk ve son satır #####, ortadaki #. 3×2 kutuda iç satır yoktur
- 11.7 True, True, False, True, True
- 11.8 `caesar("MERHABA", 3) → PHUKDED`; `caesar("PHUKDED", -3) → MERHABA`; gidiş-dönüş kontrolü True
- 11.9 `is_perfect(6) True` ($1 + 2 + 3 = 6$), `is_perfect(12) False` ($1 + 2 + 3 + 4 + 6 = 16$); 10000'e kadar mükemmel sayılar: 6, 28, 496, 8128
- 11.10 12, 35; $12/18 = 2/3$, $45/60 = 3/4$, $7/5 = 7/5$ (zaten sade)
- 11.11 Ayşe: $72,5 \cdot 0,4 + 81 \cdot 0,6 = 29 + 48,6 = 77,60 \rightarrow \text{C}$; Mehmet: $90 \cdot 0,4 + 95 \cdot 0,6 = 36 + 57 = 93,00 \rightarrow \text{A}$
- 11.12 $75 \rightarrow 2$, $2026 \rightarrow 1$ ($10 \rightarrow 1$), $7 \rightarrow 7$; kural her denemede True

Hafta 12

- 12.1 (a) Sayın Ayşe. (b) Değerli Ayşe. (c) Sayın Ayşe! (d) Merhaba Ali. (e) `SyntaxError: positional argument follows keyword argument` (f) `TypeError: missing 1 required positional argument: 'ad'`
- 12.2 kisi de `RecursionError: maximum recursion depth exceeded` verir. Birincide taban durumu yoktur (`if n == 0: return`

0 eklenmeli); ikincide taban durumu vardır ama ona hiç varılmaz, çünkü çağrı küçülüyor (`geri_say(n - 1)` olmalı)

- 12.3 $3^2 = 9$, $2^{10} = 1024$, $5^3 = 125$
- 12.4 En küçük 2, en büyük 9, fark 7; doğrulama satırı True
- 12.5 Trabzon; Python; boş metin ("")
- 12.6 1, 7, 20 basamak; dört satırda da karşılaştırma aynı sayıyı verir
- 12.7 isobarT; palindromun tersi kendisidir (True); boş metnin tersi boş metin; doğrulama True
- 12.8 `count_vowels("Programlama") = 4`; `initials("Uğur Turhal") = U.T.`; `initials("ayşe yılmaz kaya", "-") = A-Y-K-`; modülü import eden program test satırlarını görmez
- 12.9 `BOB(1071, 462) = 21`, 4 çağrı ($1071, 462 \rightarrow 462, 147 \rightarrow 147, 21 \rightarrow 21, 0$); `EKOK = 1071 \cdot 462 / 21 = 23562`; doğrulama True
- 12.10 $10 \rightarrow 100101$, $255 \rightarrow 11111111$, $0 \rightarrow 0$, $1 \rightarrow 1$, $1024 \rightarrow 10000000000$; beş satırda da doğrulama True
- 12.11 $11 = 3$ için 7 hamle; $5 \rightarrow 31$, $10 \rightarrow 1023$, $15 \rightarrow 32767$, $20 \rightarrow 1048575$; genel formül $2^n - 1$

Hafta 13

- 13.1 (a) 20 (b) 40 (c) [20, 30] (d) [10, 20] (e) [50, 40, 30, 20, 10] (f) 3 (g) `IndexError` (h) [30, 40, 50]
- 13.2 `a = [99, 2, 3, 4]`, `b = [99, 2, 3, 4]`, `c = [1, 2, 3, 4]`
- 13.3 `sort()` listeyi yerinde sıralar ve None döndürür; `sorted()` özgün listeye dokunmadan yeni bir sıralı liste döndürür. Düzeltme: `sirali = sorted(names)`
- 13.4 toplam 64, ortalama $64/5 = 12,8$; sıralı [7, 7, 12, 15, 23]; ters [15, 7, 23, 7, 12]
- 13.5 (a) [8, 12, 20] (b) [9, 64, 144, 49, 25, 400, 225] (c) [12, 20, 15] (d) 40
- 13.6 [4, 7, 2, 9]; dört farklı değer; sayımlar $4 \rightarrow 3$, $7 \rightarrow 2$, $2 \rightarrow 2$, $9 \rightarrow 1$
- 13.7 Birleşik [1, 1, 2, 3, 5, 8, 8, 9]; ortak [8, 1]; tekil birleşim [1, 2, 3, 5, 8, 9]; yalnız a'da [3, 9]
- 13.8 $7, 1, 9, 3, 5$; $5//2 = 2$ takas
- 13.9 $2 \rightarrow 2$, $2 \rightarrow 1$, $3 \rightarrow 2$, $4 \rightarrow 2$, $5 \rightarrow 1$, $6 \rightarrow 4$; en sık gelen yüz 6; sayaç toplamı 12
- 13.10 [3, 9] $\rightarrow 7$; [7, 3, 9, 4] $\rightarrow 5,5$; maaşlarda ortalama 85,6, medyan 32
- 13.11 $11 \rightarrow$ indeks 7, 2 adım; 21 $\rightarrow$ yok, 3 adım, 5. sıraya eklenir; 2 $\rightarrow$ indeks 0, 3 adım; 65 $\rightarrow$ indeks 9, 4 adım
- 13.12 $0 \rightarrow [1, 2, 3, 4, 5]$; $k = 1 \rightarrow [5, 1, 2, 3, 4]$; $k = 2 \rightarrow [4, 5, 1, 2, 3]$; $k = 7 \rightarrow [4, 5, 1, 2, 3]$ (çünkü $7 \bmod 5 = 2$)

Hafta 14

- 14.12, 8, 1, 3, 3, 15; `t[3][0]` `IndexError` verir — dış listede 0, 1, 2 indisleri vardır, 3 yoktur
- 14.13 `[0, 7, 0], [0, 7, 0]`; * kopya değil aynı iç listeye iki referans üretir. Doğrusu: `grid = [[0] * 3 for _ in range(2)]`
- 14.14 ube toplamaları 465, 460, 750; gün toplamaları 400, 400, 415, 460; genel toplam iki yoldan da 1675
- 14.15 77,50; 76,25; 80,75]; en yüksek ortalama Proje sınavında (80,75)
- 14.16 3, 47, 26.0); tür `tuple`, uzunluk 3; ortalama $156/6 = 26$
- 14.17 ortalama Zeynep 95, Burak 88, Ayşe 85, Elif 78, Can 70, Mehmet 60; ortalama $476/6 = 79,33$; ortalama üstü Ayşe, Zeynep, Burak
- 14.18 kılme sonrası 4 kayıt; olmayan ülke için `get` varsayılanı döndürür; "Paris" anahtar değil değeri olduğu için `in` ile aranınca `False` çıkar; silme sonrası 3 kayıt ve ters sözlük de 3 kayıtlıdır
- 14.19 87 farklı harf, toplam 27 harf; en sık harfler a, i ve u (üçer kez); sayıların toplamı boşluksuz metnin uzunluğuna eşittir
- 14.20 başlangıç envanter değeri $120 \cdot 7,5 + 45 \cdot 32 + 8 \cdot 4,25 = 2374,00$ TL; 20 kalem satılıp 30 silgi girildikten sonra 2351,50 TL
- 14.21 10.inci matris sihirlidir, her çizginin toplamı 15; ikincisi değildir; $n = 3$ için denetlenecek çizgi sayısı $3 + 3 + 2 = 8$
- 14.22 `b = [[58, 64], [139, 154]]`; `a` ile birim matrisin çarpımı `a`'nın kendisi; `a · a` için boyutlar uymaz, `None`
- 14.23 (a) sözlük (b) liste (c) iki boyutlu liste (d) küme (e) demet

Hafta 15

- 15.1 (a) dosyada yalnız 30 kalır (1 satır); 10 ve 20 silinir. (b) 10, 20, 30 (3 satır). Günlük için "a": günlük birikmelidir
- 15.2 (i) tampondaki veri diske yazılmaz, dosya eksik ya da boş kalır; (ii) araya bir hata girerse

- `close()` satırına hiç ulaşmaz; (iii) dosya açık kalır, işletim sistemi kaynağı tutulur ve bazı sistemlerde dosya kilitlenir. `with`, bloktan nasıl çıkılırsa çıkılsın dosyayı kapatır
- 15.3 Dosya üç satır olur; yazarken `f.write(city + "\n")`, okurken `line.strip()` gerekir. Çıktı: 1. Trabzon, 2. Rize, 3. Giresun
- 15.4 4 satır, 52 kelime, 358 karakter; satır başına ortalama 6,50 kelime
- 15.5 Geçen 6, kalan 2; dağılım AA 2, BA 1, BB 0, CB 1, CC 1, DC 1, DD 0, FF 2 (toplam 8)
- 15.6 `readlines()` uygundur: satırları listeye alır, `reverse()` listeyi ters çevirir. Sonuçta 8 satır; yeni dosyanın ilk satırı eski dosyanın son satırıdır
- 15.7 70 kelime, 31 farklı kelime; en sık beşi veri 7, model 4, ister 3, iyi 3, de 2
- 15.8 Başlık yalnız bir kez yazılır; virgüllü ad dosyada çift tırnak içine alınır ve `csv.reader` onu `tek` alan olarak okur — her satır 4 alandır
- 15.9 Sınıf ortalaması 75,17; geçme oranı %83,3 (6 öğrencinin 5'i); en yüksek Zeynep Aydın 89,6, en düşük Ali Çelik 58,0
- 15.10 Burak'ın puanı 47'den 72'ye çıkar; dosya yine 8 satırdır. Ad bulunmazsa "w" ile açma adımı hiç çalıştırılmaz
- 15.11 İlk yüklemede "0 kayıt", ikinci yüklemede "2 kayıt" yazar; olmayan bir adı silmek `pop(ad, None)` ile çökmez ve bir kayıt silinince geriye 1 kayıt kalır
- 15.12 "w" dosyayı sildi ve yazma için açtı; okuma `io.UnsupportedOperation: not readable` verir — veri zaten gitmiştir, doğrusu "r". (b) satır "Ayşe 85\n" biçimindedir; `int()` `ValueError` verir, önce `split()` gerekir (ayrıca `total` döngüden önce tanımlanmalı). (c) `encoding` yazılmadığı için işletim sisteminin varsayılan kodlaması kullanılır; Türkçe harfler bozulabilir ya da `UnicodeDecodeError` çıkar

Dizin

for döngüsü, 184

ad uzayı, 230

ad uzayına, 244

adsız fonksiyon, 230

adı, 208

adım sayısı, 46, 64, 184, 200

adımdır, 95

Akış diyagramı, 31

akış diyagramı, 24

Algoritma, 26

algoritma, 24

alt metin, 92

alt metnin, 94

ana bellek, 5

anahtar, 279, 294

anahtar sözcüklerinden, 73

anahtar sözcüklü argüman, 230, 231

anlamli kısım, 13

Argüman, 210

argüman, 206, 207

Aritmetik-mantık birimi, 5

ASCII, 4, 13

Atama, 72

atama, 24, 27, 69

ayrıştırma, 303

ayırıcı, 92

ayırıcıdan, 100

açma, 230, 279, 291

açılır, 233

bayrak, 168, 199

bayrak değişkeni, 159

bayt, 4, 6

başlık satırıdır, 316

bekçi, 136, 144

Belgeleme dizisi, 220

belgeleme dizisi, 206

bellek, 4

betik, 69

bir-eksik hatası, 148

Biriktirici, 56, 163

biriktirici, 46, 159, 196

Birim kayması, 172

birim kayması, 46, 57, 159, 163

birleştirir, 77

Bit, 6

bit, 4

bit derinliği, 16

blok, 113

bloğu, 115

boş metin, 92

Collatz dizisini, 175

CSV, 303, 316

De Morgan kuralları, 136

Demet, 290

demet, 230, 233, 279

Derleyici, 69

derleyici, 4, 17

desen, 184

desen eşleme, 136

deyim, 69, 72

değer, 279, 294

Değişken, 27, 72

değişken, 3, 24, 69

değişken sayıda argüman, 230

değiştirilebilir, 258

değiştirilebilirlik, 255

değiştirilemez, 258

değiştirilemezlik, 92

dilim, 92, 255

dilimleme, 95

Dizi, 63

dizi, 46

Donanım, 4

donanım, 4

Dosya, 19, 303

dosya, 303

dosya modu, 303

Dosya yolu, 304

dosya yolu, 303

doğrulama deyimi, 230

doğruluk değeri, 113, 125

doğruluk tablosu, 46, 51, 113, 120

doğrusal arama, 64, 255

Döngü, 53

döngü, 46, 159

döngü değişkeni, 184

döngü değişkenine, 184

dönüş değeri, 206, 207

ekleme modu, 303

eleman, 255

en anlamlı bit, 8

en az anlamlı bit, 8

etkileşimli kabuk, 69

f-string, 69, 83

Fonksiyon, 207

fonksiyon, 206

fonksiyon hiyerarşisi, 223

for döngüsü, 184

frekans, 255

frekans sayma, 279

fırlatırsa, 147

geri izleme, 86

- Girdi doğrulama, 146
- girdi doğrulama, 136
- girdi doğrulama döngüsü, 159
- girinti, 113
- giriş/çıkış birimleri, 5
- global, 216
- gölgeleme, 206
- Görelî yol, 304
- görelî yol, 19, 303
- gövde, 184, 208
- günlük, 303, 311
- ham metin, 92, 93
- ifade, 24, 27, 69, 72
- iki boyutlu liste, 279
- ikili arama, 46, 64, 255
- ikili dosya, 303
- ikili sistem, 4
- ikiye tümleyen, 12
- indeks, 46, 92, 255
- indeksi, 63
- istisna, 69
- izleme tablosu, 24, 159
- iç içe döngü, 184, 283
- iç içe koşul, 136
- işlem önceliği, 24
- işlemci, 4, 5
- işletim sistemi, 4
- işlev, 72
- joker desen, 136
- JSON, 303, 323
- kabarcık sıralaması, 255
- kalan, 79
- Kalıcılık, 303
- kalıcılık, 303
- kalın, 3
- kanal, 16
- kapsam, 206
- kapsamı, 216
- Karakter dizisi, 92
- karakter dizisi, 72, 92
- Karakter kodlaması, 13
- karar ağacı, 136, 151
- Karar tablosu, 50, 139
- karar tablosu, 46, 136
- Karar yapısı, 47
- karar yapısı, 46
- karşılaştırma operatörü, 113
- kayan nokta, 4, 13
- kaynak kod, 69
- kayıt, 279
- kaçış karakteri, 92
- Klasör, 19
- kod noktası, 13, 92, 102
- kodlama, 303
- Kontrol birimi, 5
- konumsal argüman, 206, 210
- kopya, 255
- koşul, 46, 113
- koşullu ifade, 113, 126
- koşulu, 115
- koşulun, 47
- Küme, 298
- küme, 279
- kısa devre değerlendirme, 113, 122
- Kısaltılmış atama, 81
- kısmi toplam, 184, 196
- Liste, 255
- liste, 97, 186, 255
- Liste üretici, 264
- liste üretici, 255
- listedir, 100
- makine öğrenmesi, 18
- mantıksal işleç, 46, 51
- mantıksal operatör, 113
- matris, 279
- metin dosyası, 303
- metot, 92, 255
- metottur, 97
- Mod, 306
- mod, 24, 28
- Modül, 81, 244
- modül, 230
- Mutlak yol, 304
- mutlak yol, 19, 303
- mükemmel sayı, 202
- NameError, 87
- Nöbetçi değer, 54, 165
- nöbetçi değer, 46, 159
- Onaltılık sistem, 10
- onaltılık sistem, 4
- paketleme, 230, 279, 291
- paketler, 233
- palindrom, 108
- Parametre, 210
- parametre, 206
- piksel, 4
- piksellerden, 15
- programlama dilleriyle, 17
- renk derinliği, 15
- REPL, 71
- RGB, 15
- saat hızı, 5
- Sabit, 29
- sabit, 24, 75

- sabit değer, 69
- saklı program ilkesi, 5
- satır, 279
- satır sonu, 303
- satırları, 279
- sayacın, 56
- sayaç, 46, 159, 189
- sayaç kontrollü döngü, 162
- seri, 184, 196
- seçme yapısı, 113
- seçmeli sıralama, 255
- Sonsuz döngü, 172
- sonsuz döngü, 46, 54, 159
- soyutlama, 207
- SyntaxError, 87
- Sözde kod, 30
- sözde kod, 24
- Sözlük, 294
- sözlük, 279
- sözlük sırası, 92, 102
- sütun, 279
- sütunlara, 279
- sınır değer, 113
- Sınır değer analizi, 148
- sınır değer analizi, 120, 136
- sınır değerleri, 148
- Sıralı yapı, 26
- sıralı yapı, 24
- sığ kopya, 266

- taban durumu, 230, 237
- takas, 24
- takma ad, 255, 265
- Tam bölme, 28, 79
- tam bölme, 24
- tampon, 303
- tampona, 306
- taşma, 4, 12
- Tek tırnak, 92
- tekrar yapısı, 159
- Terminal, 19
- terminal, 4
- Test durumu, 41
- test durumu, 24
- test tablosu, 136, 148
- tolerans, 127
- transpoz, 279
- tur, 46, 53, 159, 160
- TypeError, 87
- tür dönüşümü, 69, 77
- tırnaklar, 92

- ulaşılamayan kod, 119
- Unicode, 4, 13, 92
- UTF-8, 4, 13
- Uygulama yazılımı, 17
- uzantı, 19

- ValueError, 87
- varsayılan değeri, 230
- varsayılan parametre değeri, 230
- veri türü, 69, 75
- veri yolu, 5
- void fonksiyon, 206, 215

- yakınsama, 184
- yakınsıyorsa, 196
- yan etki, 255
- yan etkisi, 267
- yardımcı fonksiyon, 230, 234
- yarı açık aralık, 184
- Yazmaçlar, 5
- Yazılım, 4
- yazılım, 4
- yeniden düzenleme, 206, 223
- yerel değişken, 206
- yerel değişkendir, 216
- yineleyici, 184
- Yol, 19
- yol, 4
- Yorumlayıcı, 69
- yorumlayıcı, 4, 17, 69
- Yukarıdan aşağıya tasarım, 223
- yukarıdan aşağıya tasarım, 206
- yüksek seviyeli programlama dilleri, 69

- zaman damgası, 312
- ZeroDivisionError, 88
- zincirleme karşılaştırma, 113, 115
- zincirlenebilir, 98

- Çift tırnak, 92
- Üç tırnak, 92
- çalışma dizini, 19, 303
- çalışma dizininden, 304
- çağrı, 206, 208
- çağrı ağacı, 230
- çağrı yığını, 206
- çerçeve, 206, 208
- çevirici, 17
- çok satırlı metin, 92
- ölü kod, 212
- örnekleme, 4
- örnekleme hızı, 16
- özet, 279
- özyineleme, 230
- özyinelemeli, 237
- özyinelemeli adım, 230, 237
- üssü, 13
- üçgen eşitsizliği, 124

- İki boyutlu liste, 279
- İkili sistem, 8
- İndeks, 94
- İzleme tablosu, 32

İşaretli, [11](#)
İşaretsiz, [11](#)
İşletim sistemi, [17](#)